



Red Hat
Training and Certification

Multicluster Management with Red Hat Advanced Cluster Management for Kubernetes

Red Hat Advanced Cluster Management
for Kubernetes 2.13 DO432

Student Workbook
Edition 2

Multicloud Management with Red Hat Advanced Cluster Management for Kubernetes

Document Conventions

- Admonitions
- Use of Artificial Intelligence
- Inclusive Language

Introduction

- Multicloud Management with Red Hat Advanced Cluster Management for Kubernetes
- Orientation to the Classroom Environment
- Performing Lab Exercises

Chapter 1. Installing Red Hat Advanced Cluster Management for Kubernetes

- 1.1. The Kubernetes Multicloud and Hybrid Cloud Architecture Landscape
- 1.2. Quiz The Kubernetes Multicloud and Hybrid Cloud Architecture Landscape
- 1.3. Architecture of Red Hat Advanced Cluster Management for Kubernetes
- 1.4. Quiz Architecture of Red Hat Advanced Cluster Management for Kubernetes
- 1.5. Installing Red Hat Advanced Cluster Management for Kubernetes
- 1.6. Guided Exercise Install Red Hat Advanced Cluster Management for Kubernetes
- 1.7. Quiz Installing Red Hat Advanced Cluster Management for Kubernetes
- 1.8. Summary

Chapter 2. Managing Clusters by Using Red Hat Advanced Cluster Management for Kubernetes

- 2.1. Cluster Lifecycle Management by Using the RHACM Web Console
- 2.2. Guided Exercise Manage the Cluster Lifecycle by Using the RHACM Web Console
- 2.3. Navigating the RHACM Web Console
- 2.4. Guided Exercise Navigate the RHACM Web Console
- 2.5. Configuring Access Control for Multicloud Management
- 2.6. Guided Exercise Configure Access Control for Multicloud Management
- 2.7. Troubleshooting Common Import Issues for Managed Clusters
- 2.8. Guided Exercise Troubleshoot Common Import Issues for Managed Clusters
- 2.9. Lab Manage a Cluster by Using Red Hat Advanced Cluster Management for Kubernetes
- 2.10. Summary

Chapter 3. Deploying and Managing Policies for Multiple Clusters with Red Hat Advanced Cluster Management for Kubernetes

- 3.1. Red Hat Advanced Cluster Management Governance
- 3.2. Quiz Red Hat Advanced Cluster Management Governance
- 3.3. Deploying Policies by Using the RHACM Governance Dashboard
- 3.4. Guided Exercise Deploy Policies by Using the RHACM Governance Dashboard
- 3.5. Explain and Deploy Policies by Using Policy Generator
- 3.6. Guided Exercise Deploy Policies by Using Policy Generator
- 3.7. Deploying and Configuring the Compliance Operator for Multiple Clusters by Using RHACM

- 3.8. Guided Exercise Deploy and Configure the Compliance Operator for Multiple Clusters by Using RHACM
- 3.9. Integration of Other Policy Engines with RHACM
- 3.10. Guided Exercise Integrate Other Policy Engines with RHACM
- 3.11. Troubleshoot Governance Issues
- 3.12. Guided Exercise Troubleshoot Governance Issues
- 3.13. Lab Deploying and Managing Policies with RHACM
- 3.14. Summary

Chapter 4. Enabling and Customizing the Red Hat Advanced Cluster Management for Kubernetes Observability Stack

- 4.1. Enabling the RHACM Observability Stack
- 4.2. Guided Exercise Enable the RHACM Observability Stack
- 4.3. Customizing the RHACM Observability Stack
- 4.4. Guided Exercise Customize the RHACM Observability Stack
- 4.5. Lab Enable and Customize the RHACM Observability Stack
- 4.6. Summary

Chapter 5. Managing the Multicloud Application Lifecycle by Using GitOps Practices and Red Hat Advanced Cluster Management for Kubernetes

- 5.1. Integrating GitOps Practices and Application Lifecycle Management of RHACM
- 5.2. Quiz Integrating GitOps Practices and Application Lifecycle Management of RHACM
- 5.3. Configuring GitOps for Application Lifecycle Management in RHACM
- 5.4. Guided Exercise Configure GitOps for Application Lifecycle Management in RHACM
- 5.5. Deploying and Managing Multicloud Application Resources with RHACM
- 5.6. Guided Exercise Deploy and Manage Multicloud Application Resources with RHACM
- 5.7. Troubleshooting Common Issues for Application Lifecycle Management in RHACM
- 5.8. Guided Exercise Troubleshoot Common Issues for Application Lifecycle Management in RHACM
- 5.9. Lab Manage Applications Across Multiple Clusters with RHACM
- 5.10. Summary

Chapter 6. Managing Virtual Machines for Multiple Clusters with Red Hat Advanced Cluster Management for Kubernetes

- 6.1. Introducing OpenShift Virtualization
- 6.2. Quiz Introducing OpenShift Virtualization
- 6.3. Deploying the OpenShift Virtualization Operator by Using GitOps with RHACM
- 6.4. Guided Exercise Deploy the OpenShift Virtualization Operator by Using GitOps with RHACM
- 6.5. Deploying and Managing Multicloud Virtual Machines with RHACM
- 6.6. Guided Exercise Deploy and Manage Multicloud Virtual Machines with RHACM
- 6.7. Monitoring OpenShift Virtualization with RHACM
- 6.8. Guided Exercise Monitor OpenShift Virtualization with RHACM
- 6.9. Lab Manage Virtual Machines for Multiple Clusters with RHACM
- 6.10. Summary

Multicuster Management with Red Hat Advanced Cluster Management for Kubernetes

Red Hat Advanced Cluster Management for Kubernetes 2.13 DO432

Edition 2

Edition date 2026-01-09

Publication date 2025-08-18

Authors:	Christopher Caillouet, Guillermo Badenes Gómez, Harpal Singh, Hervé Quatremain, Maria Ordoñez, Michael Jarrett, Natalie Watkins, Niladri Roy, Randy Thomas
Course Architects:	Harpal Singh
DevOps Engineers:	Benjamin Chardi, Jordi Sola, Nikki Lucas
Editors:	Julian Cable
Contributors:	Dave Sacco, Steven Bonneville, Mariano Barbero, Diego Calbo, Shashi Singh, Chetan Tiwary

© 2025 Red Hat, Inc.

The contents of this course and all its modules and related materials, including handouts to audience members, are © 2025 Red Hat, Inc.

No part of this publication may be stored in a retrieval system, transmitted or reproduced in any way, including, but not limited to, photocopy, photograph, magnetic, electronic or other record, without the prior written permission of Red Hat, Inc.

This instructional program, including all material provided herein, is supplied without any guarantees from Red Hat, Inc. Red Hat, Inc. assumes no liability for damages or legal action arising from the use or misuse of contents or details contained herein.

If you believe Red Hat training materials are being used, copied, or otherwise improperly distributed, please send email to training@redhat.com or phone toll-free (USA) +1 (866) 626-2994 or +1 (919) 754-3700.

Red Hat, Red Hat Enterprise Linux, the Red Hat logo, JBoss, OpenShift, Fedora, Hibernate, Ansible, RHCA, RHCE, RHCSA, Ceph, and Gluster are trademarks or registered trademarks of Red Hat, Inc. or its subsidiaries in the United States and other countries.

Linux® is the registered trademark of Linus Torvalds in the United States and other countries.

Java® is a registered trademark of Oracle American, Inc. and/or its affiliates.

XFS® is a registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and/or other countries.

MySQL® is a registered trademark of MySQL AB in the United States, the European Union and other countries.

Node.js® is a trademark of Joyent. Red Hat is not formally related to or endorsed by the official Joyent Node.js open source or commercial project.

The OpenStack word mark and the Square O Design, together or apart, are trademarks or registered trademarks of OpenStack Foundation, in the United States and other countries and are used with the OpenStack Foundation's permission. Red Hat, Inc. is not affiliated with, endorsed by, or sponsored by the OpenStack Foundation or the OpenStack community.

All other trademarks are the property of their respective owners.

Document Conventions

This section describes various conventions and practices that are used throughout all Red Hat Training courses.

Admonitions

Red Hat Training courses use the following admonitions:

References

References describe where to find external documentation that is relevant to a subject.

Note

Notes are tips, shortcuts, or alternative approaches to the task at hand. Ignoring a note should have no negative consequences, but you might miss out on something that makes your life easier.

Important

Important sections provide details of information that is easily missed: configuration changes that apply only to the current session, or services that need restarting before an update applies. Ignoring these admonitions will not cause data loss, but might cause irritation and frustration.

Warning

Do not ignore warnings. Ignoring these admonitions will most likely cause data loss.

Use of Artificial Intelligence

This publication contains content or media that is generated or assisted by AI tools.

All Red Hat Training deliverables have primarily human authorship, and all content was reviewed by a human domain expert.

Inclusive Language

Red Hat Training is currently reviewing its use of language in various areas to help remove any potentially offensive terms. This process is ongoing and requires alignment with the products and services that are covered in Red Hat Training courses. Red Hat appreciates your patience during this process.

Introduction

Multicluster Management with Red Hat Advanced Cluster Management for Kubernetes

Multicluster Management with Red Hat Advanced Cluster Management for Kubernetes (DO432) addresses challenges with management, compliance, and productivity of a multicluster Kubernetes environment and its applications by using the capabilities of Red Hat Advanced Cluster Management for Kubernetes.

Objectives

- Describe and implement the Red Hat Advanced Cluster Management for Kubernetes (RHACM) architecture and its components, and follow recommended practices for its installation.
- Import and manage a cluster by using the RHACM web console, configure user access to clusters, and troubleshoot common import issues.
- Deploy and manage policies in a multicluster environment by using RHACM governance.
- Monitor a fleet of managed clusters and troubleshoot performance and availability issues by using RHACM observability components.
- Deploy and manage applications in a multicluster environment by using RHACM and GitOps.
- Deploy and manage virtual machines in a multicluster environment by using RHACM and GitOps.

Audience

- IT Operations personnel who are responsible for managing the lifecycle and monitoring the usage of multiple clusters.
- DevOps and Site Reliability Engineers who are responsible for monitoring the health of clusters and Kubernetes deployments and for maximizing their uptime. These individuals also automate the provisioning or deprovisioning of clusters and the workload placement based on capacity and policy.
- Security Operations personnel who are responsible for ensuring that Kubernetes deployments comply with regulatory and organizational standards. These individuals create security policies across diverse environments and ensure enforcement.

Prerequisites

- *Red Hat Certified Specialist in OpenShift Administration* certification (EX280) or equivalent knowledge for the roles of Red Hat OpenShift cluster administrator or SRE.
- *Red Hat Certified Systems Administrator* certification (EX200) or equivalent knowledge of Linux system administration is recommended for all roles.
- Recommended: *Red Hat OpenShift Administration III* (DO380) for advanced OpenShift platform knowledge, such as cluster-wide configuration, logging, and monitoring.

Orientation to the Classroom Environment

In this course, the main computer system that is used for hands-on learning activities (exercises) is workstation.

The workstation machine has a standard user account, student, with student as the password. Although no exercise in this course requires you to log in as root, if you must, then the root password on the workstation

Introduction

machine is redhat.

From the workstation machine, you type the oc commands to manage the OpenShift cluster, which is preinstalled as part of your classroom environment.

Also from the workstation machine, you run the commands that are required to complete the exercises for this course.

If exercises require you to open a web browser to access any application or website, then you must use the graphical console of the workstation machine and use the Firefox web browser from there.

Note

During the initial start of your classroom environment, the OpenShift clusters take more time to become fully available. The lab command at the beginning of each exercise checks and waits as required.

If you try to access your cluster by using either the oc command or the web console without first running a lab command, then your cluster might not yet be available. If the cluster is not available, then wait a few minutes and try again.

Log in to OpenShift from the Shell

To access your OpenShift cluster from the workstation machine, use the following API URL:

- https://api.ocp4.example.com:6443

```
[student@workstation ~]$ oc login -u admin -p redhatocp \
https://api.ocp4.example.com:6443
```

Besides the admin user, which has cluster administrator privileges, your OpenShift cluster also provides the developer user, with developer as the password, with no special privileges.

Verify Cluster Information

After logging in, you can verify the status and endpoints of your cluster to confirm that your workstation is communicating with the correct OpenShift cluster.

```
[student@workstation ~]$ oc cluster-info
```

Accessing the OpenShift Web Console

If you prefer to use the OpenShift web console, then open a Firefox web browser on your workstation machine and access the following URL:

- https://console-openshift-console.apps.ocp4.example.com

Click **Red Hat Identity Management** and provide the login credentials for either the admin or the developer user:

Table 1. User Credentials

Username	Password
admin	redhatocp

Username	Password
developer	developer

The Classroom Environment

Every student gets a complete remote classroom environment, which includes a dedicated OpenShift cluster for administration tasks.

This environment contains all the necessary resources for the course.

The classroom environment runs entirely as virtual machines in a large Red Hat OpenStack Platform cluster, which is shared among many students.

Red Hat Training maintains many OpenStack clusters, in data centers across the globe, to provide lower latency to students from many countries.

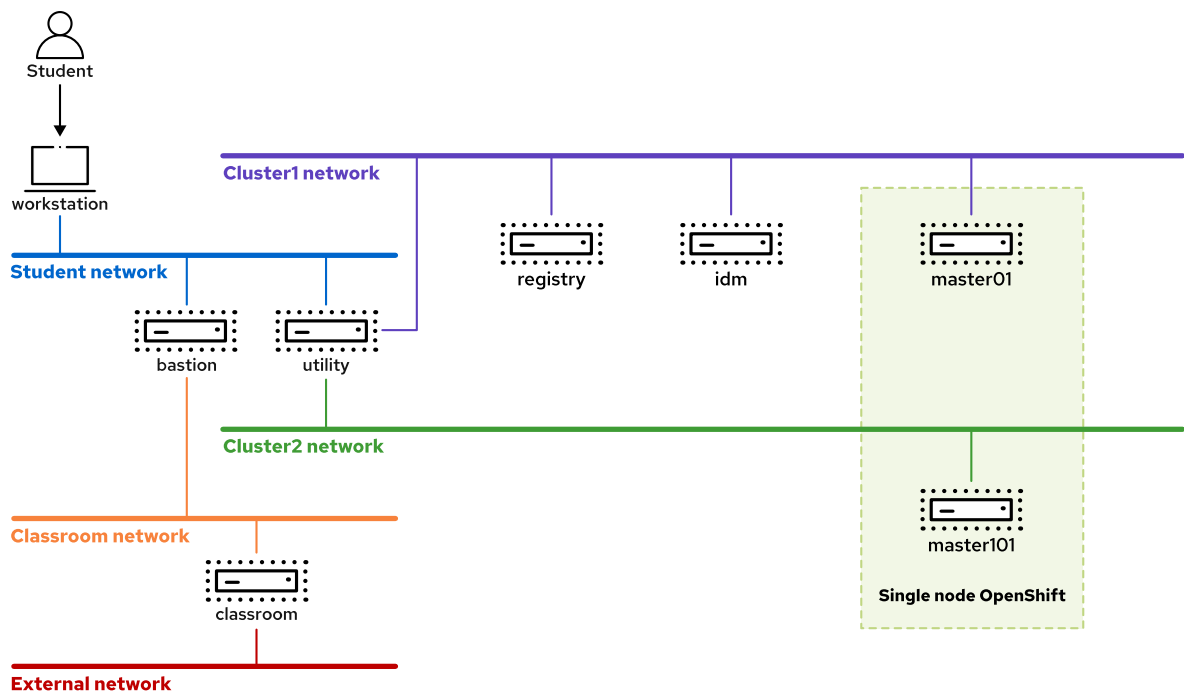


Figure 1. DO432 classroom architecture

All machines in the Student, Classroom, and Cluster networks run Red Hat Enterprise Linux 9 (RHEL 9), except those machines that are nodes of the OpenShift cluster. These cluster nodes run RHEL CoreOS.

The bastion, utility, registry, and classroom systems must always be running. These systems provide infrastructure services that the classroom environment and its OpenShift cluster require. For most exercises, you are not expected to interact with any of these services directly.

Usually, the lab commands from the exercises access these machines to set up your environment for the exercise, and require no further action from you.

For the few exercises where you must access a system other than workstation, primarily the utility system, you receive explicit instructions and connection information as part of the exercise.

Introduction

All systems in the Student network are in the `lab.example.com` DNS domain, and all systems in the Classroom network are in the `example.com` DNS domain.

The `master01` and `master101` systems are single node OpenShift 4 clusters that are part of your classroom environment.

All systems in the Cluster network are in the `ocp4.example.com` DNS domain, except the managed cluster, which is in the `ocp4-mng.example.com` domain.

Table 2. Classroom Machines

Machine name	IP addresses	Role
<code>workstation.lab.example.com</code>	172.25.250.9	Graphical workstation for system administration
<code>classroom.example.com</code>	172.25.254.254	Router to link the Classroom network to the internet
<code>bastion.lab.example.com</code>	172.25.250.254	Router to link the Student network to the Classroom network
<code>utility.lab.example.com</code>	172.25.250.253	Router to link the Student and Cluster networks
<code>registry.ocp4.example.com</code>	192.168.50.50	Server with Quay and GitLab
<code>master01.ocp4.example.com</code>	192.168.50.10	Single node OpenShift cluster that hosts RHACS Central
<code>master101.ocp4-mng.example.com</code>	192.168.51.10	Single node OpenShift cluster that RHACS Central manages
<code>idm.ocp4.example.com</code>	192.168.50.40	Red Hat Identity Management server

Performing Lab Exercises

You might see the following lab activity types in this course:

- A *guided exercise* is a hands-on practice exercise that follows a presentation section. It walks you through a procedure to perform, step by step.
- A *quiz* is typically used when checking knowledge-based learning, or when a hands-on activity is impractical for some other reason.
- An *end-of-chapter lab* is a gradable hands-on activity to help you to check your learning. You work through a set of high-level steps, based on the guided exercises in that chapter, but the steps do not walk you through every command. A solution is provided with a step-by-step walk-through.
- A *comprehensive review lab* is used at the end of the course. It is also a gradable hands-on activity, and might cover content from the entire course. You work through a specification of what to accomplish in the activity, without receiving the specific steps to do so. Again, a solution is provided with a step-by-step walk-through that meets the specification.

Introduction

To prepare your lab environment at the start of each hands-on activity, run the `lab start` command with a specified activity name from the activity's instructions. Likewise, at the end of each hands-on activity, run the `lab finish` command with that same activity name to clean up after the activity. Each hands-on activity has a unique name within a course.

The syntax for running an exercise script is as follows:

```
[student@workstation ~]$ lab action exercise
```

The *action* is a choice of `start`, `grade`, or `finish`. All exercises support the `start` and `finish` actions. Only end-of-chapter labs and comprehensive review labs support the `grade` action.

start

The `start` action verifies the required resources to begin an exercise. It might include configuring settings, creating resources, checking prerequisite services, and verifying necessary outcomes from previous exercises. You can perform an exercise at any time, even without performing preceding exercises.

grade

For gradable activities, the `grade` action directs the `lab` command to evaluate your work, and shows a list of grading criteria with the `PASS` or `FAIL` status for each. To achieve the `PASS` status for all criteria, fix the failures and rerun the `grade` action.

finish

The `finish` action cleans up resources that were configured during the exercise. You can perform an exercise as many times as you want.

The `lab` command supports tab completion. For example, to list all exercises that you can start, enter `lab start` and press the `Tab` key twice.

Chapter 1.

Installing Red Hat Advanced Cluster Management for Kubernetes

Goal

Describe and implement the Red Hat Advanced Cluster Management for Kubernetes (RHACM) architecture and its components, and follow recommended practices for its installation.

Sections

- The Kubernetes Multicluster and Hybrid Cloud Architecture Landscape (and Quiz)
- Architecture of Red Hat Advanced Cluster Management for Kubernetes (and Quiz)
- Installing Red Hat Advanced Cluster Management for Kubernetes (and Guided Exercise)
- Installing Red Hat Advanced Cluster Management for Kubernetes (Quiz)

1.1. The Kubernetes Multicluster and Hybrid Cloud Architecture Landscape

Objectives

- Identify common use cases for multiple Kubernetes clusters and the challenges that multicluster architectures present.

The Kubernetes Multicluster and Hybrid Cloud Architecture Landscape

Widespread adoption of containers, cloud-native applications, and hybrid cloud strategies drives enterprises to manage multiple Kubernetes clusters. Organizations use multicluster and hybrid cloud architectures to achieve scalability, flexibility, and resilience. Red Hat Advanced Cluster Management for Kubernetes (RHACM) uses the multicluster global hub to centralize management of distributed clusters, enforce consistent policies, and provide visibility across hybrid environments.

Cloud Features

Cloud environments vary to suit different organizational requirements, which range from fully managed to fully controlled setups. Enterprises select models based on security, scalability, and cost considerations. RHACM manages these models, for unified oversight.

Organizations choose from several cloud models to align with their operational goals. The following table describes common types of clouds:

Table 3. Common Cloud Features

Feature	Description	Example
Network Accessibility	Users access resources such as applications, services, and hardware abstractions over a network to enable seamless connectivity.	A global retailer accesses inventory management software from any store via the internet.
Resource Repository	Clouds maintain a centralized pool of IT resources, which include software services and virtualized hardware, for rapid deployment.	The retailer deploys customer-facing applications from this pool to handle peak shopping seasons.
Rapid Provisioning and Scaling	Administrators provision and scale resources quickly to match demand, to minimize delays.	The retailer scales its e-commerce platform during holiday sales to accommodate traffic spikes.

Public Cloud Limitations

There are factors that restrict public cloud usage for some enterprises. Administrators address these limitations with RHACM to integrate and manage diverse environments effectively.

Key Limitations and Solutions

Public clouds do not suit every scenario due to technical, financial, and regulatory constraints. The following table outlines key public cloud limitations and RHACM solutions:

Table 4. Public Cloud Limitations

Limitation	Description	Example	Solution
Existing On-Premise Workloads	Companies with established data centers struggle to move critical applications.	A manufacturing company runs legacy production systems on-site, which is incompatible with public clouds.	RHACM multicluster global hub manages on-premise clusters alongside public cloud resources for seamless integration.
High Migration Costs	Transitioning to cloud-native applications incurs significant expenses.	The company finds refactoring its systems cost-prohibitive.	RHACM supports legacy systems in private clouds to enable gradual modernization.
Legacy System Incompatibility	Earlier systems often fail to adapt to cloud environments.	The proprietary software lacks cloud-native support.	RHACM architecture compatibility, such as with x86 or Arm, ensures that legacy workloads run efficiently in private clouds.
Operational Costs	Public cloud expenses escalate with rapid enterprise growth.	A tech startup faces rising costs when its user base expands.	RHACM optimizes resource allocation across hybrid clouds to control expenses.
Regulatory Restrictions	Industries such as healthcare or finance face strict data storage laws, such as GDPR in the European Union.	A hospital stores patient data locally to comply with regulations.	RHACM enforces policies to meet compliance across private and public clouds.
Security and Control Concerns	Enterprises prioritize protecting infrastructure and maintaining oversight.	The hospital fears losing control over public cloud data.	RHACM secures communications with HTTPS and integrates with existing security tools for robust control.

Private Cloud Characteristics

Private clouds mirror public cloud functions within controlled environments, which is ideal for sensitive or specialized workloads. Enterprises deploy private clouds to maintain security and to optimize resource usage, which RHACM often

manages for consistency.

Key Characteristics

Private clouds deliver flexible, secure IT infrastructure for enterprise needs. The following table highlights key characteristics of private clouds:

Table 5. Private Cloud Characteristics

Characteristic	Description	Example
Self-Provisioning on Demand	Enterprises deploy private clouds to maintain security and optimize resource usage, which RHACM manages for consistency.	A financial institution provisions secure applications on demand for compliance audits.
Autoscaling Resources	Infrastructure scales dynamically based on usage to optimize performance.	The company scales compute resources during claim surges.
Efficient Resource Usage	Shared physical resources maximize allocation efficiency.	The company runs multiple applications on the same hardware to reduce waste.

Hybrid Cloud Architecture

Hybrid clouds blend public, private, and on-premise resources to create a cohesive IT infrastructure. Administrators use hybrid clouds to combine scalability with control, by using RHACM to manage complexity.

Core Components

Hybrid cloud architectures integrate diverse environments for flexibility. The following table describes the core components of the hybrid cloud architecture:

Table 6. Hybrid Cloud Architecture

Component	Description	Example
Owned Data Centers	Enterprises manage bare-metal or virtualized environments.	A telecom company operates data centers for core network functions.
Rented Data Centers	Organizations lease virtualized environments from third parties.	The telecom company rents capacity for overflow traffic.
Private Clouds	Internally hosted clouds secure sensitive workloads.	The telecom company secures customer data in a private cloud.

Component	Description	Example
Public Clouds	Providers such as Google Cloud offer scalable resources.	The telecom company hosts customer applications in a public cloud for elasticity.

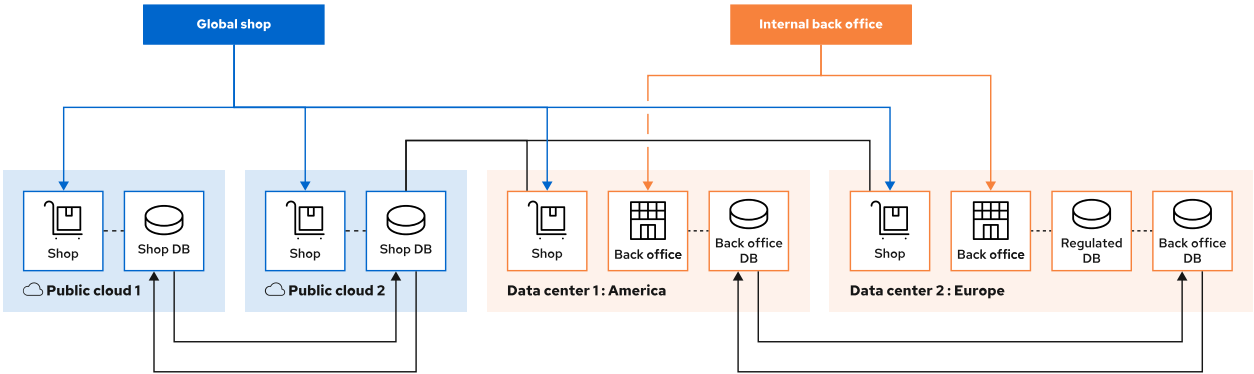


Figure 2. A hybrid cloud example

Benefits of Hybrid Clouds

Hybrid clouds deliver strategic advantages by combining the strengths of multiple environments. Enterprises achieve resilience, compliance, and high performance, all of which RHACM manages for seamless operation.

Primary Benefits

Hybrid clouds enhance IT operations with versatile capabilities. The following table outlines the primary benefits of hybrid clouds:

Table 7. Hybrid Cloud Benefits

Benefit	Description	Example
High Availability and Failover	Secondary clouds take over if the primary cloud fails to ensure uptime.	A logistics company switches to a public cloud during an on-premise outage.
Specialized Services	Administrators select optimal services from each cloud.	The company uses the public cloud AI tools for route optimization.
Geographical Proximity	Services run closer to users for faster performance.	The company deploys applications in regional clouds to reduce latency for drivers.
Regulatory Compliance	Hybrid setups adhere to data storage laws.	The company stores European Union customer data in a local private cloud to meet GDPR regulations.

Benefit	Description	Example
Reduced Network Latency	Critical applications benefit from low-latency connections.	The company prioritizes private clouds for real-time tracking to minimize delays.

For example, a global telecom company manages a hybrid cloud with the RHACM multicluster global hub. Private clouds in regional data centers secure customer data and public clouds handle scalable customer applications. The global hub synchronizes policies and monitors performance to ensure compliance with data laws and low-latency service delivery globally.

Use Cases for Multiple Kubernetes Clusters

Modern enterprises deploy multiple Kubernetes clusters to meet diverse operational demands in hybrid cloud environments. Use cases highlight needs for geographic reach, workload isolation, resilience, and scalability, all of which the RHACM multicluster global hub supports to streamline management.

Common Use Cases

Organizations use multiple Kubernetes clusters to address varied requirements in multicluster and hybrid cloud setups. The following use cases are common for multiple Kubernetes clusters:

- Geographic Distribution:** Global enterprises deploy clusters that operate globally to minimize latency and to comply with data residency regulations.
- Workload Segmentation:** Dedicated clusters for workloads such as development, testing, and production isolate environments and prevent disruptions.
- High Availability and Disaster Recovery:** Enterprises distribute workloads across clusters to ensure redundancy and reduce downtime.
- Scalability:** Large cluster fleets overwhelm a single management hub, which requires division into smaller groups, with a hub to manage each group.

For example, a global retail chain operates Kubernetes clusters in local data centers across multiple countries to handle e-commerce, inventory, and customer data.

The RHACM multicluster global hub manages these clusters centrally to enforce consistent policies and to monitor operations in real time across all locations.

Challenges of Multicluster Architectures

Multicluster setups in hybrid cloud environments present significant technical and operational hurdles. Organizations encounter issues such as application portability, security policy enforcement, network connectivity, data consistency, and management scalability. RHACM resolves the organization's issues through the multicluster global hub.

Technical and Operational Challenges

Managing multiple Kubernetes clusters presents significant hurdles, particularly in hybrid setups. The RHACM multicluster global hub resolves these challenges through platform support, secure communication, and centralized

management:

- **Portability:** Applications must run seamlessly across diverse cloud providers and on-premise environments. RHACM supports versions 2.10 to 2.13 on architectures that include x86, Arm, IBM Z, and IBM Power Systems, to ensure consistent application deployment.
- **Security:** Enterprises find that maintaining consistent security policies and compliance across clusters can be complex, especially across jurisdictions. RHACM enforces uniform security policies by using HTTPS for communication and integrates Kafka and PostgreSQL. Secrets and certificates are used to secure components through encryption and authentication.
- **Networking Complexity:** Secure, reliable communication between clusters across geographic locations or providers proves to be challenging. The RHACM multicluster global hub standardizes networking by using HTTPS on port 443 for Grafana, Kafka, and PostgreSQL to simplify secure communication.
- **Data Synchronization:** Consistent data across clusters remains critical for shared or replicated data. RHACM employs Kafka to transport data efficiently, to ensure real-time synchronization between the global hub and managed hubs.
- **Scalability Limits:** A single hub struggles to manage large cluster fleets effectively. The RHACM multicluster global hub manages multiple hub clusters from a single point to scale management for large environments.

For example, a telecommunications company deploys thousands of edge computing clusters to support 5G services. Scalability issues initially prevented managing these clusters from a single hub.

The RHACM multicluster global hub divides clusters into regional groups, with a local hub to manage each group, to provide centralized oversight for policy compliance and monitoring.

Architecture Components

The RHACM multicluster global hub relies on a robust set of components to manage and synchronize distributed Kubernetes clusters. Components collaborate to deliver centralized management, data persistence, and visualization, to ensure seamless operation across hybrid cloud environments.

Core Components

The multicluster global hub architecture comprises critical components, where each component serves a specific role to manage hub clusters and their managed clusters. Key components of the global hub architecture are as follows:

- **Global Hub Operator:** The operator deploys and manages components such as the global hub manager, Grafana, Kafka, and PostgreSQL on the global hub cluster, and the global hub agent on managed hub clusters.
- **Global Hub Manager:** The manager inputs data from Kafka into a PostgreSQL database and posts data to Kafka for synchronization with managed hubs.
- **Global Hub Agent:** The agent, which runs on managed hub clusters, synchronizes data such as cluster information and policies between the global hub and managed hubs.
- **Global Hub Visualizations (Grafana):** Grafana, which runs on the global hub cluster, provides dashboards that display policy compliance, cluster health, and middleware performance, which are accessible via a route.

For example, a financial services company automates deployment across global clusters to reduce manual configuration efforts. The company monitors compliance trends across clusters to identify noncompliant policies quickly. The company stores compliance data centrally to ensure audit-ready records. Then, the agent updates policy changes across the regional clusters in real time.

Design Considerations

Deploying the RHACM multicluster global hub demands careful planning to meet scalability, security, and performance requirements in hybrid cloud environments. Factors such as storage, networking, middleware integration, and access control shape the design to fulfill enterprise needs effectively.

Essential Design Considerations

Effective deployment of the multicluster global hub depends on addressing several design factors. Some key design considerations of the multicluster global hub are as follows:

- **Storage Requirements:** PostgreSQL requires at least 20 GB to store data for three managed hubs with 250 clusters and 50 policies for 18 months.
- **Networking:** Enterprises use secure HTTPS communication on port 443 for Grafana, Kafka, and PostgreSQL to ensure reliable connectivity.
- **Middleware Integration:** Organizations integrate existing Kafka version 3.4+, PostgreSQL version 13+, or Grafana instances instead of provided ones, to configure secrets and access appropriately.
- **Access Control:** The `cluster-admin` role remains necessary for installation, and Grafana access aligns with OpenShift console authentication.

For example, a healthcare provider can deploy the RHACM multicluster global hub to manage clusters that handle patient data across hybrid clouds. The provider allocates 30 GB for PostgreSQL to exceed minimum requirements, integrates its existing Grafana for unified monitoring, and enforces strict access controls to ensure Health Insurance Portability and Accountability Act (HIPAA) compliance and to secure operations.

Background and Evolution

The RHACM multicluster global hub builds on the evolution of Kubernetes cluster management, and draws from earlier solutions such as the Kubernetes Federation (KubeFed). Emerging operational roles such as Site Reliability Engineers (SREs), DevOps, and DevSecOps, alongside market trends toward managed services, have shaped the multicluster hub development to meet modern multicluster demands.

Historical Context and Development

Growing Kubernetes adoption, which is driven by containerization and hybrid cloud strategies, sparked the need for multicluster management. The following are key background elements from a historical and development context:

- **KubeFed Origins:** The initial aim of KubeFed to manage multiple clusters faced complexity and scalability issues, which led to its deprecation.
- **Operational Roles:** Roles such as Site Reliability Engineers (SREs) and DevSecOps emerged to handle multicluster complexity, by emphasizing automation and security. The 2024 Red Hat Kubernetes Adoption Report noted an increased adoption of managed services and hybrid cloud, which necessitates tools such as RHACM.
- **RHACM Evolution:** The RHACM multicluster global hub overcomes earlier limitations to offer a scalable, centralized solution for managing hub clusters, to replace ad-hoc federation approaches.

The RHACM Multicluster Global Hub

The RHACM multicluster global hub enables centralized control of large-scale Kubernetes environments. Centralized control, policy enforcement, and integration with observability tools address the complexities of multicluster architectures.

Core Features

The multicluster global hub centralizes management of multiple hub clusters. The following core features enhance governance, visibility, and security across hybrid environments:

- **Centralized Management:** The global hub imports hub clusters as managed hubs to enable tasks such as policy compliance reporting and inventory management from one interface.
- **Policy Enforcement:** Consistent policy application across clusters strengthens governance, with tools that report compliance status and trends.
- **Observability:** Grafana dashboards integrate to provide insights into cluster health, policy status, and middleware performance, despite unavailable full observability for the global hub.
- **Security:** Secure communication protocols and integration with existing security infrastructures bolster the multicluster environment.

For example, a global manufacturing company with production facilities worldwide manages IoT edge clusters with the RHACM multicluster global hub. Local hubs at each facility oversee edge devices and the global hub delivers a unified view to monitor production metrics, ensure safety compliance, and detect anomalies.

1.2. Quiz

The Kubernetes Multicloud and Hybrid Cloud Architecture Landscape

Choose the correct answers to the following questions:

- Which option represents a primary use case for deploying multiple Kubernetes clusters?
 - Centralized workload orchestration in a single region
 - Geographic distribution that reduces latency and that complies with data residency laws
 - Consolidation of all applications into one cluster
 - Isolation of legacy systems without scalability needs
- Which challenge do enterprises likely encounter when running applications across diverse cloud providers?
 - Simplified network configurations
 - Reduced security requirements
 - Portability issues due to varying configurations
 - Automatic data synchronization
- How does Red Hat Advanced Cluster Management (RHACM) address security challenges in multicloud environments?
 - Configures cluster-specific security policies that are tailored to each cloud provider
 - Relies on Gatekeeper constraints to enforce security without centralized policy management
 - Uses Kafka for data synchronization with optional encryption for performance optimization
 - Enforces uniform security policies by employing HTTPS and integrates Kafka and PostgreSQL
- Which component of the RHACM multicloud global hub deploys and manages other components such as Grafana and Kafka?
 - Global Hub Agent
 - Global Hub Operator
 - Global Hub Manager
 - Grafana Dashboards
- Which design consideration ensures that PostgreSQL can store data for three managed hubs with 250 clusters for 18 months?
 - Network accessibility
 - Autoscaling resources
 - Minimum storage requirement of 20 GB
 - Cluster-admin permissions
- Why did a manufacturing company transition from KubeFed to the RHACM multicloud global hub?
 - To simplify cluster provisioning across hybrid clouds.
 - To overcome scalability and complexity issues.
 - To centralize DevSecOps automation for cluster security.
 - To enhance integration with managed cloud services.

7. Which feature of the RHACM multicloud global hub enables engineers to report policy compliance from a single interface?
- ☐ a. Centralized management
 - ☐ b. Autoscaling resources
 - ☐ c. Network standardization
 - ☐ d. Data synchronization
8. How does RHACM mitigate networking complexity in multicloud setups?
- ☐ a. Mandates uniform cloud provider network configurations.
 - ☐ b. Standardizes networking by employing HTTPS on port 443 for Grafana, Kafka, and PostgreSQL.
 - ☐ c. Relies on cluster-local networking protocols.
 - ☐ d. Prioritizes low-latency UDP for all communications.
9. Which historical factor influenced the development of the RHACM multicloud global hub?
- ☐ a. Growth of single-region Kubernetes deployments
 - ☐ b. Decline in containerized application adoption
 - ☐ c. Shift away from managed service models
 - ☐ d. Deprecation of KubeFed due to complexity and scalability issues
10. Which benefit does a financial services organization gain from the RHACM Grafana dashboards?
- ☐ a. Monitors compliance trends across clusters in real time
 - ☐ b. Automatic workload migration
 - ☐ c. Generates security policies for cluster compliance
 - ☐ d. Reduces the need for data synchronization

Solution

- Which option represents a primary use case for deploying multiple Kubernetes clusters?
 - ☐ a. Centralized workload orchestration in a single region
 - ☒ b. **Geographic distribution that reduces latency and that complies with data residency laws**
 - ☐ c. Consolidation of all applications into one cluster
 - ☐ d. Isolation of legacy systems without scalability needs
- Which challenge do enterprises likely encounter when running applications across diverse cloud providers?
 - ☐ a. Simplified network configurations
 - ☐ b. Reduced security requirements
 - ☒ c. **Portability issues due to varying configurations**
 - ☐ d. Automatic data synchronization
- How does Red Hat Advanced Cluster Management (RHACM) address security challenges in multicluster environments?
 - ☐ a. Configures cluster-specific security policies that are tailored to each cloud provider
 - ☐ b. Relies on Gatekeeper constraints to enforce security without centralized policy management
 - ☐ c. Uses Kafka for data synchronization with optional encryption for performance optimization
 - ☒ d. **Enforces uniform security policies by employing HTTPS and integrates Kafka and PostgreSQL**
- Which component of the RHACM multicluster global hub deploys and manages other components such as Grafana and Kafka?
 - ☐ a. Global Hub Agent
 - ☒ b. **Global Hub Operator**
 - ☐ c. Global Hub Manager
 - ☐ d. Grafana Dashboards
- Which design consideration ensures that PostgreSQL can store data for three managed hubs with 250 clusters for 18 months?
 - ☐ a. Network accessibility
 - ☐ b. Autoscaling resources
 - ☒ c. **Minimum storage requirement of 20 GB**
 - ☐ d. Cluster-admin permissions
- Why did a manufacturing company transition from KubeFed to the RHACM multicluster global hub?
 - ☐ a. To simplify cluster provisioning across hybrid clouds.
 - ☒ b. **To overcome scalability and complexity issues.**
 - ☐ c. To centralize DevSecOps automation for cluster security.
 - ☐ d. To enhance integration with managed cloud services.

7. Which feature of the RHACM multicloud global hub enables engineers to report policy compliance from a single interface?
- ☒ a. **Centralized management**
 - ☐ b. Autoscaling resources
 - ☐ c. Network standardization
 - ☐ d. Data synchronization
8. How does RHACM mitigate networking complexity in multicloud setups?
- ☐ a. Mandates uniform cloud provider network configurations.
 - ☒ b. **Standardizes networking by employing HTTPS on port 443 for Grafana, Kafka, and PostgreSQL.**
 - ☐ c. Relies on cluster-local networking protocols.
 - ☐ d. Prioritizes low-latency UDP for all communications.
9. Which historical factor influenced the development of the RHACM multicloud global hub?
- ☐ a. Growth of single-region Kubernetes deployments
 - ☐ b. Decline in containerized application adoption
 - ☐ c. Shift away from managed service models
 - ☒ d. **Deprecation of KubeFed due to complexity and scalability issues**
10. Which benefit does a financial services organization gain from the RHACM Grafana dashboards?
- ☒ a. **Monitors compliance trends across clusters in real time**
 - ☐ b. Automatic workload migration
 - ☐ c. Generates security policies for cluster compliance
 - ☐ d. Reduces the need for data synchronization

1.3. Architecture of Red Hat Advanced Cluster Management for Kubernetes

Objectives

- Describe the benefits that RHACM provides to manage a multicluster hybrid cloud environment, and show the architecture of RHACM and its components.

Red Hat Advanced Cluster Management for Kubernetes

RHACM gives you end-to-end management visibility and control to manage your Kubernetes clusters from a single user interface.

RHACM addresses the challenges that cluster administrators and site reliability engineers face, when they work across a range of environments, including multiple data centers, private clouds, and public clouds that run Kubernetes clusters.

RHACM provides the following capabilities:

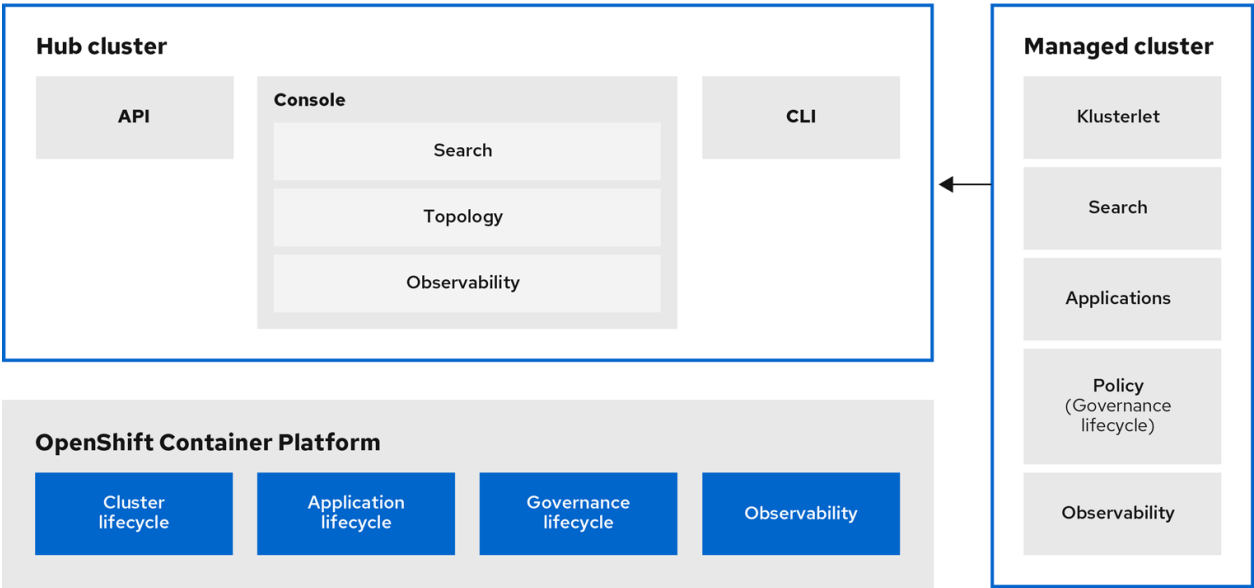
- Create Kubernetes clusters and offer cluster lifecycle management in a single console.
- Run your operations from anywhere that Red Hat OpenShift runs, and manage any Kubernetes cluster in your fleet.
- Enforce security and compliance at the target clusters.
- Deploy and maintain Day-2 operations of business applications that are distributed across your cluster landscape.
- Manage your OpenShift Virtualization virtual machine workloads across multiple clusters.
- Understand the health and use of clusters and of workloads across your fleet.

RHACM is cloud-native by design, and runs on containers. RHACM manages Kubernetes objects directly through their APIs.

RHACM Architecture

RHACM consists of several multicluster components, to access and manage your clusters. The following diagram provides a high-level overview of the RHACM architecture components:

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright



186_RHACM_1221

Figure 3. RHACM architecture

Hub Cluster

The *hub cluster* refers to the central controller that runs in an RHACM cluster. From the hub cluster, you can access the console and product components, as well as the RHACM APIs. You can use the console to search resources across clusters and to view your topology. You can enable observability on your hub cluster to monitor metrics from your managed clusters across your cloud providers.

The RHACM hub cluster uses the `MultiClusterHub` resource to manage, upgrade, and install hub cluster components, and runs in the `open-cluster-management` namespace.

The hub cluster aggregates information from multiple clusters by using an asynchronous work request model and search collectors. The hub cluster maintains the state of clusters and applications that run on it.

The *local cluster* refers to the OpenShift cluster that hosts the hub cluster. The local cluster is also a managed cluster.

Managed Cluster

A *managed cluster* is an additional Kubernetes or OpenShift cluster that reports data to, and is managed by the hub cluster. The `klusterlet` component is the agent that is installed on the managed cluster, and controls the connections between the hub cluster and the managed cluster.

The managed cluster receives and applies requests from the hub cluster and enables it to service the cluster lifecycle, the application lifecycle, governance, and observability on the managed cluster.

Cluster Lifecycle

The *cluster lifecycle* defines the process of creating, importing, managing, and deleting Kubernetes clusters across various infrastructure cloud providers, private clouds, and on-premise data centers.

The cluster lifecycle function is provided by the multicluster engine for Kubernetes operator, which is installed automatically with RHACM.

From the hub cluster console, you can do the following actions:

- View an aggregation of all cluster health statuses.
- View individual health metrics of many Kubernetes clusters.
- Upgrade managed OpenShift Container Platform clusters individually or in bulk.
- Delete any OpenShift clusters that you created, by using the hub cluster.
- Hibernate, resume, and detach clusters.

Application Lifecycle

The *application lifecycle* refers to the processes that manage the application resources across managed clusters. A multicluster application enables you to deploy resources on multiple managed clusters, as well as to provide full control of Kubernetes resource updates for all aspects of the application with high availability.

A multicluster application uses the Kubernetes specification, and provides additional automation of the deployment and lifecycle management of resources.

Red Hat OpenShift GitOps, which is based on Argo CD, is integrated with RHACM. You can set up a continuous GitOps environment to automate application consistency across clusters in development, staging, and production environments. Ansible Automation Platform jobs enable you to automate tasks.

Governance

The governance component enables you to define policies that either enforce security compliance or inform you of changes that violate the configured compliance requirements for your environment. You can view and create policies with the RHACM policy framework.

With dynamic policy templates, you can manage the policies and compliance requirements across all your managed clusters from a central interface.

Observability

Observability helps you identify and assess performance problems without additional tests and support. The observability component collects and reports the status and health of OpenShift 4.x or later versions of managed clusters to the hub cluster, which are visible from the Grafana dashboard. You can create custom alerts to inform you of problems with your managed clusters.

Observability must be enabled after the RHACM installation, because it requires persistent storage. Observability uses community open source observability tools, such as Thanos, Prometheus, and Alertmanager.

Multicluster Global Hub

The *multicluster global hub* is a set of components that enable you to import one or more hub clusters and to manage them from a single hub cluster. The multicluster global hub is useful when a single hub cluster cannot manage a large number of clusters in a high-scale environment. When this scenario happens, you divide the clusters into smaller groups of clusters and configure a hub cluster for each group.

You can use multicluster global hub to complete the following tasks across all the managed hub clusters:

- Report the policy compliance status and trend.
- Maintain an inventory of all managed hubs and managed clusters on the overview page.
- Detect and alert for any cases of irregular policy behavior.

References

For more information about Red Hat Advanced Cluster Management for Kubernetes, refer to the *Welcome to Red Hat Advanced Cluster Management for Kubernetes* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 About* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/about/index#welcome-to-red-hat-advanced-cluster-management-for-kubernetes

For more information about managing virtual machines for multiple clusters with RHACM, refer to the *Managing Virtual Machines for Multiple Clusters with RHACM* lesson.

For more information about Red Hat Advanced Cluster Management for Kubernetes Multicloud Global Hub, refer to the *Multicloud Global Hub* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Multicloud Global Hub* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/multicloud_global_hub/index#multicloud-global-hub

1.4. Quiz

Architecture of Red Hat Advanced Cluster Management for Kubernetes

Choose the correct answers to the following questions:

1. What is the primary role of the `kuberneteslet` agent in RHACM?
 - ☐ a. Deploys workloads on the hub cluster.
 - ☐ b. Connects a managed cluster to the hub cluster.
 - ☐ c. Provides observability dashboards.
 - ☐ d. Installs Red Hat OpenShift Container Platform on the managed cluster.
2. Which three open source tools use the Observability component in RHACM? (Choose three.)
 - ☐ a. Thanos
 - ☐ b. Prometheus
 - ☐ c. Red Hat Quay
 - ☐ d. Alertmanager
3. Which three options are roles of the hub cluster in RHACM? (Choose three.)
 - ☐ a. Host the central controller.
 - ☐ b. Run user workloads.
 - ☐ c. Manage the deployment of managed clusters.
 - ☐ d. Store policy compliance data.
4. Which technology does RHACM use for GitOps integration?
 - ☐ a. Flux
 - ☐ b. Tekton
 - ☐ c. Argo CD
 - ☐ d. Jenkins
5. Which are the two main functions of the Governance component in RHACM? (Choose two.)
 - ☐ a. Define and apply security and configuration policies.
 - ☐ b. Automatically deploy applications across clusters.
 - ☐ c. Monitor compliance status across managed clusters.
 - ☐ d. Scale the control plane dynamically.
6. What is the primary purpose of the multicluster global hub in RHACM?
 - ☐ a. Provide centralized image registry access.
 - ☐ b. Manage multiple hub clusters from a single hub cluster.
 - ☐ c. Deploy applications to edge devices.
 - ☐ d. Replace the Policy Controller on managed clusters.

7. What is the role of the managed cluster in RHACM?

- ☐ **a.** Receive and apply requests from the hub cluster.
- ☐ **b.** Collect observability metrics.
- ☐ **c.** Act as a backup for the hub cluster.
- ☐ **d.** Host the central controller of RHACM.

8. Which component is responsible for creating, importing, managing, and deleting Kubernetes clusters in RHACM?

- ☐ **a.** Application lifecycle
- ☐ **b.** Search service
- ☐ **c.** Cluster lifecycle
- ☐ **d.** Governance

9. What is the role of the application lifecycle component in RHACM?

- ☐ **a.** Manage authentication and RBAC across clusters.
- ☐ **b.** Deploy and manage applications across multiple clusters by using GitOps.
- ☐ **c.** Install and upgrade RHACM components.
- ☐ **d.** Enforce security policies on workloads.

Solution

1. What is the primary role of the `klusterlet` agent in RHACM?
 - ☐ a. Deploys workloads on the hub cluster.
 - ☒ b. **Connects a managed cluster to the hub cluster.**
 - ☐ c. Provides observability dashboards.
 - ☐ d. Installs Red Hat OpenShift Container Platform on the managed cluster.
2. Which three open source tools use the Observability component in RHACM? (Choose three.)
 - ☒ a. **Thanos**
 - ☒ b. **Prometheus**
 - ☐ c. Red Hat Quay
 - ☒ d. **Alertmanager**
3. Which three options are roles of the hub cluster in RHACM? (Choose three.)
 - ☒ a. **Host the central controller.**
 - ☐ b. Run user workloads.
 - ☒ c. **Manage the deployment of managed clusters.**
 - ☒ d. **Store policy compliance data.**
4. Which technology does RHACM use for GitOps integration?
 - ☐ a. Flux
 - ☐ b. Tekton
 - ☒ c. **Argo CD**
 - ☐ d. Jenkins
5. Which are the two main functions of the Governance component in RHACM? (Choose two.)
 - ☒ a. **Define and apply security and configuration policies.**
 - ☐ b. Automatically deploy applications across clusters.
 - ☒ c. **Monitor compliance status across managed clusters.**
 - ☐ d. Scale the control plane dynamically.
6. What is the primary purpose of the multicluster global hub in RHACM?
 - ☐ a. Provide centralized image registry access.
 - ☒ b. **Manage multiple hub clusters from a single hub cluster.**
 - ☐ c. Deploy applications to edge devices.
 - ☐ d. Replace the Policy Controller on managed clusters.
7. What is the role of the managed cluster in RHACM?
 - ☒ a. **Receive and apply requests from the hub cluster.**
 - ☐ b. Collect observability metrics.
 - ☐ c. Act as a backup for the hub cluster.
 - ☐ d. Host the central controller of RHACM.

8. Which component is responsible for creating, importing, managing, and deleting Kubernetes clusters in RHACM?

- ☐ a. Application lifecycle
- ☐ b. Search service
- ☒ c. **Cluster lifecycle**
- ☐ d. Governance

9. What is the role of the application lifecycle component in RHACM?

- ☐ a. Manage authentication and RBAC across clusters.
- ☒ b. **Deploy and manage applications across multiple clusters by using GitOps.**
- ☐ c. Install and upgrade RHACM components.
- ☐ d. Enforce security policies on workloads.

1.5. Installing Red Hat Advanced Cluster Management for Kubernetes

Objectives

- Install the RHACM operator from the operator hub in a Red Hat OpenShift hub cluster.

RHACM Operator Installation

It is possible to install the RHACM operator by using the OpenShift web console or the command line. For clusters with network connectivity, Red Hat recommends installing the RHACM operator from OperatorHub by using the OpenShift web console. For disconnected clusters, the operator must be available from an accessible storage endpoint for the cluster. Installing the operator is similar to the process for other operators in OpenShift.

Installation Prerequisites and Sizing for RHACM

The following items are prerequisites for installing RHACM:

- An OpenShift or Kubernetes cluster
- A supported browser, such as Google Chrome, MacOS Safari, or Microsoft Internet Explorer Edge
- Network communication between the hub cluster and the managed clusters

The following factors determine RHACM resource usage:

- The platform on which RHACM runs
- The number of managed clusters in the fleet
- The speed of storage or networks
- The number of applications in each cluster

To calculate the required resources for an RHACM hub cluster, you need to know details about the workloads to deploy. Accurately sizing the hub cluster requires testing and monitoring the resource usage of the hub cluster and knowledge of what is deployed on it.

 **Note**
To review sizing information for an RHACM installation, refer to the *Sizing Your Cluster* chapter in the installation guide at https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/install/index#sizing-your-cluster

Installing the RHACM Operator

Red Hat recommends that you install the RHACM operator from the **OperatorHub** menu in the OpenShift web console. To begin the installation, use the filter on the OperatorHub page in the web console to find the Advanced Cluster Management for Kubernetes operator from Red Hat.

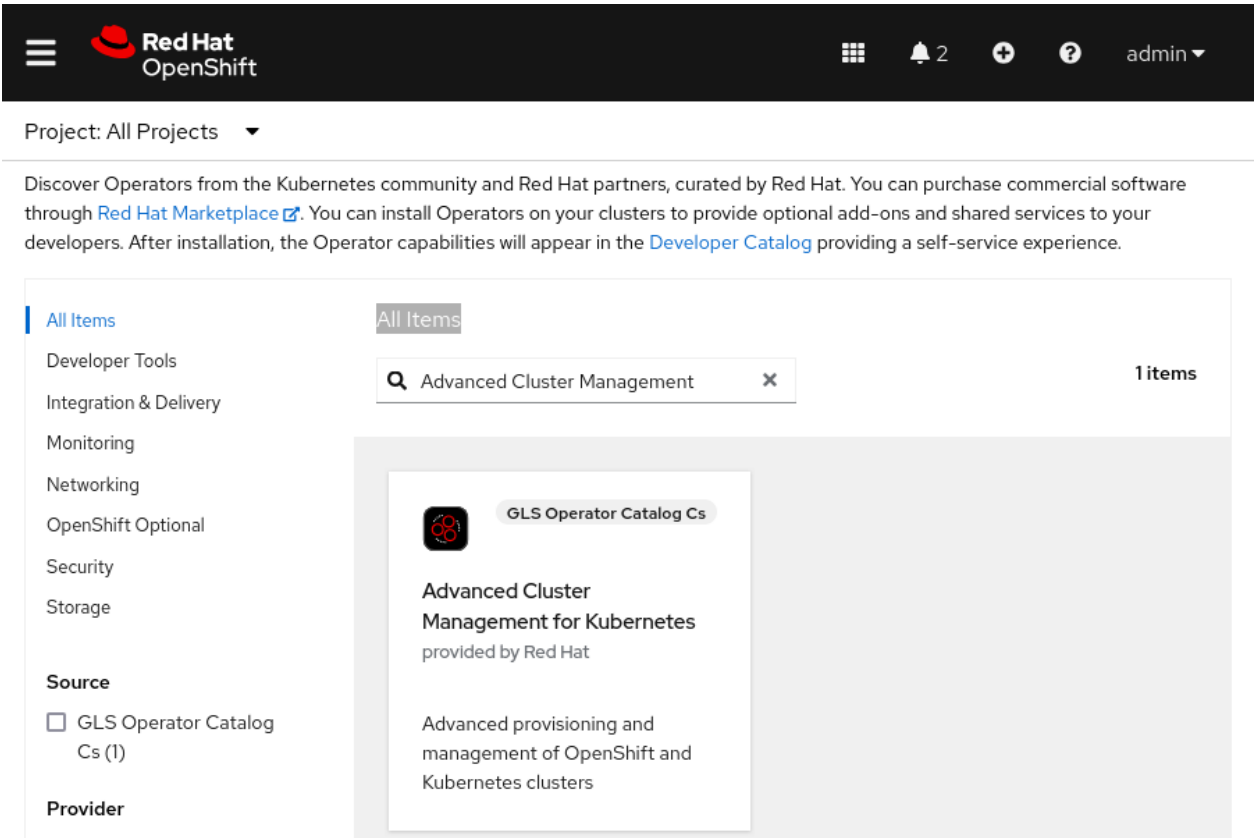


Figure 4. The RHACM operator in OperatorHub

Select the tile for the Advanced Cluster Management for Kubernetes operator and go to the installation page. The installation requires selecting an update channel. An update channel delivers streams of updates for the operator. Typically, update channels correspond to minor software releases. Select an update channel and click **Install** to continue.

Although an operator can be installed in an existing namespace, the recommended namespace for the installation is `open-cluster-management`. The option to select the namespace is on the installation page.

The operator installation also requires choosing between automatic and manual **Update approval** strategies. Choosing the manual update approval strategy requires human intervention whenever a later version of the operator is available from the update stream. Alternatively, the automatic update approval strategy causes an unattended update whenever a later operator version is available from the update stream.

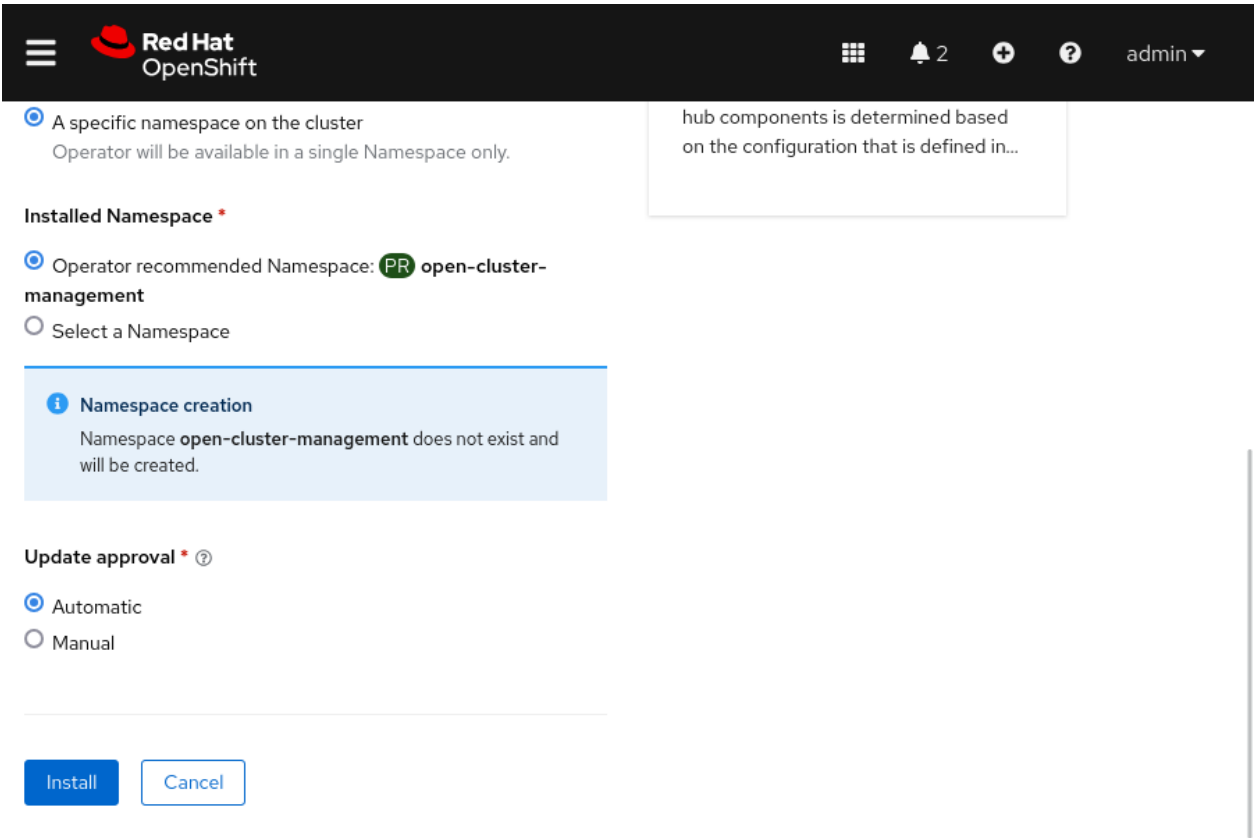


Figure 5. RHACM namespace and update strategy selection

After making these selections, click **Install**. The operator installation process takes a few minutes and shows a green checkmark after the operator installation completes.

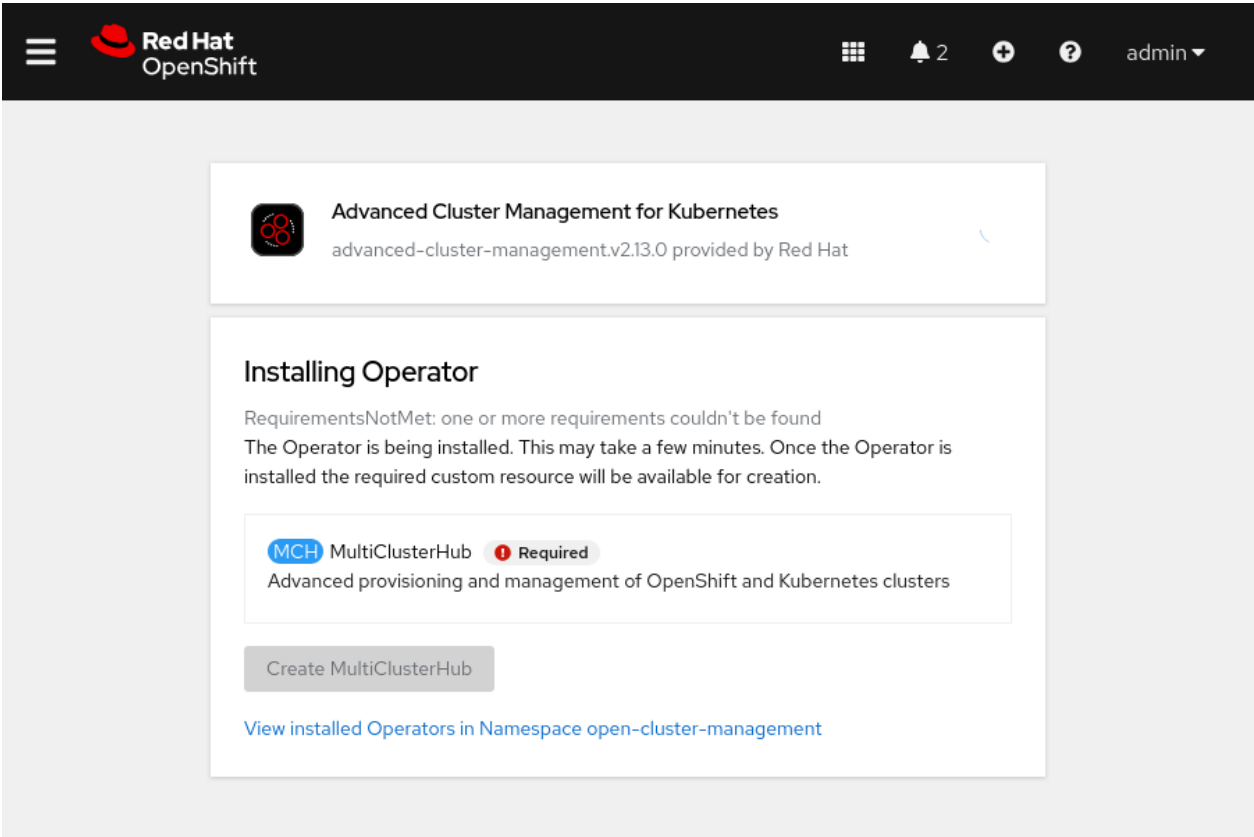


Figure 6. RHACM installation

Creating the Necessary Custom Resource

After installing the operator, you must create the `MultiClusterHub` custom resource to deploy the RHACM services. The `MultiClusterHub` custom resource is a YAML file with the configuration details for the RHACM components. To create this resource by using the web console, click **Create MultiClusterHub** on the resulting page.

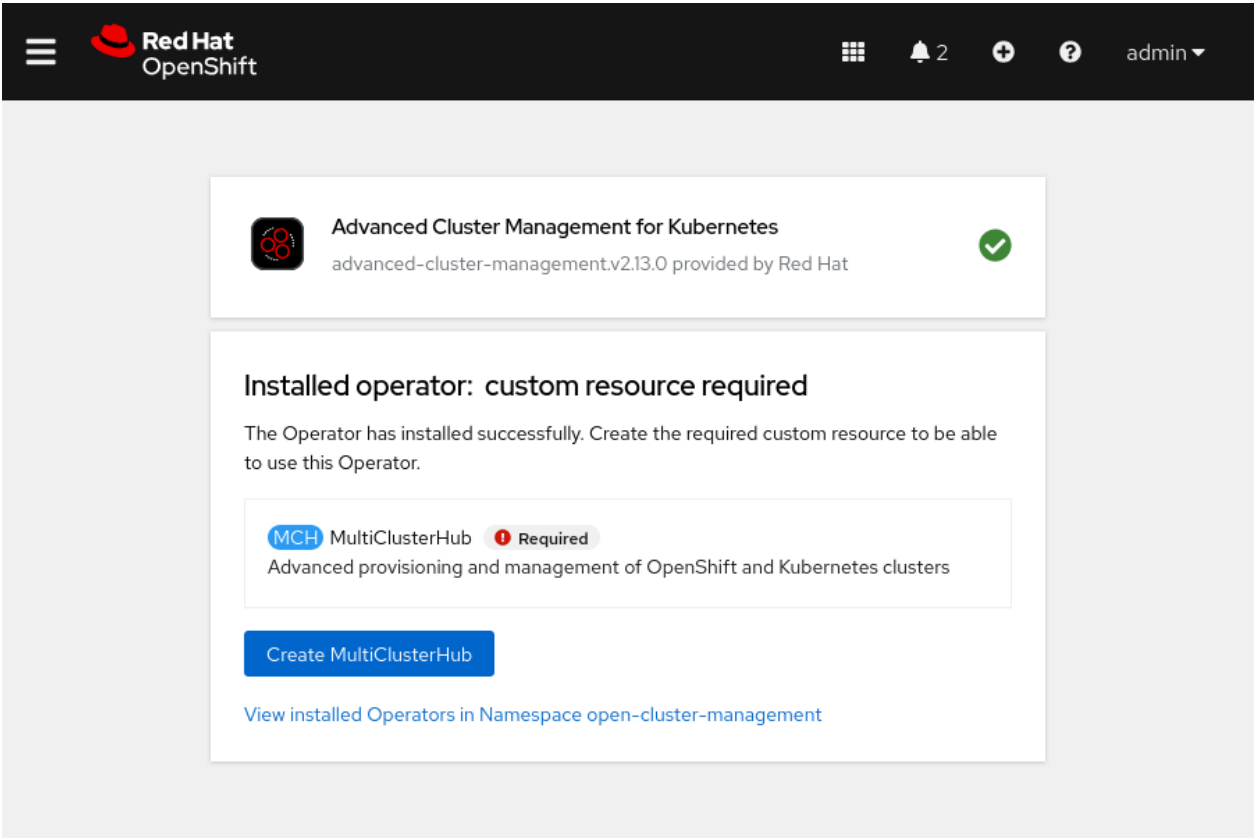


Figure 7. Create a `MultiClusterHub` custom resource

The web console displays the page for creating a `MultiClusterHub` custom resource. Provide the name and any optional labels for the new custom resource and click **Create**.

Red Hat
OpenShift

2

admin ▾

Project: open-cluster-management ▾

Name *

multiclusterhub

Labels

app=frontend

tolerations

Tolerations causes all components to tolerate any taints.

> Advanced configuration

Create

Cancel

Figure 8. RHACM MultiClusterHub custom resource form

Both the web console and command-line approaches are available in connected and disconnected environments. An example follows of the YAML file that creates the MultiClusterHub custom resource from the command line:

```
apiVersion: operator.open-cluster-management.io/v1
kind: MultiClusterHub
metadata:
  name: multiclusterhub
  namespace: open-cluster-management
spec: {}
```

The custom resource creation displays the Phase: Installing status during installation, and takes several minutes to complete. After this custom resource creation process completes, the status changes to Phase: Running.

Note

You can configure RHACM components, such as the RHACM operator and the MultiClusterHub custom resource, to run on infrastructure nodes by using node selectors and tolerations. Infrastructure nodes do not count toward the total number of required OpenShift subscriptions to run the environment.

For more information about configuring infrastructure nodes for RHACM, refer to the *Red Hat Advanced Cluster Management for Kubernetes* installation guide at https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/install/index#config-infra-node-acm

To view the new RHACM web console, set the cluster switcher at the top of the page to **All Clusters**. The resulting page shows the clusters that are registered with the RHACM hub cluster.

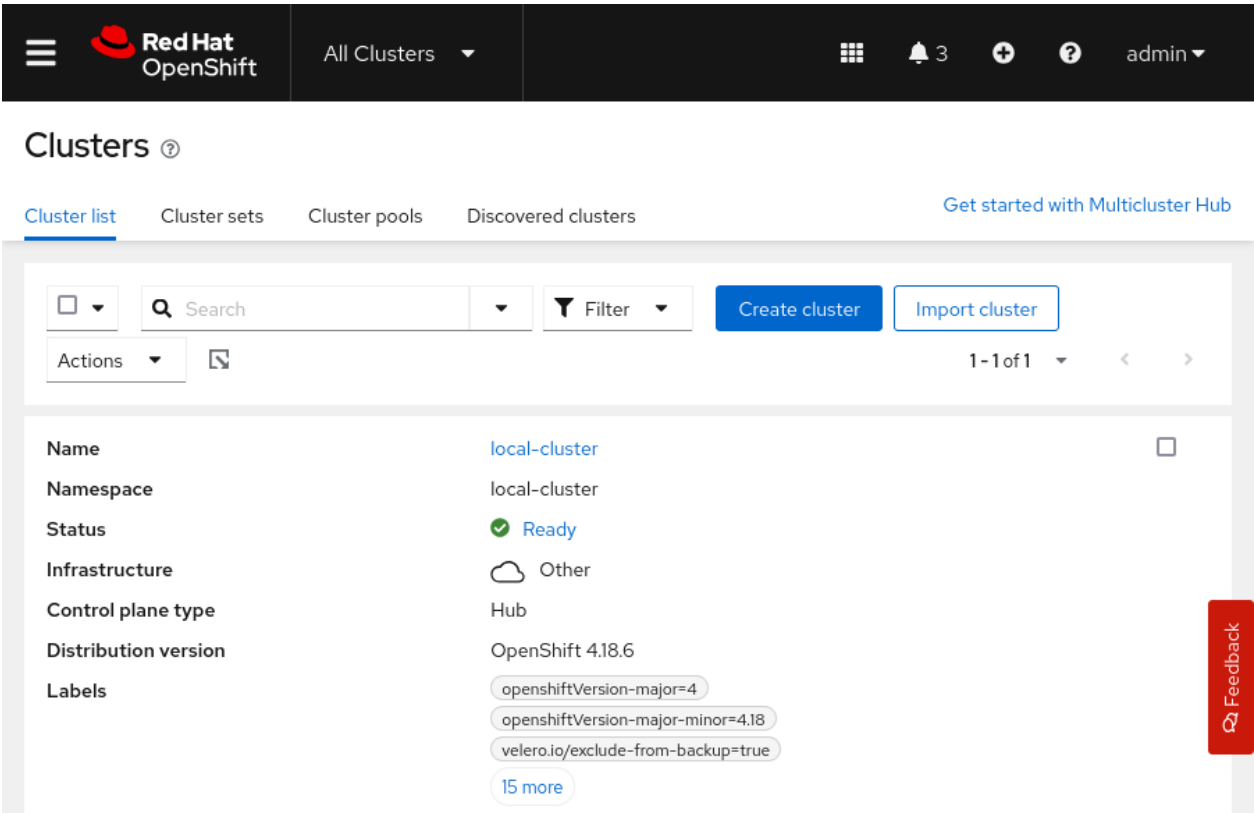


Figure 9. The RHACM Console

Note
 For more information about all the available installation methods, refer to the *Red Hat Advanced Cluster Management for Kubernetes* installation guide at https://access.redhat.com/documentation/en-us/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/install/index

Troubleshooting Installation Issues

Two common issues can occur during the installation of RHACM, which result in either a failed or a stuck installation.

The first issue is when the installation of the RHACM operator gets stuck in the Pending status. To identify this occurrence, check the pods in the installation namespace. An installation with this issue shows the following message or a similar one:

```
reason: Unschedulable
message: '0/6 nodes are available: 3 Insufficient cpu, 3 node(s) had taint {node-role.kubernetes.io/master:
}, that the pod didn't tolerate.'
```

This issue has the following possible causes:

- The OpenShift cluster is not connected to the internet, if the operator is installed from an online repository such as OperatorHub.
- The images for the RHACM operator are not available in the local endpoint, for a disconnected installation.
- The cluster nodes do not have enough resources to install the RHACM operator.

For remediation, verify that the required sizing parameters are available on the nodes for the installation. Additionally, check that the operator images are available to the cluster from a local endpoint when performing a disconnected installation, or that network access is available for a connected cluster.

Commonly, the RHACM operator might encounter installation issues after a prior failure of installation or uninstallation of the operator. This issue can occur when the operator is uninstalled, but the custom resources are not removed, or for a partial installation that was disrupted and that left artifacts in the cluster. In either scenario, it is necessary to remove the custom resources and any artifacts from the previous installation.

To remove all the RHACM installation artifacts, copy the following script to use as a template:

```
MCE_NAMESPACE=<namespace>
oc delete multiclusterengine --all
oc delete apiservice v1.admission.cluster.open-cluster-management.io v1.admission.work.open-cluster-
management.io
oc delete crd discoveredclusters.discovery.open-cluster-management.io discoveryconfigs.discovery.open-
cluster-management.io
oc delete mutatingwebhookconfiguration ocm-mutating-webhook managedclustermutators.admission.cluster.open-
cluster-management.io
oc delete validatingwebhookconfiguration ocm-validating-webhook
oc delete ns $MCE_NAMESPACE
```

Specify the namespace where the RHACM operator was installed in the MCE_NAMESPACE variable, and execute the script. After this script is run and the artifacts are removed, the installation can be retried.

References

For general information about RHACM, refer to the *About* guide in the *Red Hat Advanced Cluster Management for Kubernetes* documentation at https://access.redhat.com/documentation/en-us/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/about/index

For more information about installation, refer to the *Red Hat Advanced Cluster Management for Kubernetes* installation guide at https://access.redhat.com/documentation/en-us/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/install/index

For more information about performance and scalability, refer to the *Performance and Scalability* chapter in the *Red Hat Advanced Cluster Management for Kubernetes* installation guide at https://access.redhat.com/documentation/en-us/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/install/installing#performance-and-scalability

For more information about managing clusters with RHACM, refer to the *Clusters* guide in the *Red Hat Advanced Cluster Management for Kubernetes* documentation at https://access.redhat.com/documentation/en-us/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/clusters/index

For more information about troubleshooting RHACM installations, refer to the *Installation* chapter in the *Red Hat Advanced Cluster Management for Kubernetes* documented troubleshooting information at https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/troubleshooting/index#documented-troubleshooting

1.6. Guided Exercise

Install Red Hat Advanced Cluster Management for Kubernetes

Install the RHACM operator from the operator hub in a Red Hat OpenShift hub cluster.

Outcomes

- Deploy an RHACM multicluster hub by using the operator.
- As the admin user, confirm that the installation succeeds.

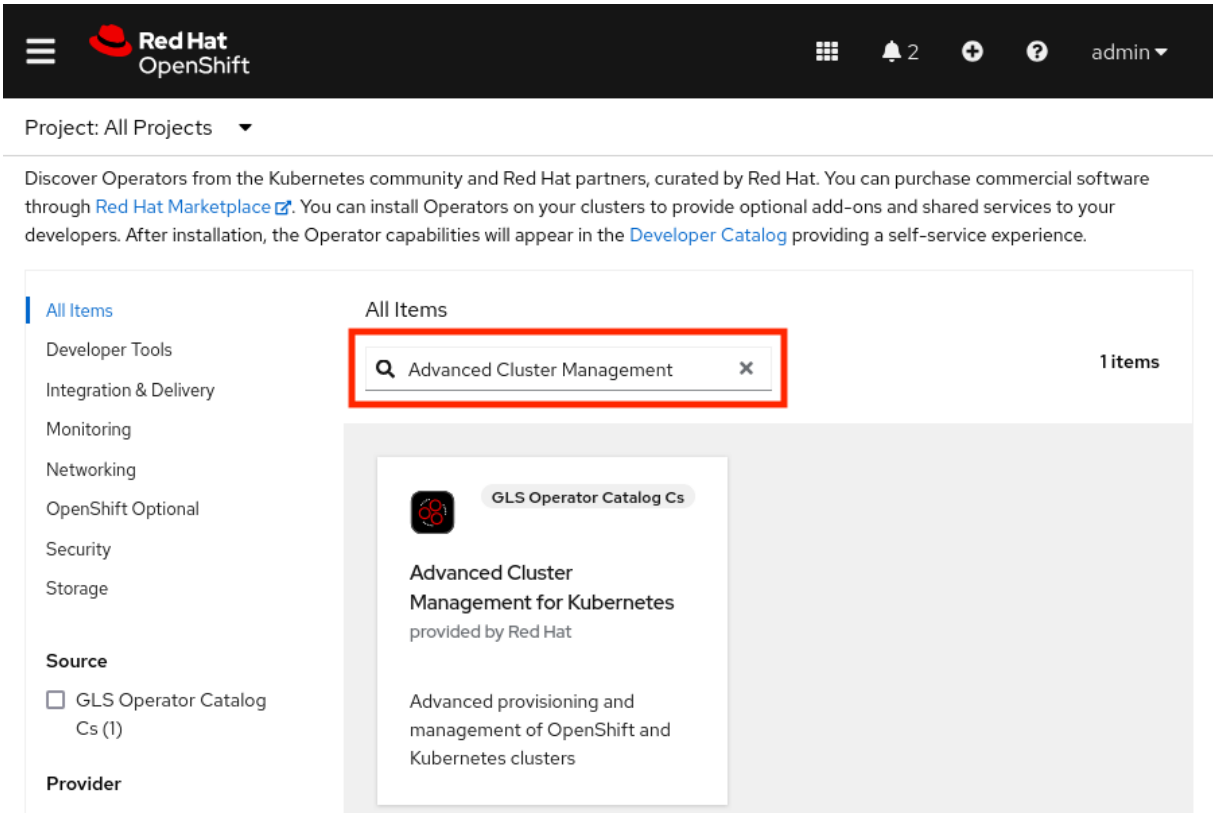
Prerequisites

As the student user on the workstation machine, use the lab command to prepare your environment for this exercise, which ensures that all required resources are available.

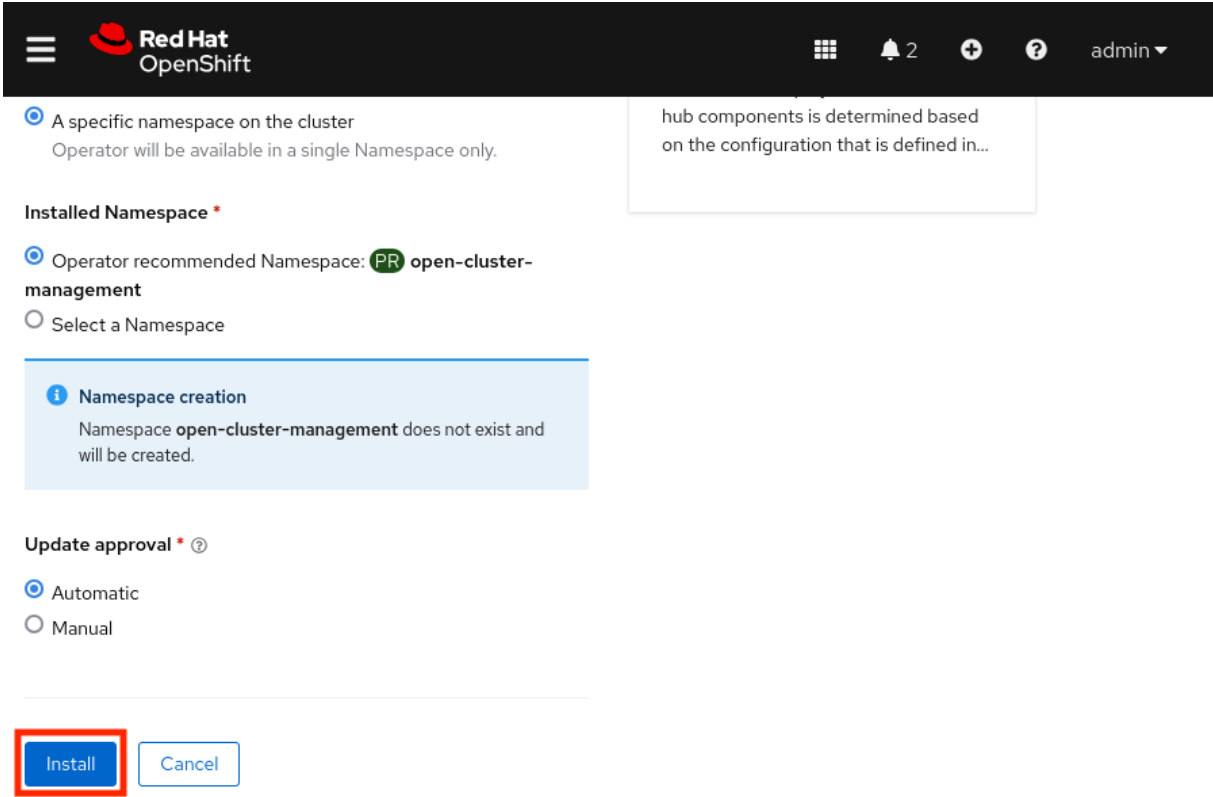
```
[student@workstation ~]$ lab install do00111 ; lab start install-rhacm
```

Instructions

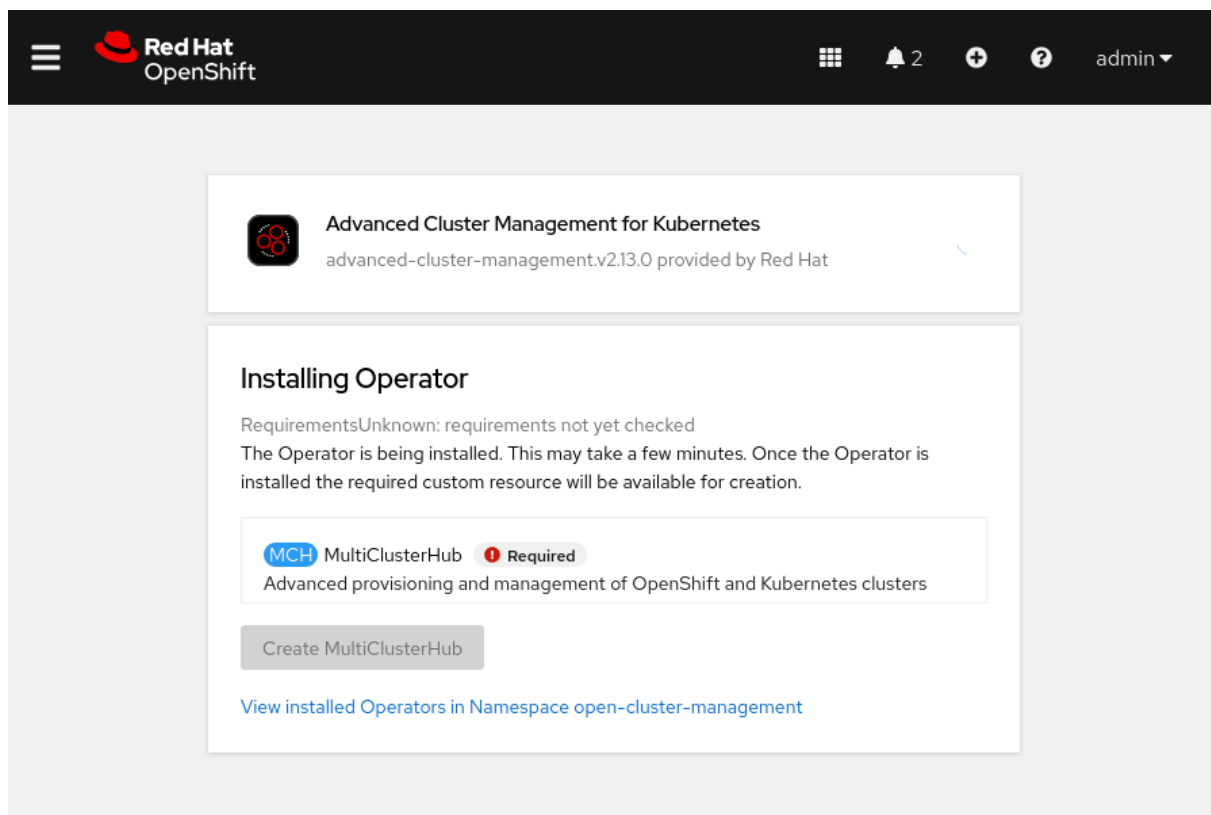
1. Using OperatorHub, install the Advanced Cluster Management for Kubernetes operator in the ocp4.example.com cluster. The web console URL is https://console-openshift-console.apps.ocp4.example.com
 - 1.1. From the workstation machine, open a browser and go to the Red Hat OpenShift web console. When prompted, click **Red Hat Identity Management** and log in as the admin user with redhatocp as the password.
 - 1.2. Install the Advanced Cluster Management for Kubernetes operator from OperatorHub.
 - 1.3. Go to **Operators > OperatorHub** and type Advanced Cluster Management in the **Filter by keyword** field.



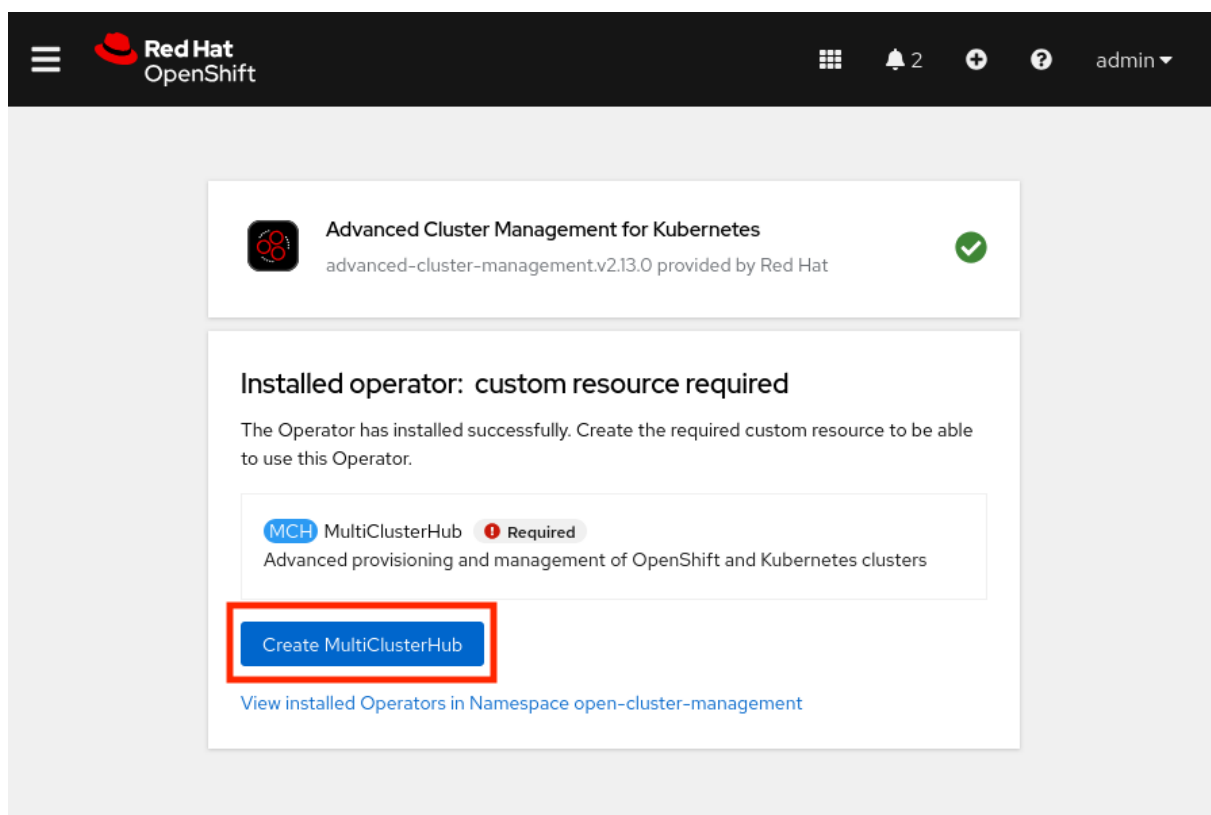
- 1.4. Click **Advanced Cluster Management for Kubernetes**, and then click **Install**.
- 1.5. Leaving the default value, scroll down and click **Install**.



The installation takes a few minutes to complete.



When the process completes, a message confirms the successful installation of the operator and the requirement to create a MultiClusterHub custom resource.

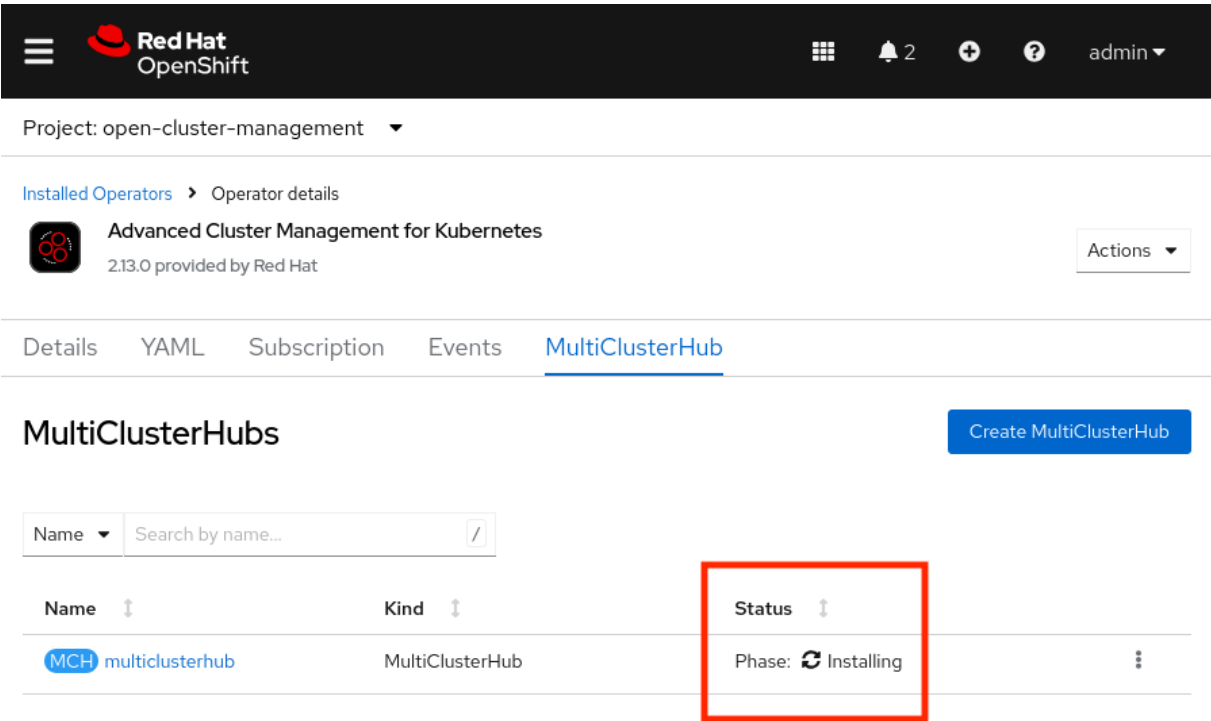


2. Create a MultiClusterHub object.

2.1. Click **Create MultiClusterHub**.

2.2. On the Create MultiClusterHub page, leave the default values and click **Create**. You are redirected to the MultiClusterHubs tab.

Initially, the MultiClusterHub object has Phase: Installing as the status.



After completing the deployment, the Status column shows Phase: Running.

- 3. Verify the deployment of the MultiClusterHub resource.
 - 3.1. Open a terminal and log in to the ocp4.example.com cluster as the admin user with redhatocp as the password. The API server address is https://api.ocp4.example.com:6443

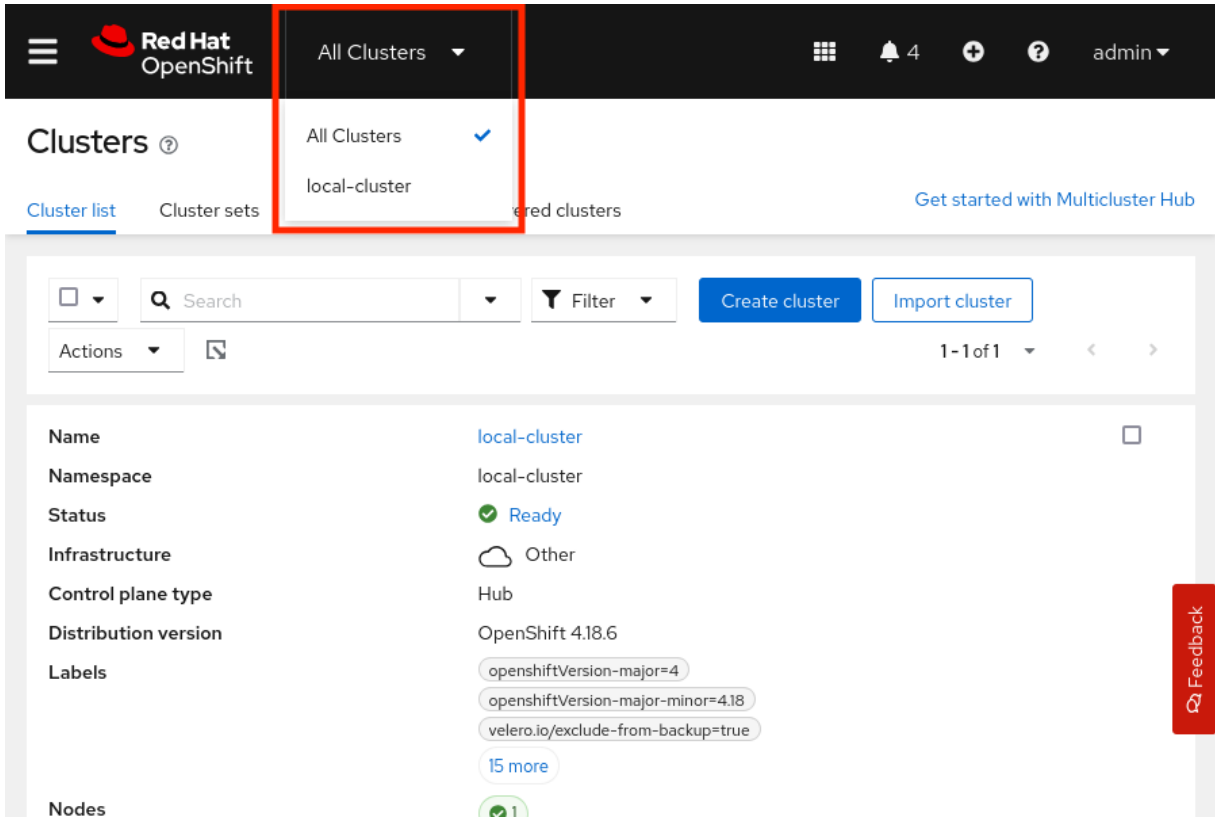
```
[student@workstation ~]$ oc login -u admin -p redhatocp \
https://api.ocp4.example.com:6443
Login successful.
...output omitted...
[student@workstation ~]$
```

- 3.2. Check the status of all the objects in the open-cluster-management namespace.

```
[student@workstation ~]$ oc get all -n open-cluster-management
...output omitted...
```

In the output, which is long, review the status of the pods, services, deployments, replica sets, and stateful sets.

- 4. Verify the functions for the new RHACM installation by using the web console.
 - 4.1. Return to the browser and refresh the web page to ensure that the recent installation is available.
 - 4.2. Use the cluster switcher at the top of the page to change from the local-cluster view to the All Clusters view.



Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish install-rhacm
```


1.7. Quiz

Installing Red Hat Advanced Cluster Management for Kubernetes

Choose the correct answers to the following questions:

1. An organization uses OpenShift clusters that are running on bare metal and on the cloud. Which three prerequisites must be met for a successful installation of RHACM? (Choose three.)
 - ☐ a. A supported OpenShift Container Platform version and the OpenShift Container Platform CLI
 - ☐ b. Access to the Red Hat Advanced Cluster Management operator in the OperatorHub catalog, and access to catalog.redhat.com
 - ☐ c. Internet connection to access the dependencies for the operator
 - ☐ d. The cluster-admin role to create the MultiClusterHub resource in OpenShift Container Platform
2. Which three major capabilities does RHACM provide? (Choose three.)
 - ☐ a. Enforce security and compliance at the target clusters.
 - ☐ b. Run your operations from anywhere that OpenShift Container Platform runs, and manage any Kubernetes cluster in your fleet.
 - ☐ c. Deploy applications to edge devices.
 - ☐ d. Manage your OpenShift Virtualization virtual machine workloads across multiple clusters.
3. How does RHACM address security challenges in multicluster environments?
 - ☐ a. Configures cluster-specific security policies that are tailored to each cloud provider.
 - ☐ b. Relies on Gatekeeper constraints to enforce security without centralized policy management.
 - ☐ c. Enforces uniform security policies by employing HTTPS and integrates Kafka and PostgreSQL.
 - ☐ d. Uses Kafka for data synchronization with optional encryption for performance optimization.
4. Which benefit does a financial services organization gain from the RHACM Grafana dashboards?
 - ☐ a. Automatic workload migration
 - ☐ b. Generates security policies for cluster compliance.
 - ☐ c. Reduces the need for data synchronization.
 - ☐ d. Monitors compliance trends across clusters in real time.
5. Which three options are roles of the hub cluster in RHACM? (Choose three.)
 - ☐ a. Run user workloads.
 - ☐ b. Host the central controller.
 - ☐ c. Manage the deployment of managed clusters.
 - ☐ d. Store policy compliance data.

6. Due to business needs, a streaming service organization that runs RHACM doubled its number of Kubernetes clusters in a short time. The current hub cluster cannot handle so many managed clusters. What would be the best solution to overcome this problem?
- ☐ a. Increase the resources of the hub cluster.
 - ☐ b. Divide the clusters into smaller groups of clusters.
 - ☐ c. Divide the clusters into smaller groups of clusters and configure a hub cluster for each group. Use the multicluster global hub to import the hub clusters and to manage them from a single hub.
 - ☐ d. Delete the necessary managed clusters until the hub cluster can handle them.
7. What is the role of the application lifecycle component in RHACM?
- ☐ a. Manage authentication and RBAC across clusters.
 - ☐ b. Install and upgrade RHACM components.
 - ☐ c. Deploy and manage applications across multiple clusters by using GitOps.
 - ☐ d. Enforce security policies on workloads.
8. Which custom resource provides the multicluster engine for Kubernetes operator?
- ☐ a. MultiCluster
 - ☐ b. Placement
 - ☐ c. MultiClusterEngine
 - ☐ d. ClusterEngine
9. What is the primary role of the `klusterlet` agent in RHACM?
- ☐ a. Connects a managed cluster to the hub cluster.
 - ☐ b. Deploys workloads on the hub cluster.
 - ☐ c. Provides observability dashboards.
 - ☐ d. Installs OpenShift Container Platform on the managed cluster.
10. Which are the two main functions of the Governance component in RHACM? (Choose two.)
- ☐ a. Automatically deploy applications across clusters.
 - ☐ b. Monitor compliance status across managed clusters.
 - ☐ c. Scale the control plane dynamically.
 - ☐ d. Define and apply security and configuration policies.
11. Which challenge do enterprises likely encounter when running applications across diverse cloud providers?
- ☐ a. Over-complex network configurations
 - ☐ b. Reduced security requirements
 - ☐ c. Portability issues due to varying configurations
 - ☐ d. Lack of automatic data synchronization

Solution

1. An organization uses OpenShift clusters that are running on bare metal and on the cloud. Which three prerequisites must be met for a successful installation of RHACM? (Choose three.)
 - ☒ a. **A supported OpenShift Container Platform version and the OpenShift Container Platform CLI**
 - ☒ b. **Access to the Red Hat Advanced Cluster Management operator in the OperatorHub catalog, and access to catalog.redhat.com**
 - ☒ c. **Internet connection to access the dependencies for the operator**
 - ☐ d. The `cluster-admin` role to create the `MultiClusterHub` resource in OpenShift Container Platform
2. Which three major capabilities does RHACM provide? (Choose three.)
 - ☒ a. **Enforce security and compliance at the target clusters.**
 - ☒ b. **Run your operations from anywhere that OpenShift Container Platform runs, and manage any Kubernetes cluster in your fleet.**
 - ☐ c. Deploy applications to edge devices.
 - ☒ d. **Manage your OpenShift Virtualization virtual machine workloads across multiple clusters.**
3. How does RHACM address security challenges in multicloud environments?
 - ☐ a. Configures cluster-specific security policies that are tailored to each cloud provider.
 - ☐ b. Relies on Gatekeeper constraints to enforce security without centralized policy management.
 - ☒ c. **Enforces uniform security policies by employing HTTPS and integrates Kafka and PostgreSQL.**
 - ☐ d. Uses Kafka for data synchronization with optional encryption for performance optimization.
4. Which benefit does a financial services organization gain from the RHACM Grafana dashboards?
 - ☐ a. Automatic workload migration
 - ☐ b. Generates security policies for cluster compliance.
 - ☐ c. Reduces the need for data synchronization.
 - ☒ d. **Monitors compliance trends across clusters in real time.**
5. Which three options are roles of the hub cluster in RHACM? (Choose three.)
 - ☐ a. Run user workloads.
 - ☒ b. **Host the central controller.**
 - ☒ c. **Manage the deployment of managed clusters.**
 - ☒ d. **Store policy compliance data.**
6. Due to business needs, a streaming service organization that runs RHACM doubled its number of Kubernetes clusters in a short time. The current hub cluster cannot handle so many managed clusters. What would be the best solution to overcome this problem?
 - ☐ a. Increase the resources of the hub cluster.
 - ☐ b. Divide the clusters into smaller groups of clusters.
 - ☒ c. **Divide the clusters into smaller groups of clusters and configure a hub cluster for each group. Use the multicloud global hub to import the hub clusters and to manage them from a single hub.**
 - ☐ d. Delete the necessary managed clusters until the hub cluster can handle them.

7. What is the role of the application lifecycle component in RHACM?
- ☐ a. Manage authentication and RBAC across clusters.
 - ☐ b. Install and upgrade RHACM components.
 - ☒ c. **Deploy and manage applications across multiple clusters by using GitOps.**
 - ☐ d. Enforce security policies on workloads.
8. Which custom resource provides the multicluster engine for Kubernetes operator?
- ☐ a. MultiCluster
 - ☐ b. Placement
 - ☒ c. **MultiClusterEngine**
 - ☐ d. ClusterEngine
9. What is the primary role of the `klusterlet` agent in RHACM?
- ☒ a. **Connects a managed cluster to the hub cluster.**
 - ☐ b. Deploys workloads on the hub cluster.
 - ☐ c. Provides observability dashboards.
 - ☐ d. Installs OpenShift Container Platform on the managed cluster.
10. Which are the two main functions of the Governance component in RHACM? (Choose two.)
- ☐ a. Automatically deploy applications across clusters.
 - ☒ b. **Monitor compliance status across managed clusters.**
 - ☐ c. Scale the control plane dynamically.
 - ☒ d. **Define and apply security and configuration policies.**
11. Which challenge do enterprises likely encounter when running applications across diverse cloud providers?
- ☐ a. Over-complex network configurations
 - ☐ b. Reduced security requirements
 - ☒ c. **Portability issues due to varying configurations**
 - ☐ d. Lack of automatic data synchronization

1.8. Summary

In this lesson, you learned to identify common use cases for multiple Kubernetes clusters, and the challenges that multicloud hybrid cloud architectures present, and how Red Hat Advanced Cluster Management for Kubernetes can help your organization in this regard.

You also learned about the RHACM architecture:

- The hub cluster hosts the central controller that runs in an RHACM cluster. From the hub cluster, you can access the RHACM web console to search resources across clusters and to view your topology.
- A managed cluster is an additional Kubernetes cluster that receives and applies requests from the hub cluster, and enables it to service the cluster lifecycle, the application lifecycle, governance, and observability on the managed cluster.
- The `klusterlet` component runs on the managed cluster, and connects the managed cluster to the hub cluster.

You installed the RHACM operator from the operator hub, and deployed the `MultiClusterHub` custom resource, which is responsible for managing, upgrading, and installing hub cluster components.

Next Steps

You might explore other learning opportunities on your own, explore further your RHACM installation, or redo some exercises.

A good next step in your learning journey would be to explore the next lessons in the series:

- Lesson 2: *Managing Clusters by Using Red Hat Advanced Cluster Management for Kubernetes*
- Lesson 3: *Deploying and Managing Policies for Multiple Clusters with Red Hat Advanced Cluster Management for Kubernetes*
- Lesson 4: *Enabling and Customizing the Observability Stack in Red Hat Advanced Cluster Management for Kubernetes*
- Lesson 5: *Managing the Multicloud Application Lifecycle by Using GitOps Practices and Red Hat Advanced Cluster Management for Kubernetes*
- Lesson 6: *Managing Virtual Machines for Multiple Clusters with Red Hat Advanced Cluster Management for Kubernetes*

Chapter 2.

Managing Clusters by Using Red Hat Advanced Cluster Management for Kubernetes

Goal

Import and manage a cluster by using the Red Hat Advanced Cluster Management for Kubernetes (RHACM) web console, configure user access to clusters, and troubleshoot common import issues.

Sections

- Cluster Lifecycle Management by Using the RHACM Web Console (and Guided Exercise)
- Navigating the RHACM Web Console (and Guided Exercise)
- Configuring Access Control for Multicluster Management (and Guided Exercise)
- Troubleshooting Common Import Issues for Managed Clusters (and Guided Exercise)

Lab

- Manage a Cluster by Using Red Hat Advanced Cluster Management for Kubernetes

2.1. Cluster Lifecycle Management by Using the RHACM Web Console

Objectives

- Describe the cluster lifecycle, explore the RHACM web console, and import and manage a managed cluster by using the RHACM web console.

Red Hat Advanced Cluster Management Lifecycle

RHACM gives you visibility into your OpenShift and Kubernetes clusters from a single user interface.

The multicluster engine operator provides cluster lifecycle management capabilities for RHACM hub clusters. From the hub cluster, you can create and manage clusters, or delete any clusters that you created. You can also hibernate, resume, or detach clusters.

The multicluster engine operator uses the Hive operator that is provided with Red Hat OpenShift Container Platform (RHOC) to provision clusters for all providers except the on-premise clusters and hosted control planes. When provisioning the on-premise clusters, multicluster engine operator uses the central infrastructure management and the assisted installer function from RHOC.

The cluster lifecycle supports all providers that are certified through the Cloud Native Computing Foundation (CNCF) Kubernetes Conformance Program. The following cluster types are supported:

- Amazon Web Services
- Amazon Web Services GovCloud
- Microsoft Azure
- Google Cloud Platform
- VMware vSphere
- Red Hat OpenStack Platform
- On-premise

Infrastructure

The Infrastructure menu in the RHACM console is where you centrally manage all the cluster resources.

Table 8. Web Console Infrastructure Menu

Infrastructure Pane	Description
Clusters	Use to create clusters or to import existing clusters.
Automation	Use to create an Ansible template. You can create prehook and posthook Ansible job instances that occur before or after creating or upgrading your clusters.
Host Inventory	Use to create a host inventory to discover physical or virtual machines that you can install your RHOC clusters on.

Infrastructure Pane	Description
Virtual Machines	Use to manage virtual machines across all clusters.

Creating a Cluster

RHACM includes the multicluster engine operator, which manages clusters for RHOCP. To create a cluster, the multicluster engine operator uses the Hive operator to provision clusters. Creating a cluster starts with creating the `ClusterDeployment` Custom Resource (CR), which is the core Hive resource to control the lifecycle of a cluster and the Hive API entry point. The `ClusterDeployment` CR references the short name of a `ClusterImageSet` CR to specify the RHOCP version to the new cluster.

The `ClusterImageSet` CR is a Hive-provided Custom Resource Definition (CRD) where you can centrally define and reuse the RHOCP installer images across many cluster-provisioning workflows. When you start new clusters, you point your `ClusterDeployment` CR at one of these cluster image sets so that all your clusters use the same RPM-hosted installer image, which makes upgrades and consistency easier to manage.

Cluster image sets provide key features that benefit the cluster creation process:

Immutable reference

Each `ClusterImageSet` CR captures a specific RHOCP installer image digest. The image digest never changes, so any `ClusterDeployment` CR that points to the `ClusterImageSet` CR always gets that same installer image.

Decoupling installer from CRs

Rather than embedding long release image URLs in each `ClusterDeployment` CR, you provide a short image set name. Upgrades and deploying new minor versions involve creating a `ClusterImageSet` CR, not editing every `ClusterDeployment` CR.

Central catalog of images

Administrators maintain a library of approved RHOCP installer images. Select the latest-approved or stable-4.x.x `ClusterImageSet` CR to avoid manually searching through quay.io or registry.redhat.io.

The following manifest shows the most important fields in a `ClusterImageSet` CR:

```
apiVersion: hive.openshift.io/v1
kind: ClusterImageSet
metadata:
  labels:
    channel: fast
    visible: 'true'
  name: img4.x.1-x86-64-appsub ①
spec:
  releaseImage: quay.io/openshift-release-dev/ocp-release:4.x.1-x86_64 ②
```

- ① A stable identifier that `ClusterDeployment.spec.imageSetRef.name` uses
- ② Full pull-spec for the RHOCP installer image (digest or tag)

In your `ClusterDeployment` CR manifest, you reference the image set by name:


```
apiVersion: hive.openshift.io/v1
kind: ClusterDeployment
metadata:
  name: mycluster
  namespace: mynamespace
spec:
  baseDomain: hive.example.com
  clusterName: mycluster
  platform:
    aws:
      credentialsSecretRef:
        name: mycluster-aws-creds
      region: us-east-1
  provisioning:
    imageSetRef:
      name: img4.x.1-x86-64-appsub
    installConfigSecretRef:
      name: mycluster-install-config
    sshPrivateKeySecretRef:
      name: mycluster-ssh-key
    pullSecretRef:
      name: mycluster-pull-secret
```

The Hive controller watches the `imageSetRef.name` field, looks up the corresponding `ClusterImageSet` CR, and uses its `spec.releaseImage` to provision the cluster.

Upgrading a Cluster

If you later create a `ClusterImageSet` CR named `ocp-4.19.0-x86_64`, teams can opt in by changing the `imageSetRef.name` field.

Importing Existing Clusters by Using the RHACM Web Console

Creating the `MultiClusterHub` CR object automatically imports the local cluster, which is the OpenShift cluster where the hub cluster runs, into RHACM. To import an existing cluster from the RHACM web console, administrators must follow these steps:

1. From the RHACM web console, go to the **Infrastructure > Clusters** menu.
2. Click **Import cluster**.
 - a. Provide the required import cluster name.
 - b. Optionally select a cluster set for the import. The import uses the default managed cluster set if nothing is selected.
 - c. Optionally add labels. If you remove a hub cluster and try to import it again, you must add the `local-cluster:true` label to the `ManagedCluster` CR.
 - d. For the Import mode, use `Run import` command manually.
- e. Click **Next**.
3. Click **Next** on the Automation page.
4. Click **Generate command** on the Review page.
5. Click **Copy command** to copy the generated command and token to the clipboard.
6. Paste and run the copied command in a terminal session on the target cluster.
7. Return to the **Infrastructure > Clusters** menu.

The newly imported cluster is immediately displayed in the **Infrastructure > Clusters** menu. It takes several minutes to see all the cluster details, because RHACM is installing the add-ons in the managed cluster.

The Import mode has the following options:

Table 9. Import Mode Options

Import Mode	Description
Run import command manually	After completing and submitting the information in the console, run the provided command on the target cluster to import the cluster.
Enter your server URL and API token for the existing cluster	Provide the server URL and API token of the cluster that you are importing.
Kubeconfig	Copy and paste the contents of the kubeconfig file of the cluster that you are importing.
Import from Red Hat OpenShift cluster manager	Provide the namespace, credentials, and cluster ID to import from Red Hat OpenShift Service on AWS (ROSA) clusters.

Scaling a Cluster

The SiteConfig operator offers a template-driven cluster provisioning solution with the ClusterInstance API. The SiteConfig operator dynamically generates installation manifests based on user-defined templates that are instantiated from the data in the ClusterInstance CR.

The following process is a high-level overview of the steps:

1. Create one or more sets of installation templates that are used on the hub cluster.
2. Create a ClusterInstance CR that references those installation templates and supporting manifests.
3. After the resources are created, the SiteConfig operator reconciles the ClusterInstance CR by populating the template fields that are referenced in the CR.
4. The SiteConfig operator validates and renders the installation manifests, and then the operator performs a dry run.
5. If the dry run is successful, then the manifests are created, and then the underlying operators consume and process the manifests.
6. The installation begins.
7. The SiteConfig operator continuously monitors for changes in the associated ClusterDeployment CR and updates the ClusterInstance status field accordingly.

To scale the cluster, set the `clusterinstance.spec.nodes` list to use the appropriate number of nodes for the installation.

Removing Imported Clusters from RHACM

To remove an imported cluster from the RHACM console, go to the Clusters page, and from the options for the cluster to remove, click **Actions > Detach cluster**. The process begins after confirming the detach cluster request.

Because RHACM removes all the add-ons and other RHACM components from the managed cluster, the detach process takes a few minutes.

To clean up all the elements that RHACM creates in an imported cluster, remove the following elements from the cluster:

1. The `klusterlet` cluster role
2. The `open-cluster-management:klusterlet-admin-aggregate-clusterrole` cluster role
3. The `klusterlet` cluster role binding

References

For more information about creating clusters with RHACM, refer to the *Clusters* guide in the *Red Hat Advanced Cluster Management for Kubernetes* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/clusters/index#create-intro

For more information about host inventories with RHACM, refer to the *Clusters* guide in the *Red Hat Advanced Cluster Management for Kubernetes* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/clusters/index#create-host-inventory-console

For more information about supported managed cluster providers, refer to the *Clusters* guide in the *Red Hat Advanced Cluster Management for Kubernetes* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/clusters/index#cluster_mce_overview

For more information about the Site Config operator, refer to the *Multicluster Engine Operator with Red Hat Advanced Cluster Management* guide at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/multicluster_engine_operator_with_red_hat_advanced_cluster_management/index

2.2. Guided Exercise

Manage the Cluster Lifecycle by Using the RHACM Web Console

Import and remove an existing cluster by using the RHACM web console.

Outcomes

- Import an existing cluster into RHACM.
- Remove an imported cluster.

Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

This command ensures that the RHACM operator is installed.

```
[student@workstation ~]$ lab install do0012l ; lab start manage-lifecycle
```

Instructions

1. Log in to the RHOCP cluster, and verify that the Red Hat Advanced Cluster Manager operator is installed and running. Open the Red Hat OpenShift web console.

- 1.1. In a terminal window, log in to the RHOCP cluster as the `admin` user with `redhatocp` as the password.

```
[student@workstation ~]$ oc login -u admin -p redhatocp \
https://api.ocp4.example.com:6443
Login successful.
...output omitted...
```

- 1.2. Check the status of all the objects in the `open-cluster-management` namespace.

```
[student@workstation ~]$ oc get deployments,pods,services,replicasets \
-n open-cluster-management
...output omitted...
```

The output is long. Review the status of the pods, services, deployments, and replica sets.

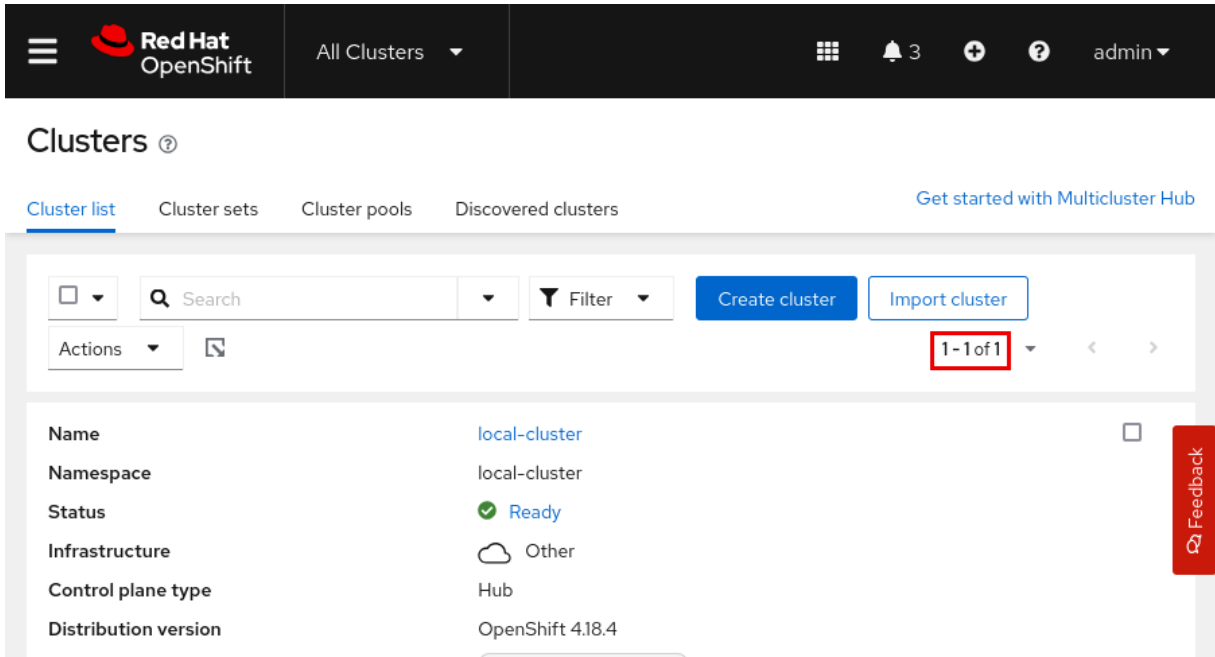
- 1.3. Retrieve the RHOCP web console URL.

```
[student@workstation ~]$ oc whoami --show-console
https://console-openshift-console.apps.ocp4.example.com
...output omitted...
```

- 1.4. Access the RHOCP web console at `https://console-openshift-console.apps.ocp4.example.com` in the browser. Click **Red Hat Identity Management** and log in as the `admin` user with `redhatocp` as the password.

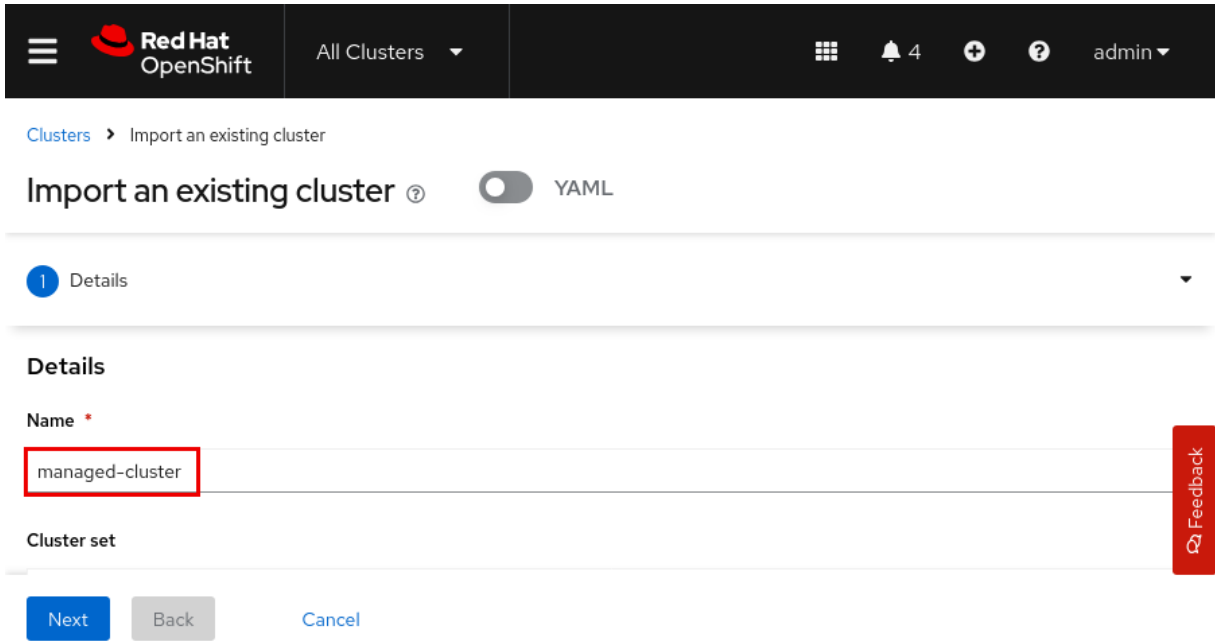
2. Import the `ocp4-mng.example.com` cluster.

- 2.1. View the available clusters. Go to the **Infrastructure > Clusters** menu.



Notice that the local cluster, which is the ocp4.example.com cluster, where the hub cluster runs, is automatically managed.

- 2.2. Generate the code to import an existing cluster. Click **Import Cluster**. On the **Import an existing cluster** page, use the managed-cluster name to identify this cluster as follows:



- 2.3. Click **Next**. On the Automation page, click **Next**. Click **Generate command**.

Red Hat OpenShift

All Clusters

Clusters > Import an existing cluster

Import an existing cluster ? ☐ YAML

3 Review

Details

Name managed-cluster

Additional labels

Import mode Run import commands manually

Namespace

Generate command Back Cancel

Feedback

2.4. Click **Copy command**.

Red Hat OpenShift

All Clusters

Clusters > managed-cluster

managed-cluster

Overview Nodes Add-ons Actions

Cluster is pending import

Run a command

1. Copy this command

For OpenShift 4 and Kubernetes 1.16.0 clusters or above

Copy command

Feedback

Note

The previous step copies a command to the clipboard to be executed in a terminal after logging in to the `ocp4-mng.example.com` managed cluster.

2.5. From the terminal, log in to the `ocp4-mng.example.com` cluster as the `admin` user with `redhatocp` as the password. The API server address is `https://api.ocp4-mng.example.com:6443`.

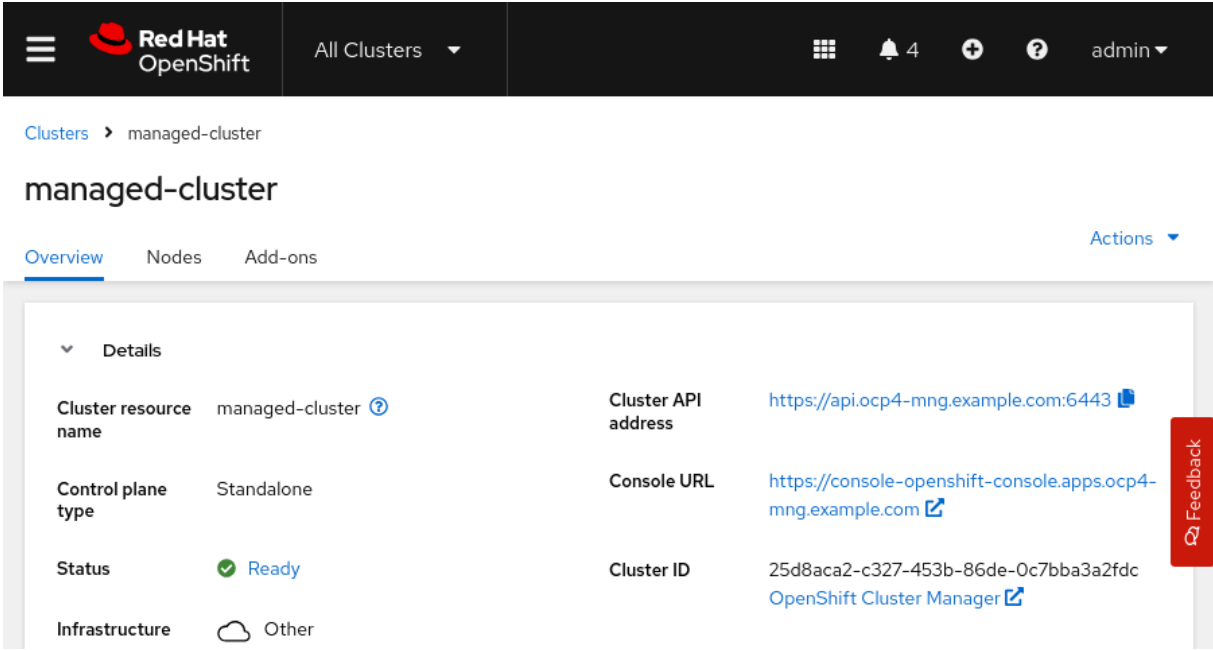
```
[student@workstation ~]$ oc login -u admin -p redhatocp \
https://api.ocp4-mng.example.com:6443
Login successful.
...output omitted...
```

2.6. Paste and run the copied command in the terminal. Paste the clipboard content with `Ctrl+Shift+V` or another method. The command, which is mostly Base64-encoded, is long.

```
[student@workstation ~]$ echo "Ci0tLQph....G9ydCBhZ2Fpbi4=" | base64 -d | \
kubectl apply -f - || echo "VGhlIGNsdXN0...hZ2Fpbi4=" | base64 -d | \
customresourcedefinition.apiextensions.k8s.io/klusterlets.operator.open-cluster-management.io
created
namespace/open-cluster-management-agent created
serviceaccount/klusterlet created
clusterrole.rbac.authorization.k8s.io/klusterlet created
clusterrole.rbac.authorization.k8s.io/klusterlet-bootstrap-kubeconfig created
clusterrole.rbac.authorization.k8s.io/open-cluster-management:klusterlet-admin-aggregate-
clusterrole created
clusterrolebinding.rbac.authorization.k8s.io/klusterlet created
deployment.apps/klusterlet created
secret/bootstrap-hub-kubeconfig created
klusterlet.operator.open-cluster-management.io/klusterlet created
secret/open-cluster-management-image-pull-credentials created
[student@workstation ~]$
```

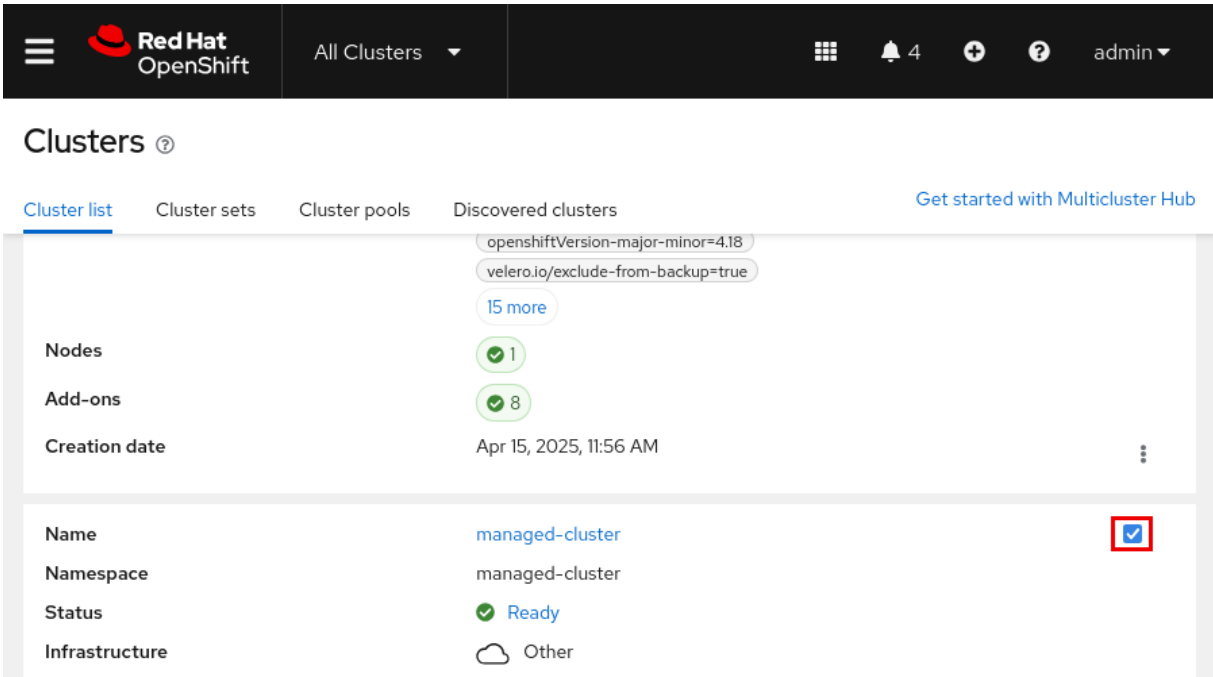
3. Verify the import of the managed cluster.

- 3.1. Return to the RHOCP web console. The console displays the managed cluster shortly after you run the import code. Initially, limited information appears. Expand the **Details** tab. After a few minutes, the cluster details appear.

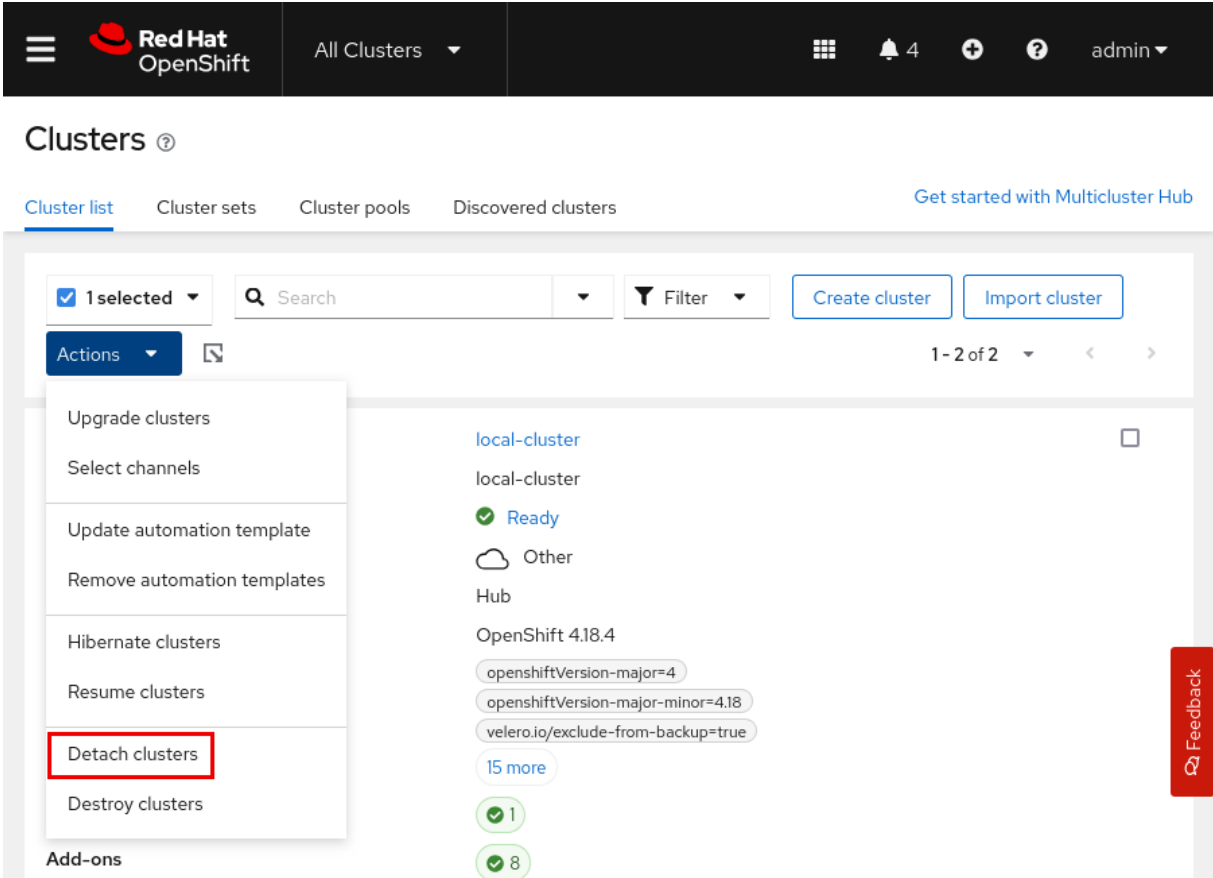


4. Remove the imported cluster.

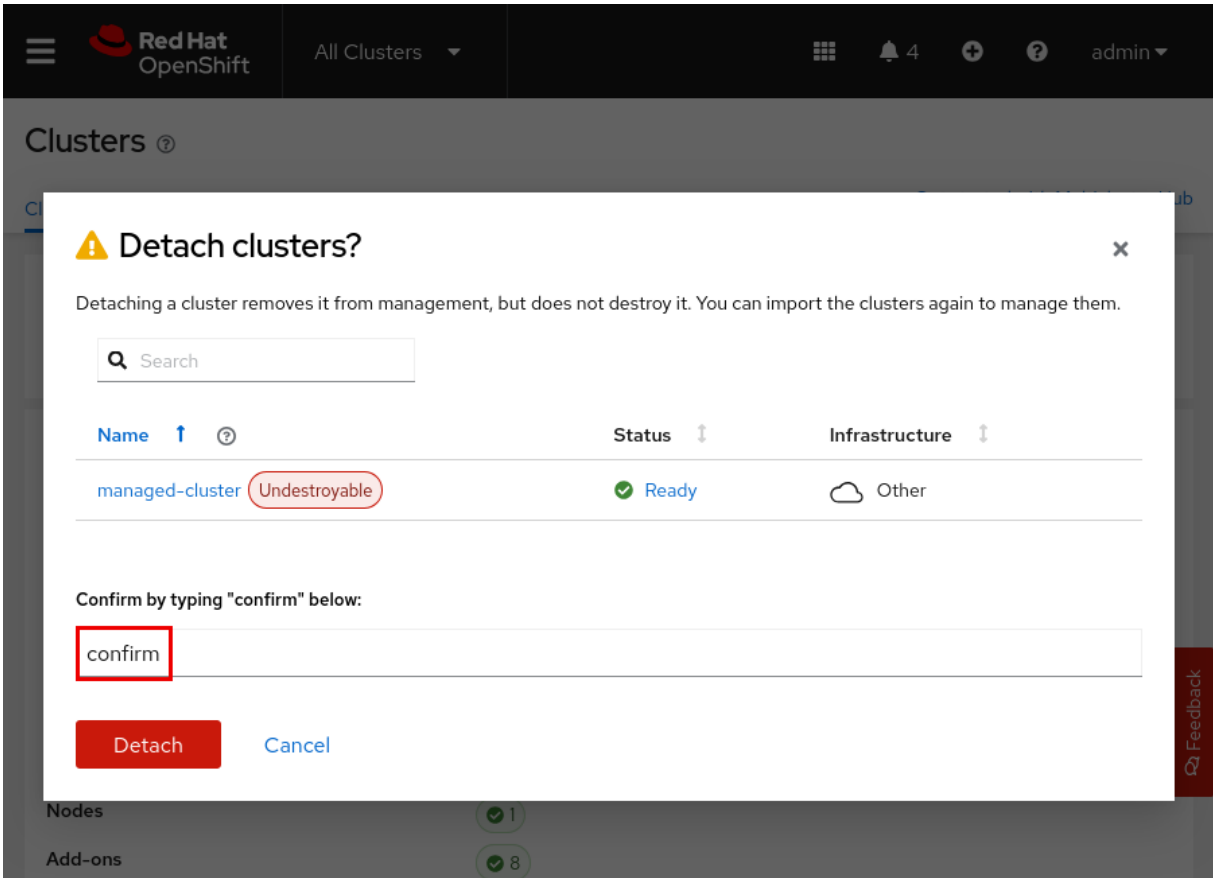
- 4.1. Go to the **Infrastructure > Clusters** pane. Use the selection icon to select only the managed cluster.



4.2. Click **Action > Detach clusters**.



4.3. Enter confirm in the confirmation dialog box and click **Detach**.



The status changes to Detaching, and the managed cluster detaches after a few minutes.

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish manage-lifecycle
```

2.3. Navigating the RHACM Web Console

Objectives

- Locate objects across a fleet of managed clusters by using the RHACM search engine and enumerate RHACM features through the web console.

RHACM Web Console Overview

The RHACM web console provides a single access point to all the management and observation features of a fleet of Kubernetes or OpenShift clusters.

The RHACM installation process deploys the console as a plug-in to the hub OpenShift web console. Consequently, you access the RHACM web console from the console of the hub cluster:

1. Log in to the OpenShift web console of the hub cluster.
2. Select **All Clusters** in the cluster switcher to access the RHACM web console.

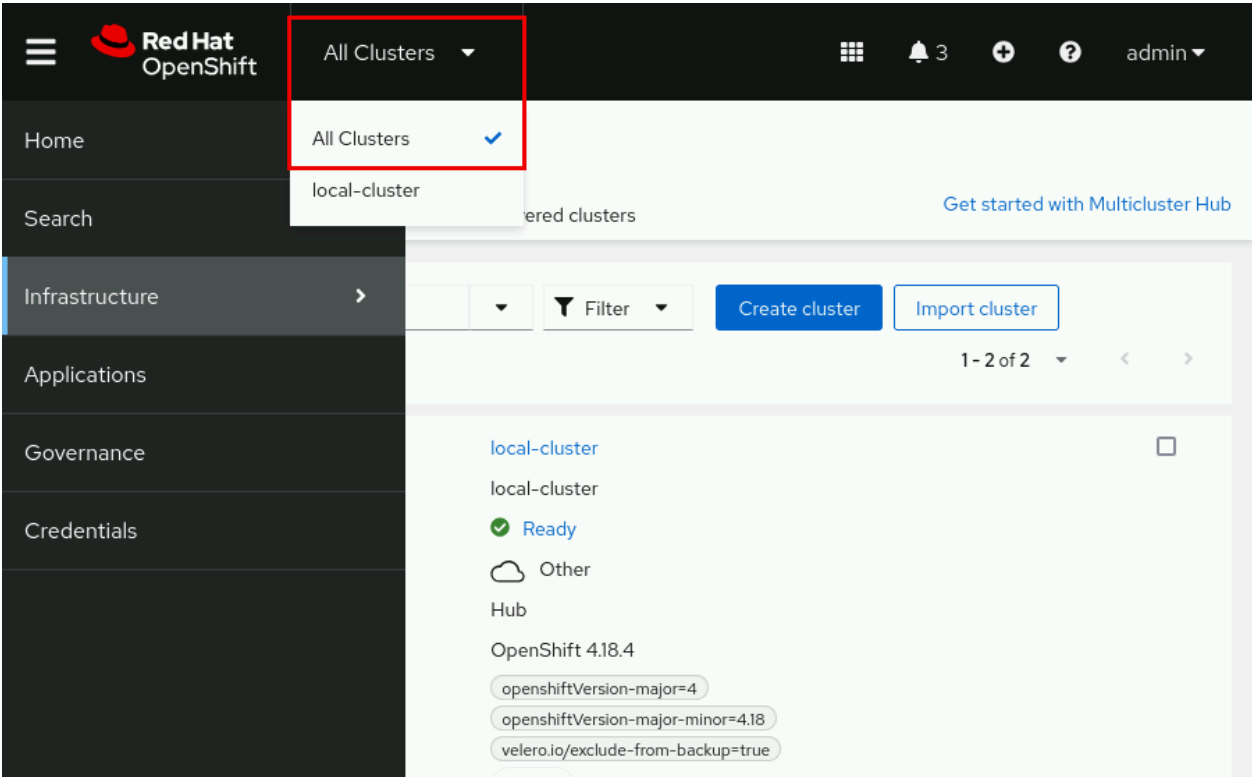


Figure 10. Accessing the RHACM web console

The RHACM web console gives you access to the RHACM components through the navigation panel. The following sections describe the items in this panel.

Home

The **Home** section provides information about RHACM and its use cases, and includes shortcuts to the main product features. The section contains two submenus:

Welcome

The **Home > Welcome** page provides information and links to access the main RHACM features. This page includes a link for contacting the Red Hat support team.

Overview

The **Home > Overview** page provides a summary and a high-level overview of the details and statuses of the managed clusters.

Infrastructure

The **Infrastructure** section gives access to cluster lifecycle management, bare-metal asset management, Ansible automation configuration, and virtual machine management. This section contains the following submenus:

Clusters

The **Infrastructure > Clusters** page gives access to the following features:

- Creating or upgrading a cluster in different public cloud providers.
- Importing an existing cluster into RHACM.
- Manually scaling the clusters, or enabling autoscaling. The process of resizing a cluster is different depending on whether you created the cluster by using RHACM, or the cluster existed and you imported it into RHACM.
- Using RHACM features such as cluster sets, cluster pools, and discovering clusters.

Automation

Use the **Infrastructure > Automation** page to create Ansible job templates and run Ansible jobs automatically during different stages of a cluster lifecycle. Creating Ansible templates requires the Red Hat Ansible Automation Platform operator to be installed on the RHACM hub cluster.

Host inventory

A host inventory is a collection of physical or virtual servers that you can configure to run your OpenShift clusters.

Virtual machines

The **Infrastructure > Virtual machines** page lists the Red Hat OpenShift Virtualization VMs that are running in your cluster fleet. You can manage your VMs directly from the RHACM web console. For example, you can start, stop, or delete a VM, or access its console. You can also edit the configuration, review the resources that the VM is using, or access the VM metrics.

Applications

Use the **Applications** section to create, deploy, or manage applications across a cluster fleet. For this component, RHACM integrates with Red Hat OpenShift GitOps, which is based on Argo CD.

Governance

Use the **Governance** section to control and enforce compliance standards for the cluster fleet. You create and manage policies and policy controllers, and apply them to the cluster fleet.

Credentials

A credential stores the connection details that RHACM uses for accessing an external entity, such as a cloud provider or an external application platform. RHACM uses the following credential types:

- Cloud provider credentials, such as Amazon Web Services, Google Cloud Platform, and Microsoft Azure

- Data center credentials, such as Red Hat OpenStack Platform and bare metal resources
- Automation and other credentials, such as Red Hat Ansible Automation Platform

Use the **Credentials** section to create and administer credentials for all your cloud providers and systems.

RHACM Search

RHACM search provides visibility of the Kubernetes objects across the cluster fleet from a single user interface. You access the search interface in the RHACM web console by selecting **Search** in the navigation panel.

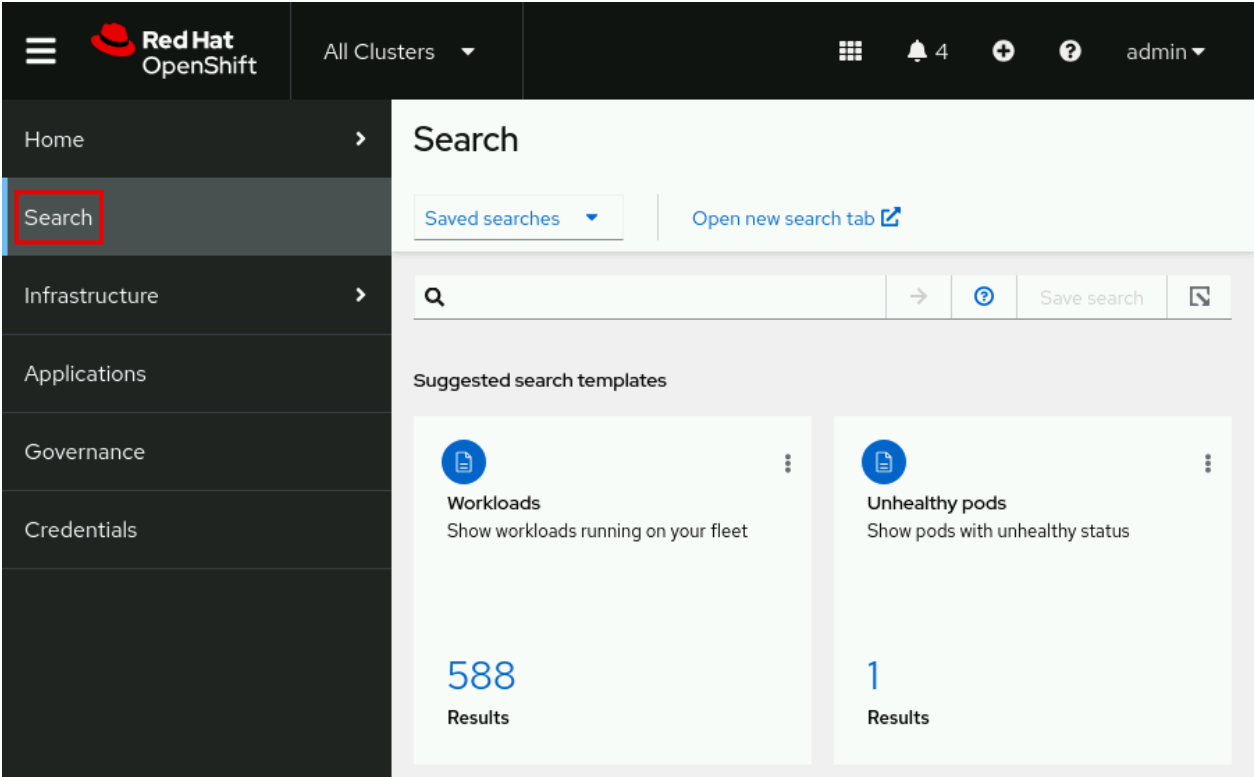


Figure 11. Searching for Kubernetes objects

The RHACM search component indexes and stores Kubernetes objects in the cluster fleet, and determines their relationships with other objects, such as the relationship between a pod and its deployment.

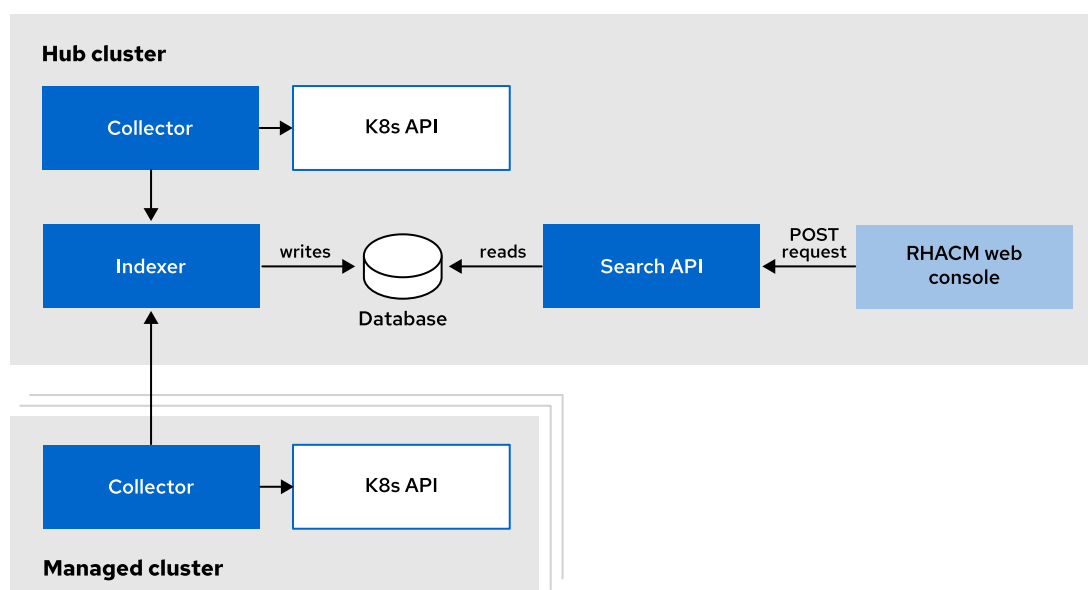


Figure 12. Search architecture

The RHACM search architecture consists of the following components:

Collector

A collector is deployed in each of the fleet clusters. In the hub cluster, the collector is deployed in the `open-cluster-management` namespace. In the other managed clusters, the collector is deployed in the `open-cluster-management-agent-addon` namespace, as part of the `search-collector` add-on.

The collector uses the Kubernetes API to collect the information about the objects, and then the collector computes relationships for objects within the managed clusters.

Note

Although by default, the collector collects all the Kubernetes resources, you can configure search to omit certain resources. Consult the *Web Console* documentation in the references section for more information.

Indexer

The indexer is deployed only in the RHACM hub cluster, in the `open-cluster-management` namespace. The indexer receives data from the collectors and writes that data to a PostgreSQL database.

Database

In the hub cluster, the PostgreSQL pod runs in the `open-cluster-management` namespace. The database saves the data that it receives from the indexer. The search API queries the database for replying to search requests.

Search API

The search API gives access to the data in the database, and enforces role-based access control (RBAC). The search API uses the RBAC of each managed cluster. For example, you can search for objects in a managed cluster only where you have authorization.

Using RHACM Search

Search comes with predefined search templates for the most common queries. Users can also save up to 10 search queries for easy retrieval.

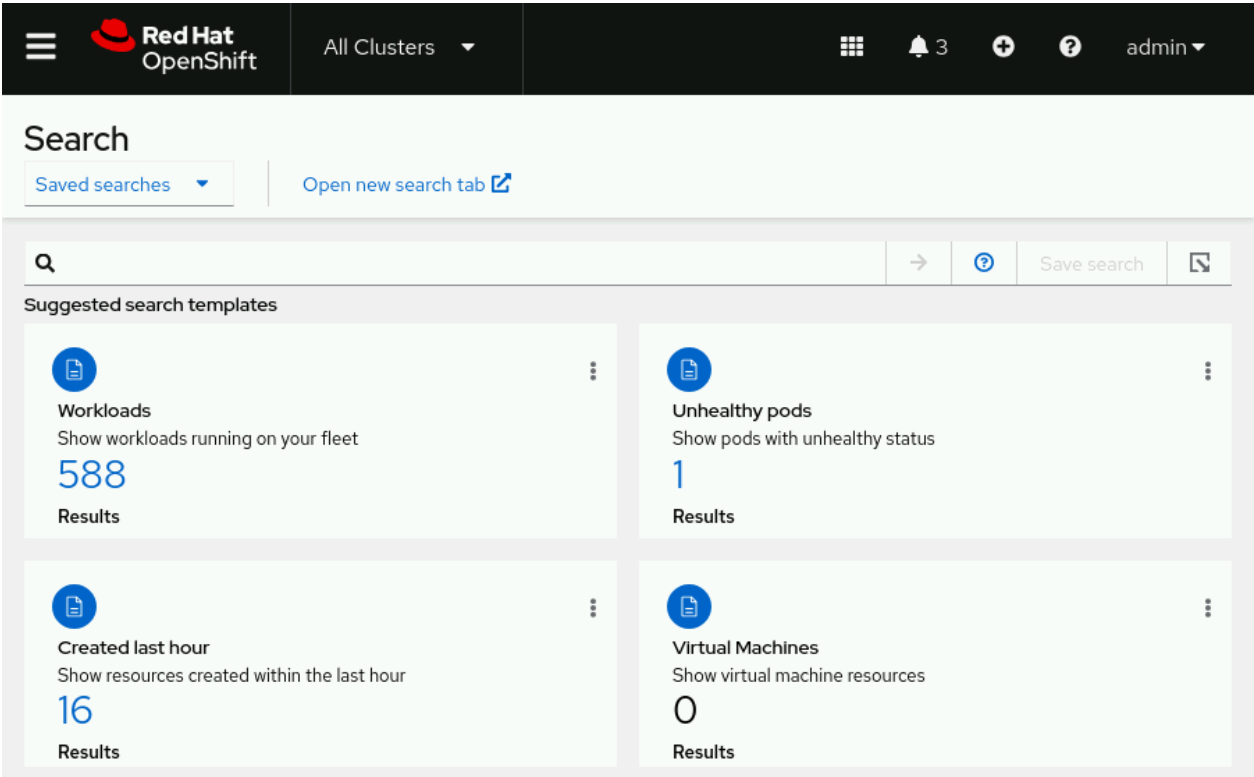


Figure 13. Search templates

You can type free-form text in the **Search** field. Search tries to locate the typed text in the PostgreSQL database of indexes, and shows results for any object that contains the typed string. The interface sorts the results in a table by type of Kubernetes or OpenShift object, which is represented by the `kind` filter.

You can refine each search by adding more filters to the query. When you add filters, the interface displays the stored values for that index.

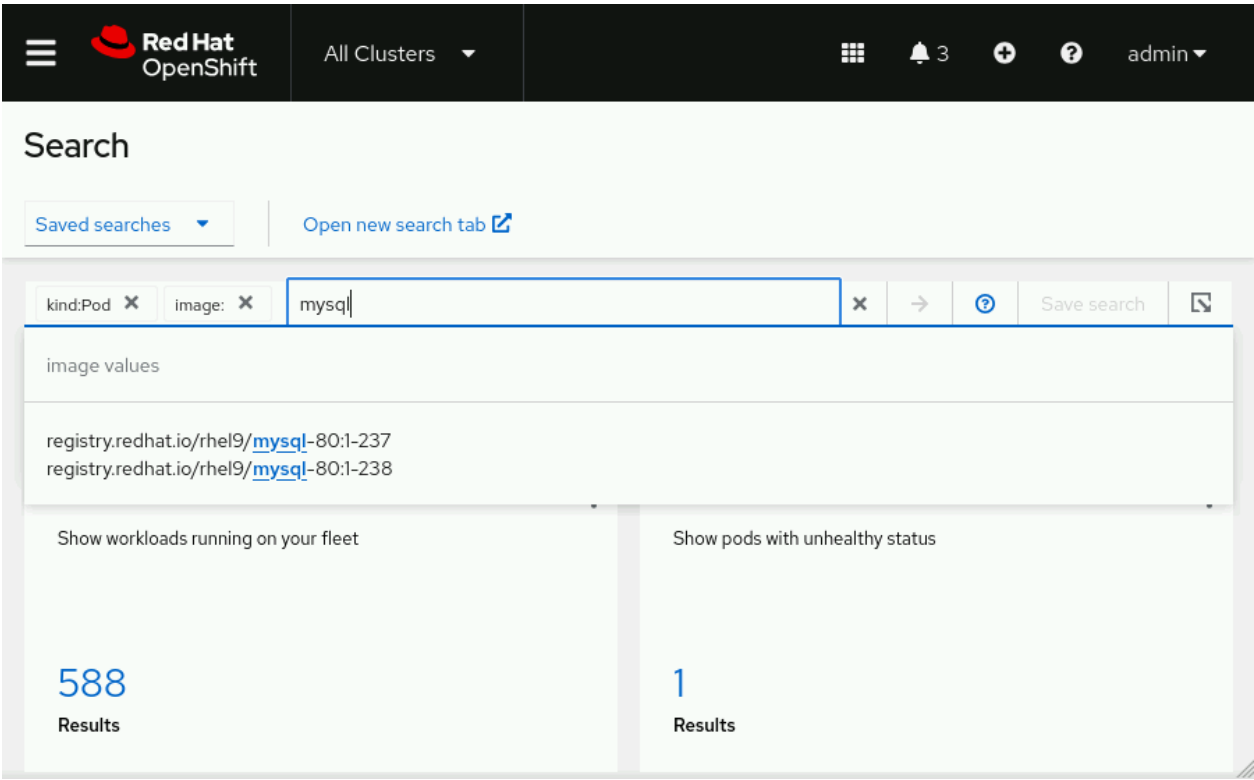


Figure 14. Search auto-completion

Some filters allow arithmetic comparators for numeric fields, such as the `cpu`, `replicas`, `capacity`, and `memory` filters.

You can also use the search interface to edit objects. If you click an object that is displayed in a result, then you can access the editable YAML definition of the object.

What Is Indexed

RHACM search uses different filters to classify the objects in the cluster fleet. Those filters are the keys of the indexing process. The **Search** interface gives access to all the available filters:

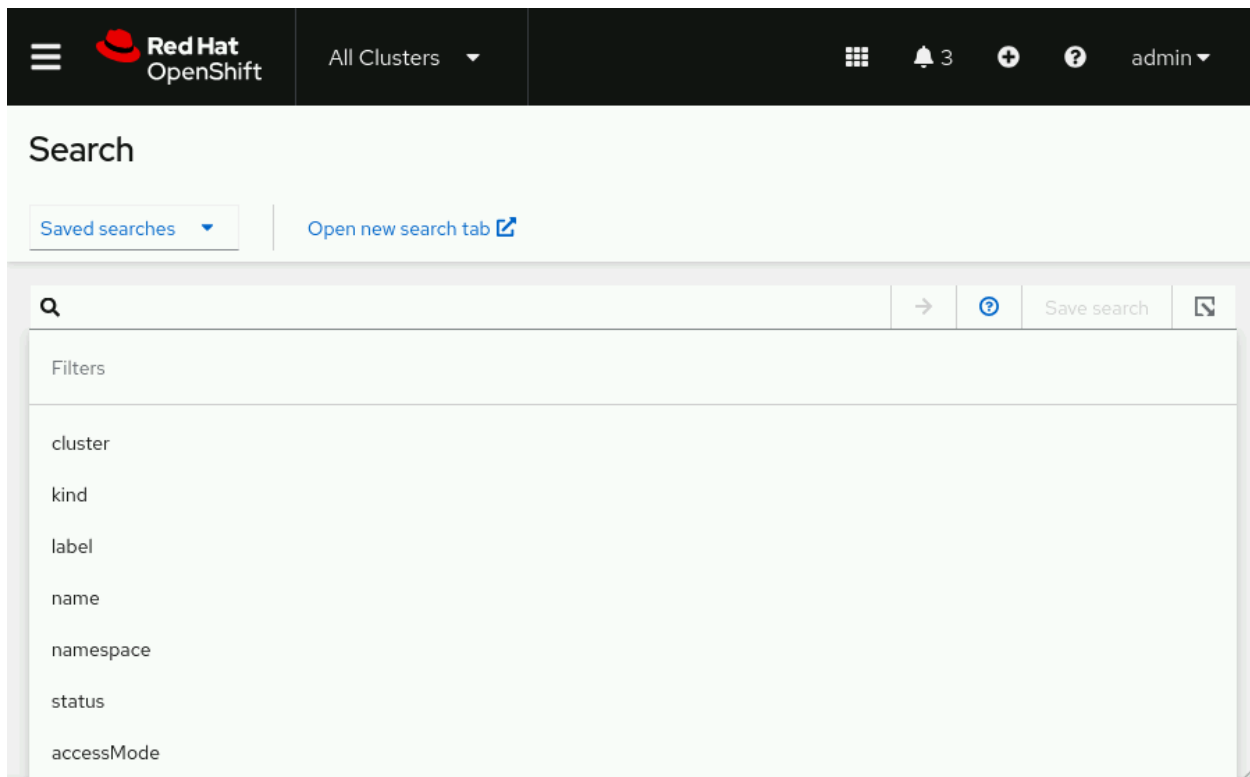


Figure 15. Filter auto-completion

You can use any of the filters to search, and you can also make free-form text searches. However, RHACM search does not index all the resource fields. For example, although the Deployment Kubernetes object has a `.spec.strategy` field, search does not index this field. Consequently, you cannot search Deployment resources by strategy.

Note
RHACM search indexes the `label` attribute of every Kubernetes object. It is a good practice to set labels on your objects for more accurate and faster searches.

Configuring RHACM Search

You can configure, tune, and customize several aspects of RHACM search.

Persistent Storage for the Database

By default, the PostgreSQL database that runs in the hub cluster stores the data in an empty directory volume. For large installations, and to improve performance, you can configure search to use a Persistent Volume Claim (PVC).

You configure this PVC by editing the Search resource named `search-v2-operator` in the `open-cluster-management` namespace:


```
apiVersion: search.open-cluster-management.io/v1alpha1
kind: Search
metadata:
  name: search-v2-operator
  namespace: open-cluster-management
spec:
  dbStorage:
    size: 10Gi
    storageClassName: fastdisk
...output omitted...
```

Resource Allocations

The search components run in pods. For large installations, you might have to increase the pods' allocated CPU and memory.

For the components that are running in the hub cluster, you set the limits in the Search resource named search-v2-operator in the open-cluster-management namespace. The following example declares resources for the collector and the indexer that are running in the hub cluster:

```
apiVersion: search.open-cluster-management.io/v1alpha1
kind: Search
metadata:
  name: search-v2-operator
  namespace: open-cluster-management
spec:
  dbStorage:
    size: 10Gi
    storageClassName: fastdisk
  deployments:
    collector:
      resources:
        limits:
          cpu: 500m
          memory: 128Mi
        requests:
          cpu: 250m
          memory: 64Mi
    database: {}
    indexer:
      resources:
        limits:
          memory: 5Gi
        requests:
          memory: 1Gi
    queryapi: {}
...output omitted...
```

For the collectors that are running on the managed clusters, you set the memory limits by annotating the search-collector resource of the ManagedClusterAddOn kind in the managed cluster namespace in the hub cluster. The following example configures the memory resources for the collector that is running in the mycluster managed cluster:

```
[user@host ~]$ oc annotate ManagedClusterAddOn search-collector -n mycluster \
addon.open-cluster-management.io/search_memory_request=512Mi \
addon.open-cluster-management.io/search_memory_limit=1Gi
```

Saved Search Queries and Search Templates

By default, each user can save up to 10 search queries in the web console. You can adjust this number by adding the `SAVED_SEARCH_LIMIT` key to the `console-mce-config` configuration map in the `multicluster-engine` namespace:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: console-mce-config
  namespace: multicluster-engine
  annotations:
    installer.multicluster.openshift.io/release-version: 2.8.0
  labels:
    backplaneconfig.name: multiclusterengine
data:
  SAVED_SEARCH_LIMIT: "20"
  LOG_LEVEL: info
  ansibleIntegration: disabled
  awsPrivateWizardStep: enabled
  singleNodeOpenshift: disabled
```

You can replace the default search templates by creating the `console-search-config` configuration map in the `open-cluster-management` namespace. The `id` attribute in each entry of the `suggestedSearches` list must be unique. The following example replaces the default templates with two custom templates:

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: console-search-config
  namespace: open-cluster-management
data:
  suggestedSearches: |-
    [
      {
        "id": "search.suggested.podmycluster.name",
        "name": "Pods in mycluster",
        "description": "Show the pods in the mycluster OCP cluster",
        "searchText": "kind:Pod cluster:mycluster"
      },
      {
        "id": "search.suggested.frontend.name",
        "name": "Frontend deployments",
        "description": "Show deployments with the tier=frontend label",
        "searchText": "kind:Deployment label:tier=frontend"
      }
    ]
```

References

Knowledgebase: "Red Hat Advanced Cluster Management for Kubernetes 2.13 Support Matrix"

For more information about cluster lifecycle, host inventories, and credentials, refer to the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Clusters* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/clusters/index

For more information about managing applications in RHACM, refer to the *Red Hat Advanced Cluster Management for Kubernetes 2.13 GitOps* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/gitops/index

For more information about governance, refer to the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Governance* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/governance/index

For more information about customizing the collected resources, refer to the *Managing Search* section in the *Search in the Console* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Web Console* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/web_console/index#creating-search-configurable-collection

For more information about the RHACM web console and tuning options, refer to the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Web Console* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/web_console/index

Search Operator

2.4. Guided Exercise

Navigate the RHACM Web Console

List failed deployments, diagnose the cause of a failed MYSQL deployment, and fix it by using the RHACM web console.

Outcomes

- Access the various RHACM components by using the web console.
- Use the predefined search templates to quickly identify failing deployments across a cluster fleet.
- Locate Kubernetes and OpenShift objects across a cluster fleet.
- Use the RHACM web console to edit Kubernetes objects.

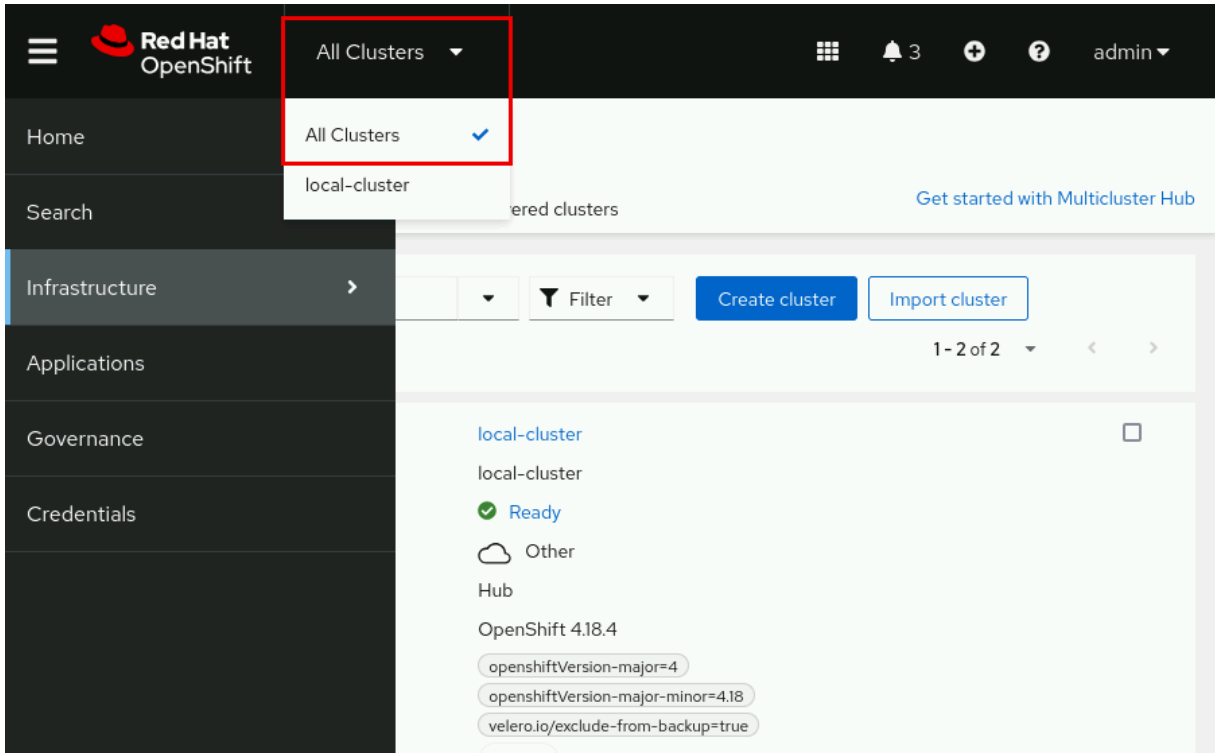
Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab install do0012l ; lab start manage-navigate
```

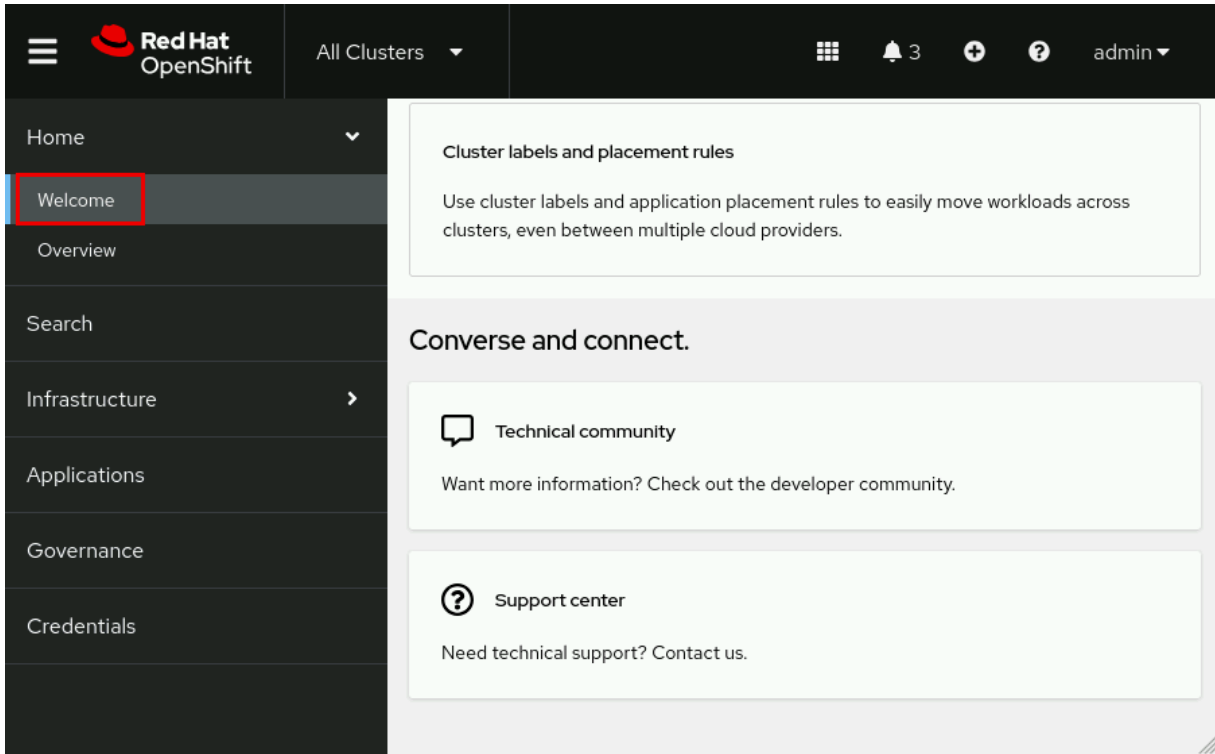
Instructions

1. Log in to the hub RHOCF cluster, and switch to the RHACM web console.
 - 1.1. Open a web browser and go to `https://console-openshift-console.apps.ocp4.example.com`
 - 1.2. Select the **Red Hat Identity Management** identity provider and log in as the admin user with `redhatocp` as the password.
 - 1.3. Ensure that the cluster switcher is set to **All Clusters**, which corresponds to the RHACM web console. If it is not set in this way, then use the cluster switcher to select **All Clusters**.

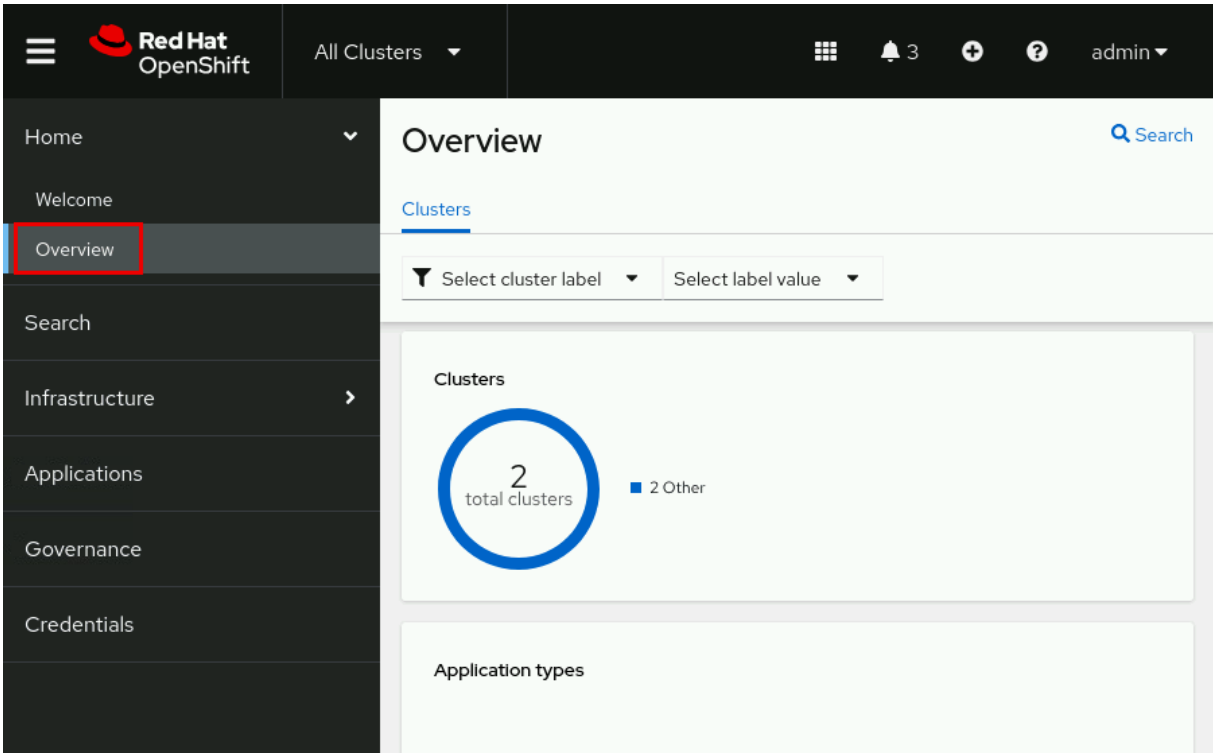


2. Review some components of RHACM. Other lessons in the series cover these components in more detail.

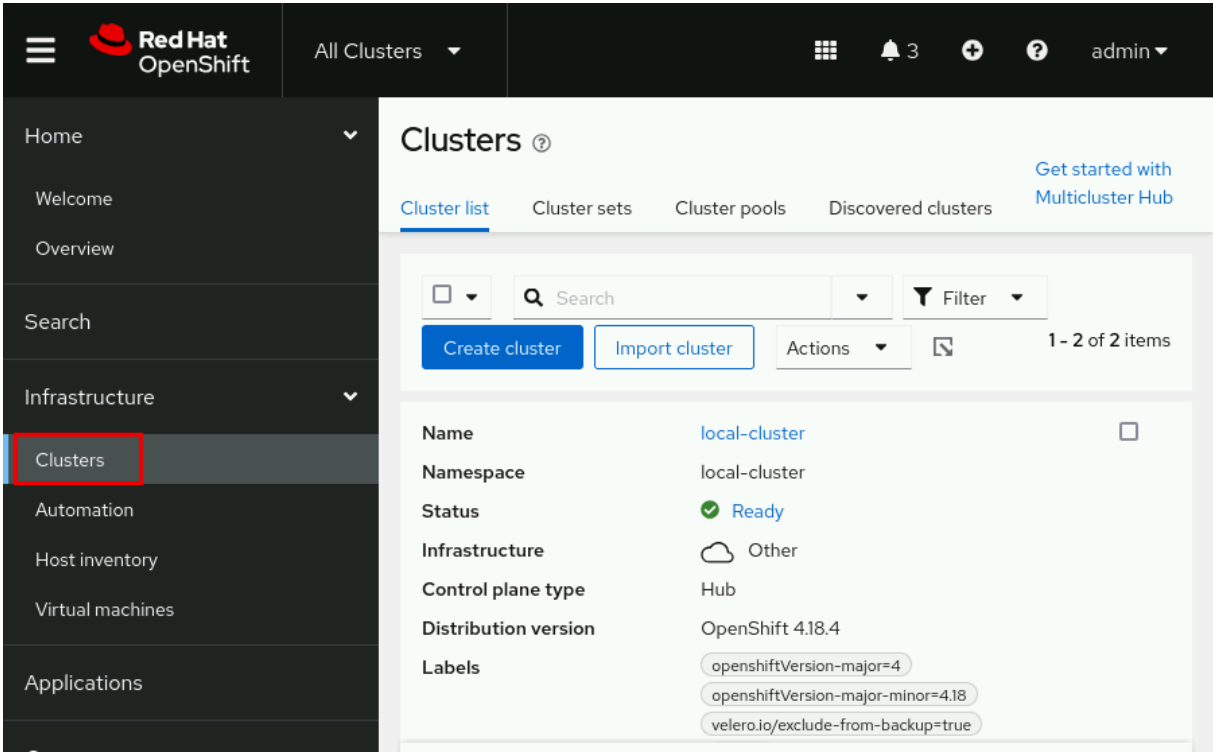
2.1. Go to **Home > Welcome**. This page gives quick access to the RHACM components and to some documentation. The **Support center** section at the bottom of the page provides a link to Red Hat support.



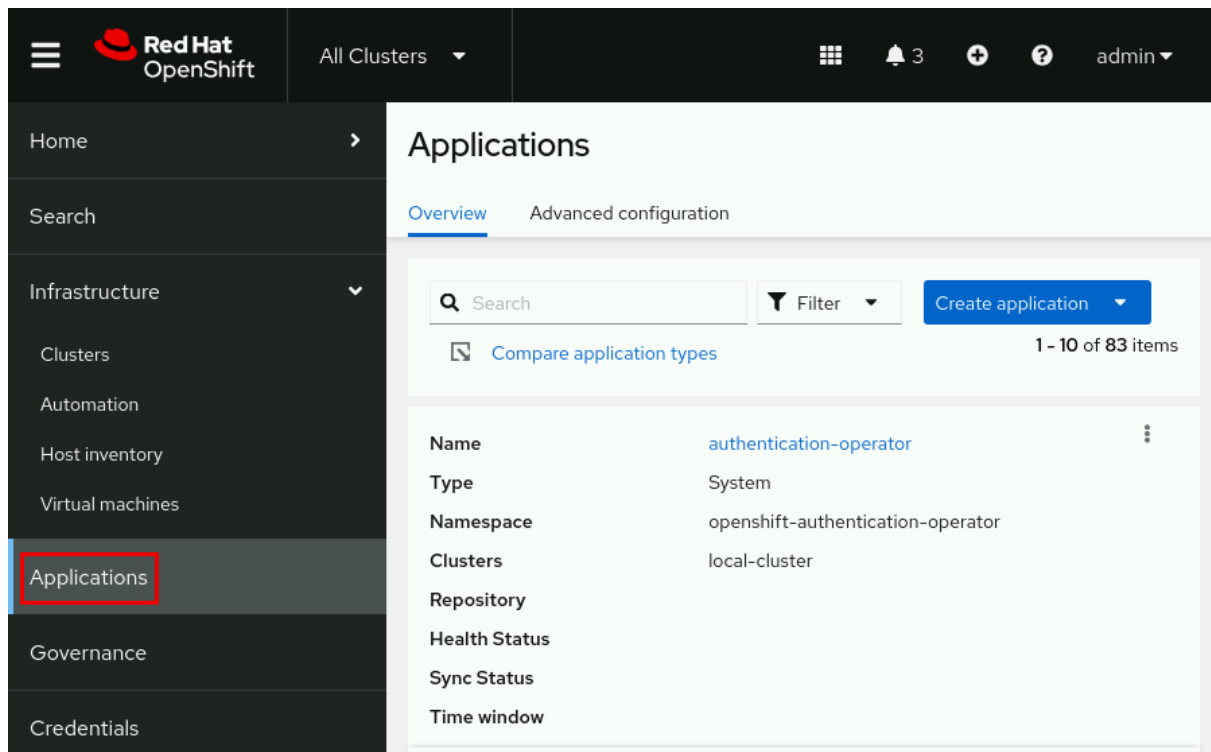
2.2. Go to **Home > Overview**. This page provides a global view of your cluster fleet. The page shows that your RHACM installation manages two clusters, and that both clusters are healthy.



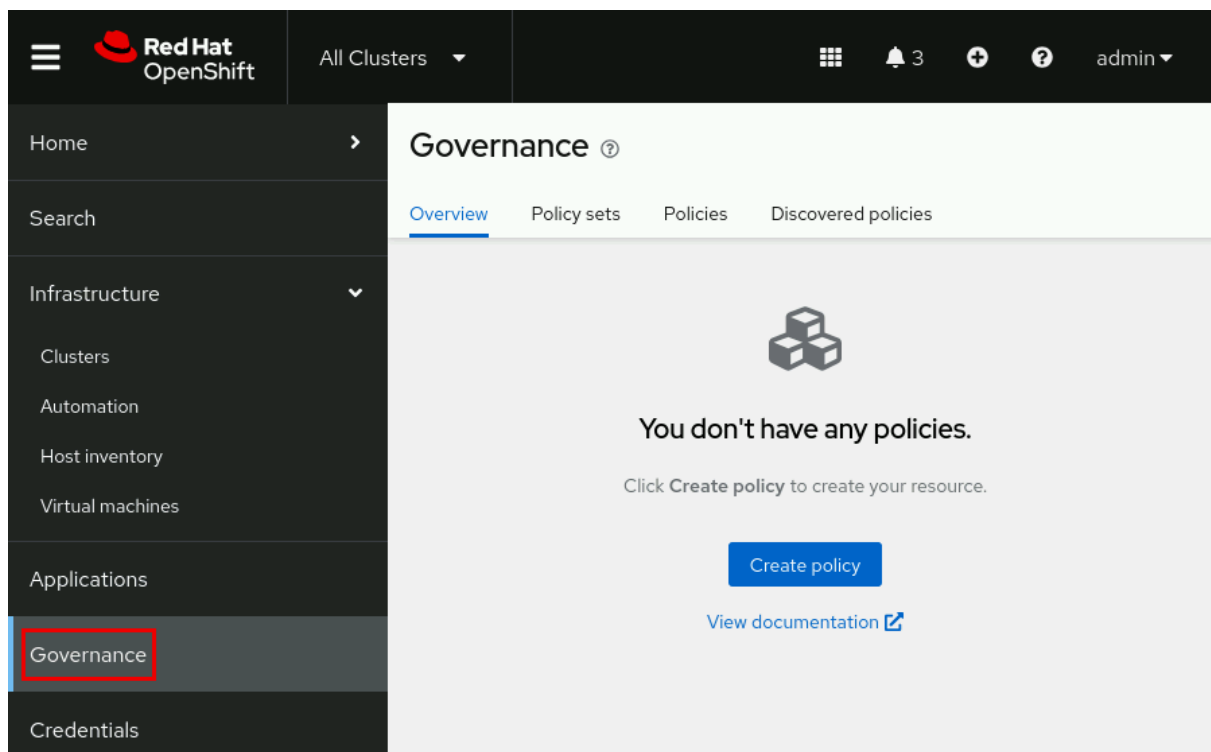
2.3. Go to **Infrastructure > Clusters**. This page lists the managed clusters, and enables you to create clusters or to import your existing clusters. Notice that the RHACM installation process automatically imported the hub cluster as a managed cluster, with `local-cluster` as the name.



2.4. Go to **Applications**. This component enables you to deploy applications on your clusters. This page lists the deployed applications, and gives you access to the application details. For this component, RHACM integrates with Red Hat OpenShift GitOps, which is based on Argo CD.

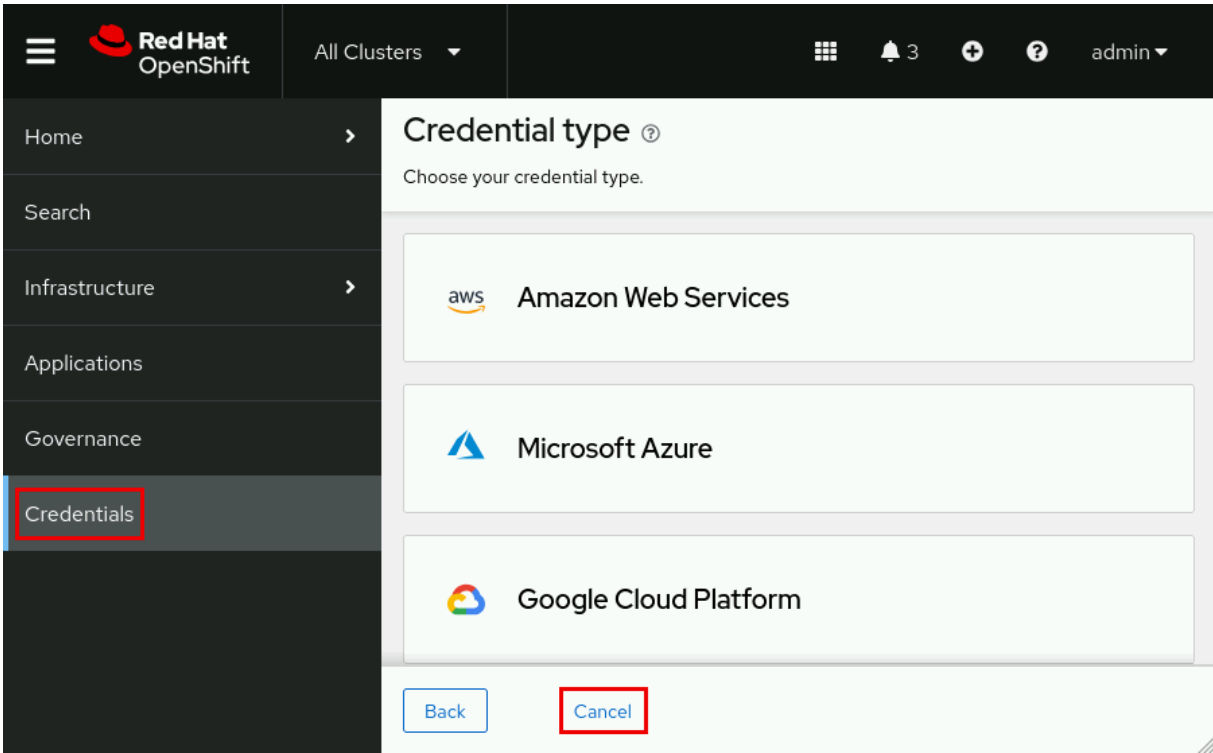


- 2.5. Go to **Governance**. This component enables you to create policies to control and enforce compliance standards.



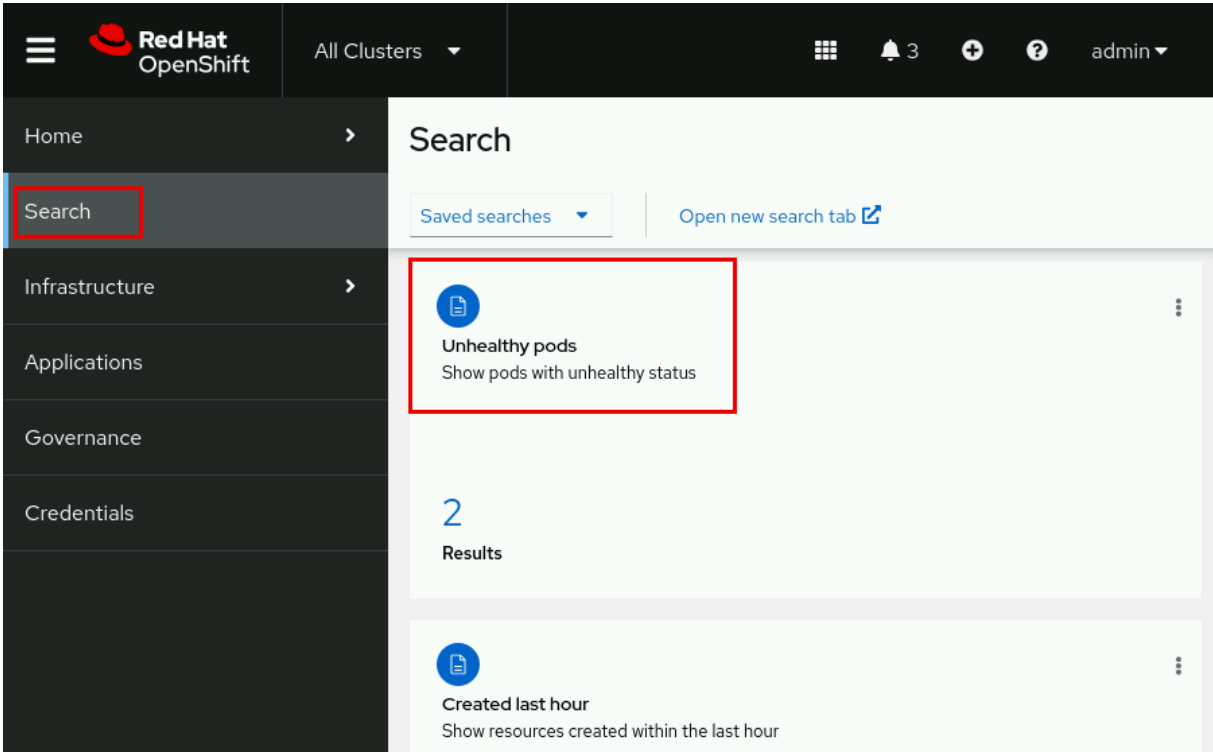
- 2.6. Go to **Credentials** and click **Add credential**. From this page, you can declare credentials to access your cloud providers, such as Amazon AWS or Microsoft Azure. RHACM uses these credentials to create and manage clusters that you host with these providers.

Click **Cancel**, because you do not need credentials for this course.



3. Search for unhealthy pods.

3.1. Go to **Search**. This page provides a list of predefined search templates. Click the **Unhealthy pods** template.



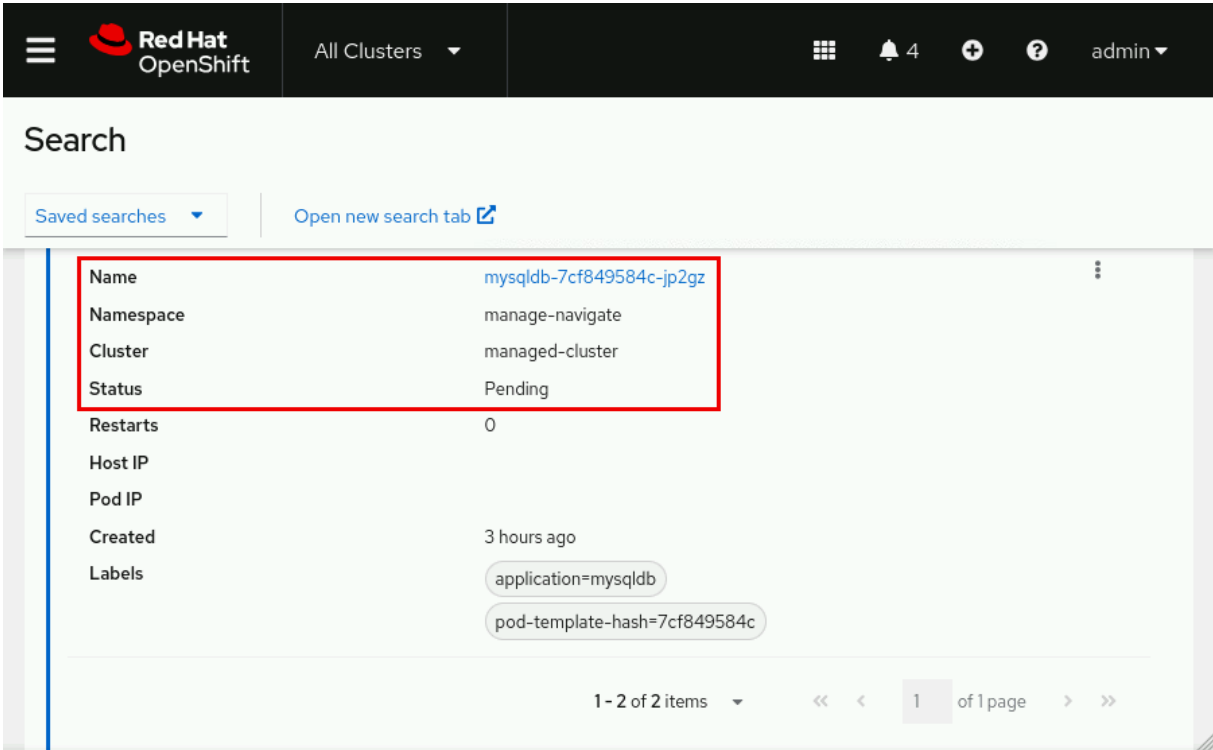
3.2. Review the syntax of the search query, which is targeting pods (kind:pod) with one of the given statuses.



The `lab` command created a misconfigured MySQL pod. Locate this failing MySQL pod in the search results. The pod is in the `manage-navigate` namespace in the `managed-cluster` RHOC cluster.

 **Note**

Ignore any search results that show pods that are unrelated to the MySQL deployment.



The pod name in the preceding screenshot is different on your system.

- 3.3. Click the pod name to investigate the issue. The **Conditions** section indicates that the pod refers to the nonexistent `my-volume` persistent volume claim (PVC).

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

mysqlpdb-7cf849584c-jp2gz

Details

YAML

Related resources

Logs

Actions

Pod details

Name

mysqlpdb-7cf849584c-jp2gz

Namespace

manage-navigate

Cluster

managed-cluster

Tolerations

2 tolerations

Created at

Apr 16, 2025, 2:54 AM

Labels

application=mysqlpdbpod-template-hash=7cf849584c

Annotations

2 annotations

Owner

mysqlpdb-7cf849584c

Conditions

Type	Status	Updated	Reason	Message
PodScheduled	False	Apr 16, 2025, 2:54 AM	Unschedulable	0/1 nodes are available: persistentvolumeclaim "my-volume" not found. preemption: 0/1 nodes are available: 1 Preemption is not helpful for scheduling.

3.4. To fix the issue, you must edit the deployment resource that created the pod. Identify this deployment by going to the **Related resources** tab and by expanding the **Deployment** section. The deployment name is mysqlpdb in the manage-navigate namespace.

mysqlpdb-7cf849584c-jp2gz

Details

YAML

Related resources

Logs

Actions

Cluster (1)

Deployment apps/v1 (1)

1 - 1 of 1

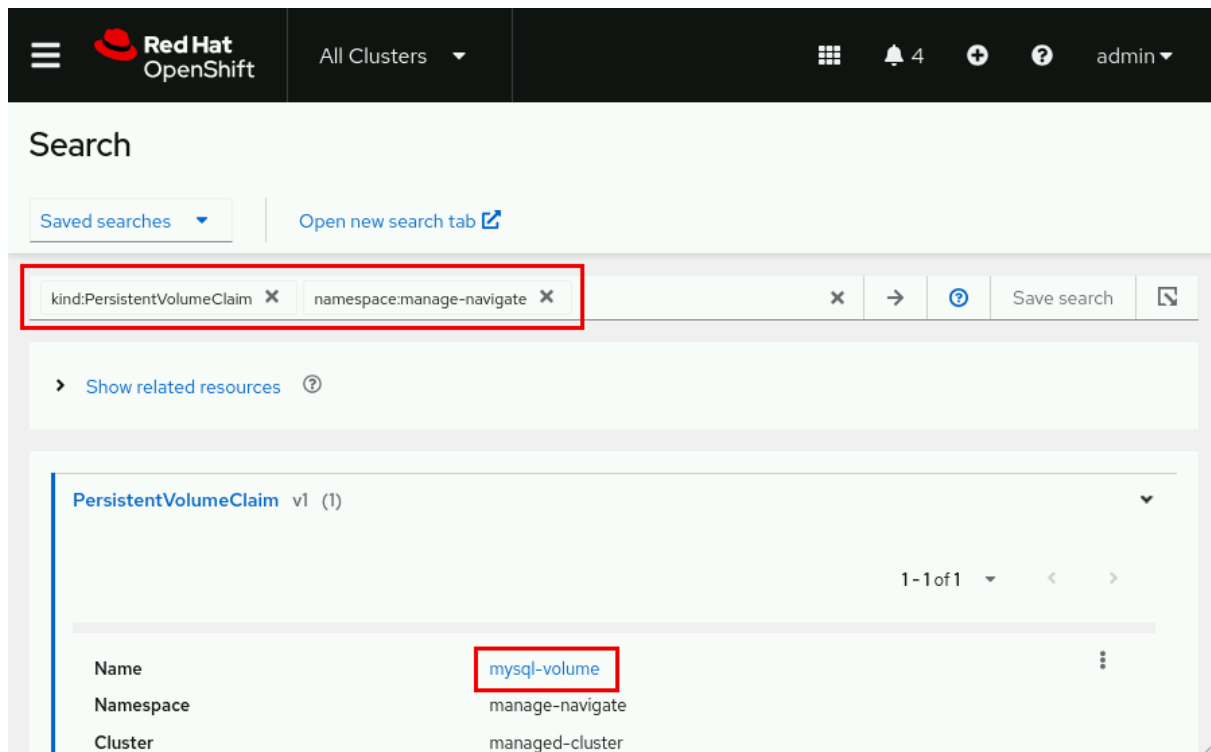
Name	Namespace	Cluster	Created	Labels
mysqlpdb	manage-navigate	managed-cluster	3 hours ago	application=mysqlpdb

1 - 1 of 1 items1 of 1 page

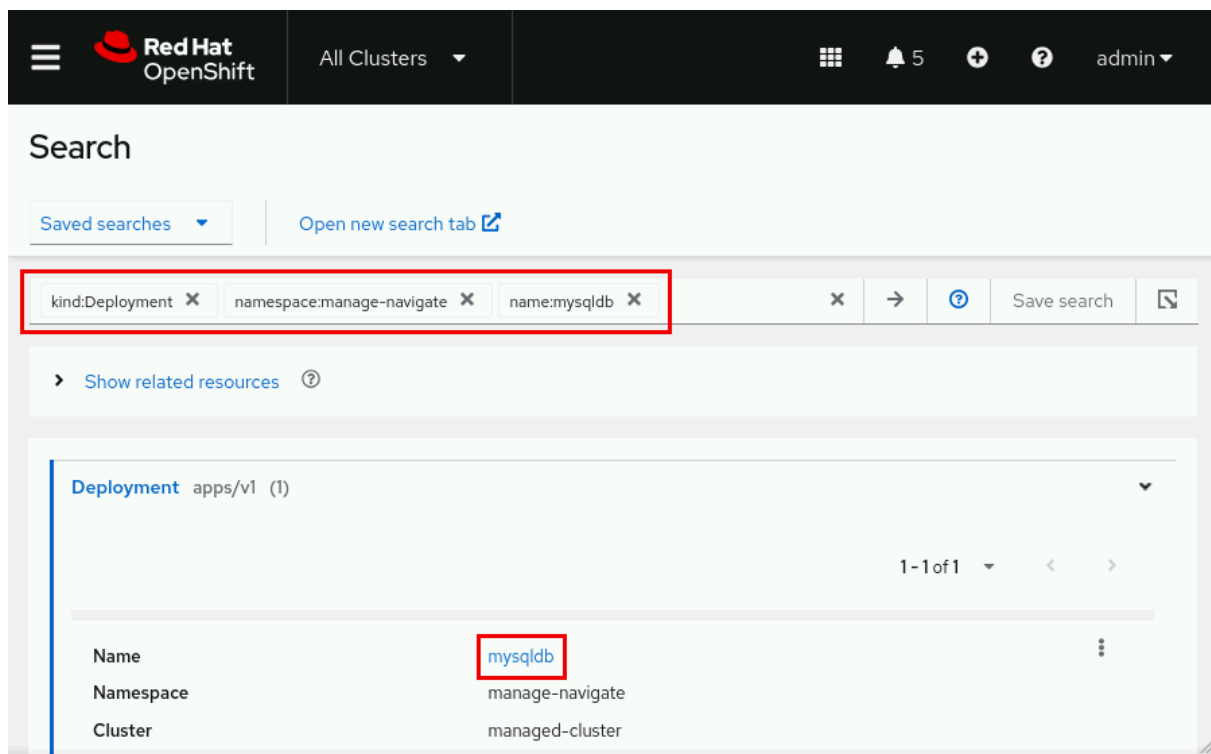
ReplicaSet apps/v1 (1)

4. Before you can edit the deployment to fix the issue, you must find the name of the available PVC in the manage-navigate namespace.

4.1. Go back to **Search** and search for PVCs in the manage-navigate namespace. Type `kind:PersistentVolumeClaim namespace:manage-navigate` in the search field. The name of the PVC is mysql-volume.

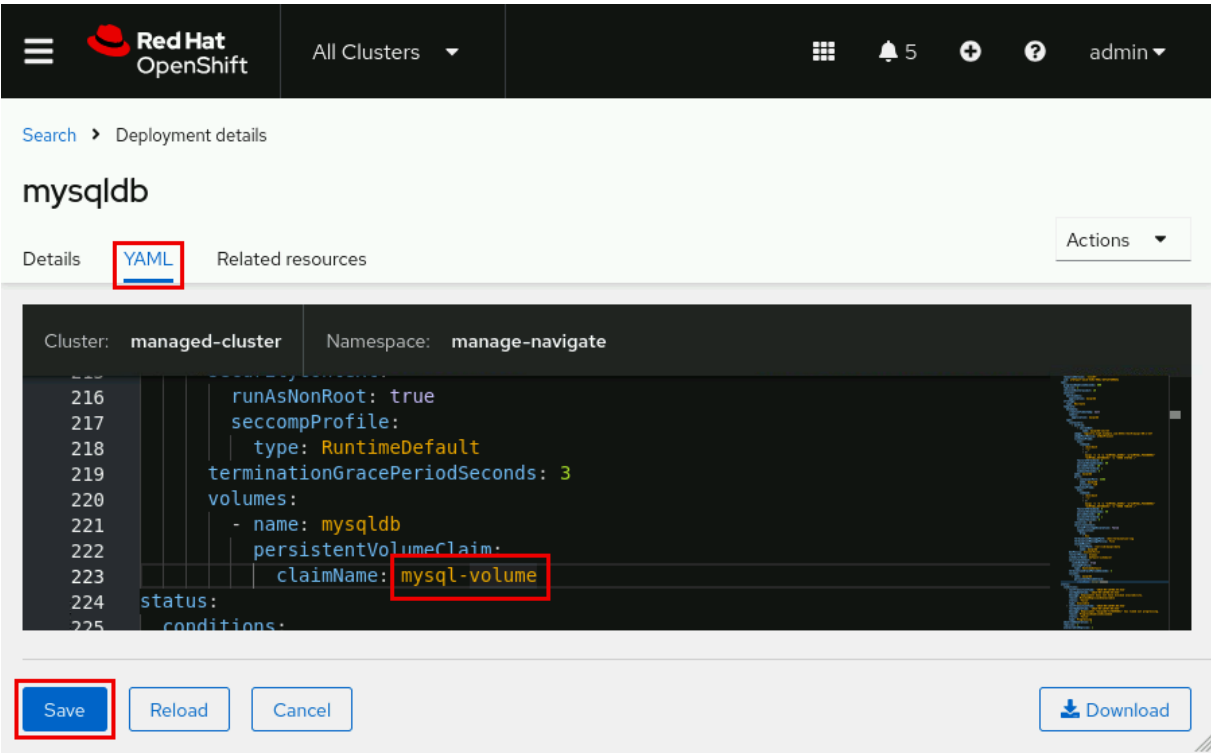


- 4.2. Locate the deployment to edit. Replace the content of the search field with `kind:Deployment` `namespace:manage-navigate` `name:mysqldb`. Click the `mysqldb` deployment.



- 4.3. Fix the PVC name in the deployment by going to the **YAML** tab and by replacing the PVC name in the `claimname` variable. Click **Save**.

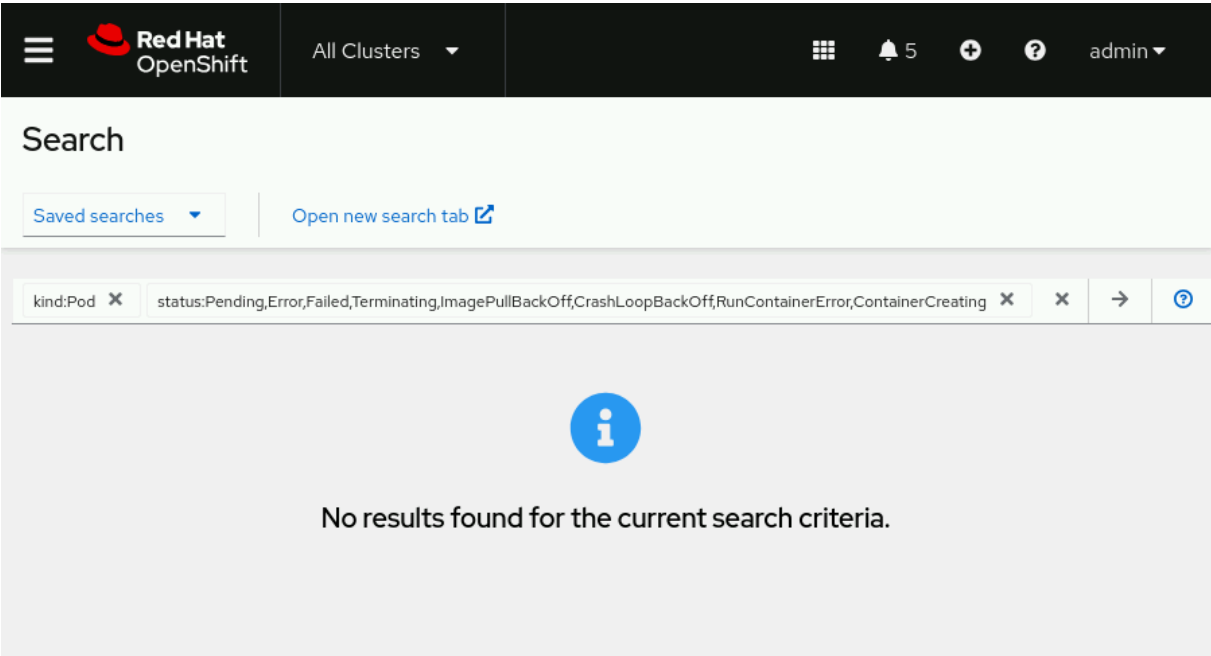
For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright



- 4.4. Go back to **Search**. Click the **Unhealthy pods** template again to verify that RHACM does not report the MySQL pod, which is no longer unhealthy. You might have to rerun the search because it takes up to two minutes for the pod to be ready.

 **Note**

Ignore any search results that show pods that are unrelated to the MySQL deployment.

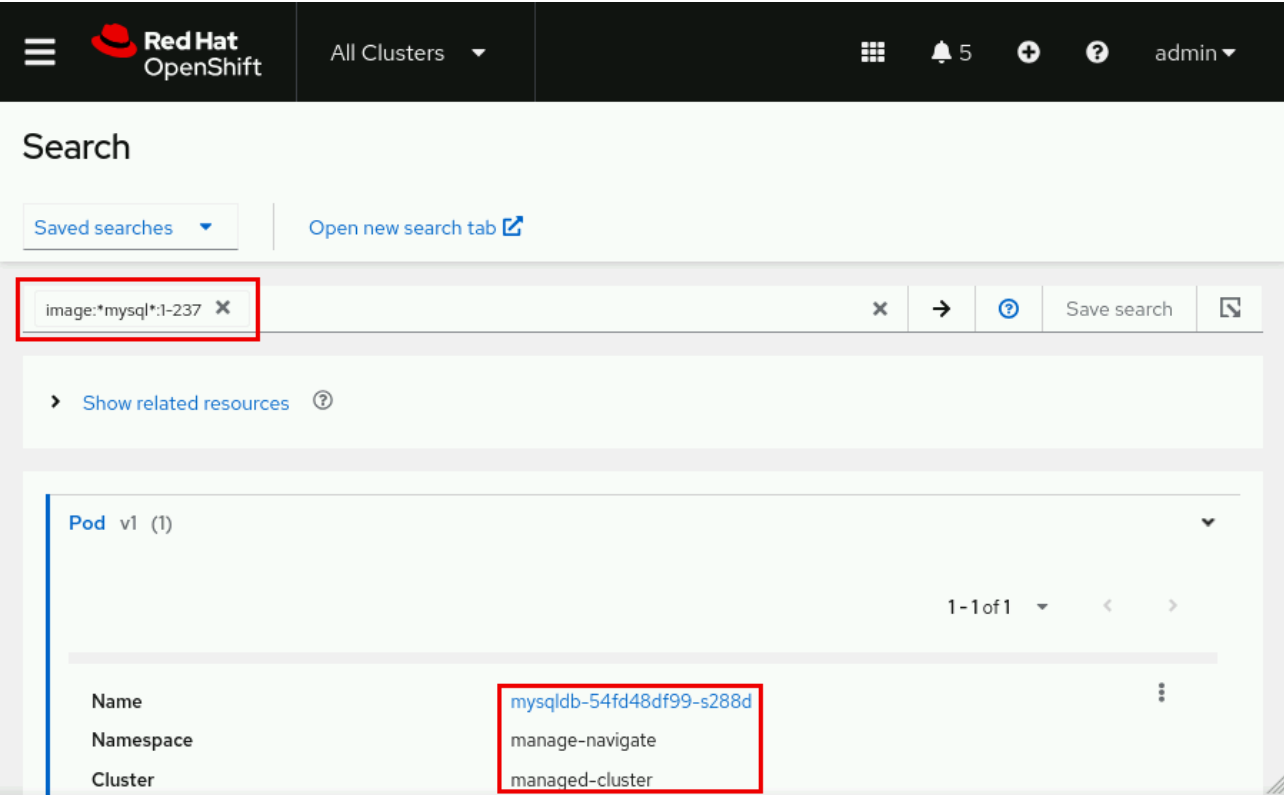


5. Your security team informs you that the MySQL container image with the 1-237 tag has a vulnerability. Verify whether any of your clusters use this image.

Replace the content of the search field with `image: *mysql*:1-237`. A MySQL pod in the `manage-navigate` namespace in the `managed-cluster` RHOC cluster is using this image. The next step would be to inform your

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

developers of the issue, or to fix the related deployment resource yourself.



The pod name in the preceding screenshot is different on your system.

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish manage-navigate
```

2.5. Configuring Access Control for Multicluster Management

Objectives

- Define an authentication model and create user roles to manage access for multicluster management in RHACM.

Role-based Access Control in RHACM

OpenShift uses the authentication and authorization security layers to enable user interaction with the cluster. When a user makes a request to the API, the API associates the user with the request. The authentication layer authenticates the user. On successful authentication, the authorization layer uses role-based access control (RBAC) to accept or reject the API request.

RHACM supports role-based access control (RBAC). Your role determines the actions that you can perform. A role binding associates users and groups with a role. Administrators can design a multicluster authentication model to define access for different tenants to different groups of clusters, with fine-grained control.

RHACM Managed Cluster Sets

To create groups of managed clusters, RHACM defines the concept of a *managed cluster set*, which is represented by the `ManagedClusterSet` resource. Managed cluster sets help you manage access to all your managed clusters, as well as perform actions on all the member clusters at once.

RHACM uses the `ManagedClusterSetBinding` resource to associate a managed cluster set with a namespace. By granting hub users RBAC permissions to the namespace, you effectively grant these users permissions to the managed cluster set.

When you install the hub cluster, RHACM creates a `ManagedClusterSet` resource named `default`. Each cluster must be a member of a managed cluster set. RHACM automatically assigns to the `default` managed cluster set the clusters that do not belong to any managed cluster set. You cannot delete or update the default managed cluster set.

RHACM automatically creates a managed cluster set named `global`, and the namespace `open-cluster-management-global-set`. RHACM also creates a `ManagedClusterSetBinding` resource named `global` that binds the `global` managed cluster set to the `open-cluster-management-global-set` namespace.

By default, the managed clusters that you create belong to the `global` managed cluster set. The `global` managed cluster set includes all managed clusters. You cannot delete or update the `global` managed cluster set. In addition to being in the `global` managed cluster set, a cluster is assigned to either the `default` managed cluster set or to a single, user-defined managed cluster set, but not to both.

A managed cluster set labels all the clusters that belong to it with the `cluster.open-cluster-management.io/clusterSet=managed_clusterSet_name` label. RHACM uses this label to identify the clusters that belong to the managed cluster set.

Role Definitions

The following tables describe the role definitions that RHACM supports. Some product resources are cluster-wide and some are namespace-scoped. You must apply cluster role bindings and namespace role bindings to your users for consistent access controls.

OpenShift Default Roles

cluster-admin

A user with a cluster role binding to this role is an OpenShift superuser, who has full access.

admin, edit, view

A user with a namespace-scoped binding to these roles has access to open-cluster-management resources in a specific namespace. A cluster-wide binding to the same roles gives access to all the open-cluster-management resources cluster-wide.

RHACM Roles

open-cluster-management:cluster-manager-admin

RHACM superuser, with full access. This role allows the user to create a ManagedCluster resource.

open-cluster-management:admin:managed_cluster_name

RHACM administrator access to the ManagedCluster resource named *managed_cluster_name*. This role is automatically created when you import a cluster into RHACM.

open-cluster-management:view:managed_cluster_name

RHACM view access to the ManagedCluster resource named *managed_cluster_name*.

open-cluster-management:managedclusterset:admin:managed_clusterset_name

RHACM administrator access to the ManagedClusterSet resource named *managed_clusterset_name*. This user has administrator access to the managedcluster, clusterclaim, clusterdeployment, and clusterpool resources, which have the `cluster.open-cluster-management.io/clusterset=managed_clusterset_name` label. A role binding is automatically generated when you are using a cluster set.

open-cluster-management:managedclusterset:view:managed_clusterset_name

RHACM view access to the ManagedClusterSet resource named *managed_clusterset_name*. This user has view access to the managedcluster, clusterclaim, clusterdeployment, and clusterpool resources, which have the `cluster.open-cluster-management.io/clusterset=managed_clusterset_name` label.

open-cluster-management:managedclusterset:bind:managed_clusterset_name

RHACM view access to the ManagedClusterSet resource named *managed_clusterset_name*. The user can bind the ManagedClusterSet resource named *managed_clusterset_name* to a namespace. This user has view access to the managedcluster, clusterclaim, clusterdeployment, and clusterpool resources, which have the `cluster.open-cluster-management.io/clusterset=managed_clusterset_name` label.

open-cluster-management:subscription-admin

A user with this role can create Git subscriptions that deploy Kubernetes resources that are specified in YAML files to multiple namespaces.

The use of the RHACM predefined roles helps organizations with multitenant environments to enforce operational boundaries and security.

For example, in a large enterprise with multiple business units, each unit manages its own applications on separate Kubernetes clusters. By using RHACM, each unit has the `open-cluster-management:managedclusterset:admin:managed_clusterset_name` role, which gives the units full control over their own clusters, such as deploying apps and managing configurations, without access to clusters that other units manage.

For example, during a security audit, an organization needs to give external auditors or internal compliance teams access to inspect configurations, policies, and cluster resources without risking any changes. By assigning the `open-cluster-management:managedclusterset:view:global` role in RHACM, these users get read-only access to the managed clusters. This role ensures transparency and accountability, and maintains the integrity and security of the environment.

Placements

A *placement* is a namespace-scoped resource that defines a rule to dynamically select a set of managed clusters in one or multiple managed cluster sets, which are bound to the placement namespace.

Placements enable you to determine a subset of managed clusters to place your application workloads on. To create the Placement resource, you need the cluster administrator and the cluster set administrator roles.

Predicates

The predicates section in the Placement resource enables you to filter managed clusters from managed cluster sets by using the `labelSelector` and the `claimSelector` parameters. The `numberOfClusters` parameter defines the number of clusters to select.

The following YAML example is of a Placement resource:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: placement1
  namespace: default
spec:
  numberOfClusters: 3 ①
  clusterSets: ②
  - dev
  - stage
  predicates: ③
  - requiredClusterSelector:
      labelSelector: ④
        matchLabels:
          purpose: test
      claimSelector: ⑤
        matchExpressions:
          - key: region.open-cluster-management.io
            operator: In
            values:
              - us-west-1
```

- ① The number of managed clusters to select.
- ② Filter by particular cluster sets.
- ③ The predicates section.
- ④ Match only clusters with the specified label.
- ⑤ Match only clusters with the specified key and value.

Taints and Tolerations

You can define tolerations in the `Placement` resource to filter clusters with matching taints. You can add a customized taint to managed clusters for selecting or deselecting these clusters from the placement.

The taints are properties of the `ManagedClusters` resource. The tolerations apply to the `Placement` resource, and allow placements to select managed clusters with matching taints.

The following YAML example is of a `ManagedCluster` resource with a taint:

```
apiVersion: cluster.open-cluster-management.io/v1
kind: ManagedCluster
metadata:
  name: cluster1
spec:
  hubAcceptsClient: true
  taints:
    - effect: NoSelect
      key: gpu
      value: "true"
      timeAdded: '2025-04-23T05:12:23Z'
```

The placement does not select the `cluster1` cluster unless you define the tolerations. The following YAML example is of a `Placement` resource, which defines the tolerations to select the `cluster1` cluster:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: placement1
  namespace: ns1
spec:
  tolerations:
    - key: gpu
      value: "true"
      operator: Equal
      tolerationSeconds: 300 ①
```

- ① Optionally defines how long a toleration stays bound to a taint. The `tolerationSeconds` parameter can automatically transfer applications that are deployed on a cluster that goes offline to another managed cluster after a specified length of time.

Prioritizers

With placements, you can sort the clusters by prioritizer scores and select the top `n` clusters from that group. You prioritize the managed clusters by using the `prioritizerPolicy` section in the `Placement` resource.

By default, the mode for the prioritizer policy is set to `Additive`, and it has two prioritizers: `Steady` and `Balance`, whose default weight is set to 1.

The following YAML example is of a `Placement` resource, which uses the prioritizers to select a cluster with the largest allocatable CPU and memory.

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: placement1
  namespace: ns1
spec:
  numberOfClusters: 1
  prioritizerPolicy:
    mode: Additive
    configurations:
      - scoreCoordinate:
          builtIn: ResourceAllocatableCPU
          weight: 2
      - scoreCoordinate:
          builtIn: ResourceAllocatableMemory
          weight: 2
```

- ① The number of managed clusters to select.
- ② The prioritizer section.
- ③ The mode can be set to Additive or Exact. If the mode is empty, then the mode is set to Additive. In Additive mode, any prioritizer that is not explicitly enumerated is enabled in its default configurations. In Exact mode, any prioritizer that is not explicitly enumerated is weighted as zero.
- ④ The configuration of prioritizers.
- ⑤ The configuration of the prioritizer and the score source.
- ⑥ The name of the prioritizer.
- ⑦ The weight of the prioritizer. The value must be in the range from -10 to 10. A higher weight indicates that the prioritizer is weighted more in the cluster selection. A weight value of 0 indicates a disabled prioritizer.

Placement Decisions

The Placement resource creates one or more PlacementDecision resources in the same namespace with the `cluster.open-cluster-management.io/placement=placement_name` label. The placement decision contains the managed clusters that the placement selected. The `status.decisions` section lists the top *n* managed clusters with the highest score and ordered by name.

If you select a managed cluster and add it to a placement decision, then the components that consume this placement might apply the workload on this managed cluster. When you remove a managed cluster from the placement decision, the workload that is applied on this managed cluster is removed.

The following YAML example is of a PlacementDecision resource:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: PlacementDecision
metadata:
  labels:
    cluster.open-cluster-management.io/placement: placement1
  name: placement1-decision-1
  namespace: default
status:
  decisions:
    - clusterName: cluster1
    - clusterName: cluster2
    - clusterName: cluster3
```

- ① The placement label
- ② The list of managed clusters with the highest score and ordered by name

Centralized Identity Management Service

One significant challenge in multicluster environments is managing the authentication and authorization mechanisms in a fully controlled manner across different Kubernetes and OpenShift clusters.

RHACM uses the OpenShift authentication and authorization layers. The OpenShift authentication layer uses the OAuth open standard as an authorization framework. By default, OpenShift uses an internal OAuth server.

Neither Kubernetes nor OpenShift currently provides a way to federate all the internal OAuth servers of each cluster. To solve this problem, you can configure OAuth to specify an identity provider such as LDAP for centrally managing the users who access the managed clusters.

References

For more information about RBAC in RHACM, refer to the *Securing Clusters* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Secure Clusters* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/secure_clusters/index#securing-cluster-intro

For more information about managed cluster sets, refer to the *ManagedClusterSets* section in the *Cluster Lifecycle with Multicluster Engine Operator Overview* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Clusters* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/clusters/index#managedclustersets-intro

For more information about placements and placement decisions, refer to the *Placement* section in the *Cluster Lifecycle with Multicluster Engine Operator Overview* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Clusters* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/clusters/index#placement-intro

For more information about authentication and authorization, refer to the *Overview of Authentication and Authorization* chapter in the *Red Hat OpenShift Container Platform 4.18 Authentication and Authorization* documentation at https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/authentication_and_authorization/index#overview-of-authentication-authorization

For more information about the supported identity providers, refer to the *Supported Identity Providers* section in the *Overview of Authentication and Authorization* chapter in the *Red Hat OpenShift Container Platform 4.18 Authentication and Authorization* documentation at

https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/authentication_and_authorization/index#supported-identity-providers

2.6. Guided Exercise

Configure Access Control for Multicluster Management

Create cluster sets by using existing Red Hat OpenShift clusters, and configure existing users in RHOCF to manage those cluster sets.

Outcomes

- Create a managed cluster set.
- Assign a managed cluster to a managed cluster set.
- Assign predefined RHACM roles to different users.
- Remove a managed cluster set.

Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab install do0012l ; lab start manage-configure
```

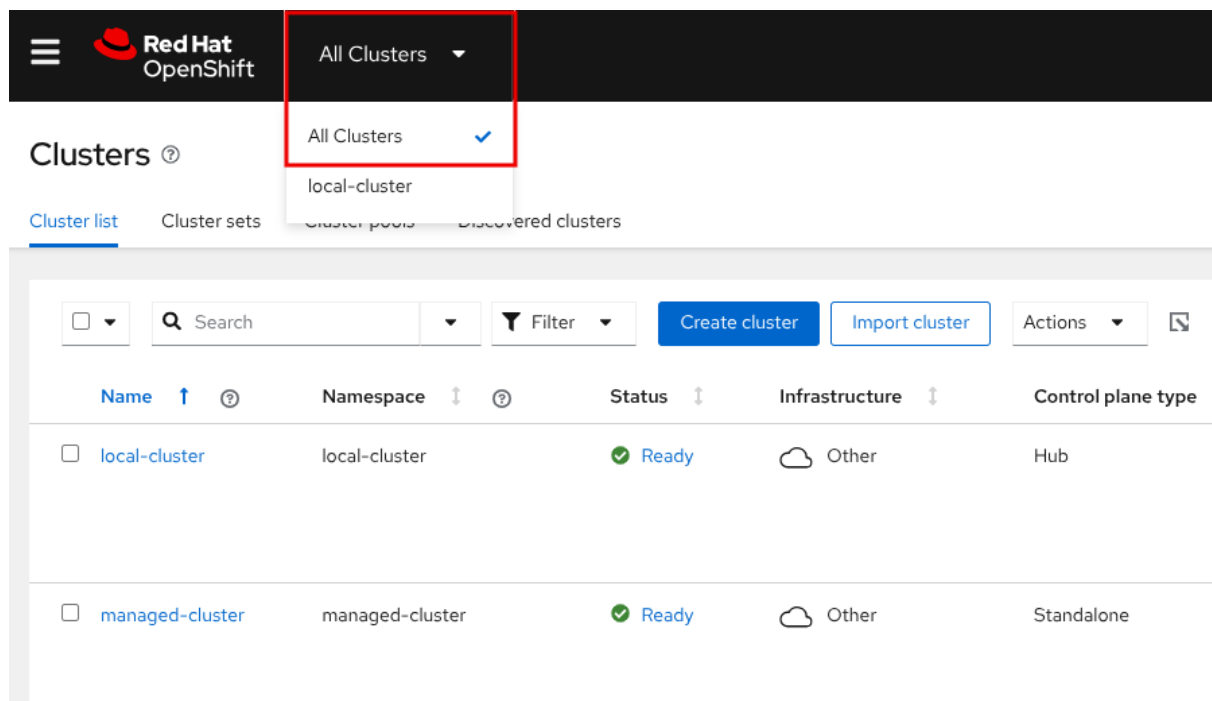
Instructions

The lab environment includes one server for Identity Management in Red Hat Enterprise Linux (IdM). The `lab start` command ensures that the necessary users and groups exist in the IdM server. The command also configures an additional LDAP identity provider in the hub cluster.

The following table summarizes the available users and groups in the IdM server for this exercise.

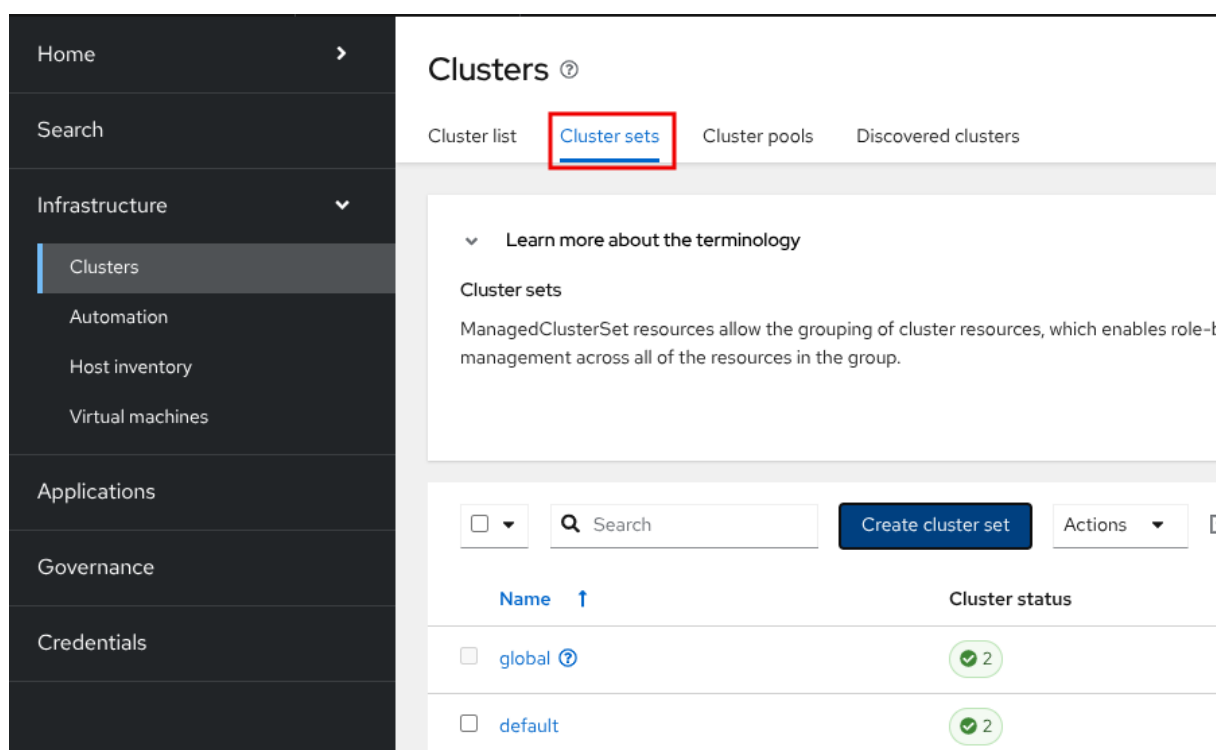
User	Group
prod-admin	production-administrators
stage-admin	stage-administrators
auditor	global-viewers

1. Log in to the RHOCF hub cluster, and switch to the RHACM web console.
 - 1.1. Open a web browser and go to `https://console-openshift-console.apps.ocp4.example.com`
 - 1.2. Select the **Red Hat Identity Management** identity provider and log in as the admin user with `redhatocp` as the password.
 - 1.3. Ensure that the cluster switcher is set to **All Clusters**, which corresponds to the RHACM web console.
 - 1.4. Verify that the `local-cluster` and the `managed-cluster` clusters are in Ready status.



2. Create the production managed cluster set, and add the local-cluster cluster by using the RHACM web console.

2.1. From the RHACM web console, go to **Infrastructure > Clusters** and click the **Cluster sets** tab. Verify that two managed cluster sets exist by default: global and default. Both managed cluster sets contain two clusters.



2.2. Click **Create cluster set** to open the dialog box.

2.3. Type production in the **Cluster set name** field and click **Create**.

Create cluster set

Creating a **ManagedClusterSet** will allow you to group resources and manage access control to all of the resources in the group together. You will be able to manage the resource assignments for access control after creating the cluster set.

Cluster set name *

production

CreateCancel

2.4. In the confirmation dialog box, click **Manage resource assignments** and select the local-cluster cluster. Click **Review**.

Manage resource assignments

Select resources to toggle their assignments to the cluster set

Resources can be added, removed, and transferred from other cluster sets (if you have permissions to remove from them from their assi

Important: assigning a resource to the cluster set will give all cluster set users permissions to the resource's namespace.

1 selected

Search

Name	Infrastructure	Distribution version
☒ local-cluster	Other	OpenShift 4.18.6
☐ managed-cluster	Other	OpenShift 4.18.6

Review

Cancel

2.5. In the confirmation dialog box, click **Save** to display the **Overview** tab of the production managed cluster set.

production

Overview Submariner add-ons Cluster list Cluster pools User management

Details

Name	production	Namespace bindings	-
Multicluster network status	Add clusters	User management	None

Status

0 Submariner add-ons Go to Submariner add-ons	1 Clusters Go to Managed clusters	0 Cluster pools Go to Cluster pools
--	--	--

2.6. In the **Cluster list** tab, click the local-cluster cluster to display the **Overview** tab of that cluster.

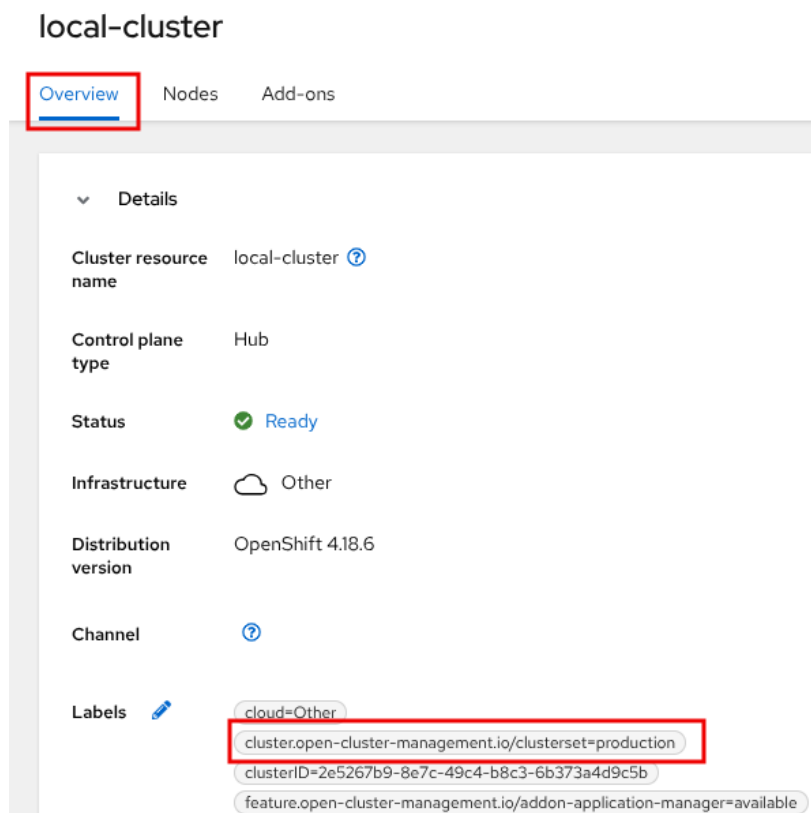
production

Overview Submariner add-ons Cluster list Cluster pools User management

☐ Search Filter Actions

Name	local-cluster
Namespace	local-cluster
Status	Ready
Infrastructure	Other
Control plane type	Hub
Distribution version	OpenShift 4.18.6
Labels	openshiftVersion-major=4 openshiftVersion-major-minor=4.18 velero.io/exclude-from-backup=true 15 more

2.7. In the **labels** section of the **Overview** tab of the local-cluster cluster, identify the cluster.open-cluster-management.io/clusterset=production label. RHACM uses this label to identify clusters that pertain to a cluster set.

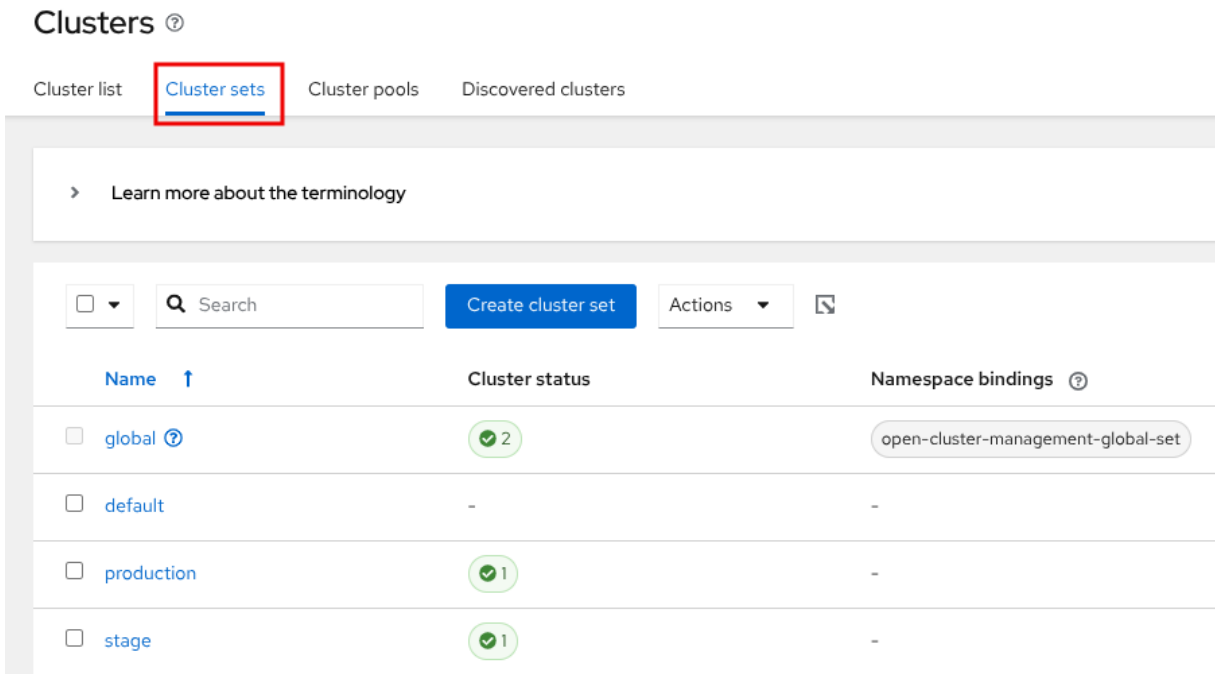


3. Create the stage managed cluster set, and add the managed-cluster cluster by using the RHACM web console.

- 3.1. From the RHACM web console, go to **Infrastructure > Clusters** and click the **Cluster sets** tab.
- 3.2. Click **Create cluster set** to open the dialog box.
- 3.3. Type stage in the **Cluster set name** field and click **Create**.
- 3.4. In the confirmation dialog box, click **Manage resource assignments** and select the managed-cluster cluster. Click **Review**.
- 3.5. In the confirmation dialog box, click **Save** to display the **Overview** tab of the stage managed cluster set.
- 3.6. In the **Cluster list** tab, click the managed-cluster cluster to display the **Overview** tab of that cluster.
- 3.7. In the **labels** section of the **Overview** tab of the managed-cluster, identify the `cluster.open-cluster-management.io/clusterset=stage` label.

4. Review the managed cluster sets.

- 4.1. Go to **Infrastructure > Clusters** and click the **Cluster sets** tab.
- 4.2. Verify that four managed cluster sets exist: global, default, production, and stage. The global managed cluster set contains the two managed clusters. The default managed cluster set does not contain any managed clusters. The production managed cluster set contains one managed cluster. The stage managed cluster set contains one managed cluster.



5. Although you could use the RHACM web console to assign roles to groups, you use the CLI for training purposes in this exercise. The following table summarizes the RHACM roles to assign to the groups of users:

Group	Roles
production-administrators	open-cluster-management:managedclusterset:admin:production open-cluster-management:managedclusterset:view:stage
stage-administrators	open-cluster-management:managedclusterset:admin:stage
global-viewers	open-cluster-management:managedclusterset:view:global

5.1. In the terminal window, log in to the hub cluster as the admin user with redhatocp as the password.

```
[student@workstation ~]$ oc login -u admin -p redhatocp \
https://api.ocp4.example.com:6443
Login successful.
...output omitted...
```

5.2. Use the `oc adm` command to assign the `open-cluster-management:managedclusterset:admin:production` role to the `production-administrators` group.

```
[student@workstation ~]$ oc adm policy add-cluster-role-to-group \
  open-cluster-management:managedclusterset:admin:production \
  production-administrators
clusterrole.rbac.authorization.k8s.io/open-cluster-management:managedclusterset:admin:production added: "production-administrators"
```

- 5.3. Use the `oc adm` command to assign the `open-cluster-management:managedclusterset:view:stage` role to the `production-administrators` group.

```
[student@workstation ~]$ oc adm policy add-cluster-role-to-group \
  open-cluster-management:managedclusterset:view:stage \
  production-administrators
clusterrole.rbac.authorization.k8s.io/open-cluster-management:managedclusterset:view:stage added: "production-administrators"
```

- 5.4. Use the `oc adm` command to assign the `open-cluster-management:managedclusterset:admin:stage` role to the `stage-administrators` group.

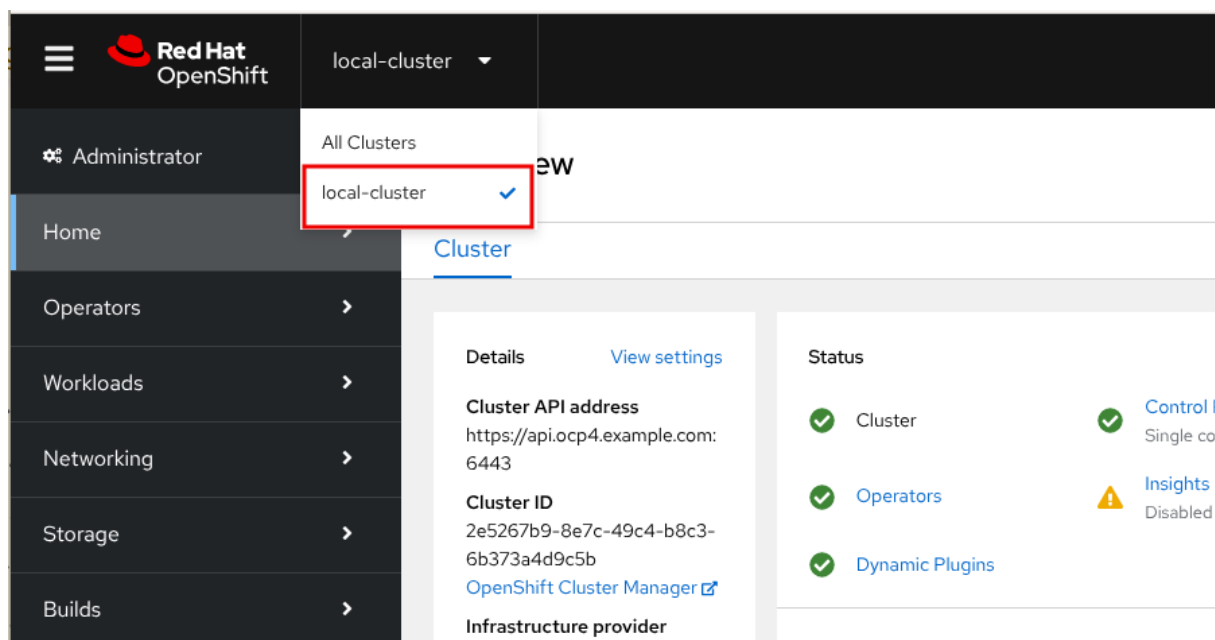
```
[student@workstation ~]$ oc adm policy add-cluster-role-to-group \
  open-cluster-management:managedclusterset:admin:stage \
  stage-administrators
clusterrole.rbac.authorization.k8s.io/open-cluster-management:managedclusterset:admin:stage added: "stage-administrators"
```

- 5.5. Use the `oc adm` command to assign the `open-cluster-management:managedclusterset:view:global` role to the `global-viewers` group.

```
[student@workstation ~]$ oc adm policy add-cluster-role-to-group \
  open-cluster-management:managedclusterset:view:global \
  global-viewers
clusterrole.rbac.authorization.k8s.io/open-cluster-management:managedclusterset:view:global added: "global-viewers"
```

6. Inspect the roles that are assigned to the groups of users and their role bindings.

- 6.1. Switch back to the browser window. From the RHACM web console, set the cluster switcher to **local-cluster**, which corresponds to the hub cluster.



- 6.2. Go to **User Management > Groups**, and click the **stage-administrators** group.

- 6.3. In the **Details** page of the group, click the **RoleBindings** tab.
- 6.4. Verify that two namespace-scoped role bindings to the admin and view roles exist in the managed-cluster namespace. Three cluster-wide role bindings to the following RHACM roles exist:
- open-cluster-management:admin:managed-cluster
 - open-cluster-management:managedclusterset:admin:stage
 - open-cluster-management:view:managed-cluster

stage-administrators

Details

YAML

RoleBindings

Filter

Name

Search by name...

Name	Role ref	Subject kind	Namespace
RB open-cluster-management:managedcluster:admin	CR admin	Group	NS managed-cluster
RB open-cluster-management:managedcluster:view	CR view	Group	NS managed-cluster
CRB open-cluster-management:managedclusterset:admin:managed-cluster	CR open-cluster-management:admin:managed-cluster	Group	All namespaces
CRB open-cluster-management:managedclusterset:admin:stage	CR open-cluster-management:managedclusterset:admin:stage	Group	All namespaces
CRB open-cluster-management:managedclusterset:view:managed-cluster	CR open-cluster-management:view:managed-cluster	Group	All namespaces

7. Verify that the users in the production-administrators group can access their corresponding managed clusters.
- 7.1. From the RHOC web console, click **admin** in the upper-right corner, and click **Log out**.
- 7.2. From the OpenShift login page, click **Red Hat Identity Management**, and log in as the prod-admin user with redhat as the password.

Log in with

kube:admin

Red Hat Identity Management

- 7.3. Ensure that the cluster switcher is set to **All Clusters**.

- 7.4. Go to **Infrastructure > Clusters** and click the **Cluster sets** tab. The production and stage managed cluster sets appear in the list.
- 7.5. Click the vertical ellipsis icon (⋮) to the right of the production managed cluster set row. Verify that the prod-admin user can edit the namespace bindings, and manage the resource assignments on the production managed cluster set.

Clusters ?

Cluster list **Cluster sets** Cluster pools Discovered clusters

> Learn more about the terminology

☐

Create cluster set

Actions ▾

1 - 2 of 2 ▾

<

>

Name ↑	Cluster status	Namespace bindings ?
☐ production	1	-
☐ stage	1	-

1 - 2 of 2 items ▾ <<

Edit namespace bindings

Manage resource assignments

Delete cluster set

- 7.6. Click the vertical ellipsis icon to the right of the stage managed cluster set row. Verify that the prod-admin user cannot perform any actions on the managed cluster set. The prod-admin user has only view access to the managed clusters in the stage managed cluster set.

Clusters ?

Cluster list **Cluster sets** Cluster pools Discovered clusters

> Learn more about the terminology

☐

Create cluster set

Actions ▾

1 - 2 of 2 ▾

<

>

Name ↑	Cluster status	Namespace bindings ?
☐ production	1	-
☐ stage	1	-

1 - 2 of 2 items ▾ <<

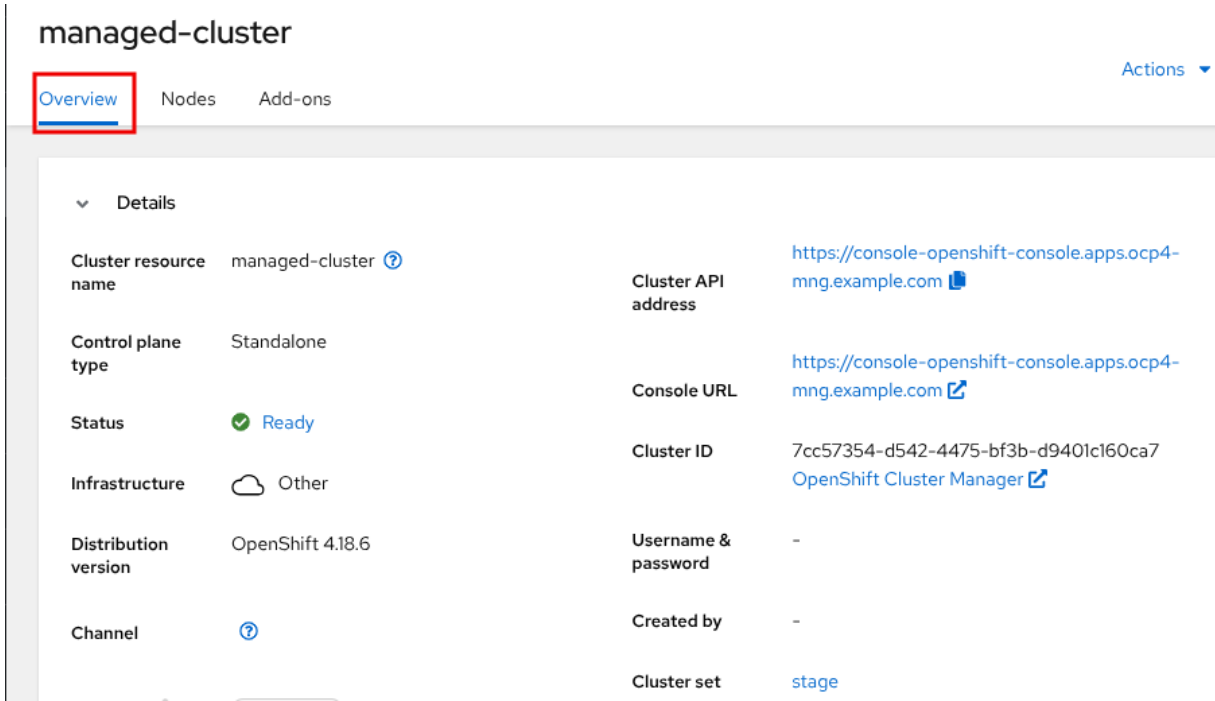
Edit namespace bindings

Manage resource assignments

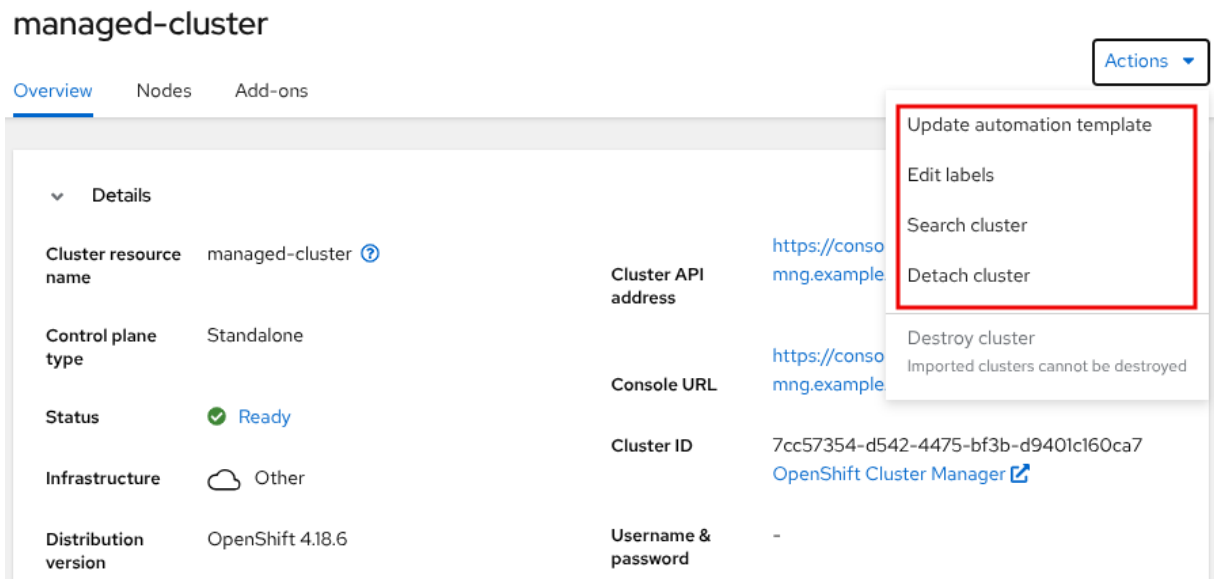
Delete cluster set

- 7.7. Click the **stage** managed cluster set. On the **Overview** page of the stage managed cluster set, click the **Cluster list** tab. The managed-cluster cluster appears in the list.
- 7.8. Click **managed-cluster** to display the **Overview** page of the managed cluster.

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright



- 7.9. Click **prod-admin** in the upper-right corner, and click **Log out**.
8. Verify that the users in the stage-administrators group can access their corresponding managed clusters.
- 8.1. From the OpenShift login page, click **Red Hat Identity Management**, and log in as the stage-admin user with redhat as the password.
- 8.2. Ensure that the cluster switcher is set to **All Clusters**.
- 8.3. Go to **Infrastructure > Clusters** and click the **Cluster sets** tab. The stage managed cluster set appears in the list.
- 8.4. Click the vertical ellipsis icon to the right of the stage managed cluster set. Verify that the stage-admin user can edit the namespace bindings, and manage the resource assignments on the stage managed cluster set.
- 8.5. Click the **stage** managed cluster set. On the **Overview** page of the stage managed cluster set, click the **Cluster list** tab. The managed-cluster cluster appears in the list.
- 8.6. Click **managed-cluster** to display the **Overview** page of the managed cluster.
- 8.7. Click **Actions** in the upper-right corner. Verify that the stage-admin user can perform actions such as editing labels and detaching the cluster.



8.8. Click **stage-admin** in the upper-right corner, and click **Log out**.

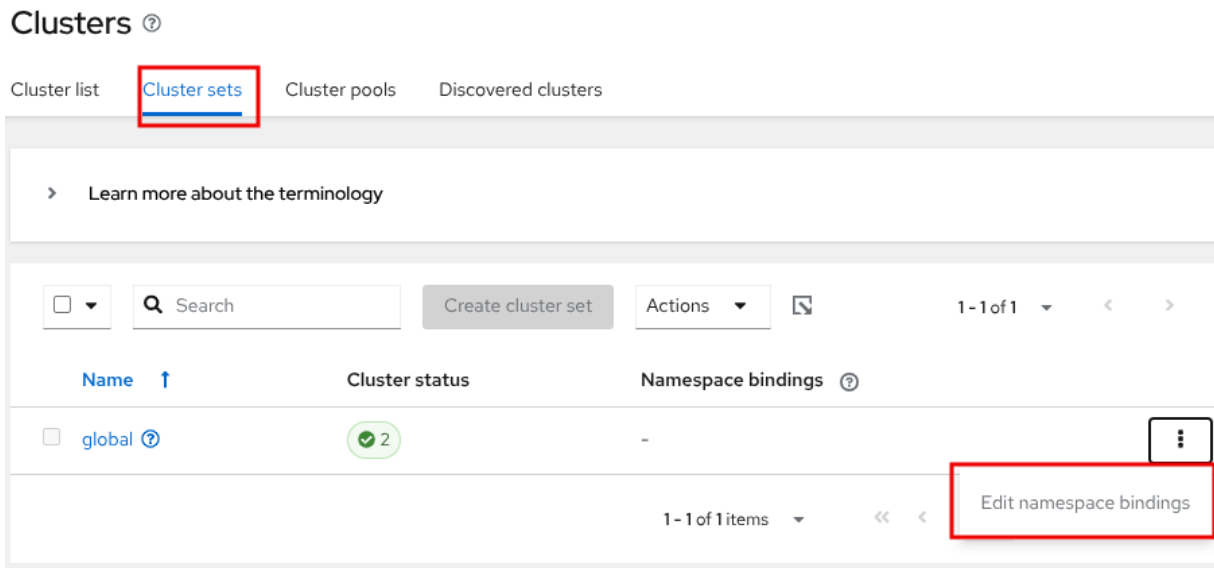
9. Verify that the users in the `global-viewers` group can access their corresponding managed clusters.

9.1. From the OpenShift login page, click **Red Hat Identity Management**, and log in as the `auditor` user with `redhat` as the password.

9.2. Ensure that the cluster switcher is set to **All Clusters**.

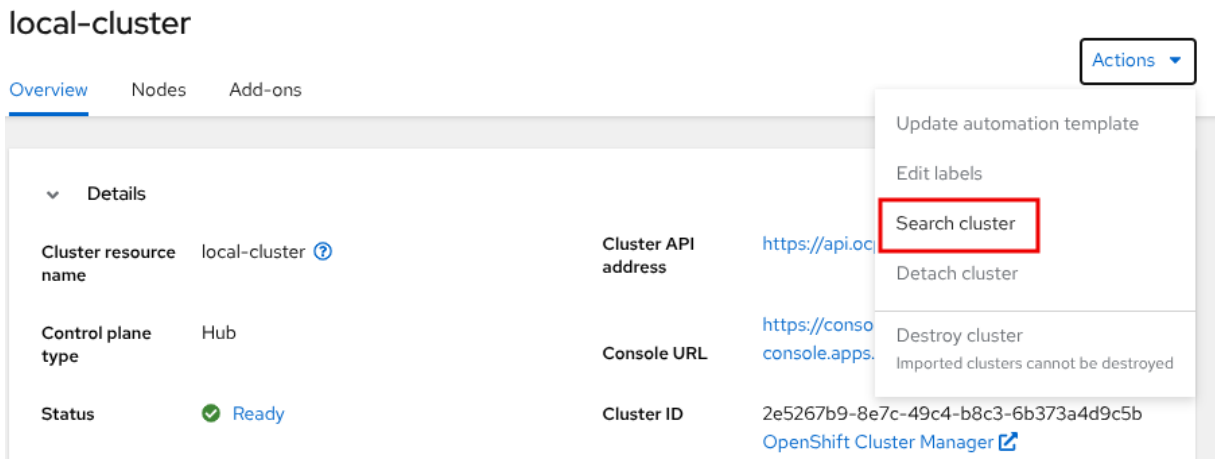
9.3. Go to **Infrastructure > Clusters** and click the **Cluster sets** tab. The default `global` managed cluster set appears in the list.

9.4. Click the vertical ellipsis icon to the right of the `global` managed cluster set. Verify that the `auditor` user cannot take any actions on the `global` managed cluster set.

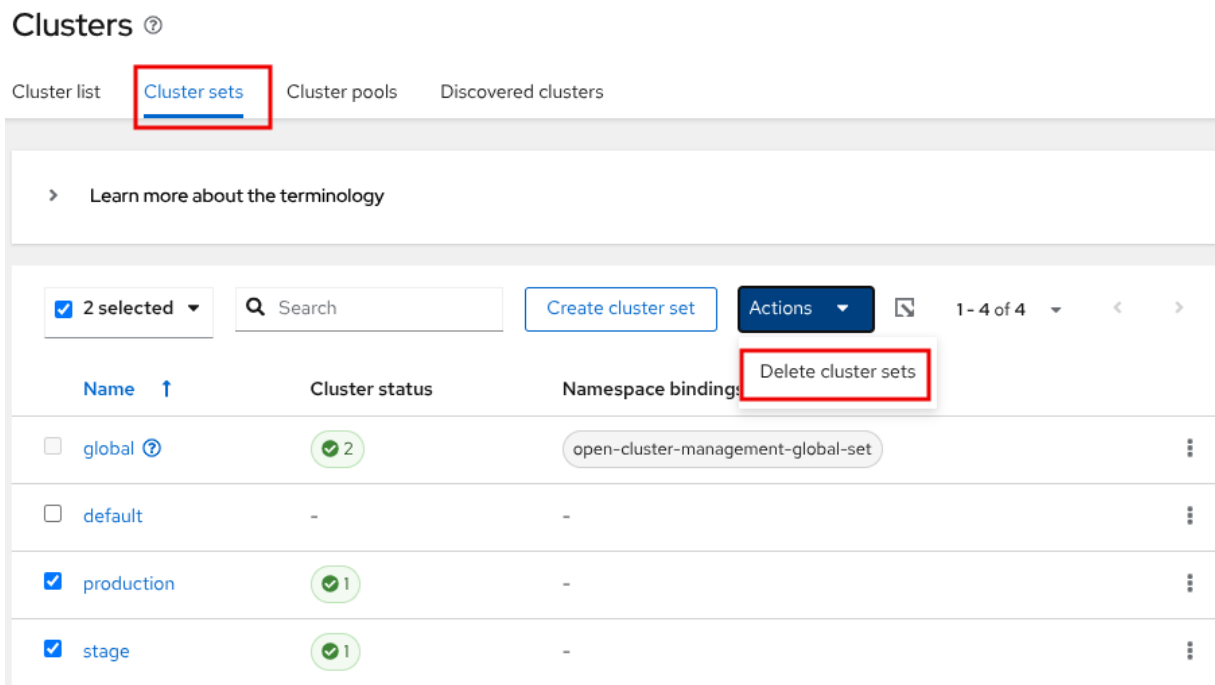


9.5. Click the **Cluster list** tab and click **local-cluster**.

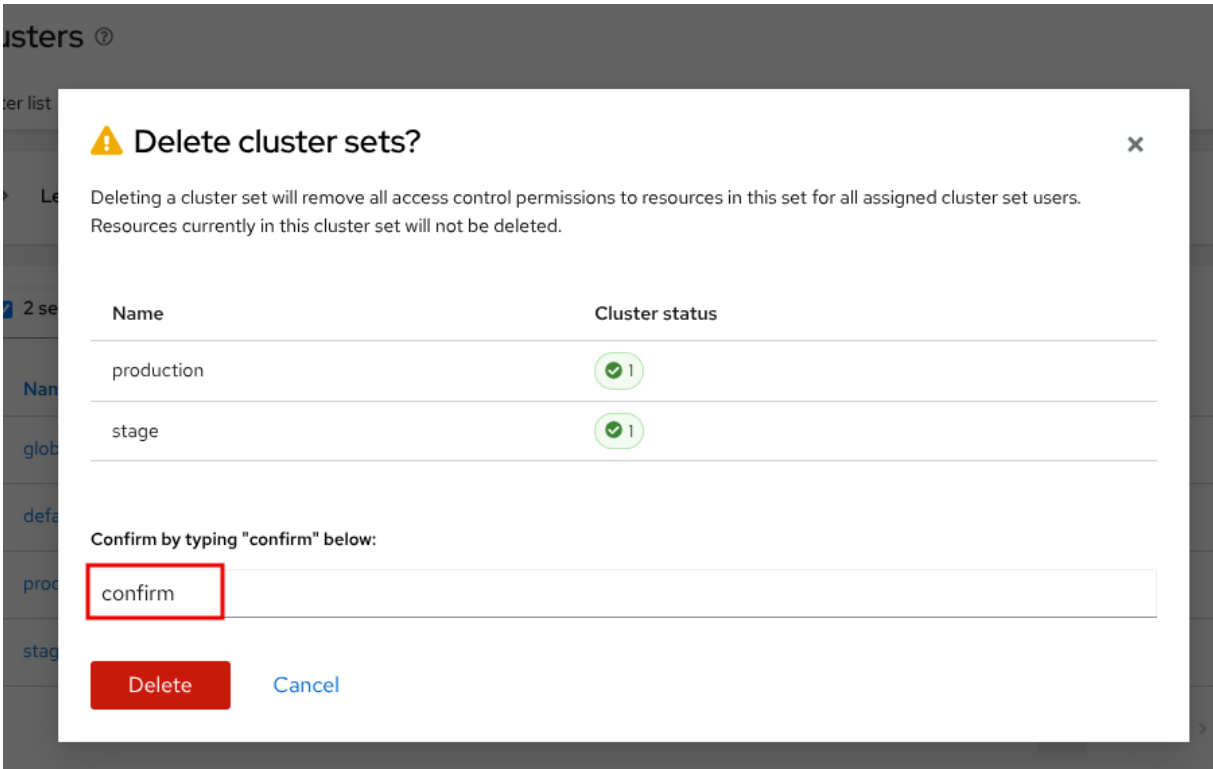
9.6. Click **Actions** in the upper-right corner. Verify that only the **Search cluster** action is enabled for the `auditor` user.



- 9.7. Go to **Infrastructure > Clusters**, and in the **Cluster list** tab, click **managed-cluster**.
- 9.8. Click **Actions** in the upper-right corner. Verify that only the **Search cluster** action is enabled for the auditor user.
- 9.9. Click **auditor** in the upper-right corner, and click **Log out**.
10. Delete the production and stage managed cluster sets.
- 10.1. From the OpenShift login page, click **Red Hat Identity Management**, and log in as the admin user with redhatocp as the password.
- 10.2. Ensure that the cluster switcher is set to **All Clusters**.
- 10.3. Go to **Infrastructure > Clusters** and click the **Cluster sets** tab. Select the production and stage managed cluster sets. Click **Actions** and click **Delete cluster sets**.



- 10.4. In the confirmation dialog box, type confirm and click **Delete**.



10.5. Click **admin** in the upper-right corner and click **Log out**.

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish manage-configure
```


2.7. Troubleshooting Common Import Issues for Managed Clusters

Objectives

- Investigate and resolve common issues that might occur when importing managed clusters.

Troubleshooting RHACM Issues

Red Hat provides extensive documentation for troubleshooting RHACM. Refer to this documentation as your first step when investigating managed cluster import issues.

The references section at the end of this lecture provides links to this troubleshooting documentation.

If you still cannot resolve your issue, then use the `must-gather` command that Red Hat provides. This command collects data about your clusters and stores the result in a directory. Some log files in this directory might help you diagnose your installation.

As a last resort, contact the Red Hat support team at <https://access.redhat.com/support>, and attach the `must-gather` directory to the case.

The following sections present common issues when importing managed clusters into RHACM.

Troubleshooting Managed Cluster Creation Failures

After creating a cluster from the RHACM web console, several issues might prevent the new cluster from reporting to RHACM:

- The network between the new cluster and the hub cluster does not meet the requirements. Specifically, TCP ports 6443 and 443 in both directions are required. Also, both clusters must resolve the DNS names of the API endpoints.
- The API endpoint TLS certificate is not trusted. The clusters must trust the certificate of their counterparts. If you use a custom certificate authority (CA) to issue these certificates, then ensure that the CA certificate is part of the trusted CA bundle.
- The time between the clusters is not synchronized. Ensure that you configured the `chrony` time service in your clusters.
- The Kubernetes managed cluster cannot pull the RHACM images from the Red Hat registries. Kubernetes clusters require a secret with your RHOCP pull secret for retrieving the images from Red Hat.

Troubleshooting Managed Cluster Import Failures

In addition to the previous issues, other conditions might prevent a managed cluster from being imported into RHACM:

- The import controller pods that are running in the hub cluster failed. Review the status and logs of these pods. The pods run in the `multicluster-engine` namespace and have the `app=managedcluster-import-controller-v2` label:

```
[user@host ~]$ oc get pods -n multicluster-engine \
-l app=managedcluster-import-controller-v2
NAME                                READY STATUS RESTARTS AGE
managedcluster-import-controller-v2-bbd65d957-5mbqc 1/1    Running 0       171m
managedcluster-import-controller-v2-bbd65d957-86wpf 1/1    Running 0       171m
[user@host ~]$ oc logs -n multicluster-engine \
-l app=managedcluster-import-controller-v2
...output omitted...
```

- The import secret resource was not created in the hub cluster. For each managed cluster, RHACM creates a namespace with the name of the cluster. In this namespace, RHACM creates the `cluster-name-import` secret. The following command lists the import secret for the `mycluster` managed cluster:

```
[user@host ~]$ oc get secret mycluster-import -n mycluster
NAME            TYPE      DATA   AGE
mycluster-import Opaque    6       9m31s
```

If the secret does not exist, then review the logs of the import controller pods:

```
[user@host ~]$ oc logs -n multicluster-engine \
-l app=managedcluster-import-controller-v2
...output omitted...
```

- The `klusterlet` resource on the managed cluster has a degraded status. Verify the status of the `RegistrationDesiredDegraded` and `WorkDesiredDegraded` conditions in the `.status.conditions` section of the resource. Both conditions must be false:

```
[user@host ~]$ oc get klusterlets klusterlet \
-o jsonpath-as-json='{.status.conditions}'
[
  [
    ...output omitted...
    {
      "lastTransitionTime": "2025-04-28T10:07:39Z",
      "message": "deployments replicas are desired: 1",
      "observedGeneration": 2,
      "reason": "DeploymentsFunctional",
      "status": "False",
      "type": "RegistrationDesiredDegraded"
    },
    {
      "lastTransitionTime": "2025-04-28T10:07:39Z",
      "message": "deployments replicas are desired: 1",
      "observedGeneration": 2,
      "reason": "DeploymentsFunctional",
      "status": "False",
      "type": "WorkDesiredDegraded"
    },
    ...output omitted...
  ]
]
```

Troubleshooting Clusters with a Pending Import Status

Importing a managed cluster that was previously attached to another RHACM hub might fail. This issue occurs when the managed cluster was not properly detached: some stale resources, such as the `klusterlet` resource, might still exist.

To resolve this issue, clean up the managed cluster by deleting the `klusterlet` resource, the `open-cluster-management-agent` and `open-cluster-management-agent-addon` namespaces, and the RHACM custom resource definitions (CRDs).

To help you in this cleanup process, Red Hat provides a script in the [Knowledgebase: "Unable to Import on RHOCP Cluster into New ACM Hub Cluster Due to Pre-existing ACM Resources"](#) article.

Troubleshooting Managed Clusters with an Error Status

You can verify the status of a managed cluster from the RHACM web console. Go to **Infrastructure > Clusters** and click the name of the managed cluster.

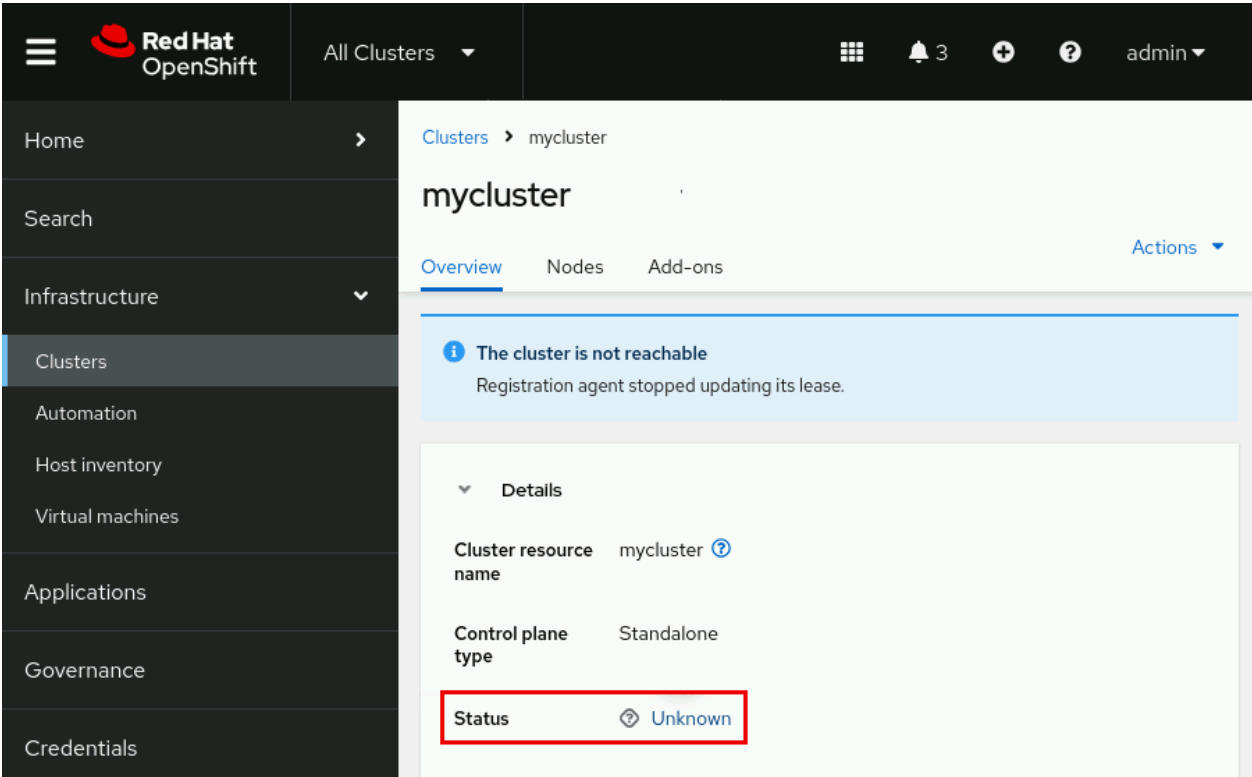


Figure 16. Managed cluster with an unknown status

You can also verify a managed cluster status from the command line. In the hub cluster, each managed cluster is represented with a `managedcluster` resource. The name of the resource is the name of the managed cluster. The `.status.conditions` section of the resource provides the status of the managed cluster.

The following command displays the status of the `mycluster` managed cluster. Under normal conditions, the command reports all the statuses as `True`. However, the `ManagedClusterConditionAvailable` condition has the `Unknown` status, which indicates that the managed cluster is not available:

```
[user@host ~]$ oc get managedcluster mycluster \
-o jsonpath-as-json='{.status.conditions}'
[
  [
    {
      "lastTransitionTime": "2025-04-30T05:23:08Z",
      "message": "Import succeeded",
      "reason": "ManagedClusterImported",
      "status": "True",
      "type": "ManagedClusterImportSucceeded"
    },
    {
      "lastTransitionTime": "2025-04-30T05:22:51Z",
      "message": "Accepted by hub cluster admin",
      "reason": "HubClusterAdminAccepted",
      "status": "True",
      "type": "HubAcceptedManagedCluster"
    },
    {
      "lastTransitionTime": "2025-04-30T05:23:01Z",
      "message": "Managed cluster joined",
      "reason": "ManagedClusterJoined",
      "status": "True",
      "type": "ManagedClusterJoined"
    },
    {
      "lastTransitionTime": "2025-04-30T06:22:55Z",
      "message": "Registration agent stopped updating its lease.",
      "reason": "ManagedClusterLeaseUpdateStopped",
      "status": "Unknown",
      "type": "ManagedClusterConditionAvailable"
    },
    {
      "lastTransitionTime": "2025-04-30T05:23:01Z",
      "message": "The clock of the managed cluster is synced with the hub.",
      "reason": "ManagedClusterClockSynced",
      "status": "True",
      "type": "ManagedClusterConditionClockSynced"
    }
  ]
]
```

If a condition is not True, then retrieve the status of the `klusterlet` resource from the managed cluster.

```
[user@host ~]$ oc get klusterlets klusterlet \
-o jsonpath-as-json='{.status.conditions}'
...output omitted...
```

Troubleshooting a Status That Alternates Between Unknown and Ready

When the network between the managed cluster and the hub cluster is unstable, the status of the managed cluster might alternate between Unknown and Ready.

In case of a network outage, the default grace period before reporting the managed cluster as offline is five minutes. You can adjust this period by updating the value of the `.spec.leaseDurationSeconds` parameter in the `managedcluster` resource for the managed cluster:

```
apiVersion: cluster.open-cluster-management.io/v1
kind: ManagedCluster
metadata:
  ...output omitted...
  name: mycluster
spec:
  hubAcceptsClient: true
  leaseDurationSeconds: 60
  managedClusterClientConfigs:
    - caBundle: LS0t...Cg==
  ...output omitted...
```

Because RHACM retries the connection five times, the total grace period is `leaseDurationSeconds` x 5. For example, if you set the `leaseDurationSeconds` parameter to 120, then the grace period is 10 minutes (120 x 5 seconds).

Collecting Data and Opening Support Cases

If you cannot fix your issue, then your next option is to collect data about your clusters and to send the data to the Red Hat support team for help:

- Collect data from the RHACM hub cluster by running the `oc adm must-gather` command. Use the `--image` option to provide the plug-in image that the `must-gather` command uses to collect RHACM data. Use the `--dest-dir` option to specify the directory that the command creates to store the data.

```
[user@host ~]$ oc adm must-gather \
  --image=registry.redhat.io/rhacm2/acm-must-gather-rhel9:v2.13 \
  --dest-dir=./must-gather-rhacm
...output omitted...
```

- Retrieve the identifier of your hub cluster. You need this information to create the archive that you send to the support team.

```
[user@host ~]$ oc get clusterversion \
  -o jsonpath='{.items[].spec.clusterID}'
2e52...9c5b
```

- Create an archive of the data directory. The name of the archive must include the current date and time, and the cluster identifier.

```
[user@host ~]$ date +"%m-%d-%Y-%H-%M-%S"
05-25-2025-12-45-36
[user@host ~]$ tar cvaf \
  must-gather-05-25-2025-12-45-36-2e52...9c5b.tar.gz \
  ./must-gather-rhacm
...output omitted...
```

- Open a support case at <https://access.redhat.com/support>, and attach the archive to the ticket.

References

Knowledgebase: "How to Troubleshoot RHACM Cluster Import Failure?"

For more information about troubleshooting RHACM, refer to the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Troubleshooting* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/troubleshooting/index

For more information about RHACM known issues, refer to the *Known Issues and Limitations for Cluster Lifecycle with Multicluster Engine Operator* section in the *Release Notes for Cluster Lifecycle with Multicluster Engine Operator* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Clusters* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/clusters/index#known-issues-mce

For more information about the required network connections, refer to the *Managed Cluster Network Configuration* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Networking* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/networking/index#managed-network-config

For more information about the `must-gather` command, refer to the *Running the `must-gather` Command to Troubleshoot Red Hat Advanced Cluster Management Components* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Troubleshooting* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/troubleshooting/index#running-the-must-gather-command-to-troubleshoot

2.8. Guided Exercise

Troubleshoot Common Import Issues for Managed Clusters

Investigate and resolve common import issues for managed clusters.

Outcomes

- Identify issues when importing managed clusters into RHACM.
- Remove existing RHACM resources from managed clusters.

Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab install do0012l ; lab start manage-troubleshoot
```

Instructions

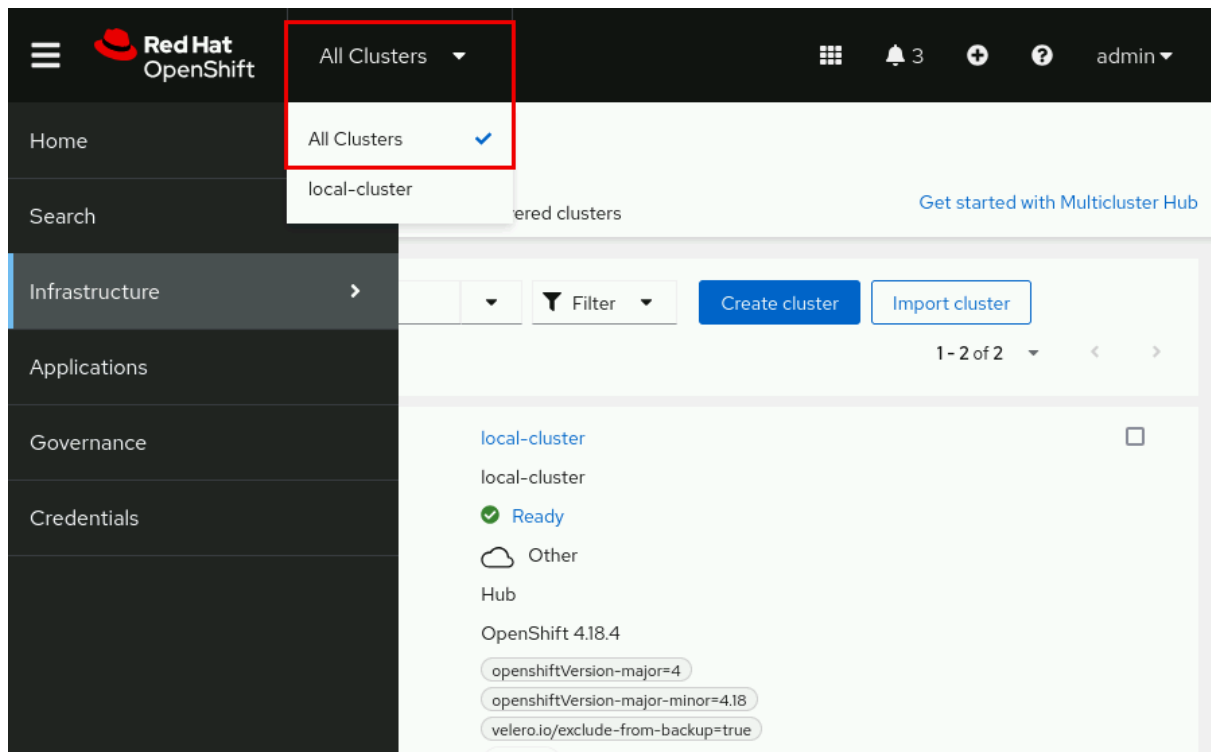
In this exercise, you import an existing cluster into RHACM. This cluster was previously used for a proof of concept, and was attached to a now decommissioned RHACM cluster.

1. Log in to the managed RHOCP cluster.

In a terminal window, log in to the RHOCP cluster as the `admin` user with `redhatocp` as the password. In the following command, ensure that you use the API URL of the managed cluster, which is `https://api.ocp4-mng.example.com:6443`

```
[student@workstation ~]$ oc login -u admin -p redhatocp \
https://api.ocp4-mng.example.com:6443
Login successful.
...output omitted...
```

2. If you are not already connected to the RHACM web console, then use the web browser to log in to the console as the `admin` user.
 - 2.1. Open a web browser and go to `https://console-openshift-console.apps.ocp4.example.com`
 - 2.2. Select the **Red Hat Identity Management** identity provider and log in as the `admin` user with `redhatocp` as the password.
 - 2.3. Ensure that the cluster switcher is set to **All Clusters**, which corresponds to the RHACM web console.



3. Import the ocp4-mng.example.com cluster from the RHACM web console.

3.1. Go to **Infrastructure > Clusters** and click **Import cluster**.

3.2. On the **Import an existing cluster** page, use the managed-cluster name to identify the cluster. Click **Next**.

3.3. On the **Automation** page, click **Next**.

3.4. Click **Generate command**. The browser redirects you to the cluster detail page. Click **Copy command** to copy the command to the clipboard to be executed from the terminal.

Keep that page open, because you copy the command again in another step.

3.5. Switch back to the terminal, paste the command, and then press **Enter** to run the command. Paste content from the clipboard by pressing **Ctrl+Shift+V** or use your preferred method. The paste command is long and mostly Base64-encoded.

The command fails because the managed cluster was previously connected to a decommissioned RHACM cluster, and the managed cluster was not detached correctly.

```
[student@workstation ~]$ echo "Ci0t...MD0K" | base64 -d | \
  kubectl apply -f - || echo "VGhl...bi4=" | base64 -d
Error from server (AlreadyExists): error when creating "STDIN":
customresourcedefinitions.apiextensions.k8s.io "klusterlets.operator.open-cluster-management.io"
already exists
The cluster cannot be imported because its Klusterlet CRD already exists.
Either the cluster was already imported, or it was not detached completely during a previous detach
process.
Detach the existing cluster before trying the import again.
```

4. Clean the managed cluster.

Note

To clean managed clusters, the [Knowledgebase: "Unable to Import an RHOC P Cluster into New ACM Hub Cluster Due to Pre-existing ACM Resources"](#) article provides a script.

You could copy this script and run it in the terminal.

For training purposes, you perform the cleaning steps manually in this exercise.

- 4.1. Verify that the `klusterlet` resource exists. This resource represents the RHACM agent.

```
[student@workstation ~]$ oc get klusterlet
NAME          AGE
klusterlet    12m
```

- 4.2. Delete the `klusterlet` resource.

```
[student@workstation ~]$ oc delete klusterlet klusterlet
klusterlet.operator.open-cluster-management.io "klusterlet" deleted
```

- 4.3. Delete the namespaces that the agent and the add-ons use.

```
[student@workstation ~]$ oc delete namespaces \
  open-cluster-management-agent open-cluster-management-agent-addon
project.project.openshift.io "open-cluster-management-agent" deleted
project.project.openshift.io "open-cluster-management-agent-addon" deleted
```

- 4.4. Delete the RHACM custom resource definitions (CRDs). The CRD names include the `open-cluster-management.io` group name. Use this string to filter and delete the CRDs.

```
[student@workstation ~]$ oc get crd -o name | grep \
  open-cluster-management | xargs oc delete
customresourcedefinition.apiextensions.k8s.io "appliedmanifestworks.work.open-cluster-
management.io" deleted
customresourcedefinition.apiextensions.k8s.io "certificatepolicies.policy.open-cluster-
management.io" deleted
customresourcedefinition.apiextensions.k8s.io "clusterclaims.cluster.open-cluster-management.io"
deleted
customresourcedefinition.apiextensions.k8s.io "configurationpolicies.policy.open-cluster-
management.io" deleted
customresourcedefinition.apiextensions.k8s.io "helmreleases.apps.open-cluster-management.io"
deleted
customresourcedefinition.apiextensions.k8s.io "klusterlets.operator.open-cluster-management.io"
deleted
customresourcedefinition.apiextensions.k8s.io "operatorpolicies.policy.open-cluster-management.io"
deleted
customresourcedefinition.apiextensions.k8s.io "policies.policy.open-cluster-management.io" deleted
customresourcedefinition.apiextensions.k8s.io "subscriptions.apps.open-cluster-management.io"
deleted
customresourcedefinition.apiextensions.k8s.io "subscriptionstatuses.apps.open-cluster-
management.io" deleted
```

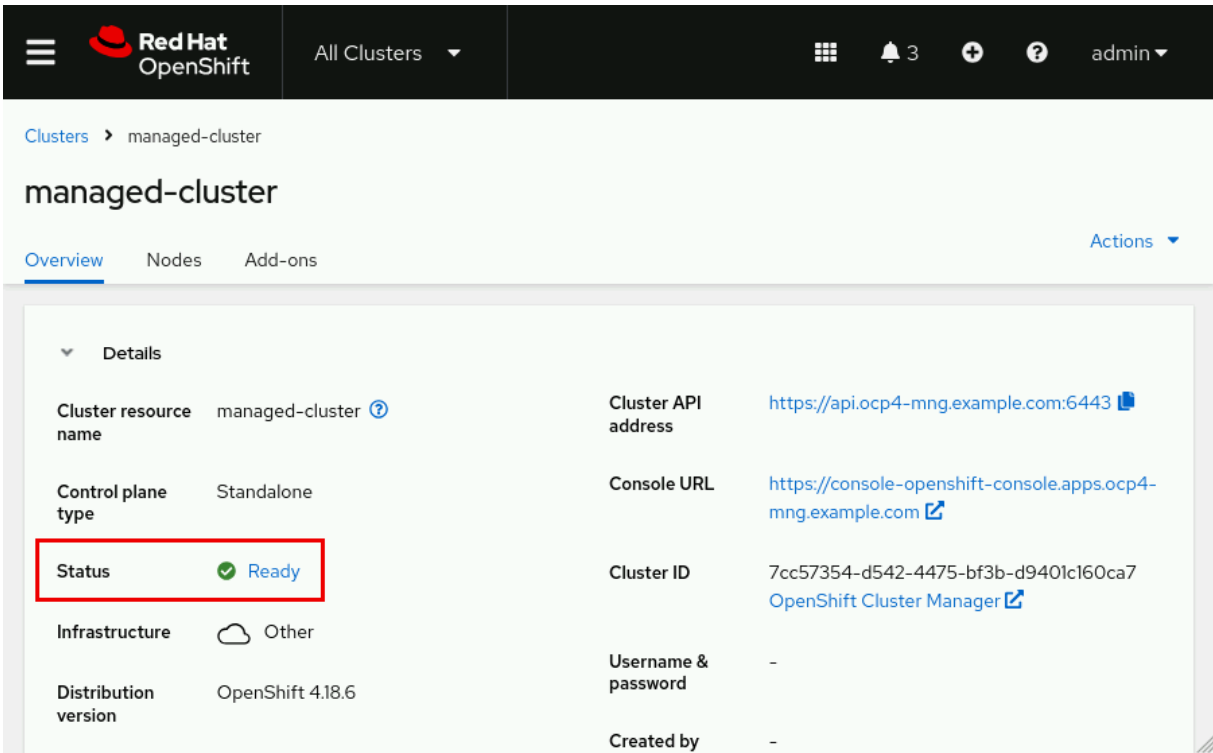
5. Rerun the import command.

- 5.1. Switch back to the web browser and click **Copy command**.

- 5.2. Switch back to the terminal, paste the command, and then press **Enter** to run the command. The command is expected to succeed.

```
[student@workstation ~]$ echo "Ci0t...MD0K" | base64 -d | \
  kubectl apply -f - || echo "VGhl...bi4=" | base64 -d
customresourcedefinition.apiextensions.k8s.io/klusterlets.operator.open-cluster-management.io
created
namespace/open-cluster-management-agent created
serviceaccount/klusterlet created
clusterrole.rbac.authorization.k8s.io/klusterlet created
clusterrole.rbac.authorization.k8s.io/klusterlet-bootstrap-kubeconfig created
clusterrole.rbac.authorization.k8s.io/open-cluster-management:klusterlet-admin-aggregate-
clusterrole created
clusterrolebinding.rbac.authorization.k8s.io/klusterlet created
deployment.apps/klusterlet created
secret/bootstrap-hub-kubeconfig created
klusterlet.operator.open-cluster-management.io/klusterlet created
secret/open-cluster-management-image-pull-credentials created
```

5.3. Switch back to the web browser to verify the import of the managed cluster. The managed cluster is displayed in the RHACM web console with the Ready status.



Finish

On the workstation machine, use the lab command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish manage-troubleshoot
```

2.9. Lab

Manage a Cluster by Using Red Hat Advanced Cluster Management for Kubernetes

Import an existing cluster. Configure view access to the managed cluster for managers, and configure administrative access for developers. Search for a deployment with fewer than three replicas as a member of the managers group. Scale the deployment to three replicas as a member of the developers group by using the RHACM web console.

Outcomes

- Import managed clusters into an RHACM hub.
- Troubleshoot import issues.
- Configure managed cluster sets and grant access rights to sets.
- Search and fix resources by using the RHACM web console.

Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab install do0012l ; lab start manage-review
```

Instructions

In this exercise, you import a managed cluster into RHACM and grant access to this cluster to the managers and the developers groups that the `lab` command prepared. You also fix a slow-running application by increasing its number of replicas.

The classroom environment provides two RHOCP clusters: an RHACM hub cluster and a managed cluster. You can access those clusters by using the following parameters:

RHACM hub cluster web console URL

`https://console-openshift-console.apps.ocp4.example.com`

RHACM hub cluster API URL

`https://api.ocp4.example.com:6443`

Managed cluster web console URL

`https://console-openshift-console.apps.ocp4-mng.example.com`

Managed cluster API URL

`https://api.ocp4-mng.example.com:6443`

Administrator username for both clusters

admin (select the **Red Hat Identity Management** identity provider)

Administrator password for both clusters

redhatocp

The lab command added two groups and two users to the hub cluster:

Groups	Users	Passwords
developers	dquincy	redhat
managers	mshika	redhat

You can log in to the cluster with these accounts by selecting the **Red Hat Identity Management** provider.

1. Import the managed cluster into RHACM with `managed-cluster` as the name. Because this cluster was previously connected to another RHACM hub cluster, you must clean the managed cluster for the import process to succeed.
2. Create a managed cluster set, named `dev-clusters`, and assign the new managed cluster to this set.
3. Grant the `developers` group administrative access to the `dev-clusters` set, and grant the `managers` group the viewing access.

You can verify your configuration by logging in to the RHACM web console as the `dquincy` user, which is a member of the `developers` group, or as the `mshika` user, which is a member of the `managers` group. When you log in, select the **Red Hat Identity Management** provider. Use `redhat` as the password for both users.

4. As the `mshika` user, use the RHACM web console to locate the Kubernetes resources for the application with the `tier=frontend` label and that runs with fewer than three current replicas.

When you log in as the `mshika` user, select the **Red Hat Identity Management** provider and use `redhat` for the password.

 Important

It takes up to five minutes for RHACM to build the search index for the new managed cluster. If your search request does not return a result, or if the current index is not available, then wait five minutes, refresh the web browser, and try again.

5. As the `dquincy` user, increase the number of replicas to three. The `dquincy` user is a member of the `developers` group, which has write access to the deployment.

When you log in as the `dquincy` user, select the **Red Hat Identity Management** provider and use `redhat` as the password.

When done, log out of the RHACM web console before grading your work.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade manage-review
```

Finish

As the student user on the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish manage-review
```

Solution

Import an existing cluster. Configure view access to the managed cluster for managers, and configure administrative access for developers. Search for a deployment with fewer than three replicas as a member of the managers group. Scale the deployment to three replicas as a member of the developers group by using the RHACM web console.

Outcomes

- Import managed clusters into an RHACM hub.
- Troubleshoot import issues.
- Configure managed cluster sets and grant access rights to sets.
- Search and fix resources by using the RHACM web console.

Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab install do0012l ; lab start manage-review
```

Instructions

In this exercise, you import a managed cluster into RHACM and grant access to this cluster to the managers and the developers groups that the `lab` command prepared. You also fix a slow-running application by increasing its number of replicas.

The classroom environment provides two RHOCP clusters: an RHACM hub cluster and a managed cluster. You can access those clusters by using the following parameters:

RHACM hub cluster web console URL

`https://console-openshift-console.apps.ocp4.example.com`

RHACM hub cluster API URL

`https://api.ocp4.example.com:6443`

Managed cluster web console URL

`https://console-openshift-console.apps.ocp4-mng.example.com`

Managed cluster API URL

`https://api.ocp4-mng.example.com:6443`

Administrator username for both clusters

admin (select the **Red Hat Identity Management** identity provider)

Administrator password for both clusters

redhatocp

The `lab` command added two groups and two users to the hub cluster:

Groups	Users	Passwords
developers	dquincy	redhat
managers	mshika	redhat

You can log in to the cluster with these accounts by selecting the **Red Hat Identity Management** provider.

1. Import the managed cluster into RHACM with managed-c luster as the name. Because this cluster was previously connected to another RHACM hub cluster, you must clean the managed cluster for the import process to succeed.
 - 1.1. To clean the managed cluster, use the terminal and log in to the cluster as the admin user with redhatocp as the password. In the following command, ensure that you use the API URL of the managed cluster, which is https://api.ocp4-mng.example.com:6443

```
[student@workstation ~]$ oc login -u admin -p redhatocp \
https://api.ocp4-mng.example.com:6443
Login successful.
...output omitted...
```

- 1.2. Verify that the klusterlet resource exists from a previous import.

```
[student@workstation ~]$ oc get klusterlet
NAME          AGE
klusterlet    12m
```

- 1.3. Delete the klusterlet resource.

```
[student@workstation ~]$ oc delete klusterlet klusterlet
klusterlet.operator.open-cluster-management.io "klusterlet" deleted
```

- 1.4. Delete the namespaces that the agent and the add-ons use.

```
[student@workstation ~]$ oc delete namespaces \
open-cluster-management-agent open-cluster-management-agent-addon
project.project.openshift.io "open-cluster-management-agent" deleted
project.project.openshift.io "open-cluster-management-agent-addon" deleted
```

- 1.5. Delete the RHACM custom resource definitions (CRDs). The CRD names include the open-c luster-management . io group name. Use this string to filter and delete the CRDs.

```
[student@workstation ~]$ oc get crd -o name | grep \
open-cluster-management | xargs oc delete
customresourcedefinition.apiextensions.k8s.io "appliedmanifestworks.work.open-cluster-
management.io" deleted
...output omitted...
```

- 1.6. To import the managed cluster, access the RHACM web console by opening a web browser and by going to https://console-openshift-console.apps.ocp4.example.com
 - 1.7. Select the **Red Hat Identity Management** identity provider and log in as the admin user with redhatocp as the password.

- 1.8. Ensure that the cluster switcher is set to **All Clusters**, which corresponds to the RHACM web console.
- 1.9. Go to **Infrastructure > Clusters** and click **Import cluster**.
- 1.10. On the **Import an existing cluster** page, use the managed-cluster name to identify the cluster. Click **Next**.
- 1.11. On the **Automation** page, click **Next**.
- 1.12. Click **Generate command**. The browser redirects you to the cluster detail page. Click **Copy command** for the command to be executed from the terminal.
- 1.13. Switch back to the terminal, paste the command, and then press **Enter** to run the command. Paste content from the clipboard by pressing **Ctrl+Shift+V** or use your preferred method. The paste command is long and mostly Base64-encoded.

```
[student@workstation ~]$ echo "Ci0t...MD0K" | base64 -d | \
  kubectl apply -f - || echo "VGhl...bi4=" | base64 -d
namespace/open-cluster-management-agent created
serviceaccount/klusterlet created
clusterrole.rbac.authorization.k8s.io/klusterlet unchanged
clusterrole.rbac.authorization.k8s.io/klusterlet-bootstrap-kubeconfig unchanged
clusterrole.rbac.authorization.k8s.io/open-cluster-management:klusterlet-admin-aggregate-
clusterrole unchanged
clusterrolebinding.rbac.authorization.k8s.io/klusterlet unchanged
deployment.apps/klusterlet created
secret/bootstrap-hub-kubeconfig created
klusterlet.operator.open-cluster-management.io/klusterlet created
secret/open-cluster-management-image-pull-credentials created
```

- 1.14. Switch back to the web browser to verify the import of the managed cluster. The managed cluster is displayed in the RHACM web console with the Ready status.

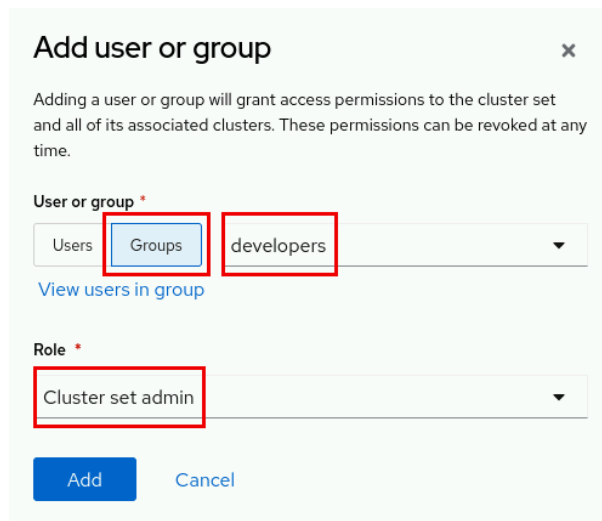
2. Create a managed cluster set, named dev-clusters, and assign the new managed cluster to this set.

- 2.1. Go to **Infrastructure > Clusters** and click the **Cluster sets** tab.
- 2.2. Click **Create cluster set** to open the dialog box.
- 2.3. Type dev-clusters in the **Cluster set name** field and click **Create**.
- 2.4. In the confirmation dialog box, click **Manage resource assignments** and select the managed-cluster cluster. Click **Review**.
- 2.5. In the confirmation dialog box, click **Save**.

3. Grant the developers group administrative access to the dev-clusters set, and grant the managers group the viewing access.

You can verify your configuration by logging in to the RHACM web console as the dquincy user, which is a member of the developers group, or as the mshika user, which is a member of the managers group. When you log in, select the **Red Hat Identity Management** provider. Use redhat as the password for both users.

- 3.1. Go to **Infrastructure > Clusters**, click the **Cluster sets** tab, and click the dev-clusters set.
- 3.2. Click the **User management** tab and click **Add user or group**.
- 3.3. Click **Groups**, select the developers group, and set **Role** to Cluster set admin. Click **Add**.



3.4. Repeat the process for the managers group. Click **Add user or group**, click **Groups**, select the managers group, and set **Role** to Cluster set view. Click **Add**.

4. As the mshika user, use the RHACM web console to locate the Kubernetes resources for the application with the tier=frontend label and that runs with fewer than three current replicas.

When you log in as the mshika user, select the **Red Hat Identity Management** provider and use redhat for the password.

Important

It takes up to five minutes for RHACM to build the search index for the new managed cluster. If your search request does not return a result, or if the current index is not available, then wait five minutes, refresh the web browser, and try again.

4.1. Log out of the RHACM web console. Click **admin** in the upper-right corner, and click **Log out**.

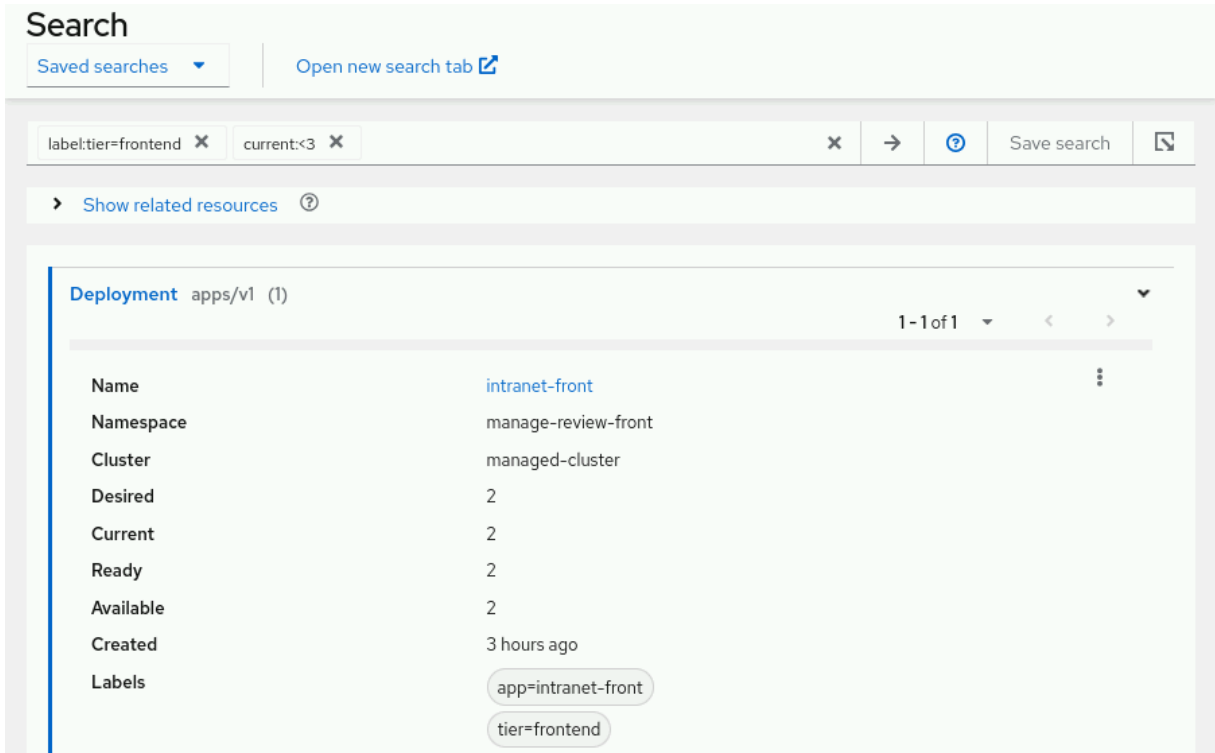
4.2. From the OpenShift login page, click **Red Hat Identity Management**, and log in as the mshika user with redhat as the password.

4.3. Go to **Search** and type label:tier=frontend current:<3 in the search field.

The intranet-front deployment in the manage-review-front namespace is displayed. The deployment has two replicas. Because the mshika user is a member of the managers group, which has view access only, you cannot modify the deployment.

Important

If your search request does not return the result, then wait five minutes, refresh the web browser, and retry the search request.



5. As the dquincy user, increase the number of replicas to three. The dquincy user is a member of the developers group, which has write access to the deployment.

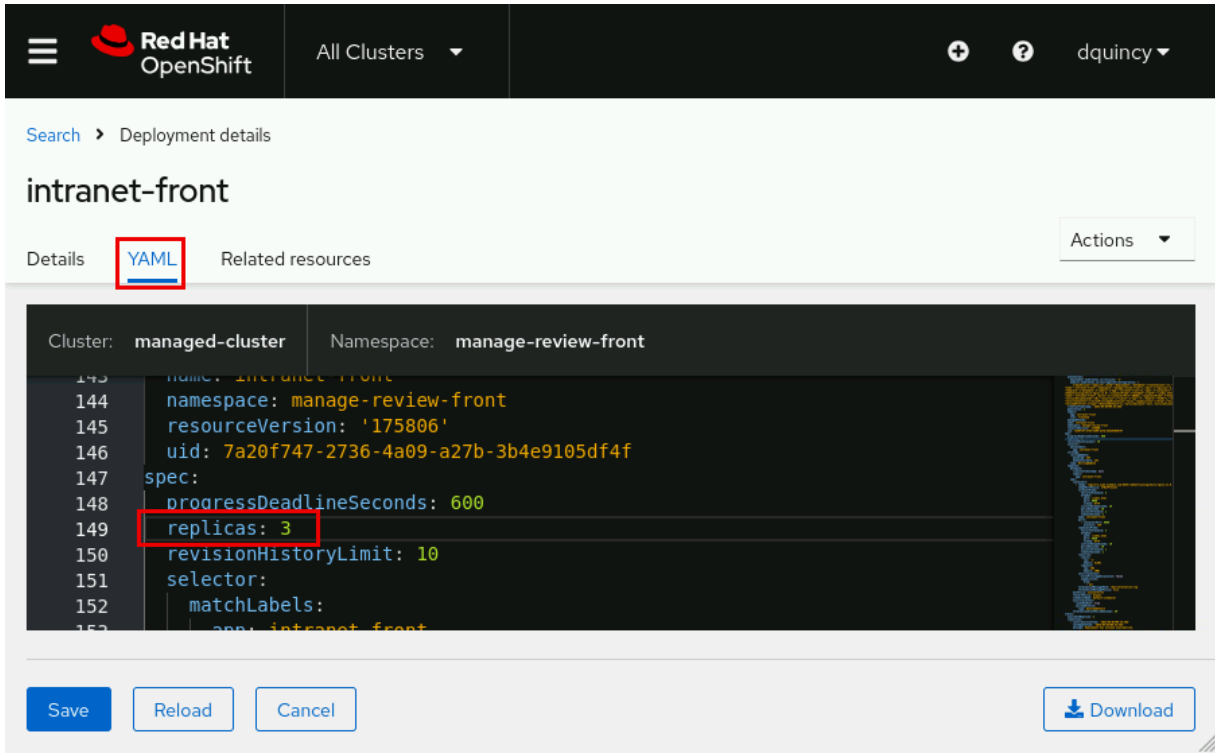
When you log in as the dquincy user, select the **Red Hat Identity Management** provider and use redhat as the password.

When done, log out of the RHACM web console before grading your work.

- 5.1. Log out of the RHACM web console. Click **mshika** in the upper-right corner, and click **Log out**.
- 5.2. From the OpenShift login page, click **Red Hat Identity Management**, and log in as the dquincy user with redhat as the password.
- 5.3. Go to **Search** and type `label:tier=frontend current:<3` in the search field.

The intranet-front deployment in the manage-review-front namespace is displayed.

- 5.4. Click the intranet-front deployment and click the **YAML** tab. In the editor, locate the `.spec.replicas` parameter and set its value to 3. Click **Save**.



- 5.5. To verify your work, rerun the search query by going to **Search** and by typing `label:tier=frontend current:<3` in the search field. The query does not return any result, which means that all the applications with the `tier=frontend` label have at least three replicas.
- 5.6. Log out of the RHACM web console. Click **dquincy** in the upper-right corner, and click **Log out**.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade manage-review
```

Finish

As the student user on the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish manage-review
```

2.10. Summary

In this lesson, you learned about cluster lifecycle management by using the RHACM web console. From the hub cluster, you can create, delete, update, hibernate, resume, or detach managed clusters.

RHACM deploys its web console as a plug-in within the hub cluster OpenShift web console. The RHACM web console provides a single access point to all the management and observation features of a fleet of Kubernetes or OpenShift clusters.

The role-based access control model of RHACM and its default roles help to define a multicluster authentication model for a cluster fleet.

RHACM defines the concept of a *managed cluster set* to create groups of managed clusters. With managed cluster sets, you can perform actions on all the member clusters at once.

RHACM defines the concept of *placement* to define rules to dynamically select a subset of managed clusters in one or more managed cluster sets.

RHACM uses the OpenShift authentication and authorization layers to provide a centralized identity management service for the users who access the managed clusters.

Next Steps

You might explore other learning opportunities on your own, explore further your RHACM installation, or redo some exercises.

A good next step in your learning journey would be to explore the next lessons in the series:

- Lesson 3: *Deploying and Managing Policies for Multiple Clusters with Red Hat Advanced Cluster Management for Kubernetes*
- Lesson 4: *Enabling and Customizing the Observability Stack in Red Hat Advanced Cluster Management for Kubernetes*
- Lesson 5: *Managing the Multicluster Application Lifecycle by Using GitOps Practices and Red Hat Advanced Cluster Management for Kubernetes*
- Lesson 6: *Managing Virtual Machines for Multiple Clusters with Red Hat Advanced Cluster Management for Kubernetes*

Chapter 3.

Deploying and Managing Policies for Multiple Clusters with Red Hat Advanced Cluster Management for Kubernetes

Goal

Deploy and manage policies in a multicluster environment by using Red Hat Advanced Cluster Management for Kubernetes (RHACM) governance.

Sections

- Red Hat Advanced Cluster Management Governance (and Quiz)
- Deploying Policies by Using the RHACM Governance Dashboard (and Guided Exercise)
- Explain and Deploy Policies by Using Policy Generator (and Guided Exercise)
- Deploying and Configuring the Compliance Operator for Multiple Clusters by Using RHACM (and Guided Exercise)
- Integration of Other Policy Engines with RHACM (and Guided Exercise)
- Troubleshoot Governance Issues (and Guided Exercise)

Lab

- Deploying and Managing Policies with RHACM

3.1. Red Hat Advanced Cluster Management Governance

Objectives

- Describe the uses of RHACM governance, and show the architecture of RHACM governance and its components.

Red Hat Advanced Cluster Management for Kubernetes Governance Overview

The multicloud infrastructure must adapt quickly to support internal standards for software engineering, secure engineering, resilience, security, and regulatory compliance by ensuring that security standards are upheld.

Governance refers specifically to the set of rules, controls, policies, and resolutions in place to enforce policy standards. RHACM governance provides a policy-based approach for companies to monitor these standards.

Use Cases for RHACM Governance

From many uses for the RHACM platform, some use cases for RHACM governance are as follows:

Manage compliance of multiple clusters

RHACM manages the lifecycle of multiple clusters to monitor and report whether they comply with a policy. If a cluster is not compliant with a policy, then RHACM remediates the cluster by using policy controllers.

Check for policy violations

Policy management enables the use of policy controllers to check for policy violations. You can set policies for the following actions:

- Enforce security by using built-in policies.
- Install operators to check for policy violations.
- Detect certificates that are close to expiring or that contain DNS names that fail to match specified patterns across your clusters.

Work across different environments

RHACM governance works across a range of environments that run Kubernetes clusters, such as data centers, private clouds, and public clouds.

Components of RHACM Governance

RHACM governance provides a graphical and a command-line interface within the policy-based framework. RHACM governance includes the following components:

RHACM governance dashboard

The RHACM governance dashboard displays a list of policies, cluster violations, and policy violations.

Policy-based governance framework

The governance framework supports policy creation and deployment to the managed clusters based on placement rules.

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

Policy controllers

The policy controllers monitor policies and generate Kubernetes events for violation when the cluster is non-compliant. The violations are propagated to the hub cluster.

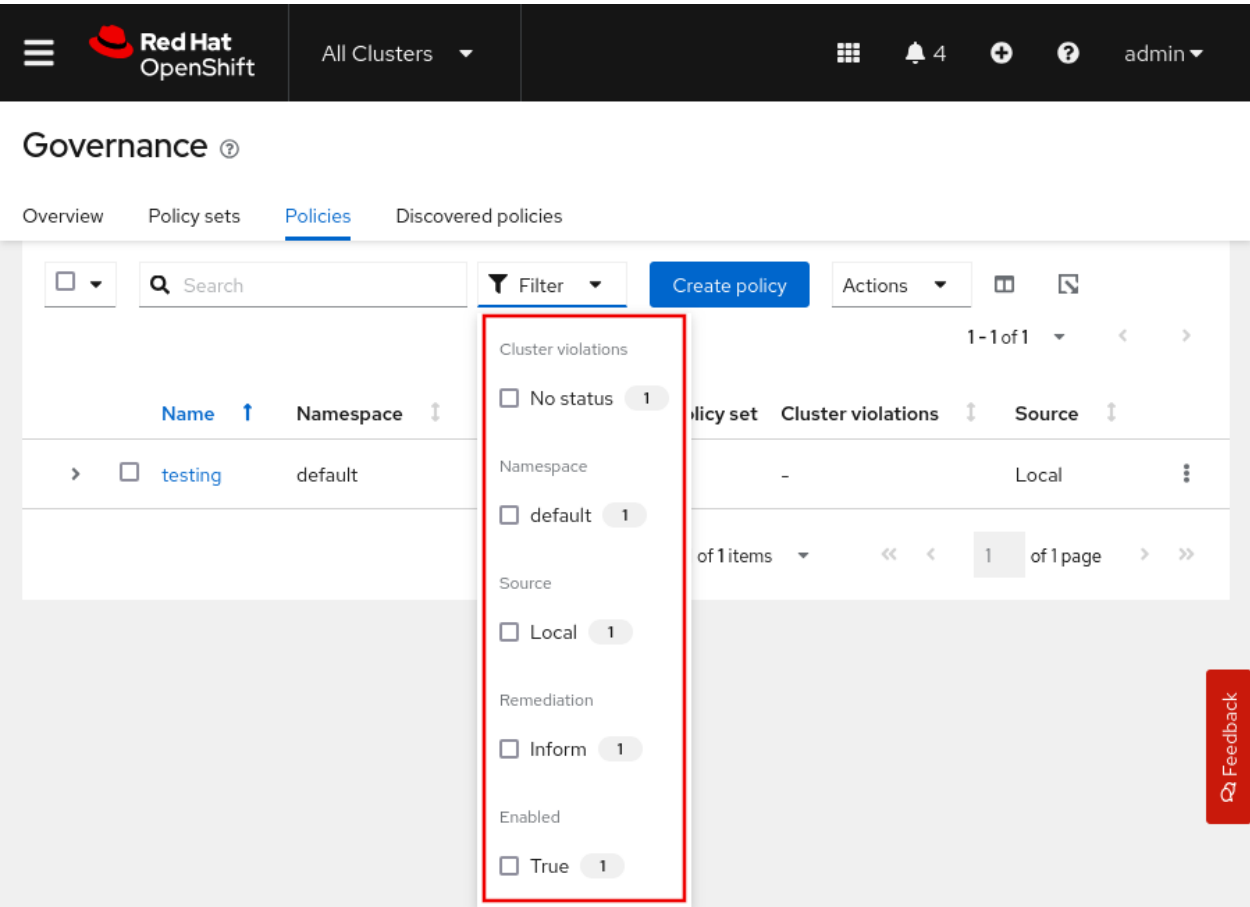
Open source community

RHACM governance supports community contribution. The `stolostron/policy-collection` repository on GitHub has stable and community policy examples. You can contribute and deploy policies by using GitOps.

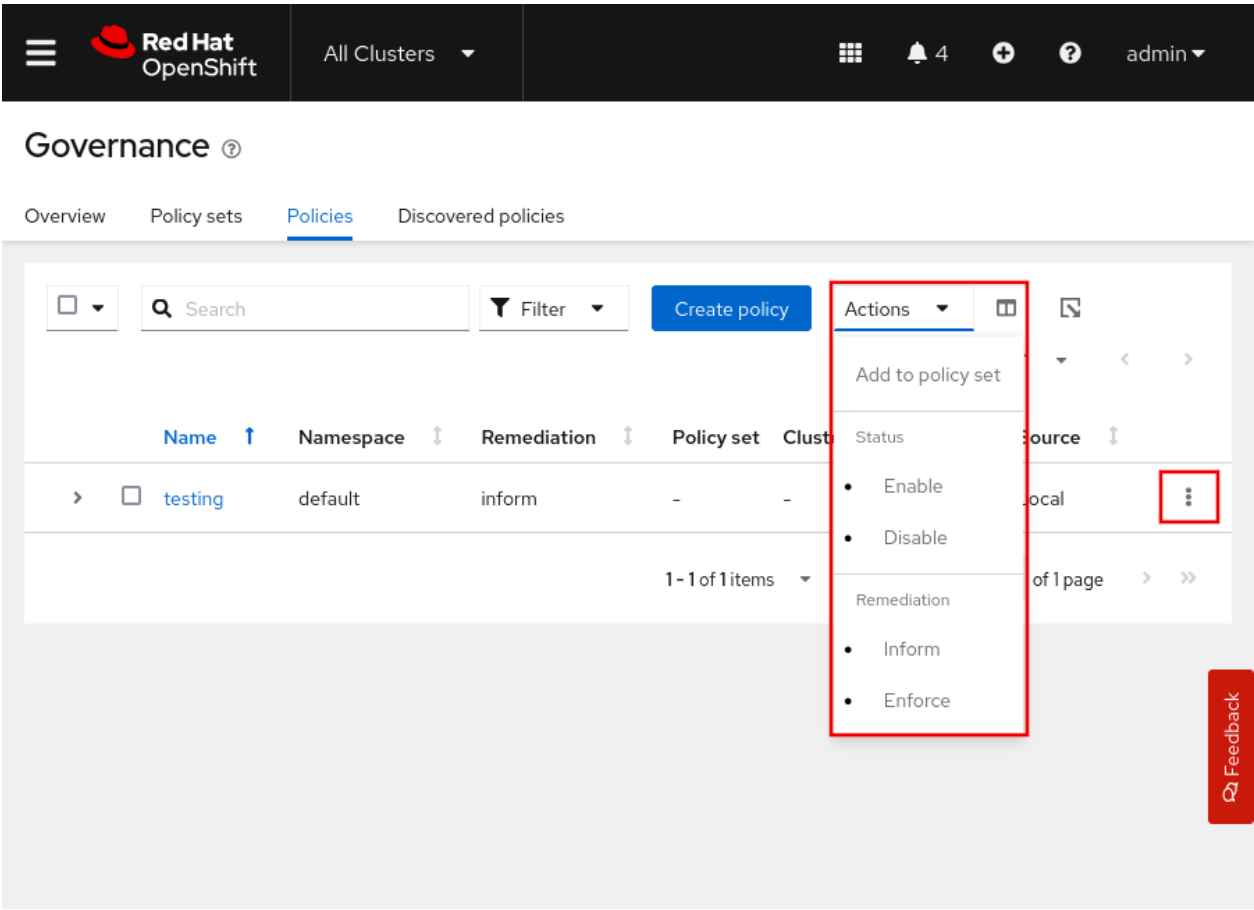
Governance Dashboard

The governance dashboard provides an option to create, deploy, and edit policies with a graphical user interface and a YAML editor.

After the first policy is created, the governance dashboard displays a list of policy violations, policy set violations, clusters, categories, controls, and standards. From the **Policies** tab, you can use the filter list to further narrow the list of policies by cluster violations, namespace, source, remediation, and whether they are enabled.



You can also change the status and remediation of selected policies from the **Actions** button. Every policy has a vertical ellipsis (⋮) menu to perform similar tasks and for policy deletion.



When you click a policy from the policy list, it redirects you to the following tabs:

Details

This tab shows policy and placement details.

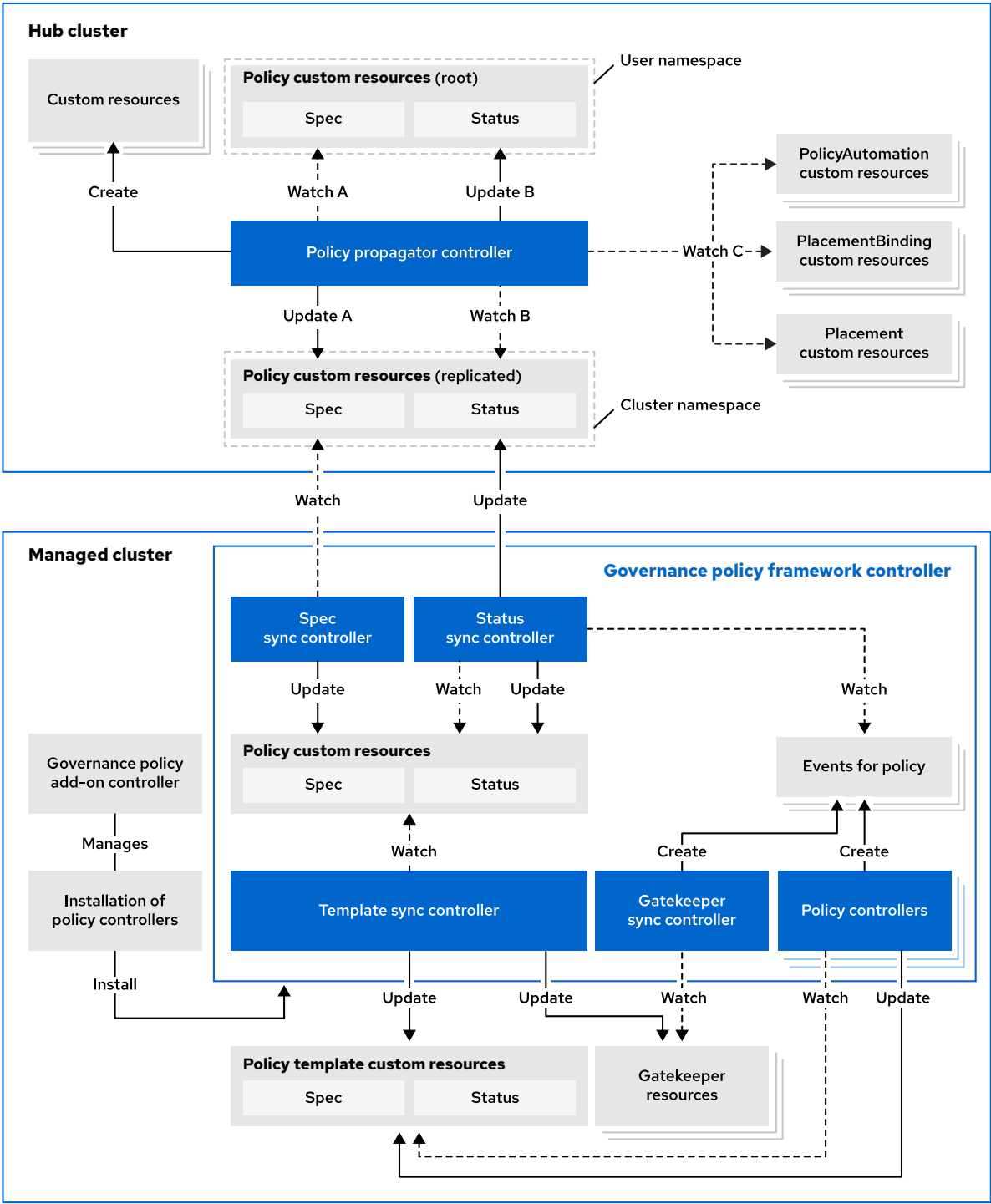
Results

This tab shows a table of all clusters that are associated with the policy.

Policy-based Governance Framework Architecture

The following diagram shows an overview of the RHACM policy-based governance framework architecture:

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright



The following components run on the hub cluster:

The policy propagator controller

This controller generates the replicated policies in the managed cluster namespaces based on the placements that are bound to the root policy. This controller also watches for Placement, PlacementBinding, and PolicyAutomation resources, adds the compliance status from replicated policies to root policy status, and starts automated processes.

The governance policy add-on controller

This controller manages the installation of policy controllers on managed clusters. This controller manages the Governance Policy Framework controller.

The following components run on the managed clusters:

The governance policy framework controller

This controller runs as the `governance-policy-framework` pod on managed clusters, and contains the following controllers:

- The `Spec Sync` controller synchronizes the replicated policy specification with the cluster namespace policies and updates the local policy specification if required.
- The `Status Sync` controller monitors events and changes from policy controllers in the replicated policies, and updates the status policies for both local and hub clusters. The status contains only relevant updates for the current policy, and ignores the past statuses when the policy is re-created.
- The `Template Sync` controller monitors policies in the cluster namespace and updates defined objects in the policy templates for any policy change.
- The `Gatekeeper Sync` controller records Gatekeeper constraint audit results as compliance events in the corresponding policies.

RHACM Policy Overview

RHACM governance uses a policy-based framework. Every `Policy` object requires the `policy-templates`, `Placement`, and `PlacementBinding` templates. A `Policy` object must have at least one `policy-templates` and can have more. The `Placement` template specifies one cluster or set of groups for deploying the policy. As the name implies, the `PlacementBinding` template binds the policy to the `Placement` resource.

Note

You cannot create a policy in the cluster namespace. For example, if the managed cluster is named `managed-cluster` in RHACM, then the reserved namespace is `managed-cluster`. If you create a policy in the `managed-cluster` namespace, then RHACM deletes it.

The policy discovery feature enables you to view the policies that control the behavior of the managed clusters from the **Discovered policies** tab in the **Governance** section from the web console. You can review information about the policies, such as the settings and the status. The **Discovered policies** tab is a useful option for supplementing an existing configuration management strategy with RHACM policies.

Policy YAML File

A policy YAML file has some required and some optional parameters. The optional fields and parameters depend on the policy controller.

```

apiVersion: policy.open-cluster-management.io/v1 ①
kind: Policy ②
metadata:
  name: policy-namespace ③
  annotations: ④
    policy.open-cluster-management.io/standards: NIST SP 800-53 ⑤
    policy.open-cluster-management.io/categories: CM Configuration Management ⑥
    policy.open-cluster-management.io/controls: CM-2 Baseline Configuration ⑦
spec:
  remediationAction: inform ⑧
  disabled: false ⑨
  policy-templates: ⑩
    - objectDefinition:
        apiVersion: policy.open-cluster-management.io/v1
        kind: ConfigurationPolicy
        metadata:
          name: policy-namespace-example
        spec:
          remediationAction: inform _# the policy-template spec.remediationAction is overridden by the
preceding parameter value for spec.remediationAction._
          severity: low
          namespaceSelector:
            exclude: ["kube-*"]
            include: ["default"]
          object-templates:
            - complianceType: musthave
              objectDefinition:
                kind: Namespace _# must have namespace 'prod'_
                apiVersion: v1
                metadata:
                  name: prod
    ---
apiVersion: policy.open-cluster-management.io/v1
kind: PlacementBinding
metadata:
  name: binding-policy-namespace
placementRef:
  name: placement-policy-namespace
  kind: Placement
apiGroup: cluster.open-cluster-management.io
subjects:
  - name: policy-namespace
    kind: Policy
    apiGroup: policy.open-cluster-management.io
    ---
apiVersion: cluster.open-cluster-management.io/v1
kind: Placement
metadata:
  name: placement-policy-namespace
spec:
  predicates:
    - requiredClusterSelector: ⑪
      labelSelector:
        matchExpressions:
          - {key: environment, operator: In, values: ["dev"]}
      tolerations:
        - key: cluster.open-cluster-management.io/unavailable
          operator: Exists
        - key: cluster.open-cluster-management.io/unreachable
          operator: Exists

```

- ① apiVersion: The value for this parameter is policy.open-cluster-management.io/v1.
- ② kind: The value for this parameter is Policy.
- ③ metadata.name: This parameter sets the name of the policy.
- ④ metadata.annotations: This parameter defines the standards for the policy to validate.

- 5 annotations.policy.open-cluster-management.io/standards: This parameter defines the name of the security standard.
- 6 annotations.policy.open-cluster-management.io/categories: This parameter defines the name of the security control category.
- 7 annotations.policy.open-cluster-management.io/controls: This parameter defines the name of the security control.
- 8 spec.remediationAction: The values for this parameter are enforce or inform.
- 9 spec.disabled: The values for this parameter are true or false.
- 10 spec.policy-templates: This parameter is used for policy creation.
- 11 requiredClusterSelector: This parameter specifies a cluster or a set of clusters for policy placement.

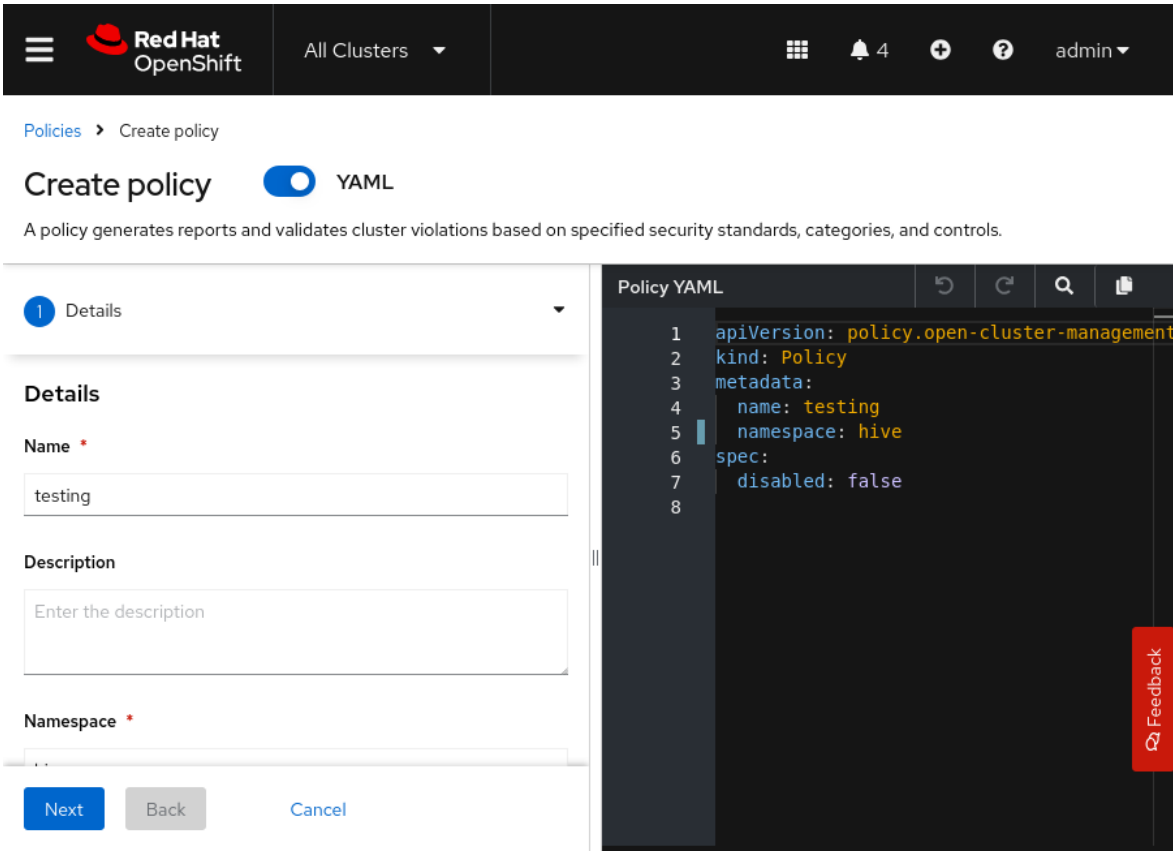
Deploying Policies on a Multicloud Architecture with RHACM

The RHACM governance supports both dashboard and command-line methods to deploy policies.

Creating a policy from the governance dashboard

You can use the governance dashboard to create policies.

The policy creation page provides a graphical interface to define the policy name, namespace, policy template, cluster selector, and remediation selection space to create the policy. The governance dashboard also provides a YAML editor to customize the policy.



Deploying a policy from the command line

You can also create and edit policies from the command line. First, log in to the hub cluster and deploy the policy from the YAML file.

```
[user@demo]$ oc login -u user -p password \
https://api.ocp4.example.com:6443
Login successful.

...output omitted...

[user@demo]$ oc create -f policy-example.yaml
policy.policy.open-cluster-management.io/policy-example created
placementbinding.policy.open-cluster-management.io/binding-policy-example created
placement.cluster.open-cluster-management.io/placement-policy-example created
```

RHACM Policy Controllers Overview

The policy controllers monitor whether your cluster complies with a policy. The policy controller also reports the policy status, which is on the RHACM governance dashboard.

The supported policy controllers are described in the following sections.

Kubernetes Configuration Policy Controller

This policy controller manages, configures, and monitors Kubernetes resources. The configuration policy controller monitors policy violations and remediates them by using the *enforce* feature. The configuration policy controller can also install, configure, or retrieve reports from other operators such as a compliance operator.

Certificate Policy Controller

This policy controller monitors certificates for the selected namespaces. The certificate policy detects when a certificate is about to expire in the default namespace. You can customize the certificate policy by adding or changing additional parameters, as described in the following table:

Parameter	Description
severity	Reports when the policy is non-compliant.
remediationAction	You can set the parameter to inform.
minimumDuration	The minimum duration for the certificate expiration. For example, if the parameter value is set to 300h, and any certificates have an expiration age of fewer than 300 hours, then the policy displays the Not-compliant status.
minimumCADuration	The minimum duration for the signing certificate expiration.
maximumDuration	The maximum duration for the certificate expiration. For example, if the parameter value is set to 300h, and any certificates have an expiration age of more than 300 hours, then the policy displays the Not-compliant status.
maximumCADuration	The maximum duration for the signing certificate expiration.

Parameter	Description
allowedSANPattern	You can specify an expression with this parameter. It compares patterns against DNS names. That expression must be present in every SAN entry in the certificates.
disallowedSANPattern	You can specify an expression with this parameter. It compares patterns against DNS names. That expression must not be present in any SAN entry in the certificates.

The certificate policy also has a namespace selector to exclude or include namespaces. The sample certificate policy template shows that the included namespace is default, and the excluded namespace is kube-*. This policy template checks that the minimum certificate duration is defined as 300h.

```
policy-templates:
- objectDefinition:
  apiVersion: policy.open-cluster-management.io/v1
  kind: CertificatePolicy
  metadata:
    name: policy-certificate-example
  spec:
    namespaceSelector:
      include:
        - default
      exclude:
        - kube-*
    remediationAction: inform
    severity: low
    minimumDuration: 300h
```

Policy Set Controller

You can use this controller to place together the policies that are in the same namespace, to deploy them at the same time to a cluster. The policy set controller checks for violations only in clusters that include the policy set placement. A PolicySet resource requires PlacementBinding to bind the PolicySet and Placement resources.

This policy set example assigns a policy to pods and a policy to namespaces that check for *Payment Card Industry* (PCI) compliance:

```
apiVersion: policy.open-cluster-management.io/v1beta1
kind: PolicySet
metadata:
  name: pci
  namespace: default
spec:
  description: Policies for PCI compliance
  policies: ❶
  - policy-pod
  - policy-namespace
status:
  compliant: NonCompliant
  placement:
  - placementBinding: binding1
    placement: placement1
    policySet: policyset-ps
```

- ❶ Policies that are assigned to the policy set.

Operator Policy Controller

You can use this controller to install and monitor the Operator Lifecycle Manager (OLM) operators in your clusters. The OLM helps users to install and update operators in a cluster. Operators that the OLM manages are also known as *add-on operators*, in contrast with cluster operators that implement platform services. The OLM must be available to use the operator policy controller.

You can use an `OperatorPolicy` policy template in a policy definition to install the OLM-managed operators on managed clusters.

This policy example installs the Quay operator through the Red Hat operator catalog:

```
apiVersion: policy.open-cluster-management.io/v1
kind: Policy ❶
metadata:
  name: install-quay
  namespace: open-cluster-management-global-set
spec:
  disabled: false
  policy-templates:
  - objectDefinition:
      apiVersion: policy.open-cluster-management.io/v1beta1
      kind: OperatorPolicy ❷
      metadata:
        name: install-quay
      spec:
        remediationAction: enforce ❸
        severity: critical
        complianceType: musthave ❹
        upgradeApproval: None ❺
        subscription: ❻
          channel: stable-3.11
          name: quay-operator
          source: redhat-operators
          sourceNamespace: openshift-marketplace ❼
```

- ❶ The `OperatorPolicy` resource must be created inside the `Policy` resource definition.
- ❷ Definition of the `OperatorPolicy` kind.
- ❸ If the value is set to `enforce`, then the controller creates resources on the targeted managed cluster to communicate to OLM to install the operator and to approve updates based on the specified versions in the policy.
- ❹ If the value is set to `musthave`, then the policy is compliant when the operator is found.
- ❺ The policy approves the version upgrades.

- 6 In this section, you configure the operator subscription.
- 7 The Red Hat operator catalog namespace.

References

Governance Policy Framework

For more information about the governance architecture, refer to the *Governance Policy Framework Architecture* section in the *Hub Cluster Policy Framework* guide in the *Red Hat Advanced Cluster Management for Kubernetes* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/governance/index#governance-architecture

For more information about the policy YAML structure, refer to the *Policy YAML Structure* section in the *Hub Cluster Policy Components* guide in the *Red Hat Advanced Cluster Management for Kubernetes* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/governance/index#policy-yaml-structure

For more information about the policy controllers, refer to the *Policy Set Controller* section in the *Governance* guide in the *Red Hat Advanced Cluster Management for Kubernetes* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/governance/index#policy-set-controller

3.2. Quiz

Red Hat Advanced Cluster Management Governance

Choose the correct answers to the following questions:

1. RHACM governance provides a policy-based approach for companies to monitor and enforce policy standards across multiple clusters. Which component of RHACM governance is responsible for displaying policy violations that affect clusters?
 - ☐ a. Policy propagator controller
 - ☐ b. RHACM governance dashboard
 - ☐ c. Policy controllers
 - ☐ d. Governance policy add-on controller
2. You are a system administrator who is responsible for managing multiple Kubernetes clusters. You must ensure that all clusters comply with your organization's security policies. You decide to use RHACM to manage these policies. You want to monitor and report compliance status and automatically remediate any noncompliant clusters. What feature of RHACM would you use to achieve these objectives?
 - ☐ a. Policy controllers
 - ☐ b. Policy-based governance framework
 - ☐ c. Governance dashboard
 - ☐ d. Policy discovery feature
3. What is the role of the policy propagator controller in RHACM?
 - ☐ a. It manages the installation of policy controllers on managed clusters.
 - ☐ b. It synchronizes the replicated policy specification with cluster namespace policies.
 - ☐ c. It records Gatekeeper constraint audit results as compliance events.
 - ☐ d. It generates replicated policies in managed cluster namespaces based on placements.
4. What does the Kubernetes Configuration Policy Controller do?
 - ☐ a. Installs OLM operators.
 - ☐ b. Deploys policies to multiple clusters.
 - ☐ c. Monitors certificate expiration.
 - ☐ d. Monitors policy violations and remediates them.
5. What is required for a PolicySet resource to function?
 - ☐ a. CertificatePolicy
 - ☐ b. PlacementBinding
 - ☐ c. ConfigurationPolicy
 - ☐ d. OperatorPolicy

6. You are deploying a set of policies that must be applied together for PCI compliance across your clusters. You want to ensure that these policies are checked for violations only in the clusters where they apply. Which controller would you use to group and deploy these policies?
- ☐ a. Certificate Policy Controller
 - ☐ b. Policy Set Controller
 - ☐ c. Kubernetes Configuration Policy Controller
 - ☐ d. Operator Policy Controller

Solution

1. RHACM governance provides a policy-based approach for companies to monitor and enforce policy standards across multiple clusters. Which component of RHACM governance is responsible for displaying policy violations that affect clusters?
 - ☐ a. Policy propagator controller
 - ☒ b. **RHACM governance dashboard**
 - ☐ c. Policy controllers
 - ☐ d. Governance policy add-on controller
2. You are a system administrator who is responsible for managing multiple Kubernetes clusters. You must ensure that all clusters comply with your organization's security policies. You decide to use RHACM to manage these policies. You want to monitor and report compliance status and automatically remediate any noncompliant clusters. What feature of RHACM would you use to achieve these objectives?
 - ☒ a. **Policy controllers**
 - ☐ b. Policy-based governance framework
 - ☐ c. Governance dashboard
 - ☐ d. Policy discovery feature
3. What is the role of the policy propagator controller in RHACM?
 - ☐ a. It manages the installation of policy controllers on managed clusters.
 - ☐ b. It synchronizes the replicated policy specification with cluster namespace policies.
 - ☐ c. It records Gatekeeper constraint audit results as compliance events.
 - ☒ d. **It generates replicated policies in managed cluster namespaces based on placements.**
4. What does the Kubernetes Configuration Policy Controller do?
 - ☐ a. Installs OLM operators.
 - ☐ b. Deploys policies to multiple clusters.
 - ☐ c. Monitors certificate expiration.
 - ☒ d. **Monitors policy violations and remediates them.**
5. What is required for a PolicySet resource to function?
 - ☐ a. CertificatePolicy
 - ☒ b. **PlacementBinding**
 - ☐ c. ConfigurationPolicy
 - ☐ d. OperatorPolicy

6. You are deploying a set of policies that must be applied together for PCI compliance across your clusters. You want to ensure that these policies are checked for violations only in the clusters where they apply. Which controller would you use to group and deploy these policies?

- ☐ a. Certificate Policy Controller
- ☒ b. **Policy Set Controller**
- ☐ c. Kubernetes Configuration Policy Controller
- ☐ d. Operator Policy Controller

3.3. Deploying Policies by Using the RHACM Governance Dashboard

Objectives

- Deploy policies on multiple clusters by using the RHACM governance dashboard.

RHACM Policy

A policy in RHACM is a definition to enforce security compliance or to notify about compliance violations in your configuration. Policies act as rule sets that monitor and manage the state of your Kubernetes resources.

RHACM policies are composed of policy template definitions, to specify the intended state of various Kubernetes objects, such as namespaces, pods, roles, or the installation of operators. To deploy these policies, the following resources are essential:

- A `Placement` resource, which defines which clusters the policy applies to. You can select clusters based on criteria, such as labels.
- A `PlacementBinding` resource, which enables binding your policy to the chosen `Placement` resource.

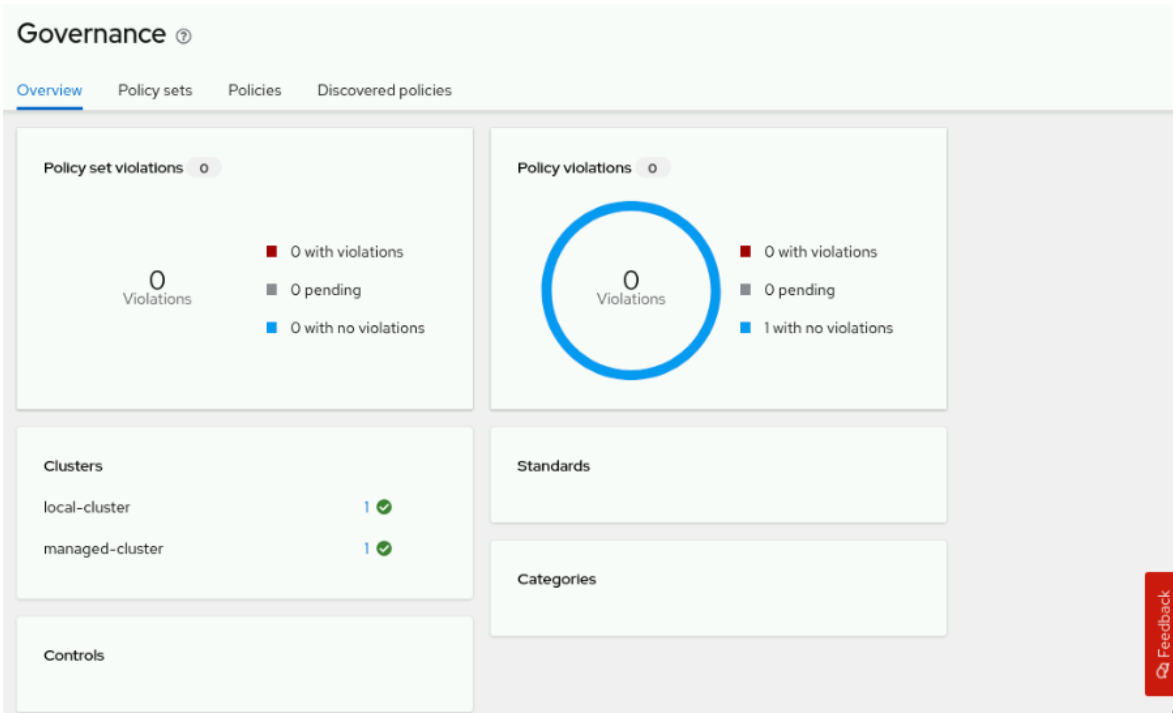
The policy framework within RHACM handles the entire lifecycle: from distributing the policies to your managed clusters, to aggregating their compliance status back to the central hub cluster.

Governance Dashboard

The Governance Dashboard in the RHACM console is your central interface for managing security policies and policy violations. The dashboard is organized into the following tabs:

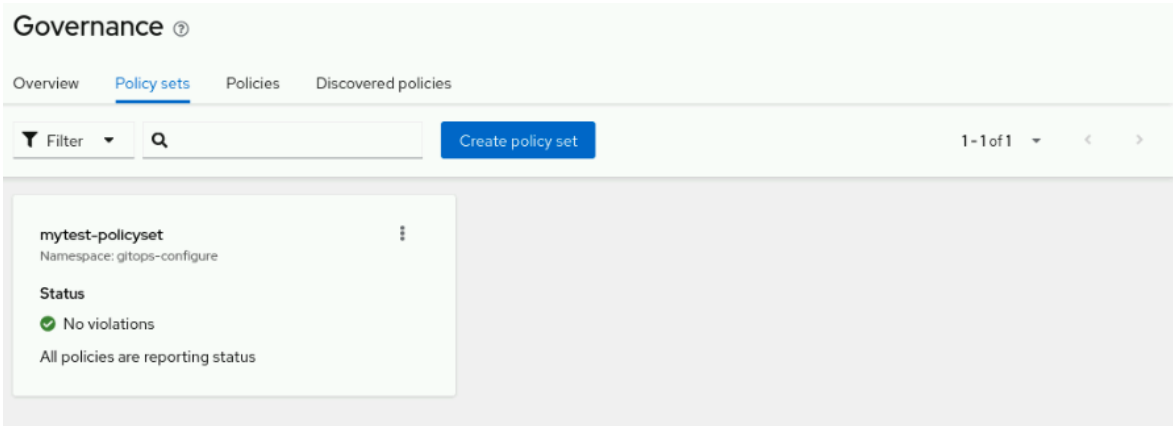
Overview Tab

This tab provides a summary of your cloud governance and risk details, for a high-level view of your compliance posture. The summary card displays information about policy set and policy violations, clusters, categories, controls, and standards.



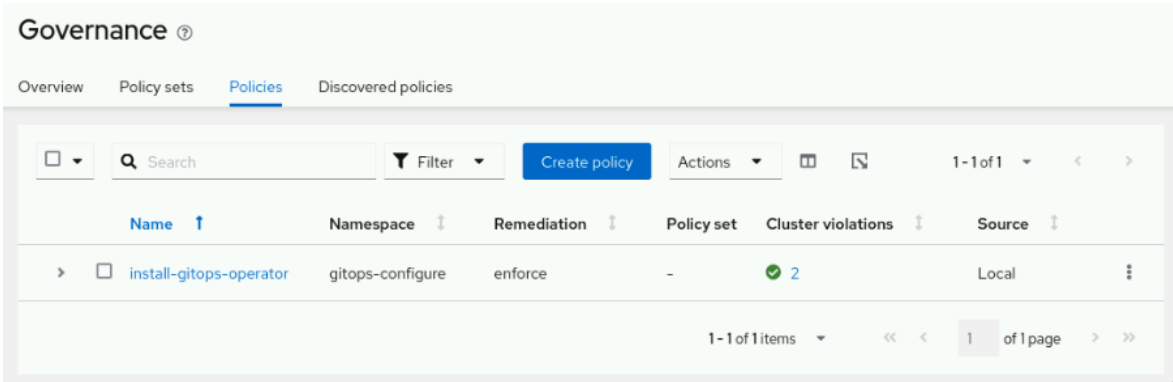
Policy Sets Tab

In this tab, you can create and manage hub cluster policy sets. A *policy set* is a logical grouping of policies in the same namespace, to help manage related policies together.



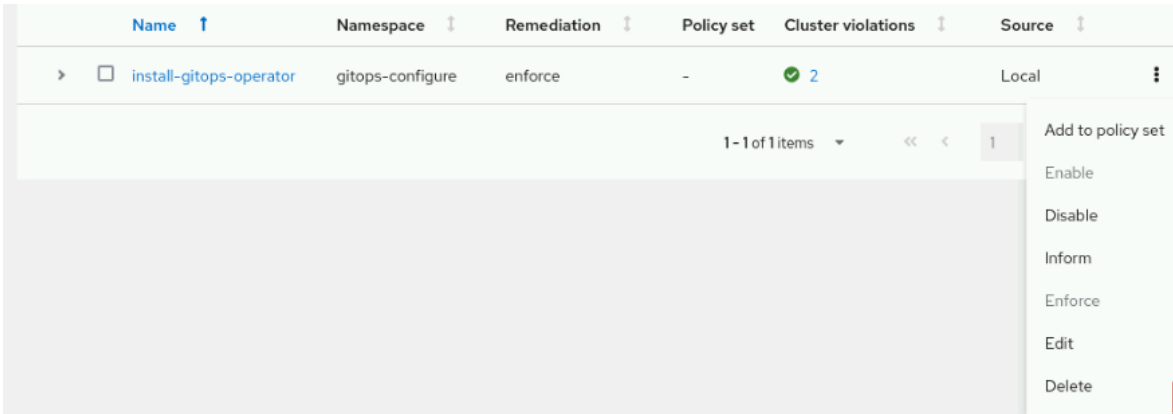
Policies Tab

This tab, where you spend much of your time, provides a table list of all your security policies, and displays details such as the name, namespace, and cluster violations.

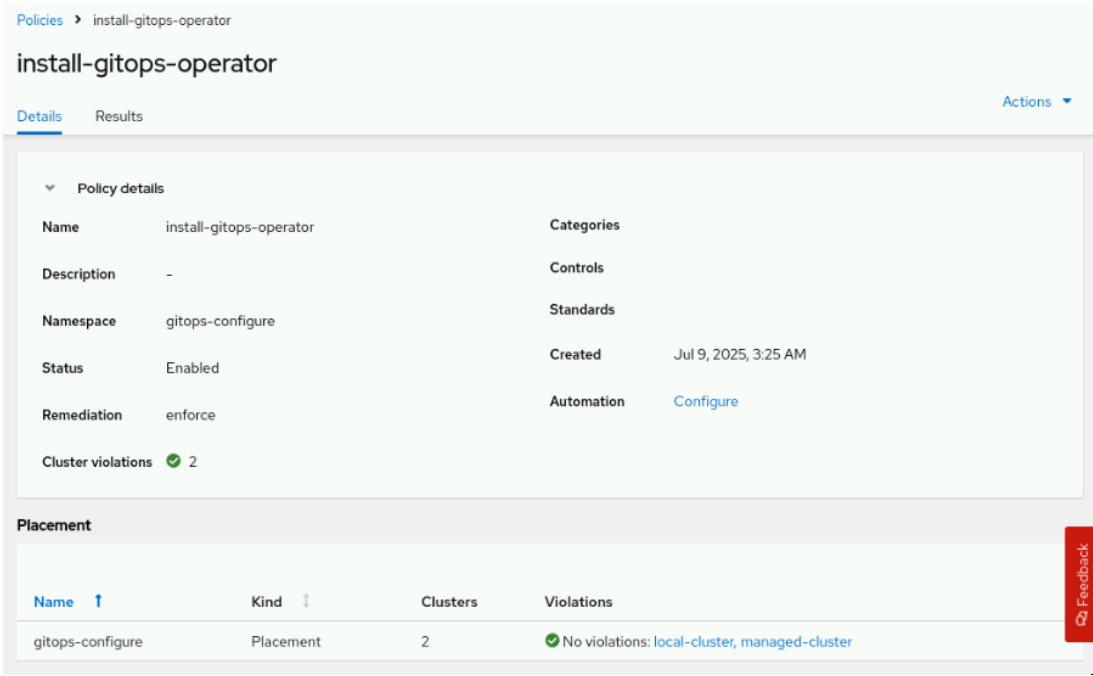


From the Policies tab, you can do the following operations:

- You can edit, enable or disable, set remediation to inform or enforce, or remove a policy by clicking the vertical ellipsis icon (⋮) next to a policy.



- You can customize your table view by reordering columns or applying filters to find specific policies. When you select an individual policy from the table, you can explore its details through the dedicated tabs:
 - The **Details** tab provides an overview of the policy and its associated placement.



- The **Results** tab lists all clusters that are associated with that policy.

Policies > install-gitops-operator

install-gitops-operator

DetailsResultsActions

Clusters

Find clusters

1 - 8 of 8

Cluster	Violations	Template	Message	Remediation	Last report	History
managed-cluster	No violations	argocd-configuration	notification - argocd [openshift-gitops] found as specified in namespace openshift-gitops - View details	enforce	Jul 10, 2025, 5:58 AM	View history
managed-cluster	No violations	clusterrolebinding	notification - clusterrolebindings [argo-admin] found as specified - View details	enforce	Jul 10, 2025, 5:58 AM	View history
managed-cluster	No violations	classroom-ca-bundle	notification - configmaps [cluster-root-ca-bundle] found as specified in namespace openshift-gitops - View details	enforce	Jul 10, 2025, 5:58 AM	View history
managed-cluster	No violations	install-gitops-operator	the policy spec is valid, the OperatorGroup matches what is required by the policy, the Subscription matches what is required by the policy, no InstallPlans requiring approval were found, ClusterServiceVersion (openshift-gitops-operator.v1.16.2) - install strategy completed with no errors, there are CRDs present for the	enforce	Jul 10, 2025, 5:58 AM	View history

Feedback

From this page, you can click **View details** to inspect template details, the policy's YAML file, and related resources.

Policies

install-gitops-operator

install-gitops-operator

DetailsResultsActions

Clusters

Find clusters

1 - 8 of 8

Cluster	Violations	Template	Message	Remediation	Last report	History
managed-cluster	No violations	argocd-configuration	notification - argocdcs [openshift-gitops] found as specified in namespace openshift-gitops - View details	enforce	Jul 10, 2025, 5:58 AM	View history
managed-cluster	No violations	clusterrolebinding	notification - clusterrolebindings [argo-admin] found as specified - View details	enforce	Jul 10, 2025, 5:58 AM	View history
managed-cluster	No violations	classroom-ca-bundle	notification - configmaps [cluster-root-ca-bundle] found as specified in namespace openshift-gitops - View details	enforce	Jul 10, 2025, 5:58 AM	View history
managed-cluster	No violations	install-gitops-operator	the policy spec is valid, the OperatorGroup matches what is required by the policy, the Subscription matches what is required by the policy, no InstallPlans requiring approval were found, ClusterServiceVersion (openshift-gitops-operator.v1.16.2) - install strategy completed with no errors, there are CRDs present for the	enforce	Jul 10, 2025, 5:58 AM	View history

Feedback

Policies

install-gitops-operator

argocd-configuration

argocd-configuration

No violations

DetailsYAML

ConfigurationPolicy details

Name	argocd-configuration	Kind	ConfigurationPolicy
Engine	Open Cluster Management	API version	policy.open-cluster-management.io/v1
Cluster	managed-cluster		

Related resources

Search

1 - 1 of 1

Name	Namespace	Kind	API version	Violations	Reason
openshift-gitops	openshift-gitops	ArgoCD	argoproj.io/v1beta1	No violations	Resource found as expected

1 - 1 of 1 items1 of 1 page

Feedback

You can click **View history** to view past violation messages and timestamps.

Policies

install-gitops-operator

install-gitops-operator

DetailsResultsActions

Clusters

Find clusters

1 - 8 of 8

Cluster	Violations	Template	Message	Remediation	Last report	History
managed-cluster	No violations	argocd-configuration	notification - argocds [openshift-gitops] found as specified in namespace openshift-gitops - View details	enforce	Jul 10, 2025, 5:58 AM	View history
managed-cluster	No violations	clusterrolebinding	notification - clusterrolebindings [argo-admin] found as specified - View details	enforce	Jul 10, 2025, 5:58 AM	View history
managed-cluster	No violations	classroom-ca-bundle	notification - configmaps [cluster-root-ca-bundle] found as specified in namespace openshift-gitops - View details	enforce	Jul 10, 2025, 5:58 AM	View history
managed-cluster	No violations	install-gitops-operator	the policy spec is valid, the OperatorGroup matches what is required by the policy, the Subscription matches what is required by the policy, no InstallPlans requiring approval were found, ClusterServiceVersion (openshift-gitops-operator.v1.16.2) - install strategy completed with no errors, there are CRDs present for the	enforce	Jul 10, 2025, 5:58 AM	View history

Policies

install-gitops-operator

History

History

managed-cluster

Template: argocd-configuration

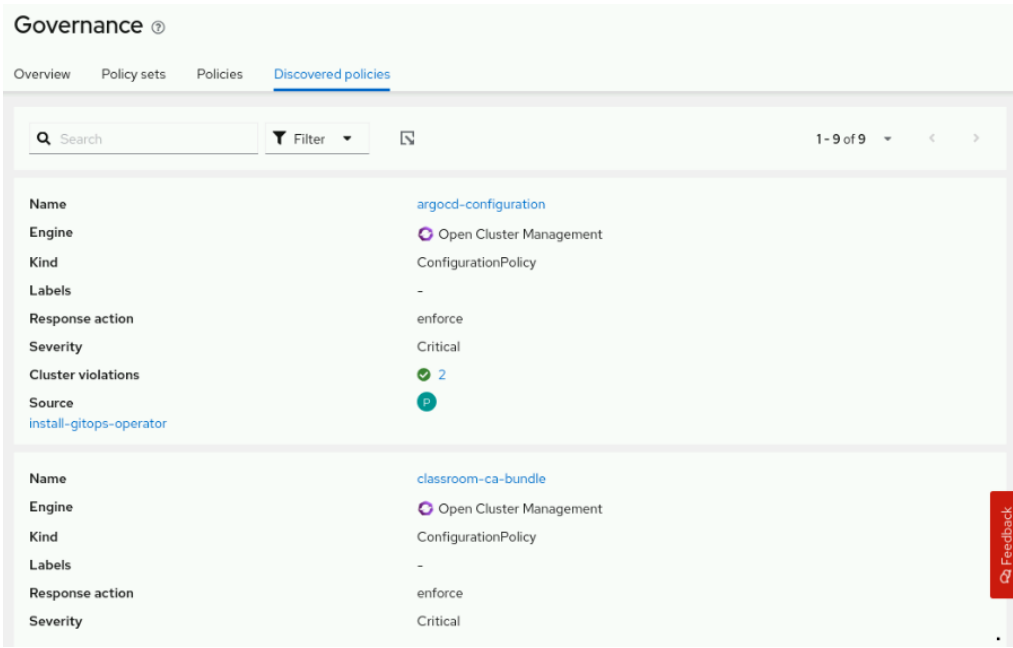
Search

1 - 7 of 7

Violations	Message	Last report
No violations	notification - argocds [openshift-gitops] found as specified in namespace openshift-gitops	Jul 10, 2025, 5:58 AM
No violations	notification - argocds [openshift-gitops] found as specified in namespace openshift-gitops	Jul 10, 2025, 5:58 AM
Pending	Dependencies were not satisfied: 1 is still pending (ConfigurationPolicy clusterrolebinding)	Jul 10, 2025, 5:58 AM
No violations	notification - argocds [openshift-gitops] found as specified in namespace openshift-gitops	Jul 9, 2025, 3:30 AM
No violations	notification - argocds [openshift-gitops] was updated successfully in namespace openshift-gitops	Jul 9, 2025, 3:30 AM
Violations	violation - argocds [openshift-gitops] found but not as specified in namespace openshift-gitops	Jul 9, 2025, 3:30 AM

Discovered Policies Tab

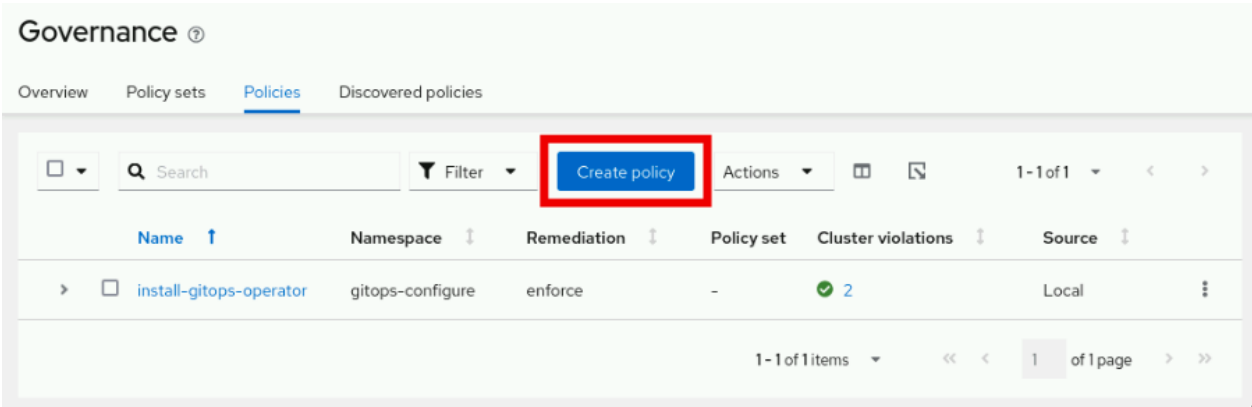
This tab shows all the policies that control the behavior of the managed clusters. RHACM discovers the policies and shows them here.



Creating a Policy from the Governance Dashboard

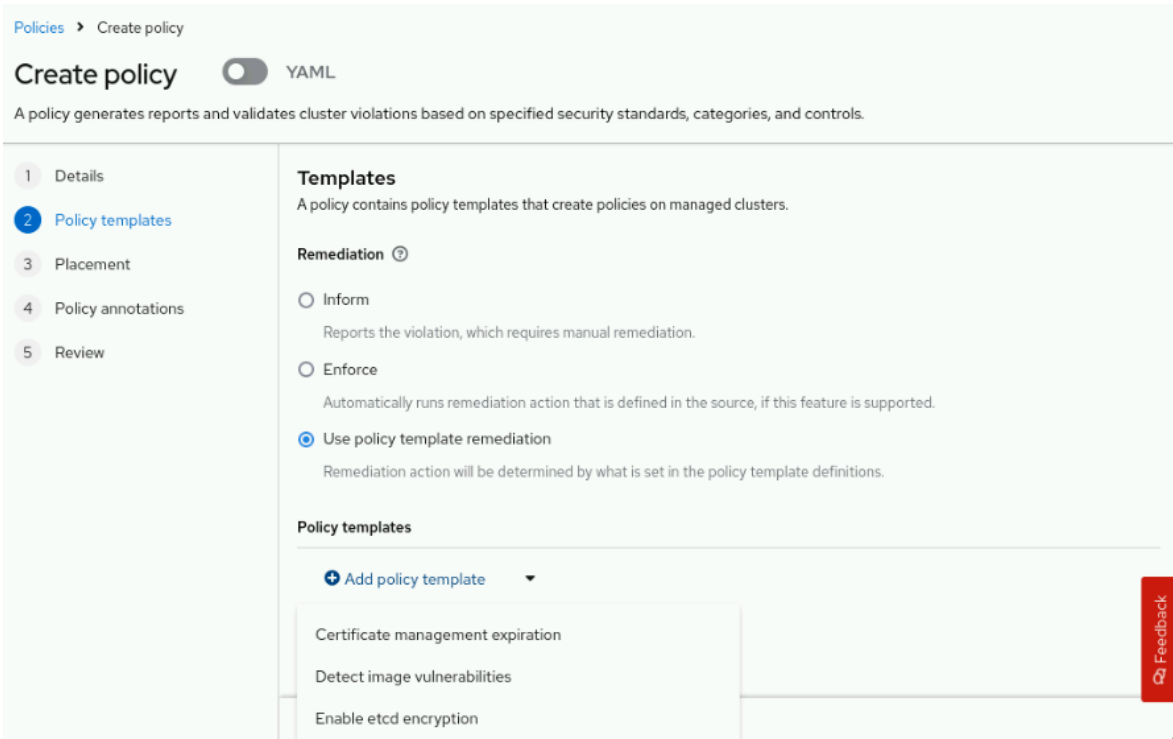
You can create policies directly from the CLI by using YAML files, or you can use the Governance dashboard. The dashboard includes a YAML editor.

To create a policy, click the **Create policy** button.

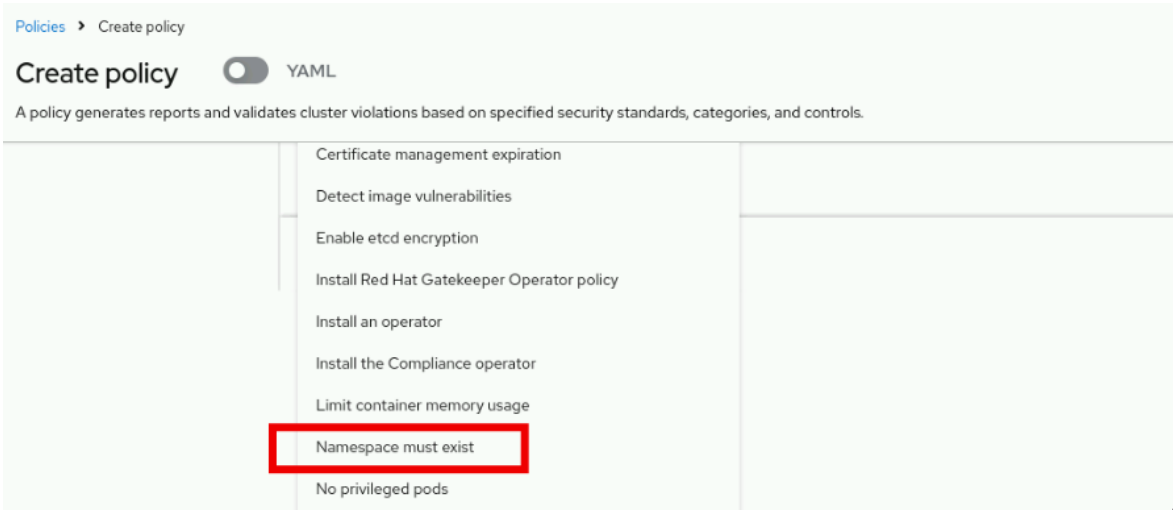


- You specify the policy name, the description, and the namespace details in the **Details** tab.
- In the **Policy templates** tab, you select an existing policy template and edit it as needed.

The **Add policy template** button shows existing policy templates. Consult the references section for a list of predefined policy templates.

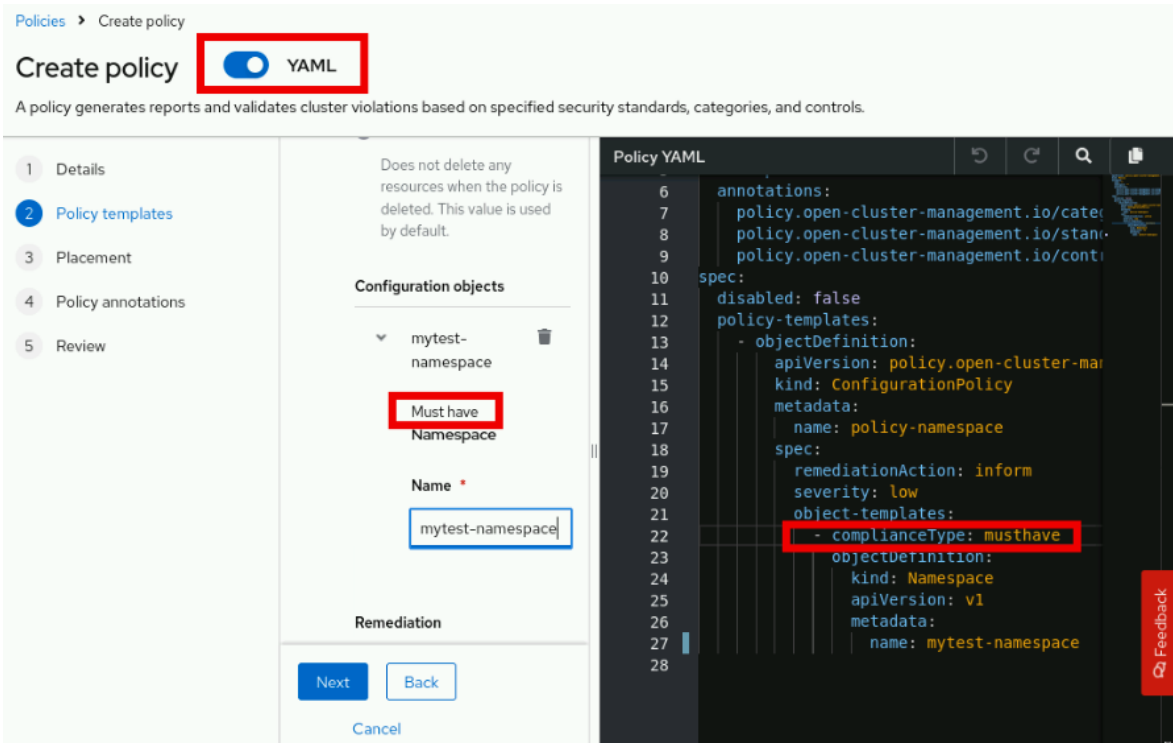


The following screenshot shows the Namespace must exist policy template. In YAML mode, you can edit settings that might not be editable in the dashboard UI.

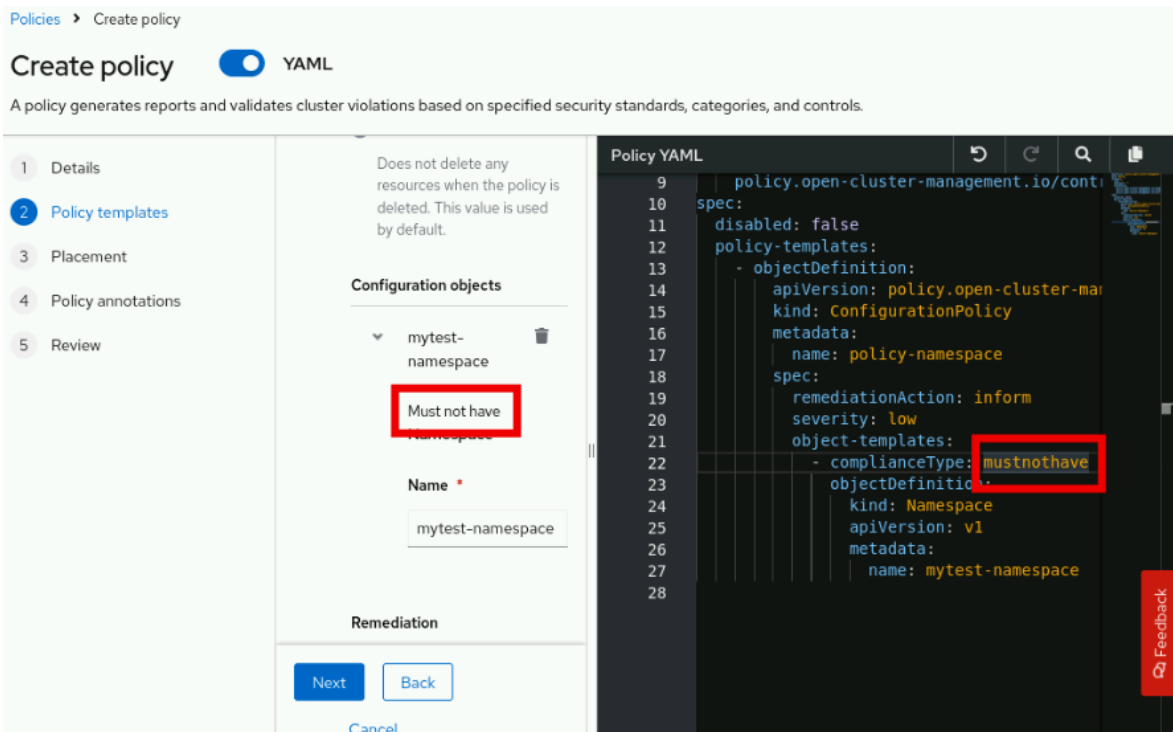


The following two screenshots show that you can change the musthave condition by using the YAML editor. In this case, the condition is changed to mustnothave.

Before:

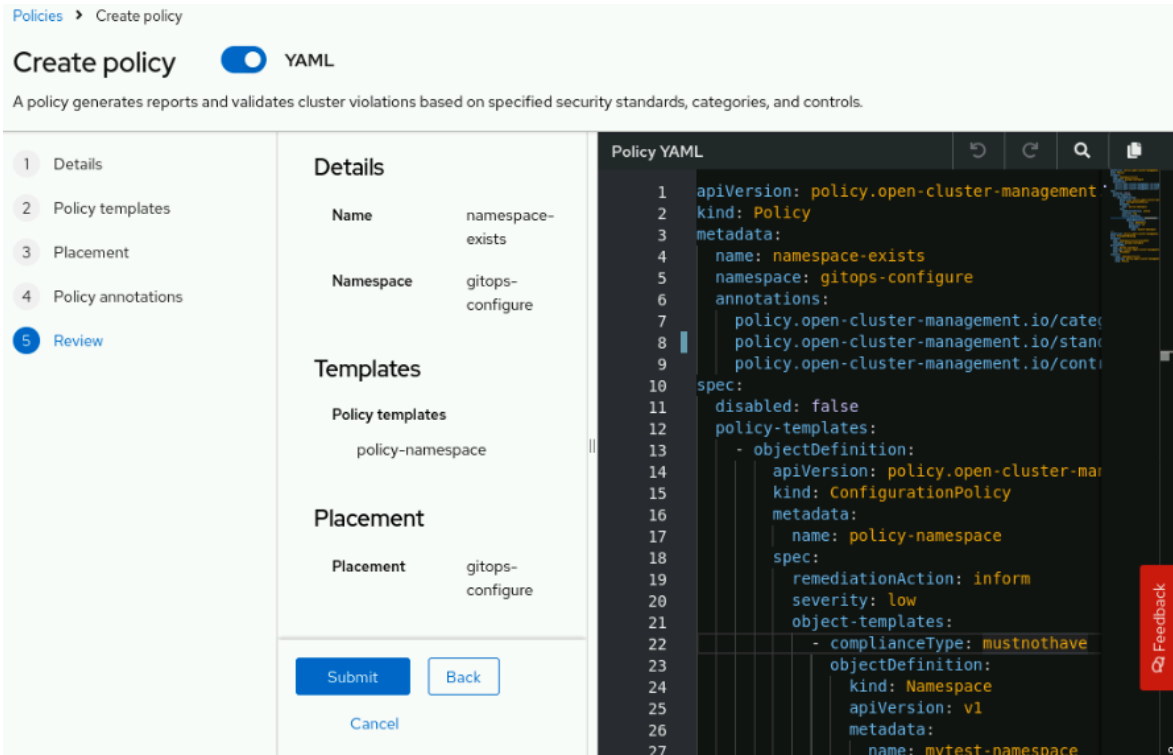


After:



- In the **Placement** tab, you can select clusters from cluster sets. This option enables RHACM to deploy policies across multiple clusters.
- You can add standards, categories and controls as annotations to the policy in the **Policy annotations** tab.
- Finally, you review your entered details before you create your policy.

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright



Policy Cleanup

Cleaning up policies and resources is essential to ensure a streamlined environment. Removing unused policies prevents interference with ongoing operations and ensures clarity in governance. For this process, you can use either the dashboard or the CLI.

To delete a policy from the dashboard, follow these steps:

- Go to Policies, select the policy, and click **Delete**.
- Confirm deletion of the associated PlacementBinding resource and the PlacementRule resource in the Placement section.

To delete a policy from the CLI, you delete the policy, its binding, and its placement rule:

```
[user@host ~]$ oc delete policy my-policy -n my-namespace
...output omitted...
[user@host ~]$ oc delete placementbinding my-placement-binding -n my-namespace
...output omitted...
[user@host ~]$ oc delete placementrule my-placement-rule -n my-namespace
...output omitted...
```

Resource Cleanup Control

Beyond deleting the policy definition, you might need to manage the lifecycle of the Kubernetes resources that the policy created or managed. By setting the pruneObjectBehavior parameter, you can specify how to clean up related objects when the configuration policy or its parent policy is deleted. It is configured within the spec section in the policy's YAML file.

The pruneObjectBehavior parameter has the following possible values:

- DeleteIfCreated: This value cleans up any resources that the policy originally created, which is useful for policies that provision new resources.

- **DeleteAll**: This option cleans up all resources that the policy managed, regardless of whether the policy originally created them.

Warning

Use this option with caution, because it can remove resources that the policy did not initially deploy.

- **None**: With this default value, no related resources are deleted when the policy is removed. You must manually clean up any associated resources.

You can set this value directly in your policy's YAML file when creating the policy via the CLI. If you are using the console, you can select this behavior in the *Prune Object Behavior* section during the *Policy templates* step of policy creation.

Note

In the console, you can set the parameter only for configuration policies. However, you can set the parameter for other policy types, such as the operator policy or the certificate policy, by turning on the YAML mode and adding the parameter to the spec section of the policy.

The following important considerations apply when you use the `pruneObjectBehavior` parameter:

- If the `pruneObjectBehavior` parameter is defined for a policy that installed an operator, then additional cleanup might be required to fully uninstall the operator, such as by deleting the `ClusterServiceVersion` object.
- If the `config-policy-addon` resource on a managed cluster is disabled, then the `pruneObjectBehavior` setting is ignored.

References

For more information, refer to the *Policy Deployment* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Governance* documentation at https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/governance/index#policy-deployment

For more information about predefined policies, refer to the *Table of Sample Configuration Policies* table in the *Supported Red Hat Advanced Cluster Management for Kubernetes policies* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Governance* documentation at https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/governance/index#configuration-policy-sample-table

For more information about controlled resource cleanup, refer to the *Cleaning up Resources That Are Created by Policies* section of the *Managing Security Policies* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Governance* documentation at https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/governance/index#cleaning-up-resources-from-policies

3.4. Guided Exercise

Deploy Policies by Using the RHACM Governance Dashboard

Create a policy in all the clusters to verify the expiration of the certificates in the `openshift-console` and `openshift-ingress` namespaces. Then, analyze the status of the policy, apply a fix, and verify the final status after the remediation.

Outcomes

- Deploy a policy to verify certificate expiration in the `openshift-console` and `openshift-ingress` namespaces.
- Analyze the status of the new policy, apply a fix, and verify the final status after remediation.

Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, which ensures that all required resources are available.

```
[student@workstation ~]$ lab install do0013l ; lab start policies-deploy
```

Instructions

1. Log in to the OpenShift hub cluster and create the `policies-deploy` project.
 - 1.1. In a terminal window, log in to the RHOC cluster as the `admin` user with `redhatocp` as the password.

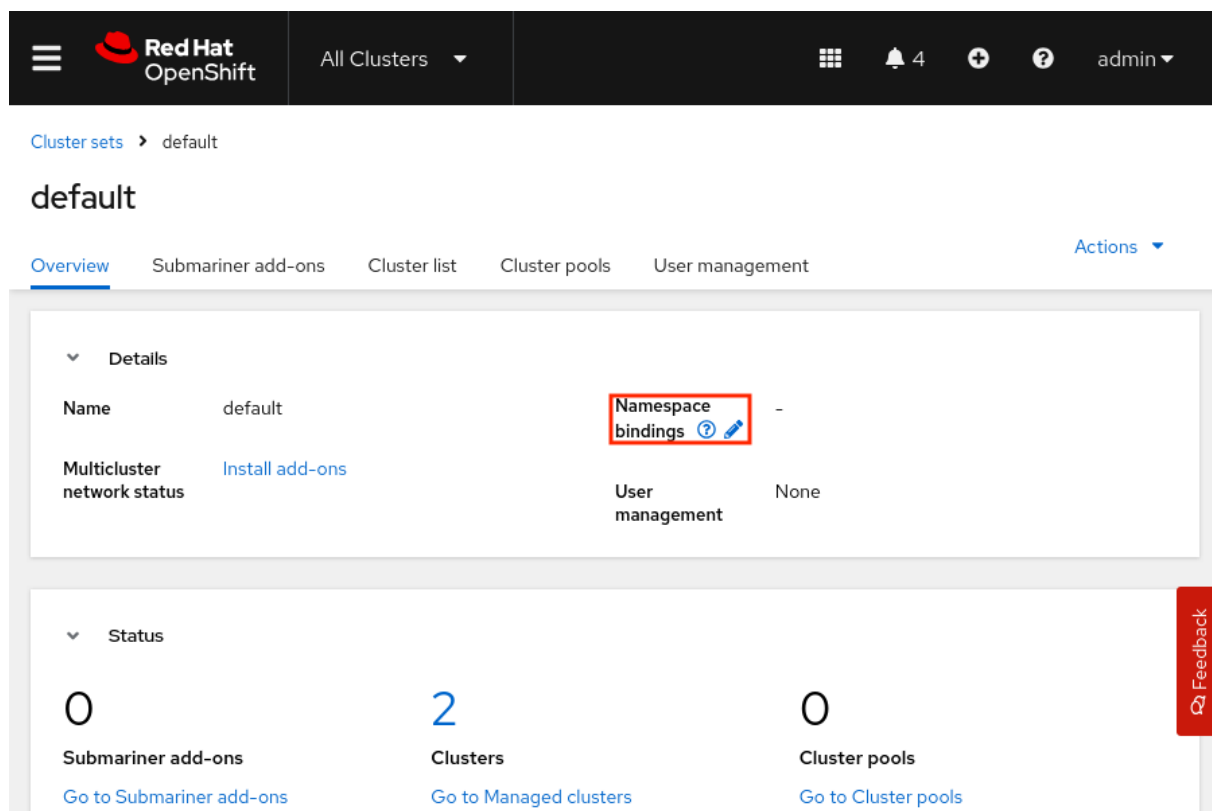
```
[student@workstation ~]$ oc login -u admin -p redhatocp \
https://api.ocp4.example.com:6443
Login successful.
...output omitted...
```

- 1.2. Create the `policies-deploy` project.

```
[student@workstation ~]$ oc new-project policies-deploy
Now using project "policies-deploy" on server "https://api.ocp4.example.com:6443".
...output omitted...
```

2. Create a `ManagedClusterSetBinding` resource that binds the `ManagedClusterSet` resource to the `policies-deploy` namespace.
 - 2.1. From the workstation machine, open Firefox and access `https://console-openshift-console.apps.ocp4.example.com`
 - 2.2. Click **Red Hat Identity Management** and log in as the `admin` user with `redhatocp` as the password.
 - 2.3. Ensure that the cluster switcher, at the top of the console, is set to `All Clusters`, which displays the RHACM console.
 - 2.4. Select **Infrastructure > Clusters**.
 - 2.5. Click **Cluster sets**. Scroll to the bottom of the page and click **default**.

2.6. Click the pencil icon to edit the namespace bindings.

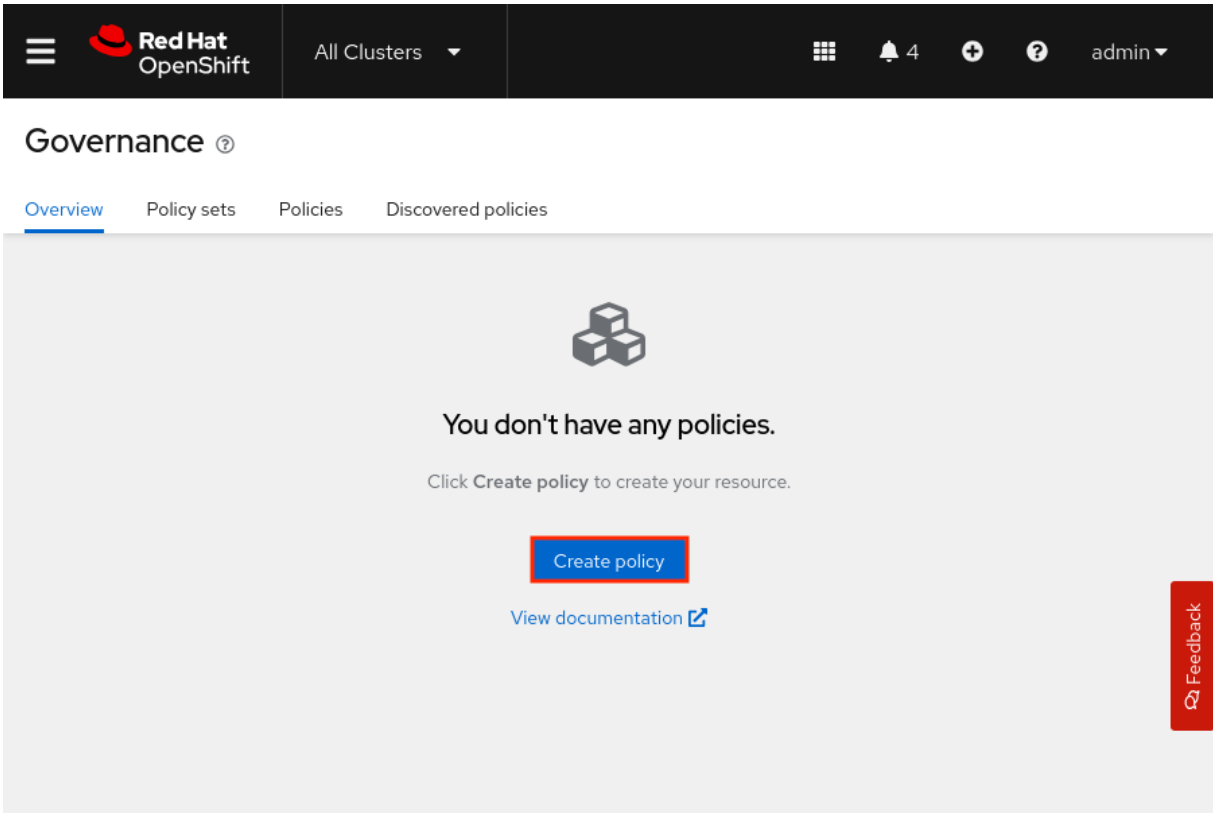


2.7. Select policies-deploy from the Namespaces drop-down list.

2.8. Click **Save**.

3. Deploy a certificate policy to verify certificate expiration in the openshift-console and openshift-ingress namespaces across all clusters by using the RHACM console UI.

3.1. From the RHACM console, go to **Governance > Policies** and click **Create policy**.



3.2. In the policy creation form, configure the fields as shown in the following table:

Field	Action
Name	certificate-policy-console-ingress
Namespace	policies-deploy
Disable policy	checkbox is cleared

Click **Next**.

3.3. In the Policy templates section, click **Add policy template** and click **Certificate management expiration**.

This policy template checks for certificates in the specified namespaces that expire in less than 300 hours.

3.4. Remove the default namespace and configure the settings as shown in the following table:

Field	Action
Name	certificate-policy-console-ingress
Include Namespace	openshift-console
Include Namespace	openshift-ingress

Field	Action
Remediation action	Inform
Severity	High

Click **Next**.

- 3.5. In the Placement section, click **New Placement** and verify that the certificate-policy-console-ingress-placement value is in the Name field.
- 3.6. Configure the placement to apply the policy to all the clusters. Select the default cluster set from the **Cluster sets** drop-down list.
- 3.7. Click **Review** on the wizard menu, and click **Submit** to deploy the policy to the cluster set.
4. Analyze the status of the certificate policy.

4.1. From the RHACM console, go to **Governance > Policies** and select the certificate-policy-console-ingress policy in the policies-deploy namespace.

4.2. Verify that the policy is applied to both the local-cluster and managed-cluster clusters.

Red Hat

OpenShift

All Clusters

3

admin

Policies

>

certificate-policy-console-ingress

certificate-policy-console-ingress

Details

Results

Actions

>

Policy details

Placement

Name	Kind	Clusters	Violations
certificate-policy-console-ingress-placement	Placement	2	Violations: local-cluster No violations: managed-cluster

Feedback

- 4.3. Click **Results** to review any policy violations that indicate that the certificates in the openshift-console or openshift-ingress namespaces will expire in less than 300 hours.

Red Hat

OpenShift

All Clusters ▾

3

admin ▾

Policies > certificate-policy-console-ingress

certificate-policy-console-ingress

DetailsResultsActions ▾

Clusters

Find clusters

1 - 2 of 2 ▾ < >

Cluster ▴ ▾	Violations ▴ ▾	Template ▴ ▾	Message ▴ ▾	Remediation ▴ ▾	Last report ▴ ▾	History
local-cluster	Violations	certificate-policy-console-ingress	1 certificates expire in less than 300h0m0s: openshift-ingress:router-cert - View details	inform	Jul 17, 2025, 2:17 PM	View history
managed-cluster	No violations	certificate-policy-console-ingress	All evaluated certificates are compliant - View details	inform	Jul 17, 2025, 2:17 PM	View history

1 - 2 of 2 items ▾ << < 1 of 1 page > >>

Feedback

A policy violation exists for the openshift-ingress namespace for the router-cert certificate.

5. Apply a fix to remediate the noncompliant certificates in the openshift-ingress namespace.
- 5.1. Return to the local cluster terminal.

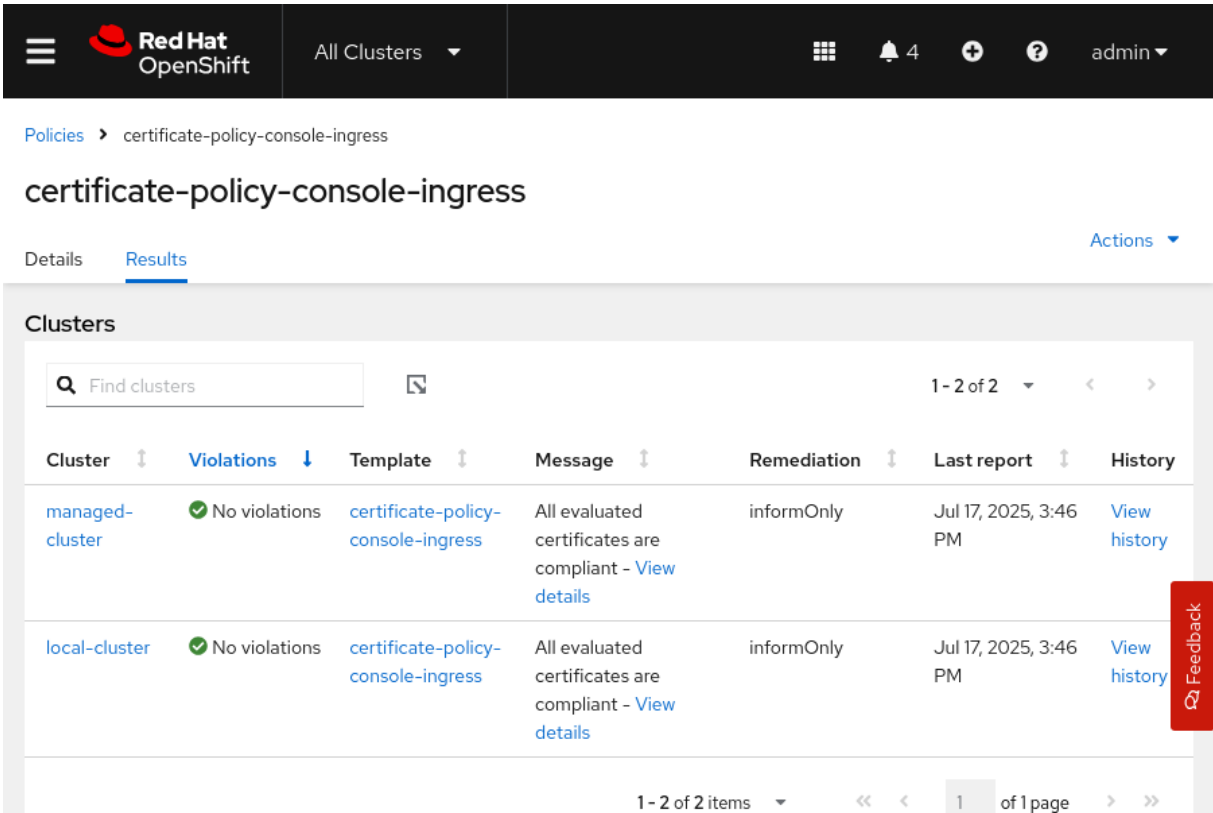
Identify the noncompliant certificate secret in the openshift-ingress namespace.

```
[student@workstation ~]$ oc get secrets -n openshift-ingress \
--field-selector type=kubernetes.io/tls
NAME          TYPE          DATA  AGE
router-certs  kubernetes.io/tls  2      1d
...output omitted...
```

- 5.2. Run the ~/D00013L/labs/policies-deploy/remediate-cert.sh script to update the router-certs secret with a new certificate in the openshift-ingress namespace. The script generates a compliant self-signed certificate that expires in 365 days and updates the existing TLS secret in the openshift-ingress namespace.

```
[student@workstation ~]$ ~/D00013L/labs/policies-deploy/remediate-cert.sh
...output omitted...
```

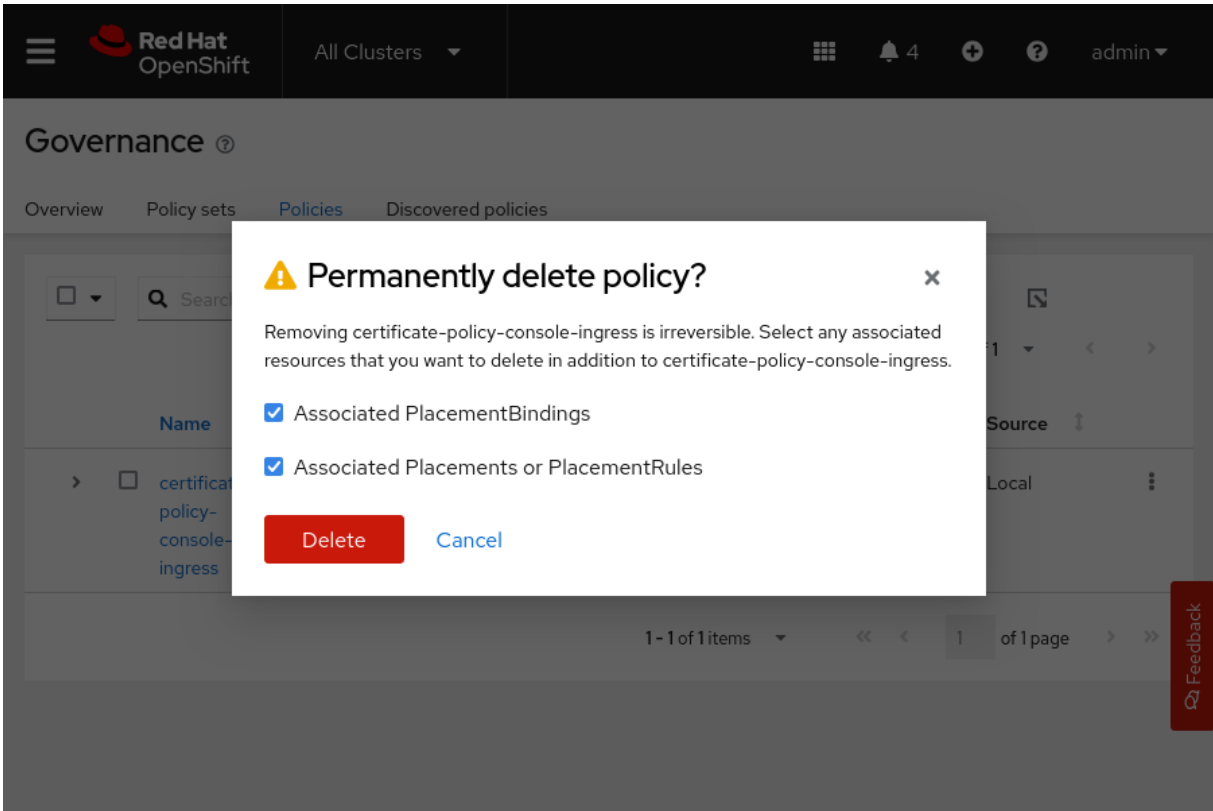
6. Verify the final status of the certificate policy after remediation.
- 6.1. From the RHACM console, go to **Governance > Policies** and select the certificate-policy-console-ingress policy in the policies-deploy namespace.
- 6.2. Verify that the local-cluster cluster is marked as Compliant for the openshift-ingress namespace. Select the **Results** tab and verify that no compliance issues exist for the remediated certificates.



7. Clean up the certificate policy and the associated resources.

7.1. Go to **Governance > Policies** and locate the certificate-policy-console-ingress policy in the policies-deploy namespace.

Click the vertical ellipsis icon  to the right of the policy and select **Delete**. Confirm the deletion of the policy and the associated resources.



7.2. In the terminal, verify that the policy is removed.

```
[student@workstation ~]$ oc get policies -n policies-deploy  
No resources found in policies-deploy namespace.
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish policies-deploy
```

3.5. Explain and Deploy Policies by Using Policy Generator

Objectives

- Describe the components of the Policy Generator and use it to deploy policies.

Overview of the Policy Generator Component

The policy generator component is included in the RHACM subscription. You can use this component with the Kustomize command-line tool to build policies from Kubernetes manifest YAML files, or you can use it as a stand-alone command-line tool. Kustomize is a configuration management tool to configure Kubernetes resources for different environments or needs. You can apply declarative changes to application configurations and components, and preserve the original base YAML files.

Developers commonly use Kustomize when they have one base deployment file that they must patch for different environments, such as development, staging, and production.

Capabilities of the Policy Generator Component

By integrating the policy generator component, you can use it to automatically build the YAML structure of RHACM policies. It simplifies the distribution of resource objects to managed clusters through RHACM policies.

Use the Policy Generator to complete the following actions:

- Convert any manifest files to RHACM configuration policies, including files that are created from the kustomize directory.
- Patch the input manifests before they are inserted into a generated RHACM policy.
- Generate additional configuration policies so you can report on Gatekeeper policy violations through RHACM.
- Generate policy sets on the hub cluster.

Policy Generator Command-line Tools

You can create YAML manifests of policies by using one of the following command-line tools that integrates the policy generator component:

The Kustomize command-line tool

You can use the Kustomize tool as part of the `kubectl` command-line tool to create YAML manifest policies by using the policy generator plug-in.

To use Kustomize in combination with the policy generator plug-in, you must download the policy generator package, and place the binary file in the `${HOME}/.config/kustomize/plugin/policy.open-cluster-management.io/v1/policygenerator` directory.

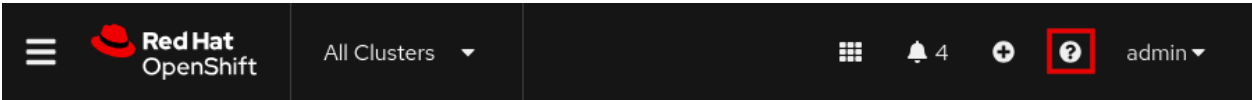
To create a policy by using a policy generator YAML manifest, you must provide a `kustomization` file, or specify the path to the `kustomization` and policy generator files.

Note

For more information about the Kustomize tool, refer to the Kustomize documentation at https://kubernetes.io/docs/reference/kubectl/generated/kubectl_kustomize/

The PolicyGenerator command-line tool

The PolicyGenerator command-line tool is available with the RHACM subscription, and you can download the appropriate version for your operating system from the **Command Line Tools** page in the RHACM web console. With the PolicyGenerator command-line tool, you can create YAML manifests of policies directly, without the use of kustomize files.



Advanced Cluster Management - 2.13.0

Enhance your multicluster experience in the terminal with the CLIs for Red Hat Advanced Cluster Management for Kubernetes. This includes:

- **policytools**
 - Interact with Red Hat Advanced Cluster Management policies locally, such as resolving templates locally and reconciling a ConfigurationPolicy locally (Dev Preview).
 - **PolicyGenerator**
 - Build Red Hat Advanced Cluster Management policies from Kubernetes manifest YAML files, which are provided through a **PolicyGenerator** manifest YAML file that is used to configure it. The Policy Generator is implemented as a Kustomize generator plug-in, allowing integration with GitOps. See the [Policy Generator documentation](#) for details.
- [Download policytools for Linux for x86_64](#)
 - [Download policytools for Linux for ARM64](#)
 - [Download policytools for Mac for x86_64](#)
 - [Download policytools for Mac for ARM64](#)
 - [Download policytools for Windows for x86_64](#)
 - [Download policytools for Windows for ARM64](#)
 - [Download PolicyGenerator for Linux for x86_64](#)
 - [Download PolicyGenerator for Linux for ARM64](#)
 - [Download PolicyGenerator for Mac for x86_64](#)
 - [Download PolicyGenerator for Mac for ARM64](#)

After you download the tool package, extract it and place the binary file in a directory in your \$PATH.

Note

This lesson covers only the use of the PolicyGenerator command-line tool to deploy policies.

The Policy Generator YAML Structure

You can configure default parameters for the policies that you create with the policy generator manifest file, by using the policyDefaults section. In this section, you can specify the fields to configure for all of your policies, such as the severity of the policy violation, a namespace for the policies, or a policy set that the policy joins to.

Except for the namespace, any default value that is configured in the policyDefaults section is overridden by an entry in the policies section, and any default value that is configured in the policySetDefaults section is overridden by an entry in the policySets section.

Note

All the fields that you can configure are listed in the reference table at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/governance/index#policy-generator-yaml-table

The fields that you can configure in the `policies` section are the name of the policy to create, the list of object manifests to include in the policy, and the path to a single file or a directory.

The `policyDefaults` and `policies` sections are mandatory for the policy generator configuration file. You can use the `policySets` section, which is not a mandatory configuration section, to define the list of policy sets to create, along with overrides to either the default values or to the values that are set in the `policySetDefaults` section.

The following YAML manifest is an example of a policy generator configuration that verifies the existence of the `testing-policies` namespace. This manifest specifies a low severity level for the policy, a default namespace named `policies`, and a policy named `namespace-existence` that calls the `manifest.yaml` file.

```
apiVersion: policy.open-cluster-management.io/v1
kind: PolicyGenerator
metadata:
  name: namespace-existence-policies
policyDefaults:
  namespace: policies ①
  severity: low ②
  remediationAction: inform ③
  complianceType: musthave ④
policies: ⑤
- name: namespace-existence
  manifests: ⑥
  - path: /manifest-files/manifest.yaml
```

- ① The `policies` configuration cannot override the default namespace that is defined in the `policyDefaults` section.
- ② The severity level is set to `low` as the default value.
- ③ The `remediationAction` field specifies the action for the policy to take. If not specified, the default value is `inform`, and can be overridden by the values that are configured in each policy entry. If set to `inform`, the policy displays only the violation. If set to `enforce`, the policy makes the corresponding changes to fix the violation.
- ④ The `complianceType` field specifies the expected behavior of the objects that are specified in the policy configuration. If not specified, the default value is `musthave`. If set to `mustonlyhave`, the object must exist with the exact name and relevant fields. If set to `musthave`, the object must exist with the same name as specified in the object's template. If set to `mustnothave`, the object with the same name or labels cannot exist and must be deleted, regardless of the specification or rules.
- ⑤ In this section, you configure each policy one by one. This policy generator manifest file configures only one policy.
- ⑥ The policy configuration calls a file named `manifest.yaml` that is included when the policy is created. You can specify a single file or a directory with multiple files.

In this example, the `manifest.yaml` file contains the configuration of a namespace:

```
apiVersion: v1
kind: Namespace
metadata:
  name: testing-policies
```

To create the policy generator resource by using the Kustomize tool, you must use a `kustomization` YAML manifest. In the `generators` section of a `kustomization` YAML manifest, specify the previous policy generator configuration file.

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
generators: ❶
- policy-generator-config.yaml
```

❶ In the generators section, you specify the previous policy generator file.

The following diagram shows the structure of the base directory that contains a kustomization YAML manifest, a policy generator YAML manifest, and a namespace YAML manifest:

```
[user@host ~]$ tree base
base/
├── kustomization.yaml
├── manifest-files
│   └── manifest.yaml
└── policy-generator-config.yaml
```

Note

The files that you call in the policies section must be separated into a different directory from where the policy generator file is located, to avoid including the PolicyGenerator resource manifest inside the Policy generated resource manifest.

Use the `kubectl kustomize --enable-alpha-plugins path-to-kustomize-directory` command to create the policy generator resource by using a kustomize configuration file. You must specify the `--enable-alpha-plugins` flag to enable the use of the external policy generator kustomize plug-in.

```
[user@host ~]$ kubectl kustomize --enable-alpha-plugins policy-generator-dir
apiVersion: apps.open-cluster-management.io/v1
kind: Placement
metadata:
  name: placement-namespace-existence
  namespace: policies
...output omitted...
---
apiVersion: policy.open-cluster-management.io/v1
kind: PlacementBinding
metadata:
  name: binding-namespace-existence
  namespace: policies
...output omitted...
---
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: namespace-existence
  namespace: policies
spec:
...output omitted...
```

To create the policy resources by using the PolicyGenerator command-line tool, you must use a policy generator YAML manifest only.

The following diagram shows the structure of the directory that contains the policy generator YAML manifest and a namespace YAML manifest:

```
[user@host ~]$ tree policy-gen
policy-gen/
├── manifest-files
│   └── manifest.yaml
└── policy-generator-config.yaml
```

Use the PolicyGenerator `policy-generator-file` command to create the policy resources by using the policy generator configuration file:

```
[user@host ~]$ PolicyGenerator policy-gen/policygenerator.yaml
apiVersion: apps.open-cluster-management.io/v1
kind: Placement
metadata:
  name: placement-namespace-existence
  namespace: policies
...output omitted...
---
apiVersion: policy.open-cluster-management.io/v1
kind: PlacementBinding
metadata:
  name: binding-namespace-existence
  namespace: policies
...output omitted...
---
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: namespace-existence
  namespace: policies
spec:
...output omitted...
```



Note

The placement and binding suffixes are added automatically to the Placement and PlacementBinding manifest resources.

After running the policy generator manifest file, the Policy, Placement, and PlacementBinding manifest resources are created.

The following manifest is an example of the Policy manifest file that is created:

```
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  annotations:
    policy.open-cluster-management.io/categories: CM Configuration Management
    policy.open-cluster-management.io/controls: CM-2 Baseline Configuration
    policy.open-cluster-management.io/standards: NIST SP 800-53
    policy.open-cluster-management.io/description: ""
  name: namespace-existence
  namespace: policies
spec:
  disabled: false
  policy-templates:
  - objectDefinition:
      apiVersion: policy.open-cluster-management.io/v1
      kind: ConfigurationPolicy
      metadata:
        name: namespace-existence
      spec:
        object-templates:
        - complianceType: musthave
          objectDefinition: ❶
            apiVersion: v1
            kind: Namespace
            metadata:
              name:
            remediationAction: inform
            severity: low
```

- ❶ In this section, you can view the namespace manifest that is specified in the manifest section from the policy generator manifest file.

The following manifest is an example of the Placement resource configuration that is created:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: placement-namespace-existence
  namespace: policies
spec:
  predicates:
  - requiredClusterSelector:
      labelSelector:
        matchExpressions: []
  tolerations:
  - key: cluster.open-cluster-management.io/unavailable
    operator: Exists
  - key: cluster.open-cluster-management.io/unreachable
    operator: Exists
```

The following manifest is an example of the PlacementBinding resource configuration that is created:

```
apiVersion: policy.open-cluster-management.io/v1
kind: PlacementBinding
metadata:
  name: binding-namespace-existence
  namespace: policies
placementRef:
  apiGroup: cluster.open-cluster-management.io
  kind: Placement
  name: placement-namespace-existence
subjects:
- apiGroup: policy.open-cluster-management.io
  kind: Policy
  name: namespace-existence ①
```

- ① The policy that is created with the use of the policy generator configuration file.

After you create the policy YAML manifests, you can use them to create the Policy, the Placement and the PlacementBinding resources by using the `oc apply` command:

```
[user@host ~]$ oc apply -f policy-definition.yaml
placement.cluster.open-cluster-management.io/placement-namespace-existence created
placementbinding.policy.open-cluster-management.io/binding-namespace-existence created
policy.policy.open-cluster-management.io/namespace-existence created
```

Note

The combined use of the Kustomize and PolicyGenerator command-line tools could cause issues in a production environment.

Updating Policy Generator Policies

You can update the policy generator YAML manifest to patch the generated policy manifest file by using the policy generator command-line tool.

The following manifest file patches the severity level:

```
apiVersion: policy.open-cluster-management.io/v1
kind: PolicyGenerator
metadata:
  name: namespace-existence-policies
policyDefaults:
  severity: medium ①
```

- ① The patch changes the severity level from low to medium.

Run the PolicyGenerator `policy-generator-file` command to generate the Policy, the Placement, and the PlacementBinding YAML manifests with the new changes.

```
[user@host ~]$ PolicyGenerator examples/policygenerator.yaml
...output omitted...
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
...output omitted...
  name: namespace-existence
  namespace: policies
spec:
...output omitted...
  severity: medium
  remediationAction: inform
```

After you generate the updated YAML manifests, you can use them to update the existing Policy, the Placement, and the PlacementBinding resources.

References

For more information about the Policy Generator command-line tool, refer to the *Policy Generator* section in the *Hub Cluster Policy Framework* guide in the *Red Hat Advanced Cluster Management for Kubernetes* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.12/html-single/governance/index#policy-generator

For more information about Kustomize, refer to the *Overview of Kustomize* section in the *Declarative Management of Kubernetes Objects Using Kustomize* documentation at

<https://kubernetes.io/docs/tasks/manage-kubernetes-objects/kustomization/>

For more information about the installation of the Policy Generator command-line tool, refer to the *Installing the Policy Generator* section in the *Policy Generator* documentation at <https://github.com/open-cluster-management-io/policy-generator-plugin>

3.6. Guided Exercise

Deploy Policies by Using Policy Generator

Deploy a policy by using Policy Generator and verify the policy status from the RHACM governance dashboard.

Outcomes

- Create a policy that requests the existence of the qa-policies and prod-policies namespaces by using the policy generator.
- Patch the policy generator resource file to remediate violations that occur.

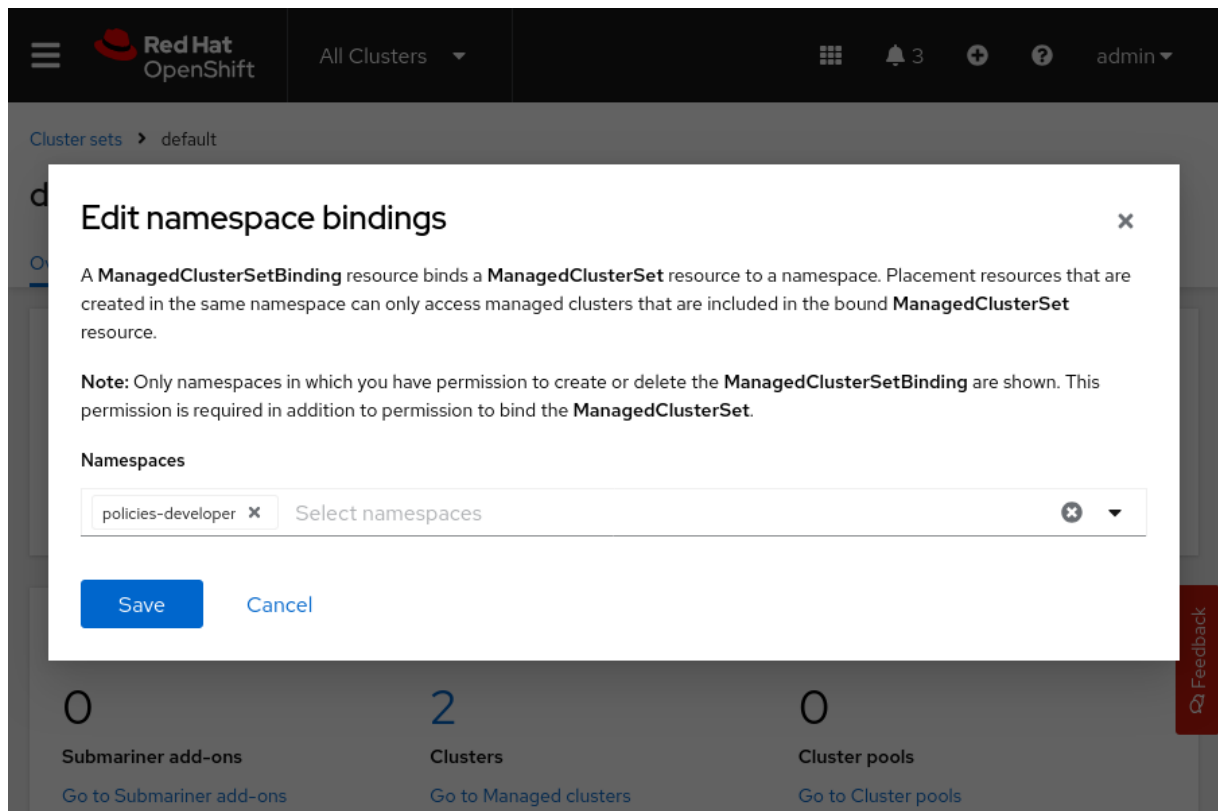
Prerequisites

As the student user on the workstation machine, use the lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab install do0013l ; lab start policies-generator
```

Instructions

1. Log in to the RHOC P hub cluster, and switch to the RHACM web console.
 - 1.1. Open a web browser and go to <https://console-openshift-console.apps.ocp4.example.com>
 - 1.2. Select the **Red Hat Identity Management** identity provider and log in as the admin user with redhatocp as the password.
 - 1.3. Ensure that the cluster switcher, at the top of the console, is set to All Clusters, which displays the RHACM web console.
2. Bind the policies-developer namespace to the default cluster set.
 - 2.1. Go to **Infrastructure > Clusters** and click **Cluster sets**.
 - 2.2. Select the default cluster set and click the pencil icon to edit the namespace bindings.
 - 2.3. Select policies-developer from the Namespaces drop-down list, and click **Save**.



3. In a terminal window, log in to the RHOCP cluster as the admin user with redhatocp as the password.

```
[student@workstation ~]$ oc login -u admin -p redhatocp \
https://api.ocp4.example.com:6443
Login successful.
...output omitted...
```

4. Create a policy generator YAML manifest file to configure a policy that requires the existence of the qa-policies and prod-policies namespaces. The policy must be created in the policies-developer namespace.

- 4.1. Create a namespace YAML manifest file for the qa-policies namespace in the ~/D00013L/policies-generator/manifest directory, to define only the name of the namespace.

```
---
apiVersion: v1
kind: Namespace
metadata:
  name: qa-policies
```

- 4.2. Create another namespace YAML manifest file for the prod-policies namespace in the ~/D00013L/policies-generator/manifest directory, to define only the name of the namespace.

```
---
apiVersion: v1
kind: Namespace
metadata:
  name: prod-policies
```

- 4.3. Create a policy generator YAML file in the ~/D00013L/policies-generator directory. Use the following table to configure the YAML manifest. The policy generator YAML file must reference the ~/D00013L/policies-generator/manifest directory.

Field	Value
namespace	policies-developer
complianceType	musthave
severity	medium

```
---
apiVersion: policy.open-cluster-management.io/v1
kind: PolicyGenerator
metadata:
  name: policy-generator-namespace
policyDefaults:
  namespace: policies-developer
  complianceType: musthave
  severity: medium
policies:
- name: policy-namespace
  manifests:
    - path: /home/student/D00013L/policies-generator/manifest
```

5. Create the Policy, Placement, and PlacementBinding manifests by using the policy generator manifest file, and store the output in a YAML file in the ~/D00013L/policies-generator directory.

5.1. Use the policy generator manifest file from the previous steps to create the Policy, Placement, and PlacementBinding manifests. Store the results in a YAML file.

```
[student@workstation ~]$ PolicyGenerator \
~/D00013L/policies-generator/policy-generator-file.yaml > \
~/D00013L/policies-generator/policy-manifest.yaml
```

5.2. Inspect the resource manifests that are generated.

```

---
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  annotations:
    policy.open-cluster-management.io/categories: CM Configuration Management
    policy.open-cluster-management.io/controls: CM-2 Baseline Configuration
    policy.open-cluster-management.io/description: ""
    policy.open-cluster-management.io/standards: NIST SP 800-53
  name: policy-namespace
  namespace: policies-developer
spec:
  disabled: false
  policy-templates:
    - objectDefinition:
        apiVersion: policy.open-cluster-management.io/v1
        kind: ConfigurationPolicy
        metadata:
          name: policy-namespace
        spec:
          object-templates:
            - complianceType: musthave
              objectDefinition:
                apiVersion: v1
                kind: Namespace
                metadata:
                  name: prod-policies
            - complianceType: musthave
              objectDefinition:
                apiVersion: v1
                kind: Namespace
                metadata:
                  name: qa-policies
              remediationAction: inform
              severity: medium
          remediationAction: inform
---
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: placement-policy-namespace
  namespace: policies-developer
spec:
  predicates:
    - requiredClusterSelector:
        labelSelector:
          matchExpressions: []
  tolerations:
    - key: cluster.open-cluster-management.io/unavailable
      operator: Exists
    - key: cluster.open-cluster-management.io/unreachable
      operator: Exists
---
apiVersion: policy.open-cluster-management.io/v1
kind: PlacementBinding
metadata:
  name: binding-policy-namespace
  namespace: policies-developer
placementRef:
  apiGroup: cluster.open-cluster-management.io
  kind: Placement
  name: placement-policy-namespace
subjects:
  - apiGroup: policy.open-cluster-management.io
    kind: Policy
    name: policy-namespace

```

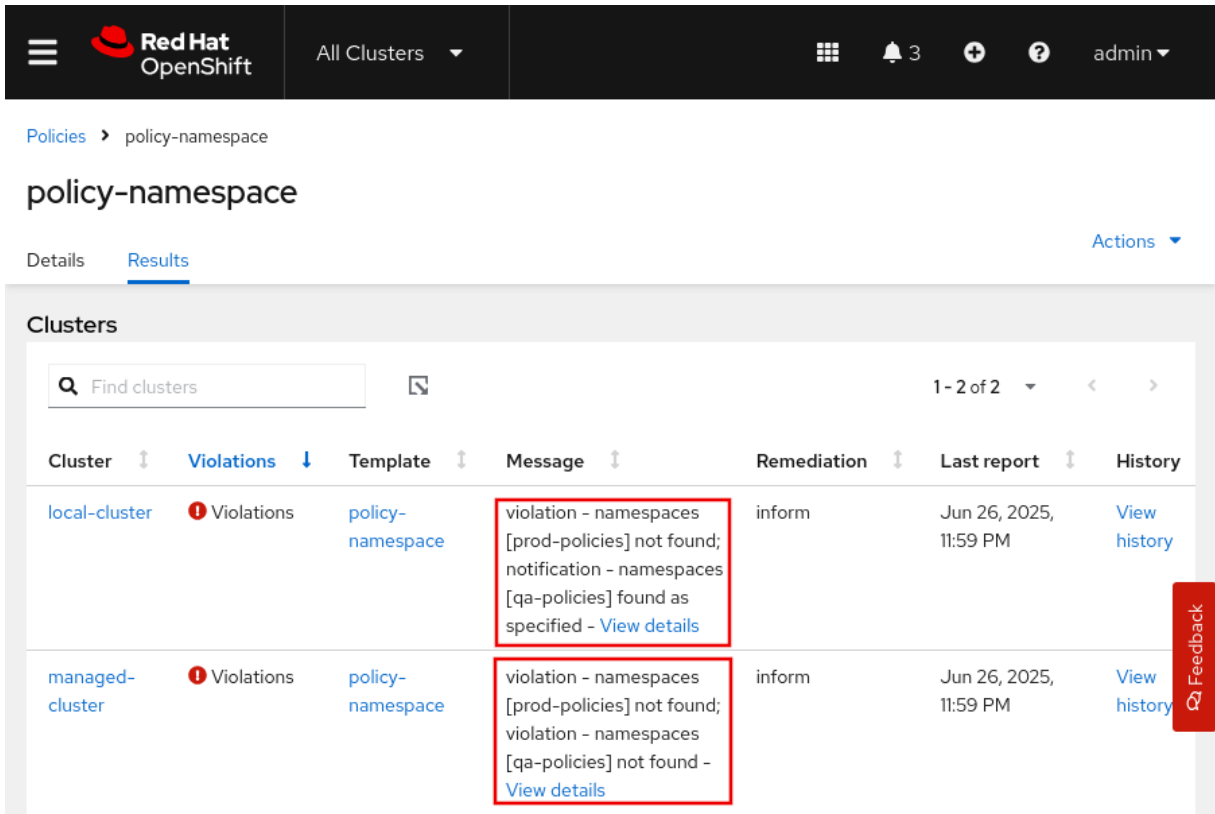
6. Create the Policy, Placement, and PlacementBinding resources by using the previously created policy manifest file.

Use the `oc apply` command to create these resources.

```
[student@workstation ~]$ oc apply -f \
~/D00013L/policies-generator/policy-manifest.yaml
policy.policy.open-cluster-management.io/policy-namespace created
placement.cluster.open-cluster-management.io/placement-policy-namespace created
placementbinding.policy.open-cluster-management.io/binding-policy-namespace created
```

7. Review the violations from the RHACM web console.

- 7.1. Go to **Governance > Policies** and select the `policy-namespace` policy.
- 7.2. Click **Results** and review the violations that are found.



The warning messages indicate that the `qa-policies` namespace is missing in the `managed-cluster` cluster, and the `prod-policies` namespace is missing in the `managed-cluster` and the `local-cluster` cluster.

8. Patch the policy generator manifest file to enable the policy to remediate the violations automatically.
- 8.1. Update the policy generator manifest file to add the `remediationAction` field with `enforce` as the value.

```
---
apiVersion: policy.open-cluster-management.io/v1
kind: PolicyGenerator
metadata:
  name: policy-generator-namespace
policyDefaults:
  remediationAction: enforce
  namespace: policies-developer
  complianceType: musthave
  severity: medium
policies:
- name: policy-namespace
  manifests:
  - path: /home/student/D00013L/policies-generator/manifest
```

8.2. Re-create the Policy, Placement, and PlacementBinding resource manifests.

```
[student@workstation ~]$ PolicyGenerator \
~/D00013L/policies-generator/policy-generator-file.yaml > \
~/D00013L/policies-generator/policy-manifest.yaml
```

9. Update the policy-namespace policy to apply the latest changes.

9.1. Apply the latest changes to the policy-namespace policy.

```
[student@workstation ~]$ oc apply -f \
~/D00013L/policies-generator/policy-manifest.yaml
policy.policy.open-cluster-management.io/policy-namespace configured
placement.cluster.open-cluster-management.io/placement-policy-namespace unchanged
placementbinding.policy.open-cluster-management.io/binding-policy-namespace unchanged
```

10. Confirm that the latest changes that were applied to the policy-namespace policy remediate the violations.

10.1. Go to **Governance > Policies** and select the policy-namespace policy.

10.2. Click **Results** and review the violations.

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

Red Hat

OpenShift

All Clusters

3

admin

Policies

policy-namespace

policy-namespace

Details

Results

Actions

Find clusters

1 - 2 of 2

Cluster	Violations	Template	Message	Remediation	Last report	History
local-cluster	No violations	policy-namespace	notification - namespaces [prod-policies] found as specified; notification - namespaces [qa-policies] found as specified - View details	enforce	Jun 27, 2025, 12:07 AM	View history
managed-cluster	No violations	policy-namespace	notification - namespaces [prod-policies] found as specified; notification - namespaces [qa-policies] found as specified - View details	enforce	Jun 27, 2025, 12:07 AM	View history

Feedback

The message states that both prod-policies and qa-policies namespaces were found.

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish policies-generator
```

Page 170 | 475

DO432-RHACM2.13-en-2-20260109

3.7. Deploying and Configuring the Compliance Operator for Multiple Clusters by Using RHACM

Objectives

- Deploy the Compliance Operator policy and view compliance reports from multiple clusters by using the command line and the RHACM governance dashboard.

Definition of Compliance

Compliance management is a continuous process to ensure that IT systems are compliant with the organization's policies and procedures. Through the entire lifecycle of IT infrastructure management, policy and standards continuously evolve to best suit the needs of the business. It is crucial to identify any systems that are noncompliant with the latest standards for any reason. Failure to recognize noncompliant systems can result in the following issues:

- Loss of the client's trust in the company, leading clients to leave the company
- Legal ramifications such as fines or being blocked from working in specific geographical locations
- Loss of IT systems certifications

Compliance management must be agile in its approach to managing security, regulatory, and configuration changes. With security threats evolving technically and strategically, compliance management aims to secure or harden cloud environments including those with multiple Kubernetes clusters.

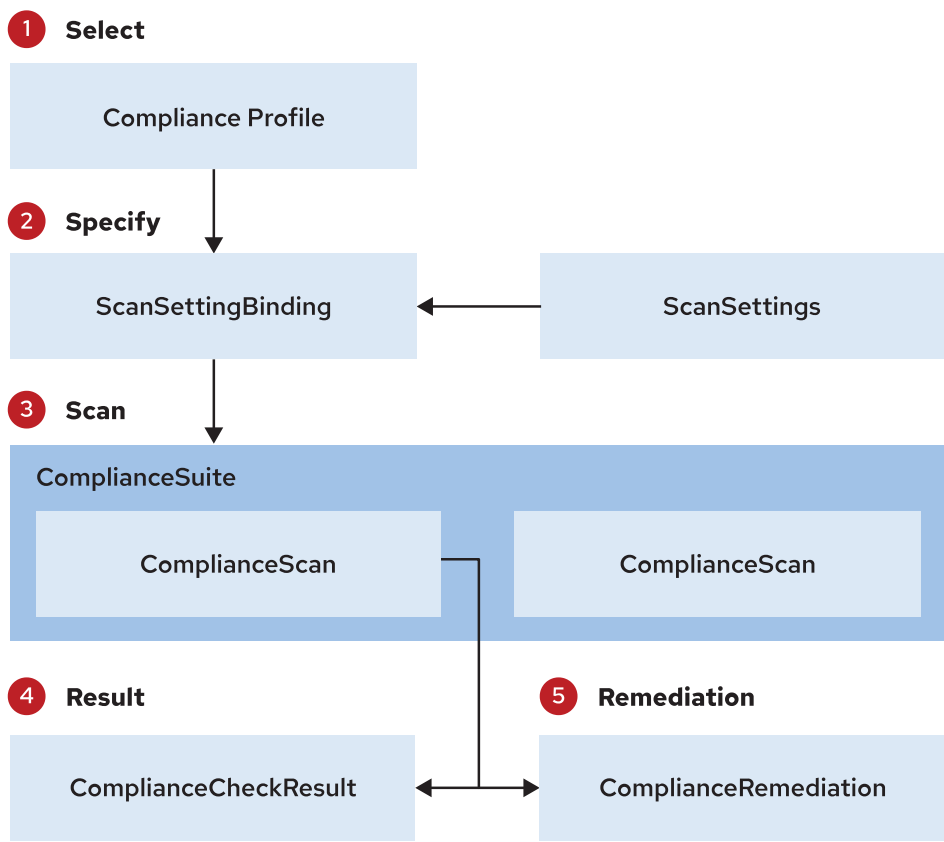
The RHOCP Compliance Operator

The compliance operator is an OpenShift operator that employs OpenSCAP and enables an administrator to run compliance scans and remediate issues that are found. The compliance operator inspects Kubernetes objects and the nodes that are running on the cluster to detect any gaps in compliance. The operator scans specific profiles and then produces a summary report to assist you in discovering noncompliant systems. The recommended remediation can be applied by manually or automatically. The process involves the following steps:

1. Select a compliance profile for scanning.
2. Specify the scan settings.
3. Start the scan.
4. Generate the reports.
5. Apply remediation.

OpenSCAP assesses for compliance, based on the policy rules that the content defines.

For use by Rajesh Ramamoorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamoorthy ra_6776 ra.6776@gmail.com Copyright



OpenSCAP

OpenSCAP is a Security Content Automation Protocol (SCAP) compliant scanner that scans for compliance and vulnerabilities. The compliance operator uses OpenSCAP to scan the OpenShift clusters and nodes. OpenSCAP employs community-based compliance content that was developed as part of the ComplianceAsCode/content project. This content is distributed as a container image and is decoupled from the operator for rapid content updates.

The compliance operator consumes the compliance content from the container image and exposes it in a custom resource.

 **Note**
SCAP is a set of specific standards to support automated vulnerability and patch monitoring, security measurement, and technical control compliance activities.

Policies

The Red Hat Advanced Cluster Management for Kubernetes policies use custom resource definitions (CRDs). Policies contain one or more policy templates.

All policies require the following two CRDs:

Placement

Defines the targeted clusters to receive the policy.

PlacementBinding

Binds the policy to the placement rule.

Note

You can create a policy in any namespace on the hub cluster except the cluster namespace. If you create a policy in the cluster namespace, then RHACM deletes it.

Compliance Operator Policy

RHACM uses the compliance operator policy to deploy the compliance operator across a fleet of clusters. The RHACM console enables administrators to create a policy that deploys the compliance operator to all the managed clusters.

Use the enforce remediation action to automatically install the operator on all managed clusters. Using the inform remediation action reports a violation if the operator is not found on a cluster, and does not install the operator. Creating a compliance operator policy results in the creation of a namespace, an OperatorGroup, and a subscription.

Namespace

The operator installation uses the openshift-compliance namespace for the compliance operator.

```
apiVersion: policy.open-cluster-management.io/v1 ①
kind: ConfigurationPolicy ②
metadata:
  name: comp-operator-ns ③
spec:
  remediationAction: inform ④
  severity: high ⑤
  object-templates:
    - complianceType: musthave ⑥
      objectDefinition:
        apiVersion: v1
        kind: Namespace ⑦
        metadata:
          name: openshift-compliance ⑧
```

- ① Specifies the RHACM policy API version.
- ② Defines a ConfigurationPolicy to enforce resource creation.
- ③ Names the policy for namespace creation.
- ④ Reports violations without automatic creation.
- ⑤ Marks violations as high priority.
- ⑥ Ensures that the namespace exists.
- ⑦ Defines a Kubernetes Namespace resource.
- ⑧ Creates the openshift-compliance namespace.

OperatorGroup

The compliance-operator operator group specifies the target namespace.

```
apiVersion: policy.open-cluster-management.io/v1 ①
kind: ConfigurationPolicy ②
metadata:
  name: comp-operator-operator-group ③
spec:
  remediationAction: inform ④
  severity: high ⑤
  object-templates:
    - complianceType: musthave ⑥
      objectDefinition:
        apiVersion: operators.coreos.com/v1 ⑦
        kind: OperatorGroup ⑧
        metadata:
          name: compliance-operator ⑨
          namespace: openshift-compliance ⑩
        spec:
          targetNamespaces:
            - openshift-compliance ⑪
```

- ① RHACM policy API version
- ② ConfigurationPolicy for OperatorGroup
- ③ Policy name for OperatorGroup creation
- ④ Reports violations; not overridden by parent policy.
- ⑤ High-priority violations
- ⑥ Ensures that OperatorGroup exists.
- ⑦ OperatorGroup API version
- ⑧ Defines an OperatorGroup resource.
- ⑨ Names the OperatorGroup.
- ⑩ Places the OperatorGroup in the openshift-compliance namespace.
- ⑪ Targets the openshift-compliance namespace.

Subscription

A comp-operator-subscription subscription references the name and channel. The subscription pulls the profile, as a container, that it supports.

```
apiVersion: policy.open-cluster-management.io/v1 ①
kind: ConfigurationPolicy ②
metadata:
  name: comp-operator-subscription ③
spec:
  remediationAction: inform ④
  severity: high ⑤
  object-templates:
    - complianceType: musthave ⑥
      objectDefinition:
        apiVersion: operators.coreos.com/v1alpha1 ⑦
        kind: Subscription ⑧
        metadata:
          name: compliance-operator ⑨
          namespace: openshift-compliance ⑩
        spec:
          installPlanApproval: Automatic ⑪
          name: compliance-operator ⑫
          source: redhat-operators ⑬
          sourceNamespace: openshift-marketplace ⑭
```

- ① RHACM policy API version
- ② ConfigurationPolicy for Subscription

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

- 3 Policy name for subscription
- 4 Reports violations.
- 5 High-priority violations
- 6 Ensures that Subscription exists.
- 7 Subscription API version
- 8 Defines a Subscription resource.
- 9 Names the subscription.
- 10 Places the subscription in openshift-compliance.
- 11 Automatically approves installation plans.
- 12 References the Compliance Operator.
- 13 Pulls from Red Hat's operator catalog.
- 14 Sources from openshift-marketplace

Deploying the Compliance Operator Policy

RHACM governance provides a compliance policy to deploy across the cluster fleet. To deploy the compliance policy, perform the following steps:

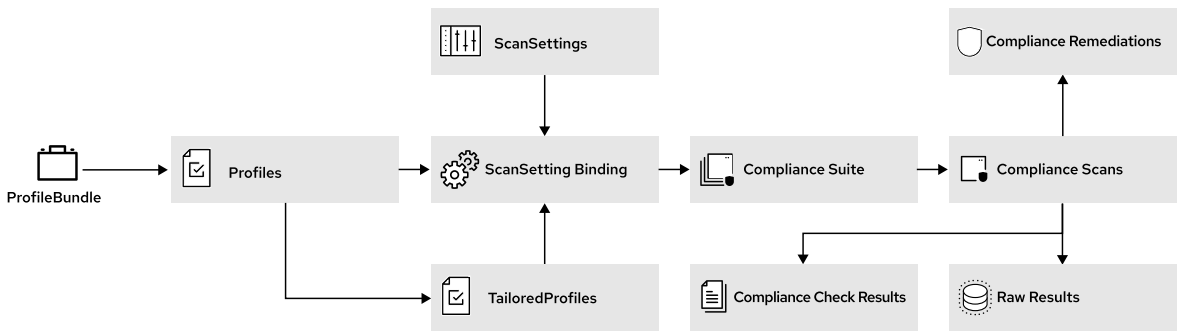
1. Log in to the RHACM console.
2. From the navigation menu in the left pane, select **Governance**.
3. Click **Create policy**.
4. Select **Install** the **Compliance operator** from the **Policy templates** field and complete the remaining fields in the forms.

Compliance Operator Policy CRDs

A CRD defines a unique object Kind to create the Custom Resource (CR) objects, and the API Server manages the entire lifecycle of CRDs.

The compliance operator uses custom resource definitions that represent security policy content, the configuration or result, and its scanning processes.

The following diagram provides an overview of the custom resource definitions.



ProfileBundle

The Compliance operator includes preconfigured ProfileBundle objects. The ProfileBundle object specifies the URL of a container image that contains the content image and the file that contains the compliance content. A ProfileBundle object can specify multiple Profile objects.

Profile

The Profile object defines the rules and variables that can be evaluated for a certain compliance standard. You cannot create or modify the Profile object manually because it is derived from a single ProfileBundle object.

TailoredProfile

Use the TailoredProfile object to modify the default Profile object based on your organization requirements. You can enable or disable rules, set variable values, and provide justification for the customization.

ScanSetting

The ScanSetting custom resource defines the scan settings:

- The nodes to scan
- The allocated storage for the results collection
- The time and frequency for the scan
- The retention policy

The compliance operator creates a ScanSetting object named default.

ScanSettingBinding

The ScanSettingBinding object binds the ScanSetting object with the profiles.

The following example of a ScanSettingBinding object references two profiles: ocp4-e8 and rhcos4-e8. The settingsRef value is a reference to the ScanSetting object. The remediationAction is defined with inform: RHACM verifies whether the object exists, and if not, it does not create the object.

```
apiVersion: policy.open-cluster-management.io/v1
kind: ConfigurationPolicy
metadata:
  name: compliance-e8-scan
spec:
  remediationAction: inform
  severity: high
  object-templates:
    - complianceType: musthave
      objectDefinition:
        apiVersion: compliance.openshift.io/v1alpha1
        kind: ScanSettingBinding
        metadata:
          name: e8
          namespace: openshift-compliance
        profiles:
          - apiGroup: compliance.openshift.io/v1alpha1
            kind: Profile
            name: ocp4-e8
          - apiGroup: compliance.openshift.io/v1alpha1
            kind: Profile
            name: rhcos4-e8
        settingsRef:
          apiGroup: compliance.openshift.io/v1alpha1
          kind: ScanSetting
          name: default
```

The ScanSetting and ScanSettingBinding objects generate the following ComplianceSuite object.

ComplianceScan

The ComplianceScan object represents a single scan and is referenced by the ComplianceSuite object.

ComplianceSuite

The ComplianceSuite object is a bundle of ComplianceScan objects and maintains a record of each scan. Although the ComplianceSuite objects are automatically created when the ScanSetting and ScanSettingBinding objects are created, you can create them directly in specific use cases.

Compliance Profiles

Compliance profiles define and package a list of compliance benchmarks, scans, and remediation steps. Use the `oc get profile.compliance` command to view the compliance profiles that are included with the compliance operator.

```
[user@host ~]$ oc get profile.compliance -n openshift-compliance \
-l compliance.openshift.io/profile-bundle=rhcos4
NAME                AGE
rhcos4-e8            127m
rhcos4-moderate      127m
rhcos4-nerc-cip      127m
```

By default, when the compliance operator is deployed, it creates two ProfileBundle objects, one for OCP and one for Red Hat CoreOS (RHCOS). Cluster-level scans use the OCP object, and node-level scans use the RHCOS object. You can find the profiles from the ProfileBundle objects that contain a reference to the container image. The operator creates profile objects that represent or define the compliance benchmarks. Profiles consist of a set of rules that validate that a system is in compliance. Red Hat provides ready-to-use profiles. To view the ProfileBundle objects, use the `oc get profilebundle.compliance` command:

```
[user@host ~]$ oc get profilebundle.compliance -n openshift-compliance
NAME      ...    CONTENTFILE    STATUS
ocp4      ...    ssg-ocp4-ds.xml    VALID
rhcos4    ...    ssg-rhcos4-ds.xml  VALID
```

To view the objects in each bundle, use the `oc get profile.compliance` command:

```
[user@host ~]$ oc get profile.compliance -n openshift-compliance \
-l compliance.openshift.io/profile-bundle=rhcos4
NAME                AGE
rhcos4-e8            127m
rhcos4-moderate      127m
rhcos4-nerc-cip      127m
```

To view the profile rules, you also use the `oc get profile.compliance` command:

```
[user@host ~]$ oc get profile.compliance -n openshift-compliance rhcos4-e8 \
-o yaml
apiVersion: compliance.openshift.io/v1alpha1
description: 'This profile contains configuration checks for Red Hat Enterprise Linux
CoreOS that align to the Australian Cyber Security Centre (ACSC) Essential Eight.

id: xccdf_org.ssgproject.content_profile_e8
kind: Profile
...output omitted...
rules:
- rhcos4-accounts-no-uid-except-zero
- rhcos4-audit-rules-dac-modification-chmod
- rhcos4-audit-rules-dac-modification-chown
- rhcos4-audit-rules-execution-chcon
- rhcos4-audit-rules-execution-restorecon
- rhcos4-audit-rules-execution-semanage
...output omitted...
```

The compliance operator enables auditing by exposing the Rule object as an API object.

Essential 8 (E8) Policy Scan

This policy assumes that the compliance operator is installed and running on the managed clusters. It deploys a scan that checks the nodes, and verifies for compliance with the E8 security profile.

```
apiVersion: policy.open-cluster-management.io/v1
kind: Policy ①
metadata:
  name: policy-e8-scan
  annotations:
    policy.open-cluster-management.io/standards: NIST SP 800-53
    policy.open-cluster-management.io/categories: CM Configuration Management
    policy.open-cluster-management.io/controls: CM-6 Configuration Settings
spec:
  remediationAction: inform
  disabled: false
  policy-templates:
    - objectDefinition:
        apiVersion: policy.open-cluster-management.io/v1
        kind: ConfigurationPolicy ②
        metadata:
          name: compliance-e8-scan ③
        spec:
          remediationAction: inform
          severity: high
          object-templates:
            - complianceType: musthave
              objectDefinition:
                apiVersion: compliance.openshift.io/v1alpha1
                kind: ScanSettingBinding
                metadata:
                  name: e8
                  namespace: openshift-compliance
                profiles: ④
                - apiGroup: compliance.openshift.io/v1alpha1
                  kind: Profile
                  name: ocp4-e8
                - apiGroup: compliance.openshift.io/v1alpha1
                  kind: Profile
                  name: rhcos4-e8
              settingsRef:
                apiGroup: compliance.openshift.io/v1alpha1
                kind: ScanSetting
                name: default
```

- ① The Policy object defines the type of policy.
- ② The ConfigurationPolicy object checks the existence of the ScanSettingBinding object.
- ③ The remediationAction parameter, which is set to inform, reports if the ScanSettingBinding object does not exist. Use enforce to automatically create the missing object.
- ④ The profiles spec defines the profiles, ocp4-e8 and rhcos4-e8, that are deployed to comply with the E8 benchmarks.

```
- objectDefinition:
  apiVersion: policy.open-cluster-management.io/v1
  kind: ConfigurationPolicy
  metadata:
    name: compliance-suite-e8 ①
  spec:
    remediationAction: inform ②
    severity: high
    object-templates:
      - complianceType: musthave ③
        objectDefinition:
          apiVersion: compliance.openshift.io/v1alpha1
          kind: ComplianceSuite
          metadata:
            name: e8
            namespace: openshift-compliance
          status:
            phase: DONE ④
```

- ① The ComplianceSuite object tracks scans and outputs reports of the scanning process.
- ② The remediationAction parameter, which is set to inform, reports if the ScanSettingBinding object does not exist. Use enforce to automatically create the missing object.
- ③ This template checks whether the scan completed, from the status field.
- ④ The ComplianceSuite object must be created and reach the DONE phase.

```
- objectDefinition:
  apiVersion: policy.open-cluster-management.io/v1
  kind: ConfigurationPolicy
  metadata:
    name: compliance-suite-e8-results
  spec:
    remediationAction: inform
    severity: high
    object-templates:
      - complianceType: mustnothave ①
        objectDefinition:
          apiVersion: compliance.openshift.io/v1alpha1
          kind: ComplianceCheckResult ②
          metadata:
            namespace: openshift-compliance
            labels:
              compliance.openshift.io/check-status: FAIL ③
              compliance.openshift.io/suite: e8 ④
```

- ① This template reports the results for the e8 scan suite by reviewing the ComplianceCheckResult CRs.
- ② The compliance operator uses ComplianceCheckResult objects to define the scan results.
- ③ Verifies that no rules failed.
- ④ The applied rules originate from an E8 scan.


```
apiVersion: policy.open-cluster-management.io/v1
kind: PlacementBinding ①
metadata:
  name: binding-policy-e8-scan
placementRef:
  name: placement-policy-e8-scan
  kind: PlacementRule
  apiGroup: apps.open-cluster-management.io
subjects:
- name: policy-e8-scan
  kind: Policy
  apiGroup: policy.open-cluster-management.io
---
apiVersion: apps.open-cluster-management.io/v1
kind: PlacementRule ②
metadata:
  name: placement-policy-e8-scan
spec:
  clusterConditions:
  - status: "True"
    type: ManagedClusterConditionAvailable
  clusterSelector: ③
    matchExpressions:
    - {key: vendor, operator: In, values: ["OpenShift"]}
```

- ① The PlacementBinding object binds the rules to the policy.
- ② The PlacementRule object defines the schedule conditions and the target cluster for the policy.
- ③ The clusterSelector field specifies a cluster or set of clusters for policy placement.

Deploying the E8 Policy Scan from the Console

RHACM enables administrators to create and deploy the E8 policy across a fleet of clusters. To deploy the compliance policy, perform the following steps:

1. Clone the policy collection repository. The policy-collection repository has policy examples for Open Cluster Management.
2. Edit the policy-compliance-operator-e8-scan.yaml file.
3. Deploy the E8-scan policy by using the following command:

```
[user@host]$ oc create -f policy-compliance-operator-e8-scan.yaml
policy.policy.open-cluster-management.io/policy-e8-scan created
placementbinding.policy.open-cluster-management.io/binding-policy-e8-scan created
placementrule.apps.open-cluster-management.io/placement-policy-e8-scan created
```

View the Details of an E8 Policy Scan from the Console

1. On the left pane, click **Governance**. Click the **Policies** tab. The dashboard displays the policy-e8-scan policy.
2. Click **policy-e8-scan**, and under Placement, select the cluster link to check the status and review the details.

References

What Is Compliance Management?

ComplianceAsCode/Content Project

OpenSCAP Tools, Security Policies, and Standards

OpenScap

For an overview of the compliance operator, refer to the *Compliance Operator Policy Overview* section in the *Red Hat Advanced Cluster Management for Kubernetes Governance Guide* at https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/governance/index#compliance-operator-policy-overview

For more information about the compliance operator, refer to the *Understanding the Compliance Operator* section in the *Red Hat OpenShift documentation* at https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/security_and_compliance/index#understanding-compliance-operator

For more information about E8 scan policy, refer to the *E8 Scan Policy* section in the *Red Hat Advanced Cluster Management for Kubernetes Governance Guide* at https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/governance/index#e8-scan-policy

3.8. Guided Exercise

Deploy and Configure the Compliance Operator for Multiple Clusters by Using RHACM

Deploy the compliance operator on all the clusters in the Asia-Pacific (APAC) location to verify conformance to the Essential 8 (E8) standard; and review the compliance report in the RHACM web console.

Outcomes

- Install the compliance operator from the RHACM governance dashboard.
- Deploy the E8 scan policy.
- Check the compliance scan result from E8 scan.

Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your system for this exercise.

```
[student@workstation ~]$ lab install do0013l ; lab start policies-compliance
```

Instructions

1. Log in to the OpenShift hub cluster and create the `policies-compliance` project.

- 1.1. In a terminal window, log in to the RHOCP cluster as the `admin` user with `redhatocp` as the password.

```
[student@workstation ~]$ oc login -u admin -p redhatocp \
https://api.ocp4.example.com:6443
Login successful.
...output omitted...
```

- 1.2. Create the `policies-compliance` project.

```
[student@workstation ~]$ oc new-project policies-compliance
Now using project "policies-compliance" on server "https://api.ocp4.example.com:6443".
...output omitted...
```

2. Place the `managed-cluster` cluster into a cluster set with `apac` as the name.

- 2.1. From the workstation machine, open Firefox and access `https://console-openshift-console.apps.ocp4.example.com`

- 2.2. Click **Red Hat Identity Management** and log in as the `admin` user with `redhatocp` as the password.

- 2.3. Ensure that the cluster switcher, at the top of the console, is set to **All Clusters**, which displays the RHACM web console.

- 2.4. Select **Infrastructure > Clusters**.

- 2.5. Click **Cluster sets**. Click **Create cluster set**.

- 2.6. Enter `apac` for the `Cluster set name` field. Click **Create**. Click **Manage resource assignments**.

2.7. Select managed-cluster.

Red Hat OpenShift

All Clusters

4

admin

Cluster sets

apac

Manage resource assignments

Manage resource assignments

Select resources to toggle their assignments to the cluster set

Resources can be added, removed, and transferred from other cluster sets (if you have permissions to remove from them from their assigned set).

Important: assigning a resource to the cluster set will give all cluster set users permissions to the resource's namespace.

1 selected

Search

1 - 3 of 3

Name	Infrastructure	Distribution version	Labels	Nodes	Current cluster set
☐ local-cluster	Other	OpenShift 4.18.4	openshiftV... openshiftV... velero.io/ex... 15 more	1	default
☒ managed-cluster	Other	OpenShift 4.18.4	openshiftV... openshiftV... 14 more	1	apac

2.8. Click **Review**. In the confirmation dialog box, click **Save**.

2.9. Click the pencil icon to edit the namespace bindings.

Red Hat OpenShift

All Clusters

3

admin

Cluster sets

apac

apac

Overview

Submariner add-ons

Cluster list

Cluster pools

User management

Details

Name

apac

Multicluster network status

Add clusters

Namespace bindings

User management

None

2.10. Select policies-compliance from the Namespaces drop-down list.

Edit namespace bindings

A **ManagedClusterSetBinding** resource binds a **ManagedClusterSet** resource to a namespace. Placement resources that are created in the same namespace can only access managed clusters that are included in the bound **ManagedClusterSet** resource.

Note: Only namespaces in which you have permission to create or delete the **ManagedClusterSetBinding** are shown. This permission is required in addition to permission to bind the **ManagedClusterSet**.

Namespaces

policies-compliance

Select namespaces

Save

Cancel

Click **Save**.

Note

To use the cluster set within the namespace, a binding must exist from the cluster set to the namespace.

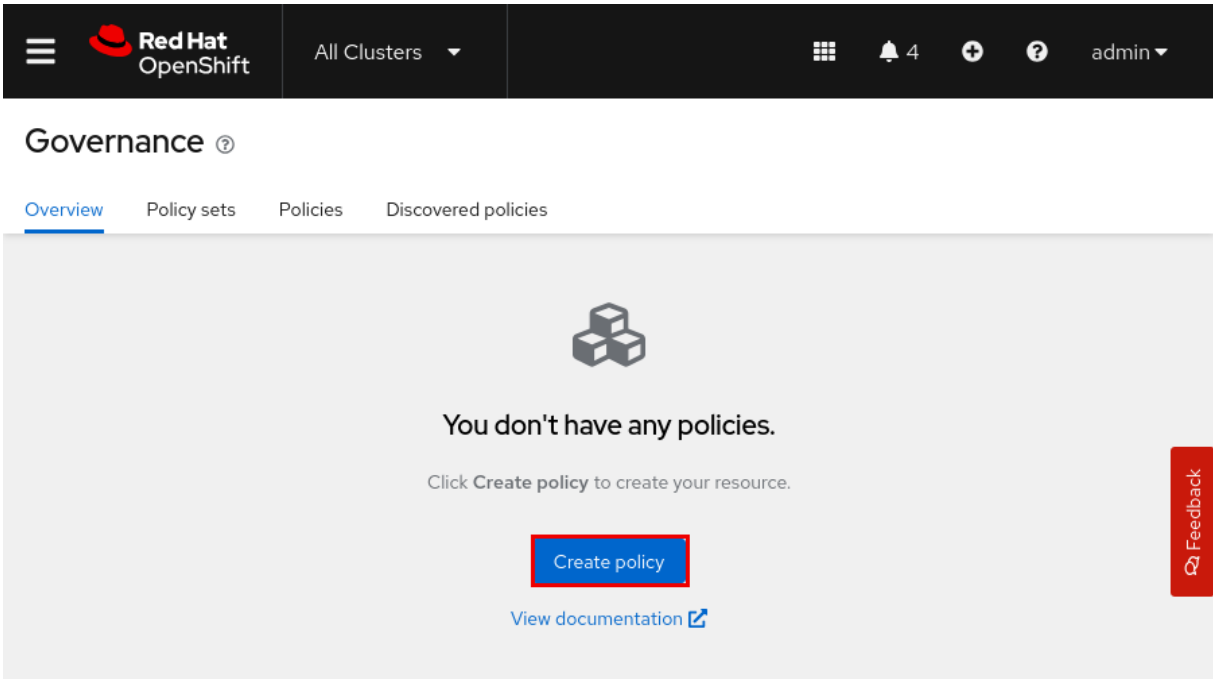
3. Create the compliance operator policy with the following parameters:

Field name	Value
Name	policy-complianceoperator
Namespace	policies-compliance
Cluster set	apac
Remediation	Inform

3.1. In the left pane, click **Governance** to display the governance dashboard, and click **Create policy**. The Create policy page is displayed.

Page 185 | 475

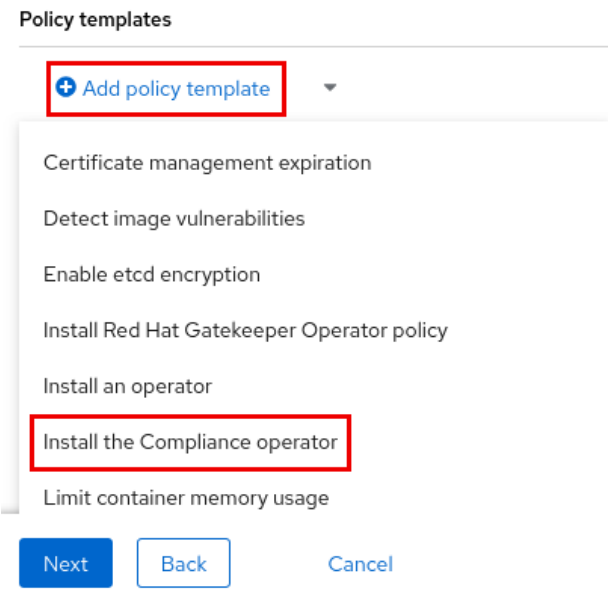
DO432-RHACM2.13-en-2-20260109



3.2. Complete the Details page with the following values. Leave the Disable policy field unchanged.

Field name	Value
Name	policy-complianceoperator
Namespace	policies-compliance

3.3. Click **Next**. On the Templates page, select Inform. Under the Policy templates label, click **Add policy template**.



Select Install the Compliance operator.

3.4. Scroll down to Operator Subscription and change the Source field to gls-catalog-cs.

Installed Namespace * ?

openshift-compliance

▼ Operator Subscription

Name * ?

compliance-operator

Channel ?

Enter the channel

Source ?

gls-catalog-cs

Source Namespace ?

openshift-marketplace

Starting CSV ?

Next

Back

Cancel

Feedback

Note

The source change is required only for the lab environment, which uses an offline operator catalog named gls-catalog-cs.

Click **Next**.

3.5. On the Placement page, click **New placement**.

Red Hat OpenShift

All Clusters ▼

4

admin ▼

Policies ▶ Create policy

Create policy

YAML

A policy generates reports and validates cluster violations based on specified security standards, categories, and controls.

3 Placement ▼

Placement

Use Placement resources to select clusters from the cluster sets that you have bound to the resource namespace. An empty Placement returns all available clusters from all bound cluster sets.

How do you want to select clusters?

New placement

Existing placement

No placement

Name *

Next

Back

Cancel

Feedback

3.6. From the cluster sets field, select apac.

Page 187 | 475

DO432-RHACM2.13-en-2-20260109

Placement

Use Placement resources to select clusters from the cluster sets that you have bound to the resource namespace. An empty Placement returns all available clusters from all bound cluster sets.

How do you want to select clusters?

New placementExisting placementNo placement

Name *

policy-complianceoperator-placement

The name of the placement should match the placement name in a placement binding so that it is bound to a policy or policy set. The placement name must be unique to the namespace.

Cluster sets ?

apac1

☒ apac

Add cluster set

Next

Back

Cancel

Click **Next**.

3.7. On the Policy annotations page, click **Next**. On the Review page, click **Submit**.

Red Hat

OpenShift

All Clusters ▾

4

admin ▾

Policies ▸ policy-complianceoperator

policy-complianceoperator

DetailsResultsActions ▾

Policy details

Name	policy-complianceoperator	Categories	CA Security Assessment and Authorization
Description	ComplianceOperator - Install the Compliance operator	Controls	CA-2 Security Assessments, CA-7 Continuous Monitoring
Namespace	policies-compliance	Standards	NIST SP 800-53
Status	Enabled	Created	May 20, 2025, 1:16 PM
Remediation	inform	Automation	Configure
Cluster violations	1		

Placement

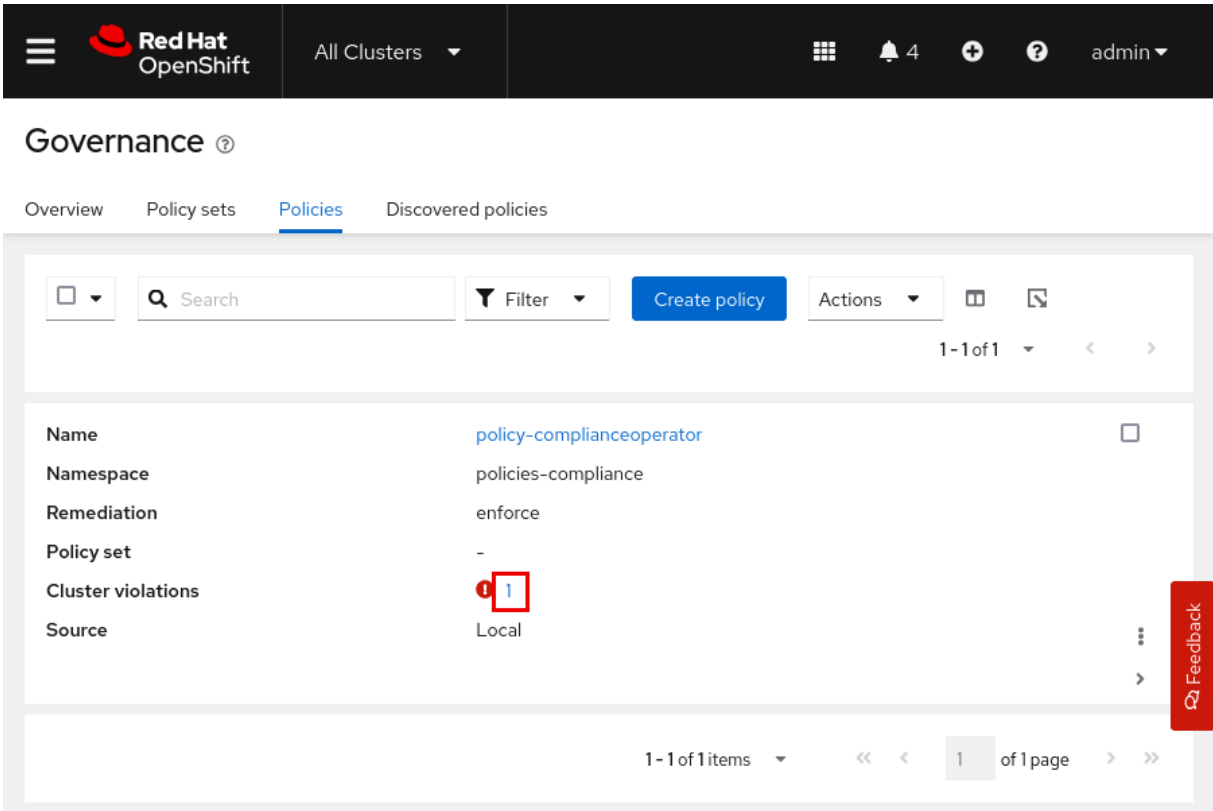
Name ↑	Kind ↑	Clusters	Violations
policy-complianceoperator-placement	Placement	1	1 Violations: managed-cluster

Feedback

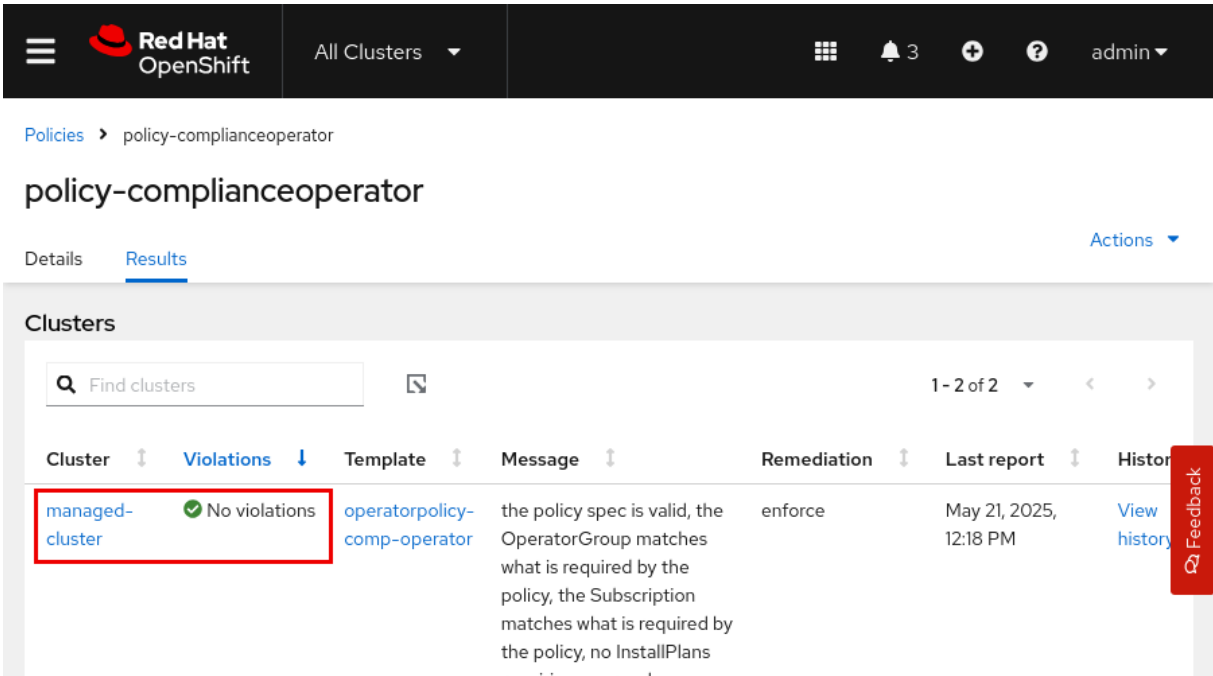
The policy status shows a cluster violation for the cluster named managed-cluster. The violation indicates that the compliance operator is not installed on the managed-cluster.

- 3.8. On the left pane, click **Governance**. Click the **Policies** tab. The dashboard shows the policy-complianceoperator policy. Click the vertical ellipsis icon (⋮) to the right of the policy field, and click **Enforce**. Click **Enforce** again in the confirmation window to change policy remediation from Inform to Enforce.

Enforce mode installs the compliance operator on managed-cluster, and policy-complianceoperator changes to Compliant status. Click 1 for the violation to see details for the cluster update.



3.9. The violation clears after the compliance operator is installed.



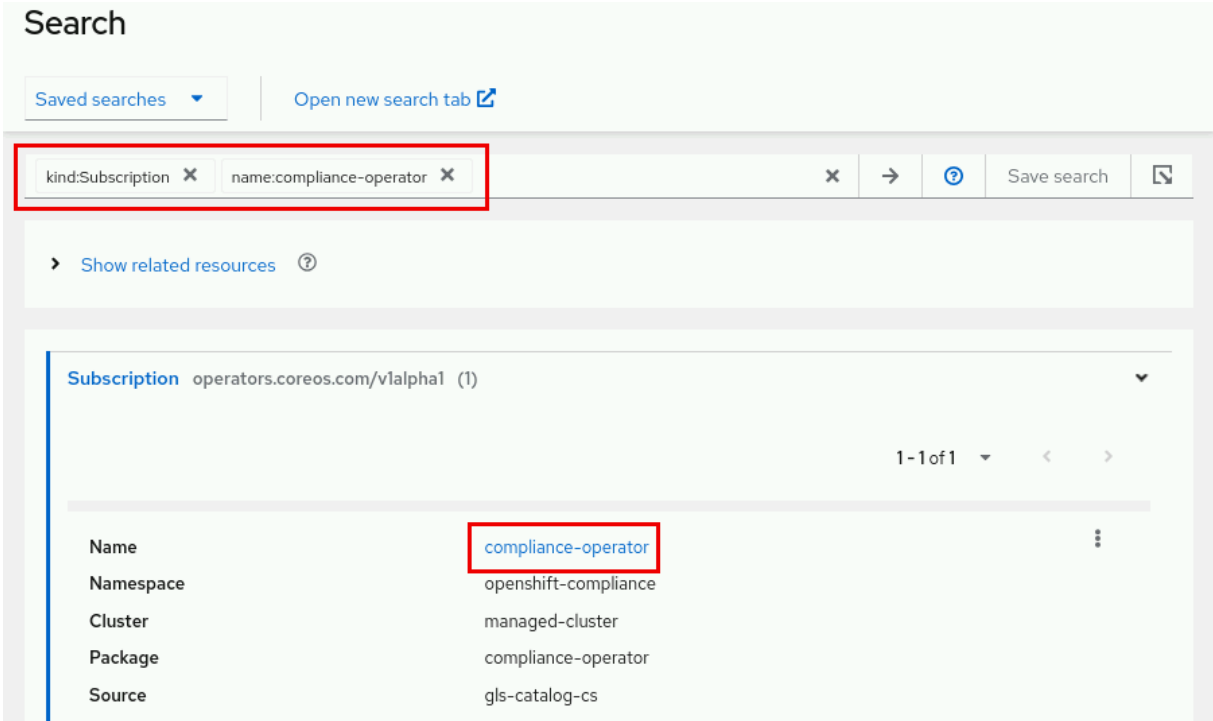
4. As the admin user, verify the compliance operator installation by using the RHACM web console.

- 4.1. On the left pane, click **Search**.
- 4.2. Type the following parameters in the search bar to search for the compliance-operator subscription.

Filter	Value
kind	subscription

Filter	Value
name	compliance-operator

The search result shows the compliance-operator subscription in the managed cluster.



5. Deploy the Essential 8 (E8) scan policy to the APAC location.

5.1. Open a terminal on the workstation machine. Change to the ~/D00013L/labs/policies-compliance/ directory.

```
[student@workstation ~]$ cd ~/D00013L/labs/policies-compliance/
```

5.2. Edit the policy-compliance-operator-e8-scan.yaml file. In the policy-e8-scan policy, set the spec.remediationAction field to enforce. At the bottom of the file, in the placement-policy-e8-scan placement rule, set the matchLabels field to cluster.open-cluster-management.io/clusterset: apac.

```
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: policy-e8-scan
  annotations:
    policy.open-cluster-management.io/standards: NIST SP 800-53
    policy.open-cluster-management.io/categories: CM Configuration Management
    policy.open-cluster-management.io/controls: CM-6 Configuration Settings
spec:
  remediationAction: enforce
  disabled: false
  ...output omitted...

  ...output omitted...
spec:
  clusterConditions:
    - status: "True"
      type: ManagedClusterConditionAvailable
  clusterSelector:
    matchLabels:
      cluster.open-cluster-management.io/clusterid: apac
```

5.3. Deploy the E8-scan policy in the policies-compliance namespace.

```
[student@workstation policies-compliance]$ oc create -f \
policy-compliance-operator-e8-scan.yaml -n policies-compliance
policy.policy.open-cluster-management.io/policy-e8-scan created
placementbinding.policy.open-cluster-management.io/binding-policy-e8-scan created
placementrule.apps.open-cluster-management.io/placement-policy-e8-scan created
```

5.4. Change to the home directory.

```
[student@workstation policies-compliance]$ cd ~
[student@workstation ~]$
```

6. Check the compliance operator E8 scan results.

- 6.1.** In the left pane, click **Governance**. Click the **Policies** tab. In the dashboard, click **policy-e8-scan**. Clusters that violate the policy appear under Placement.

Red Hat OpenShift

All Clusters

4

admin

Policies

policy-e8-scan

policy-e8-scan

Actions

DetailsResults

Policy details

Name	policy-e8-scan	Categories	CM Configuration Management
Description	-	Controls	CM-6 Configuration Settings
Namespace	policies-compliance	Standards	NIST SP 800-53
Status	Enabled	Created	May 21, 2025, 3:03 PM
Remediation	enforce	Automation	Configure
Cluster violations	1		

Placement

Name	Kind	Clusters	Violations
placement-policy-e8-scan	PlacementRule	1	1 Violations: managed-cluster

6.2. Click the **managed-cluster** link. The results show three resources: compliance-e8-scan, compliance-suite-e8, and compliance-suite-e8-result. The Compliance-suite-e8-result shows a violation.

Page 193 | 475

DO432-RHACM2.13-en-2-20260109

Red Hat

OpenShift

All Clusters ▾

4

admin ▾

Policies ▸ policy-e8-scan

policy-e8-scan

Details Results Actions ▾

Clusters

Q managed-cluster 3 / 3 X

1 - 3 of 3 ▾

Cluster ↑	Violations ↑	Template ↑	Message ↑	Remediation ↑	Last report ↑	History
managed-cluster	✔ No violations	compliance-e8-scan	notification - scansettingbindings [e8] found as specified in namespace openshift-compliance - View details	enforce	May 21, 2025, 3:14 PM	View history
managed-cluster	✔ No violations	compliance-suite-e8	notification - compliancesuites [e8] found as specified in namespace openshift-compliance - View details	enforce	May 21, 2025, 3:16 PM	View history
managed-cluster	❗ Violations	compliance-suite-e8-results	violation - compliancecheckresults [ocp4-e8-ocp-allowed-registries-for-import, ocp4-e8-api-server-tls-cipher-suites, ocp4-e8-ocp-allowed-registries, ocp4-e8-api-server-encryption-provider-cipher, ocp4-e8-	enforce	May 21, 2025, 3:16 PM	View history

Feedback

Note

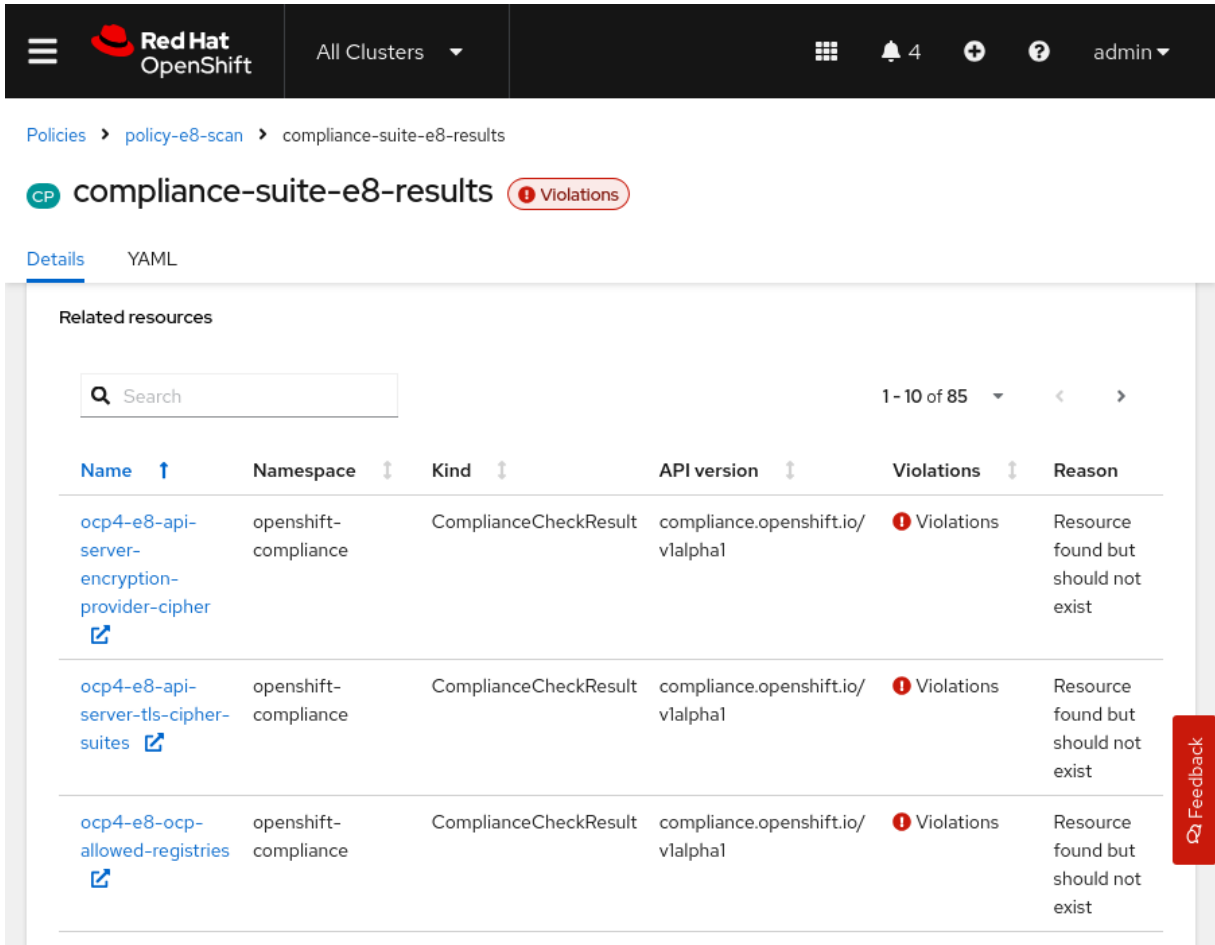
It takes 2-3 minutes for Compliance-suite-e8-result to show the violation.

The compliance-suit-e8-result object is noncompliant because the policy applies the mustnothave condition to the ComplianceCheckResult object.

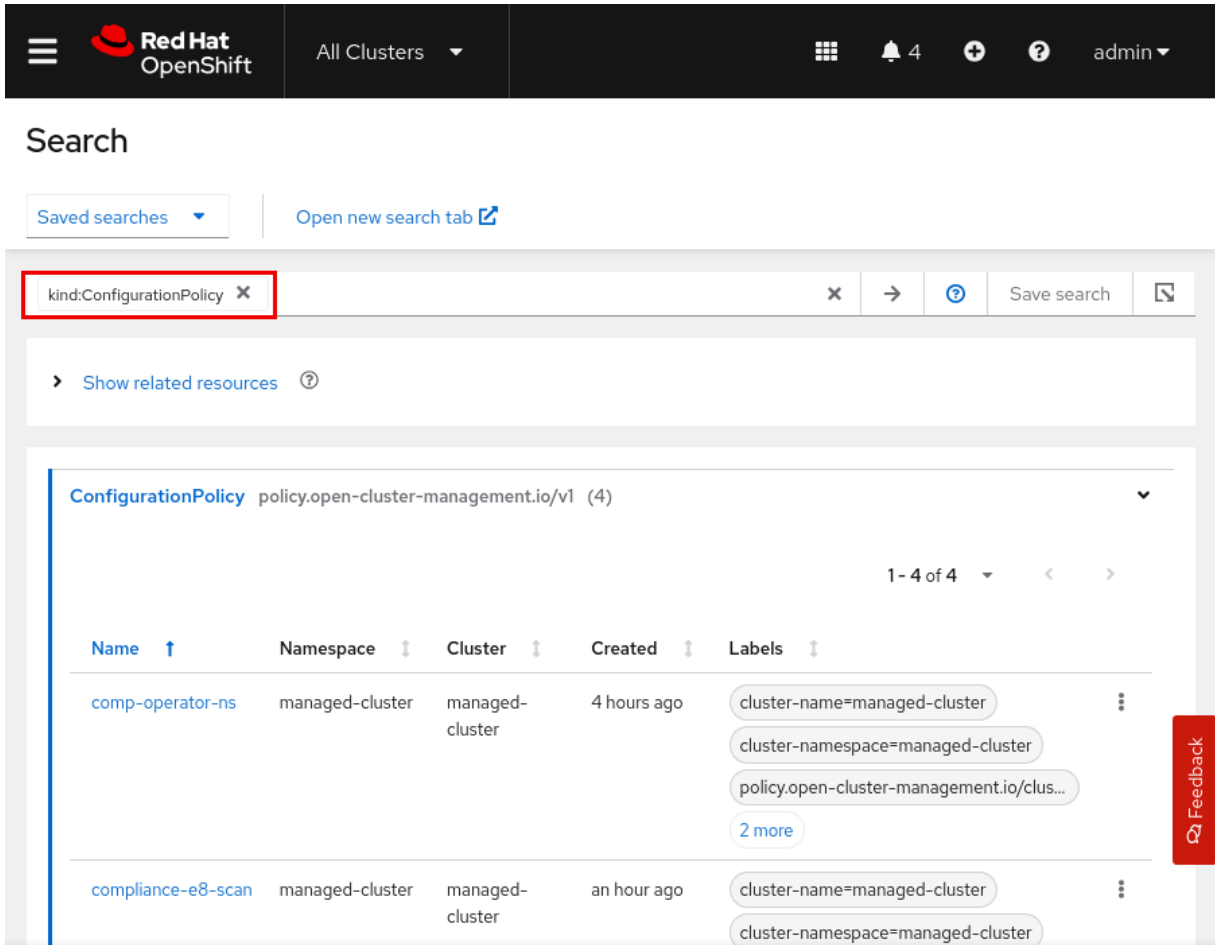
6.3. Click the **Compliance-suite-e8-result** link and review the details. On the details page, you can see the name of the noncompliant objects.

Page 194 | 475

DO432-RHACM2.13-en-2-20260109



- 6.4. To list the compliance policies, click **Search** in the RHACM web console and type `kind:configurationpolicy`, and press **Enter**.
- 6.5. The search results show a list of all the configuration policies.



Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish policies-compliance
```


3.9. Integration of Other Policy Engines with RHACM

Objectives

- Deploy the gatekeeper policy and gatekeeper constraints to multiple clusters by using the command line and the RHACM governance dashboard.

Open Policy Agent and Open Policy Agent Gatekeeper

Implementing policies is a challenging task. It usually requires policy management software to implement the policies, a policy language for writing the rules, and a policy engine to evaluate the policies.

Open Policy Agent (OPA)

OPA is an open source policy engine that unifies policy enforcement across the stack. OPA uses a high-level declarative language named Rego to write the policies, which removes the policy decision-making load from the application. OPA is a general-purpose policy engine, to enforce policies in microservices, Kubernetes, CI/CD pipelines, API gateways, and more.

OPA gatekeeper

The OPA gatekeeper is a use case of OPA for Kubernetes environments. It is built to work as a Kubernetes admission controller webhook. In Kubernetes, when an API request is authenticated and authorized, the Kubernetes admission controllers process and validate the API request. If the request is not compliant with the defined policies, then an admission controller can block it.

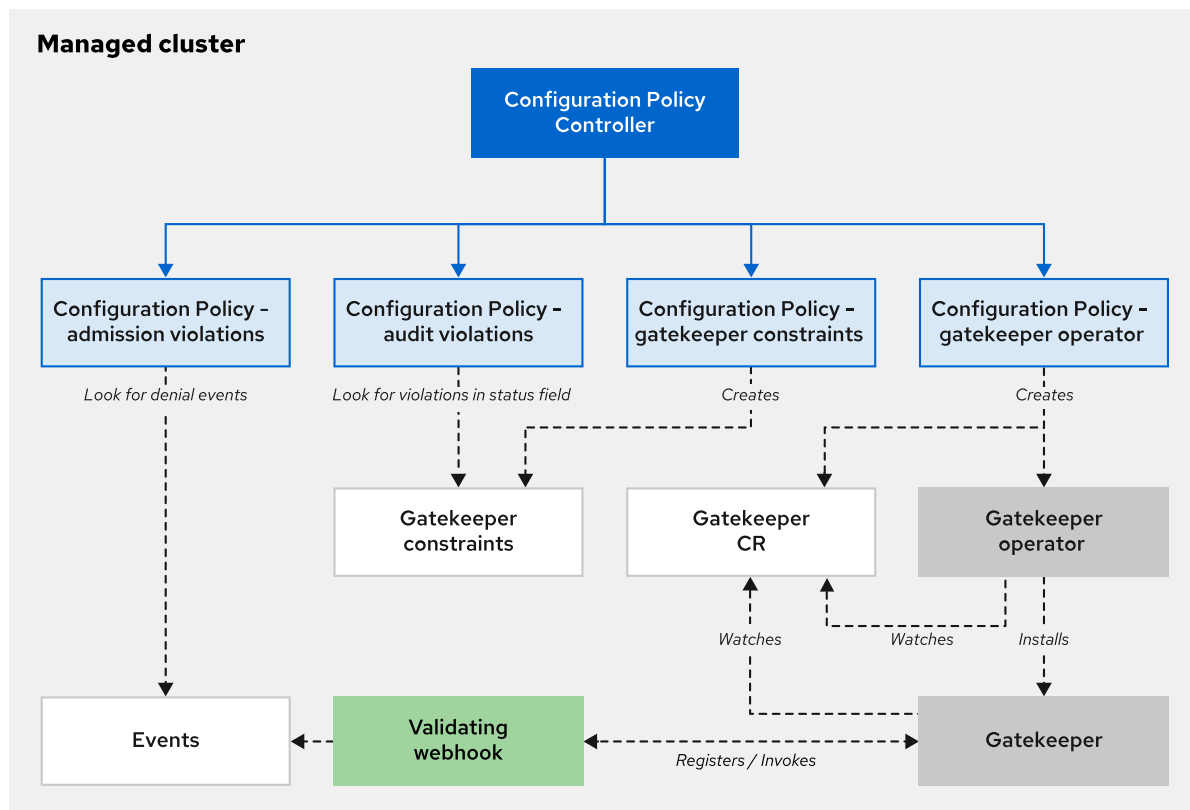
OPA Gatekeeper Integration with RHACM

RHACM provides a built-in policy to deploy the OPA gatekeeper operator. This way, RHACM can use OPA gatekeeper as a policy controller and you can use constraints and constraint templates to define admission and audit templates for multiple clusters.

OPA gatekeeper integration with RHACM is a two-step process.

The first step is to deploy the gatekeeper operator and to create the gatekeeper custom resource. Next, you can create constraint, audit, and admission templates. The Kubernetes configuration policy controller creates all the policy templates.

The following diagram provides an overview of how the gatekeeper policy engine integrates with the RHACM configuration policy controller.



Installing the Gatekeeper Operator by Using RHACM Governance

With RHACM, you can use a policy to install the Gatekeeper operator by using the governance framework. The policy installs the Gatekeeper operator in the targeted cluster or set of clusters based on a placement and creates the Gatekeeper custom resource.

The RHACM configuration policy controller checks the Gatekeeper operator policy and supports the enforce remediation action. When you set the controller to enforce, it automatically creates the Gatekeeper operator objects on the managed clusters.

To create the policy from the RHACM web console, go to **Governance** and click **Create policy**.

On the **Create policy** page, you define the policy name and the namespace on the hub cluster to create the policy resources.

Create policy

YAML

A policy generates reports and validates cluster violations based on specified security standards, categories, and controls.

1Details

2Policy templates

3Placement

4Policy annotations

5Review

Details

Name *

policy-gatekeeper-operator

Description

Enter the description

Namespace *

policy-gatekeeper

The namespace on the hub cluster where the policy resources will be created.

☐ Disable policy

Select to disable the policy from being propagated to managed clusters.

Figure 17. Policy details

In the **Policy templates** section, you set **Remediation** to **Enforce**, and then you add the **Install Red Hat Gatekeeper Operator policy** policy template.

1Details

2Policy templates

3Placement

4Policy annotations

5Review

Templates

A policy contains policy templates that create policies on managed clusters.

Remediation ?

☐ Inform

Reports the violation, which requires manual remediation.

☒ Enforce

Automatically runs remediation action that is defined in the source, if this feature is supported.

☐ Use policy template remediation

Remediation action will be determined by what is set in the policy template definitions.

Policy templates

+ Add policy template

Certificate management expiration

Detect image vulnerabilities

Enable etcd encryption

Install Red Hat Gatekeeper Operator policy

Install an operator

Figure 18. Policy templates

The policy template, which defines the operator installation details, installs the Gatekeeper operator on the managed clusters. You select the default installation mode, and review the default values for the operator subscription.

Page 199 | 475

DO432-RHACM2.13-en-2-20260109

Name *

operatorpolicy-gatekeeper-operator

Name needs to be unique to the namespace on each of the managed clusters.

Installation Mode *

☒ All namespaces on the cluster (default)

☐ A specific namespace on the cluster

Installed Namespace ?

openshift-operators

Operator Subscription

Name * ?

gatekeeper-operator-product

Channel ?

stable

Source ?

redhat-operators

Source Namespace ?

openshift-marketplace

Figure 19. Gatekeeper operator details

The **Placement** section creates or selects a placement to select the clusters from the cluster sets that you bound to the resource namespace. In the following example, you create a placement named `policy-gatekeeper-operator-placement` that uses the default cluster set.

1 Details

2 Policy templates

3 Placement

4 Policy annotations

5 Review

Placement

Use Placement resources to select clusters from the cluster sets that you have bound to the resource namespace

How do you want to select clusters?

New placement

Existing placement

No placement

Name *

policy-gatekeeper-operator-placement

The name of the placement should match the placement name in a placement binding so that it is bound to a

Cluster sets ?

default

1

If no cluster sets are selected, all clusters will be selected from the cluster sets bound to the namespace.

Label expressions ?

+ Add label expression

Figure 20. Policy placement

The **Policy annotations** section defines the annotations for the policy.

1

Details

2

Policy templates

3

Placement

4

Policy annotations

5

Review

Policy annotations

Standards ?

NIST SP 800-53

+ Add

Categories ?

CM Configuration Management

+ Add

Controls ?

CM-2 Baseline Configuration

+ Add

Figure 21. Policy annotations

To create the policy, go to the **Review** section and click **Submit**.

1

Details

2

Policy templates

3

Placement

4

Policy annotations

5

Review

Details

Name

policy-gatekeeper-operator

Namespace

policy-gatekeeper

Templates

Remediation

Enforce

Policy templates

operatorpolicy-gatekeeper-operator

gatekeeper

gatekeeper-status

Placement

Submit

Back

Cancel

Figure 22. Policy review

The policy installs the Gatekeeper operator in the managed clusters. To verify the operator installation, in the OpenShift console of the managed clusters, go to **Operators > Installed Operators** and select **All Projects**. The list of operators shows the Gatekeeper operator.

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

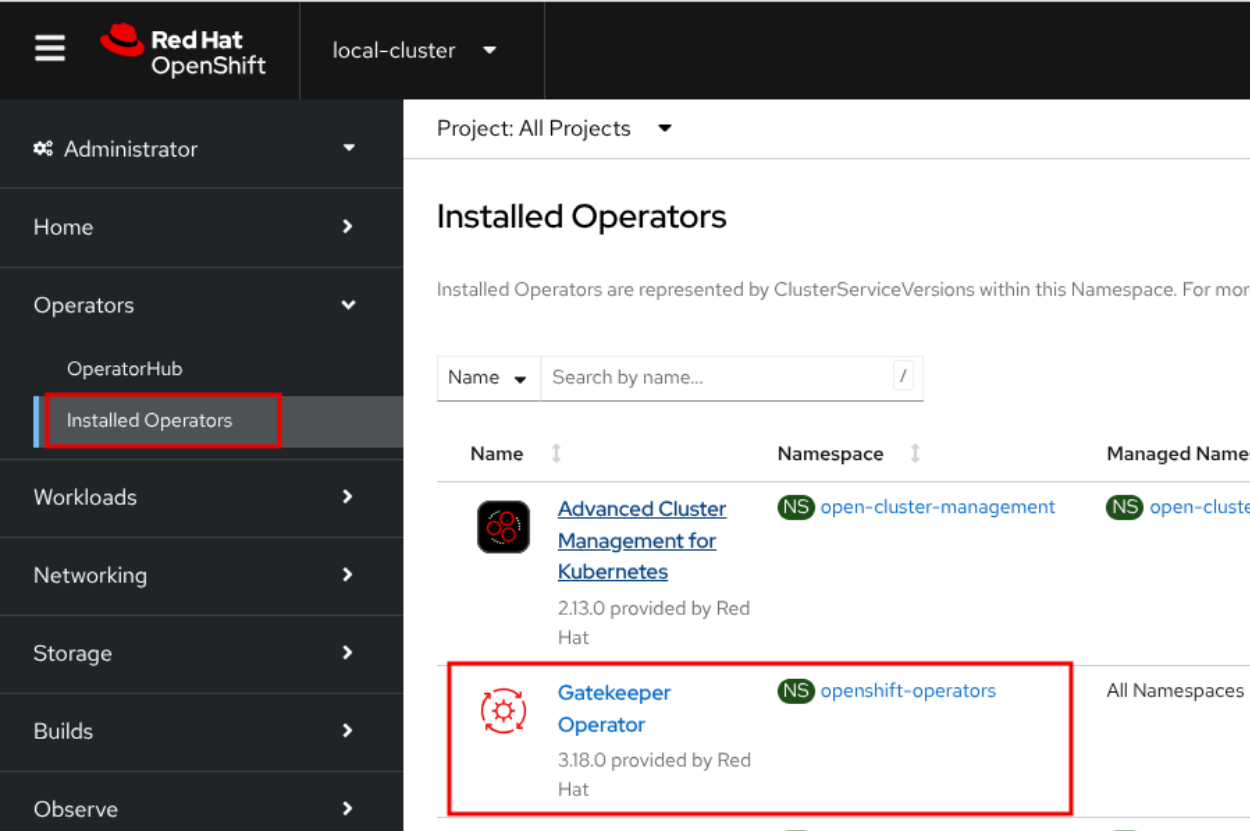


Figure 23. Installed operators

In the RHACM web console, go to the **Governance** page and click the **Policies** tab. Click the **policy-gatekeeper-operator** policy, and click the **Results** tab. Three policy templates appear for each managed cluster.

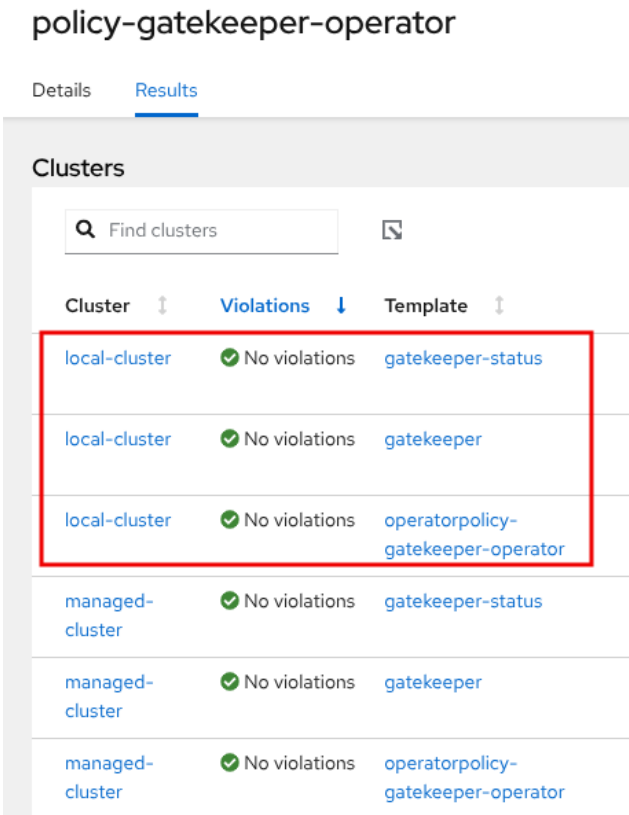


Figure 24. Policy results

Gatekeeper Custom Resource Configuration

You can edit the Gatekeeper object to exclude namespaces from certain processes for all constraints on your hub cluster.

The following config section excludes the namespaces that start with test -, and the my -namespace namespace from all processes.

```
apiVersion: operator.gatekeeper.sh/v1alpha1
kind: Gatekeeper
metadata:
  name: gatekeeper
  ...output omitted...
spec:
  audit:
    logLevel: INFO
    replicas: 1
  mutatingWebhook: Disabled
  validatingWebhook: Enabled
  webhook:
    emitAdmissionEvents: Enabled
    logLevel: INFO
    replicas: 2
  config: ①
    matches:
      - excludedNamespaces: ②
        - test-*
        - my-namespace
    processes: ③
      - '*'
  disableDefaultMatches: false ④
```

- ① The config section
- ② The list of namespaces to exclude

- ③ The list of processes, with valid values of audit, webhook, sync, mutation-webhook, and * to include all listed processes and any future processes
- ④ This parameter disables appending the default exempt namespaces that the Gatekeeper operator provides. The default exempt namespaces are RHOCP or Kubernetes system namespaces. This parameter is set to false by default to append the default namespaces.

The Gatekeeper operator updates the Config resource within the openshift-gatekeeper-system namespace, and appends the namespaces to exclude.

```
[student@workstation ~]$ oc get configs config \
  -n openshift-gatekeeper-system -oyaml
apiVersion: config.gatekeeper.sh/v1alpha1
kind: Config
metadata:
  name: config
  ...output omitted...
spec:
  match:
    - excludedNamespaces: ①
      - kube-*
      - multicluster-engine
      - hypershift
      - hive
      - rhacs-operator
      - open-cluster-*
      - openshift-*
    processes:
      - webhook
      - mutation-webhook
    - excludedNamespaces: ②
      - test-*
      - my-namespace
    processes:
      - '*'
  ...output omitted...
```

- ① The default exempt namespaces that the Gatekeeper operator provides
- ② The list of namespaces to exclude that the Gatekeeper object defines

Deploying Constraints by Using Gatekeeper Policies

You can deploy gatekeeper policies when the gatekeeper operator is deployed to the targeted clusters. To create Gatekeeper policies, use constraint templates and constraints. Add constraint templates and constraints to the policy-templates section of a Policy resource.

The ConstraintTemplate object defines the schema for the Constraint object, and contains the Rego code to evaluate the policy violation. The Constraint object provides the value of the parameters that are defined in ConstraintTemplate object.

The following YAML example defines a Policy resource with a ConstraintTemplate object named k8srequiredlabels and a constraint named ns-must-have-gk to ensure that the gatekeeper label is set on the test-labels-ns namespace:


```

apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: require-gatekeeper-labels-on-ns
spec:
  remediationAction: inform ①
  disabled: false
  policy-templates:
    - objectDefinition: ②
      apiVersion: templates.gatekeeper.sh/v1beta1
      kind: ConstraintTemplate
      metadata:
        name: k8srequiredlabels
        annotations:
          policy.open-cluster-management.io/severity: low ③
      spec:
        crd:
          spec:
            names:
              kind: K8sRequiredLabels
            validation:
              openAPIV3Schema: ④
                properties:
                  labels:
                    type: array
                    items: string
        targets:
          - target: admission.k8s.gatekeeper.sh
            rego: | ⑤
              package k8srequiredlabels
              violation[{"msg": msg, "details": {"missing_labels": missing}}] {
                provided := {label | input.review.object.metadata.labels[label]}
                required := {label | label := input.parameters.labels[_]}
                missing := required - provided
                count(missing) > 0
                msg := sprintf("you must provide labels: %v", [missing])
              }
    - objectDefinition: ⑥
      apiVersion: constraints.gatekeeper.sh/v1beta1
      kind: K8sRequiredLabels
      metadata:
        name: ns-must-have-gk
        annotations:
          policy.open-cluster-management.io/severity: low
      spec:
        enforcementAction: dryrun ⑦
        match:
          kinds:
            - apiGroups: [""]
              kinds: ["Namespace"]
          namespaces: ⑧
            - test-labels-ns
        parameters: ⑨
          labels: ["gatekeeper"]

```

- ① Specifies the remediation of the policy. The valid values are enforce and inform. When the remediationAction is set to inform, the enforcementAction field of the Gatekeeper constraint is overridden to warn. Gatekeeper detects and warns you about creating or updating a namespace that is missing the gatekeeper label. If the remediationAction policy is set to enforce, then the Gatekeeper constraint enforcementAction field is overridden to deny. Gatekeeper prevents any user from creating or updating a namespace that is missing the gatekeeper label.
- ② The ConstraintTemplate object definition
- ③ Optional, sets the severity value for each Gatekeeper constraint or constraint template. The valid values are low, medium, high, or critical.
- ④ The schema for the parameters field

- ⑤ The Rego code to evaluate the policy violation
- ⑥ The Constraint object definition
- ⑦ Sets the constraint enforcement action. The valid values are dry-run, warn, or deny.
- ⑧ The list of namespaces to apply the constraint
- ⑨ The value of the parameters that are defined in the ConstraintTemplate object

With the previous policy, you receive the following policy status message if you create the test-labels-ns namespace without the gatekeeper label:

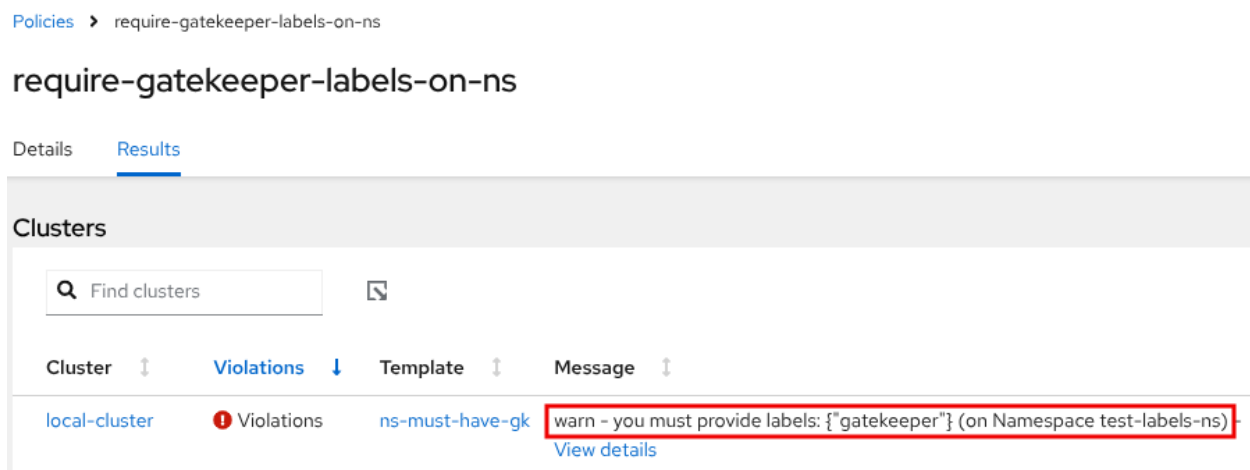


Figure 25. Policy violation

Gatekeeper Admission Webhook

Use the policy-gatekeeper-admission configuration policy within a RHACM policy to check for Kubernetes API requests that are denied by the Gatekeeper admission webhook.

The object template defines parameters such as event_type, constraint_name, constraint_kind, and constraint_action, and watches for specific events in the targeted clusters.

The following YAML example creates a ConfigurationPolicy object named policy-gatekeeper-admission:

```
...output omitted...
- objectDefinition:
  apiVersion: policy.open-cluster-management.io/v1
  kind: ConfigurationPolicy
  metadata:
    name: policy-gatekeeper-admission
  spec:
    remediationAction: inform ①
    severity: low
    object-templates:
      - complianceType: mustnothave ②
        objectDefinition:
          apiVersion: v1
          kind: Event
          metadata:
            namespace: openshift-gatekeeper-system ③
            annotations:
              constraint_action: deny ④
              constraint_kind: K8sRequiredLabels ⑤
              constraint_name: ns-must-have-gk ⑥
              event_type: violation ⑦
```

- ① The configuration policy remediationAction parameter is overwritten by remediationAction in the parent policy.

- 2 Sets how objects on the cluster compare with the object definition of the configuration policy. The valid values are MustHave, MustOnlyHave, or MustNotHave.
- 3 The namespace where Gatekeeper runs
- 4 The constraint action
- 5 The constraint kind
- 6 The constraint name
- 7 The event type

If you create a test-labels-ns namespace without the gatekeeper label, then the Gatekeeper admission webhook triggers the following violation event:

Policies > require-gatekeeper-labels-on-ns

require-gatekeeper-labels-on-ns

DetailsResults

Clusters

Find clusters

Cluster	Violations	Template	Message
local-cluster	Violations	policy-gatekeeper-admission	violation - events [test-labels-ns.18410b56703f223e] found in namespace openshift-gatekeeper-system - View details
local-cluster	Violations	ns-must-have-gk	warn - you must provide labels: {"gatekeeper"} (on Namespace test-labels-ns) - View details

Figure 26. Gatekeeper admission webhook

References

Open Policy Agent

For more information about installing Gatekeeper by using a Gatekeeper operator policy in RHACM, refer to the *Installing Gatekeeper Using a Gatekeeper Operator Policy* section in the *Gatekeeper Operator Overview* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Governance* documentation at https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/governance/index#installing-gk-operator-policy

For more information about integrating Gatekeeper constraints and constraint templates in RHACM, refer to the *Integrating Gatekeeper Constraints and Constraint Templates* section in the *Gatekeeper Operator Overview* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Governance* documentation at https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/governance/index#integrate-gk-constraints-templates

3.10. Guided Exercise

Integrate Other Policy Engines with RHACM

Install the gatekeeper operator in all clusters and enforce all running containers in the production cluster not to use the latest tag.

Outcomes

- Use a policy to install the Gatekeeper operator by using the RHACM governance framework.
- Exclude a given namespace for all constraints.
- Deploy a gatekeeper policy to enforce containers not to use images with the latest tag.

Prerequisites

As the student user on the workstation machine, use the lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab install do0013l ; lab start policies-integration
```

Instructions

The lab start command labels the local-cluster cluster with the environment=stage label, and the managed-cluster cluster with the environment=production label.

1. Create the policy-gatekeeper namespace on the hub cluster.

1.1. In the terminal window, log in to the hub cluster as the admin user with redhatocp as the password.

```
[student@workstation ~]$ oc login -u admin -p redhatocp \
https://api.ocp4.example.com:6443
Login successful.
...output omitted...
```

1.2. Create the policy-gatekeeper project. RHACM uses the project to create the policy resources that install the Gatekeeper operator. You use the namespace in a later step.

```
[student@workstation ~]$ oc new-project policy-gatekeeper
Now using project "policy-gatekeeper" on server "https://api.ocp4.example.com:6443".
...output omitted...
```

2. Log in to the RHOC hub cluster, and switch to the RHACM web console.

2.1. Open a web browser and go to <https://console-openshift-console.apps.ocp4.example.com>

2.2. Select the **Red Hat Identity Management** identity provider and log in as the admin user with redhatocp as the password.

2.3. Switch to the RHACM web console and verify that the local-cluster and the managed-cluster clusters are in Ready status.

3. Bind the default managed cluster set to the policy-gatekeeper namespace.

3.1. From the RHACM web console, go to **Infrastructure > Clusters** and click the **Cluster sets** tab.

- 3.2. Click the vertical ellipsis icon (...) to the right of the default managed cluster set row, and click **Edit namespace bindings**.
- 3.3. From the **Edit namespace bindings** dialog box, select the policy-gatekeeper namespace from the drop-down list and click **Save**.

Edit namespace bindings ✕

A **ManagedClusterSetBinding** resource binds a **ManagedClusterSet** resource to a namespace. Placement resources that are created in the same namespace can only access managed clusters that are included in the bound **ManagedClusterSet** resource.

Note: Only namespaces in which you have permission to create or delete the **ManagedClusterSetBinding** are shown. This permission is required in addition to permission to bind the **ManagedClusterSet**.

Namespaces

policy-gatekeeper ✕ Select namespaces ✕ ▼

Save **Cancel**

4. Install the Gatekeeper operator by using RHACM governance.
 - 4.1. From the RHACM web console, go to **Governance** and click **Create policy**.
 - 4.2. At the top of the **Create policy** page, set the **YAML** toggle to on, to view the YAML file. The policy creation steps that you complete are automatically reflected in updates to the policy YAML file. Set the policy name to policy-gatekeeper-operator, and select the policy-gatekeeper namespace. Click **Next**.

Details

Name *

policy-gatekeeper-operator

Description

Enter the description

Namespace *

policy-gatekeeper

☐ Disable policy

Select to disable the policy from being propagated to managed clusters.

Next

Back

Cancel

Policy YAML

```
1  apiVersion: policy.open-cluster-management.io/v1
2  kind: Policy
3  metadata:
4    name: policy-gatekeeper-operator
5    namespace: policy-gatekeeper
6  spec:
7    disabled: false
8
```

4.3. In the **Policy templates** section, set **Remediation** to **Enforce**. Click **Add policy template** and select the **Install Red Hat Gatekeeper Operator policy** policy template.

1 Details

2 Policy templates

3 Placement

4 Policy annotations

5 Review

Templates

A policy contains policy templates that create policies on managed clusters.

Remediation ⓘ

☐ Inform

Reports the violation, which requires manual remediation.

☒ Enforce

Automatically runs remediation action that is defined in the source, if this feature is supported.

☐ Use policy template remediation

Remediation action will be determined by what is set in the policy template definitions.

Policy templates

+ Add policy template

Certificate management expiration

Detect image vulnerabilities

Enable etcd encryption

Install Red Hat Gatekeeper Operator policy

4.4. Ensure that the operator policy and the operator subscription have the following values, and click **Next**.

Operator policy

Name	operatorpolicy-gatekeeper-operator
Installation Mode	All namespaces on the cluster (default)
Installed Namespace	openshift-operators

1Details

2Policy templates

3Placement

4Policy annotations

5Review

Policy templates

operatorpolicy-gatekeeper-operator

Operator policy

An Operator policy creates operators on managed clusters.

Name *

operatorpolicy-gatekeeper-operator

Name needs to be unique to the namespace on each of the managed clusters.

Installation Mode *

All namespaces on the cluster (default)

A specific namespace on the cluster

Installed Namespace ?

openshift-operators

Operator Subscription

Name	gatekeeper-operator-product
Channel	stable
Source	gls-catalog-cs
Source Namespace	openshift-marketplace

Page 211 | 475

DO432-RHACM2.13-en-2-20260109

1

Details

2

Policy templates

3

Placement

4

Policy annotations

5

Review

Operator Subscription

Name *

gatekeeper-operator-product

Channel

stable

Source

gls-catalog-cs

Source Namespace

openshift-marketplace

Starting CSV

Enter the ClusterServiceVersion

Next

Back

Cancel

4.5. In the **Placement** section, click **New placement**. The name of the placement is policy-gatekeeper-operator-placement. Select the default cluster set from the **Cluster sets** drop-down list. Click **Next**.

1

Details

2

Policy templates

3

Placement

4

Policy annotations

5

Review

Placement

Use Placement resources to select clusters from the cluster sets that you have bound to the resource namespace. An empty Placement returns all available clusters from all bound cluster sets.

How do you want to select clusters?

New placement

Existing placement

No placement

Name *

policy-gatekeeper-operator-placement

The name of the placement should match the placement name in a placement binding so that it is bound to a policy or policy set. The placement name must be unique to the namespace.

Cluster sets

default

1

If no cluster sets are selected, all clusters will be selected from the cluster sets bound to the namespace.

Label expressions

+ Add label expression

Next

Back

Cancel

4.6. In the **Policy annotations** section, leave the default values and click **Next**.

1

Details

2

Policy templates

3

Placement

4

5

Policy annotations

Standards ?

NIST SP 800-53

+ Add

Categories ?

CM Configuration Management

+ Add

Controls ?

CM-2 Baseline Configuration

+ Add

Next

Back

Cancel

4.7. In the **Review** section, click **Submit** to create the policy.

1

Details

2

Policy templates

3

4

5

Details

Namepolicy-gatekeeper-operator

Namespacespolicy-gatekeeper

Templates

RemediationEnforce

Policy templates

operatorpolicy-gatekeeper-operator

gatekeeper

gatekeeper-status

Placement

Namepolicy-gatekeeper-operator-placement

Cluster setsdefault

Policy annotations

StandardsNIST SP 800-53

CategoriesCM Configuration Management

ControlsCM-2 Baseline Configuration

Submit

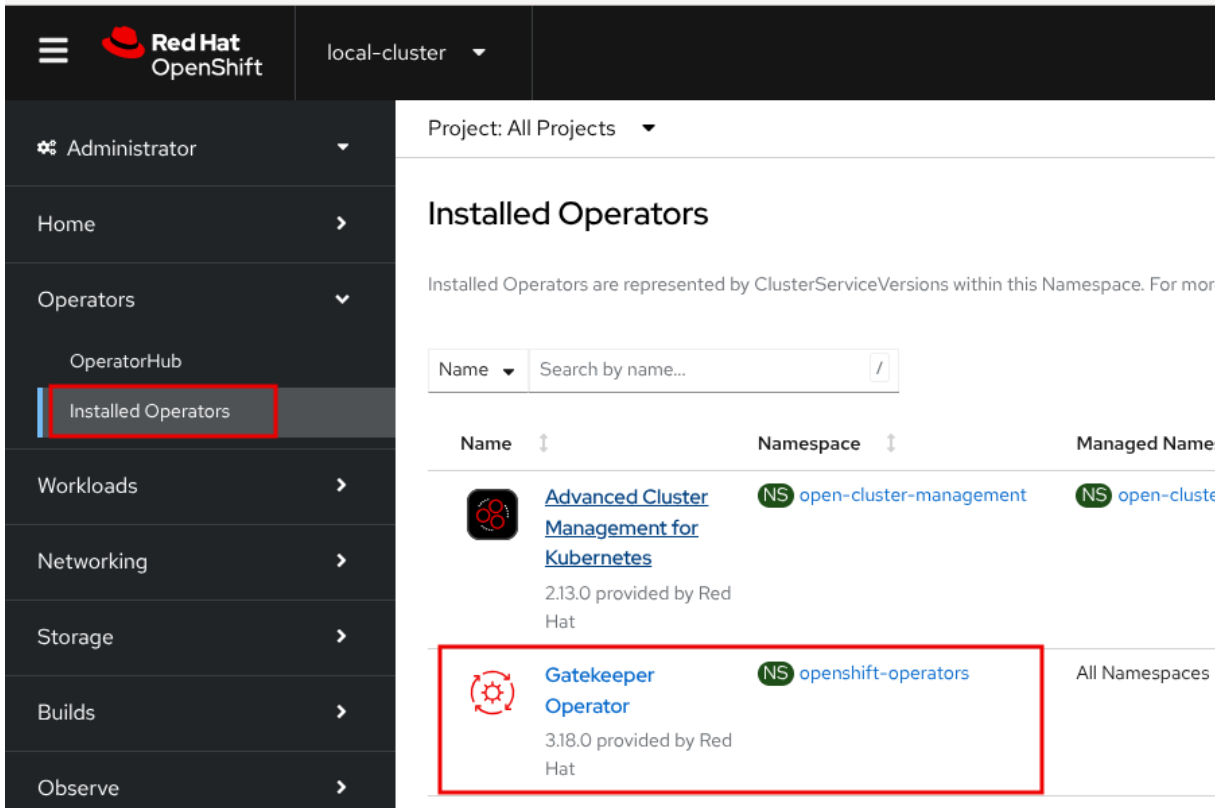
Back

Cancel

5. Verify the Gatekeeper operator installation on the managed clusters.
- 5.1. From the RHACM web console, set the cluster switcher to **local-cluster**, which corresponds to the OpenShift web console of the hub cluster.

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

5.2. Go to **Operators > Installed Operators** and select **All Projects**. The list of operators shows the Gatekeeper operator.



- 5.3. Log in to the RHOCP managed cluster. Open a tab in the web browser and go to <https://console-openshift-console.apps.ocp4-mng.example.com>
- 5.4. Select the **Red Hat Identity Management** identity provider and log in as the admin user with redhatocp as the password.
- 5.5. Go to **Operators > Installed Operators** and select **All Projects**. The list of operators shows the Gatekeeper operator.
- 5.6. From the OpenShift web console of the hub cluster, set the cluster switcher to **All Clusters** to switch to the RHACM web console.
- 5.7. From the RHACM web console, go to **Governance** and click the **Policies** tab. Click the **policy-gatekeeper-operator** policy, and click the **Results** tab. Three policy templates appear for each managed cluster with no violations.

policy-gatekeeper-operator

Details Results

Clusters

Find clusters



Cluster Violations Template

local-cluster	✓ No violations	gatekeeper-status
local-cluster	✓ No violations	gatekeeper
local-cluster	✓ No violations	operatorpolicy-gatekeeper-operator
managed-cluster	✓ No violations	gatekeeper-status
managed-cluster	✓ No violations	gatekeeper
managed-cluster	✓ No violations	operatorpolicy-gatekeeper-operator

6. Update the Gatekeeper resource to exclude app-stage namespace for all constraints in the stage environment.

6.1. In the terminal window, edit the policy-gatekeeper-operator policy to update the Gatekeeper object by adding the config section, which excludes the app-stage namespace from all constraints.

```
[student@workstation ~]$ oc edit policy \
  policy-gatekeeper-operator -n policy-gatekeeper
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
...output omitted...
- objectDefinition:
  apiVersion: policy.open-cluster-management.io/v1
  kind: ConfigurationPolicy
  metadata:
    name: gatekeeper
    ...output omitted...
  spec:
    audit:
      logLevel: INFO
      replicas: 1
    mutatingWebhook: Disabled
    validatingWebhook: Enabled
    webhook:
      emitAdmissionEvents: Enabled
      logLevel: INFO
      replicas: 2
    config:
      disableDefaultMatches: false
      matches:
        - excludedNamespaces:
            - app-stage
        processes:
            - '*'
...output omitted...
```

6.2. Inspect the Config object within the openshift-gatekeeper-system namespace to verify that the Gatekeeper operator adds the app-stage namespace to the list of excluded namespaces.

```
[student@workstation ~]$ oc get configs.config.gatekeeper.sh \
-n openshift-gatekeeper-system -oyaml
apiVersion: v1
items:
- apiVersion: config.gatekeeper.sh/v1alpha1
  kind: Config
...output omitted...
  processes:
  - webhook
  - mutation-webhook
- excludedNamespaces:
  - app-stage
  processes:
  - '*'
...output omitted...
```

7. Deploy a gatekeeper policy in the policy-gatekeeper namespace to force containers not to use images with the latest tag for both stage and production environments.

- 7.1. In the terminal window, change to the ~/D00013L/labs/policies-integration/ directory.

```
[student@workstation ~]$ cd ~/D00013L/labs/policies-integration/
```

- 7.2. Edit the policy-latest.yaml file and modify the placeholder text in the Placement resource to match the following content:

```
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: policy-gatekeeper-containerimage-latest
...output omitted...
kind: Placement
metadata:
  name: placement-policy-gatekeeper-containerimage-latest
spec:
  clusterSets:
  - default
  predicates:
  - requiredClusterSelector:
      labelSelector:
        matchExpressions:
        - key: environment
          operator: In
          values: [stage, production]
```

- 7.3. Create the policy-gatekeeper-containerimage-latest policy.

```
[student@workstation policies-integration]$ oc apply -f policy-latest.yaml
policy.policy.open-cluster-management.io/policy-gatekeeper-containerimage-latest created
placementbinding.policy.open-cluster-management.io/binding-policy-gatekeeper-containerimage-latest
created
placement.cluster.open-cluster-management.io/placement-policy-gatekeeper-containerimage-latest
created
```

- 7.4. From the RHACM web console, go to **Governance** and click the **Policies** tab. Click the **policy-gatekeeper-containerimage-latest** policy to open the **Details** page of the policy.

- 7.5. Verify that the **Placement** selects the local-cluster and the managed-cluster managed clusters.

Placement			
Name ↑	Kind ↑	Clusters	Violations
placement-policy-gatekeeper-containerimage-latest	Placement	2	✓ No violations: local-cluster, managed-cluster

8. Deploy the stage-hello application to the stage environment by using the `~/D00013L/labs/policies-integration/stage-hello.yaml` file. The stage-hello application creates the app-stage namespace, and the stage-hello deployment, service, and route. The deployment uses an image with the latest tag. You can review the `stage-hello.yaml` file for more details.

- 8.1. Switch back to the terminal window, and create the stage-hello application resources. Gatekeeper allows you to create the deployment because the app-stage namespace is excluded from all constraints in the stage environment.

```
[student@workstation policies-integration]$ oc create -f stage-hello.yaml
project.project.openshift.io/app-stage created
deployment.apps/stage-hello created
service/stage-hello created
route.route.openshift.io/stage-hello created
```

- 8.2. Test the stage-hello application.

```
[student@workstation policies-integration]$ curl \
  stage-hello.apps.ocp4.example.com
Hello Application!
Image version : latest
```

- 8.3. Use the `~/D00013L/labs/policies-integration/hello-app.yaml` file to deploy the stage-hello application in the app-test namespace. The Gatekeeper policy denies the request with the following error message:

```
[student@workstation policies-integration]$ oc create -f hello-app.yaml
project.project.openshift.io/app-test created
service/hello-app created
route.route.openshift.io/hello-app created
Error from server (Forbidden): error when creating "hello-app.yaml": admission webhook
"validation.gatekeeper.sh" denied the request: [containerimagelatest] Deployment/hello-app:
container 'hello-app' is using the latest tag for its image
(registry.ocp4.example.com:8443/redhattraining/hello-app:latest), which is an anti-pattern.
```

- 8.4. From the RHACM web console, go to **Governance** and click the **Policies** tab. Click **policy-gatekeeper-containerimage-latest**, and click the **Results** tab. The policy-gatekeeper-admission-latest template shows the violation.

policy-gatekeeper-containerimage-latest

Details Results Actions

Clusters

Find clusters 1 - 4 of 4

Cluster	Violations	Template	Message	Remediation	Last report	History
local-cluster	Violations	policy-gatekeeper-admission-latest	violation - events [hello-app.1844064396921e4b] found in namespace openshift-gatekeeper-system - View details	enforce	May 29, 2025, 4:49 PM	View history
managed-cluster	No violations	policy-gatekeeper-admission-latest	notification - events missing as expected in namespace openshift-gatekeeper-system - View details	enforce	May 29, 2025, 4:39 PM	View history

9. Deploy the prod-hello application to the production environment by using the `~/D00013L/labs/policies-integration/prod-hello.yaml` file. The prod-hello application creates the app-prod namespace, and the prod-hello deployment, service, and route. The deployment uses an image with the latest tag. You can review the prod-hello.yaml file for more details.

- 9.1. In the terminal window, log in to the managed-cluster cluster as the admin user with redhatocp as the password.

```
[student@workstation policies-integration]$ oc login -u admin -p redhatocp \
https://api.ocp4-mng.example.com:6443
Login successful.
...output omitted...
```

- 9.2. Create the prod-hello application resources. The Gatekeeper policy denies the request with the following error message:

```
[student@workstation policies-integration]$ oc create -f prod-hello.yaml
project.project.openshift.io/app-prod created
service/prod-hello created
route.route.openshift.io/prod-hello created
Error from server (Forbidden): error when creating "prod-hello.yaml": admission webhook
"validation.gatekeeper.sh" denied the request: [containerimagelatest] Deployment/prod-hello:
container 'prod-hello' is using the latest tag for its image
(registry.ocp4.example.com:8443/redhattraining/hello-app:latest), which is an anti-pattern.
```

- 9.3. From the RHACM web console, go to **Governance** and click the **Policies** tab. Click the **policy-gatekeeper-containerimage-latest**, and click the **Results** tab to view the violation.

- 9.4. In the terminal window, edit the prod-hello.yaml file to use the v1.0 image tag.

```
...output omitted...
spec:
  containers:
    - name: prod-hello
      image: registry.ocp4.example.com:8443/redhattraining/hello-app:v1.0
...output omitted...
```

- 9.5. Delete any orphan prod-hello application resources. You can safely ignore the error message in the output.

```
[student@workstation policies-integration]$ oc delete -f prod-hello.yaml
project.project.openshift.io "app-prod" deleted
service "prod-hello" deleted
route.route.openshift.io "prod-hello" deleted
Error from server (NotFound): error when deleting "prod-hello.yaml": deployments.apps "prod-hello"
not found
```

9.6. Create the prod-hello application resources. The policy allows you to create the deployment.

```
[student@workstation policies-integration]$ oc create -f prod-hello.yaml
project.project.openshift.io/app-prod created
deployment.apps/prod-hello created
service/prod-hello created
route.route.openshift.io/prod-hello created
```

9.7. Test the prod-hello application.

```
[student@workstation policies-integration]$ curl \
  prod-hello.apps.ocp4-mng.example.com
Hello Application!
Image version : v1.0
```

9.8. Change to the home directory.

```
[student@workstation policies-integration]$ cd ~
[student@workstation ~]$
```

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish policies-integration
```

3.11. Troubleshoot Governance Issues

Objectives

- Troubleshoot common issues with RHACM Governance for managed clusters.

Troubleshooting RHACM Governance Issues

RHACM simplifies configuring policies across multiple clusters by providing a policy-based governance framework and a management console. This approach to policy definition is a declarative way to manage multiple clusters with a unified governance model.

However, issues can occur with the policy definition, syntax, or placement, or with conflicts with existing policies. It can be challenging to troubleshoot these issues, especially if you are new to RHACM governance or to cluster administration.

Governance Issues with Policies

Issues often arise when configuring deployment policies.

These issues can be broadly categorized into the following types:

Definition Issues

These issues arise when the policy is not defined correctly, which leads to unexpected behavior. Common mistakes include incorrect syntax, missing information in configuration fields, or providing incorrect parameters.

Placement Issues

Policies must be placed in the correct namespace and cluster for the intended effect. Issues can occur if the policy is applied to an incorrect namespace or cluster, or if the placement rules are not correctly configured.

Conflict Issues

Conflicts can arise for several reasons, most often when multiple policies are applied to the same resource. When these conflicts occur, it can lead to unpredictable behavior and might be difficult to identify and resolve. Additionally, if two policies have conflicting rules, it can be challenging to determine which policy takes precedence, or if the existing policy should be modified or removed.

Evaluation Issues

Policies might not evaluate as expected due to issues with the policy engine or the underlying infrastructure. This outcome becomes increasingly likely with more clusters, and as the infrastructure becomes more diverse and complex. This complexity can make it difficult to determine why a policy is not being enforced.

Remediation Issues

If a policy violation occurs, the configured remediation actions might not be applied correctly. This outcome can occur if the remediation actions violate other policies, cannot be performed successfully, or if issues arise with the policy controller. Also consider human error as a potential cause of remediation issues, such as manually changing a resource that introduces an unintentional policy violation.

Troubleshooting Tools

RHACM provides the following tools to help troubleshoot governance issues:

- The RHACM web console, where you can graphically manage policies across clusters
- The oc command-line tool, to inspect resources and view logs
- Robust logging capabilities to identify issues and audit policy evaluations or enforcement
- Use the Open Policy Agent (OPA) and OPA Gatekeeper in RHACM for an alternative approach to defining and enforcing policies in complex deployments
- Integrated development environments (IDEs) which can provide syntax highlighting and validation for policy files, to help identify syntax errors or conflicts

RHACM Web Console

The RHACM web console provides an interface for managing policies across clusters. Through the console, you can view the details of each policy, including its rules and remediation actions. You can also view the status of policies, their placement, and any violations.

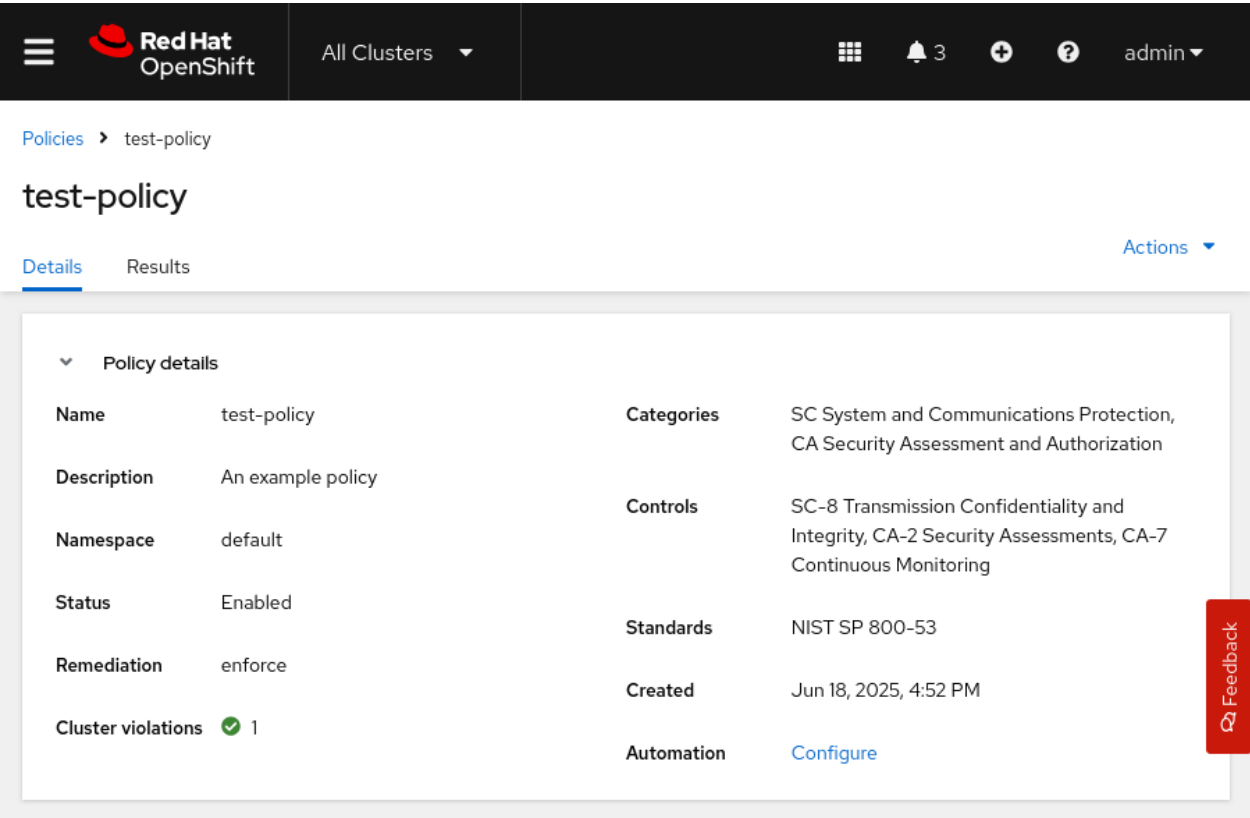


Figure 27. Policy details in the RHACM web console

Command Line Tools

The oc command-line tool can be useful for inspecting policies. For example, you can use the oc get policies command to view the status of all policies. You can also use the oc get policy <policy-name> -n <namespace> command to view the status of a specific policy, or the oc describe policy <policy-name> -n <namespace> command to view policy details, including its placement and status.

Use the oc describe policy command to review details, such as the policy deployment date, status, and any events:

```
[student@workstation ~]$ oc describe policy test-policy
Name:          test
Namespace:     developer ①

Kind:          Policy
Metadata:
  Creation Timestamp:  2025-06-12T02:45:11Z ②
  Generation:         1
  Resource Version:    343357
  UID:                a4234311-4f11-43a2-9124-f40a6878c623

...output omitted...

Status: ③
  Placement:
    Placement:        test-policyset-placement
    Placement Binding: test-policyset-placement
    Policy Set:        test-policyset
  Events: ④
    Type      Reason              Age   From              Message
    ----      -
    Normal    PolicyPropagation  162m  policy-propagator Policy default/test-policy was propagated to cluster managed-cluster
    Normal    PolicyPropagation  162m  policy-propagator Policy default/test-policy was propagated to cluster local-cluster
```

- ① Confirm that the Namespace is correct for the project.
- ② Verify that the Creation Timestamp shows the expected deployment date.
- ③ Review the Status information for the correct placement, binding, and set.
- ④ The Events field contains additional information to aid troubleshooting.

Logging

RHACM provides robust logging capabilities to aid in identifying issues and auditing policies. You can use the `oc logs` command to view the logs of the policy controller, to help identify issues with policy evaluation or enforcement.

Use the `oc logs` command in the `open-cluster-management` namespace to view the `grc-policy-propagator-xxx` and `grc-policy-addon-controller-xxx` component logs. These logs capture the controller activities and policy propagation across clusters.

The logs for the `grc-policy-addon-controller` can help identify issues with policy evaluation and enforcement, including certificate management details. Inspecting the logs for the `grc-policy-propagator` can help identify issues with policy propagation across clusters. The following command output displays the logs for the `grc-policy-propagator`, and shows the successful deployment of the `test_policy_set`.

```
[student@workstation ~]$ oc logs -n open-cluster-management \
  grc-policy-propagator-6c6668b5f-kbxgd

...output omitted...

2025-06-18T19:14:22.828Z info policy-set policyset/policyset
controller.go:85 Policy set successfully processed, reconcile ①
complete. {"Request.Namespace": "default", "Request.Name": "test_policy_set"}
```

OPA Gatekeeper

You can use the Open Policy Agent (OPA) and OPA Gatekeeper as an alternative approach to defining and enforcing policies. Inspect installation details and controller health to ensure that the OPA Gatekeeper is functioning correctly. Use the `oc describe operator gatekeeper` to review the Gatekeeper operator status to ensure that the policy controller is correctly installed and functioning.

Verifying a successful installation of the Gatekeeper operator can reveal issues with the deployment. Additionally, the Gatekeeper operator provides output for any events that occurred.

Use the `oc describe operator` command to view the Gatekeeper operator details:

```
[student@workstation ~]$ oc describe operator gatekeeper \
-n open-cluster-management
Name:          gatekeeper-operator-product.openshift-operators
Namespace:
Labels:        <none>
Annotations:   <none>
API Version:   operators.coreos.com/v1
Kind:          Operator
Metadata:
  Creation Timestamp:  2025-06-12T03:09:45Z ①
  Generation:         1
  Resource Version:    1746288
  UID:                 b98c0047-45d3-412e-883f-ccb662f07cc1

...output omitted...

Events:          <none> ②
```

- ① The Creation Timestamp shows the Gatekeeper operator installation timestamp. Verify that this field shows the expected installation date.
- ② Events contain additional information to aid troubleshooting. An absence of events indicates that the Gatekeeper operator is functioning correctly.

IDEs

Many issues are introduced through human error, such as spelling errors or incorrect syntax. Most modern IDEs, such as VSCode, provide syntax highlighting, spell checking, and argument validation features when editing policy files. Configure your IDE with the appropriate extensions to help identify issues.

Additional Considerations

Events can inform you about the deployment, status, and configuration details for RHACM resources and policies. You can use the `oc get events` command to view the events feed for the environment. Refining this output by namespace or through `grep` filtering can help you focus on any specific issues.

The `oc get managedcluster` command lists all managed clusters. Checking the events from the `oc describe managedcluster` command output provides configuration information, conditions, and events for RHACM clusters. Verify that the details are accurate for the related areas for the policy issue that occurs.

Deploying PolicySets can encounter conflicts when multiple policies require a specific order of application. When control of the order is necessary, policy dependencies can be added. Update the policy resource YAML file with dependencies and extradependencies to define the order for correct deployment.

Listing 1. Example Policy Dependency

```
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  annotations:
    policy.open-cluster-management.io/categories: CM Configuration Management
    policy.open-cluster-management.io/controls: CM-2 Baseline Configuration
    policy.open-cluster-management.io/standards: NIST SP 800-53
    policy.open-cluster-management.io/description:
  name: compliance-scan
  namespace: default
spec:
  dependencies: ①
  - apiVersion: policy.open-cluster-management.io/v1
    compliance: Compliant
    kind: Policy
    name: compliance-operator
    namespace: default
  disabled: false
  policy-templates:
  - extraDependencies: ②
    - apiVersion: policy.open-cluster-management.io/v1
      kind: ConfigurationPolicy
      name: scan-setting-binding
      compliance: Compliant
      ignorePending: false
```

- ① The dependency verifies the installation of the compliance operator.
- ② The extra dependency proceeds if the compliance value is met.



References

Open Policy Agent

For more information about installing Gatekeeper by using a Gatekeeper operator policy in RHACM, refer to the *Installing Gatekeeper Using a Gatekeeper Operator Policy* section in the *Gatekeeper Operator Overview* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Governance* documentation at https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/governance/index#installing-gk-operator-policy

For more information about policy dependencies, refer to the *Policy Dependencies* section in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Governance* documentation at https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/governance/governance#policy-dependencies

3.12. Guided Exercise

Troubleshoot Governance Issues

Apply your acquired knowledge in troubleshooting common problems with deploying and configuring policies in RHACM.

Outcomes

- The production-scan policy needs to target the managed-cluster cluster in the policies-troubleshoot namespace.
- Inspect the policy enforcement to ensure that the scan is targeting the production-clusters cluster set, which must contain only the managed-cluster member.
- Remediate any discrepancies that you find, so the policy applies to the cluster set that contains the single cluster.

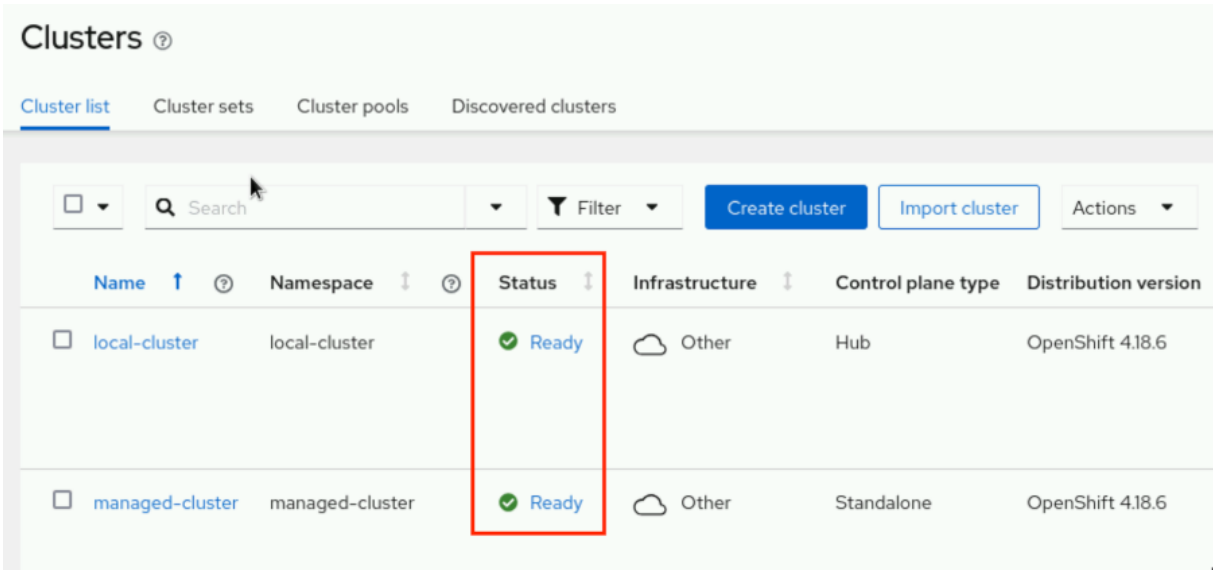
Prerequisites

As the student user on the workstation machine, use the lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab install do0013l ; lab start policies-troubleshoot
```

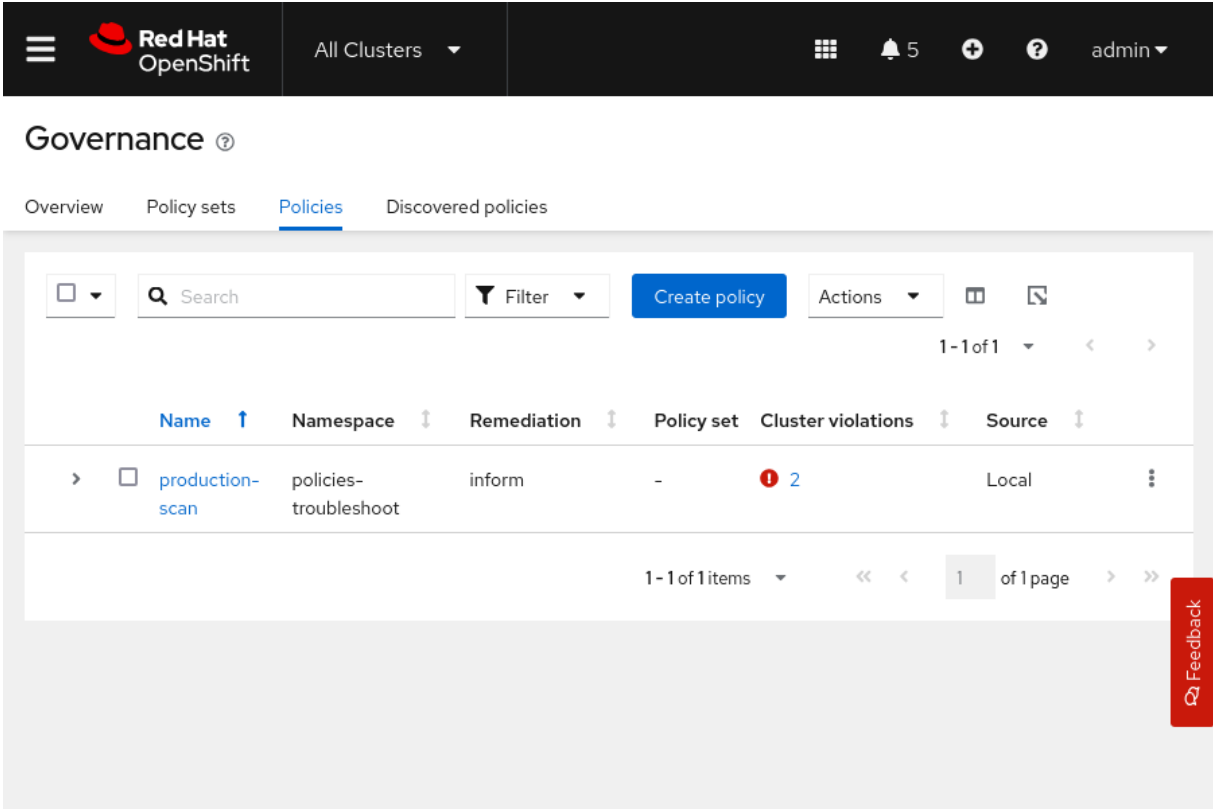
Instructions

1. Log in to the RHOC hub cluster, and switch to the RHACM web console.
 - 1.1. Open a web browser and go to <https://console-openshift-console.apps.ocp4.example.com>
 - 1.2. Select the **Red Hat Identity Management** identity provider and log in as the admin user with redhatocp as the password.
 - 1.3. Switch to the RHACM web console, go to **Infrastructure > Clusters**, and verify that the local-cluster and the managed-cluster clusters are in the Ready status.



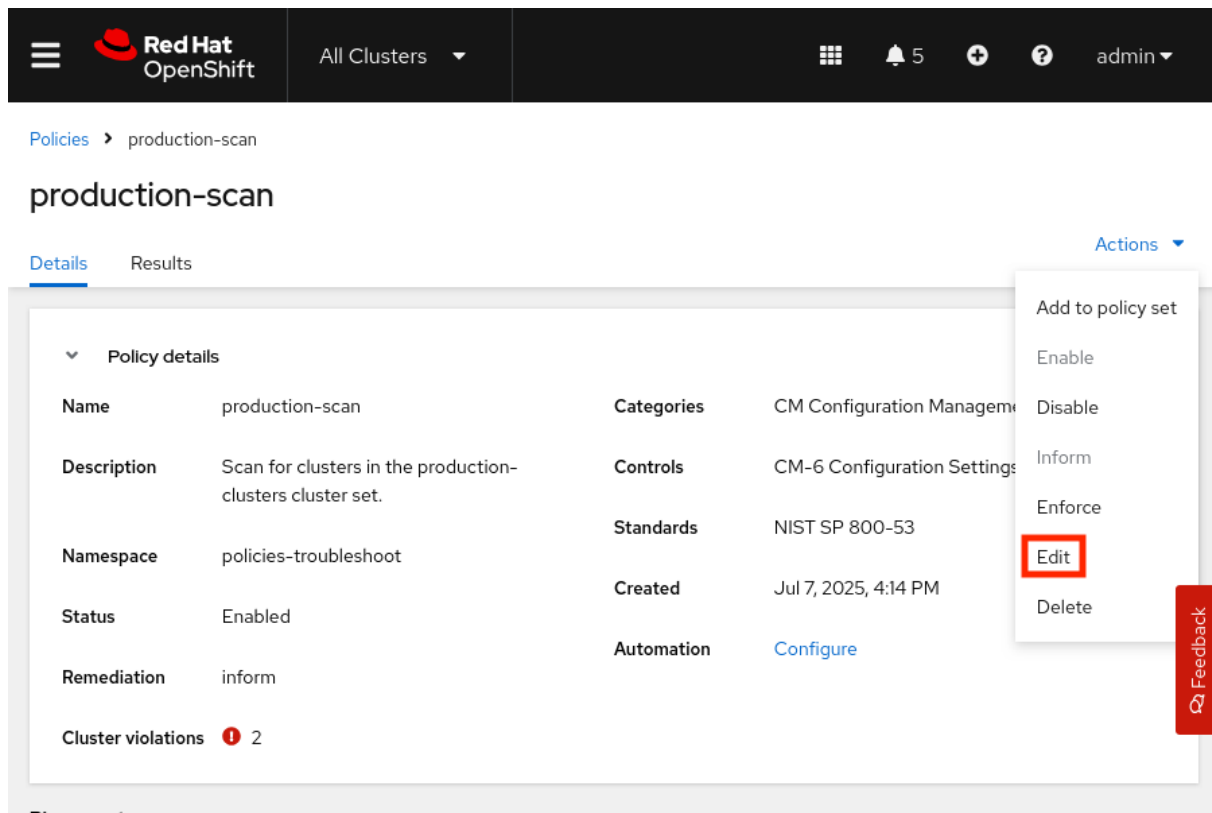
2. Inspect the production-scan policy to identify which cluster set is configured.

2.1. Go to **Governance** and click **Policies** to view the list of configured policies.



2.2. Select the production-scan policy, and go to the **Details** tab. In the Placement section, inspect the Clusters value, which shows two clusters, instead of only the managed-clusters cluster. This placement is incorrect and must be corrected.

2.3. From the **Actions** menu, select Edit to review the policy configuration details.



- 2.4.** Inspect the `clusterSets` field in the YAML resource definition file. If the YAML window is not shown, then enable the **YAML** viewer selector to display the code. The Placement resource `clusterSets` value is set to `default` instead of `production-clusters`. It means that the policy is targeting the clusters that the default cluster set defines, which is not the intended behavior, as shown in the following excerpt:

```
...output omitted...
---
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: production-scan-placement
  namespace: policies-troubleshoot
  annotations:
    kubectrl.kubernetes.io/last-applied-configuration: ...
  creationTimestamp: 2025-07-03T08:31:15Z
  generation: 1
  resourceVersion: "666027"
  uid: fca69d11-fe2e-494a-a963-e6020747a29c
spec:
  clusterSets:
    - default
...output omitted...
```

- 3.** Update the policy placement to use the specified `production-clusters` cluster set, to replace the default value.
- 3.1.** By using the configuration wizard, click **Next** on the Details and Policy templates pages to get to the Placement configuration section. Remove the default cluster set by clicking the **x** next to it, and select `production-clusters` from the drop-down list.

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

Red Hat OpenShift

All Clusters

5

admin

[Policies](#) > production-scan

Edit policy

YAML

A policy generates reports and validates cluster violations based on specified security standards, categories, and controls.

3 Placement

Cluster sets

production-clusters1

☐ default

☒ production-clusters

Next

Back

Cancel

Feedback

3.2. Click **Next** on the Placement and Policy annotations pages, and then click **Submit** to apply the changes.

Red Hat OpenShift

All Clusters

4

admin

[Policies](#) > production-scan

Edit policy

YAML

A policy generates reports and validates cluster violations based on specified security standards, categories, and controls.

5 Review

Placement

Name

production-scan-placement

Cluster sets

production-clusters

Policy annotations

Standards

NIST SP 800-53

Categories

CM Configuration Management

Submit

Back

Cancel

Feedback

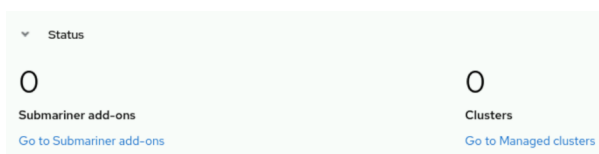
3.3. Return to the wizard editor from the policy details page, and verify that the policy is now applied to only the managed-cluster cluster by reviewing the clusterSets value in the YAML file.


```
...output omitted...
---
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: production-scan-placement
  namespace: policies-troubleshoot
  annotations:
    kubectrl.kubernetes.io/last-applied-configuration: ...
  creationTimestamp: 2025-07-03T08:31:15Z
  generation: 1
  resourceVersion: "666027"
  uid: fca69d11-fe2e-494a-a963-e6020747a29c
spec:
  clusterSets:
    - production-clusters
...output omitted...
```

4. Review the policy details to ensure that the policy is now applied to the managed-cluster cluster only.

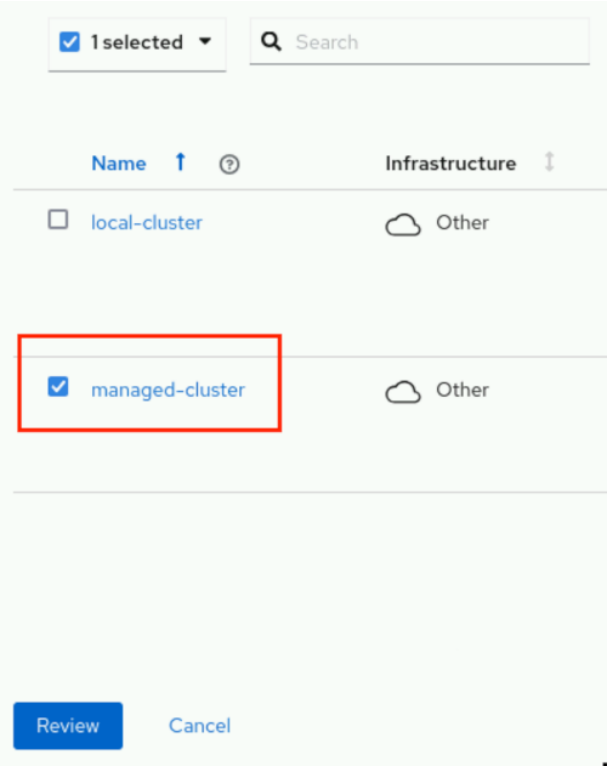
4.1. Go to **Governance** and click **Policies**. Select the production-scan policy. Select the **Details** tab and review the Placement section. The placement information now shows that no clusters are targeted when using the production-clusters cluster set.

4.2. Go to the Cluster set tab on the **Infrastructure > Clusters** page and click **Cluster sets**. Select the production-clusters cluster set. In the Status section, inspect the Clusters value, which now shows zero clusters. Again, this value deviates from the expected configuration, which is intended to target a single cluster named managed-cluster.

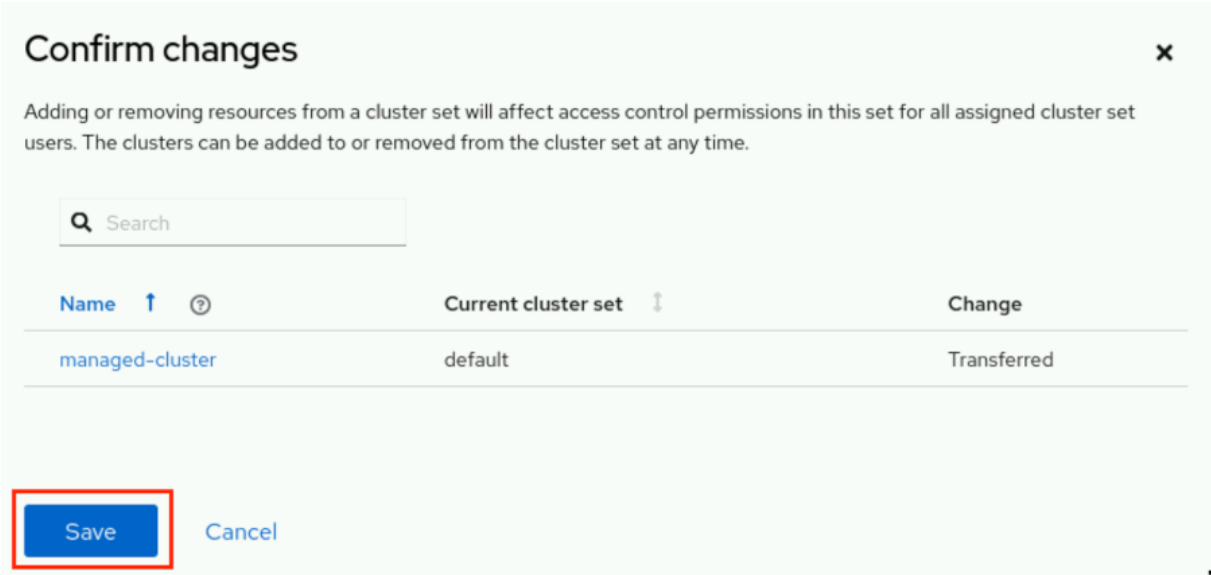


5. Correct and verify the members of the production-clusters cluster set.

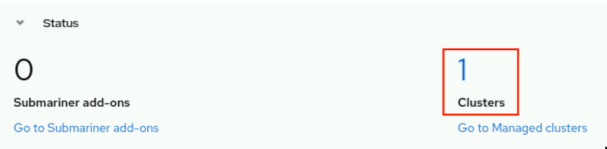
5.1. Click **Go to Managed clusters** and click **Manage resource assignments**. Add the intended cluster to the production-clusters cluster set by checking the box next to managed-cluster and clicking **Review**.



5.2. Confirm the transfer of the managed-cluster cluster by clicking **Save**.

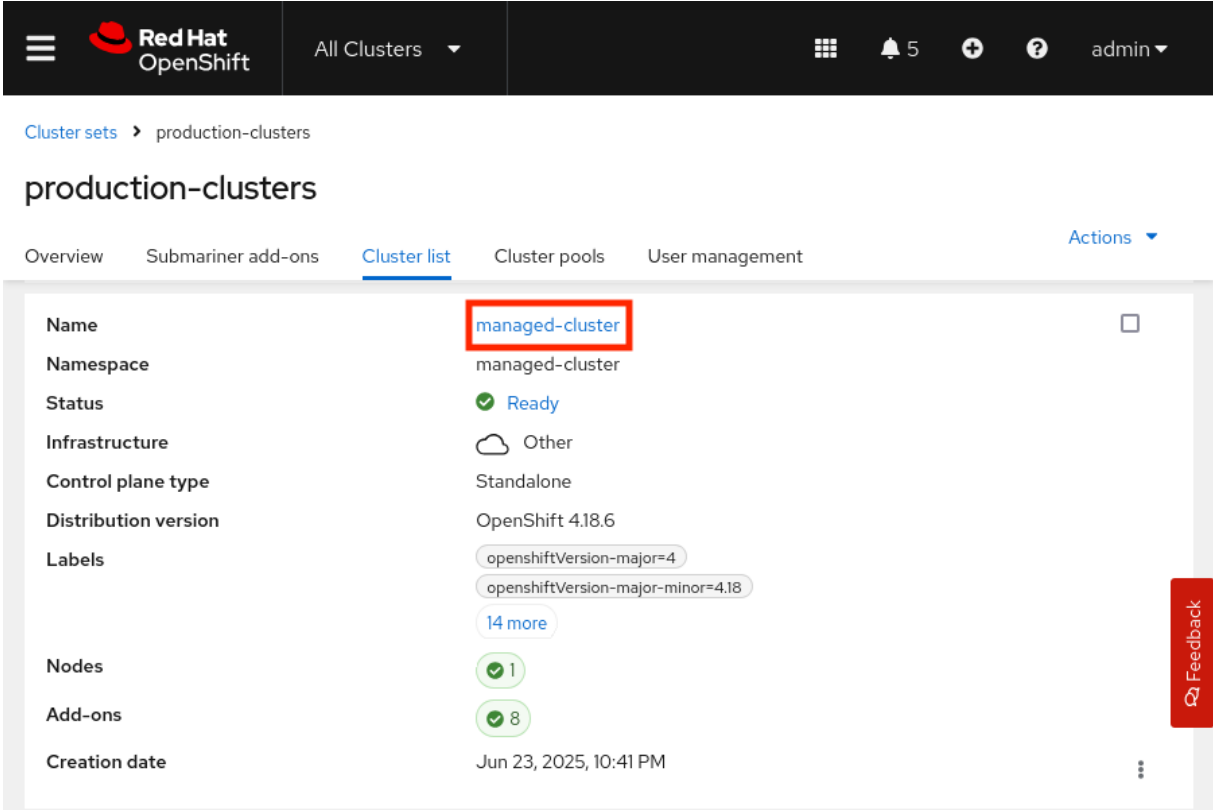


5.3. In the Status section, verify that the Clusters value shows one cluster.



5.4. Go to the Cluster list tab and verify that the managed-cluster cluster is the only member.

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright



6. Verify that the production-scan policy is now applied to the managed-cluster cluster only.
- 6.1. Go to **Governance** and click **Policies**. Select the production-scan policy, and go to the **Details** tab.
- 6.2. In the Placement section, inspect the Clusters value, which shows one cluster, and inspect the Violations value, which shows Violations: managed-cluster. This placement is correct and targeting only managed-cluster cluster.

Finish

On the workstation machine, use the lab command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish policies-troubleshoot
```

3.13. Lab

Deploying and Managing Policies with RHACM

Create an RHACM policy to prevent the creation of the test-project namespace. Then, troubleshoot the existing RHACM policy to ensure the successful installation of the Gatekeeper operator, and deploy an additional Gatekeeper constraints policy to enforce container security by preventing pods from running with elevated privileges.

Outcomes

- Create a policy that prevents the creation of the test-project namespace in the targeted cluster.
- The policy-gatekeeper policy installs the Gatekeeper operator on all clusters in the default cluster set.
- Deploy a gatekeeper policy to enforce containers not to have elevated privileges in the secure-prod namespace of the targeted cluster.

Prerequisites

As the student user on the workstation machine, use the lab command to prepare your environment for this lab, which ensures that all required resources are available.

```
[student@workstation ~]$ lab install do0013l ; lab start policies-review
```

Instructions

In this lab, you deploy a namespace policy that prevents the creation of the test-project namespace in the production cluster.

You troubleshoot the deployed policy-gatekeeper policy to ensure that the Gatekeeper operator is installed on all clusters in the default cluster set. You also deploy a gatekeeper policy to enforce containers not to have elevated privileges in the secure-prod namespace of the production cluster.

The classroom environment provides two RHOC clusters: an RHACM hub cluster and a managed cluster. You can access these clusters by using the following parameters:

RHACM hub cluster web console URL

<https://console-openshift-console.apps.ocp4.example.com>

RHACM hub cluster API URL

<https://api.ocp4.example.com:6443>

Managed cluster web console URL

<https://console-openshift-console.apps.ocp4-mng.example.com>

Managed cluster API URL

<https://api.ocp4-mng.example.com:6443>

Administrator username for both clusters

admin (select the **Red Hat Identity Management** identity provider)

Administrator password for both clusters

redhatocp

1. Bind the default managed cluster set to the `policies-review` namespace.
2. Deploy a namespace policy in the `policies-review` namespace with the following specification:
 - The policy prevents the creation of the `test-project` namespace.
 - The name of the policy is `policy-namespace`.
 - The policy must be deployed in the `policies-review` namespace.
 - The policy placement uses the default cluster set and selects clusters with the `environment=production` label.

Note

You can use either the RHACM web console or the policy generator tool to deploy this policy. The `lab start` command installs the policy generator tool. The sample policy generator configuration and manifest files are available in the `/home/student/DO0013L/labs/policies-review/namespace-policy` directory.

3. Ensure that the `policy-gatekeeper` policy installs the Gatekeeper operator on all clusters in the default cluster set.
4. Deploy a gatekeeper policy to enforce containers not to have elevated privileges in the `secure-prod` namespace of the targeted cluster, with the following specification:
 - The name of the policy is `policy-privileged-containers`.
 - The policy placement is using the `environment=production` label to select the cluster.
 - The policy must be deployed in the `policies-review` namespace.
 - The sample policy generator configuration and manifest files are available in the `/home/student/DO0013L/labs/policies-review/constraint-policy` directory.
 - The PolicyGenerator tool is already installed in the lab environment.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade policies-review
```

Finish

As the student user on the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish policies-review
```

Solution

Create an RHACM policy to prevent the creation of the test-project namespace. Then, troubleshoot the existing RHACM policy to ensure the successful installation of the Gatekeeper operator, and deploy an additional Gatekeeper constraints policy to enforce container security by preventing pods from running with elevated privileges.

Outcomes

- Create a policy that prevents the creation of the test-project namespace in the targeted cluster.
- The policy-gatekeeper policy installs the Gatekeeper operator on all clusters in the default cluster set.
- Deploy a gatekeeper policy to enforce containers not to have elevated privileges in the secure-prod namespace of the targeted cluster.

Prerequisites

As the student user on the workstation machine, use the lab command to prepare your environment for this lab, which ensures that all required resources are available.

```
[student@workstation ~]$ lab install do0013l ; lab start policies-review
```

Instructions

In this lab, you deploy a namespace policy that prevents the creation of the test-project namespace in the production cluster.

You troubleshoot the deployed policy-gatekeeper policy to ensure that the Gatekeeper operator is installed on all clusters in the default cluster set. You also deploy a gatekeeper policy to enforce containers not to have elevated privileges in the secure-prod namespace of the production cluster.

The classroom environment provides two RHOCP clusters: an RHACM hub cluster and a managed cluster. You can access these clusters by using the following parameters:

RHACM hub cluster web console URL

<https://console-openshift-console.apps.ocp4.example.com>

RHACM hub cluster API URL

<https://api.ocp4.example.com:6443>

Managed cluster web console URL

<https://console-openshift-console.apps.ocp4-mng.example.com>

Managed cluster API URL

<https://api.ocp4-mng.example.com:6443>

Administrator username for both clusters

admin (select the **Red Hat Identity Management** identity provider)

Administrator password for both clusters

redhatocp

1. Bind the default managed cluster set to the policies-review namespace.
 - 1.1. From the workstation machine, open Firefox and access <https://console-openshift-console.apps.ocp4.example.com>
 - 1.2. Click **Red Hat Identity Management** and log in as the admin user with redhatocp as the password.
 - 1.3. Ensure that the cluster switcher, at the top of the console, is set to All Clusters, which displays the RHACM console.
 - 1.4. Select **Infrastructure > Clusters** and click the **Cluster sets** tab.
 - 1.5. Click the vertical ellipsis icon (⋮) to the right of the default managed cluster set row, and click **Edit namespace bindings**.
 - 1.6. From the **Edit namespace bindings** dialog box, select the policies-review namespace from the drop-down list and click **Save**.
2. Deploy a namespace policy in the policies-review namespace with the following specification:
 - The policy prevents the creation of the test-project namespace.
 - The name of the policy is policy-namespace.
 - The policy must be deployed in the policies-review namespace.
 - The policy placement uses the default cluster set and selects clusters with the environment=production label.

 Note

You can use either the RHACM web console or the policy generator tool to deploy this policy. The lab start command installs the policy generator tool. The sample policy generator configuration and manifest files are available in the `/home/student/DO0013L/labs/policies-review/namespace-policy` directory.

- 2.1. From the RHACM console, go to **Governance** and click **Policies**. Click **Create policy**.
- 2.2. At the top of the **Create policy** page, set the **YAML** toggle to on, to view the YAML file. The policy creation steps that you complete are automatically reflected in updates to the policy YAML file.
- 2.3. In the policy creation form, configure the fields as shown in the following table:

Field	Action
Name	policy-namespace
Namespace	policies-review
Disable policy	checkbox is cleared

Click **Next**.

- 2.4. In the **Policy templates** section, set **Remediation** to **Enforce**. Click **Add policy template** and select the **Namespace must exist** policy template.
- 2.5. Scroll down to the **Configuration objects** section and enter test-project as the value for the Name field.
- 2.6. In the YAML editor, update the value of the complianceType field to mustnothave.

```
...output omitted...
spec
  ...output omitted...
  object-templates:
    - complianceType: mustnothave
      objectDefinition:
        kind: Namespace
        apiVersion: v1
        metadata:
          name: test-project
```

Verify that the **Configuration objects** section also shows the **Must not have** compliance type. Click **Next**.

- 2.7. In the **Placement** section, click **New placement**. The name of the placement is policy-namespaces-placement. Click **Add label expression**, and select environment for the Label field, and select production for the Values field. Click **Next**.
 - 2.8. In the **Policy annotations** section, leave the default values and click **Next**.
 - 2.9. In the **Review** section, click **Submit** to create the policy.
3. Ensure that the policy-gatekeeper policy installs the Gatekeeper operator on all clusters in the default cluster set.
 - 3.1. From the RHACM console, go to **Governance** and click **Policies**. Click the **policy-gatekeeper-operator** policy. Verify that the **Placement** section shows the No placement selectors found message.
 - 3.2. From the **Actions** menu, select Edit to review the policy configuration details.
 - 3.3. In the **Edit policy** form, click **Placement** to check the placement details. In the **Placement** section, verify that No placement is shown for the policy-gatekeeper-operator policy.
 - 3.4. Click **New placement**. The name of the placement is policy-gatekeeper-operator-placement. Select the default cluster set from the **Cluster sets** drop-down list. Click **Next**.
 - 3.5. In the **Policy annotations** section, leave the default values and click **Next**.
 - 3.6. In the **Review** section, click **Submit** to update the policy. The **Details** page shows that the policy-gatekeeper-operator policy is placed on both clusters.
 - 3.7. Verify that the policy shows a violation for both clusters and that **Remediation** shows inform as the status.
 - 3.8. Click **Results** to check the violation.

In the **Results** section, the gatekeeper, operatorpolicy-gatekeeper-operator, and gatekeeper-status templates each show a violation and all templates have inform remediation.

The inform remediation shows that the resources are not available. The policy does not create the resources.

3.9. From the **Actions** menu, select Enforce to change the remediation action. Click **Enforce** to confirm.

It takes about three minutes for the policy to report the **No violations** status for all the templates.

4. Deploy a gatekeeper policy to enforce containers not to have elevated privileges in the secure-prod namespace of the targeted cluster, with the following specification:

- The name of the policy is policy-privileged-containers.
- The policy placement is using the environment=production label to select the cluster.
- The policy must be deployed in the policies-review namespace.
- The sample policy generator configuration and manifest files are available in the /home/student/D00013L/labs/policies-review/constraint-policy directory.
- The PolicyGenerator tool is already installed in the lab environment.

4.1. In a terminal window, log in to the RHOC cluster as the admin user with redhatocp as the password.

```
[student@workstation ~]$ oc login -u admin -p redhatocp \
https://api.ocp4.example.com:6443
Login successful.
...output omitted...
```

4.2. Change to the directory where the sample policy generator configuration and manifest files are available.

```
[student@workstation ~]$ cd ~/D00013L/labs/policies-review/constraint-policy
[student@workstation constraint-policy]$
```

4.3. Review the manifest/constraint.yaml file. You do not have to change its contents.

```
---
apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: k8sprivilegedcontainer
...output omitted...
---
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sPrivilegedContainer
metadata:
  name: disallow-privileged-containers
spec:
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
    namespaces:
      - "secure-prod"
---
```

The manifest file has ConstraintTemplate and K8sPrivilegedContainer resources. For the K8sPrivilegedContainer resource, the spec.match.namespace field is set to the secure-prod value.

4.4. Update the policy-generator-config.yaml policy generator file. Use the following table to configure the YAML manifest. The policy generator YAML file must reference the ~/D00013L/labs/policies-review/constraint-policy/manifest directory.

Field	Value
namespace	policies-review
remediationAction	enforce
policies.name	policy-privileged-containers
matchLabels	environment: production

```
---
apiVersion: policy.open-cluster-management.io/v1
kind: PolicyGenerator
metadata:
  name: policy-generator-constraint
policyDefaults:
  namespace: policies-review
  complianceType: musthave
  severity: medium
  remediationAction: "enforce"
policies:
- name: policy-privileged-containers
  manifests:
    - path: /home/student/DO0013L/labs/policies-review/constraint-policy/manifest
  placement:
    clusterSelector:
      matchLabels:
        environment: production
```

4.5. Use the policy generator manifest file from the previous steps to create the Policy, Placement, and PlacementBinding manifests. Store the results in a YAML file.

```
[student@workstation constraint-policy]$ PolicyGenerator \
policy-generator-config.yaml > policy-manifest.yaml
```

4.6. Inspect the resource manifests that are generated.

```
---
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  namespace: policies-review
  name: policy-privileged-containers
...output omitted...
policy-templates:
  - objectDefinition:
      apiVersion: policy.open-cluster-management.io/v1
      kind: ConfigurationPolicy
      metadata:
        name: policy-privileged-containers
      spec:
        object-templates:
          - complianceType: musthave
...output omitted...
          spec:
            match:
              kinds:
                - apiGroups:
                    - ""
                  kinds:
                    - Pod
                  namespaces:
                    - secure-prod
            remediationAction: enforce
            severity: medium
...output omitted...
---
apiVersion: apps.open-cluster-management.io/v1
kind: PlacementRule
metadata:
  name: placement-policy-privileged-containers
  namespace: policies-review
spec:
  clusterSelector:
    matchLabels:
      environment: production
---
apiVersion: policy.open-cluster-management.io/v1
kind: PlacementBinding
...output omitted...
```

- 4.7.** Create the Policy, Placement, and PlacementBinding resources by using the previously created policy manifest file.

Use the `oc create` command to create these resources.

```
[student@workstation constraint-policy]$ oc create -f policy-manifest.yaml
policy.policy.open-cluster-management.io/policy-privileged-containers created
placementrule.apps.open-cluster-management.io/placement-policy-privileged-containers created
placementbinding.policy.open-cluster-management.io/binding-policy-privileged-containers created
```

- 4.8.** Change to the home directory.

```
[student@workstation constraint-policy]$ cd ~
[student@workstation ~]$
```

- 4.9.** Go to **Governance > Policies** and verify that the policy-privileged-containers policy is available.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade policies-review
```

Finish

As the student user on the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish policies-review
```

3.14. Summary

In this lesson, you learned about the governance policy engine that is provided through RHACM. From the web console and the CLI, you can administer policies that govern the configuration and behavior of managed clusters. With the centralized approach to policy management, you can define and enforce policies across a fleet of managed clusters, to ensure compliance and consistency.

RHACM provides a set of predefined policies to get started quickly. You can customize these policies or create your own to meet the requirements of your environment.

Using the RHACM concept of a *cluster set*, you can group similar clusters, apply policy sets to those groups, and review violations for any of the member clusters, all from within a unified web console.

You create RHACM *placement* resources to define rules to dynamically select a subset of managed clusters in one or more managed cluster sets. You can use placement rules to target specific clusters for policy application, to ensure that policies are enforced only where they are needed.

Next Steps

You might explore other learning opportunities on your own, or explore your RHACM installation further, or redo some exercises.

A good next step in your learning journey would be to explore the next lessons in the series:

- Lesson 4: *Enabling and Customizing the Observability Stack in Red Hat Advanced Cluster Management for Kubernetes*
- Lesson 5: *Managing the Multicluster Application Lifecycle by Using GitOps Practices and Red Hat Advanced Cluster Management for Kubernetes*
- Lesson 6: *Managing Virtual Machines for Multiple Clusters with Red Hat Advanced Cluster Management for Kubernetes*

Chapter 4.

Enabling and Customizing the Red Hat Advanced Cluster Management for Kubernetes Observability Stack

Goal

Monitor a fleet of managed clusters and troubleshoot performance and availability issues by using Red Hat Advanced Cluster Management for Kubernetes (RHACM) observability components.

Sections

- Enabling the RHACM Observability Stack (and Guided Exercise)
- Customizing the RHACM Observability Stack (and Guided Exercise)

Lab

- Enable and Customize the RHACM Observability Stack

4.1. Enabling the RHACM Observability Stack

Objectives

- Enable the RHACM observability stack and describe the architecture and the benefits of observability in a multicluster environment.

Multicluster Observability in RHACM

The RHACM observability component is a service that provides a unified display of metrics to assess the health and usage of clusters, and workloads across your fleet.

In production environments, monitoring, processing, and storing metrics from the fleet of clusters is crucial to anticipating issues, improving the performance of the clusters, and conducting postmortem analysis. By using the observability service, you can automate and manage the components that are within observability.

RHACM Observability Components

The observability service uses the following open source observability tools from the Cloud Native Computing Foundation (CNCF):

- Prometheus: A monitoring and alerting tool to collect and store metrics from your applications as time-series data. Prometheus enables you to store all scraped samples locally, run rules to aggregate and record new time series from existing data, and generate alerts.
- Thanos: A toolkit of components that gathers data from multiple Prometheus instances that are running on different OpenShift clusters, and enables global querying. You can use any S3-compatible storage for long-term storage of Prometheus data.
- Alertmanager: A tool to manage and receive alerts from Prometheus. You can deduplicate, group, and route alerts to third-party applications such as email, Slack, and PagerDuty. Alertmanager enables you to silence and inhibit specific alerts.

The following diagram shows the components of the observability service:

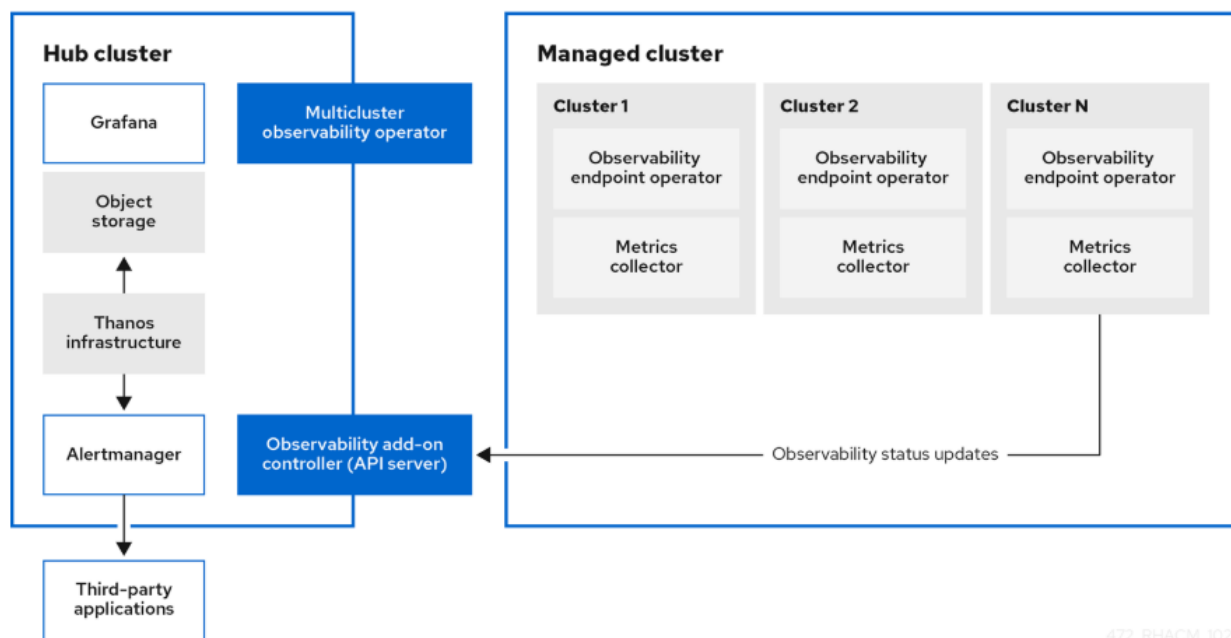


Figure 28. Observability architecture

- The multiclusterhub-operator pod deploys the multicluster-observability-operator pod by default. The multicluster-observability-operator pod sends hub cluster data to the managed clusters.
- The observability-endpoint-operator pod is automatically deployed to each imported or created managed cluster. This controller starts a metrics collector that collects and sends the data from OpenShift Prometheus to the RHACM hub cluster.
- The observability add-on controller is the API server that automatically updates the log of the managed clusters.
- The Thanos infrastructure includes the Thanos Compactor, which the multicluster-observability-operator pod deploys.

The Thanos Compactor ensures that queries are performing well by using the retention configuration and compaction of the data in storage.

- Grafana enables data visualization with many predefined dashboards, and you can also create your own custom dashboards to make cluster resource information visible, help you save cost, and prevent service disruptions. RHACM supports version 11.1.5 of Grafana.

Differences Between the RHOCP Monitoring Stack and the RHACM Observability Service

The Grafana and Alertmanager instances from RHACM complement the OpenShift cluster-level metrics.

The RHOCP monitoring stack includes a preconfigured, preinstalled, and self-updating monitoring stack that monitors core platform components. Starting with RHOCP version 4.6, you can also enable the option to enable monitoring for user-defined projects.

Enabling the RHACM Observability Service

The RHACM observability service is always installed in the hub cluster, although it is not enabled by default, due to the requirements for persistent storage, CPU, and memory.

When you enable the observability service on your hub cluster, the `multicluster-observability-operator` pod watches for new managed clusters and automatically deploys metric and alert collection services to the managed clusters.

Prerequisites for the Observability Service

You must meet the following conditions to enable the observability service:

- A user with the cluster administrator role, the `open-cluster-management:cluster-manager-admin` role, or an S3 administrator
- To define a storage class in the `MultiClusterObservability` custom resource, if no default storage class is specified
- To have direct network access to the hub cluster
- To configure an object store to create a storage solution. RHACM supports Amazon Web Services S3, Google Cloud Storage, and Red Hat OpenShift Data Foundation, among other cloud providers.
- The observability components require 2701 m CPU and 11972 Mi memory to enable the observability service.

Enabling RHACM Observability from the CLI

To enable the RHACM observability service from the CLI, create the `open-cluster-management-observability` namespace.

```
[user@host ~]$ oc create namespace open-cluster-management-observability
```

Create a secret named `multiclusterhub-operator-pull-secret` in the `open-cluster-management-observability` namespace.

You can extract the secret named `multiclusterhub-operator-pull-secret` from the `open-cluster-management` namespace.

```
[user@host ~]$ DOCKER_CONFIG_JSON=oc extract \  
secret/multiclusterhub-operator-pull-secret \  
-n open-cluster-management --to=-
```

If the `multiclusterhub-operator-pull-secret` is not defined in the namespace, then extract the secret named `pull-secret` from the `openshift-config` namespace.

```
[user@host ~]$ DOCKER_CONFIG_JSON=oc extract \  
secret/pull-secret -n openshift-config --to=-
```

Create the pull secret in the `open-cluster-management-observability` namespace by running the following command:

```
[user@host ~]$ oc create secret generic multiclusterhub-operator-pull-secret \  
-n open-cluster-management-observability \  
--from-literal=.dockerconfigjson="$DOCKER_CONFIG_JSON" \  
--type=kubernetes.io/dockerconfigjson
```

The RHACM observability service needs a persistent object store. The following YAML example defines an `ObjectBucketClaim` named `thanos-obc` that uses the `openshift-storage.noobaa.io` storage class that Red Hat OpenShift Data Foundation provides.

```
apiVersion: objectbucket.io/v1alpha1
kind: ObjectBucketClaim
metadata:
  name: thanos-objc
  namespace: open-cluster-management-observability
spec:
  storageClassName: openshift-storage.noobaa.io
  generateBucketName: observability-bucket
```

After creating the object bucket claim, Red Hat OpenShift Data Foundation creates, with the same name, a matching secret and a configuration map, which contain the bucket information and credentials.

To extract the information from the configuration map, run the following command:

```
[user@host ~]$ oc extract --to=- cm/thanos-objc \
-n open-cluster-management-observability
# BUCKET_REGION

# BUCKET_SUBREGION

# BUCKET_HOST
s3.openshift-storage.svc
# BUCKET_NAME
thanos-objc-b2cdf931-0ec5-41a9-9b31-033d360d3a38
# BUCKET_PORT
443
```

To extract the bucket credentials from the secret, run the following command:

```
[user@host ~]$ oc extract --to=- secret/thanos-objc \
-n open-cluster-management-observability
# AWS_SECRET_ACCESS_KEY
MWYkFQTUVUHLw3Lgjtvs+WUBu2QH6hF1XLuXYsR
# AWS_ACCESS_KEY_ID
Z9VYa0jvJVveQ1Az4rkX
```

Create the thanos-object-storage secret within the open-cluster-management-observability namespace by using the information from the configuration map and the credentials from the secret.

The following YAML example creates the thanos-object-storage secret:

```
apiVersion: v1
kind: Secret
metadata:
  name: thanos-object-storage
  namespace: open-cluster-management-observability
type: Opaque
stringData:
  thanos.yaml: |
    type: s3
    config:
      bucket: thanos-objc-b2cdf931-0ec5-41a9-9b31-033d360d3a38
      endpoint: s3.openshift-storage.svc
      insecure: true
      access_key: Z9VYa0jvJVveQ1Az4rkX
      secret_key: MWYkFQTUVUHLw3Lgjtvs+WUBu2QH6hF1XLuXYsR
```

Creating the Multicluster Observability Custom Resource

To enable the observability service, you must create a MultiClusterObservability custom resource instance in the open-cluster-management-observability namespace. The MultiClusterObservability object

requires a secret that contains the credentials of the S3 bucket storage.

The following YAML example is of a `MultiClusterObservability` resource:

```
apiVersion: observability.open-cluster-management.io/v1beta2
kind: MultiClusterObservability
metadata:
  name: observability
spec:
  enableDownsampling: true ①
  observabilityAddonSpec: ②
    enableMetrics: true ③
    interval: 30
  storageConfig: ④
    metricObjectStorage: ⑤
      key: thanos.yaml
      name: thanos-object-storage ⑥
    alertmanagerStorageSize: 1Gi ⑦
    compactStorageSize: 20Gi ⑧
    receiveStorageSize: 20Gi ⑨
    ruleStorageSize: 1Gi ⑩
    storeStorageSize: 10Gi ⑪
    storageClass: ocs-external-storagecluster-ceph-rbd ⑫
  nodeSelector: ⑬
    node-role.kubernetes.io/infra:
```

- ① When set to `true`, optimizes queries for long time intervals by creating pre-aggregated data points. Thanos can quickly retrieve aggregated values for these intervals.
- ② Defines the global settings for all managed clusters that enable the observability add-on.
- ③ When set to `true`, indicates the observability add-on to push metrics to the hub cluster.
- ④ Specifies the storage configurations that the observability service uses.
- ⑤ Defines the object store secret for metrics.
- ⑥ The name of the secret that contains the object store information and credentials
- ⑦ The storage for the Alertmanager stateful sets
- ⑧ The storage for the Thanos compact stateful sets
- ⑨ The storage for the Thanos receive stateful sets
- ⑩ The storage for the Thanos rule stateful sets
- ⑪ The storage for the Thanos store stateful sets
- ⑫ The name of the storage class for the Thanos and Alertmanager stateful sets
- ⑬ Defines the nodes to deploy the observability service components

The creation of the `MultiClusterObservability` resource creates the pods for Thanos, Alertmanager, and Grafana in the `open-cluster-management-observability` namespace.

Verify that all the pods in the `open-cluster-management-observability` namespace have the `Running` status.

```
[user@host ~]$ oc get pods \
-n open-cluster-management-observability
NAME                                                    READY   STATUS
endpoint-observability-operator-86d846cc45-bsp4t        1/1     Running
metrics-collector-deployment-b7798dd6b-p5rn6          1/1     Running
observability-alertmanager-0                           4/4     Running
...output omitted...
```

To validate the enablement of the observability service, verify that the `multicluster-observability-operator` deployment runs the `multicluster-observability-operator` pod in the `open-cluster-management` namespace.

```
[user@host ~]$ oc get deploy multicluster-observability-operator \
-n open-cluster-management
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
multicluster-observability-operator	1/1	1	1	35m

You can validate that the observability service populates the data from the managed clusters by launching the Grafana dashboards. From the RHACM web console, go to **Infrastructure > Clusters** and click the **Grafana** link in the upper-right corner of the page.

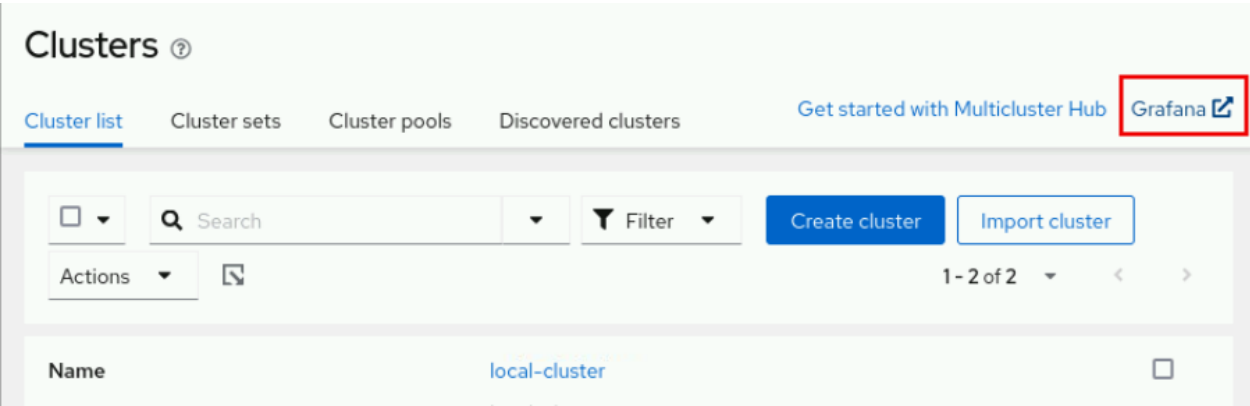


Figure 29. Access Grafana

Grafana displays information about the managed clusters in the dashboards.

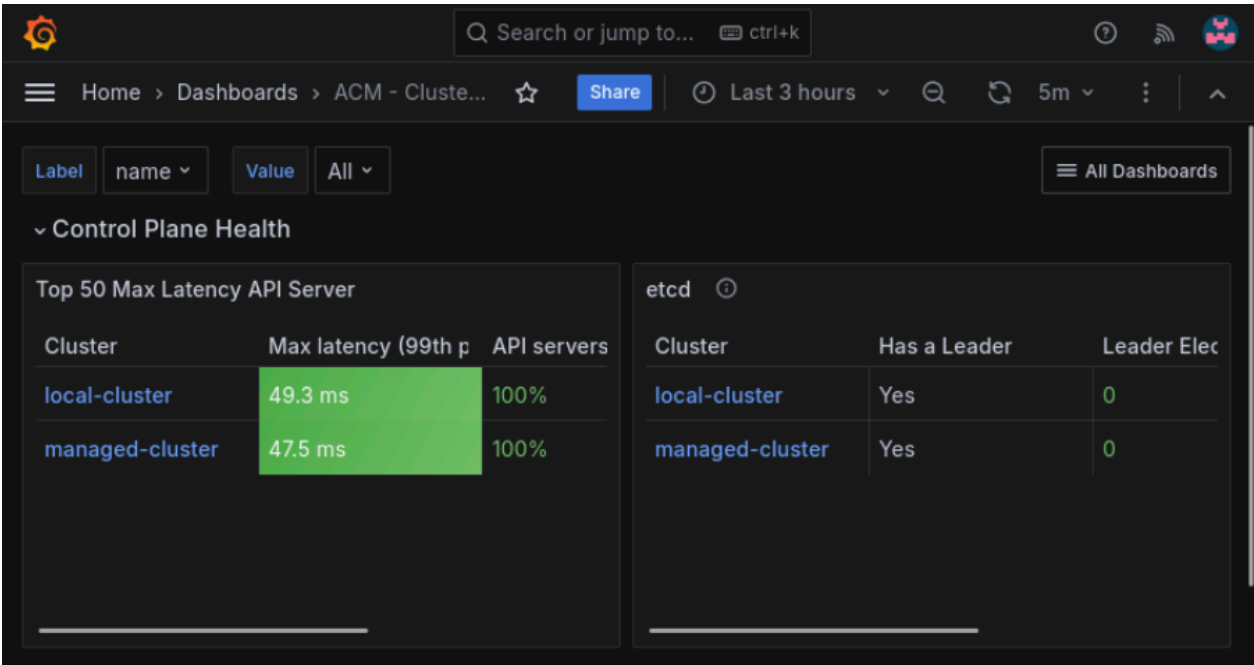


Figure 30. Grafana dashboard

References

For more information about the observability service in RHACM, refer to the *Observability Service* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Observability* documentation at https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/observability/index#observing-environments-intro

For more information about enabling the observability service in RHACM, refer to the *Enabling the Observability Service* section in the *Observability Service* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Observability* documentation at https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/observability/index#enabling-observability-service

4.2. Guided Exercise

Enable the RHACM Observability Stack

Enable the RHACM observability stack and verify that the observability components are deployed successfully in the hub cluster and the managed clusters.

Outcomes

- Enable the RHACM observability components.
- Verify the correct deployment of these components in the managed clusters.
- Locate and access the RHACM Grafana dashboard.
- Go to the Grafana dashboard and analyze the CPU and memory usage of the managed clusters.
- Disable the observability agent in one of the managed clusters.

Prerequisites

As the student user on the workstation machine, use the lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab install do0014l ; lab start observability-enable
```

Instructions

1. Create the open-cluster-management-observability namespace in the hub cluster.

- 1.1. In the terminal window, log in to the hub cluster as the admin user with redhatocp as the password.

```
[student@workstation ~]$ oc login -u admin -p redhatocp \
https://api.ocp4.example.com:6443
Login successful.
...output omitted...
```

- 1.2. Create the open-cluster-management-observability namespace for the observability service. RHACM creates in this namespace the components that enable the observability service.

```
[student@workstation ~]$ oc create namespace \
open-cluster-management-observability
namespace/open-cluster-management-observability created
```

2. Create the multiclusterhub-operator-pull-secret secret in the open-cluster-management-observability namespace.

- 2.1. Extract the secret named pull-secret from the openshift-config namespace, and store it in the DOCKER_CONFIG_JSON variable.

```
[student@workstation ~]$ DOCKER_CONFIG_JSON=$(oc extract \
secret/pull-secret -n openshift-config --to=)
# .dockerconfigjson
```

- 2.2. Verify that the content of the DOCKER_CONFIG_JSON variable contains the pull-secret content.

```
[student@workstation ~]$ echo $DOCKER_CONFIG_JSON
{"auths": {"registry.ocp4.example.com:8443": {"auth": ...output omitted...
```

- 2.3.** Create the `multicloudhub-operator-pull-secret` secret in the `open-cluster-management-observability` namespace by running the following command:

```
[student@workstation ~]$ oc create secret generic \
  multicloudhub-operator-pull-secret \
  -n open-cluster-management-observability \
  --from-literal=.dockerconfigjson="$DOCKER_CONFIG_JSON" \
  --type=kubernetes.io/dockerconfigjson
secret/multicloudhub-operator-pull-secret created
```

- 3.** The `lab_start` command installs OpenShift Data Foundation and deploys the Multicloud Object Gateway component, which provides the S3 endpoint in the central cluster.

- 3.1.** Verify that the `StorageCluster` object is ready by running the following command:

```
[student@workstation ~]$ oc get storagecluster -n openshift-storage
NAME                AGE    PHASE    ...
ocs-storagecluster  8m49s  Ready    ...
```

- 3.2.** Verify that the `NooBaa` object is ready by running the following command:

```
[student@workstation ~]$ oc get noobaa -n openshift-storage
NAME    ...    PHASE    AGE
noobaa  ...    Ready    6m48s
```

- 3.3.** Verify that the `openshift-storage.noobaa.io` storage class exists by running the following command:

```
[student@workstation ~]$ oc get storageclasses -o \
  custom-columns='NAME:metadata.name,PROVISIONER:provisioner'
NAME                PROVISIONER
...output omitted...
openshift-storage.noobaa.io  openshift-storage.noobaa.io/obc
```

- 4.** Create the persistent object store for the observability service.

- 4.1.** Change to the `~/D00014L/labs/observability-enable/` directory.

```
[student@workstation ~]$ cd ~/D00014L/labs/observability-enable/
```

- 4.2.** Edit the `~/D00014L/labs/observability-enable/obc.yaml` file and modify the placeholder text to match the following content:

```
apiVersion: objectbucket.io/v1alpha1
kind: ObjectBucketClaim
metadata:
  name: thanos-obc
  namespace: open-cluster-management-observability
spec:
  storageClassName: openshift-storage.noobaa.io
  generateBucketName: observability-bucket
```

- 4.3.** Create the `thanos-obc` object bucket claim.

```
[student@workstation observability-enable]$ oc apply -f obc.yaml
objectbucketclaim.objectbucket.io/thanos-obc created
```

- 4.4.** Verify that the thanos-obc object bucket claim exists in the open-cluster-management-observability namespace, and that it displays Bound as the status in the PHASE column.

```
[student@workstation observability-enable]$ oc get objectbucketclaim \
-n open-cluster-management-observability
NAME          STORAGE-CLASS          PHASE ...
thanos-obc    openshift-storage.noobaa.io Bound
```

- 5.** Retrieve the S3 bucket information and credentials. You use this data in a later step.

When an object bucket claim is created, OpenShift Data Foundation creates, with the same name, a matching secret and a configuration map, which contain the bucket information and credentials.

- 5.1.** Retrieve the bucket name from the generated configuration map.

```
[student@workstation observability-enable]$ oc extract \
--to=- cm/thanos-obc -n open-cluster-management-observability
# BUCKET_REGION

# BUCKET_SUBREGION

# BUCKET_HOST
s3.openshift-storage.svc
# BUCKET_NAME
observability-bucket-030b2b89-6c7e-4883-a26e-ea8939dfc875
# BUCKET_PORT
443
```

The BUCKET_NAME in the preceding output is different on your system.

- 5.2.** Retrieve the bucket credentials from the generated secret.

```
[student@workstation observability-enable]$ oc extract \
--to=- secret/thanos-obc -n open-cluster-management-observability
# AWS_ACCESS_KEY_ID
n0v00rBFB1F4Bc1NZaIv
# AWS_SECRET_ACCESS_KEY
sc8Z3KpNF8Ik035inA0ji9Nkz3g2hBMnVe7EFWIT
```

The credentials in the preceding output are different on your system.

- 5.3.** Identify the public URL for the S3 endpoint from the s3 route in the openshift-storage namespace. You use this URL to connect to the S3 bucket from the workstation machine in the next step.

```
[student@workstation observability-enable]$ oc get route s3 \
-n openshift-storage
NAME    HOST/PORT          ...
s3      s3-openshift-storage.apps.ocp4.example.com ...
```

- 6.** Create the thanos-object-storage secret within the open-cluster-management-observability namespace to access the object storage.

- 6.1.** Edit the ~/DO0014L/labs/observability-enable/secret.yaml file and update the placeholder text. Use the S3 credentials, the S3 endpoint URL, and the bucket name from the previous step.


```
apiVersion: v1
kind: Secret
metadata:
  name: thanos-object-storage
  namespace: open-cluster-management-observability
type: Opaque
stringData:
  thanos.yaml: |
    type: s3
    config:
      bucket: observability-bucket-030b2b89-6c7e-4883-a26e-ea8939dfc875
      endpoint: s3-openshift-storage.apps.ocp4.example.com
      insecure: true
      access_key: n0v00rBFB1F4Bc1NZaIv
      secret_key: sc8Z3KPnF8Ik035inA0ji9Nkz3g2hBMnVe7EFWIT
```

6.2. Create the thanos-object-storage secret.

```
[student@workstation observability-enable]$ oc apply -f secret.yaml
secret/thanos-object-storage created
```

7. Create the MultiClusterObservability custom resource instance to enable the observability service.

7.1. Edit the ~/DO0014L/labs/observability-enable/mcobs.yaml file and modify the placeholder text to match the following content:

```
apiVersion: observability.open-cluster-management.io/v1beta2
kind: MultiClusterObservability
metadata:
  name: observability
spec:
  observabilityAddonSpec:
    enableMetrics: true
  storageConfig:
    metricObjectStorage:
      key: thanos.yaml
      name: thanos-object-storage
    storageClass: nfs-storage
    alertmanagerStorageSize: 1Gi
    compactStorageSize: 1Gi
    receiveStorageSize: 1Gi
    ruleStorageSize: 1Gi
    storeStorageSize: 1Gi
```

7.2. Create the MultiClusterObservability object.

```
[student@workstation observability-enable]$ oc apply -f mcobs.yaml
multiclusterobservability.observability.open-cluster-management.io/observability created
```

7.3. Verify that the multicluster-observability-operator deployment runs the multicluster-observability-operator pod in the open-cluster-management namespace.

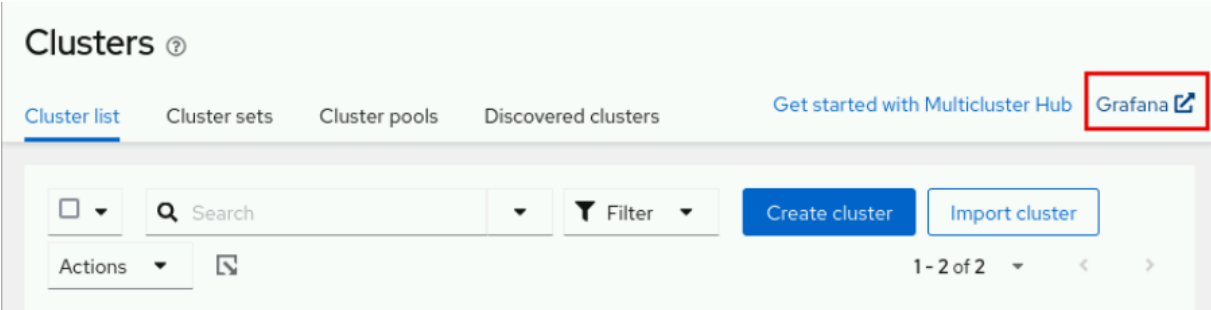
```
[student@workstation observability-enable]$ oc get deploy \
  multicluster-observability-operator \
  -n open-cluster-management
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
multicluster-observability-operator	1/1	1	1	5m

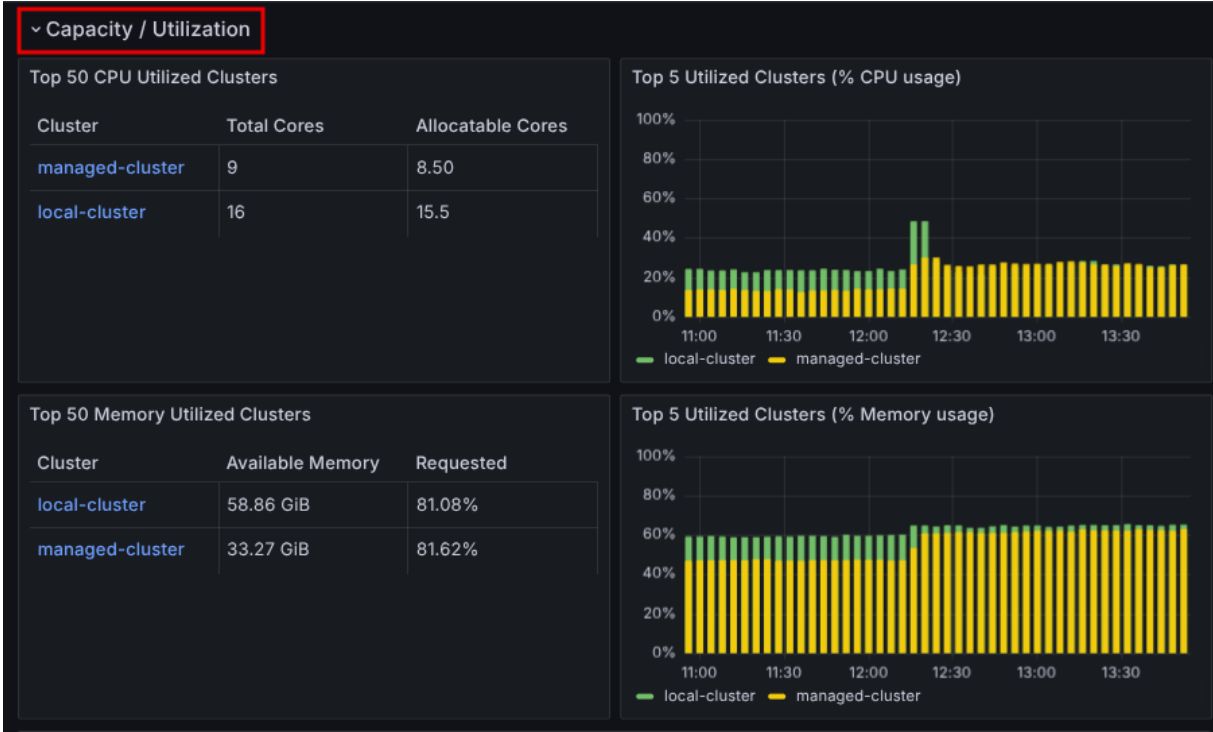
7.4. Verify that all the pods in the open-cluster-management-observability namespace display Running in the STATUS column. It can take up to three minutes for the pods to reach the Running status.

```
[student@workstation observability-enable]$ oc get po -n \
open-cluster-management-observability
NAME                                     READY   STATUS   ...
endpoint-observability-operator-86d846cc45-bsp4t   1/1     Running
metrics-collector-deployment-b7798dd6b-p5rn6      1/1     Running
observability-alertmanager-0                    4/4     Running
observability-alertmanager-1                    4/4     Running
observability-alertmanager-2                    4/4     Running
observability-grafana-7c8ddc4d5-bnbwn            3/3     Running
observability-grafana-7c8ddc4d5-d9gjb            3/3     Running
observability-observatorium-api-6f9748959-xbj2j   1/1     Running
observability-observatorium-api-6f9748959-z2l7v   1/1     Running
...output omitted...
```

8. Go to the Grafana dashboards and analyze the CPU and memory usage of the managed clusters.
- 8.1. Open a web browser and go to <https://console-openshift-console.apps.ocp4.example.com>
- 8.2. Select the **Red Hat Identity Management** identity provider and log in as the admin user with redhatocp as the password.
- 8.3. Switch to the RHACM web console, and go to **Infrastructure > Clusters**.
- 8.4. Click the **Grafana** link in the upper-right corner.



- 8.5. In a new browser tab that opens, log in as the admin user with redhatocp as the password.
- 8.6. Grafana loads the **Home:Dashboards > ACM - Clusters Overview** dashboard by default. Scroll down to the **Capacity / Utilization** section, and locate the CPU and memory usage of the managed clusters. The data can take up to seven minutes to appear after the observability service is installed.



9. Disable the metrics collection from the managed-c cluster cluster. This feature is useful to avoid overloading the observability service with metrics from clusters that are used for stage environments, for example.

- 9.1. Switch to the RHACM web console, and go to **Infrastructure > Clusters**.
- 9.2. In the **Cluster list** tab, click **managed-cluster**.
- 9.3. On the **managed-cluster** page, click the **Add-ons** tab. Locate the **observability-controller** add-on in the list.

Clusters > managed-cluster

managed-cluster

OverviewNodesAdd-ons

Q Search

Name ↑	Status	Message
application-manager	✓ Available	application-manager add-on is available.
cert-policy-controller	✓ Available	cert-policy-controller add-on is available.
cluster-proxy	✓ Available	cluster-proxy add-on is available.
config-policy-controller	✓ Available	config-policy-controller add-on is available.
governance-policy-framework	✓ Available	governance-policy-framework add-on is available.
managed-serviceaccount	✓ Available	managed-serviceaccount add-on is available.
observability-controller	✓ Available	observability-controller add-on is available.
search-collector	✓ Available	search-collector add-on is available.
work-manager	✓ Available	work-manager add-on is available.

9.4. Switch to the terminal window, and change to the home directory.

```
[student@workstation observability-enable]$ cd ~  
[student@workstation ~]$
```

9.5. List the managed clusters.

```
[student@workstation ~]$ oc get managedclusters  
NAME                HUB ACCEPTED   MANAGED CLUSTER URLS    ...  
local-cluster       true           https://api.ocp4.example.com:6443  
managed-cluster     true           https://console-openshift-console...
```

9.6. Add the `observability=disabled` label to the ManagedCluster custom resource named `managed-cluster`.

```
[student@workstation ~]$ oc label managedcluster managed-cluster \  
  observability=disabled -n open-cluster-management  
managedcluster.cluster.open-cluster-management.io/managed-cluster labeled
```

9.7. Log in to the `managed-cluster` cluster as the `admin` user with `redhatocp` as the password.

```
[student@workstation ~]$ oc login -u admin -p redhatocp \  
  https://api.ocp4-mng.example.com:6443  
Login successful.  
...output omitted...
```

9.8. Verify that no pods are running in the `open-cluster-management-addon-observability` namespace.

```
[student@workstation ~]$ oc get pod -n \  
  open-cluster-management-addon-observability  
No resources found in open-cluster-management-addon-observability namespace.
```

9.9. Switch to the RHACM web console, and go to **Infrastructure > Clusters**. Verify that the **observability-controller** add-on is not in the list.

The Grafana dashboard can show cached information about the `managed-cluster` cluster after disabling the observability add-on. All the information about the `managed-cluster` cluster disappears in a few hours.

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish observability-enable
```

4.3. Customizing the RHACM Observability Stack

Objectives

- Configure the RHACM observability stack with custom rules to efficiently process frequently needed or expensive queries, and to trigger alerts when certain conditions or events occur.

Observability Service Customizations in RHACM

In production environments, you might need to adapt the observability service to the specific needs of your fleet of clusters.

This customization usually involves creating custom Prometheus metrics and alerting rules, updating the MultiClusterObservability custom resource replicas, and adding or modifying existing dashboards in Grafana.

With the RHACM observability service, you can also configure Prometheus to forward metrics to an external or centralized alerting engine.

Configuring the Multicluster Observability Custom Resource

The MultiClusterObservability custom resource contains all the required parameters to configure the observability service, such as the persistent storage for Thanos and Alertmanager, the metrics retention policy, and the number of replicas of different components.

In a heavily loaded production environment, you might need to increase the number of replicas of the observability pods. You update the replicas parameter value for the relevant component.

The following YAML example is of a MultiClusterObservability object that sets the number of replicas of the Thanos receive stateful set to 6.

```
apiVersion: observability.open-cluster-management.io/v1beta2
kind: MultiClusterObservability
metadata:
  name: observability
spec:
  enableDownsampling: true
  observabilityAddonSpec:
    enableMetrics: true
    interval: 30
  advanced:
    receive:
      replicas: 6
  ...output omitted...
```

 **Note**

Consult the References section to learn the complete definition of the MultiClusterObservability custom resource.

Prometheus Rules

Prometheus supports two types of rules that can be configured and evaluated at regular intervals: recording rules and alerting rules.

Recording rules

Recording rules enable you to precompute expressions that are often needed or that are computationally expensive. Prometheus saves the resulting metric as a new set of time series. This process can significantly speed up queries, especially for dashboards that repeatedly query the same expressions.

A recording rule requires a `record` field for the name of the output time series, an `expr` field for the PromQL expression to evaluate, and an optional `labels` field for additional labels. The following YAML example defines the `code:prometheus_http_requests_total:sum` metric:

```
- record: code:prometheus_http_requests_total:sum
  expr: sum by (code) (prometheus_http_requests_total)
  labels:
    env: stage
```

Alerting rules

Alerting rules enable you to define conditions for alerts based on Prometheus expressions, and to receive notifications when these conditions are met.

An alerting rule requires an `alert` field, an `expr` field for the PromQL expression, and optional `for`, `keep_firing_for`, `labels`, and `annotations` clauses. The following YAML example defines the `HighRequestLatency` alerting rule:

```
- alert: HighRequestLatency
  expr: job:request_latency_seconds:mean5m{job="myjob"} > 0.5
  for: 10m
  keep_firing_for: 5m
  labels:
    severity: page
  annotations:
    summary: High request latency
```

RHACM Metrics

The RHACM observability service provides a set of default metrics; the service receives in-cluster platform metrics from Prometheus, and federates a subset of the metrics into Thanos.

To view the default metrics, see the `observability-metrics-allowlist` configuration map within the `open-cluster-management-observability` namespace. The configuration map also defines a set of recording rules. You cannot change the default metrics in the `observability-metrics-allowlist` configuration map.

RHACM Custom Metrics

With RHACM, you can add custom metrics. The metrics can be exported as platform metrics or as user workload metrics. User workload metrics are application-specific measurements that users or third-parties define. You can also collect custom metrics from a single managed cluster only.

You define both platform metrics and user workload metrics by creating the `observability-metrics-custom-allowlist` configuration map within the `open-cluster-management-observability` namespace.

Creating the configuration map causes the `metrics-collector-deployment` pod to restart in the `open-cluster-management-observability` namespace.

The following YAML example is of a `observability-metrics-custom-allowlist` configuration map, which adds the `node_memory_MemTotal_bytes` metric and the

`apiserver_request_duration_seconds:histogram_quantile_90` metric.

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: observability-metrics-custom-allowlist
  namespace: open-cluster-management-observability
data:
  metrics_list.yaml: | ①
    names: ②
    - node_memory_MemTotal_bytes
    recording_rules: ③
    - record: apiserver_request_duration_seconds:histogram_quantile_90
      expr: histogram_quantile(0.90,sum(rate(apiserver_request_duration_seconds_bucket {job=\"apiserver\",
      verb!=\"WATCH\"}[5m])) by (verb,le))
```

- ① The resulting file's name, which you cannot change
- ② The names of the metrics to collect from the managed clusters
- ③ The recording rules that you create

To collect custom metrics from only a single managed cluster, apply the `observability-metrics-custom-allowlist` configuration map on your managed cluster in the `open-cluster-management-addon-observability` namespace.

To add user workload metrics, you set the namespace to capture the metric in the configuration map. The following example monitors the user workload metric named `sample_metrics` from the `monitored_namespace` namespace.

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: observability-metrics-custom-allowlist
  namespace: monitored_namespace ①
data:
  uwl_metrics_list.yaml: ②
    names: ③
    - sample_metrics
```

- ① The name of the namespace to monitor
- ② The resulting file's name, which you cannot change
- ③ The names of the metrics to collect from the namespace

RHACM Alerts

The RHACM observability service provides a set of default alerting rules. To view the default alerts, refer to the `thanos-ruler-default-rules` configuration map within the `open-cluster-management-observability` namespace.

RHACM Custom Alerts

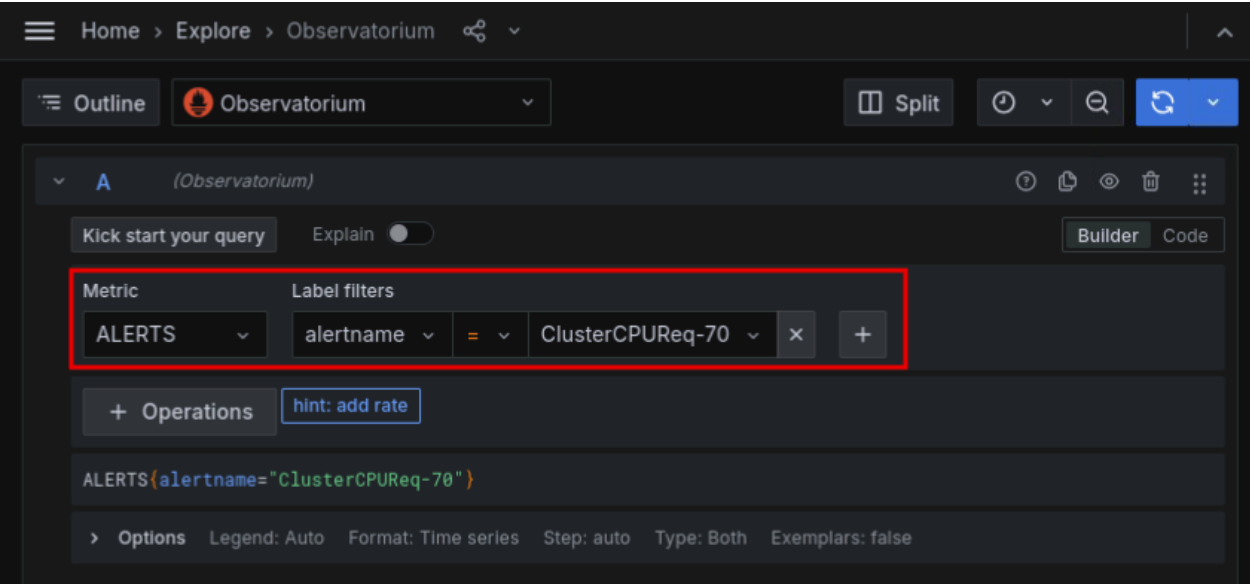
You add custom alerting rules by creating the `thanos-ruler-custom-rules` configuration map within the `open-cluster-management-observability` namespace. Creating the configuration map causes the `observability-thanos-rule` pods to restart in the `open-cluster-management-observability` namespace.

The following YAML example is of a `thanos-ruler-custom-rules` configuration map, which adds a custom rule named `ClusterCPUReq-70`. The alert notifies when the CPU usage on a cluster is greater than 70%.

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: thanos-ruler-custom-rules
  namespace: open-cluster-management-observability
data:
  custom_rules.yaml: | ①
    groups: ②
      - name: cluster-health
        rules:
          - alert: ClusterCPUReq-70 ③
            annotations:
              summary: Notify when CPU requests on a cluster are greater than the defined utilization limit
              description: "The cluster has a high CPU usage: {{ $value }} core for {{ $labels.cluster }} {{ $labels.clusterID }}." ④
            expr: | ⑤
              sum(namespace_cpu:kube_pod_container_resource_requests:sum) by (clusterID, cluster) /
              sum(kube_node_status_allocatable{resource="cpu"}) by (clusterID, cluster) > 0.7
            for: 5s ⑥
            labels:
              severity: critical
```

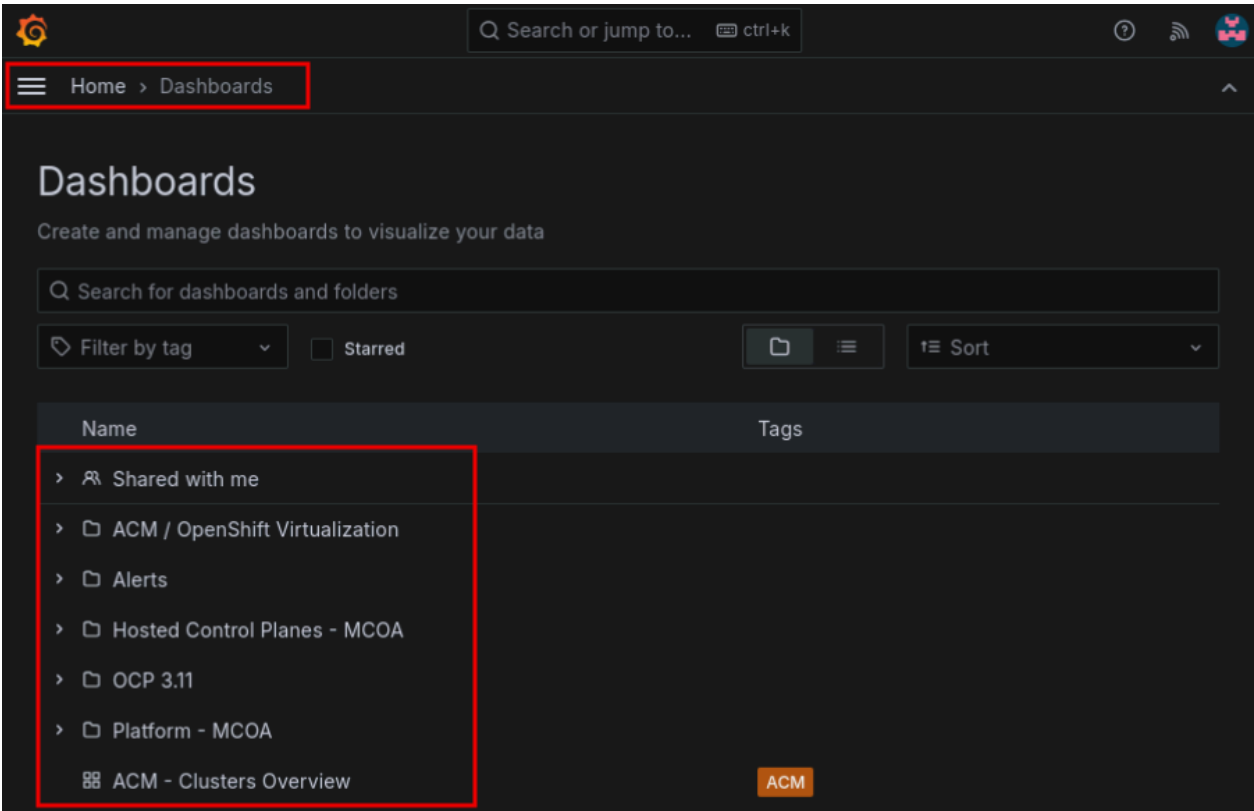
- ① The resulting file's name, which you cannot change
- ② Defines a group of rules
- ③ The name of the alert
- ④ The notification message to the user when the alert fires
- ⑤ The PromQL expression to evaluate
- ⑥ The time period that Prometheus checks that the alert continues to be active before firing the alert

You can check the custom alert in Grafana from the **Explore** page. Select **ALERTS** from the **Metric** drop-down list, select a **alername** from the **Label filters** drop-down list, and select the custom alert from the **Select value** drop-down list.



Grafana Dashboards in RHACM

When you enable the observability service in RHACM, the MultiClusterObservability custom resource creates a Grafana console with more than 15 dashboards. From the Grafana dashboard, go to **Home > Dashboards** to view the default dashboards grouped by type.



Some of the Grafana dashboards come from the Red Hat OpenShift Container Platform monitoring stack, and include a cluster filter, which refers to each managed cluster in the fleet.

Review the ConfigMap objects in the `open-cluster-management-observability` namespace that have the `grafana-dashboard-` prefix to learn the definitions of the dashboards.

RHACM enables you to create your own custom dashboards by setting up a Grafana developer instance. From a Grafana developer instance, you can design your own dashboards from the Grafana UI, or by creating a configuration map from the command line.

 **Note**
Consult the References section to learn how to set up a Grafana developer instance, and how to design your own dashboards.

Forwarding Metrics to External Services

After you enable observability, RHACM automatically sends the alerts from your managed clusters to the hub cluster.

With RHACM, you can integrate external messaging tools such as email, Slack, and PagerDuty to receive notifications from Alertmanager. You must override the `alertmanager-config` secret in the `open-cluster-management-observability` namespace to add integrations or to configure routes for Alertmanager.

The following YAML example configures the `alertmanager-config` secret to send alerts to Slack:

```
...output omitted...
route:
  receiver: 'slack-notifications'
  group_by: [alertname, datacenter, app]

receivers:
- name: 'slack-notifications'
  slack_configs:
  - channel: '#alerts'
    text: 'https://internal.myorg.net/wiki/alerts/{{ .GroupLabels.app }}/{{ .GroupLabels.alertname }}'
```

To disable alert forwarding for managed clusters, add the `mco-disable-alerting: "true"` annotation to the `MultiClusterObservability` custom resource.

References

For more information about adding custom metrics, refer to the *Adding Custom Metrics* section of the *Observability Service* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Observability* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/observability/index#adding-custom-metrics

For more information about creating custom rules, refer to the *Creating Custom Rules* section of the *Observability Service* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Observability* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/observability/index#creating-custom-rules

For more information about designing custom Grafana dashboards, refer to the *Using Grafana Dashboards* section of the *Observability Service* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Observability* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/apis/index#rhacm-docs_apis_multiclusterobservability_jsondefinitions

For more information about forwarding alerts, refer to the *Forwarding Alerts* section of the *Observability Service* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Observability* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/observability/index#forward-alerts

For more information about all the fields of the `MultiClusterObservability` custom resource, refer to the *MultiClusterObservability* section of the *APIS Guide* at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/apis/index#rhacm-docs_apis_multiclusterobservability_jsonmulticlusterobservability

For more information about recording rules, visit the Prometheus documentation at

https://prometheus.io/docs/prometheus/latest/configuration/recording_rules/

For more information about alerting rules, visit the Prometheus documentation at

https://prometheus.io/docs/prometheus/latest/configuration/alerting_rules/

For more information about PromQL, visit the Prometheus documentation at

<https://prometheus.io/docs/prometheus/latest/querying/basics/>

4.4. Guided Exercise

Customize the RHACM Observability Stack

Configure an existing RHACM observability service to handle a higher load of monitoring data from the managed clusters. Then, you create a custom recording rule, and a custom alerting rule that triggers when the CPU requests of a cluster are above a certain value. Finally, you disable the RHACM observability service.

Outcomes

- Increase the number of replicas of the observability metrics receiver pods.
- Create a Prometheus custom alerting rule that triggers an alert when the CPU requests of a cluster are greater than a certain value.
- Use Grafana to verify the creation and the status of the alerting rule.
- Add a platform metric by creating a custom platform recording rule.
- Remove the observability components from RHACM.

Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab install do0014l ; lab start observability-customize
```

Instructions

The `lab start` command enables the observability service.

1. To handle a higher load of monitoring data from the managed clusters, increase the receiver pod replicas to six for the observability metrics.

- 1.1. In the terminal window, log in to the hub cluster as the `admin` user with `redhatocp` as the password.

```
[student@workstation ~]$ oc login -u admin -p redhatocp \
https://api.ocp4.example.com:6443
Login successful.
...output omitted...
```

- 1.2. Verify that the current number of replicas of the `observability-thanos-receive-default` stateful set is three.

```
[student@workstation ~]$ oc get sts \
observability-thanos-receive-default \
-n open-cluster-management-observability
```

NAME	READY	AGE
observability-thanos-receive-default	3/3	10m

- 1.3. Edit the `MultiClusterObservability` object, and update the `spec` section so that it contains the following highlighted configuration block. The `MultiClusterObservability` custom resource is not a namespace-scoped resource.

```
[student@workstation ~]$ oc edit multiclusterobservability \
  observability
...output omitted...
spec:
  advanced:
    receive:
      replicas: 6
    enableDownsampling: true
...output omitted...
```

- 1.4. Verify that the number of observability-thanos-receive-default pods increases to six.

```
[student@workstation ~]$ oc get pods -w \
  -n open-cluster-management-observability \
  -l app.kubernetes.io/name=thanos-receive
NAME                                READY   STATUS   ...
observability-thanos-receive-default-0 1/1     Running
observability-thanos-receive-default-1 1/1     Running
observability-thanos-receive-default-2 1/1     Running
observability-thanos-receive-default-3 1/1     Running
observability-thanos-receive-default-4 1/1     Running
observability-thanos-receive-default-5 1/1     Running
```

Press Ctrl+C to exit the watch command.

2. Create a Prometheus alerting rule so that cluster administrators receive alerts when the CPU requests of any cluster are above 60% of the total allocatable CPU.

- 2.1. Change to the ~/D00014L/labs/observability-customize directory.

```
[student@workstation ~]$ cd ~/D00014L/labs/observability-customize
[student@workstation observability-customize]$
```

- 2.2. Review the custom_rule.yaml file that defines the thanos-ruler-custom-rules configuration map. The ConfigMap object defines the alerting rule by using the PromQL expression language.

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: thanos-ruler-custom-rules
  namespace: open-cluster-management-observability
data:
  custom_rules.yaml: |
    groups:
      - name: cluster-health
        rules:
          - alert: ClusterCPUReq-60
            annotations:
              summary: Notify when CPU requests on a cluster are greater than the defined utilization
            limit
              description: "The cluster {{ $labels.cluster }} has an elevated percentage of CPU
            requests: {{ $labels.clusterID }}."
            expr: |
              sum(namespace_cpu:kube_pod_container_resource_requests:sum) by (clusterID, cluster) /
              sum(kube_node_status_allocatable{resource="cpu"}) by (clusterID, cluster) > 0.6
            for: 5s
            labels:
              cluster: "{{ $labels.cluster }}"
              severity: critical
```

- 2.3. Create the thanos-ruler-custom-rules configuration map in the open-cluster-management-observability namespace.

```
[student@workstation observability-customize]$ oc apply -f custom-rules.yaml  
configmap/thanos-ruler-custom-rules created
```

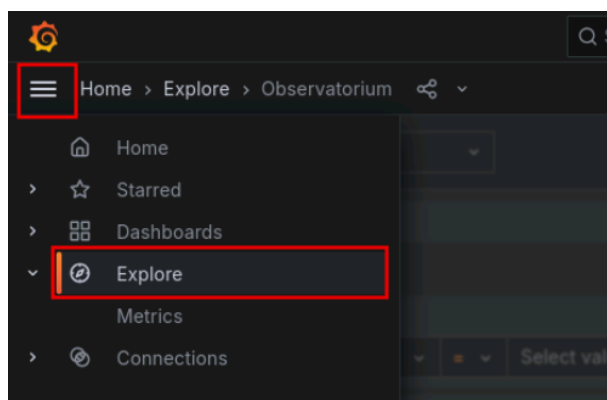
The creation of the configuration map restarts the observability-thanos-rule pods in the open-cluster-management-observability namespace.

- 2.4. Verify that the new configuration is applied. You can monitor the restart of the pods in the open-cluster-management-observability namespace by running the following watch command:

```
[student@workstation observability-customize]$ oc get pods \
-n open-cluster-management-observability -w | grep rule  
observability-thanos-rule-0      2/2      Running    0      1m  
observability-thanos-rule-1      2/2      Running    0      1m  
observability-thanos-rule-2      2/2      Running    0      1m
```

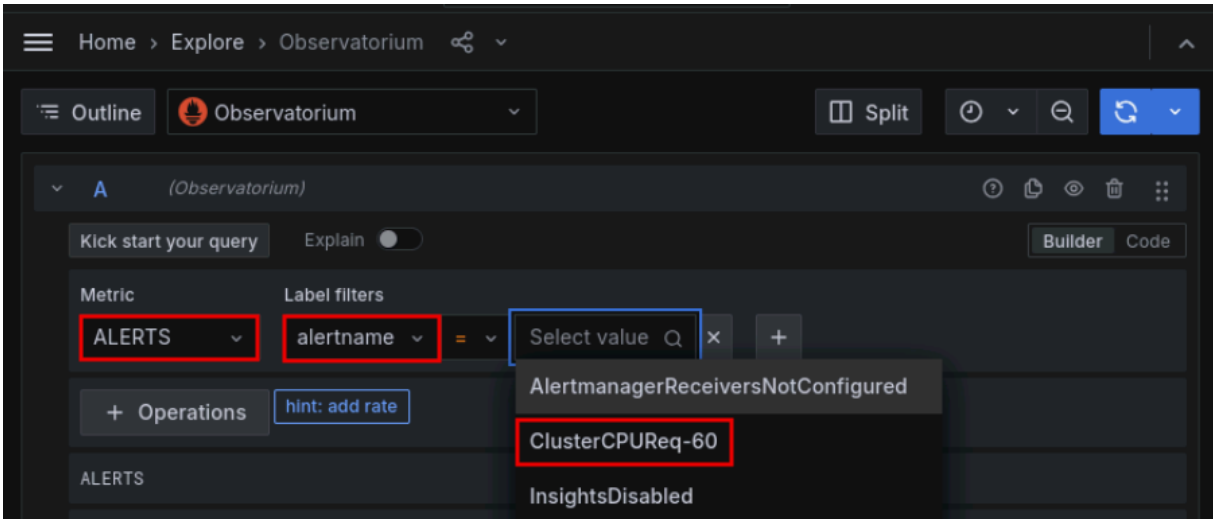
Press Ctrl+C to exit the watch command. After the observability-thanos-rule pods restart, the alerting rule is visible from the RHACM Grafana dashboard.

3. Go to the Grafana dashboard to verify that the custom alert is visible for the cluster named local-cluster.
- 3.1. On the workstation machine, open a web browser and go to <https://console-openshift-console.apps.ocp4.example.com>
- 3.2. Select the **Red Hat Identity Management** identity provider and log in as the admin user with redhatocp as the password.
- 3.3. Switch to the RHACM web console, and go to **Infrastructure > Clusters**.
- 3.4. Click the **Grafana** link in the upper-right corner.
- 3.5. In a new browser tab that opens, log in as the admin user with redhatocp as the password.
- 3.6. From the main menu of the Grafana dashboard, click **Explore**.

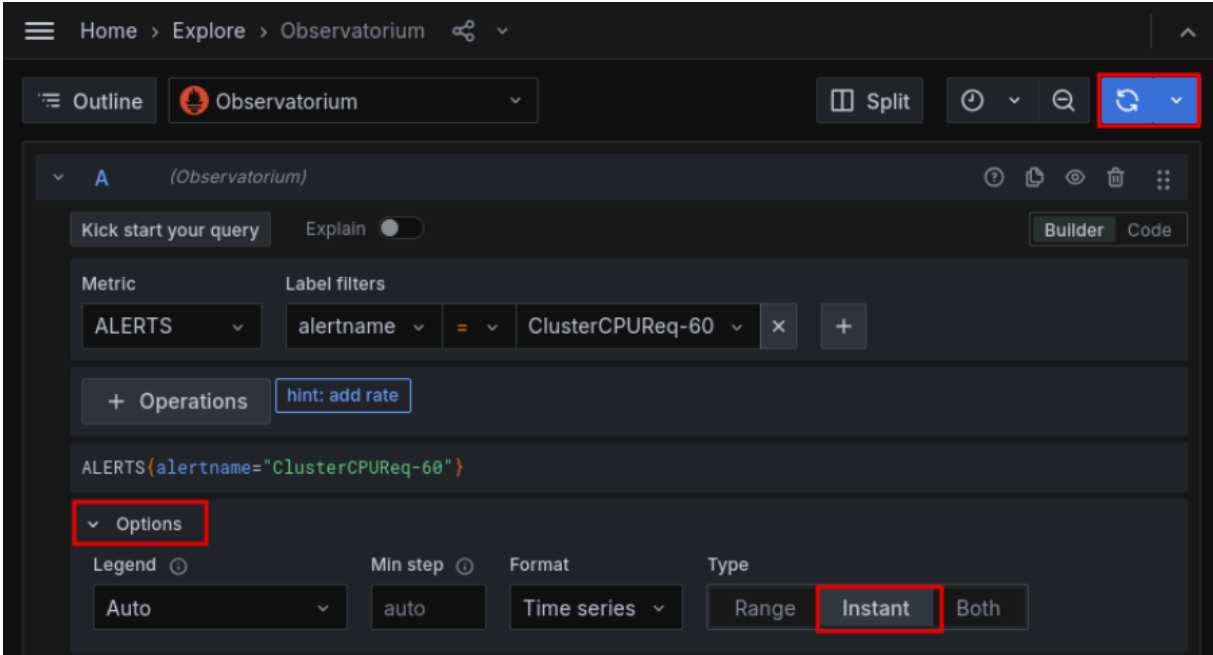


- 3.7. On the explore page, select ALERTS from the **Metric** drop-down list. Select alertname from the **Label filters** drop-down list, and select **ClusterCPUReq-60** as the value. If the drop-down list does not show alertname, then type it.

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright



3.8. Expand the **Options** section, and select the **Instant** query type. Click the **Run query** button in the upper-right corner.



3.9. Verify that the **Table** section contains a row with the ClusterCPUReq-60 custom alerting rule that is firing in the alertstate column for the local-cluster cluster.

Table					
Time	__name__	alertname	alertstate	cluster	cluste
2025-07-03 05:13:17	ALERTS	ClusterCPUReq-60	firing	local-cluster	6c8c6

4. Add a platform metric by creating a custom platform recording rule. The recording rule expression calculates the 90th percentile latency over the past 5 minutes for each type of API request, excluding WATCH requests, to the Kubernetes API server.

4.1. Review the platform-recording.yaml file, which contains the observability-metrics-custom-allowlist configuration map. The configuration map defines the metric named apiserver_request_duration_seconds:histogram_quantile_90 by using the PromQL expression language.

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: observability-metrics-custom-allowlist
  namespace: open-cluster-management-observability
data:
  metrics_list.yaml: |
    recording_rules:
      - record: apiserver_request_duration_seconds:histogram_quantile_90
        expr:
          histogram_quantile(0.90,sum(rate(apiserver_request_duration_seconds_bucket{job=\"apiserver\",verb!=
            \"WATCH\"}[5m])) by (verb,le))
```

- 4.2. Create the observability-metrics-custom-allowlist configuration map in the open-cluster-management-observability namespace.

```
[student@workstation observability-customize]$ oc apply -f \
  platform-recording.yaml
configmap/observability-metrics-custom-allowlist created
```

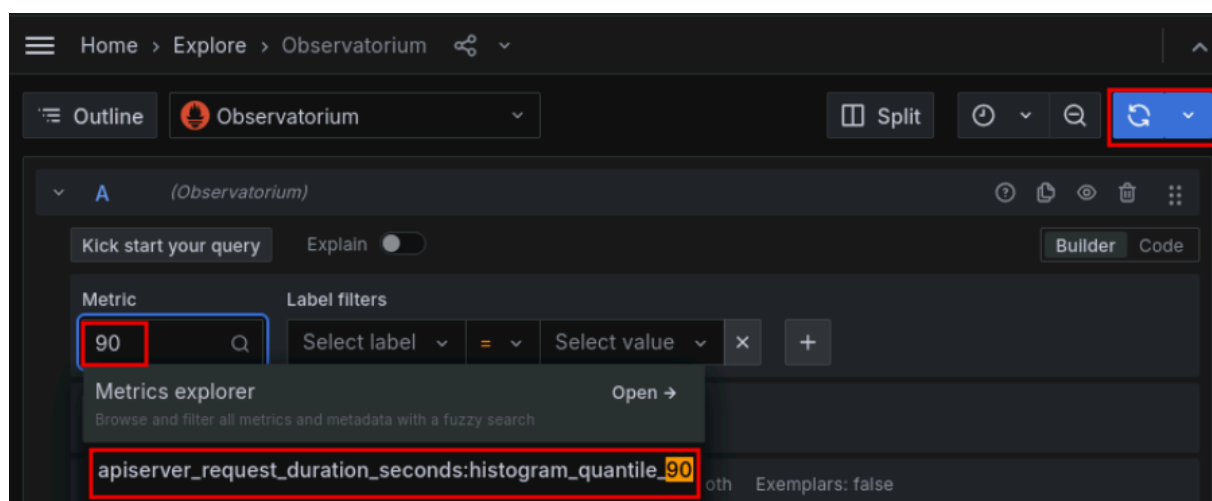
Creating the configuration map causes the metrics-collector-deployment pod to restart in the open-cluster-management-observability namespace.

- 4.3. Verify that the new configuration is applied. You can monitor the restart of the pod in the open-cluster-management-observability namespace by running the following watch command:

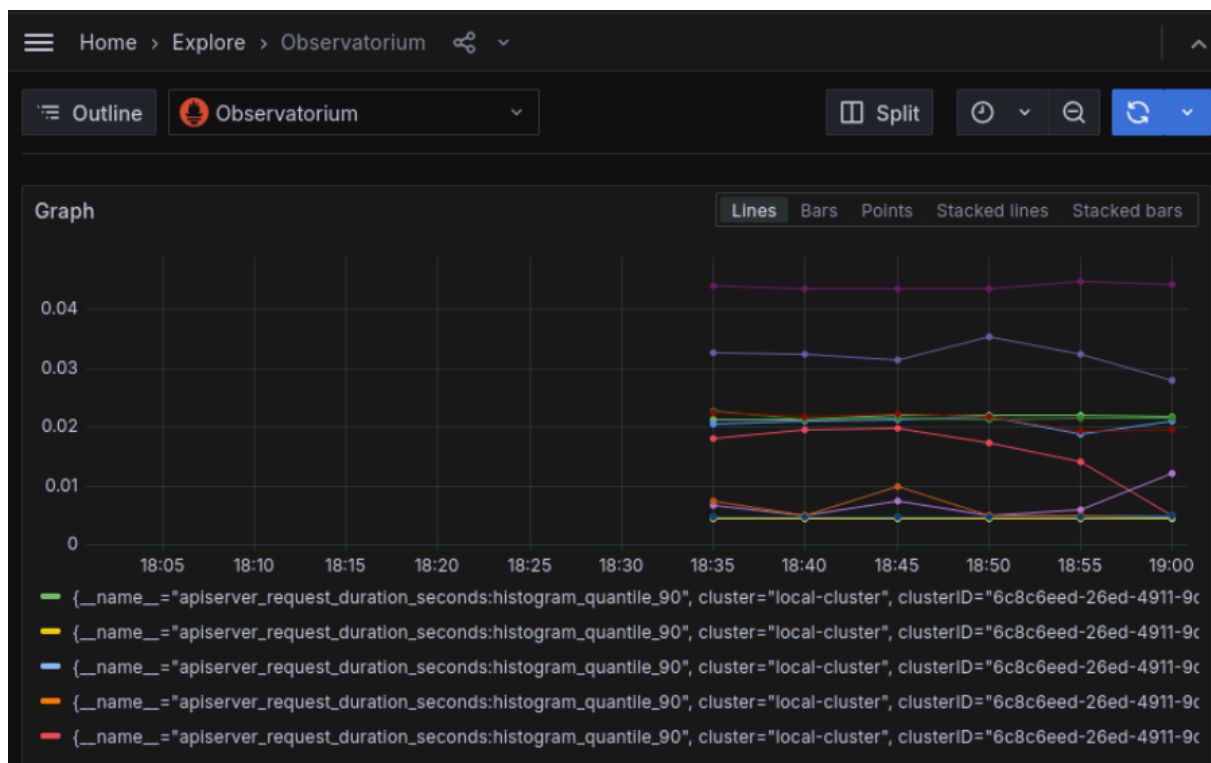
```
[student@workstation observability-customize]$ oc get pods \
  -n open-cluster-management-observability -w | grep "^metrics-collector"
metrics-collector-deployment-... 1/1 Running 0 1m
```

Press Ctrl+C to exit the watch command. After the metrics-collector-deployment pod restarts, the recording rule is visible from the RHACM Grafana dashboard.

- 4.4. Switch to the Grafana dashboard. From the main menu of the Grafana dashboard, click **Explore**.
- 4.5. On the explore page, type 90 in the **Metric** drop-down list, and the metric name appears. Select the **apiserver_request_duration_seconds:histogram_quantile_90** metric, and click the **Run query** button in the upper-right corner.



- 4.6. Verify that the **Graph** section shows the data from the metric. The data might take a few minutes to appear in the graph.



4.7. Switch to the terminal window, and change to the home directory.

```
[student@workstation observability-customize]$ cd ~
[student@workstation ~]$
```

5. Remove the observability components from RHACM.

5.1. Remove the MultiClusterObservability object named observability by running the following command:

```
[student@workstation ~]$ oc delete multiclusterobservability \
  observability
multiclusterobservability.observability.open-cluster-management.io "observability" deleted
```

5.2. Verify that all the pods in the open-cluster-management-observability namespace are deleted by running the following watch command. You might see the metrics-collector-deployment pod in the Error status before it is deleted.

```
[student@workstation ~]$ watch oc get pods \
  -n open-cluster-management-observability
No resources found in open-cluster-management-observability namespace.
```

5.3. Remove the open-cluster-management-observability namespace. The deletion of the namespace takes a few seconds to complete.

```
[student@workstation ~]$ oc delete namespace \
  open-cluster-management-observability
namespace "open-cluster-management-observability" deleted
```

Finish

On the workstation machine, use the lab command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.


```
[student@workstation ~]$ lab finish observability-customize
```

4.5. Lab

Enable and Customize the RHACM Observability Stack

Enable the RHACM observability stack. Increase the number of replicas of the observability components. Create a custom alerting rule that triggers an alert when the memory requests of a cluster are greater than 70%. Create a platform metric by creating a recording rule. Use Grafana to view the managed cluster dashboards, the new alerting rule, and the new metric.

Outcomes

- Enable the RHACM observability stack.
- Verify the deployment of the observability stack in the managed clusters.
- Increase the number of replicas of the Grafana pods.
- Create a custom alerting rule that triggers an alert when the memory requests of a cluster are greater than a defined value.
- Create a platform metric for a single managed cluster.
- Use Grafana to verify the creation and the status of the alerting rule, and the platform metrics.

Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab install do0014l ; lab start observability-review
```

Instructions

In this exercise, you enable the RHACM observability service, and customize the `MultiClusterObservability` object to increase the number of replicas of the observability components.

You create a custom alerting rule that triggers an alert when the memory requests are greater than 70%. You also create a platform metric for the cluster named `managed-cluster` by creating a recording rule. Finally, you use Grafana to view the managed cluster dashboards, the new alerting rule, and the new metrics.

The classroom environment provides two RHOCP clusters: an RHACM hub cluster and a managed cluster. You can access those clusters by using the following parameters:

RHACM hub cluster web console URL

`https://console-openshift-console.apps.ocp4.example.com`

RHACM hub cluster API URL

`https://api.ocp4.example.com:6443`

Managed cluster web console URL

`https://console-openshift-console.apps.ocp4-mng.example.com`

Managed cluster API URL

`https://api.ocp4-mng.example.com:6443`

Administrator username for both clusters

admin (select the **Red Hat Identity Management** identity provider)

Administrator password for both clusters

redhatocp

The lab command installs OpenShift Data Foundation and deploys the Multicloud Object Gateway component, which provides the S3 endpoint in the hub cluster.

1. Create the `open-cluster-management-observability` namespace in the hub cluster, and create the `multicloudhub-operator-pull-secret` secret in the `open-cluster-management-observability` namespace. To create the `multicloudhub-operator-pull-secret` secret, you use the `DOCKER_CONFIG_JSON` variable, which stores the contents of the secret named `pull-secret` from the `openshift-config` namespace.
2. Create an object bucket claim named `thanos-obc`, and use the `openshift-storage.noobaa.io` storage class. Create the `thanos-object-storage` secret within the `open-cluster-management-observability` namespace that uses the `thanos-obc` object bucket. The secret contains the object bucket information and credentials. You can use the `~/DO0014L/solutions/observability-review/obc.yaml` file to create the object bucket claim. You can use the `~/DO0014L/labs/observability-review/secret.yaml` file as a reference to create the `thanos-object-storage` secret.
3. Create the `MultiClusterObservability` custom resource instance to enable the observability service. To create the `MultiClusterObservability` object, use the following information:

`metadata.name`

`observability`

`spec.observabilityAddonSpec.enableMetrics`

`true`

`spec.storageConfig.metricObjectStorage.name`

`thanos-object-storage`

`spec.storageConfig.storageClass`

`nfs-storage`

`spec.storageConfig.alertmanagerStorageSize`

`1Gi`

`spec.storageConfig.compactStorageSize`

`1Gi`

`spec.storageConfig.receiveStorageSize`

`1Gi`

`spec.storageConfig.ruleStorageSize`

`1Gi`

`spec.storageConfig.storeStorageSize`

`1Gi`

4. Increase the Grafana deployment replicas to four by updating the `MultiClusterObservability` object.
5. Create a custom alerting rule named `ClusterMemReq-60` that triggers an alert when the memory requests of a cluster are greater than 60%. To create the alerting rule, use the following information:

alert

`ClusterMemReq-60`

expr

`sum(namespace_memory:kube_pod_container_resource_requests:sum) by (clusterID, cluster) / sum(kube_node_status_allocatable{resource="memory"}) by (clusterID, cluster) > 0.6`

summary

Notify when Memory requests on a cluster are greater than the defined utilization limit

description

The cluster `{{ $labels.cluster }}` has an elevated percentage of Memory requests: `{{ $labels.clusterID }}`.

for

`5s`

cluster

`{{ $labels.cluster }}`

severity

`warning`

6. Create a custom platform metric named `pod:top_25_fs_usage_mb` by creating a recording rule. The metric applies only to the cluster named `managed-cluster`. The metric returns the top 25 pods with the highest file-system usage. The result is expressed in megabytes, and it is sorted in descending order. To create the recording rule, use the following information:

record

`pod:top_25_fs_usage_mb`

expr

`topk(25, sort_desc(sum(pod:container_fs_usage_bytes:sum{container!="",pod!=""})) BY (pod, namespace))/1000000)`

7. Go to the Grafana dashboard and verify that the `ClusterMemReq-60` alert is visible and that it is firing for the `local-cluster` cluster.
8. Go to the Grafana dashboard and verify that you can run a query that uses the `pod:top_25_fs_usage_mb` metric. The query returns values for only the `managed-cluster` cluster.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade observability-review
```

Finish

As the student user on the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish observability-review
```

Solution

Enable the RHACM observability stack. Increase the number of replicas of the observability components. Create a custom alerting rule that triggers an alert when the memory requests of a cluster are greater than 70%. Create a platform metric by creating a recording rule. Use Grafana to view the managed cluster dashboards, the new alerting rule, and the new metric.

Outcomes

- Enable the RHACM observability stack.
- Verify the deployment of the observability stack in the managed clusters.
- Increase the number of replicas of the Grafana pods.
- Create a custom alerting rule that triggers an alert when the memory requests of a cluster are greater than a defined value.
- Create a platform metric for a single managed cluster.
- Use Grafana to verify the creation and the status of the alerting rule, and the platform metrics.

Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab install do0014l ; lab start observability-review
```

Instructions

In this exercise, you enable the RHACM observability service, and customize the `MultiClusterObservability` object to increase the number of replicas of the observability components.

You create a custom alerting rule that triggers an alert when the memory requests are greater than 70%. You also create a platform metric for the cluster named `managed-cluster` by creating a recording rule. Finally, you use Grafana to view the managed cluster dashboards, the new alerting rule, and the new metrics.

The classroom environment provides two RHOCP clusters: an RHACM hub cluster and a managed cluster. You can access those clusters by using the following parameters:

RHACM hub cluster web console URL

`https://console-openshift-console.apps.ocp4.example.com`

RHACM hub cluster API URL

`https://api.ocp4.example.com:6443`

Managed cluster web console URL

`https://console-openshift-console.apps.ocp4-mng.example.com`

Managed cluster API URL

`https://api.ocp4-mng.example.com:6443`

Administrator username for both clusters

admin (select the **Red Hat Identity Management** identity provider)

Administrator password for both clusters

redhatocp

The lab command installs OpenShift Data Foundation and deploys the Multicloud Object Gateway component, which provides the S3 endpoint in the hub cluster.

1. Create the `open-cluster-management-observability` namespace in the hub cluster, and create the `multicloudhub-operator-pull-secret` secret in the `open-cluster-management-observability` namespace. To create the `multicloudhub-operator-pull-secret` secret, you use the `DOCKER_CONFIG_JSON` variable, which stores the contents of the secret named `pull-secret` from the `openshift-config` namespace.

- 1.1. To create the `open-cluster-management-observability` namespace in the hub cluster, use the terminal and log in to the hub cluster as the `admin` user with `redhatocp` as the password.

```
[student@workstation ~]$ oc login -u admin -p redhatocp \
https://api.ocp4.example.com:6443
Login successful.
...output omitted...
```

- 1.2. Create the `open-cluster-management-observability` namespace in the hub cluster.

```
[student@workstation ~]$ oc create namespace \
open-cluster-management-observability
namespace/open-cluster-management-observability created
```

- 1.3. Extract the secret named `pull-secret` from the `openshift-config` namespace, and store it in the `DOCKER_CONFIG_JSON` variable.

```
[student@workstation ~]$ DOCKER_CONFIG_JSON=$(oc extract \
secret/pull-secret -n openshift-config --to=)
# .dockerconfigjson
```

- 1.4. Create the `multicloudhub-operator-pull-secret` secret in the `open-cluster-management-observability` namespace by using the `DOCKER_CONFIG_JSON` variable.

```
[student@workstation ~]$ oc create secret generic \
multicloudhub-operator-pull-secret \
-n open-cluster-management-observability \
--from-literal=.dockerconfigjson="$DOCKER_CONFIG_JSON" \
--type=kubernetes.io/dockerconfigjson
secret/multicloudhub-operator-pull-secret created
```

2. Create an object bucket claim named `thanos-obc`, and use the `openshift-storage.noobaa.io` storage class. Create the `thanos-object-storage` secret within the `open-cluster-management-observability` namespace that uses the `thanos-obc` object bucket. The secret contains the object bucket information and credentials. You can use the `~/DO0014L/solutions/observability-review/obc.yaml` file to create the object bucket claim. You can use the `~/DO0014L/labs/observability-review/secret.yaml` file as a reference to create the `thanos-object-storage` secret.

- 2.1. Create the `thanos-obc` object bucket claim within the `open-cluster-management-observability` namespace

```
[student@workstation ~]$ oc apply -f \
~/D00014L/solutions/observability-review/obc.yaml
objectbucketclaim.objectbucket.io/thanos-obc created
```

- 2.2.** Verify that the thanos-obc object bucket claim exists in the open-cluster-management-observability namespace, and wait until it displays Bound as the status in the PHASE column. The object bucket claim might take a few minutes to appear as Bound.

```
[student@workstation ~]$ watch oc get objectbucketclaim \
-n open-cluster-management-observability
NAME                STORAGE-CLASS          PHASE    ...
thanos-obc          openshift-storage.noobaa.io  Bound
```

Press Ctrl+C to exit the watch command.

- 2.3.** Retrieve the bucket name from the thanos-obc configuration map within the open-cluster-management-observability namespace.

```
[student@workstation ~]$ oc extract --to=- cm/thanos-obc \
-n open-cluster-management-observability
...output omitted...
# BUCKET_NAME
observability-bucket-030b2b89-6c7e-4883-a26e-ea8939dfc875
# BUCKET_PORT
443
```

The BUCKET_NAME in the preceding output is different on your system.

- 2.4.** Retrieve the bucket credentials from the thanos-obc secret within the open-cluster-management-observability namespace.

```
[student@workstation ~]$ oc extract --to=- secret/thanos-obc \
-n open-cluster-management-observability
# AWS_ACCESS_KEY_ID
n0v00rBFB1F4Bc1NZaIv
# AWS_SECRET_ACCESS_KEY
sc8Z3KpNF8Ik035inA0ji9Nkz3g2hBMnVe7EFWIT
```

The credentials in the preceding output are different on your system.

- 2.5.** Identify the public URL for the S3 endpoint from the s3 route in the openshift-storage namespace.

```
[student@workstation ~]$ oc get route s3 -n openshift-storage
NAME    HOST/PORT          ...
s3      s3-openshift-storage.apps.ocp4.example.com  ...
```

- 2.6.** Edit the ~/D00014L/labs/observability-review/secret.yaml file and update the placeholder text. Use the S3 credentials, the S3 endpoint URL, and the bucket name from the previous steps.


```
---
apiVersion: v1
kind: Secret
metadata:
  name: thanos-object-storage
  namespace: open-cluster-management-observability
type: Opaque
stringData:
  thanos.yaml: |
    type: s3
    config:
      bucket: observability-bucket-030b2b89-6c7e-4883-a26e-ea8939dfc875
      endpoint: s3-openshift-storage.apps.ocp4.example.com
      insecure: true
      access_key: n0v00rBFB1F4Bc1NZaIv
      secret_key: sc8Z3KpnF8Ik035inA0ji9Nkz3g2hBMnVe7EFWIT
```

2.7. Create the thanos-object-storage secret.

```
[student@workstation ~]$ oc apply -f \
~/D00014L/labs/observability-review/secret.yaml
secret/thanos-object-storage created
```

3. Create the MultiClusterObservability custom resource instance to enable the observability service. To create the MultiClusterObservability object, use the following information:

metadata.name

observability

spec.observabilityAddonSpec.enableMetrics

true

spec.storageConfig.metricObjectStorage.name

thanos-object-storage

spec.storageConfig.storageClass

nfs-storage

spec.storageConfig.alertmanagerStorageSize

1Gi

spec.storageConfig.compactStorageSize

1Gi

spec.storageConfig.receiveStorageSize

1Gi

spec.storageConfig.ruleStorageSize

1Gi

spec.storageConfig.storeStorageSize

1Gi

- 3.1. To create the MultiClusterObservability object, create a YAML file with the following content. You can use the ~/D00014L/solutions/observability-review/mcobs.yaml file to create the MultiClusterObservability object.

```
---
apiVersion: observability.open-cluster-management.io/v1beta2
kind: MultiClusterObservability
metadata:
  name: observability
spec:
  observabilityAddonSpec:
    enableMetrics: true
  storageConfig:
    metricObjectStorage:
      key: thanos.yaml
      name: thanos-object-storage
    storageClass: nfs-storage
    alertmanagerStorageSize: 1Gi
    compactStorageSize: 1Gi
    receiveStorageSize: 1Gi
    ruleStorageSize: 1Gi
    storeStorageSize: 1Gi
```

3.2. Create the MultiClusterObservability object.

```
[student@workstation ~]$ oc apply -f \
~/D00014L/solutions/observability-review/mcobs.yaml
multiclusterobservability.observability.open-cluster-management.io/observability created
```

3.3. Verify that all the pods in the open-cluster-management-observability namespace display Running in the STATUS column. It can take up to three minutes for the pods to reach the Running status.

```
[student@workstation ~]$ oc get po -n \
open-cluster-management-observability
```

NAME	READY	STATUS	...
endpoint-observability-operator-86d846cc45-bsp4t	1/1	Running	
metrics-collector-deployment-b7798dd6b-p5rn6	1/1	Running	
observability-alertmanager-0	4/4	Running	
observability-alertmanager-1	4/4	Running	
observability-alertmanager-2	4/4	Running	
observability-grafana-7c8ddc4d5-bnbwn	3/3	Running	
observability-grafana-7c8ddc4d5-d9gjb	3/3	Running	
observability-observatorium-api-6f9748959-xbj2j	1/1	Running	
observability-observatorium-api-6f9748959-z2l7v	1/1	Running	

...output omitted...

4. Increase the Grafana deployment replicas to four by updating the MultiClusterObservability object.

4.1. Edit the MultiClusterObservability object, and update the spec section to add the advanced.grafana.replicas configuration block. Set the replicas to 4.

```
[student@workstation ~]$ oc edit multiclusterobservability \
observability
...output omitted...
spec:
  advanced:
    grafana:
      replicas: 4
    enableDownsampling: true
...output omitted...
```

5. Create a custom alerting rule named ClusterMemReq-60 that triggers an alert when the memory requests of a cluster are greater than 60%. To create the alerting rule, use the following information:

alert

ClusterMemReq-60

expr

sum(namespace_memory:kube_pod_container_resource_requests:sum) by (clusterID, cluster) / sum(kube_node_status_allocatable{resource="memory"}) by (clusterID, cluster) > 0.6

summary

Notify when Memory requests on a cluster are greater than the defined utilization limit

description

The cluster {{ \$labels.cluster }} has an elevated percentage of Memory requests: {{ \$labels.clusterID }}.

for

5s

cluster

{{ \$labels.cluster }}

severity

warning

- 5.1. To create the alerting rule, create a YAML file for the thanos-ruler-custom-rules configuration map with the following content. You can use the ~/DO0014L/solutions/observability-review/custom-rules.yaml file to create the configuration map.

```
---
kind: ConfigMap
apiVersion: v1
metadata:
  name: thanos-ruler-custom-rules
  namespace: open-cluster-management-observability
data:
  custom_rules.yaml: |
    groups:
      - name: cluster-health
        rules:
          - alert: ClusterMemReq-60
            annotations:
              summary: Notify when Memory requests on a cluster are greater than the defined
              utilization limit
              description: "The cluster {{ $labels.cluster }} has an elevated percentage of Memory
              requests: {{ $labels.clusterID }}."
            expr: |
              sum(namespace_memory:kube_pod_container_resource_requests:sum) by (clusterID, cluster)
              / sum(kube_node_status_allocatable{resource="memory"}) by (clusterID, cluster) > 0.6
            for: 5s
            labels:
              cluster: "{{ $labels.cluster }}"
              severity: warning
```

- 5.2. Create the thanos-ruler-custom-rules configuration map in the open-cluster-management-observability namespace.

```
[student@workstation ~]$ oc apply -f \
~/D00014L/solutions/observability-review/custom-rules.yaml
configmap/thanos-ruler-custom-rules created
```

6. Create a custom platform metric named `pod:top_25_fs_usage_mb` by creating a recording rule. The metric applies only to the cluster named `managed-cluster`. The metric returns the top 25 pods with the highest file-system usage. The result is expressed in megabytes, and it is sorted in descending order. To create the recording rule, use the following information:

record

`pod:top_25_fs_usage_mb`

expr

`topk(25, sort_desc(sum(pod:container_fs_usage_bytes:sum{container!="",pod!="\"})) BY (pod, namespace))/1000000)`

- 6.1. Log in to the `managed-cluster` cluster as the `admin` user with `redhatocp` as the password. In the following command, ensure that you use the API URL of the managed cluster, which is `https://api.ocp4-mng.example.com:6443`

```
[student@workstation ~]$ oc login -u admin -p redhatocp \
https://api.ocp4-mng.example.com:6443
Login successful.
...output omitted...
```

- 6.2. To create the recording rule, create a YAML file for the `observability-metrics-custom-allowlist` configuration map with the following content. You can use the `~/D00014L/solutions/observability-review/platform-recording.yaml` file to create the configuration map.

```
---
kind: ConfigMap
apiVersion: v1
metadata:
  name: observability-metrics-custom-allowlist
  namespace: open-cluster-management-addon-observability
data:
  metrics_list.yaml: |
    recording_rules:
      - record: pod:top_25_fs_usage_mb
        expr: topk(25, sort_desc(sum(pod:container_fs_usage_bytes:sum{container!="",pod!="\"})) BY (pod, namespace))/1000000)
```

- 6.3. Create the `observability-metrics-custom-allowlist` configuration map in the `open-cluster-management-addon-observability` namespace.

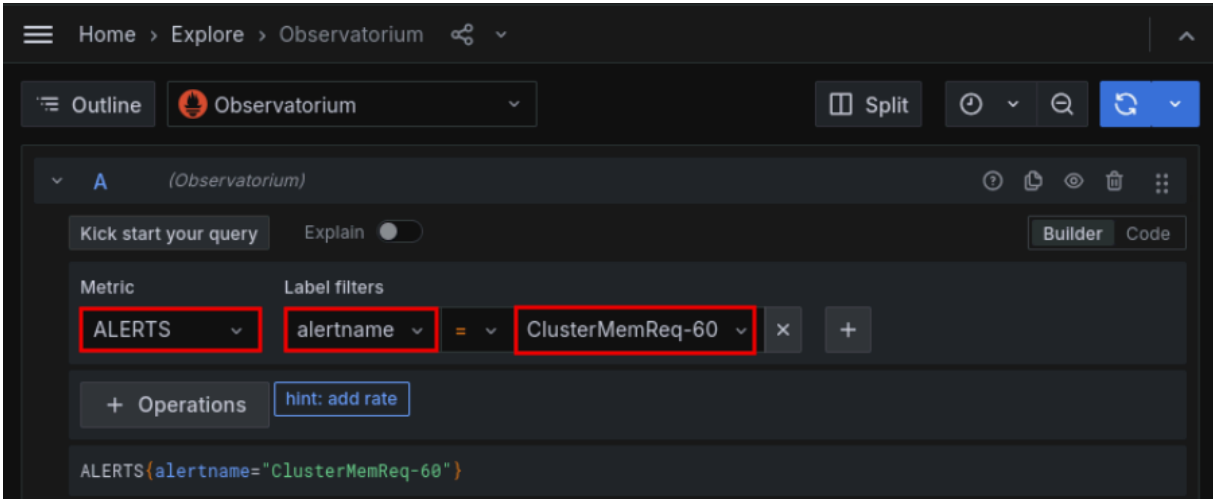
```
[student@workstation ~]$ oc apply -f \
~/D00014L/solutions/observability-review/platform-recording.yaml
configmap/observability-metrics-custom-allowlist created
```

7. Go to the Grafana dashboard and verify that the `ClusterMemReq-60` alert is visible and that it is firing for the `local-cluster` cluster.

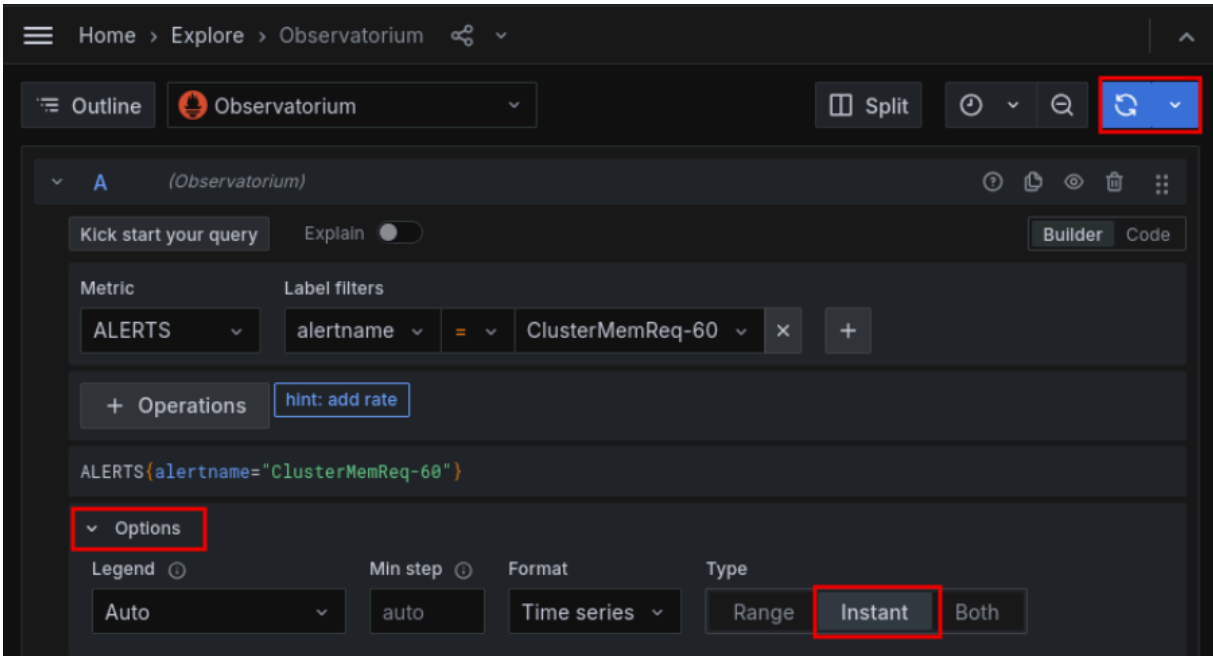
- 7.1. On the workstation machine, open a web browser and go to `https://console-openshift-console.apps.ocp4.example.com`

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

- 7.2. Select the **Red Hat Identity Management** identity provider and log in as the admin user with redhatocp as the password.
- 7.3. Switch to the RHACM web console, and go to **Infrastructure > Clusters**.
- 7.4. Click the **Grafana** link in the upper-right corner.
- 7.5. In a new browser tab that opens, log in as the admin user with redhatocp as the password.
- 7.6. From the main menu of the Grafana dashboard, click **Explore**.
- 7.7. On the explore page, select ALERTS from the **Metric** drop-down list. Select alertname from the **Label filters** drop-down list, and select **ClusterMemReq-60** as the value. If the drop-down list does not show alertname, then type it.



- 7.8. Expand the **Options** section, and select the **Instant** query type. Click the **Run query** button in the upper-right corner.

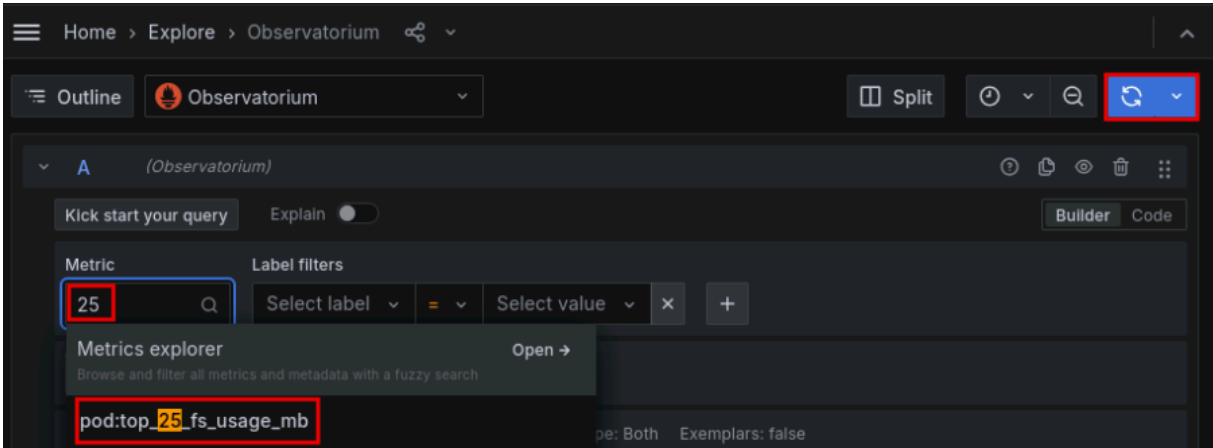


- 7.9. Verify that the **Table** section contains a row with the ClusterMemReq-60 custom alerting rule that is firing in the alertstate column for the local-cluster cluster.

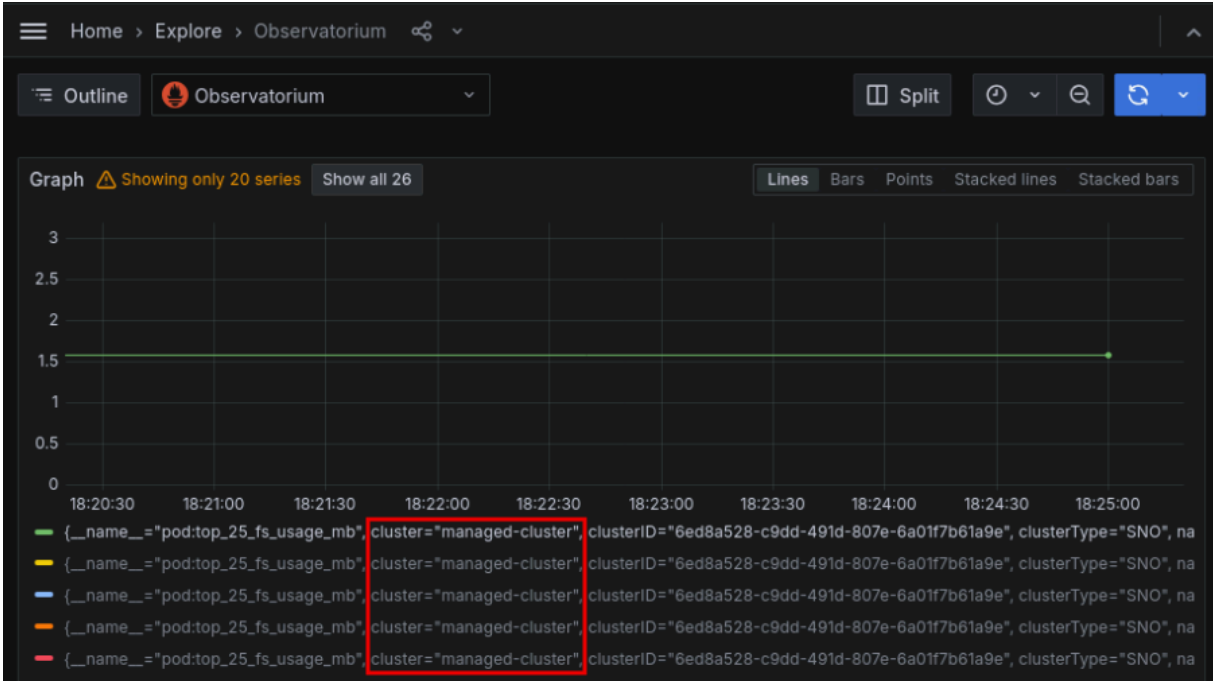
For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

Table					
Time	__name__	alertname	alertstate	cluster	cluste
2025-07-03 05:25:30	ALERTS	ClusterMemReq-60	firing	local-cluster	6c8c6

8. Go to the Grafana dashboard and verify that you can run a query that uses the `pod:top_25_fs_usage_mb` metric. The query returns values for only the managed-cluster cluster.
- 8.1. From the main menu of the Grafana dashboard, click **Explore**.
- 8.2. On the explore page, type 25 in the **Metric** drop-down list, and the metric name appears. Select the `pod:top_25_fs_usage_mb` metric, and click the **Run query** button in the upper-right corner.



- 8.3. Verify that the **Graph** section shows the data from the metric only for the managed-cluster cluster. The data might take a few minutes to appear in the graph.



Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade observability-review
```

Finish

As the student user on the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish observability-review
```

4.6. Summary

In this lesson, you learned about the Red Hat Advanced Cluster Management for Kubernetes observability stack, and how to enable and customize the RHACM observability components. The RHACM observability service provides a unified display of metrics to assess the health and usage of clusters and workloads across your fleet. This service is crucial for anticipating issues, improving performance, and conducting postmortem analysis.

The RHACM observability service uses key open source tools from the Cloud Native Computing Foundation (CNCF). Prometheus acts as a monitoring and alerting tool, by collecting and storing metrics as time-series data. Thanos gathers data from multiple Prometheus instances across clusters, and provides global querying capabilities. Alertmanager manages and receives alerts from Prometheus. Grafana offers data visualization with default and custom dashboards.

To enable the RHACM observability service, you create the `MultiClusterObservability` custom resource. The RHACM observability service requires specific prerequisites, such as a dedicated namespace, a pull secret, and the object store secret with the S3 bucket information and credentials.

To customize the RHACM observability stack, you adapt the service to the specific needs of your cluster fleet by updating the `MultiClusterObservability` custom resource.

With RHACM, you can create custom Prometheus rules, which include recording rules to precompute complex expressions for faster querying, and alerting rules to define conditions that trigger notifications.

Grafana comes with predefined dashboards, and enables you to create your own custom dashboards.

RHACM can integrate external messaging tools for forwarding alerts.

Next Steps

You might explore other learning opportunities on your own, explore further your RHACM installation, or redo some exercises.

A good next step in your learning journey would be to explore the next lessons in the series:

- Lesson 5: *Managing the Multicluster Application Lifecycle by Using GitOps Practices and Red Hat Advanced Cluster Management for Kubernetes*
- Lesson 6: *Managing Virtual Machines for Multiple Clusters with Red Hat Advanced Cluster Management for Kubernetes*

Chapter 5.

Managing the Multicloud Application Lifecycle by Using GitOps Practices and Red Hat Advanced Cluster Management for Kubernetes

Goal

Deploy and manage applications in a multicloud environment by using Red Hat Advanced Cluster Management for Kubernetes (RHACM) and GitOps.

Sections

- Integrating GitOps Practices and Application Lifecycle Management of RHACM (and Quiz)
- Configuring GitOps for Application Lifecycle Management in RHACM (and Guided Exercise)
- Deploying and Managing Multicloud Application Resources with RHACM (and Guided Exercise)
- Troubleshooting Common Issues for Application Lifecycle Management in RHACM (and Guided Exercise)

Lab

- Manage Applications Across Multiple Clusters with RHACM

5.1. Integrating GitOps Practices and Application Lifecycle Management of RHACM

Objectives

- Integrate GitOps practices into the application lifecycle management of Red Hat Advanced Cluster Management (RHACM).

Introduction to GitOps

GitOps is an operational framework that applies DevOps best practices to application delivery and infrastructure management, by using Git as the single source of truth. It is a method for deploying and continuously managing applications on Kubernetes clusters. You declaratively describe and store the entire intended state of your system in a Git repository.

Red Hat OpenShift GitOps and Argo CD integrate with RHACM. The OpenShift GitOps Operator enables users to use the latest advancements in Argo CD development and to receive support through their OpenShift GitOps operator subscription.

The primary application deployment patterns are the Push model and the Pull model. These models are explained later in this lecture.

Benefits of the GitOps Approach

Adopting a GitOps approach offers several compelling benefits for application lifecycle management:

Declarative Deployments

GitOps mandates that you declaratively define the intended state of applications and infrastructure within Git repositories. The Git repository always reflects the live state of your environment, which enables predictable and repeatable deployments.

Version Control and Auditability

Using Git automatically version-controls every change to your application and infrastructure configurations. This approach provides a complete and immutable audit trail, for tracking who made which changes, when, and why. It also simplifies rollbacks to previous stable states if issues arise.

Consistency Across Environments

GitOps helps automate and ensure application consistency across various environments, such as development, staging, and production, by driving deployments from a unified source of truth. GitOps reduces configuration drift and potential errors.

Enhanced Scalability

The GitOps Pull model uses the hub-spoke pattern of RHACM to offer improved scalability for managing a large fleet of Kubernetes clusters. It enhances the distribution of the workload of the application deployment.

Improved Security

This approach inherently enhances security because you do not have to store cluster credentials in a centralized environment for all operations. The attack surface reduces and minimizes the impact of a potential breach in a central system.

Reduced Impact of Centralized Failure

By enabling a decentralized approach, particularly with the Pull model, GitOps can reduce the impact of a single point of centralized failure for application delivery. If the hub experiences issues, then managed clusters might still be able to reconcile their state from Git.

Increased Flexibility

GitOps provides greater flexibility, by enabling managed clusters to pull updates on their own schedule. This method can help in scenarios with intermittent connectivity or specific maintenance windows.

Application Lifecycle Management by Using GitOps in RHACM

RHACM offers enhanced application management capabilities based on a Kubernetes resource-based application model. Red Hat designed this model to unify and simplify creating and managing your applications across all your managed clusters.

Application Model in RHACM

The current application model primarily uses the `ApplicationSet` custom resource, which represents Argo CD applications that a controller generates. The following key components work with `ApplicationSet`:

Placement

This resource defines the target clusters that receive application deployments. It selects clusters from a `ClusterSet` resource where you deliver applications.

ClusterSet

Managed `ClusterSet` resources group cluster resources for role-based access control (RBAC), which enables you to control access to resources within the group.

GitOpsCluster

This resource represents an Argo CD server and its associated placement, to help integrating Argo CD with RHACM's cluster management capabilities.

Application

An `ApplicationSet` resource deploys these resources.

 **Important**
Earlier versions of RHACM used an Application Subscription Model, which included components such as *Channel*, *Subscription*, and *Placement Rule*. The RHACM developers deprecated these earlier components in favor of the later and integrated Application Set Model.

Push and Pull Models

RHACM's integration with OpenShift GitOps and Argo CD supports two primary application deployment patterns: the Push model and the Pull model. Both models use the same `ApplicationSet` Custom Resource Definition (CRD) to define and deploy applications to managed clusters.

In the *Push* model, the Argo CD server, which is running on the hub cluster, takes direct action to deploy application resources to the managed clusters. The Argo CD application on the hub cluster actively communicates with the Git repository and then directly applies the manifests to the managed clusters. This model requires the hub cluster to own the necessary credentials to access and interact with the managed clusters.

Conversely, in the *Pull* model, the hub cluster propagates the application resources to the managed clusters by using a *ManifestWork* object. Crucially, with the Pull model, each managed cluster individually communicates with the Git repository to deploy the resource manifests locally. To do so, an Argo CD server must be running on each target managed cluster, which then deploys the replicated application resources locally.

Scenarios for Push and Pull Models

Choosing appropriately between the Push and Pull models is critical for efficient and secure application deployment:

The Push model is the default method for deploying applications unless you use explicit annotations to enable the Pull model. The following scenarios apply to the Push model:

Network Connectivity

The Push model works best in environments where the centralized hub cluster has reliable and direct network connectivity to all remote clusters. If the hub cannot start connections to the managed clusters, then the Push model might not be suitable.

Limited Managed Cluster Capabilities

In scenarios where a managed cluster cannot reach the Git server directly due to network constraints or security policies, the Push model becomes the only viable solution, because the hub can proxy the Git access.

Resource Considerations

When managing many managed clusters, deploying applications by using the Push model can potentially overload the OpenShift GitOps controller's memory and CPU usage on the hub cluster. For such large-scale deployments, carefully consider resource quotas or requests for the controller.

The following scenarios apply to the Pull model:

Network Segmentation or Unreachability

The Pull model helps in network architectures where the centralized hub cluster cannot start connections to remote clusters, but the remote clusters can communicate back to the centralized cluster. Such architectures have firewalls that allow outbound but not inbound connections.

Scalability for Large Deployments

The Pull model provides significant scalability relief by offloading the direct deployment workload from the hub cluster's controller. Although status reporting still occurs, the primary reconciliation and deployment work happens locally on each managed cluster.

Decentralized Control

This model offers decentralized control, where each managed cluster maintains its own copy of the configuration and independently pulls updates from Git. This approach can simplify management in highly distributed environments.

Enhanced Security by Using Decentralized Credentials

This model can enhance security because you do not need to centrally store sensitive cluster credentials on the hub cluster for direct push operations to every managed cluster. Each managed cluster needs credentials only to access its Git source.

Resilience

The Pull model can reduce the impact of a single point of centralized failure for application delivery, because the system distributes the execution of the application deployment across the managed clusters. However, the hub

cluster still plays an orchestration role, and Red Hat recommends that administrators make it redundant for overall system resilience.



Note

Currently, in the Pull model, the system excludes the local cluster as a target managed cluster for deployments.



References

For more information about using GitOps with RHACM, refer to the *GitOps Overview* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 GitOps* documentation at https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/gitops/index#gitops-overview

[Introducing the Argo CD Application Pull Controller for Red Hat Advanced Cluster Management](#)

[Argo CD Application Pull Controller](#)

5.2. Quiz

Integrating GitOps Practices and Application Lifecycle Management of RHACM

Choose the correct answers to the following questions:

1. Which operational framework uses Git as the single source of truth for application delivery and infrastructure management?
☐ a. DevOps
☐ b. Kubernetes
☐ c. GitOps
☐ d. OpenShift
2. In the GitOps application model in RHACM, which custom resource represents Argo CD applications that are generated by a controller?
☐ a. Placement
☐ b. ClusterSet
☐ c. GitOpsCluster
☐ d. ApplicationSet
3. In which application deployment model does the Argo CD server on the hub cluster directly apply manifests to managed clusters, which requires the hub to own the necessary credentials?
☐ a. Pull model
☐ b. Decentralized model
☐ c. Push model
☐ d. Hybrid model
4. Which scenario is best suited to the GitOps Pull model?
☐ a. Environments where the centralized hub cluster has reliable and direct network connectivity to all remote clusters.
☐ b. Scenarios where a managed cluster cannot reach the Git server directly due to network constraints.
☐ c. When managing many managed clusters, to centralize the deployment workload on the hub cluster.
☐ d. Network architectures where the centralized hub cluster cannot initiate connections to remote clusters, but remote clusters can communicate back to the hub.
5. How does the Pull model contribute to enhanced security in GitOps deployments?
☐ a. By requiring all sensitive cluster credentials to be centrally stored on the hub cluster.
☐ b. By consolidating all Git access through a single proxy on the hub cluster.
☐ c. By allowing the hub cluster to perform direct push operations without credentials.
☐ d. By not needing to centrally store sensitive cluster credentials on the hub cluster for direct push operations to every managed cluster.

6. In the context of GitOps deployment models, what is the default method for deploying applications?

- ☐ **a.** The Hybrid model
- ☐ **b.** The Decentralized model
- ☐ **c.** The Push model
- ☐ **d.** The Proxy model

7. Which statement is true regarding the local cluster as a target managed cluster in the Pull model?

- ☐ **a.** The local cluster is always prioritized as a target.
- ☐ **b.** The local cluster acts as the central point for all Pull model deployments.
- ☐ **c.** The system excludes the local cluster as a target managed cluster for deployments.
- ☐ **d.** The local cluster must have unique credentials to be a Pull model target.

Solution

- Which operational framework uses Git as the single source of truth for application delivery and infrastructure management?
 - ☐ a. DevOps
 - ☐ b. Kubernetes
 - ☒ c. **GitOps**
 - ☐ d. OpenShift
- In the GitOps application model in RHACM, which custom resource represents Argo CD applications that are generated by a controller?
 - ☐ a. Placement
 - ☐ b. ClusterSet
 - ☐ c. GitOpsCluster
 - ☒ d. **ApplicationSet**
- In which application deployment model does the Argo CD server on the hub cluster directly apply manifests to managed clusters, which requires the hub to own the necessary credentials?
 - ☐ a. Pull model
 - ☐ b. Decentralized model
 - ☒ c. **Push model**
 - ☐ d. Hybrid model
- Which scenario is best suited to the GitOps Pull model?
 - ☐ a. Environments where the centralized hub cluster has reliable and direct network connectivity to all remote clusters.
 - ☐ b. Scenarios where a managed cluster cannot reach the Git server directly due to network constraints.
 - ☐ c. When managing many managed clusters, to centralize the deployment workload on the hub cluster.
 - ☒ d. **Network architectures where the centralized hub cluster cannot initiate connections to remote clusters, but remote clusters can communicate back to the hub.**
- How does the Pull model contribute to enhanced security in GitOps deployments?
 - ☐ a. By requiring all sensitive cluster credentials to be centrally stored on the hub cluster.
 - ☐ b. By consolidating all Git access through a single proxy on the hub cluster.
 - ☐ c. By allowing the hub cluster to perform direct push operations without credentials.
 - ☒ d. **By not needing to centrally store sensitive cluster credentials on the hub cluster for direct push operations to every managed cluster.**
- In the context of GitOps deployment models, what is the default method for deploying applications?
 - ☐ a. The Hybrid model
 - ☐ b. The Decentralized model
 - ☒ c. **The Push model**
 - ☐ d. The Proxy model

7. Which statement is true regarding the local cluster as a target managed cluster in the Pull model?

- ☐ a. The local cluster is always prioritized as a target.
- ☐ b. The local cluster acts as the central point for all Pull model deployments.
- ☒ c. **The system excludes the local cluster as a target managed cluster for deployments.**
- ☐ d. The local cluster must have unique credentials to be a Pull model target.

5.3. Configuring GitOps for Application Lifecycle Management in RHACM

Objectives

- Describe and deploy the Red Hat OpenShift GitOps operator and configure GitOps for application lifecycle management in RHACM.

Introduction to Red Hat OpenShift GitOps

RHACM is ready for GitOps, and the web console provides a dashboard to create and manage applications. The GitOps feature in RHACM relies on the Argo CD tool, which synchronizes Kubernetes resources from definitions that are stored in Git repositories.

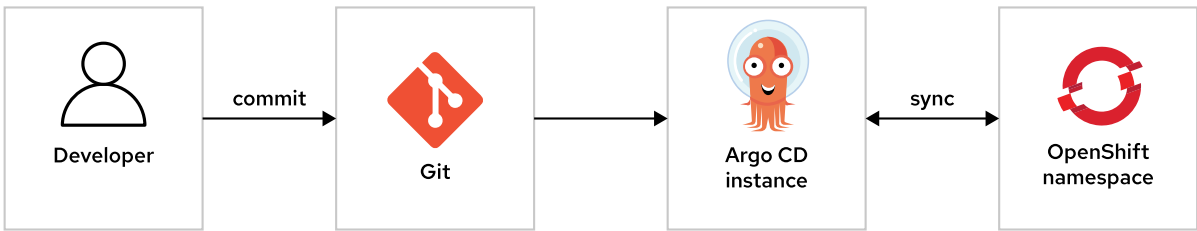


Figure 31. Argo CD architecture

However, the configuration is not complete, because the RHACM operator does not include Argo CD. Instead, you must install Argo CD by deploying the OpenShift GitOps operator.

The OpenShift GitOps operator manages Argo CD instances in OpenShift clusters. During deployment, the operator creates a default Argo CD instance in the `openshift-gitops` namespace.

OpenShift GitOps is an operator in OperatorHub, from the Red Hat operator catalog.

Argo CD Applications

In Argo CD, an *application* is a resource that references a Git repository with Kubernetes resource definitions.

Applications define a synchronization policy. Argo CD can detect and synchronize changes automatically, or users can trigger synchronizations on demand.

Argo CD Web Console

Users can create and manage applications from the Argo CD web console, where the status of application resources is displayed graphically. In this representation, you can view whether the resources are synchronized with the application definition, and inspect their status.

When you integrate OpenShift GitOps with RHACM, the RHACM web console provides a similar representation of the applications. You also create and manage applications directly within the RHACM web console, without having to use the Argo CD web console.

Although the Argo CD web console stays accessible, you should restrict its usage for troubleshooting purposes.

Deploying the OpenShift GitOps Operator

Application lifecycle management in RHACM relies on Argo CD and requires the installation of the OpenShift GitOps operator.

RHACM provides two models of application lifecycle management:

Push model

With the Push model, the Argo CD instance that is running in the hub cluster manages the deployment of the applications into the managed clusters. In this model, you install the OpenShift GitOps operator in the hub cluster.

Pull model

With the Pull model, the Argo CD instance that is running in the hub cluster delegates the deployment of the applications to the Argo CD instance that is running in each managed cluster. In this model, you install the OpenShift GitOps operator in the managed clusters as well.

You can install the OpenShift GitOps operator in a similar way to any other operator, for example by using the installation interface of the OpenShift web console.

However, Red Hat recommends that you use an RHACM governance policy to automate the process and to maintain compliance. In the following exercise, you install and configure the OpenShift GitOps operator by using this method.

Configuring Argo CD Instances

The OpenShift GitOps operator creates a default Argo CD instance in the `openshift-gitops` namespace. For RHACM to use this instance, you must configure the instance with the following adjustments:

- Grant the OpenShift administrators access to the Argo CD web console.
- Grant the Argo CD instance the required permissions to deploy applications.
- Install your custom certificate authority (CA) certificates into the Argo CD instance so that Argo CD can pull Git repositories from private Git systems.

Granting Access to the Argo CD Web Console

Users can interact with Argo CD through the Argo CD API, web console, or command-line tool. Each Argo CD instance has a role-based access control (RBAC) system to manage permissions in the Argo CD interfaces.

OpenShift manages the Argo CD instances by using ArgoCD resources. The following excerpt from the default Argo CD instance shows the default authentication configuration in the `rbac` section:

```
---
apiVersion: argoproj.io/v1beta1
kind: ArgoCD
metadata:
  name: openshift-gitops
  namespace: openshift-gitops
spec:
  ...output omitted...
  rbac:
    defaultPolicy: "" ①
    policy: |
      g, system:cluster-admins, role:admin ②
      g, cluster-admins, role:admin ③
      scopes: '[groups]'
  ...output omitted...
```

- ① The defaultPolicy parameter specifies the Argo CD role that authenticated users get by default. Although all OpenShift users can log in to the Argo CD web console, they have no permissions, because the defaultPolicy parameter is empty.
- ② The policy rule grants the Argo CD admin role to the kube-admin user. Because you usually do not use this kube-admin credential, you can ignore this rule.
- ③ The policy rule grants the Argo CD admin role to the members of the OpenShift cluster-admins group. With the admin role, the users have full control of the instance.

With this rule, you can grant your OpenShift administrators access to the Argo CD web console by adding the administrator accounts to the cluster-admins group.

You can grant other groups the admin role by adding a policy rule in the policy section. The following example grants the Argo CD admin role to the members of the OpenShift ocpadmins group:

```
---
apiVersion: argoproj.io/v1beta1
kind: ArgoCD
metadata:
  name: openshift-gitops
  namespace: openshift-gitops
spec:
  ...output omitted...
  rbac:
    defaultPolicy: ""
    policy: |
      g, system:cluster-admins, role:admin
      g, cluster-admins, role:admin
      g, ocpadmins, role:admin
      scopes: '[groups]'
  ...output omitted...
```

Granting Cluster Administration Permissions to Argo CD

The default Argo CD instance has a set of permissions to support cluster administration, and can manage some cluster-scoped Kubernetes resources. OpenShift administrators commonly use the instance to manage the cluster itself, instead of managing the organization's applications.

However, when it is used for RHACM, the instance does not have sufficient permissions to manage all the required Kubernetes resources to deploy applications. For RHACM, you must grant the cluster-admin permissions to Argo CD.

To grant cluster-admin permissions, create a ClusterRoleBinding resource for the openshift-gitops-argocd-application-controller service account. Argo CD uses this service account to interact with Kubernetes

for managing resources.

The following resource grants the `cluster-admin` permissions to Argo CD:

```
---
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: argo-admin
subjects:
  - kind: ServiceAccount
    name: openshift-gitops-argocd-application-controller
    namespace: openshift-gitops
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
```

Providing Argo CD with Custom CA Certificates

Argo CD applications reference Git repositories. During an application deployment, the Argo CD instance pulls the repository contents from the Git system.

When you use a secured protocol, such as HTTPS, to access the Git repository, Argo CD validates the remote TLS certificate by using the CA that issued this certificate. When the Git system is public, such as github.com, the certificate is signed by a well-known CA that is part of the CA bundle that comes with Argo CD.

However, if you use a private Git system, such as self-managed GitLab, and you protect it with a certificate that your company's private CA issued, then you must add this private CA to the bundle. Otherwise, Argo CD cannot validate the certificate and raises an error.

To provide the private CA certificate to Argo CD, perform the following steps:

- Add the private CA certificate to a configuration map in the `openshift-gitops` namespace.
- Mount the configuration map as a volume inside the Argo CD instance.

If the private CA certificate is already available in OpenShift, then create a configuration map with the `config.openshift.io/inject-trusted-cabundle` label. When this label is set, OpenShift automatically populates the configuration map with the cluster CA bundle:

```
---
kind: ConfigMap
apiVersion: v1
metadata:
  name: cluster-root-ca-bundle
  namespace: openshift-gitops
  labels:
    config.openshift.io/inject-trusted-cabundle: 'true'
```

To mount the configuration map into Argo CD, update the ArgoCD resource. In the `repo` section, add the `volumeMounts` and `volumes` subsections:

```
---
apiVersion: argoproj.io/v1beta1
kind: ArgoCD
metadata:
  name: openshift-gitops
  namespace: openshift-gitops
  ...output omitted...
spec:
  ...output omitted...
  repo:
    volumeMounts:
      - mountPath:
          /etc/pki/ca-trust/extracted/pem/tls-ca-bundle.pem
        subPath: ca-bundle.crt
        name: cluster-root-ca-bundle
    volumes:
      - configMap:
          name: cluster-root-ca-bundle
        name: cluster-root-ca-bundle
  ...output omitted...
```

Importing Managed Clusters into Argo CD

Argo CD accesses the clusters' Kubernetes API to manage application resources. Consequently, you must declare your managed clusters to Argo CD.

After you installed the OpenShift GitOps operator, you register your clusters with Argo CD by creating a `GitOpsCluster` resource:

```
---
apiVersion: apps.open-cluster-management.io/v1beta1
kind: GitOpsCluster
metadata:
  name: gitops-clusters
  namespace: openshift-gitops
spec:
  argoServer: ①
  argoNamespace: openshift-gitops
  placementRef: ②
  kind: Placement
  apiVersion: cluster.open-cluster-management.io/v1beta1
  name: clusters-for-gitops
  namespace: openshift-gitops
```

- ① The `argoServer` section specifies the namespace of the Argo CD instance to configure in the hub cluster.

Note

You do not need to register the clusters with the Argo CD instances that are running in the managed clusters.

- ② The `placementRef` section provides the `placement` resource that selects the clusters to register with Argo CD. The following placement example selects all the OpenShift clusters from the cluster sets that are mapped to the `openshift-gitops` project:

```
---
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: clusters-for-gitops
  namespace: openshift-gitops
spec:
  tolerations:
    - key: cluster.open-cluster-management.io/unreachable
      operator: Exists
    - key: cluster.open-cluster-management.io/unavailable
      operator: Exists
  predicates:
    - requiredClusterSelector:
        labelSelector:
          matchLabels:
            vendor: OpenShift
```

With this configuration, you enable RHACM to deploy Argo CD applications to any of the imported managed clusters.

References

Knowledgebase: "How to Add Additional Trusted Root CAs to Argo CD"

For more information about installing the OpenShift GitOps operator with a governance policy, refer to the *Generating a Policy to Install GitOps Operator* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 GitOps* documentation at https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/gitops/index#gitops-policy-generator

For more information about providing a custom CA certificate to OpenShift, refer to the *Configuring a Custom PKI* chapter in the *Red Hat OpenShift Container Platform 4.18 Networking* documentation at https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/configuring_network_settings/index#configuring-a-custom-pki

For more information about importing managed cluster into Argo CD, refer to the *Registering Managed Clusters to Red Hat OpenShift GitOps Operator* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 GitOps* documentation at https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/gitops/index#gitops-register

For more information about adding Argo CD instances, refer to the *Setting up an Argo CD Instance* chapter in the *Red Hat OpenShift GitOps 1.16 Argo CD Instances* documentation at https://docs.redhat.com/en/documentation/red_hat_openshift_gitops/1.16/html-single/argo_cd_instance/index#setting-up-argocd-instance

For more information about Argo CD RBAC systems, refer to the *Configuring Argo CD RBAC* chapter in the *Red Hat OpenShift GitOps 1.16 Argo CD Access Control and User Management* documentation at https://docs.redhat.com/en/documentation/red_hat_openshift_gitops/1.16/html-single/access_control_and_user_management/index#configuring-argo-cd-rbac

[Getting Started with OpenShift GitOps](#)

5.4. Guided Exercise

Configure GitOps for Application Lifecycle Management in RHACM

Deploy the Red Hat OpenShift GitOps operator and configure GitOps for application lifecycle management in RHACM.

Outcomes

- Deploy and configure the OpenShift GitOps operator by using a governance policy.
- Import clusters into Argo CD.

Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab install do0015l ; lab start gitops-configure
```

Instructions

1. Log in to the RHOC hub cluster, and create the `gitops-configure` project. You use this project to deploy the OpenShift GitOps operator by using a governance policy in another step.

- 1.1. In a terminal window, log in to the RHOC cluster as the `admin` user with `redhatocp` as the password.

```
[student@workstation ~]$ oc login -u admin -p redhatocp \
https://api.ocp4.example.com:6443
Login successful.
...output omitted...
```

- 1.2. Create the `gitops-configure` project.

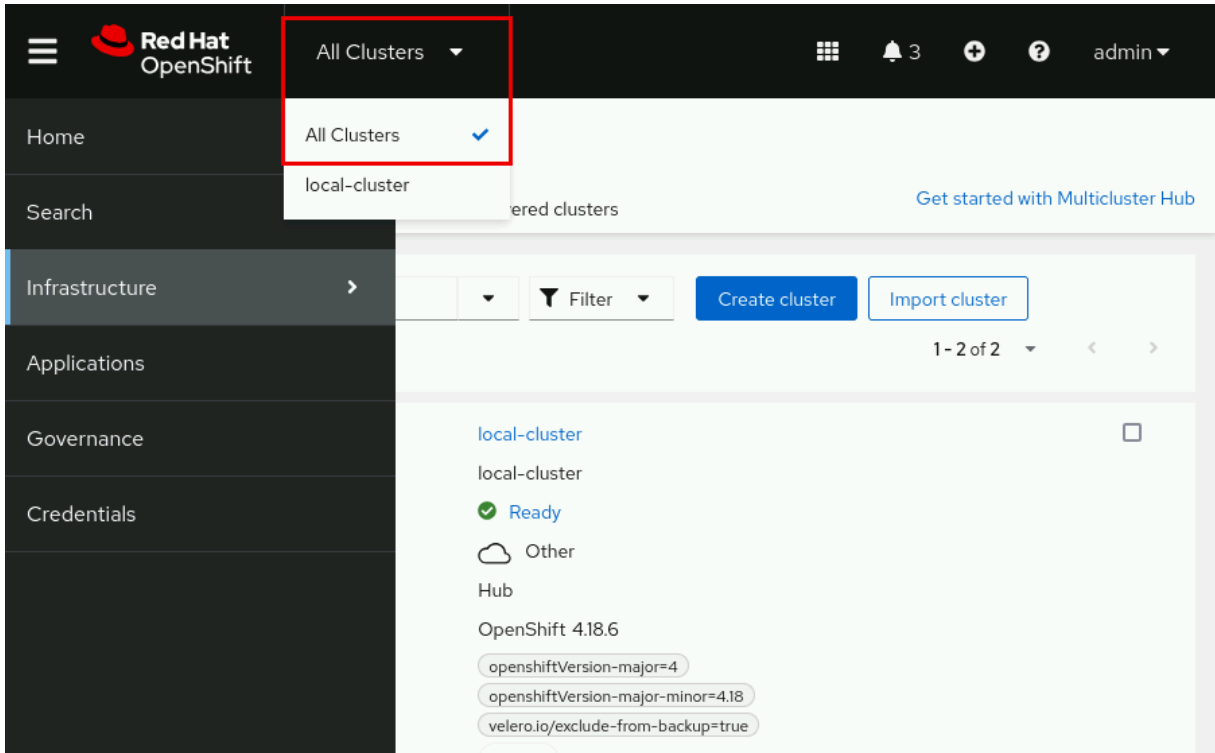
```
[student@workstation ~]$ oc new-project gitops-configure
Now using project "gitops-configure" on server "https://api.ocp4.example.com:6443".
...output omitted...
```

2. Log in to the RHOC hub cluster, and switch to the RHACM web console.

- 2.1. Open a web browser and go to `https://console-openshift-console.apps.ocp4.example.com`

- 2.2. Select the **Red Hat Identity Management** identity provider and log in as the `admin` user with `redhatocp` as the password.

- 2.3. Ensure that the cluster switcher is set to **All Clusters**, which corresponds to the RHACM web console. If it is not set in this way, then use the cluster switcher to select **All Clusters**.



3. Create a cluster set that groups all the clusters where the OpenShift GitOps operator must be installed. The placement resource that you create in another step relies on this cluster set to select the clusters.

3.1. Go to **Infrastructure > Clusters**, select the **Cluster sets** tab, and click **Create cluster set**.

3.2. Set the name of the cluster set to `gitops-configure` and click **Create**.

Create cluster set

Creating a **ManagedClusterSet** will allow you to group resources and manage access control to all of the resources in the group together. You will be able to manage the resource assignments for access control after creating the cluster set.

Cluster set name *

gitops-configure

CreateCancel

3.3. Click **Manage resource assignments**. Select the two clusters, click **Review**, and click **Save**.

Select resources to toggle their assignments to the cluster set

Resources can be added, removed, and transferred from other cluster sets (if you have permissions to remove from them from their assigned set).

Important: assigning a resource to the cluster set will give all cluster set users permissions to the resource's namespace.

2 selected

Search

1 - 2 of 2

Name	Infrastructure	Distribution version	Labels	Nodes	Current cluster set
☒ local-cluster	Other	OpenShift 4.18.6	openshiftV... openshiftV... velero.io/ex... 15 more	1	default
☒ managed-cluster	Other	OpenShift 4.18.6	openshiftV... openshiftV... 14 more	1	default

1 - 2 of 2 items

1 of 1 page

Review

Cancel

3.4. Bind the new cluster set to the gitops-configure namespace by clicking **Actions > Edit namespace bindings**.

gitops-configure

OverviewSubmariner add-onsCluster listCluster poolsUser management

Details

Namegitops-configure

Multicenter network statusInstall add-ons

Namespace bindings-
User managementNone

Actions

Edit namespace bindings
Manage resource assignments
Delete cluster set

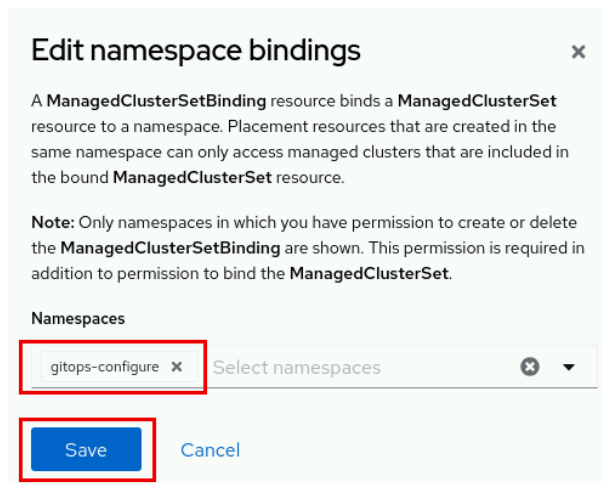
Status

0Submariner add-ons
Go to Submariner add-ons

2Clusters
Go to Managed clusters

0Cluster pools
Go to Cluster pools

Select the gitops-configure namespace, and click **Save**.



4. To install and configure the OpenShift GitOps operator, you use a governance policy. The `lab` command provides the policy generator resources that you review in this step.

4.1. From the terminal window, change to the `~/D00015L/labs/gitops-configure/policy` directory.

```
[student@workstation ~]$ cd ~/D00015L/labs/gitops-configure/policy
```

4.2. Review the `policy-generator.yaml` file that defines the `PolicyGenerator` resource. You do not have to change its contents.

The `manifests` section lists all the resources to include in the generated policy.

```
---
apiVersion: policy.open-cluster-management.io/v1
kind: PolicyGenerator
metadata:
  name: gitops-policy-generator
policyDefaults:
  namespace: gitops-configure
  orderManifests: true
  consolidateManifests: false
  remediationAction: enforce
  placement:
    name: gitops-configure
    labelSelector:
      vendor: OpenShift
  placementBindingDefaults:
    name: gitops-configure
  policies:
    - name: gitops-configure
      manifests:
        - path: gitops-operator.yaml
        - path: cluster-role-binding.yaml
        - path: ca-bundle.yaml
        - path: argocd-configuration.yaml
```

- ① The `gitops-operator.yaml` file declares the subscription resource that installs the OpenShift GitOps operator.
- ② The `cluster-role-binding.yaml` file declares a `ClusterRoleBinding` resource that grants Argo CD the `cluster-admin` cluster role.
- ③ The `ca-bundle.yaml` file declares a configuration map that includes the classroom custom CA certificate. The Argo CD instance mounts this configuration map, which allows Argo CD to trust the certificate from the classroom GitLab system. You use this GitLab system in another exercise to deploy an application with GitOps.

- ④ The `argocd-configuration.yaml` file configures the Argo CD instance to mount the configuration map. The file also grants the OpenShift admin user access to the Argo CD web console.

4.3. Review the `cluster-role-binding.yaml` file. You do not have to change its contents.

The cluster role binding grants the `cluster-admin` role to the `openshift-gitops-argocd-application-controller` service account in the `openshift-gitops` namespace.

The Argo CD instance uses this service account to run. Consequently, the instance gets cluster administrator rights, which enables Argo CD to create and manage applications in the clusters.

```
---
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: argo-admin
subjects:
- kind: ServiceAccount
  name: openshift-gitops-argocd-application-controller
  namespace: openshift-gitops
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
```

4.4. Review the `argocd-configuration.yaml` file. You do not have to change its contents.

The resource patches the ArgoCD resource in the `openshift-gitops` namespace.

```
---
apiVersion: argoproj.io/v1beta1
kind: ArgoCD
metadata:
  name: openshift-gitops
  namespace: openshift-gitops
spec:
  rbac: ①
    defaultPolicy: ""
    policy: |
      g, system:cluster-admins, role:admin
      g, cluster-admins, role:admin
      g, ocpadmins, role:admin
    scopes: '[groups]'
  repo: ②
    volumeMounts:
      - mountPath:
          /etc/pki/ca-trust/extracted/pem/tls-ca-bundle.pem
        subPath: ca-bundle.crt
        name: cluster-root-ca-bundle
    volumes:
      - configMap:
          name: cluster-root-ca-bundle
        name: cluster-root-ca-bundle
    ...output omitted...
```

- ① The `rbac` section grants the Argo CD admin role to the members of the OpenShift `ocpadmins` group. This Argo CD admin role enables access to the Argo CD web console.

The OpenShift admin user is a member of the OpenShift `ocpadmins` group, and therefore the user is granted access to the Argo CD web console.

- 2 The repo section mounts the cluster-root-ca-bundle configuration map as a volume. This configuration map includes the classroom custom CA certificate, which enables Argo CD to trust the classroom GitLab system to pull Git repositories.

5. Apply the governance policy.

5.1. Run the PolicyGenerator command, and apply the resulting resources.

```
[student@workstation policy]$ PolicyGenerator policy-generator.yaml | \
oc apply -f -
policy.policy.open-cluster-management.io/gitops-configure created
placement.cluster.open-cluster-management.io/gitops-configure created
placementbinding.policy.open-cluster-management.io/binding-gitops-configure created
```

5.2. Switch to the RHACM web console for following the remediation process. Go to **Governance** and select the **Policies** tab. Wait for both clusters to be compliant, which takes up to five minutes.

Name	Namespace	Remediation	Policy set	Cluster violations	Source
gitops-configure	gitops-configure	enforce	-	2	Local

Name	Namespace	Remediation	Policy set	Cluster violations	Source
gitops-configure	gitops-configure	enforce	-	1 1	Local

Name	Namespace	Remediation	Policy set	Cluster violations	Source
gitops-configure	gitops-configure	enforce	-	2	Local

5.3. Switch to the local-cluster view to verify that the OpenShift GitOps operator is installed in the local cluster.

Red Hat OpenShift

local-cluster

admin

Overview

local-cluster

Cluster

Status

Cluster

Control Plane

Operators

Insights

Dynamic Plugins

5.4. Go the **Operators > Installed operators**, select All Projects, and verify that the page lists the Red Hat OpenShift GitOps operator.

Project: All Projects

Installed Operators

Installed Operators are represented by ClusterServiceVersions within this Namespace. For more information, see the [Understanding Operators documentation](#). Or create an Operator and ClusterServiceVersion using the [Operator SDK](#).

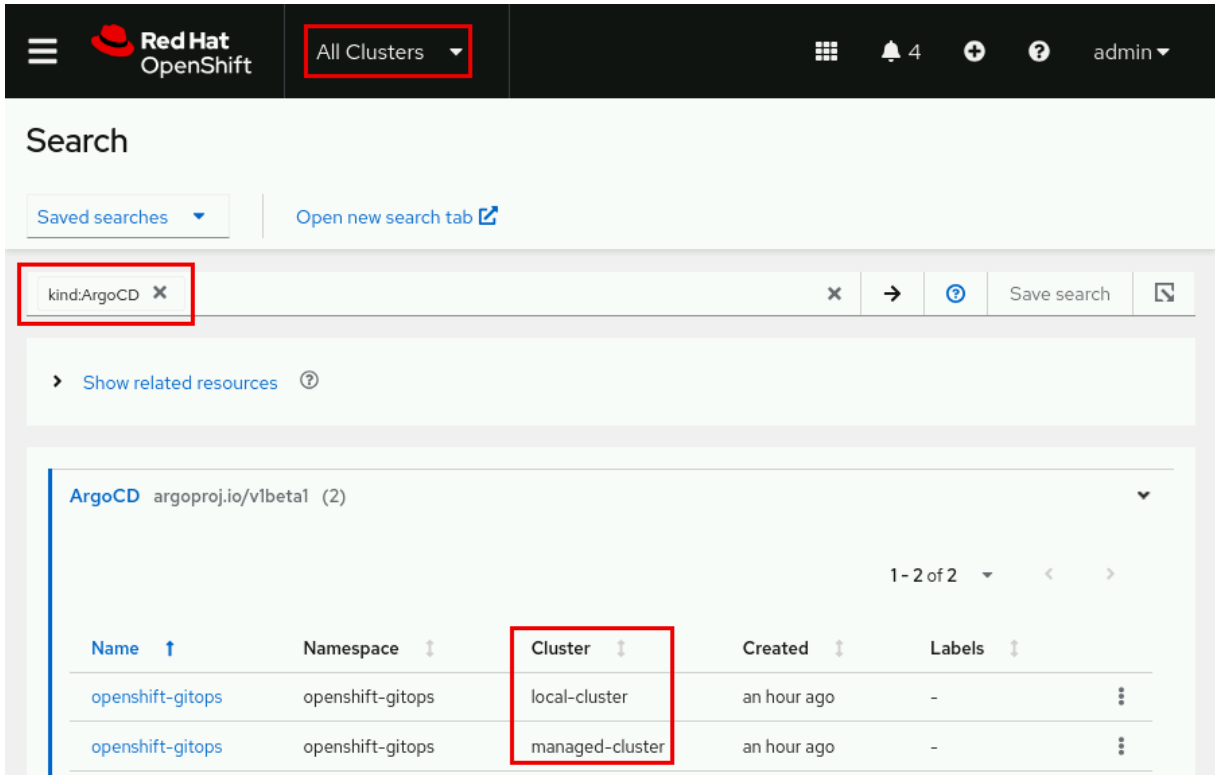
Name	Namespace	Managed Namespaces
 Advanced Cluster Management for Kubernetes 2.13.0 provided by Red Hat NS open-cluster-management NS open-cluster-management		
 LVM Storage 4.18.2 provided by Red Hat NS openshift-storage NS openshift-storage		
 MetalLB Operator 4.18.0-202505200035 provided by Red Hat NS metallb-system All Namespaces		
 multicenter engine for Kubernetes 2.8.0 provided by Red Hat NS multicenter-engine NS multicenter-engine		
 Red Hat OpenShift GitOps 1.16.1 provided by Red Hat Inc NS openshift-gitops-operator All Namespaces		

5.5. Switch back to the All Clusters view. Go to **Search**, and use `kind:ArgoCD` for the search request. The result lists two instances, with one instance in each cluster.

 **Note**

The Push model requires only the OpenShift GitOps operator on the hub cluster. The Pull model requires the operator on all the clusters.

To experience the Pull model in this course, you installed the operator in both clusters.



6. Import the clusters into Argo CD.
- 6.1. The lab command prepared the ~/D00015L/labs/gitops-configure/gitops-register.yaml resource file. Review the file from the terminal window. You do not have to change its contents.


```
---
apiVersion: cluster.open-cluster-management.io/v1beta2
kind: ManagedClusterSetBinding ❶
metadata:
  name: gitops-configure
  namespace: openshift-gitops
spec:
  clusterSet: gitops-configure
---
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement ❷
metadata:
  name: gitops-configure
  namespace: openshift-gitops
spec:
  tolerations:
    - key: cluster.open-cluster-management.io/unreachable
      operator: Exists
    - key: cluster.open-cluster-management.io/unavailable
      operator: Exists
  clusterSets:
    - gitops-configure
---
apiVersion: apps.open-cluster-management.io/v1beta1
kind: GitOpsCluster ❸
metadata:
  name: gitops-configure
  namespace: openshift-gitops
spec:
  argoServer:
    argoNamespace: openshift-gitops
  placementRef:
    kind: Placement
    apiVersion: cluster.open-cluster-management.io/v1beta1
    name: gitops-configure
    namespace: openshift-gitops
```

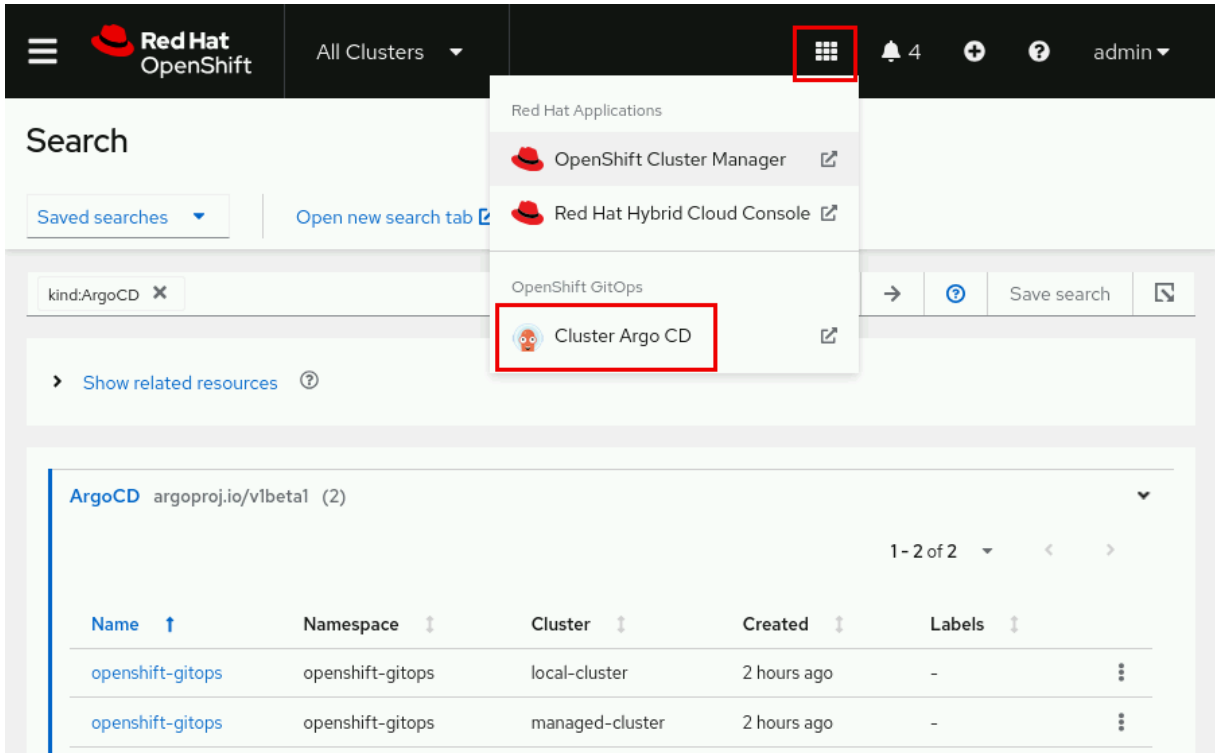
- ❶ Bind the `gitops-configure` cluster set to the `openshift-gitops` namespace, which hosts the Argo CD instance. This cluster set groups all your clusters.
- ❷ The placement resource selects the clusters to import into Argo CD from the cluster set. With this resource, all the clusters from the cluster set are selected.
- ❸ The `GitOpsCluster` resource imports the clusters from the placement resource into the Argo CD instance that is running in the hub cluster.

6.2. Import the cluster by applying the `gitops-register.yaml` resource file.

```
[student@workstation policy]$ oc apply -f \
~/DO0015L/labs/gitops-configure/gitops-register.yaml
managedclustersetbinding.cluster.open-cluster-management.io/gitops-configure created
placement.cluster.open-cluster-management.io/gitops-configure created
gitopscluster.apps.open-cluster-management.io/gitops-configure created
```

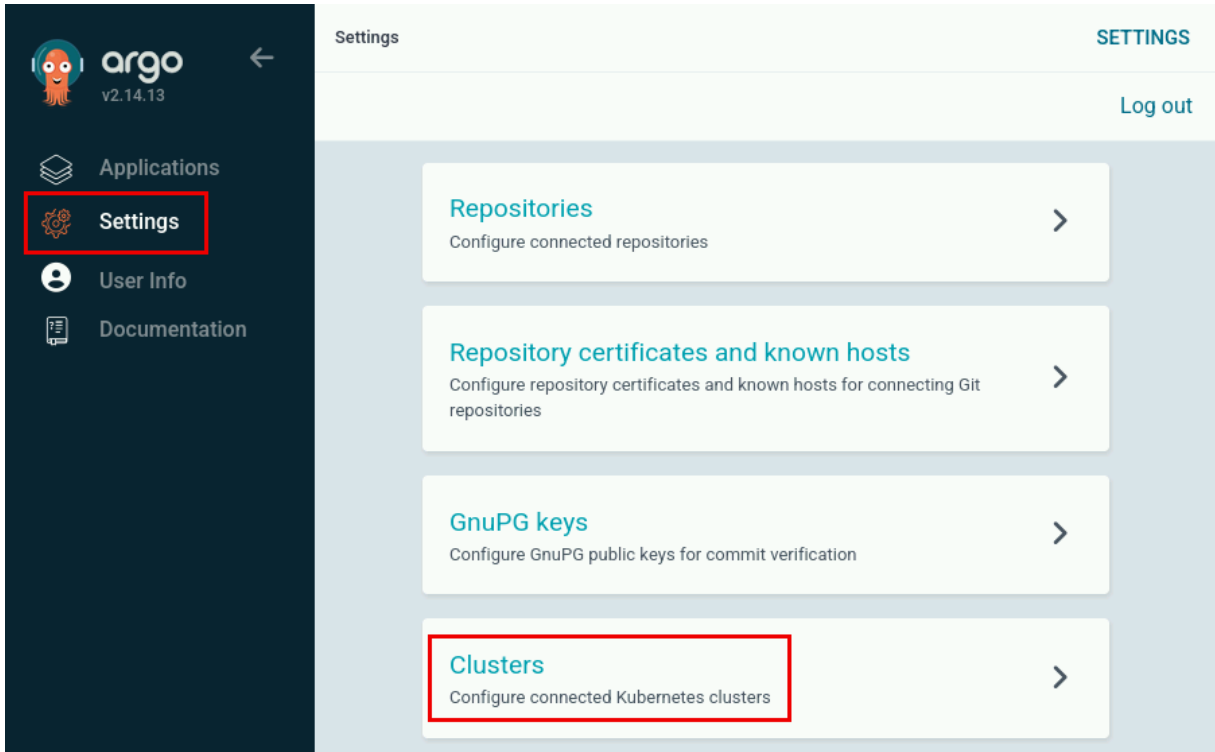
7. Verify that the clusters are imported into Argo CD.

7.1. Switch to the RHACM web console, and access the Argo CD web console by using the Applications menu.



7.2. Click **Log in via OpenShift**. Select the **Red Hat Identity Management** identity provider and log in as the admin user with redhatocp as the password. Allow the access by clicking **Allow selected permissions**.

7.3. Go to **Settings > Clusters**.



The two clusters are displayed. The connection status is unknown, because Argo CD did not attempt yet to access the clusters. The instance accesses the clusters when Argo CD has an application to deploy.

Settings / Clusters CLUSTERS

Refer to CLI [Documentation](#) for adding clusters. Log out

NAME	URL	VERSION	CONNECTION STATUS
in-cluster	https://kubernetes.default.svc		Unknown
local-cluster	https://api.ocp4.example.com:6443		Unknown
managed-cluster	https://api.ocp4-mng.example.com:6443		Unknown

7.4. From the terminal window, change to the /home/student directory.

```
[student@workstation policy]$ cd
```

Do not delete your work, because you use your installation to deploy an application in the following exercise.

Finish

On the workstation machine, use the lab command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish gitops-configure
```

5.5. Deploying and Managing Multicuster Application Resources with RHACM

Objectives

- Deploy applications across multiple clusters by using RHACM GitOps.

Kustomize

When you create an application with RHACM, Argo CD retrieves the application's resources from a Git repository. Argo CD supports several content types, including the following technologies:

- A directory of YAML or JSON resources
- Kustomize
- Helm charts

Kustomize is a tool that customizes Kubernetes resources for different environments or needs. Kustomize is a solution that helps reuse configurations and patch any resource.

The most common use case is the need to define different resources for environments such as development, staging, and production.

Kustomize separates these configuration sets into two types: base and overlays. *Base* holds all configuration and resources that are common to all the derivatives. *Overlays* represent the differences from the base for a specific environment.

Kustomize uses directories to represent these configuration sets. The following directory structure is an example of such organization:

```
myapp
├── base
│   ├── deployment.yaml
│   ├── kustomization.yaml
│   ├── route.yaml
│   └── service.yaml
└── overlays
    ├── development
    │   └── kustomization.yaml
    ├── production
    │   ├── kustomization.yaml
    │   └── pvc.yaml
```

Kustomization Files

Each configuration set needs a `kustomization.yaml` file. The `kustomization.yaml` files define how the settings in the overlay interact with the base YAML files to build new manifests. The new manifest builds are called *variants*, and consist of the base YAML files and their overlays.

Generally, in a multicuster infrastructure, each environment has a corresponding `kustomization.yaml` file that contains specific settings for this environment.

For the base configuration set, the `kustomization.yaml` file contains a `resources` section that lists the application's files:

```
---
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
- deployment.yaml
- route.yaml
- service.yaml
```

For the other environments, the `kustomization.yaml` file reuses the resources from the base configuration set, and can include some additional resources or modifications to the base resources.

The following example shows the `kustomization.yaml` file for a production environment. The file reuses the resources from the base configuration set and adds the `pvc.yaml` resource file. The file also patches the deployment resource from the base configuration set.

```
---
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
- ../../base
- pvc.yaml

patches:
- patch: |-
    apiVersion: apps/v1
    kind: Deployment
    metadata:
      name: myapp
    spec:
      replicas: 3
```

Deploying Applications with Kustomize

Argo CD automatically detects the Kustomize structure of the application in the Git repository, and uses Kustomize to deploy the application.

You can also use Kustomize as a stand-alone tool with the `kustomize` or the `oc apply -k` command.

These commands take the directory with the configuration set to apply as a parameter. The following example uses the `kustomize` command to deploy the application for the production environment:

```
[user@host ~]$ kustomize build myapp/overlays/production | oc apply -f -
```

You can obtain the same result by using the `oc apply -k` command:

```
[user@host ~]$ oc apply -k myapp/overlays/production
```

Managing Applications with RHACM

With RHACM, you manage applications across multicloud environments by implementing `ApplicationSet` and `Placement` resources.

An Argo CD `ApplicationSet` resource describes the application to deploy on managed clusters. The `ApplicationSet` resource acts as a template that Argo CD uses to create an `Application` resource for each cluster. The `Application` resource contains the details of the application to deploy, such as the Git repository.

The following example shows an `ApplicationSet` resource that deploys the `myapp` application:

```
---
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: myapp
  namespace: openshift-gitops
spec:
  generators:
    - clusterDecisionResource:
        configMapRef: acm-placement
        labelSelector:
          matchLabels:
            cluster.open-cluster-management.io/placement: myapp-placement ①
        requeueAfterSeconds: 180
  template: ②
    metadata:
      labels:
        velero.io/exclude-from-backup: "true"
      name: myapp-{{name}} ③
    spec:
      destination:
        namespace: myapp
        server: '{{server}}' ④
      project: default
      sources: ⑤
        - path: kustomize/overlays/production
          repoURL: https://github.com/myorganization/myapp.git
          targetRevision: main
      syncPolicy:
        automated:
          prune: true
          selfHeal: true
        syncOptions:
          - CreateNamespace=true
          - PruneLast=true
```

- ① Argo CD uses the `generators` section to retrieve the list of clusters where the application must be deployed. The section refers to the `myapp-placement` resource, which Argo CD uses to select the clusters.
- ② The `template` section provides a template for the `Application` resources that Argo CD generates for each cluster.
- ③ In templates, you enclose variable substitutions within braces. In this example, Argo CD replaces the `{{name}}` string with the cluster name in the resulting `Application` resource. For example, if you deploy the application on a managed cluster named `test-cluster`, then the name of the `Application` resource is `myapp-test-cluster`.
- ④ The `server` parameter provides the OpenShift API URL of the managed cluster. Argo CD uses this API to manage resources in the cluster. For generating the `Application` resources, Argo CD replaces the `{{server}}` string with the API URL of the selected managed clusters.
- ⑤ The `sources` section provides the details of the Git repository that hosts the application resources.

Argo CD generates the `Application` resources in the `openshift-gitops` namespace.

Note

Because several Kubernetes controllers define an Application CRD, you must specify the `argoproj.io` API group when referring to Argo CD applications:

```
[user@host ~]$ oc get applications.argoproj.io -n openshift-gitops
NAME                               SYNC STATUS  HEALTH STATUS
myapp-prod-cluster                 Synced       Healthy
myapp-qa-cluster                   Synced       Healthy
myapp-test-cluster                 Synced       Healthy
```

A Placement resource selects clusters from a cluster set. ApplicationSet resources refer to Placement resources in the `openshift-gitops` namespace.

The following example shows a Placement resource that selects OpenShift clusters by label:

```
---
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: myapp-placement
  namespace: openshift-gitops
spec:
  clusterSets: ①
  - my-openshift-clusters
  numberOfClusters: 3 ②
  predicates: ③
  - requiredClusterSelector:
      labelSelector:
        matchLabels:
          openshiftVersion: 4.18.6
```

- ① The placement selects the clusters from the given cluster set. You must bind this cluster set to the `openshift-gitops` namespace by creating a `ManagedClusterSetBinding` resource.
- ② By default, the placement selects all the clusters that match the predicate. You can limit the number of clusters by setting the `numberOfClusters` parameter. If the number of clusters that match the predicate is larger than the value of the `numberOfClusters` parameter, then the placement selects a random set.
- ③ The predicates section specifies the criteria to select the clusters from the set. The predicates are OR functions. In this example, you select the managed clusters with the `openshiftVersion=4.18.6` label.

Instead of creating these resources from YAML files, you can also use the RHACM web console.

Creating Applications by Using the RHACM Web Console

The **Applications** section enables you to create application sets from the web interface.

To initiate the creation process, go to **Create application > Argo CD ApplicationSet - Push model** or **Create application > Argo CD ApplicationSet - Pull model**. The interface guides you through the following pages:

1. On the first page, you provide the application name.

Create application set - push model

YAML

1 General

2 Template

3 Sync policy

4 Placement

5 Review

General

Name *
myapp

Argo server * ?
Select the Argo server
openshift-gitops
name: gitops-configure; namespace: openshift-gitops
Add Argo Server

Next Back Cancel

- 1 Name of the ApplicationSet resource.
- 2 The Argo CD instance that is running in the hub cluster. If you did not import your cluster to Argo CD during RHACM configuration, then you can add the cluster from this page.

2. On the template page, you provide the application repository.

1 General

2 Template

3 Sync policy

4 Placement

5 Review

Repository

Type
Git https://github.com/myorganization/myapp

1

Git
Use a Git repository

2

URL * ?
https://github.com/myorganization/myapp.git

3

Revision ?
main

4

Path ?
kustomize/overlays/production

Next Back Cancel

- 1 Argo CD supports Git and Helm repositories.
- 2 For Git, provide the URL of the Git repository that hosts the application.
- 3 Provide the tag, branch, or commit to use from the Git repository.

4. Provide the path in the Git repository to the application resources. The example specifies the path to a Kustomize configuration set.
3. The synchronization policy page provides Argo CD with instructions for managing several aspects of the application lifecycle. For example, you can instruct Argo CD to automatically revert local changes to a resource by reapplying the resource from the Git repository.

1

General

2

Template

3

Sync policy

4

Placement

5

Review

Sync policy

Settings used to configure application syncing when there are differences between the desired state and the live cluster state.

☒ Delete resources that are no longer defined in the source repository

☒ Delete resources that are no longer defined in the source repository at the end of a sync operation

☐ Replace resources instead of applying changes from the source repository

☐ Allow applications to have empty resources

☐ Only synchronize out-of-sync resources

☒ Automatically sync when cluster state changes

☒ Automatically create namespace if it does not exist

☐ Disable kubectl validation

Next

Back

Cancel

4. From the placement page, you can select an existing Placement resource in the openshift-gitops namespace. You can also create the Placement resource directly from the page.

Placement

How do you want to select clusters?

1 New placement Existing placement

Cluster sets ?

my-openshift-clusters 1

If no cluster sets are selected, all clusters will be selected from the cluster sets bound to the namespace.

Label expressions ?

openshiftVersion equals 4.18.6

Label *	Operator *	Values *
openshiftVersion	equals any of	4.18.6 1

+ Add label expression

Next Back Cancel

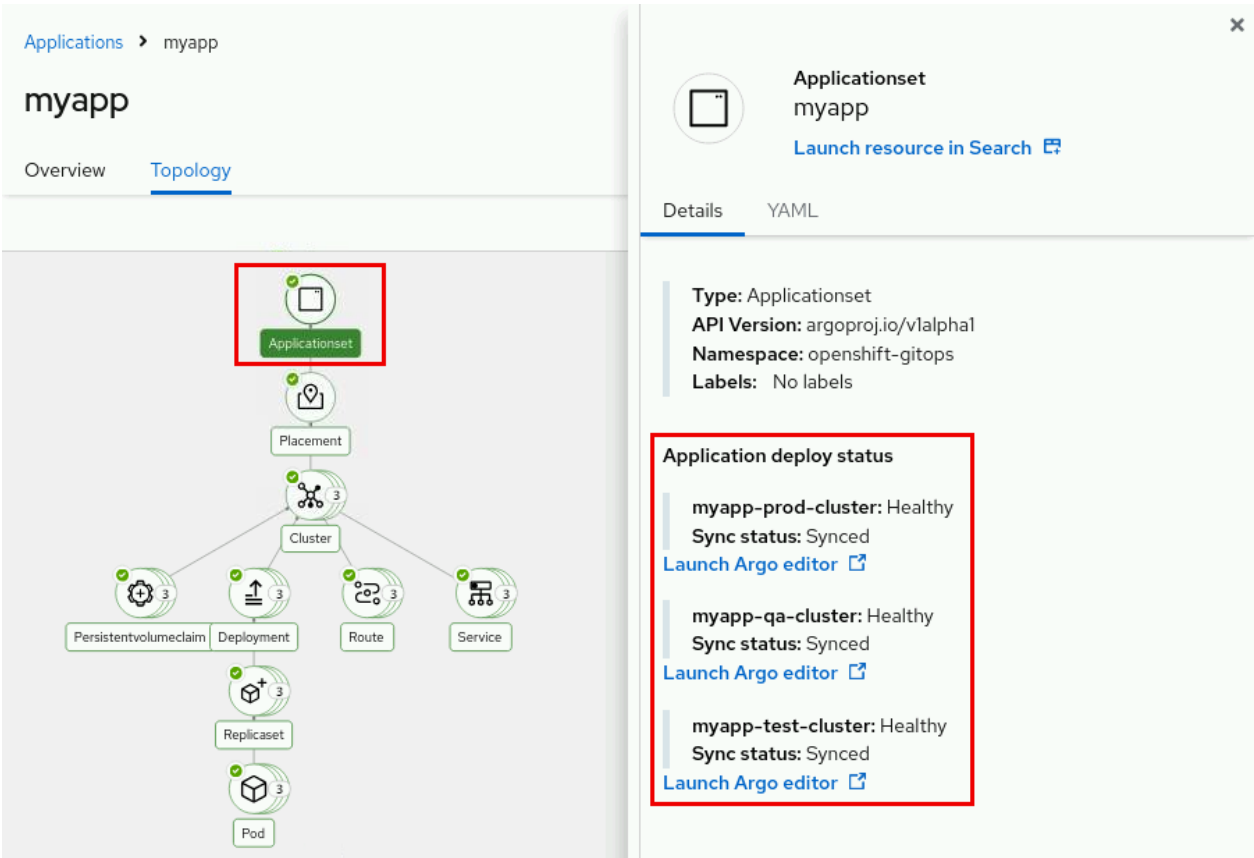
- Whether to create a Placement resource or use an existing one.
- Select a cluster set that you bound to the openshift-gitops project. The Placement resource selects clusters from this set.
- Provide the labels that the Placement resource uses to select clusters.

5. From the review page, you can review your configuration and submit the ApplicationSet resource.

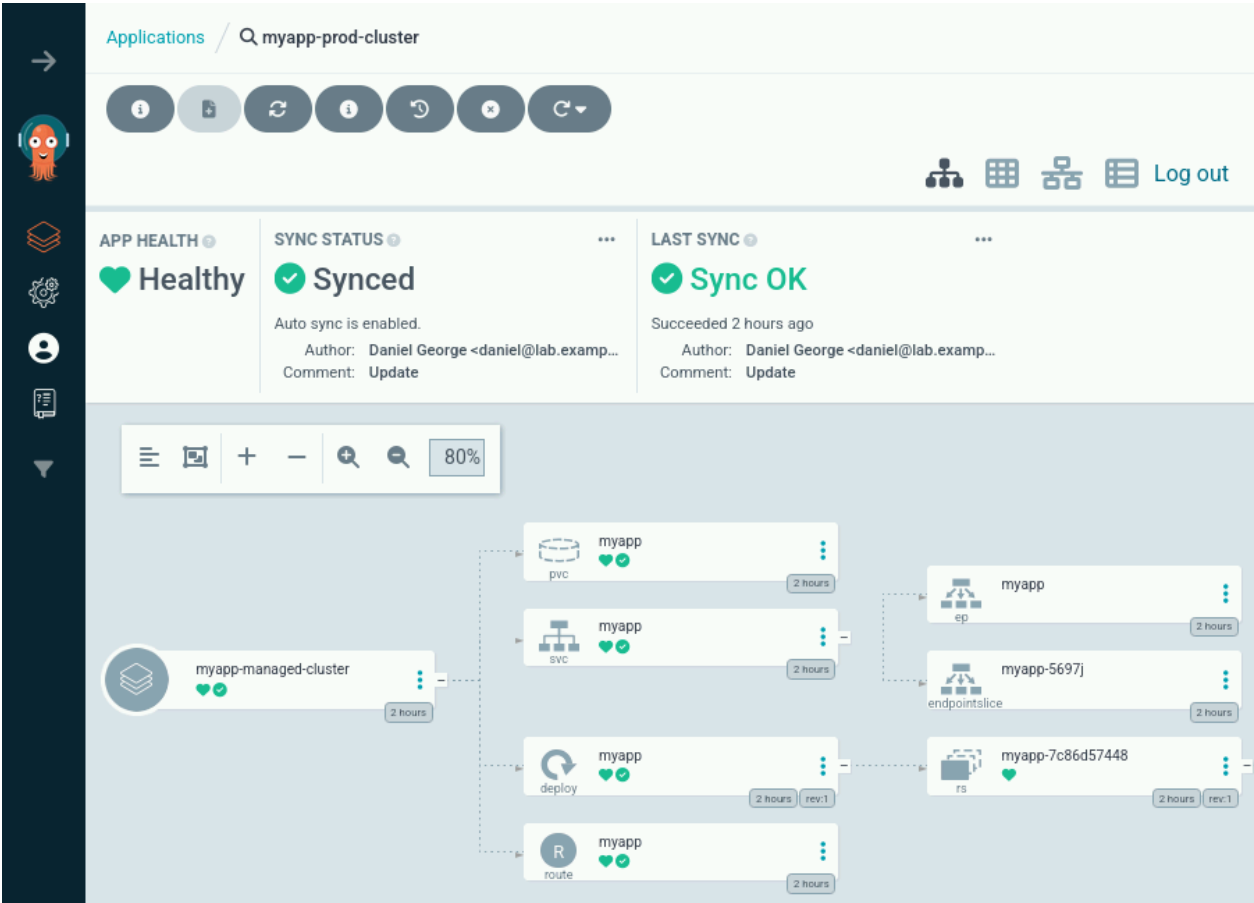
Reviewing Application Deployments

After the application is deployed, use the Topology view to review the status of the application.

When you select an icon, the side panel provides details for the selected resource and enables you to review the resource's YAML manifest. By selecting the application set, you can review the ApplicationSet YAML manifest, and you can launch the Argo CD web console to review the resulting Application resource:



Argo CD provides a similar view to the topology page:



Deploying an Application by Using the Pull Model

Using the Pull model requires you to install the Red Hat OpenShift GitOps operator in each managed cluster where you

plan to deploy applications with GitOps.

Deploying an application by using the Pull model is similar to using the Push model, with a few differences:

- You cannot deploy applications on the hub cluster. Your Placement resource must exclude this cluster.
- If you create the ApplicationSet resource in YAML format, without using the RHACM web console, then the template section requires additional annotations and labels:

```
---
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: myapp-pull
  namespace: openshift-gitops
spec:
  generators:
    - clusterDecisionResource:
        configMapRef: acm-placement
        labelSelector:
          matchLabels:
            cluster.open-cluster-management.io/placement: myapp-pull-placement
        queueAfterSeconds: 180
  template:
    metadata:
      annotations:
        apps.open-cluster-management.io/ocm-managed-cluster: '{{name}}'
        apps.open-cluster-management.io/ocm-managed-cluster-app-namespace:
          openshift-gitops
        argocd.argoproj.io/skip-reconcile: "true"
      labels:
        apps.open-cluster-management.io/pull-to-ocm-managed-cluster: "true"
        velero.io/exclude-from-backup: "true"
      name: myapp-pull-{{name}}
    spec:
      destination:
        namespace: myapp-pull
        server: '{{server}}'
      project: default
      sources:
        - path: kustomize/overlays/production
          repoURL: https://github.com/myorganization/myapp.git
          targetRevision: main
      syncPolicy:
        automated:
          prune: true
          selfHeal: true
        syncOptions:
          - CreateNamespace=true
          - PruneLast=true
```

With such an ApplicationSet resource, RHACM deploys the application with the following steps:

1. The Argo CD instance that is running in the hub cluster processes the ApplicationSet resource and creates the corresponding Application resources.

The `argocd.argoproj.io/skip-reconcile="true"` annotation instructs Argo CD not to process the Application resources further. In other words, the hub cluster Argo CD instance creates the Application resources, but does not deploy them.

2. RHACM detects the new Application resources with the `apps.open-cluster-management.io/pull-to-ocm-managed-cluster="true"` label.

3. RHACM creates ManifestWork resources in the namespaces of the managed clusters.

RHACM uses the `apps.open-cluster-management.io/ocm-managed-cluster` and `apps.open-cluster-management.io/ocm-managed-cluster-app-namespace` annotations to create these resources. A ManifestWork resource embeds the Application resource definition for the cluster.

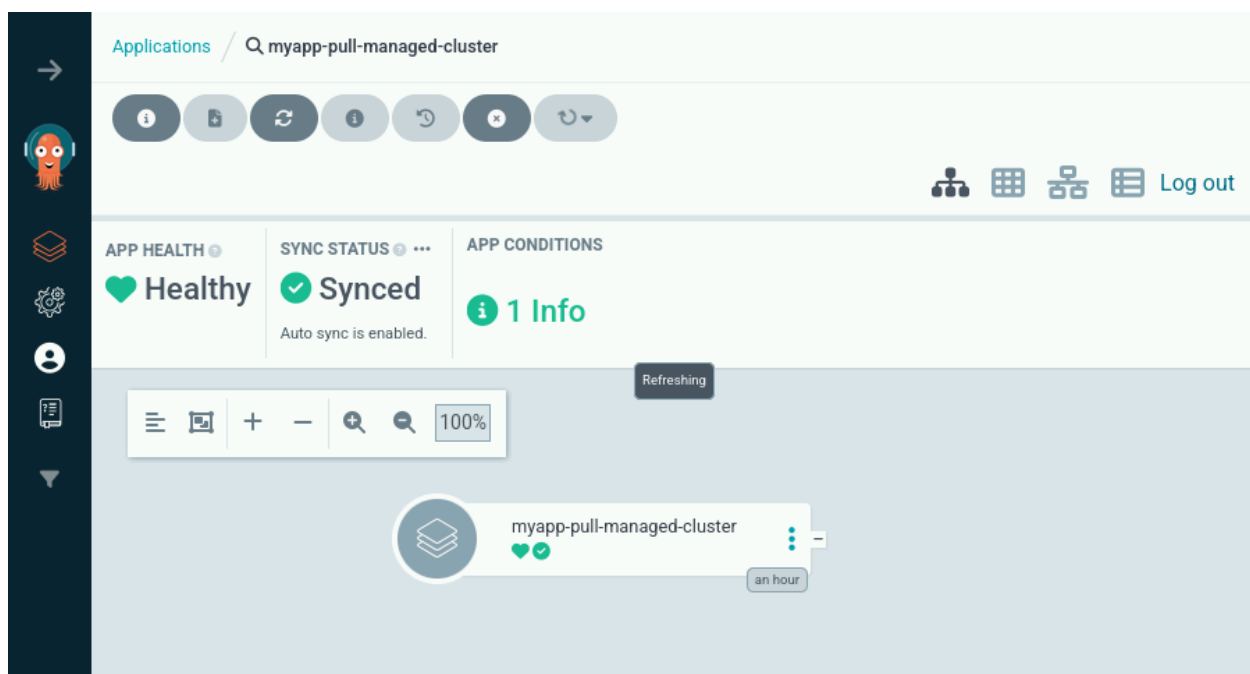
4. The managed cluster retrieves and processes the ManifestWork resource from the hub cluster. The managed cluster creates the Application resource.

5. The Argo CD instance that is running in the managed cluster detects and processes the Application resource, which deploys the application.

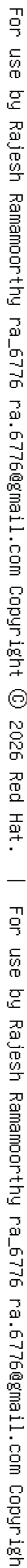
6. The RHACM hub cluster collects the status of the application by creating a MulticloudApplicationSetReport resource.

7. The topology view uses the MulticloudApplicationSetReport resource for displaying the application status.

The Argo CD web console on the hub cluster shows the application, but no resources are displayed:



The Argo CD web console on the managed cluster shows the application and its resources:



For use by Rajesh Ramamorthy ra-6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra-6776 ra.6776@gmail.com Copyright

For use by Rajesh Ramamorthy ra-6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra-6776 ra.6776@gmail.com Copyright

```
[user@host ~]$ oc delete ApplicationSet myapp -n openshift-gitops
applicationset.argoproj.io "myapp" deleted
```

For use by Rajesh Ramamorthy ra-6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra-6776 ra.6776@gmail.com Copyright

For use by Rajesh Ramamorthy ra-6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra-6776 ra.6776@gmail.com Copyright

```
[user@host ~]$ oc delete placement myapp-placement -n openshift-gitops  
placement.cluster.open-cluster-management.io "myapp-placement" deleted
```

References

For more information about deploying applications, refer to the *Deploying Argo CD with Push and Pull Model* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 GitOps* documentation at https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/gitops/index#gitops-push-pull

[Introducing the Argo CD Application Pull Controller for Red Hat Advanced Cluster Management](#)

[Reference Documentation for Kustomize](#)

[Argo CD Supported Tools](#)

5.6. Guided Exercise

Deploy and Manage Multicenter Application Resources with RHACM

Deploy a task list application on multiple clusters by using the RHACM GitOps workflow.

Outcomes

- Create GitOps applications that deploy resources from Git repositories.
- Inspect the status of deployed applications by using the RHACM topology view and the Argo CD web console.

Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab install do0015l ; lab start gitops-deploy
```

Instructions

In this exercise, you use GitOps with RHACM to deploy a task list application in your two clusters.

1. Retrieve and review the task list application.
 - 1.1. In a terminal window, clone the application from the classroom GitLab system. The application is available in the `https://git.ocp4.example.com/do0015l/gitops-deploy.git` repository.

```
[student@workstation ~]$ git clone \
https://git.ocp4.example.com/do0015l/gitops-deploy.git
Cloning into 'gitops-deploy'...
...output omitted...
```

- 1.2. Review the application files. The developers use Kustomize to deploy the application in the development and production environments.

```
[student@workstation ~]$ tree gitops-deploy
gitops-deploy/
├── README.md
├── kustomize
│   ├── base ①
│   │   ├── deployment.yaml
│   │   ├── kustomization.yaml
│   │   ├── route.yaml
│   │   └── service.yaml
│   └── overlays
│       ├── dev
│       │   └── kustomization.yaml
│       └── prod ②
│           └── kustomization.yaml
5 directories, 7 files
```

- ① The base directory includes a deployment, a service, and a route resource.
- ② The prod directory contains the `kustomization.yaml` file to adjust the base resources to the production environment.

- 1.3.** Review the deployment resource in the base directory in the `gitops-deploy/kustomize/base/deployment.yaml` file. You do not have to change its contents.

The resource deploys one replica.

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: todo-app
    name: todo-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: todo-app
  template:
    metadata:
      labels:
        app: todo-app
    spec:
      containers:
        - name: todo-app
          image: registry.ocp4.example.com:8443/redhattraining/todo-angular:v1.1
...output omitted...
```

- 1.4.** Review the route resource in the `gitops-deploy/kustomize/base/route.yaml` file. You do not have to change its contents.

The route resource uses the HTTP plain-text protocol.

```
---
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  labels:
    app: todo-app
    name: todo-app
spec:
  to:
    kind: Service
    name: todo-app
```

- 1.5.** Review the `gitops-deploy/kustomize/overlays/prod/kustomization.yaml` file that customizes the resources for the production environment. You do not have to change its contents.

```
---
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
- ../../base

labels:
- pairs:
  env: production

patches:
- patch: |- ①
  apiVersion: apps/v1
  kind: Deployment
  metadata:
    name: todo-app
  spec:
    replicas: 3
- patch: |- ②
  apiVersion: route.openshift.io/v1
  kind: Route
  metadata:
    name: todo-app
  spec:
    tls:
      termination: edge
```

- ① For production, the deployment consists of three replicas.
- ② The customization configures a secured route that uses an edge TLS termination.

2. Log in to the RHOC hub cluster, and switch to the RHACM web console.

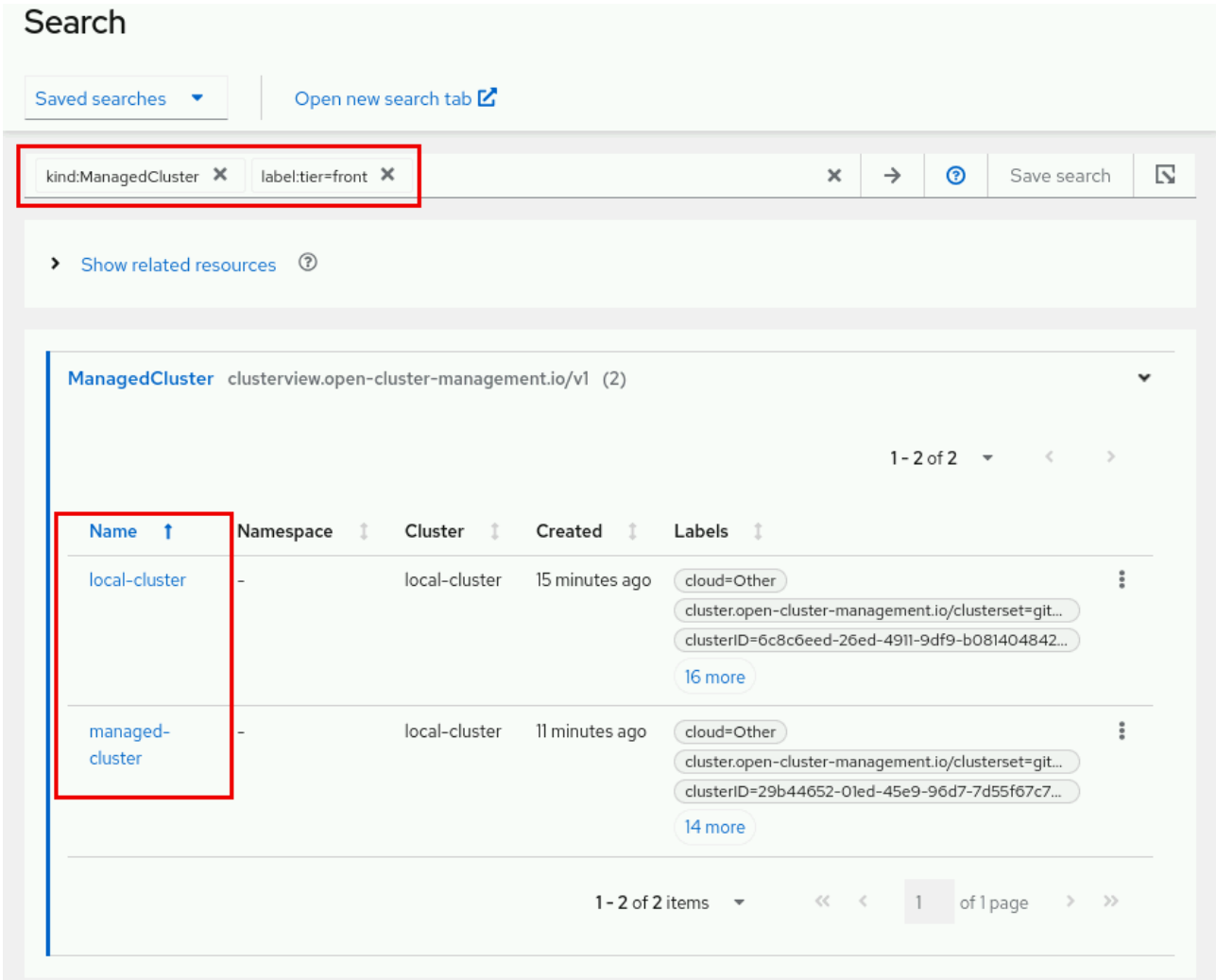
2.1. Open a web browser and go to <https://console-openshift-console.apps.ocp4.example.com>

2.2. Select the **Red Hat Identity Management** identity provider and log in as the admin user with redhatocp as the password.

2.3. Ensure that the cluster switcher is set to **All Clusters**, which corresponds to the RHACM web console.

3. Verify that the lab command added the tier=front label to the managed clusters.

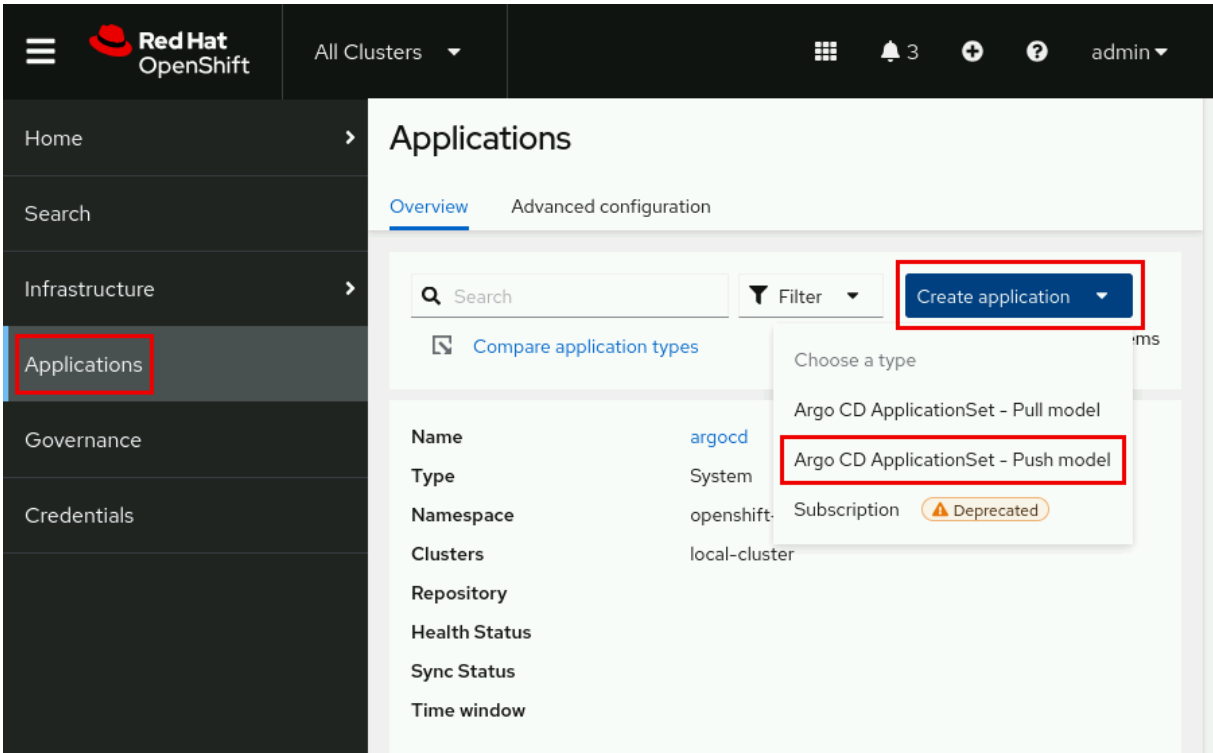
Go to **Search** and use the kind:ManagedCluster label:tier=front request.



You use this label to select the two clusters where the task list application must be installed.

4. Deploy the task list application by using the GitOps Push model.

4.1. Go to **Applications** and select **Create application > Argo CD ApplicationSet - Push model**.



4.2. Set the name of the application to todo-app, select the openshift-gitops Argo server, and click **Next**.

Create application set - push model ☐ YAML

1 General

General

Name *

todo-app

Argo server * ?

openshift-gitops

Requeue time * ?

180

Next

Back

Cancel

4.3. Configure the repository as follows, and click **Next**.

Parameter	Value
Type	Git
URL	https://git.ocp4.example.com/do0015l/gitops-deploy.git

Parameter	Value
Revision	main
Path	kustomize/overlays/prod
Remote namespace	todo-app

Repository

Type

Githttps://git.ocp4.example.com/do0015l/gitops-deploy.git

Git

Use a Git repository

HELM

Helm

Use a Helm repository

URL * ?

https://git.ocp4.example.com/do0015l/gitops-deploy.git

Revision ?

main

✕

▼

↺

Path ?

kustomize/overlays/prod

✕

▼

↺

+ Add another repository

Destination

Remote namespace *

todo-app

Next

Back

Cancel

- 4.4. For the synchronization policy, clear the **Automatically sync when cluster state changes** option, and click **Next**. With this configuration, Argo CD does not revert changes that you make to resources directly from the system, instead of updating the Git repository.

Sync policy

Settings used to configure application syncing when there are differences between the desired state and the live cluster state.

- ☒ Delete resources that are no longer defined in the source repository
- ☒ Delete resources that are no longer defined in the source repository at the end of a sync operation
- ☐ Replace resources instead of applying changes from the source repository
- ☐ Allow applications to have empty resources
- ☐ Only synchronize out-of-sync resources
- ☐ Automatically sync when cluster state changes
- ☒ Automatically create namespace if it does not exist
- ☐ Disable kubectrl validation
- ☐ Prune propagation policy

Next

Back

Cancel

- 4.5. Create a placement to select the clusters with the tier=front label. Select the gitops-configure cluster set and click **Add label expression**. Set **Label** to tier, set **Operator** to equals any of, and set **Values** to front. Set **Limit the number of clusters selected** to 2. You might have to scroll down to access this control. Click **Next**.

Placement

How do you want to select clusters?

New placement

Existing placement

Cluster sets ?

gitops-configure

1

If no cluster sets are selected, all clusters will be selected from the cluster sets bound to the namespace.

Label expressions ?

tier equals front

Label *

Operator *

Values *

tier

equals any of

front

1

+ Add label expression

Limit the number of clusters selected

-

2

+

Next

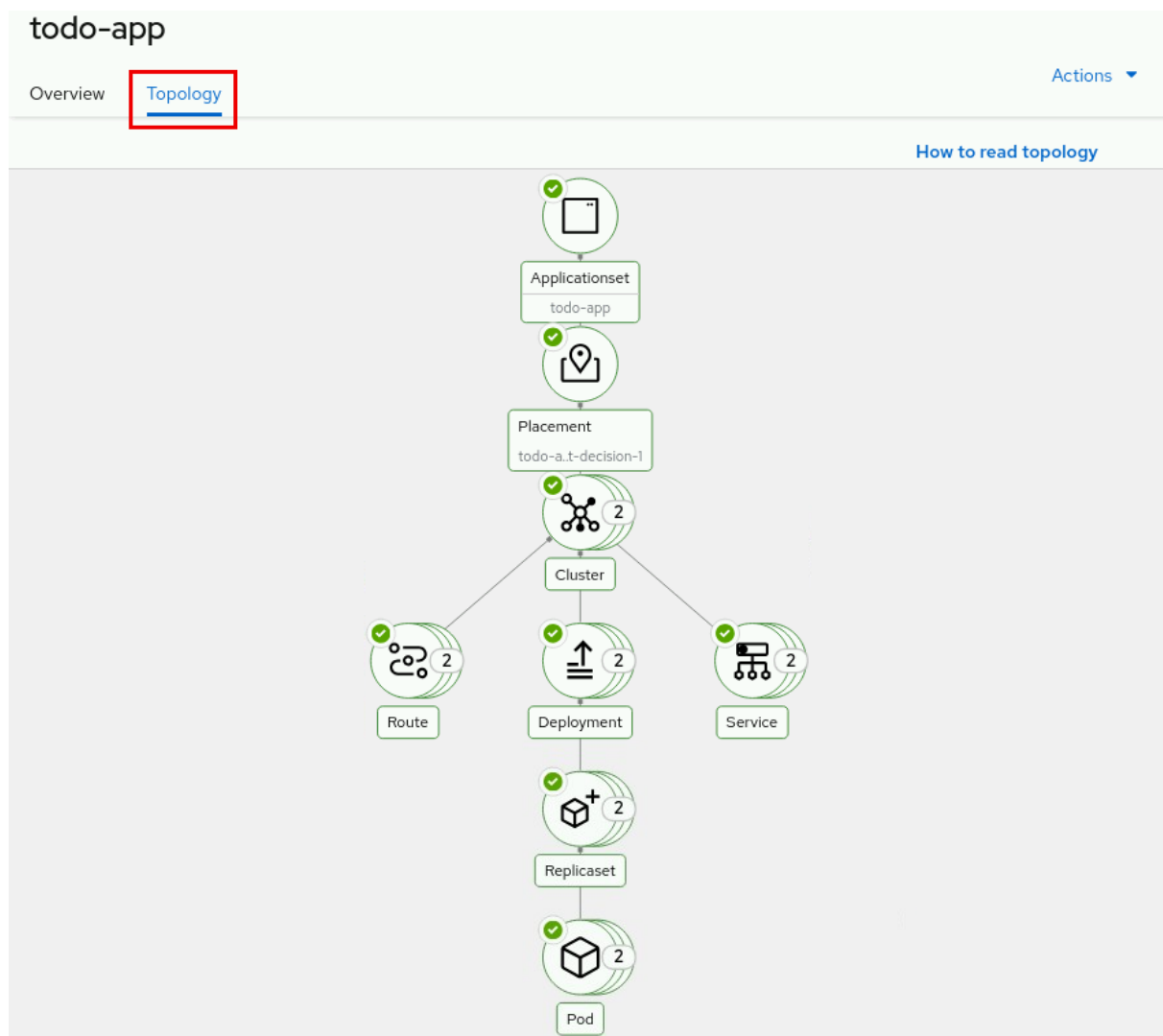
Back

Cancel

- 4.6. Review your configuration and click **Submit**.

5. Verify that the task list application runs in the two clusters.

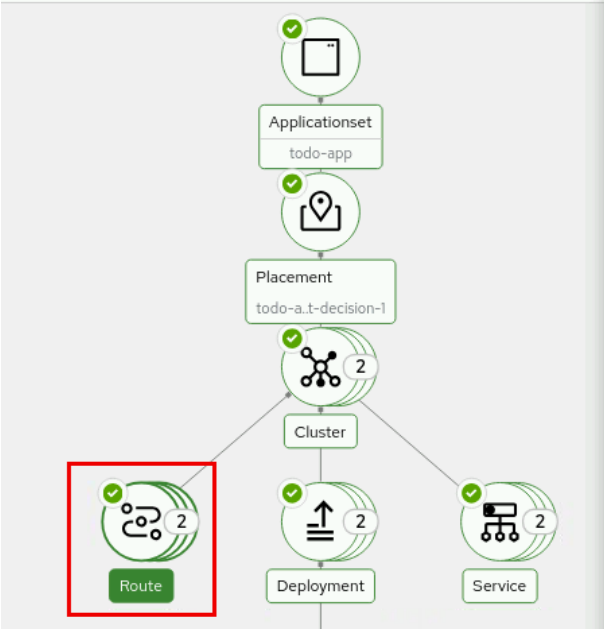
- 5.1. Go to **Topology** and wait up to three minutes for the topology to report the application status and for the application to be ready.



- 5.2. Verify that the application is available from the managed cluster. Select the Route icon and click todo-app for the managed cluster.

todo-app

Overview Topology



Route

Name	Namespace	Cl...
✓ todo-app	todo-app	local-cluster
✓ todo-app	todo-app	managed-cluster

1 - 2 of 2

1 of 1

5.3. Click **Launch Route URL**.

< Back to all route resources

Route

todo-app

Launch resource in Search

Details

YAML

Type: Route

Namespace: todo-app

Labels: app=todo-app,app.kubernetes.io/instance=todo-app-managed-cluster; env=production

Cluster deploy status

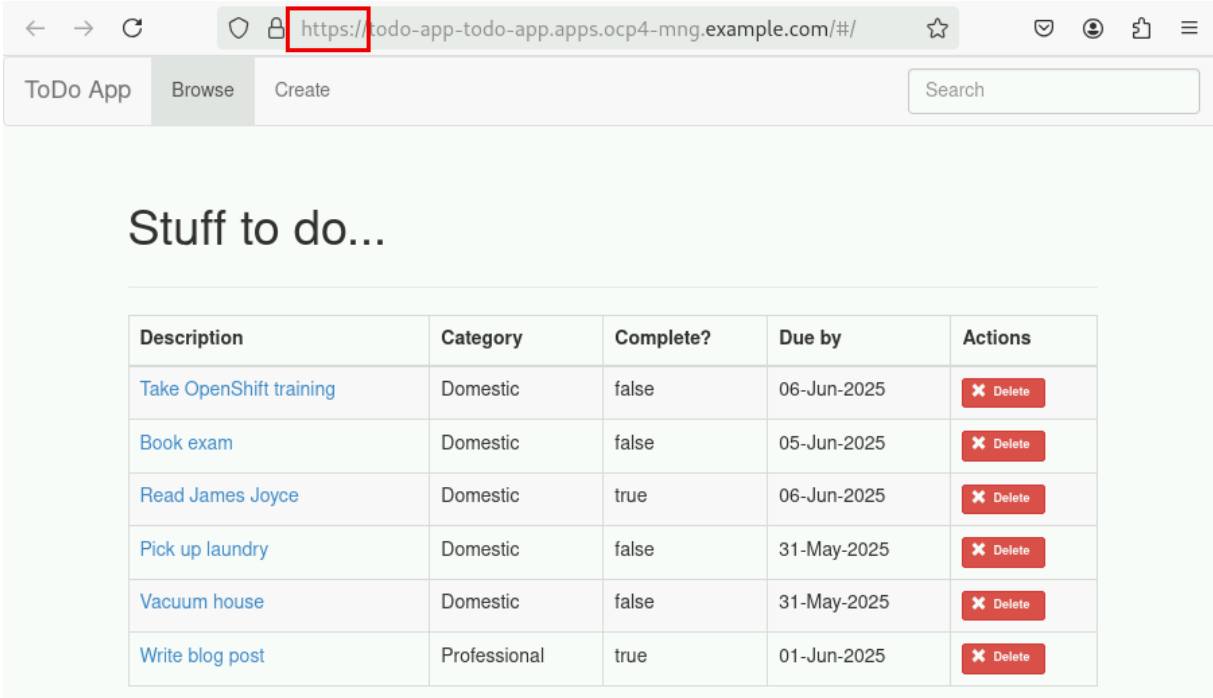
Cluster name: managed-cluster

✓ todo-app: Deployed

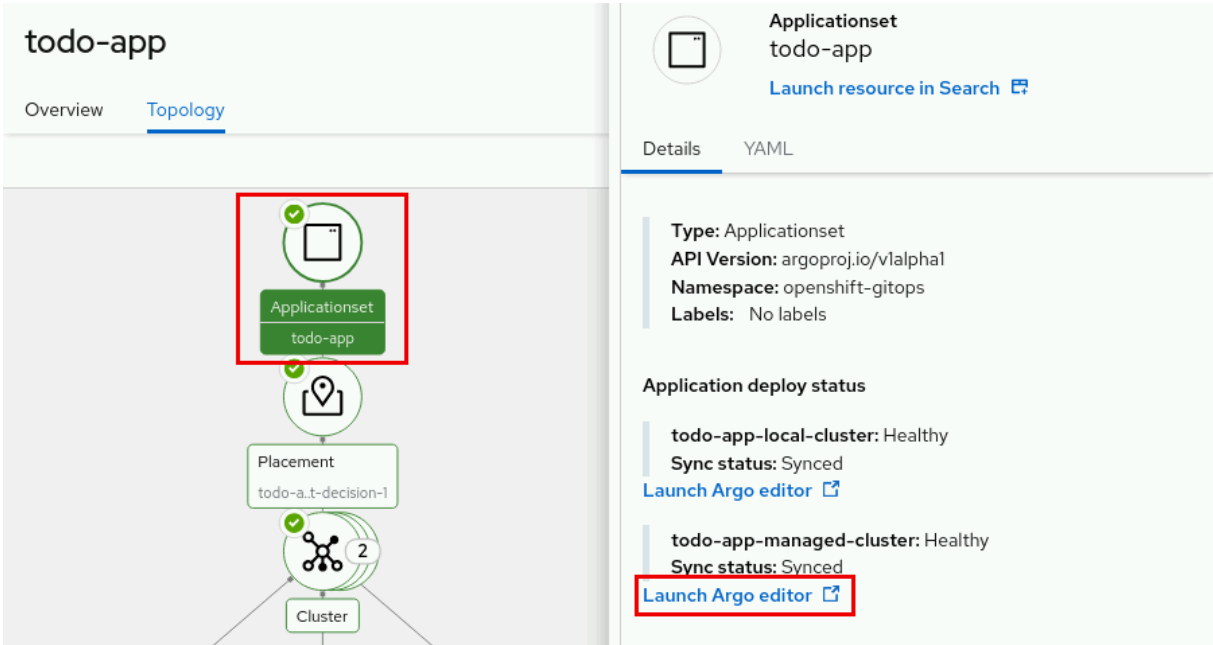
Launch Route URL

View resource YAML

The application opens in a new browser tab. The application uses HTTPS, which indicates that Kustomize created the application for the production environment. Close the browser tab.

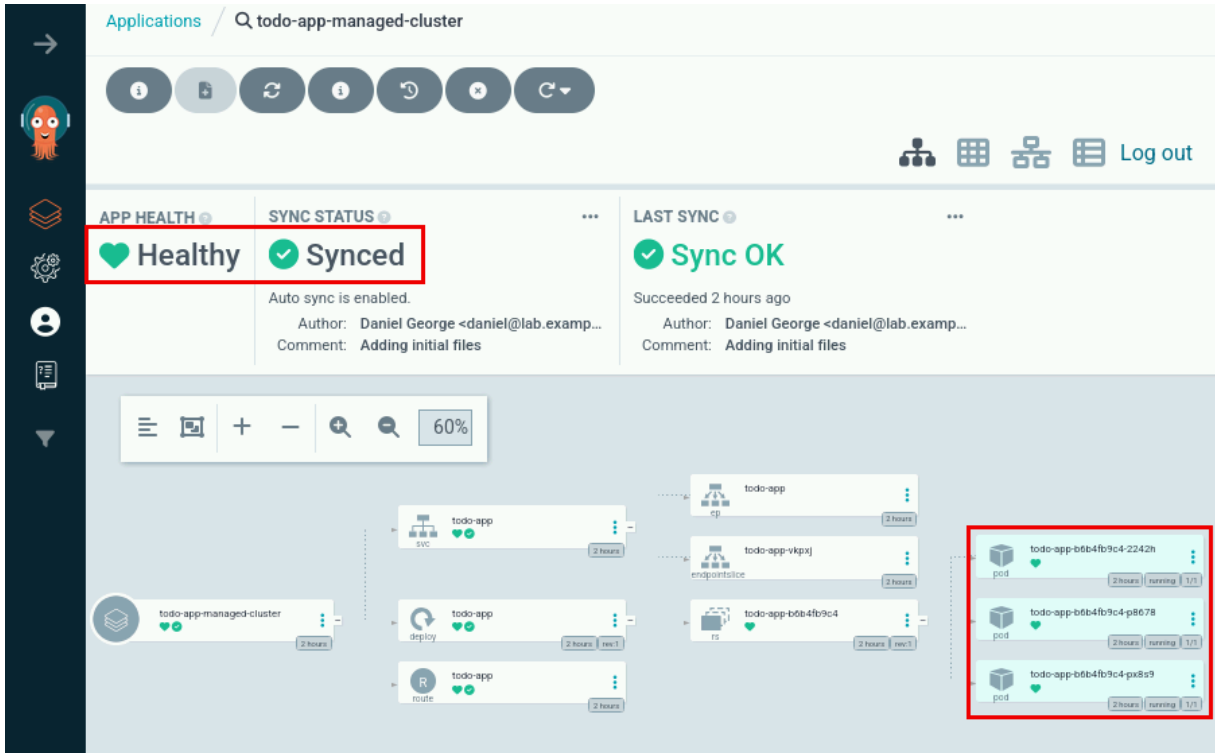


5.4. From the Argo CD web console, review the status of the application that is running on the managed cluster. Select the Applicationset icon and click Launch Argo editor for the managed cluster.



5.5. If you are not already authenticated with Argo CD, then click **Log in via OpenShift**. Select the **Red Hat Identity Management** identity provider and log in as the admin user with redhatocp as the password. Click **Allow selected permissions**.

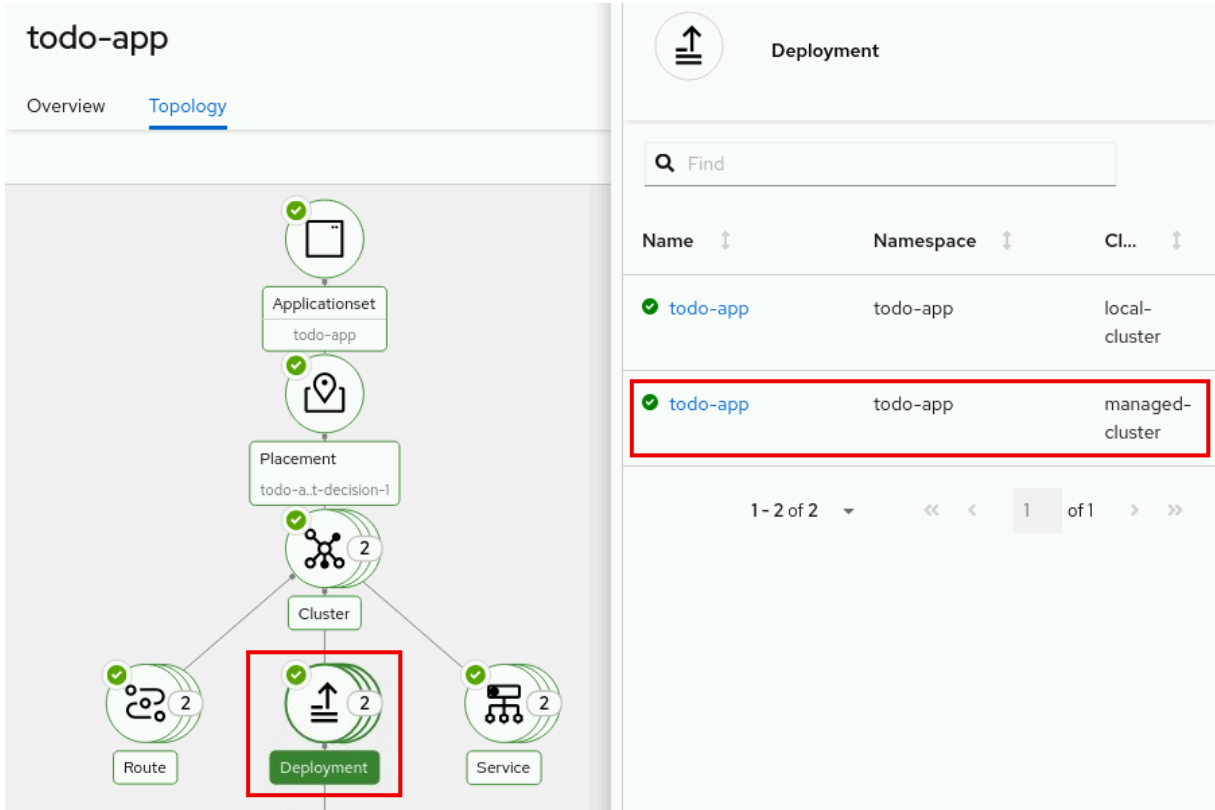
5.6. The application is healthy and synchronized. All the resources are healthy, and three replicas are running.



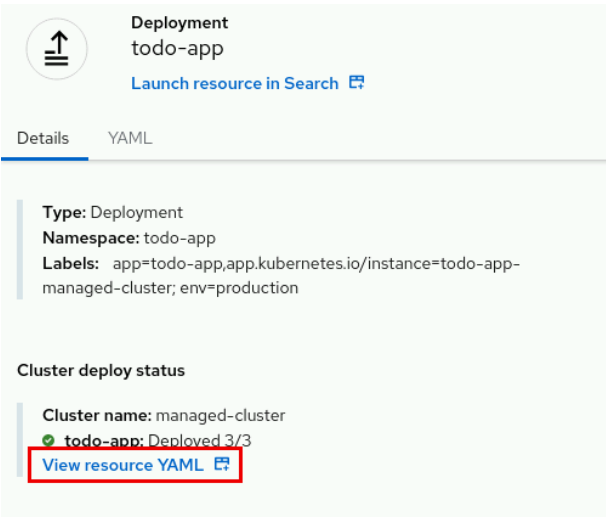
Do not close this Argo CD browser tab.

Optionally, redo the steps to verify the status of the task list application that is running in the local cluster.

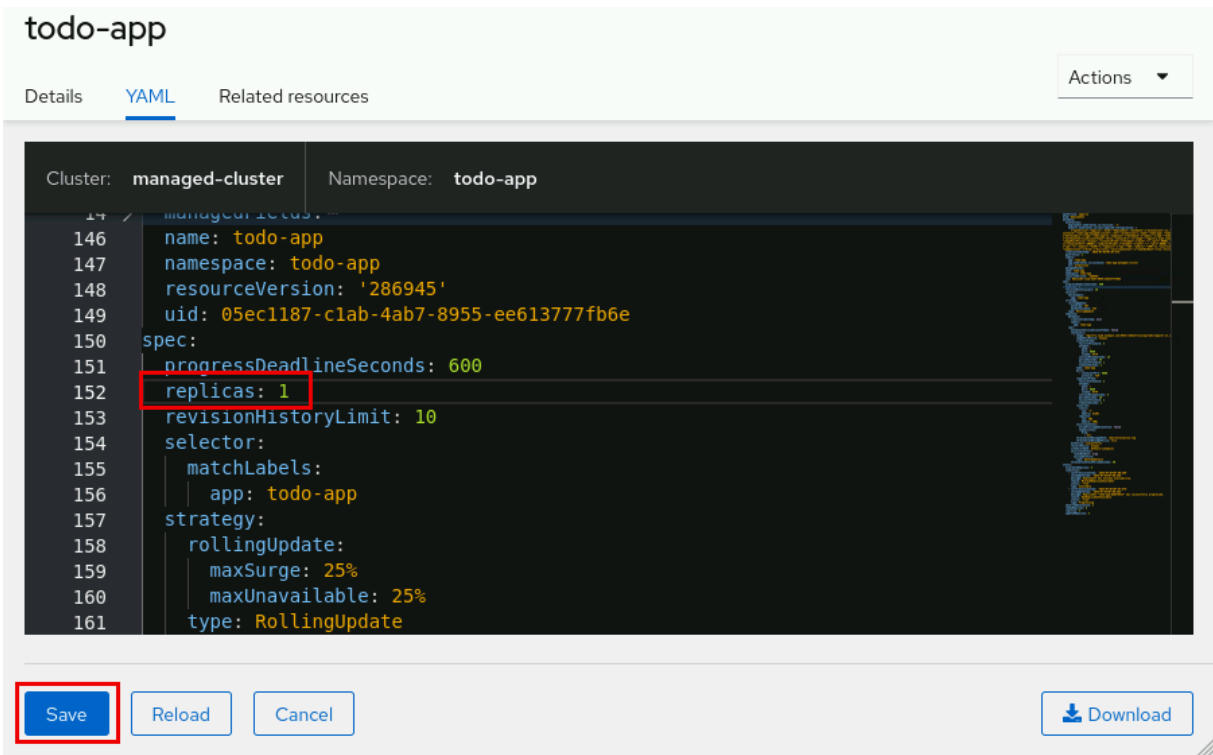
- 6. Update the deployment resource and set the number of replicas to one.
 - 6.1. Switch back to the browser tab with the RHACM web console. Select the Deployment icon and click todo-app for the managed cluster.



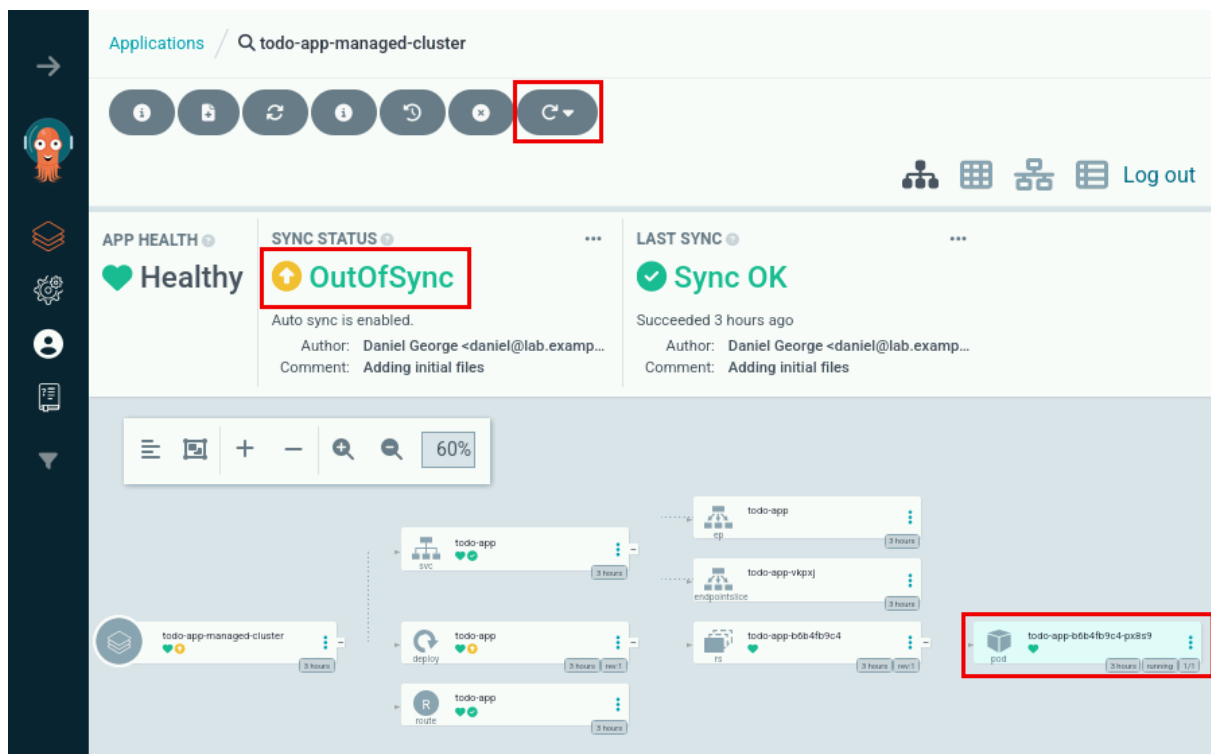
- 6.2. Click **View resource YAML** to edit the deployment resource. The YAML editor opens in a new browser tab.



6.3. Set the number of replicas to one and click **Save**. Close the browser tab.



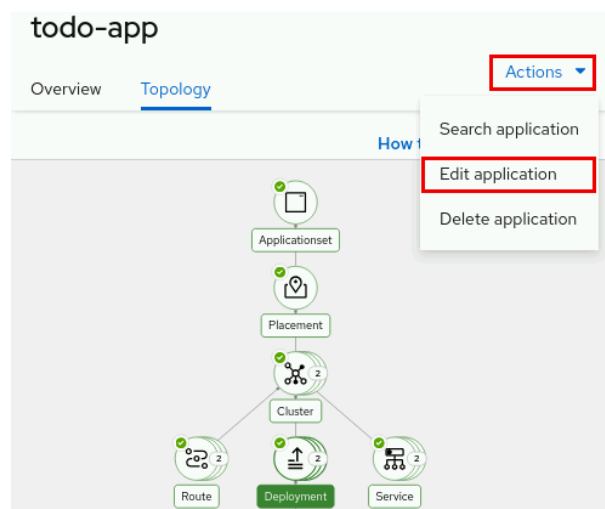
6.4. Switch to the Argo CD web console and click **Refresh**. The application is out of sync, because the deployment resource no longer matches the resource from the Git repository. Only one pod is running.



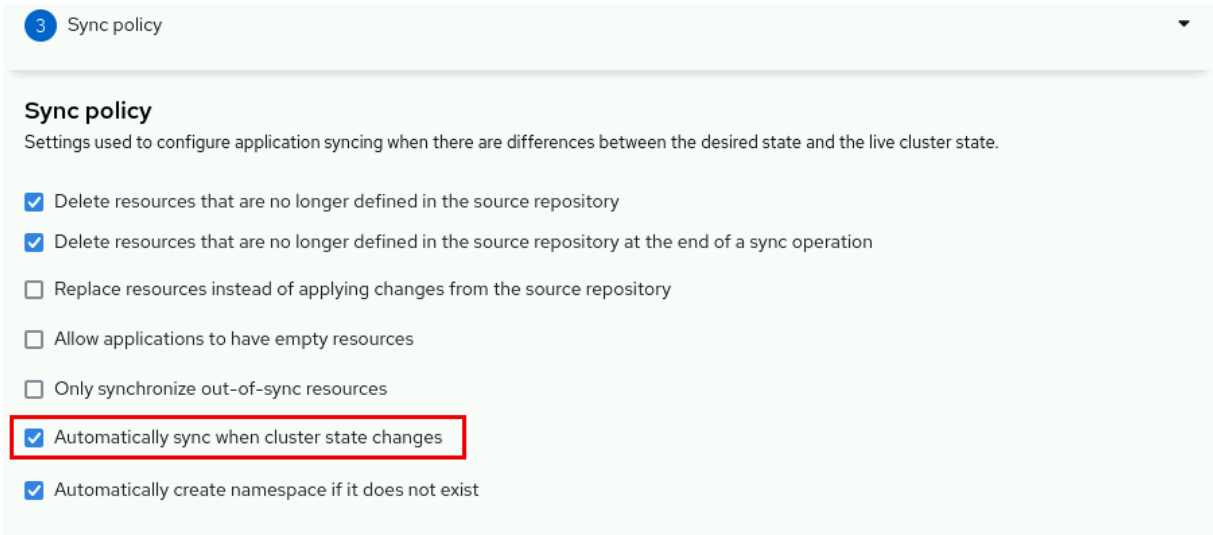
Do not close this Argo CD browser tab.

7. With GitOps, use Git for all modifications to the application. Configure RHACM to force reverting your changes from OpenShift.

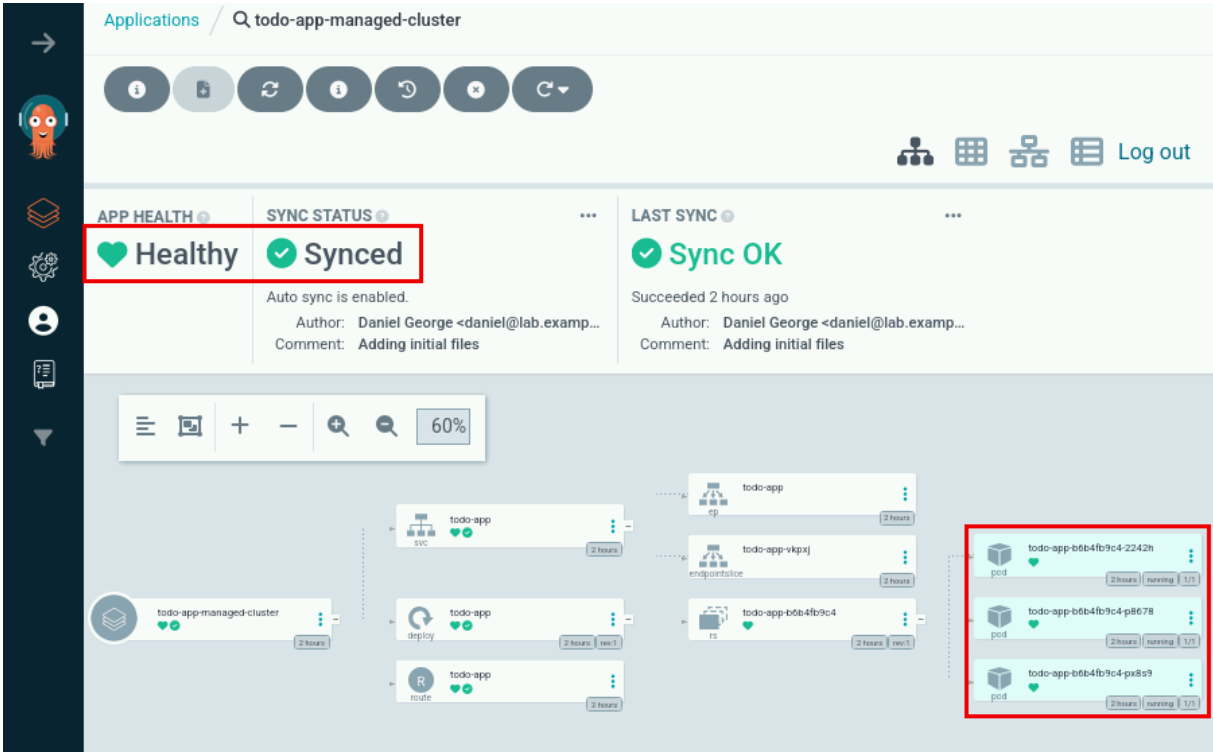
7.1. Switch back to the browser tab with the RHACM web console. Go to **Actions > Edit application**.



7.2. Go to step 3 of the form, **Sync policy**, and select **Automatically sync when cluster state changes**.



- 7.3.** Go to step 5 of the form, **Review**, and click **Submit**.
- 7.4.** Switch to the Argo CD web console and click **Refresh**. Argo CD reverted your change to the deployment resource. The application is healthy and synchronized, and three pods are running again. Close the Argo CD browser tab.



Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish gitops-deploy
```

5.7. Troubleshooting Common Issues for Application Lifecycle Management in RHACM

Objectives

- Investigate and resolve common issues when deploying and managing applications on the managed clusters.

Troubleshooting RHACM Applications

The topology view in the RHACM web console is a good starting point when troubleshooting applications. The view reports the status of your application and aggregates the error messages.

When an application component fails, click the component icon to access the error message, the YAML manifest, and other details.

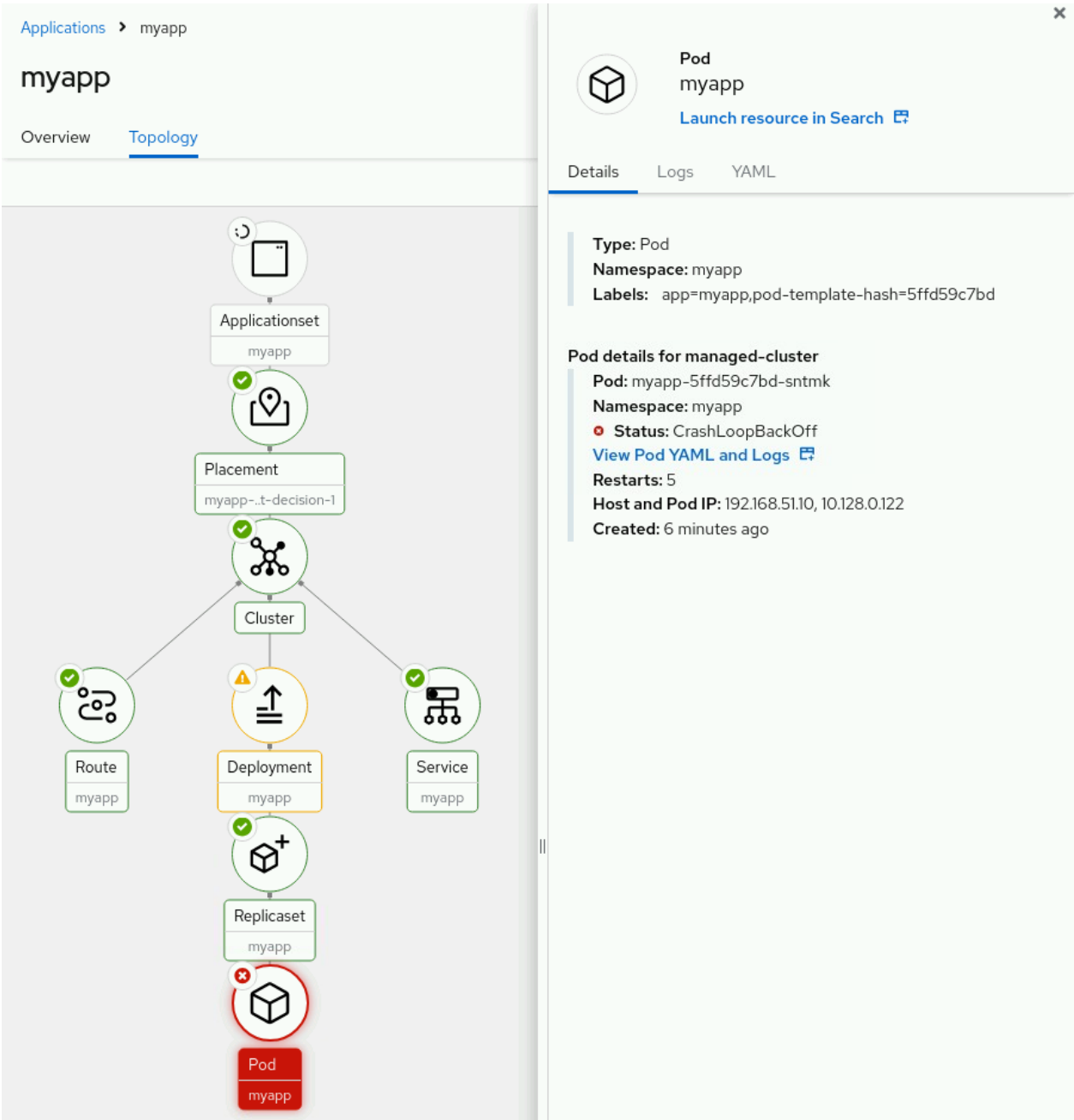


Figure 32. Topology view of an application with errors

The topology view might not report all application issues. In this case, use the Argo CD web console to get a more precise view of the application status.

For the Push model, use the web console of the Argo CD instance that is running in the hub cluster. For the Pull model, use the web console of the Argo CD instance that is running in the managed cluster that hosts the application resources.

Consult the concluding references section for Red Hat documentation about troubleshooting RHACM applications.

If you still cannot resolve your issue, then use the `must-gather` command to collect data about your clusters, and contact the Red Hat support team at <https://access.redhat.com/support>

The following sections present common issues with applications in RHACM.

Troubleshooting Applications That Fail to Deploy

Often, applications fail to deploy because of issues in the `ApplicationSet` or the `Placement` resource.

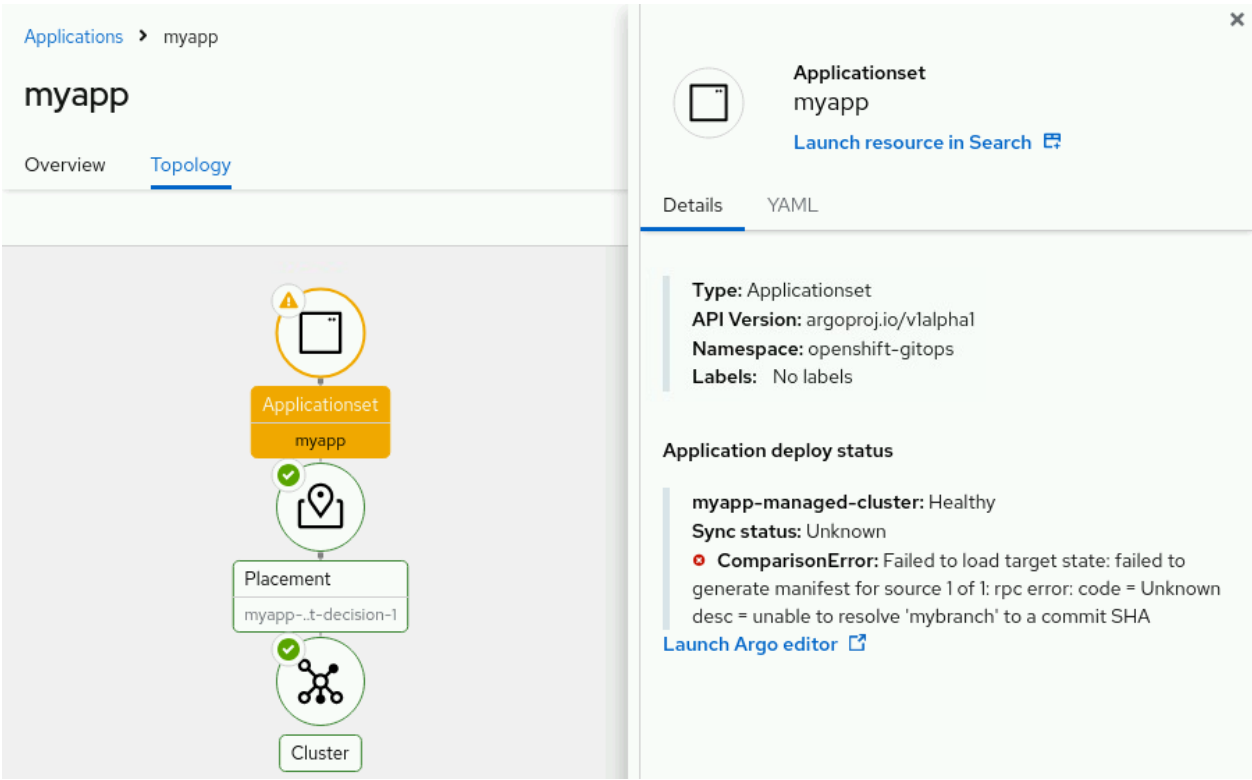


Figure 33. Wrong Git branch in the application set resource

Fix these issues by reviewing the ApplicationSet resource. Specifically, ensure the validity of the Git repository.

If you use a private Git repository that requires authentication, then ensure that you configured Argo CD for that repository. Also, verify in your Git system that the username, password, or token did not expire.

The references section links to the Argo CD documentation about configuring private Git repositories.

The issue might also come from the Placement resource that targets the wrong managed clusters. Review the clusters' labels and verify that the Placement resource refers to the correct labels.

Troubleshooting Unsynchronized Applications

When you update the application resources directly on the cluster, without using Git, Argo CD reverts your change by using the manifests from the Git repository.

An OutOfSync status in the Argo CD web console indicates that Argo CD detected the change but did not revert it.

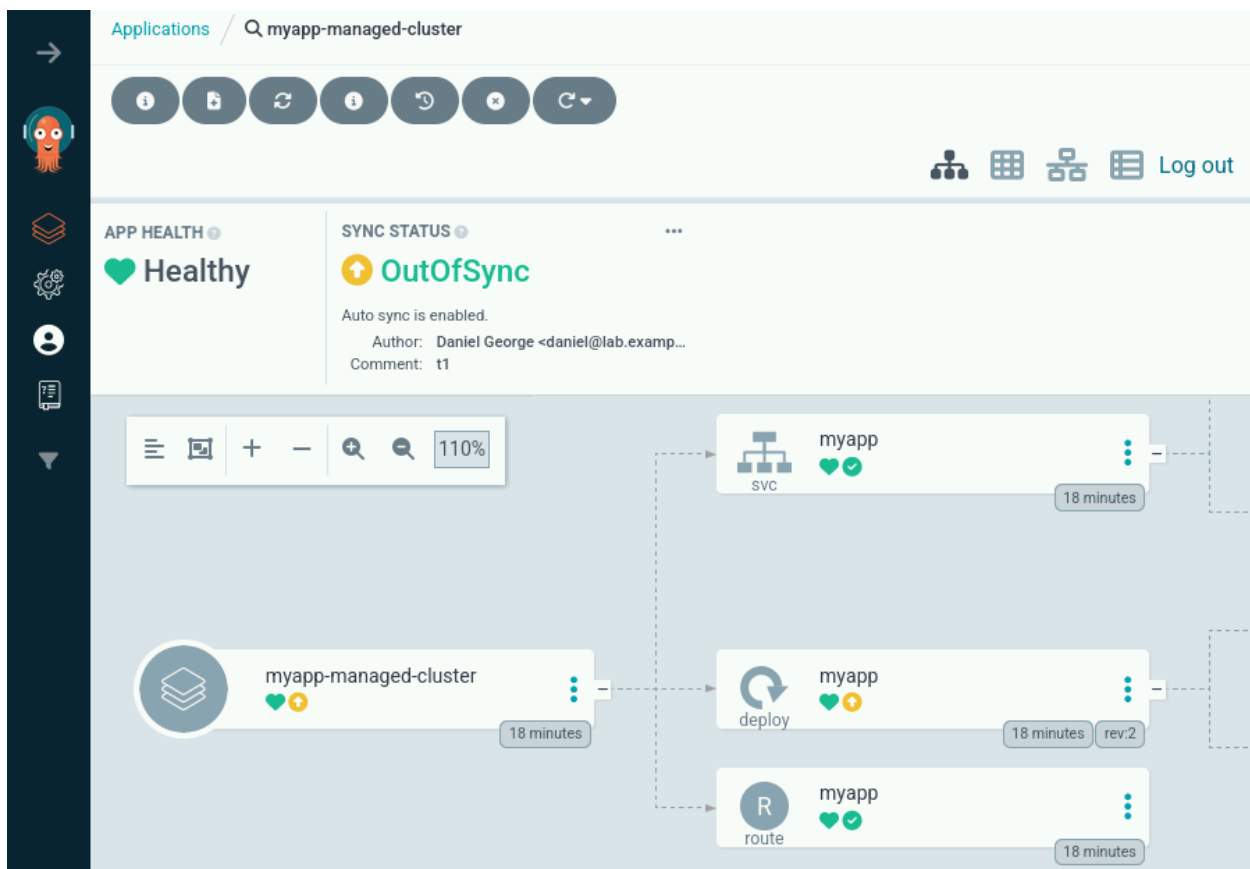


Figure 34. Unsynchronized application

If this behavior is not expected, then use the RHACM web console to edit the application and select **Automatically sync when cluster state changes** in the **Sync policy** section.

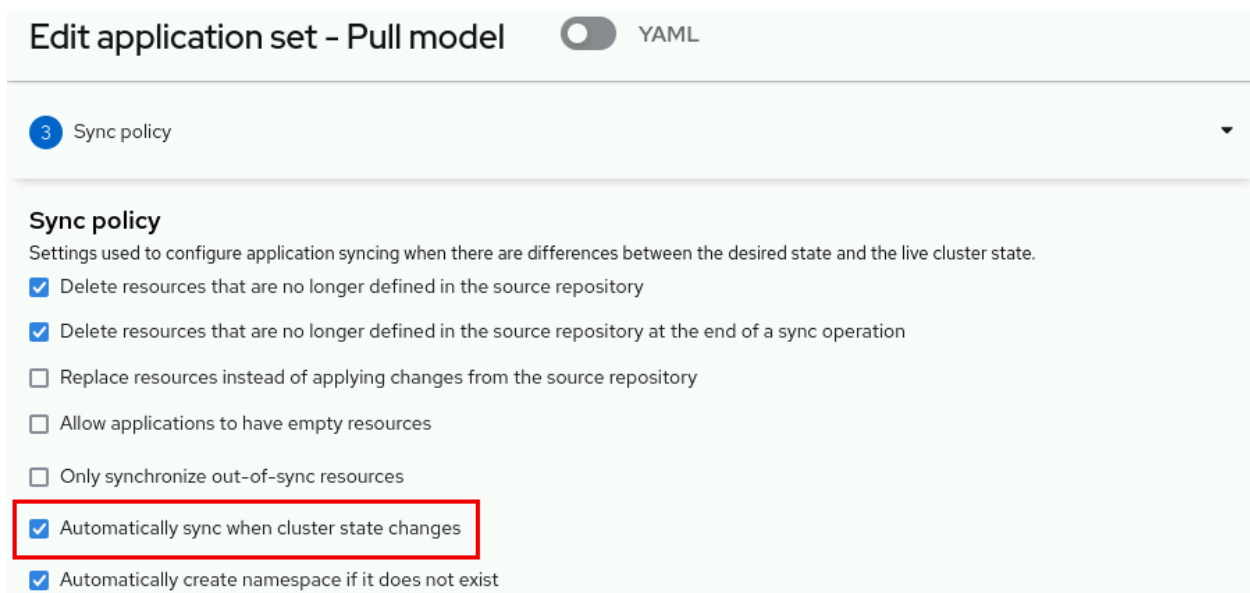


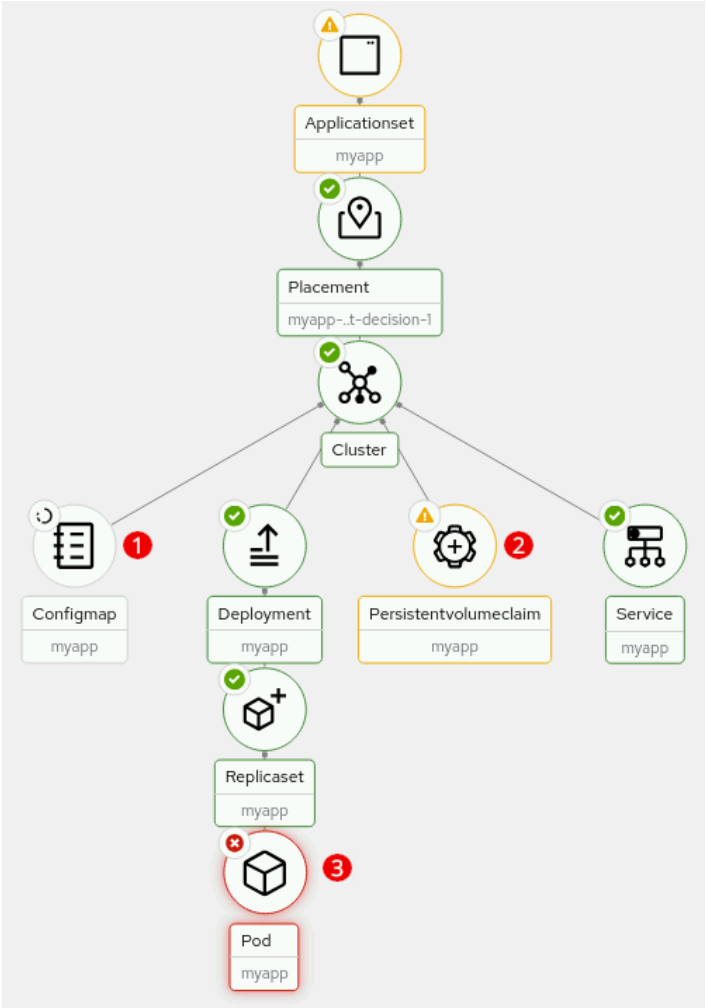
Figure 35. Selecting automatic synchronization

Alternatively, edit the ApplicationSet resource from the CLI and set the `selfHeal` parameter to `true`:

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: myapp
  namespace: openshift-gitops
spec:
  ...output omitted...
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
  syncOptions:
    - CreateNamespace=true
    - PruneLast=true
```

Troubleshooting Dysfunctional Applications

After Argo CD deploys your application, the topology view might report different error statuses for the resources. These issues usually come from errors in the application manifests that you must fix in the application's Git repository.



- 1 The Not deployed or OutOfSync status indicates that OpenShift rejected the resource. This status often indicates a syntax error in the resource, such as the presence of a field that does not exist in the custom resource definition (CRD). Select the application set icon to get more information about the issue.
- 2 The Pending status indicates that OpenShift accepted the resource and is waiting for the control loop to reconcile the current state with the intended state. Examples of Pending status include a persistent volume claim (PVC) that refers to a nonexistent storage class, or a deployment resource that is waiting for its pods to be ready.
- 3 The Error status indicates a failing resource, such as a pod with failing readiness or liveness probes.

OpenShift GitOps Issues

If the OpenShift GitOps operator is incorrectly integrated with RHACM, then application deployments might fail. For example, for Argo CD to pull from private Git repositories, you must add your company's private certificate authority (CA) to the Argo CD bundle.

Review the *Configuring GitOps for Application Lifecycle Management in RHACM* section, which describes the integration process.

When using the Push model, the Argo CD instance that is running in the hub cluster manages all the GitOps applications. With large installations, the instance might fail because of insufficient resources. In that situation, increase the resources that the cluster allocates to the instance. Consult the documentation in the references section about configuring Argo CD with more resources.

Collecting Data and Opening Support Cases

If you cannot fix your issue, then collect and send data about your clusters to the Red Hat support team for help:

- Collect data from the RHACM hub cluster by running the `oc adm must-gather` command. Use the `--image` option to provide the plug-in image that the `must-gather` command uses to collect RHACM data. Use the `--dest-dir` option to specify the directory that the command creates to store the data.

```
[user@host ~]$ oc adm must-gather \
--image=registry.redhat.io/rhacm2/acm-must-gather-rhel9:v2.13 \
--dest-dir=./must-gather-rhacm
...output omitted...
```

- Retrieve the identifier of your hub cluster. You need this information to create the archive that you send to the support team.

```
[user@host ~]$ oc get clusterversion \
-o jsonpath='{.items[].spec.clusterID}'
2e52...9c5b
```

- Create an archive of the data directory. The name of the archive must include the current date and time, and the cluster identifier.

```
[user@host ~]$ date +"%m-%d-%Y-%H-%M-%S"
05-25-2025-12-45-36
[user@host ~]$ tar cvaf \
must-gather-05-25-2025-12-45-36-2e52...9c5b.tar.gz \
./must-gather-rhacm
...output omitted...
```

- Open a support case at <https://access.redhat.com/support>, and attach the archive.

References

Knowledgebase: "ACM Application Is Not Getting Deployed on the Managed Clusters and It Is Showing "Cluster Is Offline" on the Application Topology Page"

For more information about troubleshooting applications, refer to the *Troubleshooting Application Kubernetes Deployment Version* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13*

Troubleshooting documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/troubleshooting/index#troubleshooting-application-kubernetes-deployment-version

For more information about adjusting resources for Argo CD, refer to the *Configuring Resource Quota or Requests* chapter in the *Red Hat OpenShift GitOps 1.16 Managing Resource Use* documentation at

https://docs.redhat.com/en/documentation/red_hat_openshift_gitops/1.16/html-single/managing_resource_use/index#configuring-resource-quota

For more information about the `must-gather` command, refer to the *Running the must-gather Command to Troubleshoot Red Hat Advanced Cluster Management Components* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Troubleshooting* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/troubleshooting/index#running-the-must-gather-command-to-troubleshoot

Argo CD: Private Repositories

5.8. Guided Exercise

Troubleshoot Common Issues for Application Lifecycle Management in RHACM

Investigate and resolve common issues when deploying and managing applications on the managed clusters.

Outcomes

- Review the status of applications from the topology view.
- Diagnose application issues.
- Fix ApplicationSet and Placement resources.
- Fix application resources from Git.

Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab install do0015l ; lab start gitops-troubleshoot
```

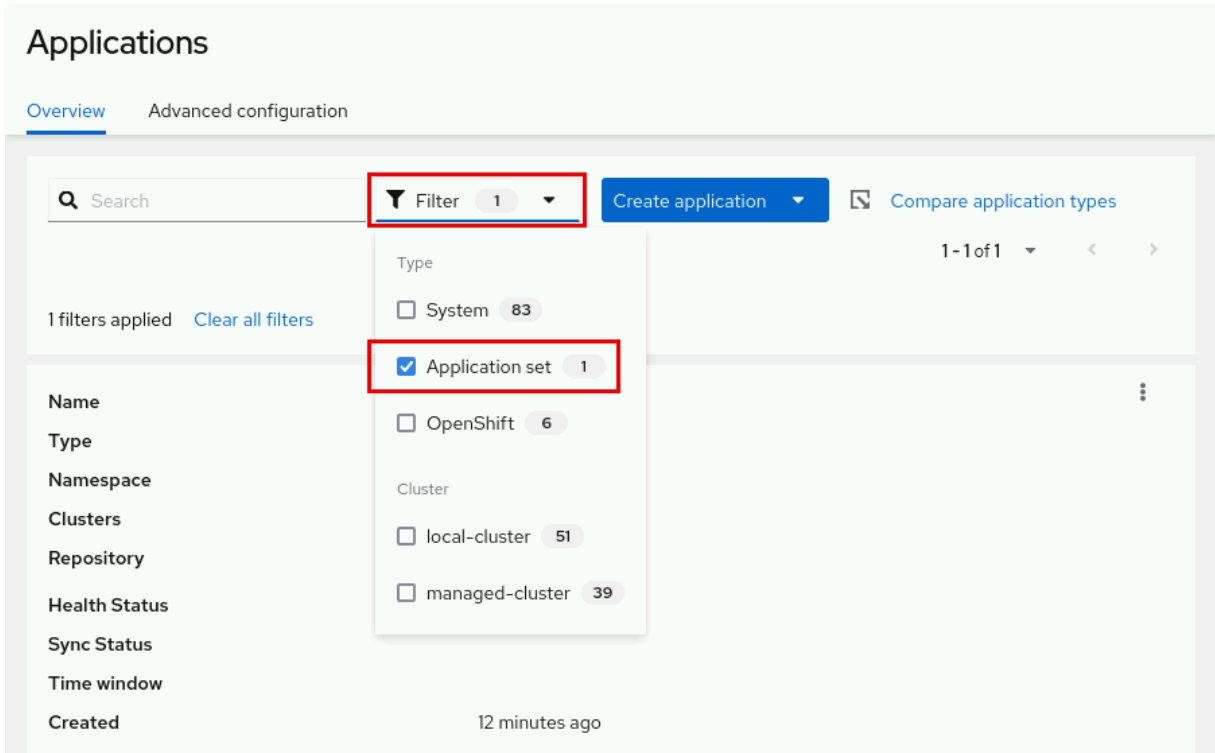
Instructions

The `lab` command initiated the deployment of a task-list application in your managed cluster by using the GitOps Pull model.

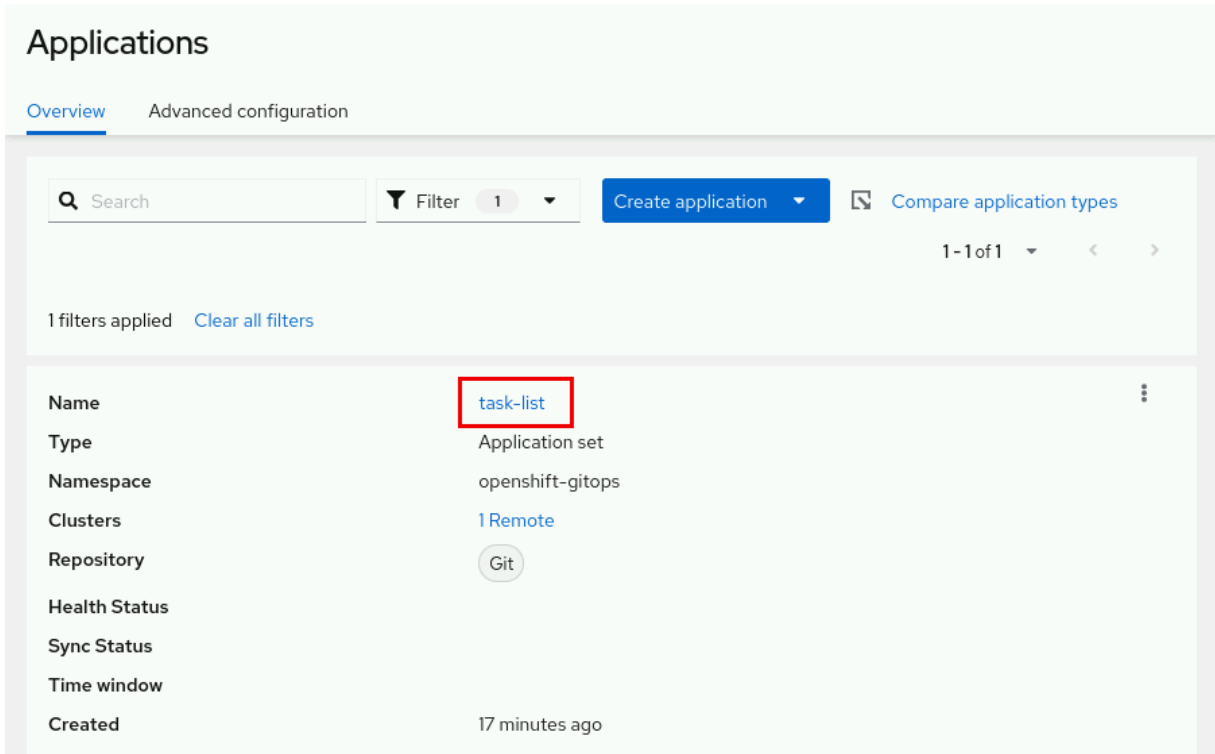
The application's manifests are available in the `main` branch of the `https://git.ocp4.example.com/do0015l/gitops-troubleshoot.git` Git repository. The path to the Kustomize overlay is `kustomize/overlays/dev`.

Diagnose and fix the application, which fails to deploy.

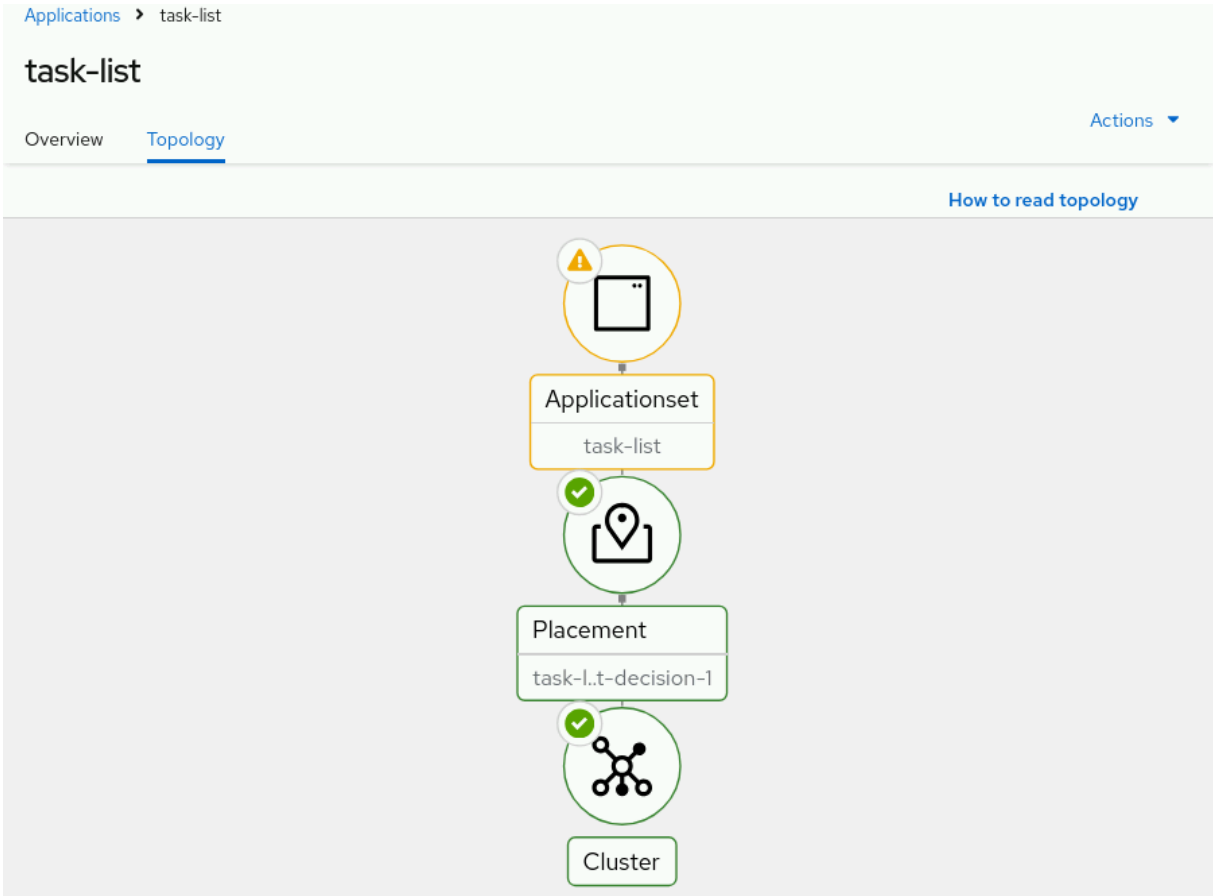
1. Log in to the RHOC hub cluster, switch to the RHACM web console, and review the application status from the topology view.
 - 1.1. Open a web browser and go to `https://console-openshift-console.apps.ocp4.example.com`
 - 1.2. Select the **Red Hat Identity Management** identity provider and log in as the `admin` user with `redhatocp` as the password.
 - 1.3. Ensure that the cluster switcher is set to **All Clusters**, which corresponds to the RHACM web console.
 - 1.4. Go to **Applications**, and activate the `Application` set filter to locate the application by its name.



Click the `task-list` application.



- 1.5. Click the **Topology** tab. The application is not deployed. You might have to wait up to five minutes for the application set to display the warning indicator.



1.6. Click the application set icon to review the warning messages.

task-list

OverviewTopology

Applicationset

task-list

Placement

task-l..t-decision-1

Cluster

Applicationset

task-list

Launch resource in Search

DetailsYAML

Type: Applicationset

API Version: argoproj.io/v1alpha1

Namespace: openshift-gitops

Labels: No labels

Warning

The ArgoCD pull model does not support the hub cluster as a destination cluster. Filter out the hub cluster from the placement resource.

Application deploy status

task-list-managed-cluster: Healthy

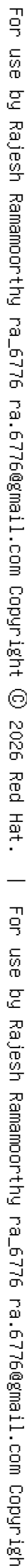
Sync status: Unknown

ComparisonError

Failed to load target state: failed to generate manifest for source 1 of 1: rpc error: code = Unknown desc = Manifest generation error (cached): DEV: app path does not exist

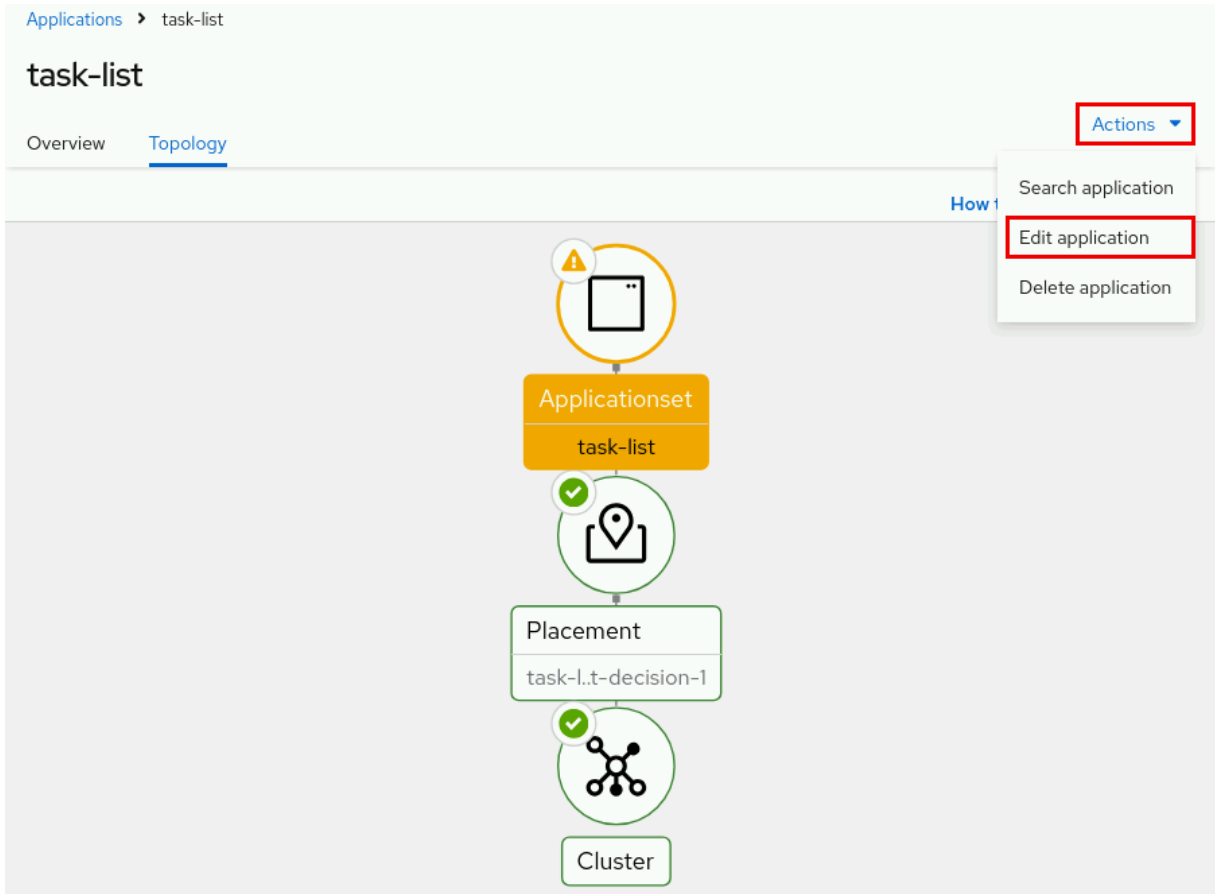
Launch Argo editor

2. Take a few minutes to investigate the first issue regarding the Pull model. When done, continue with the exercise to verify your discovery and to fix the issue.

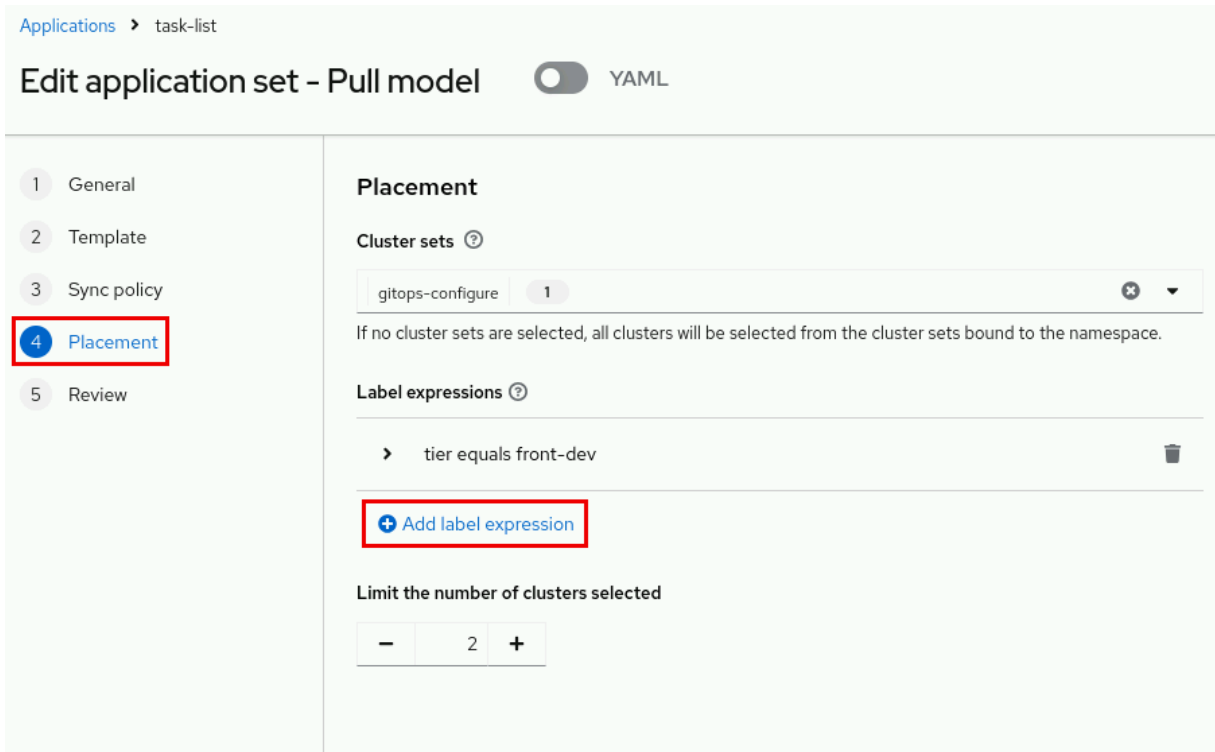


- For use by Rajesh Ramamorthy ra-6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra-6776 ra.6776@gmail.com Copyright

For use by Rajesh Ramamorthy ra-6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra-6776 ra.6776@gmail.com Copyright



2.2. Go to **Placement** and click **Add label expression** for declaring a rule to exclude the hub cluster.



2.3. Exclude the hub cluster, which is named local-cluster. Set **Label** to name, set **Operator** to does not equal any of, and set **Values** to local-cluster. Because you deploy the application on one cluster only, set **Limit the number of clusters selected** to 1.

Placement

Cluster sets ⓘ

gitops-configure 1

If no cluster sets are selected, all clusters will be selected from the cluster sets bound to the namespace.

Label expressions ⓘ

tier equals front-dev

name does not equal local-cluster

Label *

Operator *

Values *

name

does not equal any of

local-cluster 1

+ Add label expression

Limit the number of clusters selected

- 1 +

2.4. Go to **Review**, verify the new placement expression, and click **Submit**.

Applications > task-list

Edit application set - Pull model ☐ YAML

1 General

2 Template

3 Sync policy

4 Placement

5 Review

☒ Automatically sync when cluster state changes

☒ Automatically create namespace if it does not exist

Placement

Cluster sets gitops-configure

Label expressions

tier equals front-dev

name does not equal local-cluster

Limit the number of clusters selected 1

Submit

Back

Cancel

2.5. Go to **Topology**. Click the application set icon to verify that the warning message about the placement issue is no longer shown. It might take up to two minutes for the message to disappear.

You fix the remaining issue in the next step.

3. Take a few minutes to investigate the remaining issue. When done, continue with the exercise to verify your discovery and to fix the issue.

task-list

OverviewTopology

Applicationset

task-list

Placement

task-l.t-decision-1

Cluster

Applicationset

task-list

Launch resource in Search

DetailsYAML

Type: Applicationset

API Version: argoproj.io/v1alpha1

Namespace: openshift-gitops

Labels: No labels

Application deploy status

task-list-managed-cluster: Healthy

Sync status: Unknown

ComparisonError: Failed to load target state: failed to generate manifest for source 1 of 1: rpc error: code = Unknown desc = Manifest generation error (cached): DEV: app path does not exist

Launch Argo editor

- 3.1. The message suggests that the path to the application is wrong. Go to **Actions > Edit application** to verify the application Git repository.
- 3.2. Go to **Template**. The path to the Kustomize overlay is wrong.

Applications > task-list

Edit application set - Pull model

YAML

1 General

2 Template

3 Sync policy

4 Placement

5 Review

URL * ?

https://git.ocp4.example.com/do0015/gitops-troubleshoot.git

Revision ?

main

Path ?

DEV

+ Add another repository

NextBackCancel

Set the path to kustomize/overlays/dev.

Applications > task-list

Edit application set - Pull model

☒ YAML

1 General

2 Template

3 Sync policy

4 Placement

5 Review

URL * ?

https://git.ocp4.example.com/do0015/gitops-troubleshoot.git

Revision ?

main

Path ?

kustomize/overlays/dev

+ Add another repository

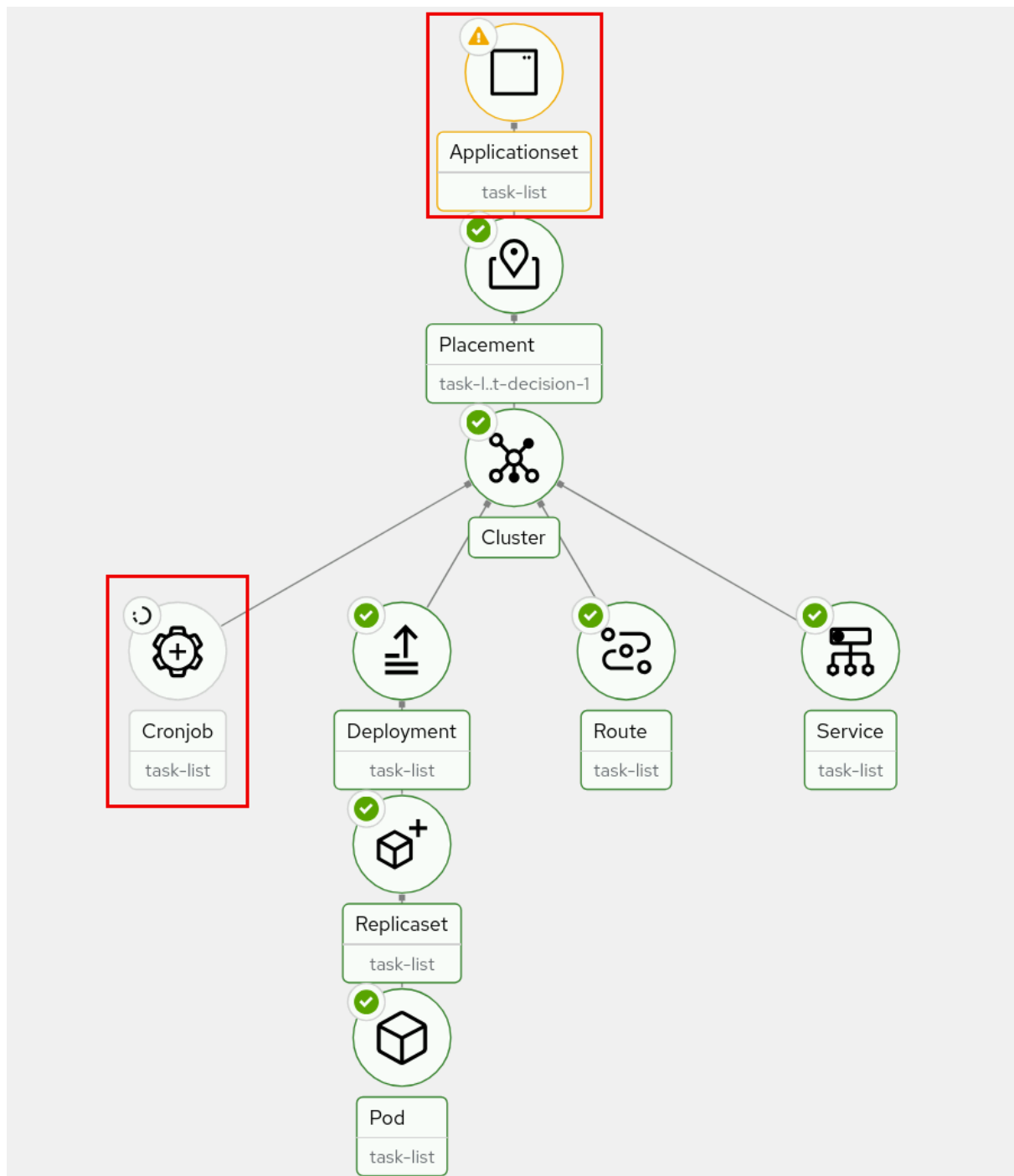
Next

Back

Cancel

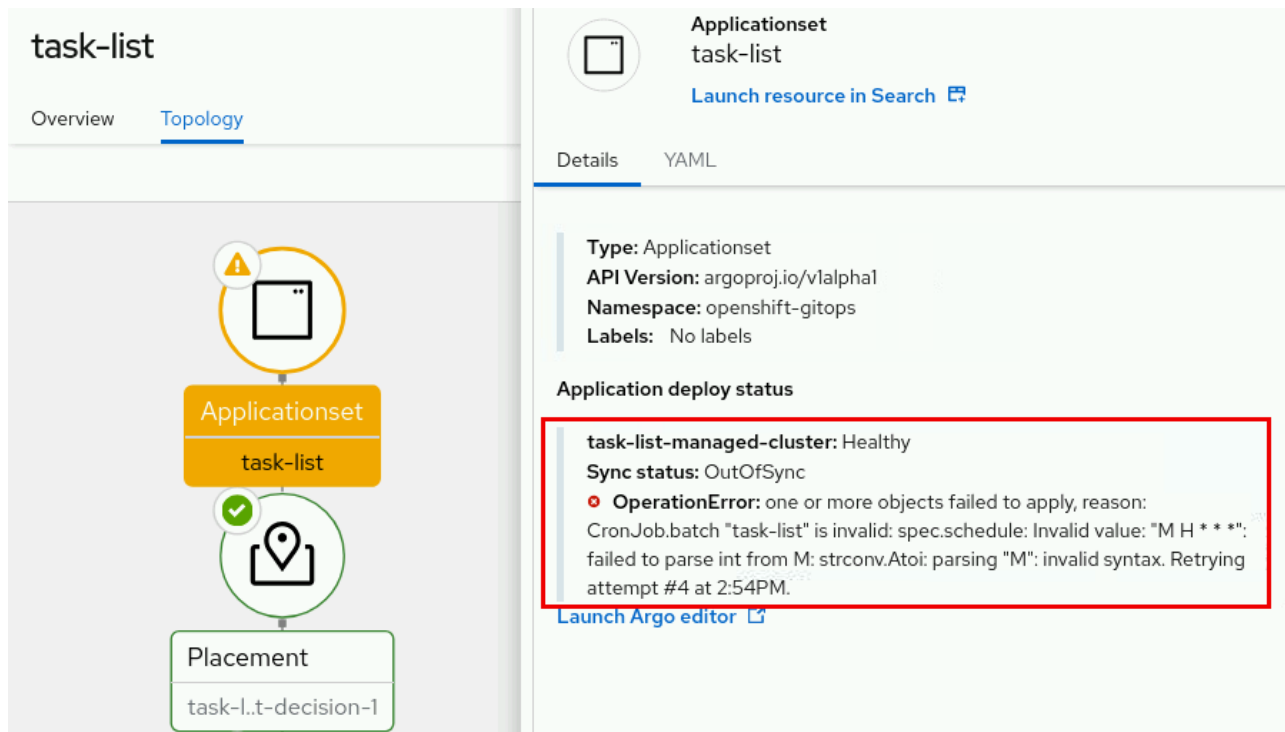
3.3. Go to **Review** and click **Submit**.

3.4. Go to **Topology** and wait up to three minutes for the topology to report the application status. The application is deployed, except for a CronJob resource. The application set still has a warning indicator.



Click the application set icon to access the message.

- Take a few minutes to investigate the issue with the CronJob resource. When done, continue with the exercise to verify your discovery and to fix the issue.



- 4.1. The error message indicates that the schedule parameter of the CronJob resource is invalid.

Follow the GitOps principles by fixing the resource manifest in the Git repository, which is the source of truth. Do not edit the resource directly from OpenShift.

In a terminal window, clone the application from the classroom GitLab system. The application is available in the <https://git.ocp4.example.com/do0015l/gitops-troubleshoot.git> repository.

```
[student@workstation ~]$ git clone \
  https://git.ocp4.example.com/do0015l/gitops-troubleshoot.git
Cloning into 'gitops-troubleshoot'...
...output omitted...
```

- 4.2. Change to the gitops-troubleshoot directory.

```
[student@workstation ~]$ cd gitops-troubleshoot
```

- 4.3. List the manifest files in the kustomize/overlays/dev directory, which corresponds to the path that you fixed in the RHACM application. The cronjob.yaml file probably has the issue.

```
[student@workstation gitops-troubleshoot]$ ls kustomize/overlays/dev
cronjob.yaml kustomization.yaml
```

- 4.4. Edit the kustomize/overlays/dev/cronjob.yaml file. The schedule parameter has an invalid syntax. Fix the parameter by configuring the job to start at 3 AM.

```
---
apiVersion: batch/v1
kind: CronJob
metadata:
  labels:
    app: task-list
    name: task-list
spec:
  schedule: "0 3 * * *"
  jobTemplate:
    ...output omitted...
```

- 4.5.** Add the file to the index, commit your change, and push the commit to the remote Git repository.

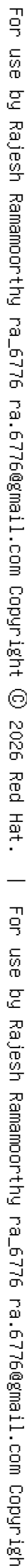
```
[student@workstation gitops-troubleshoot]$ git add \
  kustomize/overlays/dev/cronjob.yaml
[student@workstation gitops-troubleshoot]$ git commit -m "Fix cron job"
[main 8fc7b85] Fix cron job
1 file changed, 1 insertion(+), 1 deletion(-)
[student@workstation gitops-troubleshoot]$ git push
...output omitted...
```

- 4.6.** Change to the /home/student directory.

```
[student@workstation gitops-troubleshoot]$ cd
```

- 4.7.** Switch back to the RHACM web console. Go to **Applications**, select the task-list application, and go to **Topology**.

Wait up to three minutes for the topology to report the application status as successful.



For use by Rajesh Ramamorthy ra-6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra-6776 ra.6776@gmail.com Copyright

For use by Rajesh Ramamorthy ra-6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra-6776 ra.6776@gmail.com Copyright

```
[student@workstation ~]$ lab finish gitops-troubleshoot
```


5.9. Lab

Manage Applications Across Multiple Clusters with RHACM

Deploy MySQL applications with customized settings for the managed clusters by using RHACM and GitOps.

Outcomes

- Deploy applications by using RHACM and GitOps.
- Review application statuses and troubleshoot issues.
- Use Git to manage application lifecycles.

Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab install do0015l ; lab start gitops-review
```

Instructions

In this exercise, you deploy a MySQL database on your managed cluster by using GitOps and the Pull model. Your installation is already configured with the GitOps operator, and the `lab` command created a Kustomize Git repository with the resources for creating the database.

The public Git repository is available at <https://git.ocp4.example.com/do0015l/gitops-review.git>. The main branch stores the database resources in the `kustomize/overlays/qa` path.

The classroom environment provides two RHOC clusters: an RHACM hub cluster and a managed cluster. You can access those clusters by using the following parameters:

RHACM hub cluster web console URL

<https://console-openshift-console.apps.ocp4.example.com>

RHACM hub cluster API URL

<https://api.ocp4.example.com:6443>

Managed cluster web console URL

<https://console-openshift-console.apps.ocp4-mng.example.com>

Managed cluster API URL

<https://api.ocp4-mng.example.com:6443>

Administrator username for both clusters

admin (select the **Red Hat Identity Management** identity provider)

Administrator password for both clusters

redhatocp

1. Add the `tier=backend` label to the managed cluster. You use this label in another step to select the cluster for the MySQL database.

2. Create the application by using the GitOps Pull model and verify that RHACM deploys the database. The application has an issue that you fix in the next main step.

Use the following parameters:

Parameter	Value
GitOps model	Pull model
Application set name	intranet-db
Git URL	https://git.ocp4.example.com/do0015l/gitops-review.git
Git revision	main
Git path	kustomize/overlays/qa
Remote namespace	intranet-db
Placement cluster set	gitops-configure
Managed cluster label	tier=backend

3. Investigate and fix the issue. You might have to clone the Git repository on your workstation machine to fix issues in the resource files.

Note

To clone the repository, run the following command:

```
[student@workstation ~]$ git clone \
https://git.ocp4.example.com/do0015l/gitops-review.git
```

After you edit a file, add it to the Git index, commit your change, and push the commit to the classroom GitLab system:

```
[student@workstation gitops-review]$ git add updated-file
[student@workstation gitops-review]$ git commit -m "Fix issue"
[student@workstation gitops-review]$ git push
```

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade gitops-review
```

Finish

As the student user on the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish gitops-review
```

Solution

Deploy MySQL applications with customized settings for the managed clusters by using RHACM and GitOps.

Outcomes

- Deploy applications by using RHACM and GitOps.
- Review application statuses and troubleshoot issues.
- Use Git to manage application lifecycles.

Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab install do0015l ; lab start gitops-review
```

Instructions

In this exercise, you deploy a MySQL database on your managed cluster by using GitOps and the Pull model. Your installation is already configured with the GitOps operator, and the `lab` command created a Kustomize Git repository with the resources for creating the database.

The public Git repository is available at <https://git.ocp4.example.com/do0015l/gitops-review.git>. The main branch stores the database resources in the `kustomize/overlays/qa` path.

The classroom environment provides two RHOC clusters: an RHACM hub cluster and a managed cluster. You can access those clusters by using the following parameters:

RHACM hub cluster web console URL

<https://console-openshift-console.apps.ocp4.example.com>

RHACM hub cluster API URL

<https://api.ocp4.example.com:6443>

Managed cluster web console URL

<https://console-openshift-console.apps.ocp4-mng.example.com>

Managed cluster API URL

<https://api.ocp4-mng.example.com:6443>

Administrator username for both clusters

admin (select the **Red Hat Identity Management** identity provider)

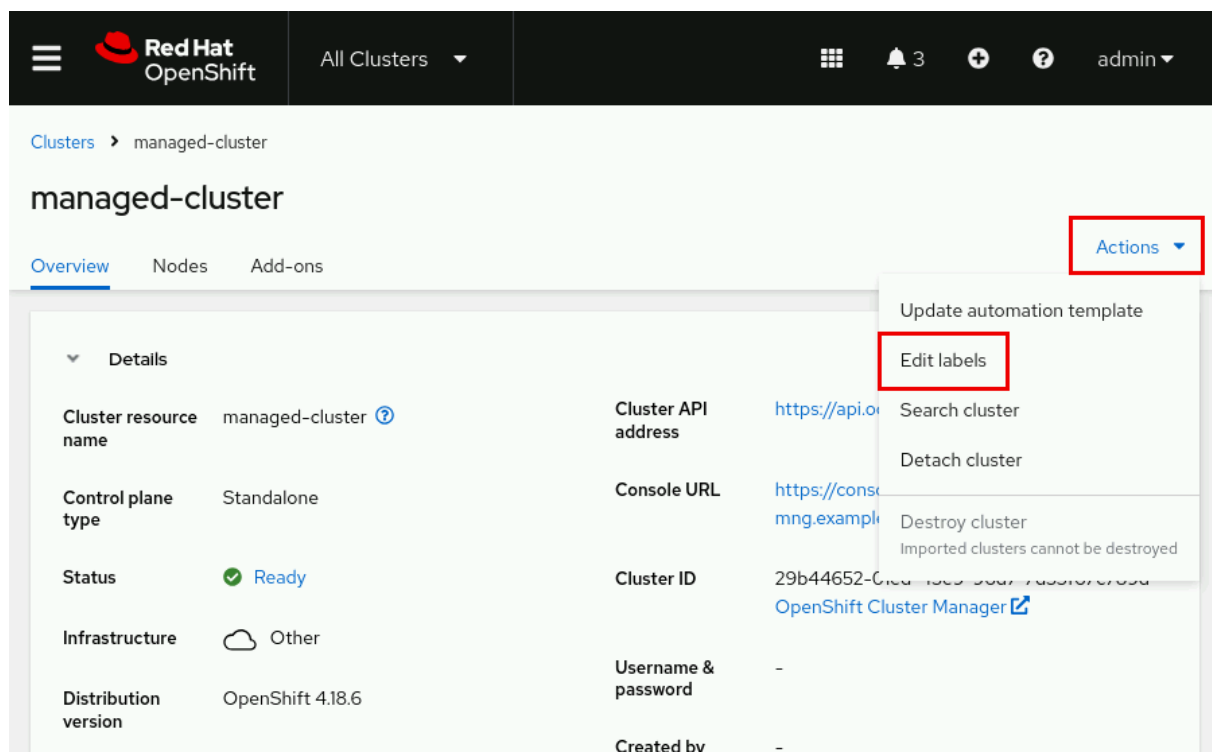
Administrator password for both clusters

redhatocp

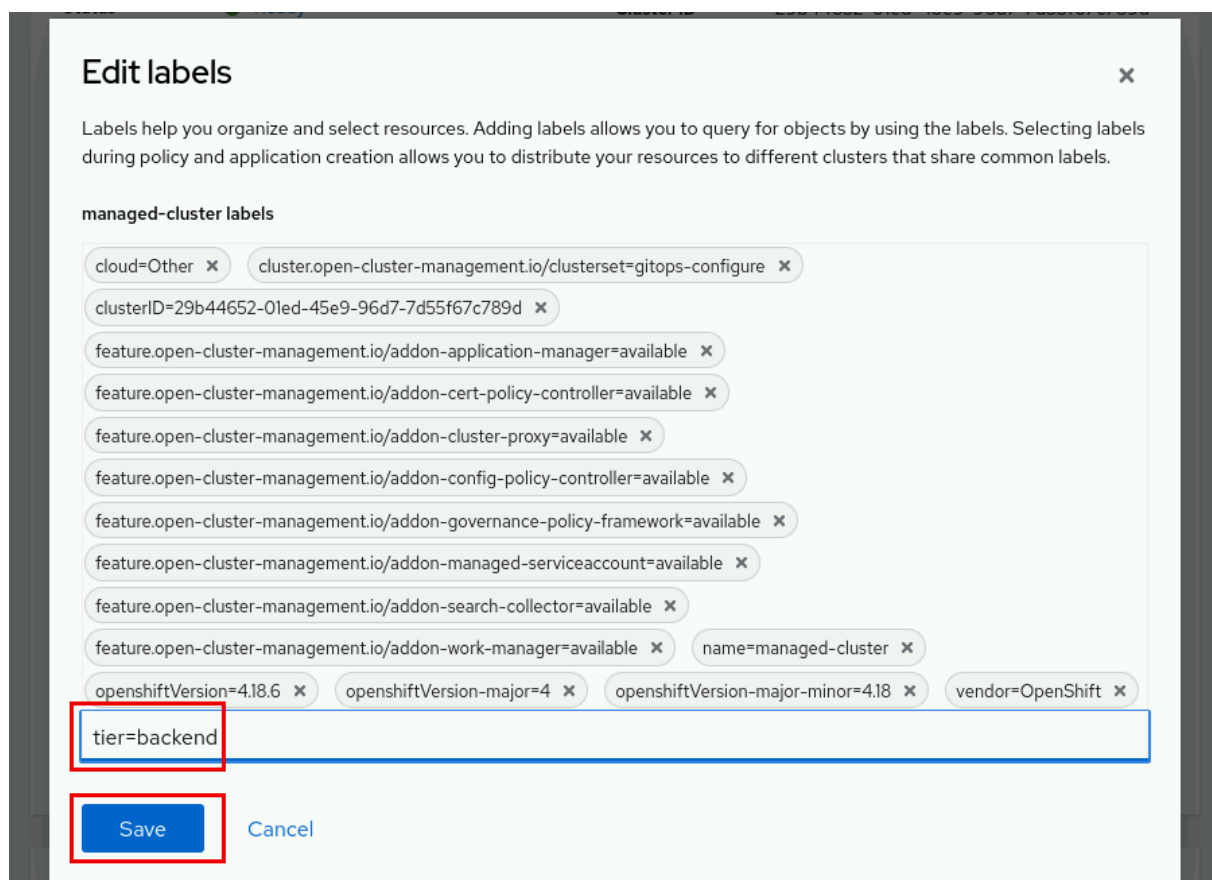
1. Add the `tier=backend` label to the managed cluster. You use this label in another step to select the cluster for the MySQL database.

1.1. Open a web browser and go to <https://console-openshift-console.apps.ocp4.example.com>

- 1.2. Select the **Red Hat Identity Management** identity provider and log in as the admin user with redhatocp as the password.
- 1.3. Ensure that the cluster switcher is set to **All Clusters**, which corresponds to the RHACM web console.
- 1.4. Go to **Infrastructure > Clusters**, click the managed-cluster name, and go to **Actions > Edit labels**.



- 1.5. Add the tier=backend label and click **Save**.

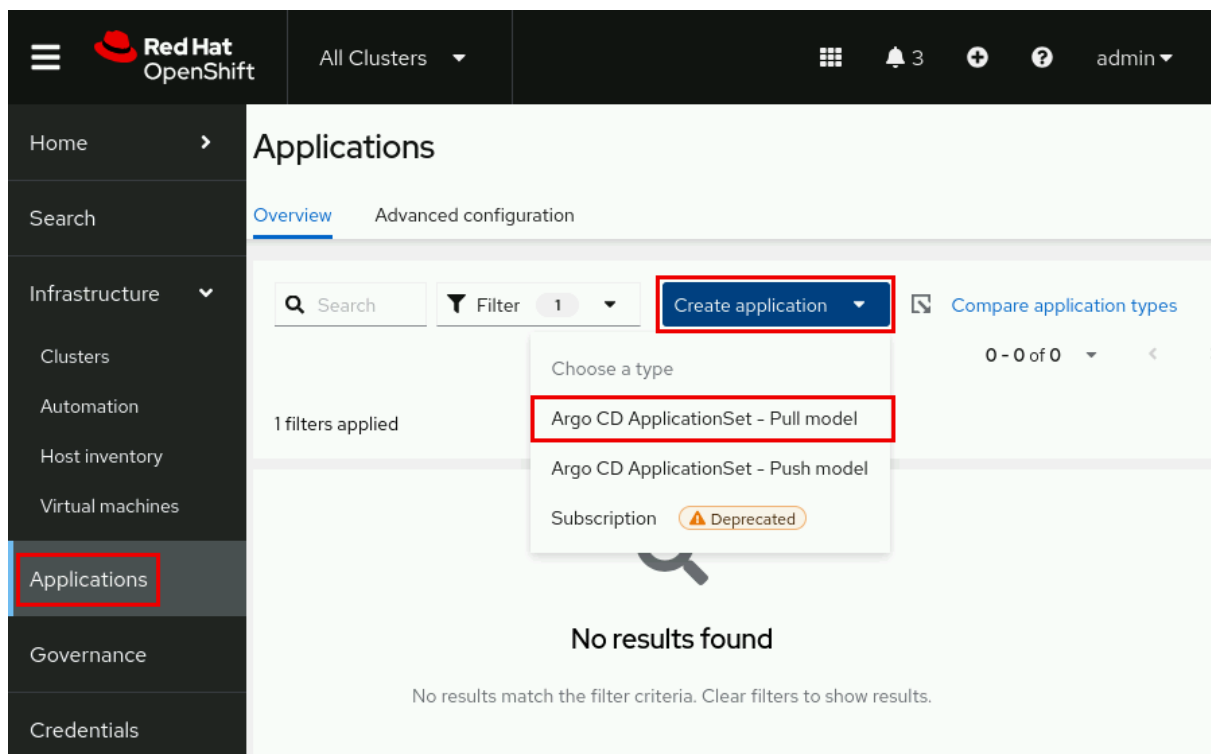


2. Create the application by using the GitOps Pull model and verify that RHACM deploys the database. The application has an issue that you fix in the next main step.

Use the following parameters:

Parameter	Value
GitOps model	Pull model
Application set name	intranet-db
Git URL	https://git.ocp4.example.com/do0015l/gitops-review.git
Git revision	main
Git path	kustomize/overlays/qa
Remote namespace	intranet-db
Placement cluster set	gitops-configure
Managed cluster label	tier=backend

- 2.1. Go to **Applications** and select **Create application > Argo CD ApplicationSet - Pull model**.



- 2.2. Complete the form with the provided parameters. In the **General** section, set the application name and select the Argo CD server. Click **Next**.

1 General

Operator required
The OpenShift GitOps Operator is required on the managed clusters to create an application set pull model type. Make sure the operator is installed on all managed clusters you are targeting.

General

Name *
intranet-db

Argo server * ?
openshift-gitops

Requeue time * ?
180

Next Back Cancel

- 2.3. In the **Template** section, select **Git** and provide the Git parameters. Click **Next**.

2 Template


Git
Use a Git repository


Helm
Use a Helm repository

URL * ?
https://git.ocp4.example.com/do0015l/gitops-review.git

Revision ?
main

Path ?
kustomize/overlays/qa

+ Add another repository

Destination

Remote namespace *
intranet-db

Next Back Cancel

- 2.4. Do not change anything in the **Sync policy** section. Click **Next**.
- 2.5. In the Placement section, select **New placement** and select the gitops-configure cluster set. Keep the default label expression that excludes the local cluster and add the expression that selects the cluster with the tier=backend label. Click **Next**.

4 Placement

Placement

How do you want to select clusters?

New placement

Existing placement

Cluster sets

gitops-configure 1

If no cluster sets are selected, all clusters will be selected from the cluster sets bound to the namespace.

Label expressions

name does not equal local-cluster

Label *

name

Operator *

does not equal any of

Values *

local-cluster 1

tier equals backend

Label *

tier

Operator *

equals any of

Values *

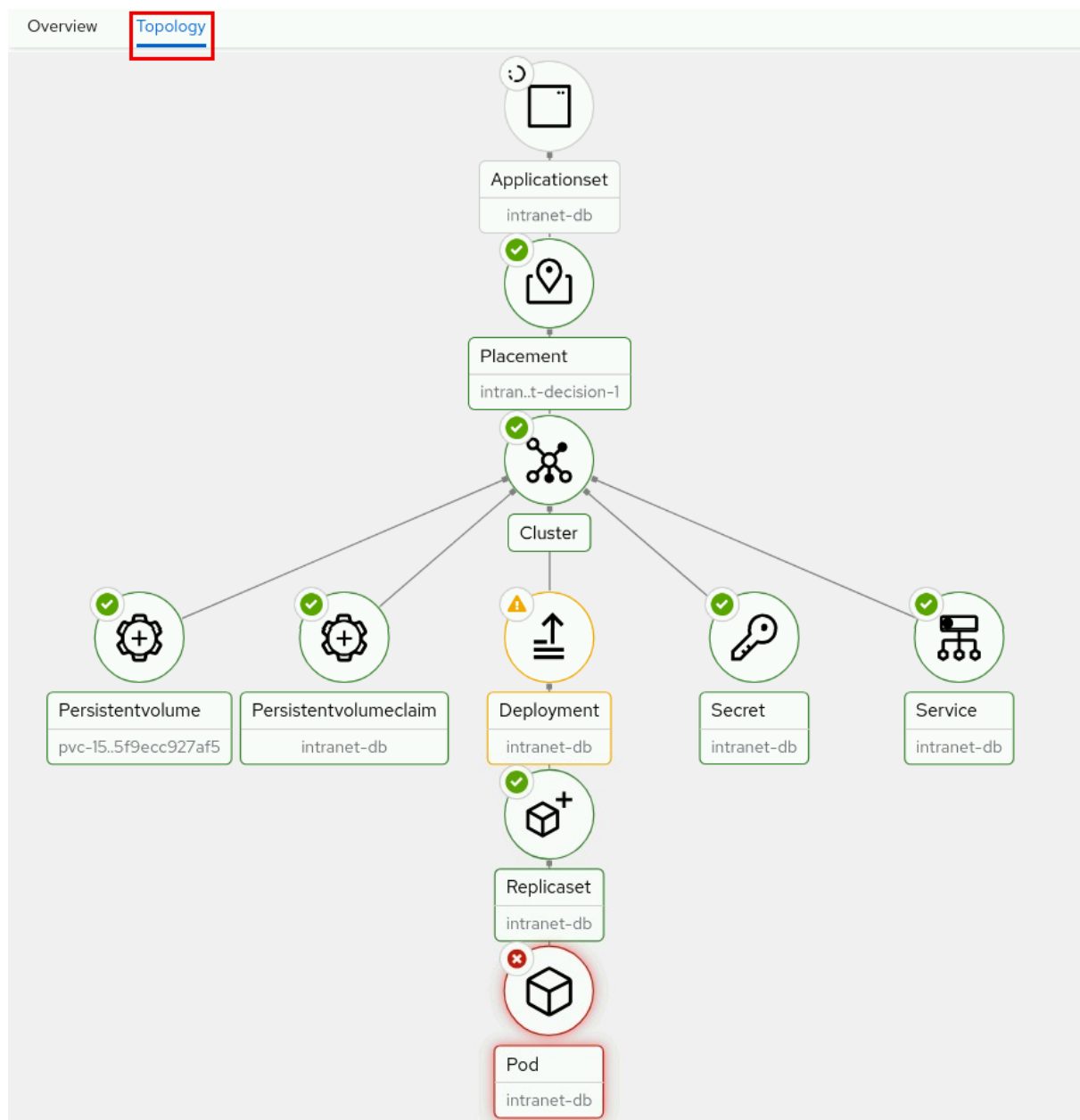
backend 1

2.6. In the Review section, verify the parameters and click **Submit**.

2.7. Go to **Topology** and wait up to three minutes for the topology to report the application status. The pod status alternates between Running and Error. You fix this issue in the next step.

Page 365 | 475

DO432-RHACM2.13-en-2-20260109



3. Investigate and fix the issue. You might have to clone the Git repository on your workstation machine to fix issues in the resource files.

Note

To clone the repository, run the following command:

```
[student@workstation ~]$ git clone \
https://git.ocp4.example.com/do0015l/gitops-review.git
```

After you edit a file, add it to the Git index, commit your change, and push the commit to the classroom GitLab system:

```
[student@workstation gitops-review]$ git add updated-file
[student@workstation gitops-review]$ git commit -m "Fix issue"
[student@workstation gitops-review]$ git push
```

- 3.1. The topology view indicates an error with the pod. Click the Pod icon to get more information about the error. The pod is in a CrashLoopBackOff cycle.

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

Click **View Pod YAML and Logs** to access the pod logs.

intranet-db

Overview Topology

intranet-db

Deployment

intranet-db

Replicaset

intranet-db

Pod

intranet-db

Pod

intranet-db

Launch resource in Search

Details Logs YAML

Type: Pod
Namespace: intranet-db
Labels: app=intranet-db,pod-template-hash=789966587b

Pod details for managed-cluster
Pod: intranet-db-789966587b-jnxxv
Namespace: intranet-db
Status: CrashLoopBackOff
View Pod YAML and Logs
Restarts: 5
Host and Pod IP: 192.168.51.10, 10.128.0.108
Created: 4 minutes ago

Pod: intranet-db-789966587b-jnxxv
Namespace: intranet-db
Status: CrashLoopBackOff
View Pod YAML and Logs
Restarts: 5
Host and Pod IP: 192.168.51.10, 10.128.0.108
Created: 4 minutes ago

3.2. Go to **Logs** and wait for the logs to be displayed. The `MYSQL_*` environment variables are missing. Usually, MySQL deployments get these variables from secret resources.

Close the browser tab.

Search pod details

intranet-db-789966587b-jnxxv

Details YAML Related resources **Logs** Actions

intranet-db Current log Search

☐ Wrap lines ☒ Raw ☒ Download ☒ Expand

Cluster: managed-cluster Namespace: intranet-db 25 lines

```
1 => sourcing 20-validate-variables.sh ...
2 You must either specify the following environment variables:
3 MYSQL_USER (regex: '[a-zA-Z0-9_]+$')
4 MYSQL_PASSWORD (regex: '[a-zA-Z0-9_~!@#%&*()-=<>,.;:|]+$')
5 MYSQL_DATABASE (regex: '[a-zA-Z0-9_]+$')
6 Or the following environment variable:
7 MYSQL_ROOT_PASSWORD (regex: '[a-zA-Z0-9_~!@#%&*()-=<>,.;:|]+$')
8 Or both.
9 Optional Settings:
10 MYSQL_LOWER_CASE_TABLE_NAMES (default: 0)
11 MYSQL_LOG_QUERIES_ENABLED (default: 0)
12 MYSQL_MAX_CONNECTIONS (default: 151)
13 MYSQL_FT_MIN_WORD_LEN (default: 4)
14 MYSQL_FT_MAX_WORD_LEN (default: 20)
15 MYSQL_AIO (default: 1)
```

3.3. Review the application manifests. In a terminal window, clone the application from the classroom GitLab system and change to the `gitops-review` directory.

```
[student@workstation ~]$ git clone \
https://git.ocp4.example.com/do0015l/gitops-review.git
Cloning into 'gitops-review'...
...output omitted...
[student@workstation ~]$ cd gitops-review
```

- 3.4.** Review the `kustomize/overlays/qa/kustomization.yaml` file, which Argo CD uses to deploy the application. You do not have to change its contents.

Kustomize uses the resources from the base directory.

```
---
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
- ../../base

labels:
- pairs:
    env: qa
```

- 3.5.** List the resource files in the `kustomize/base` directory. In most cases, the `secret.yaml` file stores the `MYSQL_*` environment variables for the database.

```
[student@workstation gitops-review]$ ls kustomize/base
deployment.yaml kustomization.yaml pvc.yaml secret.yaml service.yaml
```

- 3.6.** Review the `kustomize/base/secret.yaml` file. The manifest declares the variables, but the `MYSQL_` prefix is missing.

One solution to fix the issue is to edit the file by adding the missing prefix to the variables. The possible solution that is provided in this lab involves instead declaring the prefix in the deployment manifest. Therefore, do not change the contents of this `secret.yaml` file.

```
---
apiVersion: v1
kind: Secret
metadata:
  labels:
    app: intranet-db
    name: intranet-db
stringData:
  USER: appuser
  PASSWORD: securepassword
  DATABASE: quotesdb
  ROOT_PASSWORD: supersecurepassword
```

- 3.7.** Edit the `kustomize/base/deployment.yaml` file and add the `prefix: MYSQL_` parameter to the container's `envFrom` section.

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: intranet-db
  name: intranet-db
spec:
  replicas: 1
  selector:
    matchLabels:
      app: intranet-db
  template:
    metadata:
      labels:
        app: intranet-db
    spec:
      containers:
        - name: intranet-db
          image: registry.ocp4.example.com:8443/rhel9/mysql-80:1-237
          envFrom:
            - secretRef:
                name: intranet-db
                prefix: MYSQL_
          resources:
            requests:
              cpu: 10m
              memory: 256Mi
...output omitted...
```

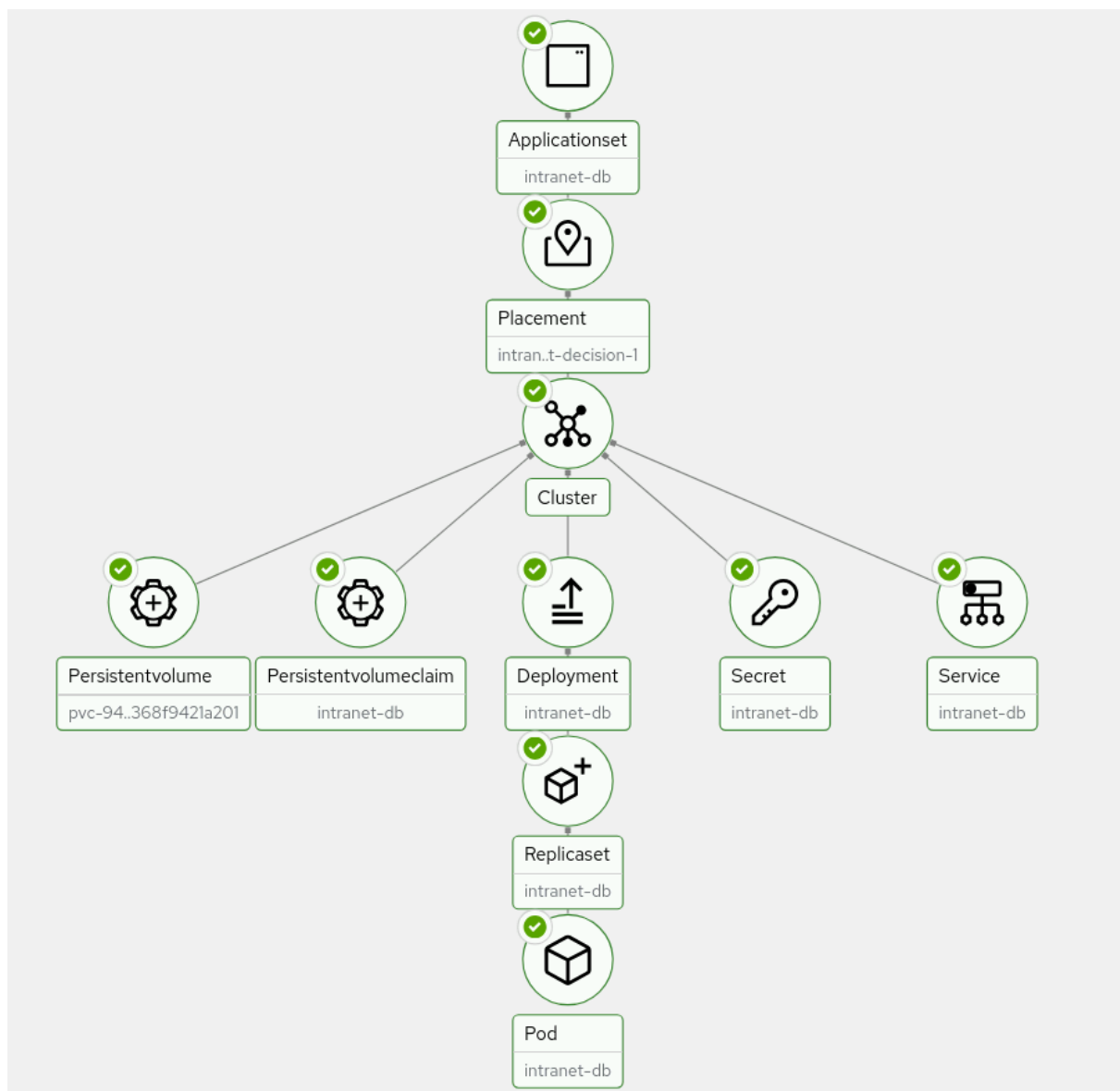
3.8. Add the file to the Git index, commit, and push your update.

```
[student@workstation gitops-review]$ git add kustomize/base/deployment.yaml
[student@workstation gitops-review]$ git commit -m 'Fix MySQL variables'
[main 3f3c5db] Fix MySQL variables
1 file changed, 1 insertion(+), 1 deletion(-)
[student@workstation gitops-review]$ git push
...output omitted...
```

3.9. Change to the /home/student directory.

```
[student@workstation gitops-review]$ cd
```

3.10. Switch back to the RHACM web console. If you are not already on the topology view, then go to **Application**, locate the intranet-db application, click the application name, and go to **Topology**. Wait about five minutes for the view to report a successful deployment.



Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade gitops-review
```

Finish

As the student user on the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish gitops-review
```

5.10. Summary

In this lesson, you learned about GitOps as an operational framework that applies DevOps best practices to application delivery by using Git as the single source of truth. This approach offers benefits such as declarative deployments, version control, consistency across environments, enhanced scalability, and improved security.

RHACM relies on Red Hat OpenShift GitOps and Argo CD to deploy applications on a cluster fleet.

RHACM supports the Push and Pull models for application deployment. In the Push model, the Argo CD instance on the hub cluster directly deploys applications to the selected remote clusters. The Pull model propagates application resources to the Argo CD instances that are running in each managed cluster.

Configuring application management in RHACM involves deploying the OpenShift GitOps operator and adjusting Argo CD instances. For example, you configure the instance with necessary deployment permissions. You register managed clusters with Argo CD by creating a `GitOpsCluster` resource, which uses a `Placement` resource to select the clusters.

To manage applications with RHACM, you create `ApplicationSet` and `Placement` resources. An `ApplicationSet` resource acts as a template for Argo CD to create `Application` resources for each targeted cluster.

You can use the topology view in the RHACM web console to review the application status. The Argo CD web console provides a similar view.

Troubleshooting RHACM applications starts with the topology view. Failed deployments are often caused by errors in `ApplicationSet` or `Placement` resources, such as an invalid Git repository configuration or an incorrect cluster selection. Dysfunctional applications often point to errors in the Git repository's manifests.

Next Steps

You might explore other learning opportunities on your own, explore further your RHACM installation, or redo some exercises. For example, you could research configuring Argo CD for a private Git repository that requires authentication.

A good next step in your learning journey would be to explore the next lesson in the series:

- Lesson 6: *Managing Virtual Machines for Multiple Clusters with Red Hat Advanced Cluster Management for Kubernetes*



References

[Argo CD: Private Repositories](#)

[Introduction to ApplicationSet Controller](#)

Chapter 6.

Managing Virtual Machines for Multiple Clusters with Red Hat Advanced Cluster Management for Kubernetes

Goal

Deploy and manage virtual machines in a multicluster environment by using Red Hat Advanced Cluster Management for Kubernetes (RHACM) and GitOps.

Sections

- Introducing OpenShift Virtualization (and Quiz)
- Deploying the OpenShift Virtualization Operator by Using GitOps with RHACM (and Guided Exercise)
- Deploying and Managing Multicluster Virtual Machines with RHACM (and Guided Exercise)
- Monitoring OpenShift Virtualization with RHACM (and Guided Exercise)

Lab

- Manage Virtual Machines for Multiple Clusters with RHACM

6.1. Introducing OpenShift Virtualization

Objectives

- Describe the features and benefits of OpenShift Virtualization.

Virtualization Concepts

System virtualization partitions a single physical computer into multiple, concurrent virtual computers. Each of these *virtual machines* (VMs) can run its own independent operating system. These VMs are fully isolated from each other, as if they are running on private hardware. VMs have their own network interfaces, IP addresses, file systems, and other peripherals. Each VM can run any supported operating system or version.

The *hypervisor* component is a crucial software layer that manages and supports the physical server virtualization, which enables and runs VMs for each virtualized operating system (OS). The hypervisor provides access to virtual CPUs, memory, disks, networking, and other peripherals, and generally prevents VMs from having direct access to hardware or to other VMs.

The physical machine and operating system that provide the hypervisor are called a *host*. A *guest* is an operating system that runs in a VM on a hypervisor.

Virtualization management is an effective solution for managing VMs and hosts that run in your infrastructure. Typically, virtualization management tools create an integrated virtualization environment to create, manage, and monitor VMs, hypervisors, storage, and networking resources. A virtualization manager provides centralized control for your virtualization infrastructure, rather than requiring you to individually manage and monitor hundreds of hypervisors and their VMs.

Red Hat OpenShift Virtualization

OpenShift Virtualization is an add-on operator to Red Hat OpenShift. Its primary purpose is to enable you to run and manage traditional VM workloads directly alongside your container workloads. This integration transforms VMs into native Kubernetes objects within your OpenShift cluster; you manage them by using familiar OpenShift tools such as the web console or the `oc` command-line interface (CLI). This unified approach to management provides both administrative and economical advantages, and improves application reliability and reduces operational overhead. OpenShift Virtualization is supported with all OpenShift editions, to enable managing VMs in a hybrid cloud environment.

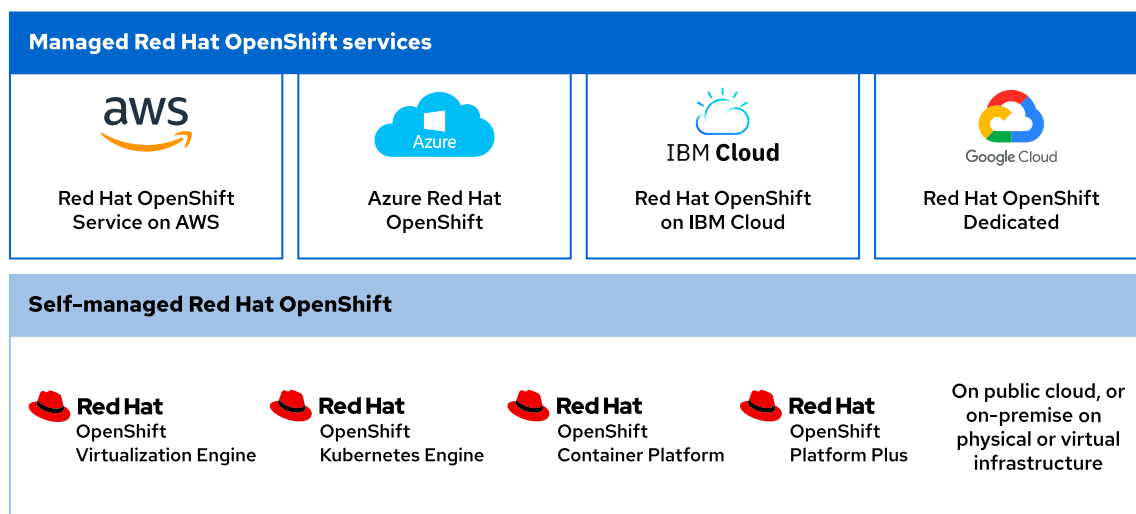


Figure 36. Product editions

OpenShift Virtualization is a Kubernetes operator that is based on the *KubeVirt* Kubernetes virtualization add-on. KubeVirt delivers container-native virtualization by using the *Kernel-based Virtual Machine* (KVM) Linux Kernel hypervisor within a Kubernetes container. KVM is an open source virtualization technology that is built into the Linux kernel, and enables a host machine to run multiple, isolated virtual environments and guest VMs. KubeVirt includes similar services to traditional virtualization platforms, to provide the best of mature virtualization management technology and Kubernetes container orchestration. Started by Red Hat, KubeVirt is now a Cloud Native Computing Foundation Incubation project and an open source upstream project.

OpenShift Virtualization uses the KVM technology as the hypervisor. Alongside KVM, OpenShift Virtualization uses libvirt libraries for defining VMs. KVM and libvirt run within a container, and are deployed on Kubernetes and OpenShift platforms alongside other containerized applications. The *Quick EMUlator* (QEMU) guest agent is another component that runs inside the VM to pass information about the VM to the host.

Essentially, KubeVirt, and by extension OpenShift Virtualization, integrates this robust KVM-based virtualization deeply into the Kubernetes environment, and treats VMs as another type of workload that Kubernetes can orchestrate.

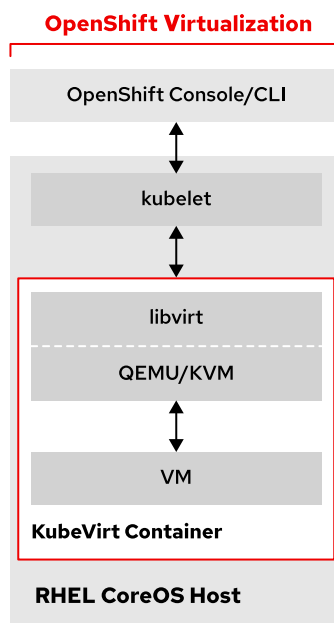


Figure 37. Containerizing KVM

Features of OpenShift Virtualization

Adding OpenShift Virtualization to a Red Hat OpenShift cluster enables deploying and managing virtualized workloads with the same hardware, environment, and tool set that the containerized applications use. OpenShift Virtualization extends the capabilities of your OpenShift cluster by offering a set of features for managing virtualized workloads. The following list describes some standout features:

Unified Resource Control

OpenShift Virtualization enables you to control the usage and access of CPU, memory, storage, and networking resources for VMs by using the same Kubernetes features that you would use for containers. This approach centralizes infrastructure management and streamlines operations.

Flexible VM Creation

You can create and manage both Linux and Windows VMs. You create VMs from *predefined templates* and *instance types*, or from custom templates and instance types that you create. Both templates and instance types define resources and characteristics to apply to new VMs, such as compute resources and *boot sources*, which are fully configured disk images that include drivers.

OpenShift Virtualization provides boot sources for select operating systems, such as Red Hat Enterprise Linux (RHEL) 9 and Fedora. Provided boot sources are updated automatically to the latest version of the operating system in connected clusters.

OpenShift Virtualization supports creating VMs by cloning existing VMs and storage volumes.

Access and Connectivity

You can access VMs through various consoles, which include *Virtual Network Computing* (VNC) and serial consoles, via the OpenShift web console or by using the `virtctl` CLI tool. OpenShift Virtualization also supports SSH protocol access for Linux, and Remote Desktop Protocol (RDP) for Windows VMs.

Advanced Resource Management

You can dynamically add or remove memory, CPU, and disks from running VMs without a restart.

You can define node placement rules to control where VMs are scheduled. OpenShift Virtualization 4.18 incorporates the `LongLifecycle` descheduler profile, which balances resource usage and enables VMs to live migrate to more appropriate nodes.

High Availability and Live Migration

OpenShift Virtualization offers features to provide *high availability* (HA) to VMs by using standard Kubernetes features and extensions, such as readiness and liveness probes.

You perform *live migration* to move a running VM from one node to another without interruption. Live migration has limitations, such as not supporting storage volumes with `ReadWriteOnce` (RWO) access mode or other passthrough features. For live migration, the VM's *Persistent Volume Claims* (PVCs) must use the `ReadWriteMany` access mode.

Note

High availability is not supported on single-node OpenShift clusters.

Storage Management

VMs in OpenShift Virtualization use the Kubernetes *Persistent Volume* (PV) and *Persistent Volume Claim* (PVC) paradigms for storage management. OpenShift Virtualization supports most storage providers with *Container Storage Interface* (CSI) drivers, such as Red Hat OpenShift Data Foundation and IBM Fusion.

In an environment without available network storage, you can deploy the *Hostpath Provisioner Operator* (HPP) to provision VMs on a node's local storage or on mounted volumes.

You can migrate VM disks to a different storage class by using the web console. When you perform storage class migration on a running VM, the VM is not interrupted and the data on the migrated disks remains accessible.

Note

Storage class migration is a Technology Preview feature for OpenShift Virtualization 4.18. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using Technology Preview features in production.

Backup and Restore

OpenShift Virtualization supports snapshots for VMs. A *snapshot* is a view of a VM that includes the operating system and applications on all the attached disks at a given time. You create and restore snapshots by using the web console or the CLI.

The *OpenShift API for Data Protection* (OADP) operator provides a native solution for backing up and restoring VM workloads. You can back up and restore VMs by using the same OADP resources as for backing up and restoring Kubernetes resources. You can also use other supported backup providers, such as Veeam Kasten and Portworx.

Monitoring

OpenShift Virtualization integrates with the OpenShift monitoring stack, which is based on Prometheus. Administrators can view detailed CPU, memory, storage, and network usage metrics for individual VMs through the OpenShift web console.

Disaster Recovery

OpenShift Virtualization supports both *Metro-DR* and *Regional-DR* methods, to provide robust solutions for business continuity.

Migration from Other Hypervisors

The *Migration Toolkit for Virtualization* (MTV) operator enables you to migrate multiple VMs from compatible hypervisors, such as VMware vSphere or Red Hat Virtualization, to OpenShift Virtualization.

OpenShift Virtualization is a solution for organizations that seek to modernize their infrastructure by converging traditional VM workloads with containerized applications on a single, unified platform. By using the Kubernetes robust orchestration capabilities, OpenShift Virtualization provides enterprise-grade virtualization features such as live migration, high availability, advanced storage management, and comprehensive monitoring. With its continuous enhancements, OpenShift Virtualization continues to evolve as a powerful tool for hybrid cloud environments.

References

For more information, refer to the *OpenShift Virtualization* documentation at https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/virtualization/index

The upstream KubeVirt project is hosted at <https://kubevirt.io>

For more information about the hostpath provisioner, refer to the *Configuring Local Storage for Virtual Machines* guide at https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/virtualization/index#virt-configuring-local-storage-with-hpp

For more information about creating virtual machines, refer to the *Creating Virtual Machines from Red Hat Images Overview* section at https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/virtualization/index#virt-creating-vms-from-rh-images-overview

6.2. Quiz

Introducing OpenShift Virtualization

Choose the correct answers to the following questions:

1. What is the primary purpose of OpenShift Virtualization as an add-on operator to Red Hat OpenShift?
 - ☐ a. Manage traditional VMs without integration with container workloads
 - ☐ b. Enable running and managing traditional VM workloads directly alongside container workloads
 - ☐ c. Replace existing Kubernetes clusters with a new virtualization platform
 - ☐ d. Provide a stand-alone hypervisor solution that is separate from OpenShift
2. Which component is a crucial software layer that manages and supports physical server virtualization, which enables and runs VMs for each virtualized OS?
 - ☐ a. Guest
 - ☐ b. Host
 - ☐ c. Hypervisor
 - ☐ d. Virtual Machine
3. Which open source virtualization technology, which is built into the Linux kernel, does KubeVirt use within a Kubernetes container to enable a host machine to run multiple, isolated virtual environments?
 - ☐ a. Red Hat Virtualization
 - ☐ b. Hyper-V
 - ☐ c. KVM
 - ☐ d. VirtualBox
4. Which is a key feature of OpenShift Virtualization for resource management of VMs?
 - ☐ a. VMs have separate resource controls from containers.
 - ☐ b. Resources for VMs (CPU, memory, storage, networking) are controlled by using the same Kubernetes features as for containers.
 - ☐ c. Dynamic addition and removal of memory, CPU, and disks for running VMs require a restart.
 - ☐ d. Advanced resource management features are handled through a separate external system.
5. What is a limitation for live migration of VMs in OpenShift Virtualization?
 - ☐ a. Live migration is fully supported for all storage volumes, including volumes with the ReadWriteOnce (RWO) access mode.
 - ☐ b. Live migration requires restarting the VM during the migration process.
 - ☐ c. Live migration is supported on single-node OpenShift clusters.
 - ☐ d. Live migration does not support storage volumes with the ReadWriteOnce (RWO) access mode, and requires the VM's PVCs to use the ReadWriteMany (RWX) access mode.

6. How does OpenShift Virtualization primarily handle storage management for VMs?
- ☐ a. It uses proprietary storage solutions that are separate from Kubernetes.
 - ☐ b. It uses the Kubernetes PV and PVC paradigms.
 - ☐ c. It supports only local storage on the host machine and cannot integrate with network storage.
 - ☐ d. It provides a dedicated storage interface only for VMs.
7. Which operator enables migrating multiple VMs from compatible hypervisors, such as VMware vSphere or Red Hat Virtualization, to OpenShift Virtualization?
- ☐ a. OpenShift API for Data Protection (OADP) operator
 - ☐ b. Hostpath Provisioner Operator (HPP)
 - ☐ c. Migration Toolkit for Virtualization (MTV) operator
 - ☐ d. Cluster Monitoring Operator

Solution

1. What is the primary purpose of OpenShift Virtualization as an add-on operator to Red Hat OpenShift?
 - ☐ a. Manage traditional VMs without integration with container workloads
 - ☒ b. **Enable running and managing traditional VM workloads directly alongside container workloads**
 - ☐ c. Replace existing Kubernetes clusters with a new virtualization platform
 - ☐ d. Provide a stand-alone hypervisor solution that is separate from OpenShift
2. Which component is a crucial software layer that manages and supports physical server virtualization, which enables and runs VMs for each virtualized OS?
 - ☐ a. Guest
 - ☐ b. Host
 - ☒ c. **Hypervisor**
 - ☐ d. Virtual Machine
3. Which open source virtualization technology, which is built into the Linux kernel, does KubeVirt use within a Kubernetes container to enable a host machine to run multiple, isolated virtual environments?
 - ☐ a. Red Hat Virtualization
 - ☐ b. Hyper-V
 - ☒ c. **KVM**
 - ☐ d. VirtualBox
4. Which is a key feature of OpenShift Virtualization for resource management of VMs?
 - ☐ a. VMs have separate resource controls from containers.
 - ☒ b. **Resources for VMs (CPU, memory, storage, networking) are controlled by using the same Kubernetes features as for containers.**
 - ☐ c. Dynamic addition and removal of memory, CPU, and disks for running VMs require a restart.
 - ☐ d. Advanced resource management features are handled through a separate external system.
5. What is a limitation for live migration of VMs in OpenShift Virtualization?
 - ☐ a. Live migration is fully supported for all storage volumes, including volumes with the ReadWriteOnce (RWO) access mode.
 - ☐ b. Live migration requires restarting the VM during the migration process.
 - ☐ c. Live migration is supported on single-node OpenShift clusters.
 - ☒ d. **Live migration does not support storage volumes with the ReadWriteOnce (RWO) access mode, and requires the VM's PVCs to use the ReadWriteMany (RWX) access mode.**
6. How does OpenShift Virtualization primarily handle storage management for VMs?
 - ☐ a. It uses proprietary storage solutions that are separate from Kubernetes.
 - ☒ b. **It uses the Kubernetes PV and PVC paradigms.**
 - ☐ c. It supports only local storage on the host machine and cannot integrate with network storage.
 - ☐ d. It provides a dedicated storage interface only for VMs.

7. Which operator enables migrating multiple VMs from compatible hypervisors, such as VMware vSphere or Red Hat Virtualization, to OpenShift Virtualization?
-  a. OpenShift API for Data Protection (OADP) operator
 -  b. Hostpath Provisioner Operator (HPP)
 -  c. **Migration Toolkit for Virtualization (MTV) operator**
 -  d. Cluster Monitoring Operator

6.3. Deploying the OpenShift Virtualization Operator by Using GitOps with RHACM

Objectives

- Deploy the OpenShift Virtualization Operator on multiple clusters by using RHACM.

RHACM provides a single point of control for managing the cluster and application lifecycle across multiple OpenShift Container Platform clusters. A key task in managing a fleet of clusters is deploying and maintaining operators, such as the Red Hat OpenShift Virtualization operator, in a consistent and automated way.

Deploying operators manually on each cluster is inefficient, error-prone, and does not scale in an enterprise environment. By using RHACM, you can automate the deployment of OpenShift Virtualization to ensure that all targeted clusters have a consistent installation, which simplifies management, governance, and future upgrades.

This process uses GitOps principles and RHACM policy enforcement to deploy operators declaratively, and treats your cluster configuration as code that is version-controlled and auditable.

Benefits of Deploying OpenShift Virtualization by Using RHACM

In an enterprise environment, managing virtualization across many OpenShift clusters can be complex. Each cluster requires installing and configuring the OpenShift Virtualization operator and its components. Manually performing these tasks on each cluster is inefficient, prone to human error, and difficult to scale. RHACM provides a centralized management hub to overcome these challenges. By using RHACM, you gain the following benefits:

- **Automate deployments:** Automate the installation and configuration of OpenShift Virtualization on multiple clusters simultaneously.
- **Enforce consistency:** Ensure that all managed clusters have a consistent installation and configuration of OpenShift Virtualization, which reduces configuration drift.
- **Scalability:** Deploy OpenShift Virtualization to new clusters as your environment grows.
- **Centralized management:** Manage the lifecycle of OpenShift Virtualization and other applications from a single console.

Core Concepts

Before deploying the OpenShift Virtualization operator by using RHACM, it is important to understand some RHACM components that enable multicluster management.

Cluster Sets

A `ClusterSet` resource provides a logical grouping of managed OpenShift clusters within RHACM. You can use cluster sets to organize clusters by environment, such as development or production, by business unit, or by geographic region. When you define application deployments or policies, you can target a cluster set to apply the configuration to all member clusters simultaneously, which simplifies management at scale.

GitOps and Argo CD

GitOps is an operational framework that applies DevOps best practices to application delivery and infrastructure management, by using a Git repository as the single source of truth. RHACM integrates Argo CD as its built-in GitOps

engine.

Argo CD continuously monitors a Git repository for changes and uses the repository to define the intended state of your infrastructure and applications. Changes to the cluster are made by updating the configuration in Git, which then triggers an automated process to align the live state with the defined state in the repository.

Argo CD Applications

In Argo CD, an *application* is a resource that references a Git repository with Kubernetes resource definitions.

Applications define a synchronization policy. Argo CD can detect and synchronize changes automatically, or users can trigger synchronizations on demand.

Push and Pull Models

RHACM provides two models of application lifecycle management:

Push model

With the Push model, the Argo CD instance that is running in the hub cluster manages the deployment of the applications into the managed clusters.

Pull model

With the Pull model, the Argo CD instance that is running in the hub cluster delegates the deployment of the applications to the Argo CD instance that is running in each managed cluster.

Argo CD Sync Waves

When deploying an operator and then creating instances of its custom resources (CRs), you must control the order of deployment. You can create a CR only after the operation registers its Custom Resource Definition (CRD) with the cluster.

Argo CD uses *sync waves*, which are defined by the `argocd.argoproj.io/sync-wave` annotation, to manage this ordering. Resources with a lower sync wave number are applied first. Resources without the annotation are considered to be in wave 0.

For OpenShift Virtualization, you deploy the `OperatorGroup` CR and `Subscription` CR in an earlier wave, such as wave 1, and the `HyperConverged` CR in a later wave, such as wave 3.

Deploying with GitOps

A recommended approach for deploying applications with RHACM is to use a GitOps workflow. This method uses an RHACM `Application` resource to deploy the OpenShift Virtualization operator and its custom resource from manifests in a Git repository.

Preparing the Git Repository

First, create a Git repository with the Kubernetes manifests that are required to install and enable OpenShift Virtualization. Use a consistent target namespace in each manifest. The OpenShift Virtualization operator typically uses the `openshift-cnv` namespace. Create a directory structure in your Git repository, for example the `openshift-virtualization/operator/` directory, to hold the configurations. The directory includes the `OperatorGroup`, `Subscription`, and `HyperConverged` CR manifests.

Creating the Operator Group

Before deploying the OpenShift Virtualization operator, you must create an OperatorGroup CR. The OperatorGroup makes the operator available to the target namespace. You place the OperatorGroup in the first sync wave. The following example shows an `operatorgroup.yaml` file that defines the necessary resource:

```
---
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: ocp-virt-operatorgroup
  namespace: openshift-cnv
  annotations:
    argocd.argoproj.io/sync-wave: "1" ①
spec:
  targetNamespaces:
    - openshift-cnv
```

- ① The sync-wave annotation ensures that the OperatorGroup CR is created before objects in other sync-wave groups.

Creating the Subscription

The Subscription resource directs the Operator Lifecycle Manager (OLM) to install the OpenShift Virtualization operator. This resource is placed in the second sync wave.

```
---
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: kubevirt-hyperconverged
  namespace: openshift-cnv
  annotations:
    argocd.argoproj.io/sync-wave: "2" ①
spec:
  source: redhat-operators
  sourceNamespace: openshift-marketplace
  name: kubevirt-hyperconverged
  channel: "stable"
  installPlanApproval: Automatic
```

- ① The sync-wave annotation ensures that the Subscription CR is created after the OperatorGroup CR and before the HyperConverged CR.

Creating the HyperConverged Custom Resource

The OpenShift Virtualization operator requires a HyperConverged CR. This CR activates the operator and deploys the virtualization components. It is placed in a later sync wave to ensure that the operator is ready.

```
---
apiVersion: hco.kubevirt.io/v1beta1
kind: HyperConverged
metadata:
  name: kubevirt-hyperconverged
  namespace: openshift-cnv
  annotations:
    argocd.argoproj.io/sync-options: SkipDryRunOnMissingResource=true ①
    argocd.argoproj.io/sync-wave: "3" ②
  labels:
    app: kubevirt-hyperconverged
    operators.coreos.com/kubevirt-hyperconverged.openshift-cnv: ""
spec: {}
```

- ① The SkipDryRunOnMissingResource=true annotation enables Argo CD to create the resource without using a dry run to verify dependencies.
- ② The sync-wave annotation ensures that the HyperConverged CR is created after the OperatorGroup CR and the Subscription CR.

 Note

The empty spec object in the HyperConverged CR directs the operator to install OpenShift Virtualization with the default configuration, which is suitable for most initial deployments. You can customize the installation by adding fields to the spec object.

Creating an Application from the RHACM Console

Now that the Git repository is prepared, you can create an RHACM application to deploy the OpenShift Virtualization operator.

- 1. In the RHACM console, go **Applications**.
- 2. Select **Create application > Argo CD ApplicationSet - Push model**.
- 3. On the **General** page, provide the following details:

Table 10. Create Application Set - Push Model - General

Parameter	Value
Name	The name for the application, such as openshift-virtualization-app
Argo Server	The name of the Argo CD server, such as openshift-gitops

Configure the repository settings, and proceed to the **Template** page.

Table 11. Create Application Set - Push Model - Template

Parameter	Value
Type	Git
URL	The URL of your Git repository

Parameter	Value
Revision	The branch that contains your manifests, such as the main branch
Path	The path from the root of the repository to the manifests, such as the openshift-virtualization/operator/ directory
Remote namespace	openshift-cnv

Go to the **Placement** page, and create or choose a placement that targets your intended managed clusters. Review the configuration settings and submit the request to add the application.

Argo CD detects the resources, uses the sync wave annotations to deploy the operator first, and then creates the HyperConverged resource to complete the installation.

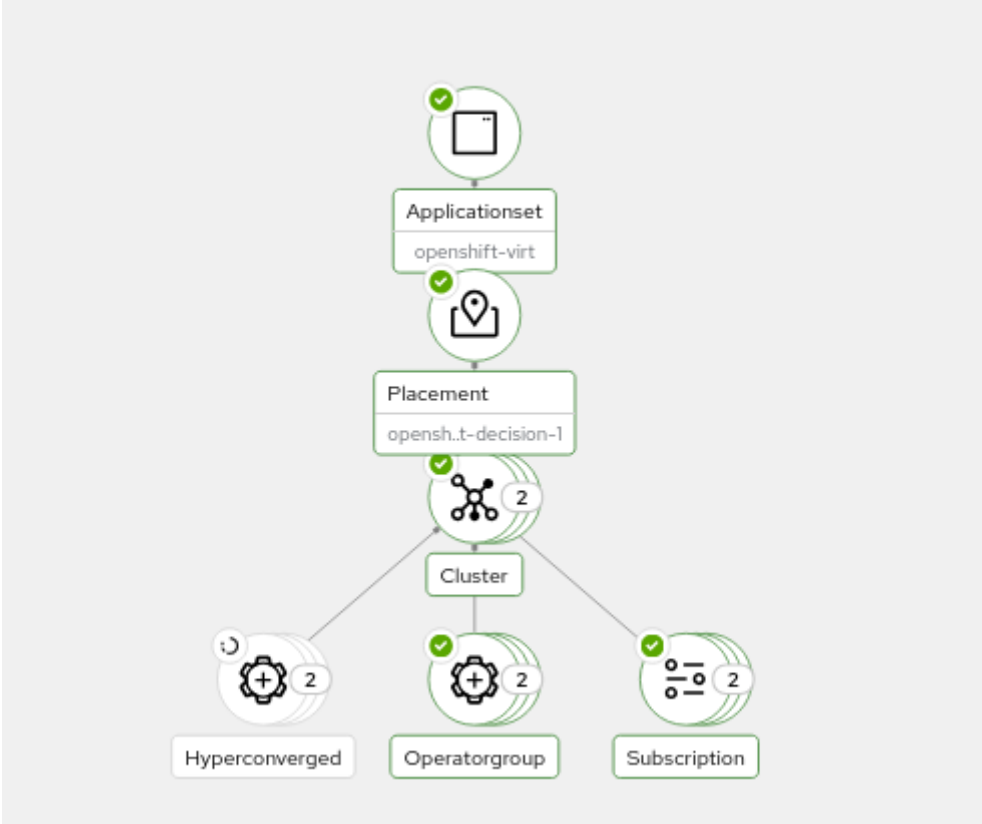


Figure 38. OpenShift virtualization application topology on two clusters

Deploying OpenShift Virtualization by Using RHACM Policies

Another method for deploying the OpenShift Virtualization operator is by using RHACM policies. Policies are a powerful way to enforce an intended state across your managed clusters. You can create policies to ensure that the OpenShift Virtualization operator is installed and configured correctly. Use this approach to ensure compliance and for environments where you must enforce specific configurations.

RHACM Policies for Governance

RHACM includes a policy and governance framework that defines and enforces configuration standards across your fleet of clusters. A policy is a set of rules that specifies an intended state for a configuration resource. RHACM continuously checks whether the managed clusters are compliant with the applied policies and can report on or remediate any drift from the intended state.

Creating a Policy to Install the Operator

RHACM includes an `OperatorPolicy` template that supports creating policies for operator installation. To create a policy to install the OpenShift Virtualization operator, go to **Governance > Policies** and click **Create Policy**. In the policy creation **Details** page, enter the following field values:

Table 12. Policy Create - Details

Parameter	Value
Name	A name for the Policy CR, such as <code>ocp-virt-install-policy</code>
Description	An optional description for the policy, which provides the value for the <code>policy.open-management-cluster.io/description</code> annotation
Namespace	The namespace on the hub cluster for the Policy CR, such as <code>install-ocp-virt</code> This namespace must have a binding to the <code>ClusterSet</code> CR that this policy uses.

On the policy creation **Policy Template** page, select **Add Policy Template > Install an operator**. This option corresponds to the `OperatorPolicy` policy template. In the fields, add values to define the operator policy:

Table 13. Policy Create - Operator Policy

Parameter	Value
Name	A name for the <code>OperatorPolicy</code> CR, such as <code>ocp-virt-install-operator-policy</code>
Installation Mode	This field provides information about the target namespace for the operator. The OpenShift Virtualization operator uses A specific namespace on the cluster.
Installed Namespace	When the operator requires a specific namespace, this field provides the name of the target namespace, such as <code>openshift-cnv</code> .

After the operator policy, provide the values for the operator subscription:

Table 14. Policy Create - Operator Subscription

Parameter	Value
Name	Use the package name from the packageManifest CR. The OpenShift Virtualization operator uses kubevirt-hyperconverged.
Channel	The packageManifest CR provides the default channel, such as stable.
Source	The packageManifest CR provides the default source for the operator catalog at metadata.labels.catalog. The OpenShift Virtualization operator in the classroom uses the gls-catalog-cs source.
Source Namespace	The packageManifest CR provides the default namespace for the operator catalog at metadata.labels.catalog-namespace. The OpenShift Virtualization operator in the classroom uses the openshift-marketplace source namespace.
Starting CSV	Use the starting CSV to specify the OpenShift Virtualization operator version to install. The packageManifest CR provides the current CSV for the operator at status.channels[0].currentCSV. The classroom uses the kubevirt-hyperconverged-operator.v4.18.0 version.
Allowed Cluster Service Versions	When a starting CSV value is provided, the same value must be added to the list of allowed versions. In this case, kubevirt-hyperconverged-operator.v4.18.0 must be included.
Remediation	Use Enforce to direct RHACM to install the operator.

On the policy creation **Placement** page, click **New Placement**. Use either a cluster set or labels to specify the target clusters for the OpenShift Virtualization operator.

Additional policy annotations are optional. Go to the **Review** page and select **Submit**.

The following YAML example of the policy ensures the creation of the openshift-cnv namespace, the OperatorGroup, and the Subscription for the OpenShift Virtualization operator:


```

apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: ocp-virt-install-policy
  namespace: install-ocp-virt ①
  annotations:
    policy.open-cluster-management.io/description: This policy installs the OpenShift Virtualization
operator
    policy.open-cluster-management.io/categories: ""
    policy.open-cluster-management.io/standards: ""
    policy.open-cluster-management.io/controls: ""
spec:
  disabled: false
  remediationAction: enforce
  policy-templates:
    - objectDefinition:
        apiVersion: policy.open-cluster-management.io/v1beta1
        kind: OperatorPolicy ②
        metadata:
          name: ocp-virt-install-operator-policy
        spec:
          remediationAction: enforce
          severity: critical
          complianceType: musthave
          subscription:
            name: kubevirt-hyperconverged
            namespace: openshift-cnv
            channel: stable
            source: gls-catalog-cs
            sourceNamespace: openshift-marketplace
            startingCSV: kubevirt-hyperconverged-operator.v4.18.0
          upgradeApproval: Automatic
          versions:
            - kubevirt-hyperconverged-operator.v4.18.0
          operatorGroup:
            name: default
            targetNamespaces:
              - openshift-cnv
  ---
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: ocp-virt-install-policy-placement
  namespace: install-ocp-virt
spec:
  tolerations:
    - key: cluster.open-cluster-management.io/unreachable
      operator: Exists
    - key: cluster.open-cluster-management.io/unavailable
      operator: Exists
  clusterSets:
    - gitops-configure
  ---
apiVersion: policy.open-cluster-management.io/v1
kind: PlacementBinding
metadata:
  name: ocp-virt-install-policy-placement
  namespace: install-ocp-virt
placementRef:
  name: ocp-virt-install-policy-placement
  apiGroup: cluster.open-cluster-management.io
  kind: Placement
subjects:
  - name: ocp-virt-install-policy
    apiGroup: policy.open-cluster-management.io
    kind: Policy

```

- 1 Before building the policy, ensure that a binding exists from this namespace to the target cluster set.
- 2 The `OperatorPolicy` CR supports operator installations.

Creating a Policy for the HyperConverged Instance

Create a `HyperConverged` CR so the OpenShift Virtualization operator becomes fully functional. Use a `RHACM ConfigurationPolicy` CR policy to ensure that the `HyperConverged` CR is created after the operator is installed. This policy depends on the successful installation of the operator. Use a YAML manifest to create this policy as follows:

```

apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: policy-create-hyperconverged
  namespace: install-ocp-virt
spec:
  dependencies:
    - name: ocp-virt-install-policy ①
      namespace: install-ocp-virt
      apiVersion: policy.open-cluster-management.io/v1
      compliance: Compliant
      kind: Policy
  disabled: false
  policy-templates:
    - objectDefinition:
        apiVersion: policy.open-cluster-management.io/v1
        kind: ConfigurationPolicy
        metadata:
          name: policy-hyperconverged-cr-def
        spec:
          object-templates:
            - complianceType: musthave
              objectDefinition:
                apiVersion: hco.kubevirt.io/v1beta1
                kind: HyperConverged
                metadata:
                  name: kubevirt-hyperconverged
                  namespace: openshift-cnv ②
                spec: {}
          remediationAction: enforce
          severity: high
      remediationAction: enforce
    ---
  apiVersion: cluster.open-cluster-management.io/v1beta1
  kind: Placement ③
  metadata:
    name: policy-create-hyperconverged-placement
    namespace: install-ocp-virt
  spec:
    clusterSets:
      - gitops-configure
    tolerations:
      - key: cluster.open-cluster-management.io/unreachable
        operator: Exists
      - key: cluster.open-cluster-management.io/unavailable
        operator: Exists
    ---
  apiVersion: policy.open-cluster-management.io/v1
  kind: PlacementBinding
  metadata:
    name: policy-create-hyperconverged-placement
    namespace: install-ocp-virt
  placementRef:
    name: policy-create-hyperconverged-placement
    apiGroup: cluster.open-cluster-management.io
    kind: Placement
  subjects:
    - name: policy-create-hyperconverged
      apiGroup: policy.open-cluster-management.io
      kind: Policy

```

- ① Use a dependency on the policy that installs the OpenShift Virtualization operator.
- ② Install the HyperConverged CR in the same namespace as the OpenShift Virtualization operator.
- ③ Use the same placement as the OpenShift Virtualization operator installation.

Verifying the Deployment

After deploying, you can verify the installation by using the RHACM console, the command line, or the OpenShift web console on a managed cluster.

Verifying with the RHACM Console

If the OpenShift Virtualization operator is installed via GitOps, then review the GitOps applications to verify the installation. Go to **Applications** in the RHACM console. If the `openshift-virtualization-app` application shows green as the status, then it indicates successful synchronization and deployment.

If the OpenShift Virtualization operator is installed via governance, then review the policies to verify the installation. Go to **Governance > Policies**. Both the `install-openshift-virt` policy and the `policy-create-hyperconverged` policy are expected to show green status.

Verifying with the Command Line

Log in to one of the managed clusters by using the `oc` command-line tool. Check for the running OpenShift Virtualization pods in the `openshift-cnv` namespace.

```
[user@host ~]$ oc get pods -n openshift-cnv
```

Several pods, such as `hco-operator-`, `virt-api-`, `virt-controller-`, and `virt-handler-`, are shown in the Running state.

Verify that the HyperConverged resource installation succeeded.

```
[user@host ~]$ oc get hco -n openshift-cnv kubevirt-hyperconverged \
-o yaml
```

The status of the resource is expected to show the `Available` and `ReconcileComplete` conditions.

Verifying with the OpenShift Web Console

You can also verify the deployment status directly in the OpenShift web console of the managed cluster.

1. Log in to the web console of a managed cluster where the operator is deployed. Use the **Administrator** perspective.
2. Go **Operators > Installed Operators** to confirm that the OpenShift Virtualization installation succeeded.
3. From the **Project** drop-down list, select the `openshift-cnv` project. Confirm that the OpenShift Virtualization operator has the Succeeded status.
4. Click **OpenShift Virtualization** to view the operator details.
5. Click **All instances**, and verify that the `kubevirt-hyperconverged` object status shows the **ReconcileComplete** and **Available** conditions. These conditions confirm that all components are deployed and running correctly.

References

For more information, refer to the *OpenShift Virtualization* documentation at

https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/virtualization/index

For more information about RHACM and GitOps, refer to the *Red Hat Advanced Cluster Management for Kubernetes* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/gitops/index

For more information about RHACM Governance Policies, refer to the *Red Hat Advanced Cluster Management for Kubernetes* documentation at

https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/governance/index

6.4. Guided Exercise

Deploy the OpenShift Virtualization Operator by Using GitOps with RHACM

Deploy the OpenShift Virtualization Operator on multiple clusters by using GitOps with RHACM.

Outcomes

- Create an application in Argo CD to deploy and configure the OpenShift Virtualization operator on the managed clusters.

Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available. The `lab start` command deploys the GitOps operator and creates a cluster set named `gitops-configure` that contains the hub cluster and the managed cluster.

```
[student@workstation ~]$ lab install do0016l ; lab start virtualization-deploy
```

Instructions

In this exercise, you use GitOps with RHACM to deploy the OpenShift Virtualization operator in your two clusters.

1. Retrieve and review the OpenShift Virtualization operator manifests.
 - 1.1. In a terminal window, clone the application from the classroom GitLab system. The application is available in the `https://git.ocp4.example.com/do0016l/virtualization-deploy.git` repository.

```
[student@workstation ~]$ git clone \
https://git.ocp4.example.com/do0016l/virtualization-deploy.git
Cloning into 'virtualization-deploy'...
...output omitted...
```

- 1.2. Review the manifest files.

```
[student@workstation ~]$ tree virtualization-deploy
virtualization-deploy
├── virtualization
│   └── manifests
│       ├── hco.yaml ①
│       ├── operatorgroup.yaml ②
│       └── subscription.yaml ③
```

- ① The HyperConverged CR deploys in sync-wave 3.
- ② The OperatorGroup CR deploys in sync-wave 1.
- ③ The Subscription CR deploys in sync-wave 2.

- 1.3. Review the contents of the OperatorGroup resource in the `virtualization-deploy/virtualization/manifests/operatorgroup.yaml` file. You do not have to change its contents.

```
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: openshift-cnv
  namespace: openshift-cnv
  annotations:
    argocd.argoproj.io/sync-wave: "1"
spec:
  targetNamespaces:
    - openshift-cnv
```

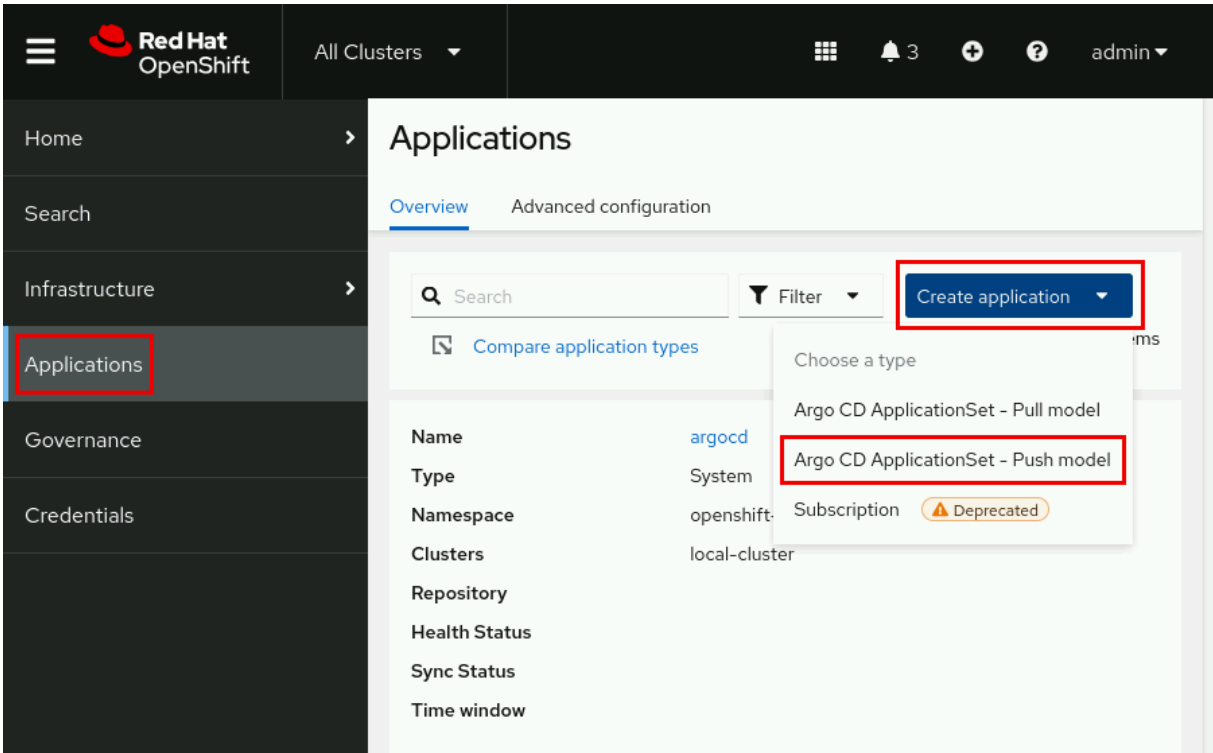
- 1.4. Review the contents of the Subscription resource in the virtualization-deploy/virtualization/manifests/subscription.yaml file. You do not have to change its contents.

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: kubevirt-hyperconverged
  namespace: openshift-cnv
  annotations:
    argocd.argoproj.io/sync-wave: "2"
spec:
  channel: "stable"
  installPlanApproval: Automatic
  name: kubevirt-hyperconverged
  source: gls-catalog-cs
  sourceNamespace: openshift-marketplace
```

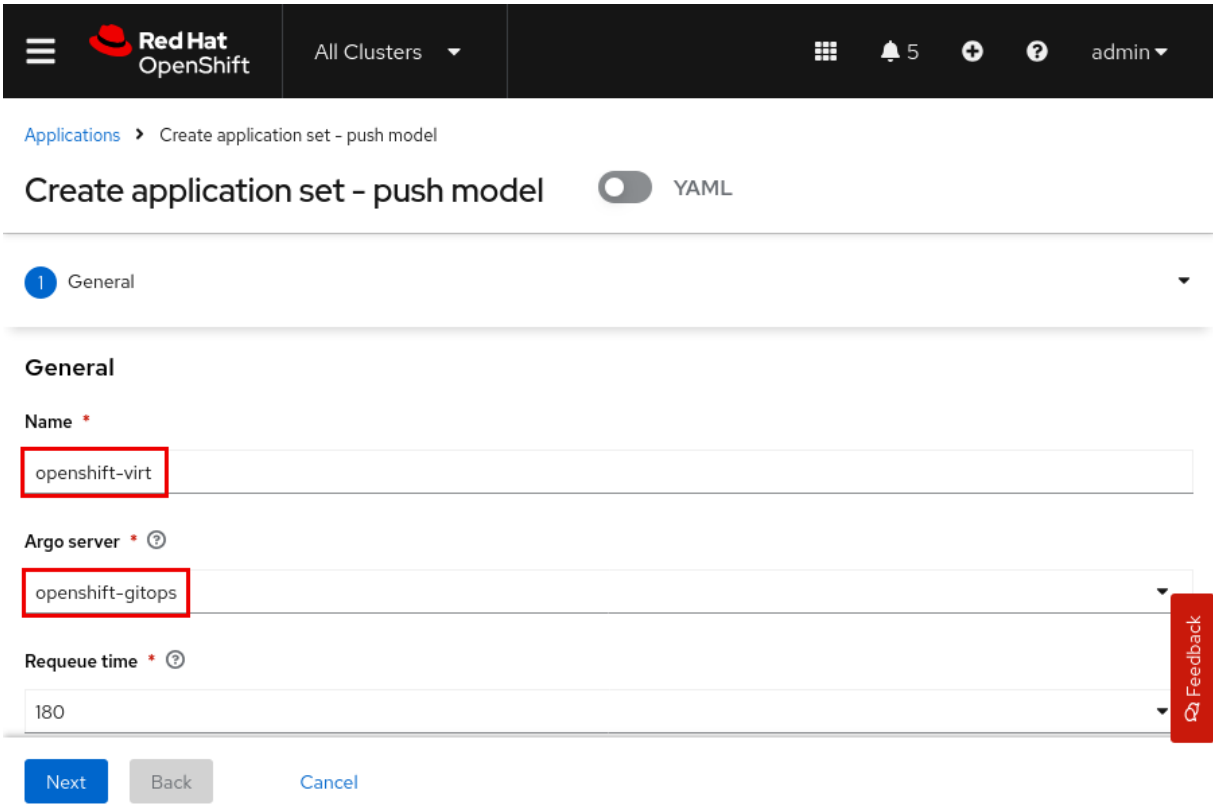
- 1.5. Review the contents of the HyperConverged resource in the virtualization-deploy/virtualization/manifests/hco.yaml file. You do not have to change its contents.

```
apiVersion: hco.kubevirt.io/v1beta1
kind: HyperConverged
metadata:
  name: kubevirt-hyperconverged
  namespace: openshift-cnv
  annotations:
    deployOVS: "false"
    argocd.argoproj.io/sync-options: SkipDryRunOnMissingResource=true
    argocd.argoproj.io/sync-wave: "3"
  labels:
    app: kubevirt-hyperconverged
    operators.coreos.com/kubevirt-hyperconverged.openshift-cnv: ""
spec:
  ksmConfiguration:
    nodeLabelSelector: {}
    ...output omitted...
```

2. Log in to the RHOCp hub cluster, and switch to the RHACM web console.
 - 2.1. Open a web browser and go to <https://console-openshift-console.apps.ocp4.example.com>
 - 2.2. Select the **Red Hat Identity Management** identity provider and log in as the admin user with redhatocp as the password.
 - 2.3. Ensure that the cluster switcher is set to **All Clusters**, which corresponds to the RHACM web console.
3. Deploy the OpenShift Virtualization operator by using the GitOps push model.
 - 3.1. Go to **Applications** and select **Create application > Argo CD ApplicationSet - Push model**.



3.2. Set the name of the application to openshift-virt, select the openshift-gitops Argo server, and click **Next**.



3.3. Configure the repository as follows, and click **Next**.

Parameter	Value
Type	Git

Parameter	Value
URL	https://git.ocp4.example.com/do0016l/virtualization-deploy.git
Revision	main
Path	virtualization/manifests
Remote namespace	openshift-cnv


Git
Use a Git repository


Helm
Use a Helm repository

URL * ?

https://git.ocp4.example.com/do0016l/virtualization-deploy.git

Revision ?

main

Path ?

virtualization/manifests

+ Add another repository

Destination

Remote namespace *

openshift-cnv

Next

Back

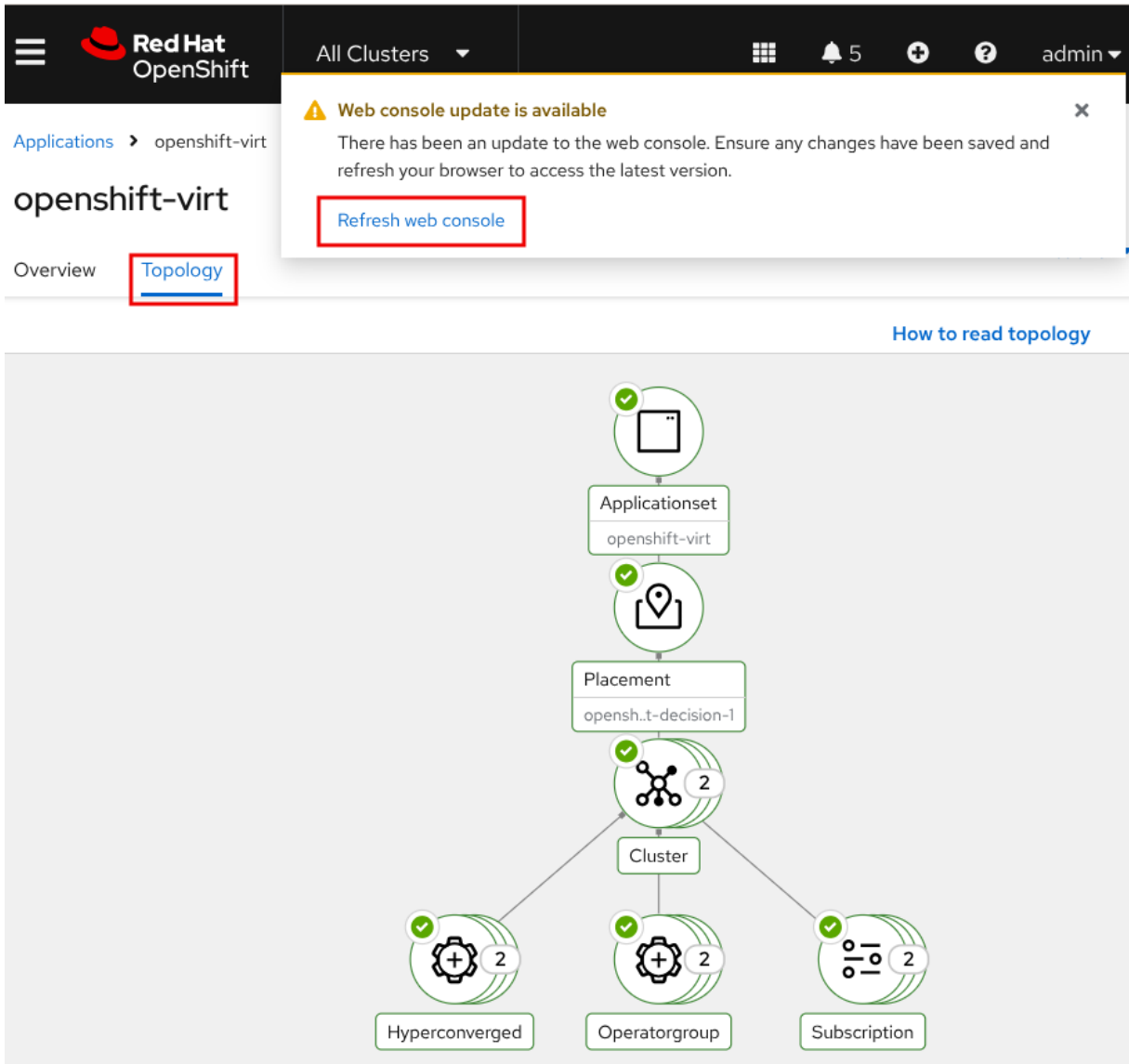
Cancel

3.4. In the Sync policy section, leave the default values and click **Next**.

3.5. Create a placement to select the gitops-configure cluster set. Set the **Limit the number of clusters selected** to 2. Click **Next**.

The screenshot shows the Red Hat OpenShift web console interface for creating an application set. The top navigation bar includes the Red Hat OpenShift logo, a dropdown for 'All Clusters', and user information 'admin'. The breadcrumb trail is 'Applications > Create application set - push model'. The main heading is 'Create application set - push model' with a 'YAML' toggle. The 'Placement' tab is selected, indicated by a blue circle with the number 4. Below the heading, the question 'How do you want to select clusters?' is followed by two buttons: 'New placement' (highlighted) and 'Existing placement'. Under 'Cluster sets', a search bar contains 'gitops-configure' and a dropdown shows '1' selected. A note states: 'If no cluster sets are selected, all clusters will be selected from the cluster sets bound to the namespace.' Below this, the 'Label expressions' section has a '+ Add label expression' button. The 'Limit the number of clusters selected' section shows a numeric input with minus, '2', and plus buttons. At the bottom are 'Next', 'Back', and 'Cancel' buttons. A red 'Feedback' button is on the right side.

- 3.6. Review your configuration and click **Submit**.
- 3.7. Click the **Topology** tab and wait up to six minutes for the topology to report the application status for the application to be ready.
- 3.8. The web console displays the **Web console update is available** pop-up message when the application is ready. Click **Refresh web console** to enable the Virtualization menu option in the local-cluster cluster.



- 4. Verify that the OpenShift Virtualization operator runs in the two clusters.
 - 4.1. Verify that the operator is available in the local-cluster cluster. Ensure that the cluster switcher is set to `local-cluster`, which corresponds to the RHOCP web console.
 - 4.2. From the left panel of the OpenShift web console, go to **Operators > Installed Operators** to confirm that the OpenShift Virtualization installation succeeded.

Project: All Projects ▾

Installed Operators

Installed Operators are represented by ClusterServiceVersions within this Namespace. For more information, see the [Understanding Operators documentation](#). Or create an Operator and ClusterServiceVersion using the [Operator SDK](#).

Name ▾

Search by name... /

Name	Namespace	Managed Namespaces	Status
 Advanced Cluster Management for Kubernetes 2.13.0 provided by Red Hat	NS open-cluster-management	NS open-cluster-management	✓ Succeeded Up to date
 OpenShift Virtualization 4.18.0 provided by Red Hat	NS openshift-cnv	NS openshift-cnv	✓ Succeeded Up to date

- 4.3. Click **OpenShift Virtualization** to view the operator details.
- 4.4. Click **All instances**, and verify that the **kubevirt-hyperconverged** object status shows the **ReconcileComplete**, **Available**, and **Upgradeable** conditions.

Project: openshift-cnv ▾

Installed Operators ▸ Operator details



OpenShift Virtualization
4.18.0 provided by Red Hat

Actions ▾

Details

YAML

Subscription

Events

All instances

OpenShift Virtualization Deployments

All Instances

Create new ▾

Filter ▾

Name ▾

Search by name... /

Name	Kind	Status
 kubevirt-hyperconverged	HyperConverged	Conditions: ReconcileComplete, Available, Upgradeable

- 4.5. Open a browser tab and go to <https://console-openshift-console.apps.ocp4-mng.example.com> to log in to the RHOCP managed-cluster cluster.

- 4.6. Select the **Red Hat Identity Management** identity provider and log in as the admin user with redhatocp as the password.
- 4.7. From the left panel of the OpenShift web console, go to **Operators > Installed Operators** to confirm that the OpenShift Virtualization installation succeeded.
- 4.8. Click **OpenShift Virtualization** to view the operator details.
- 4.9. Click **All instances**, and verify that the **kubevirt-hyperconverged** object status shows the **ReconcileComplete**, **Available**, and **Upgradeable** conditions.

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish virtualization-deploy
```

6.5. Deploying and Managing Multicluster Virtual Machines with RHACM

Objectives

- Deploy and manage virtual machines across a multicluster environment by using RHACM.

Virtual Machine Resources

A VM is a virtual environment that simulates the resources of a physical machine, which includes CPU, memory, network interfaces, and storage. Typically, VMs maintain the application state and persistent data, similar to `StatefulSet` pods.

A VM object specifies a template to create a running instance of the VM inside your cluster. The running instance of a VM is a VMI. A container inside a pod executes and manages the VMI.

The relationship between a VM and a VMI is similar to the relationship between a deployment and a replica set. A VM object specifies the template for a VMI pod. If a VMI is deleted, then Kubernetes generates another instance that is based on the VM object configuration.

When a VM is created and started, a `virt-controller` pod signals to a `virt-handler` pod on a cluster node to create a `virt-launcher` pod for the VMI. The `virt-launcher` pod consists of a running `libvirtd` container instance to execute the VMI as an isolated process.

Components of the Virtual Machine Creation Process

For the Red Hat OpenShift components that provide virtualization, the cluster structures components into a hierarchy, where higher components (such as the `virt-controller` operator) orchestrate lower components (such as the `virt-handler` daemon set and the VM's pod).

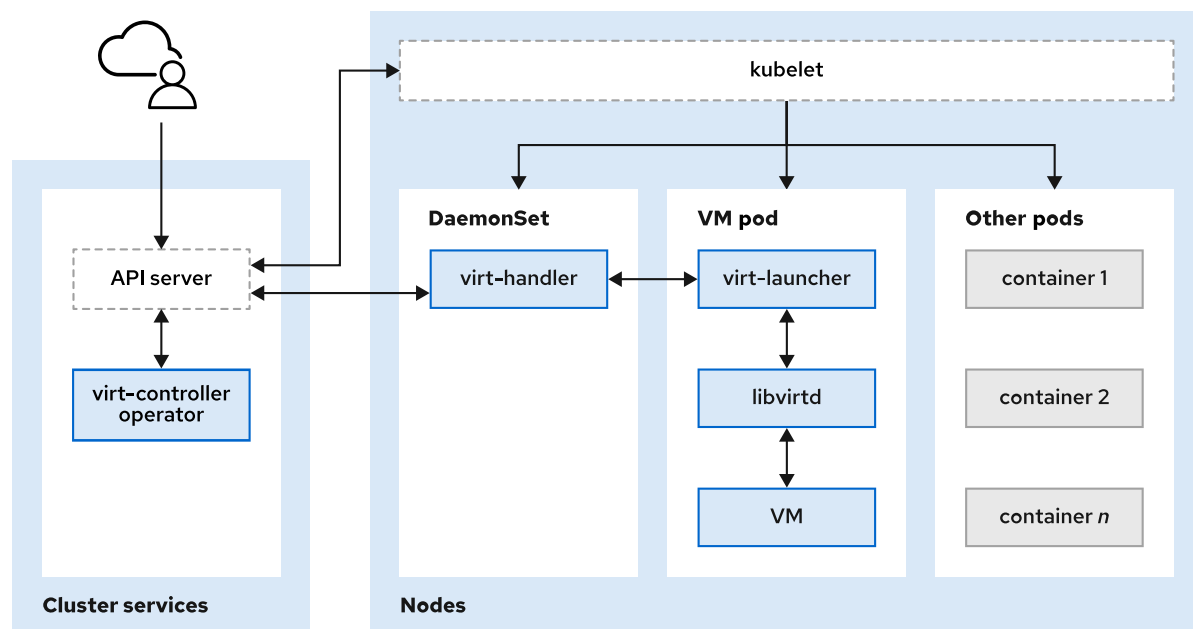


Figure 39. Components of the VM creation

The `virt-controller` operator monitors for new VMI objects. The `virt-handler` daemon set runs on each node to execute any necessary actions to meet a VM object's defined state. The `virt-handler` daemon set monitors for changes in a VM, and creates the `virt-launcher` container.

The `virt-launcher` container runs within each pod of a VM, and instantiates the VM with the use of a local `libvirtd` instance, which provides a virtualization architecture and interfaces with the kernel to manage the lifecycle of the VM process. When the VMI is provisioned, the `virt-launcher` pod routes IPv4 traffic to the *Dynamic Host Configuration Protocol (DHCP)* address of the VMI. This routing enables you to connect to a VMI with a port-forwarding connection.

Deploying Virtual Machines on Multiple Clusters

RHACM provides comprehensive tools for managing the lifecycle of Kubernetes and OpenShift clusters, which includes deploying and managing VMs across a fleet of clusters.

Argo CD is a declarative continuous delivery tool for Kubernetes either as a stand-alone tool or as part of your CI/CD workflow to deliver needed resources from a Git repository to your OpenShift clusters.

The Argo CD tool consists of two resources:

Application resource

An Application resource is a CRD that defines a source of Kubernetes manifests and a target cluster for their application.

Application Set resource

An ApplicationSet resource is a CRD that extends Argo CD applications. The purpose of the application set is to deploy and manage Application resources across multiple clusters by using the same manifest or declaration.

Deploying Virtual Machines with an Application Set

With an application set, which is integrated with OpenShift GitOps, you can define, synchronize, and deploy VMs across multiple managed clusters.

You can use an application set to deploy VMs in multiple managed clusters by using the same declaration of a VM for all clusters.

The following YAML example shows a VM manifest:

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora ①
  annotations:
    argocd.argoproj.io/sync-wave: "3"
spec:
  dataVolumeTemplates:
    - metadata:
        name: fedora ②
      spec:
        sourceRef:
          kind: DataSource
          name: fedora ③
          namespace: openshift-virtualization-os-images
        storage:
          accessModes:
            - ReadWriteOnce
          resources:
            requests:
              storage: 10Gi ④
          storageClassName: nfs-storage
 instancetype:
    name: u1.small ⑤
  preference:
    name: fedora
  runStrategy: Always ⑥
  template:
    metadata:
      labels:
        network.kubevirt.io/headlessService: headless
  spec:
    domain:
      devices:
        autoattachPodInterface: false
        disks: []
        interfaces:
          - masquerade: {}
            name: default
    networks:
      - name: default
        pod: {}
    subdomain: headless
    volumes:
      - dataVolume:
          name: fedora
          name: rootdisk
```

- ① The VM name
- ② The data volume name
- ③ The data source name
- ④ The size of the requested storage for the data volume
- ⑤ The instance type to control resource sizing of the VM

- ⑥ The VMI behavior. If set to Always, then a VMI is always present when a VM is created. A new VMI is created if the original VMI stops for any reason. If set to Manual, then no VMI is present when a VM is created. You manually start, stop, or pause the VM to control the VMI state.

To create an application set, go to **Applications** and select **Create application > Argo CD ApplicationSet - Push model**.

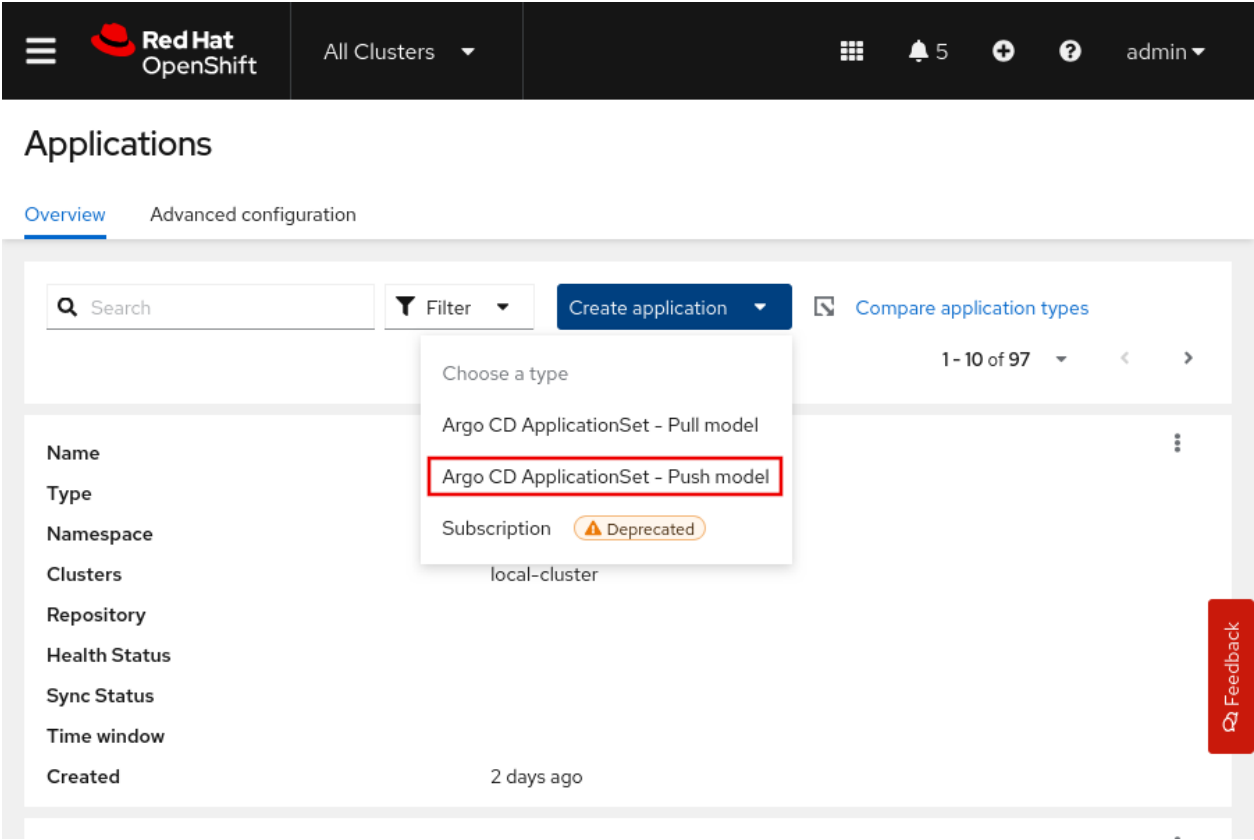


Figure 40. Create an application set

In the following section, you can define a name and select the Argo CD server:

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

Red Hat OpenShift

All Clusters

5

admin

Applications > Create application set - push model

Create application set - push model

YAML

1 General

General

Name *

test-vms

Argo server * ?

openshift-gitops

Requeue time * ?

180

Next

Back

Cancel

Feedback

Figure 41. General page of the application set creation

Note

Enable the YAML view to show the YAML manifest of the application set.

On the **Template** page, you must provide the configuration of the Git repository that contains the manifest files of the VM to deploy. Specify a namespace to deploy the VMs in the **Remote** namespace field.

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

Red Hat OpenShift

All Clusters

5

admin

Applications > Create application set - push model

Create application set - push model

YAML

2 Template

URL

https://git.ocp4.example.com/test-git/virtualmachines.git

Revision

main

Path

virtualmachines/manifests

Feedback

NextBackCancel

Figure 42. Template page of the application set creation

In the following section, you can select the settings used to configure the application synchronizing when differences exist between the intended state and the live cluster state.

Red Hat OpenShift

All Clusters

5

admin

Applications > Create application set - push model

Create application set - push model

YAML

3 Sync policy

Sync policy

Settings used to configure application syncing when there are differences between the desired state and the live cluster state.

☒

Delete resources that are no longer defined in the source repository

☒

Delete resources that are no longer defined in the source repository at the end of a sync operation

☐

Replace resources instead of applying changes from the source repository

☐

Allow applications to have empty resources

Feedback

NextBackCancel

Figure 43. Sync policy page of the application set creation

In the **Placement** section, you must select a cluster set that you previously configured. You can limit the number of clusters that the application set can use to deploy VMs. From this section, you can also configure label expressions to

select clusters from the cluster set.

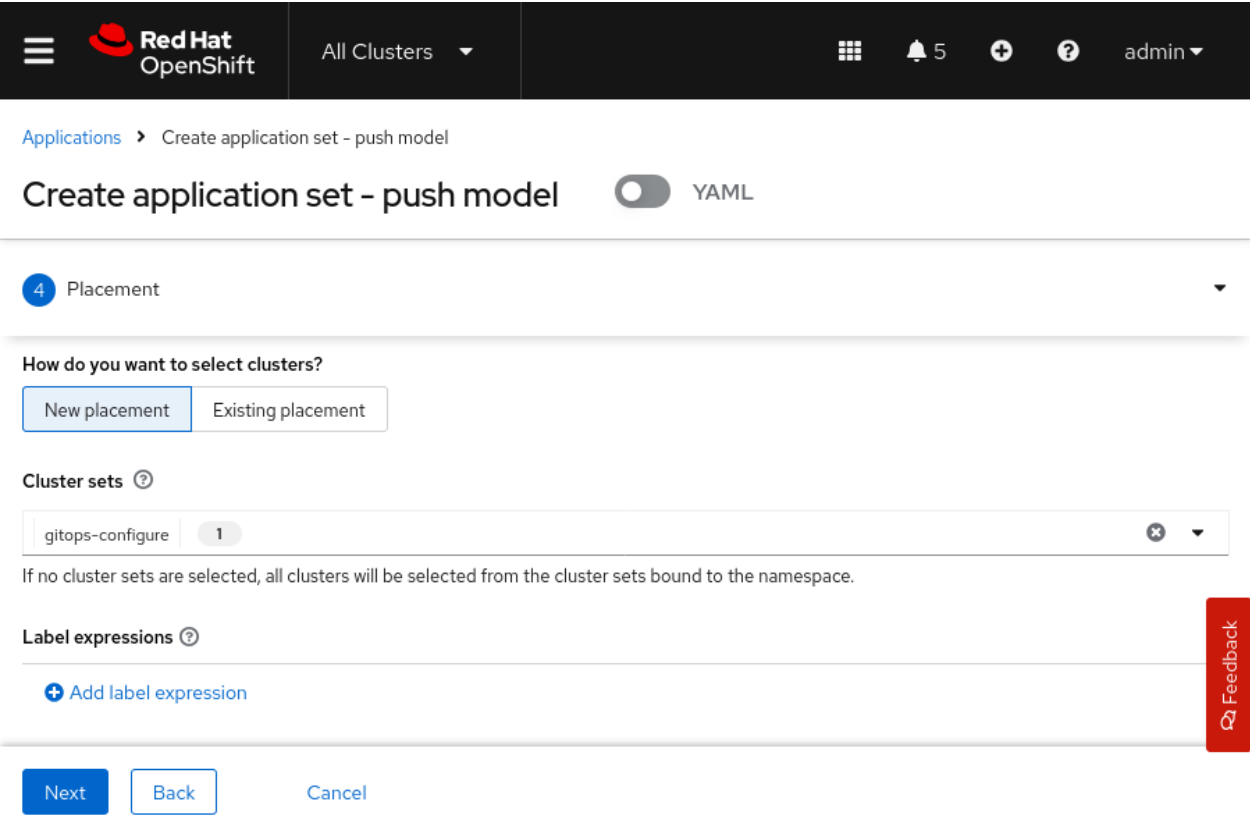


Figure 44. Placement page of the application set creation

From the **Review** section, you can review the settings that you configured.

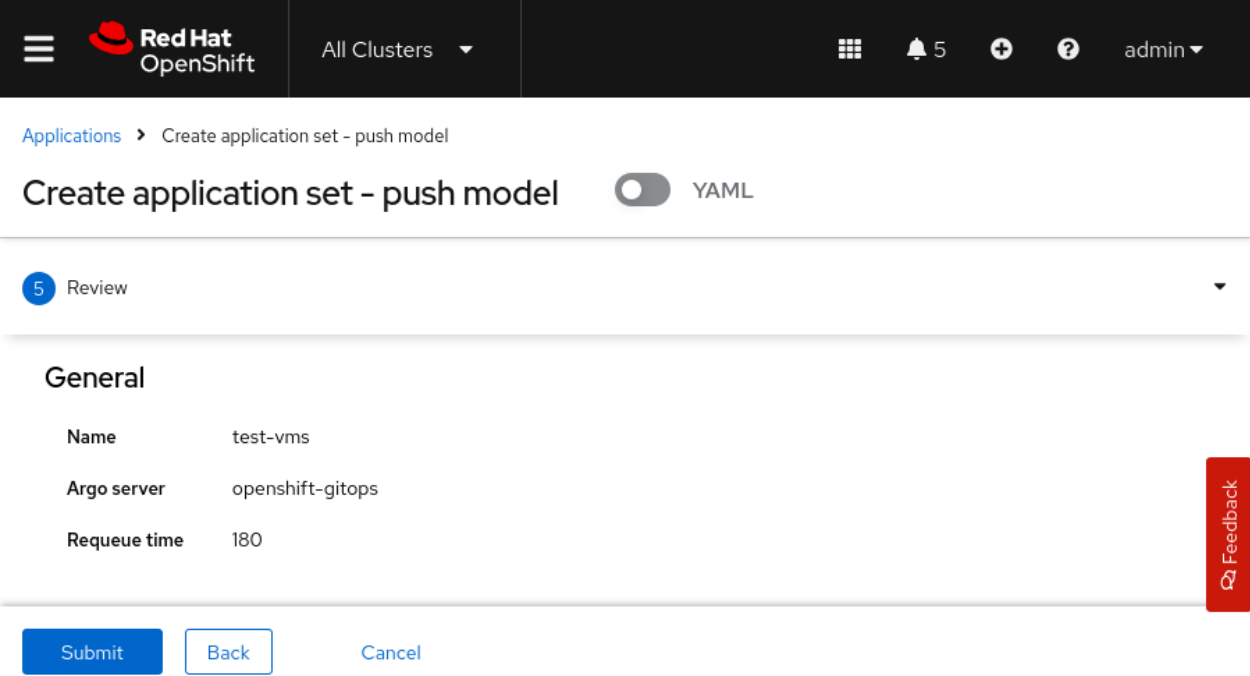


Figure 45. Review page of the application set creation

After you create the application set, click **Topology** to review the status of the resources.

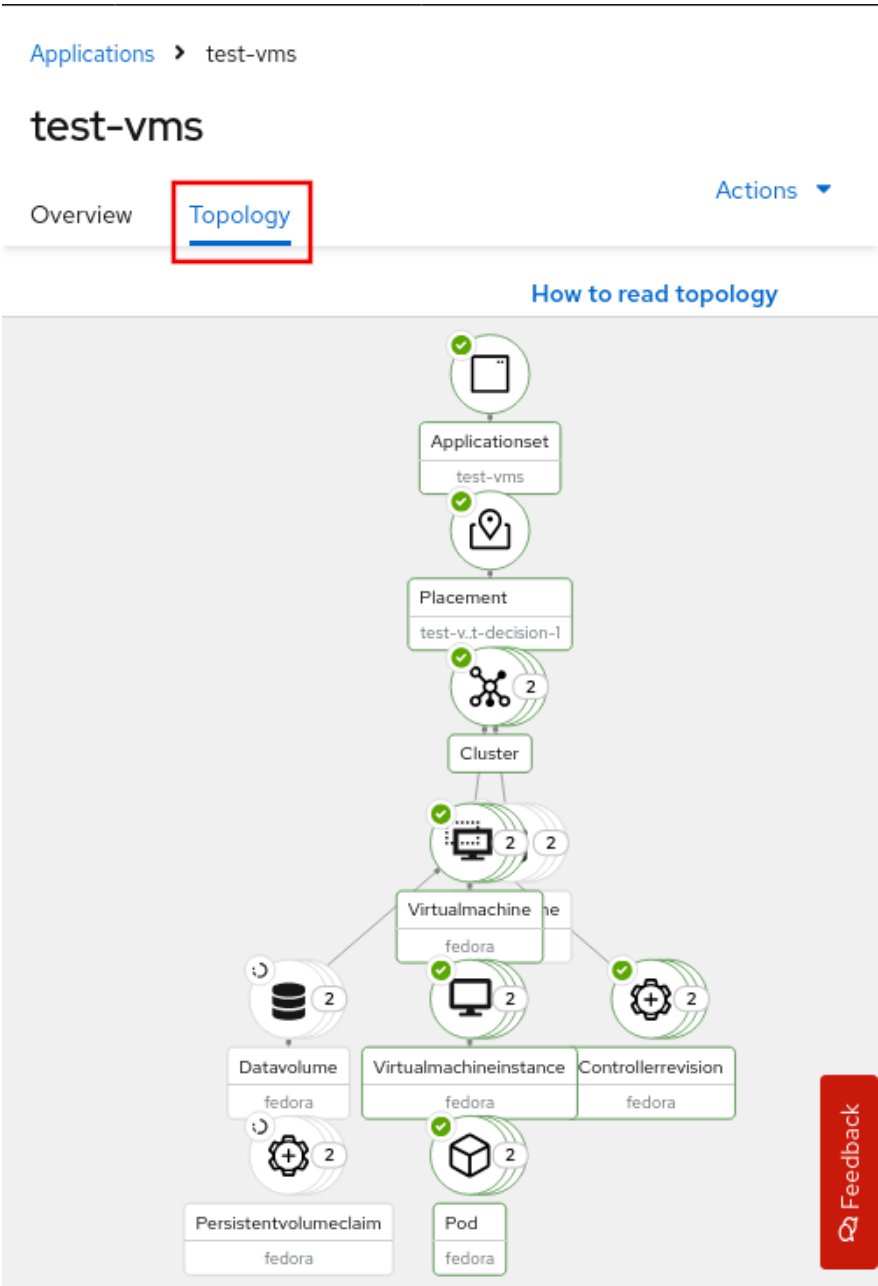


Figure 46. Topology of the application set

Note

In the **Topology** view, you might see the unknown status icon for DataVolume and PVC. It is a known display issue and does not affect the functioning of the application, and will be resolved in a future update.

Click the **Virtualmachine** icon to list the VM that the application set deployed. You can view the name of the VM, the namespace, and the cluster where the resource is deployed. Select a VM from the list to show more details of the VM and the YAML manifest.

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

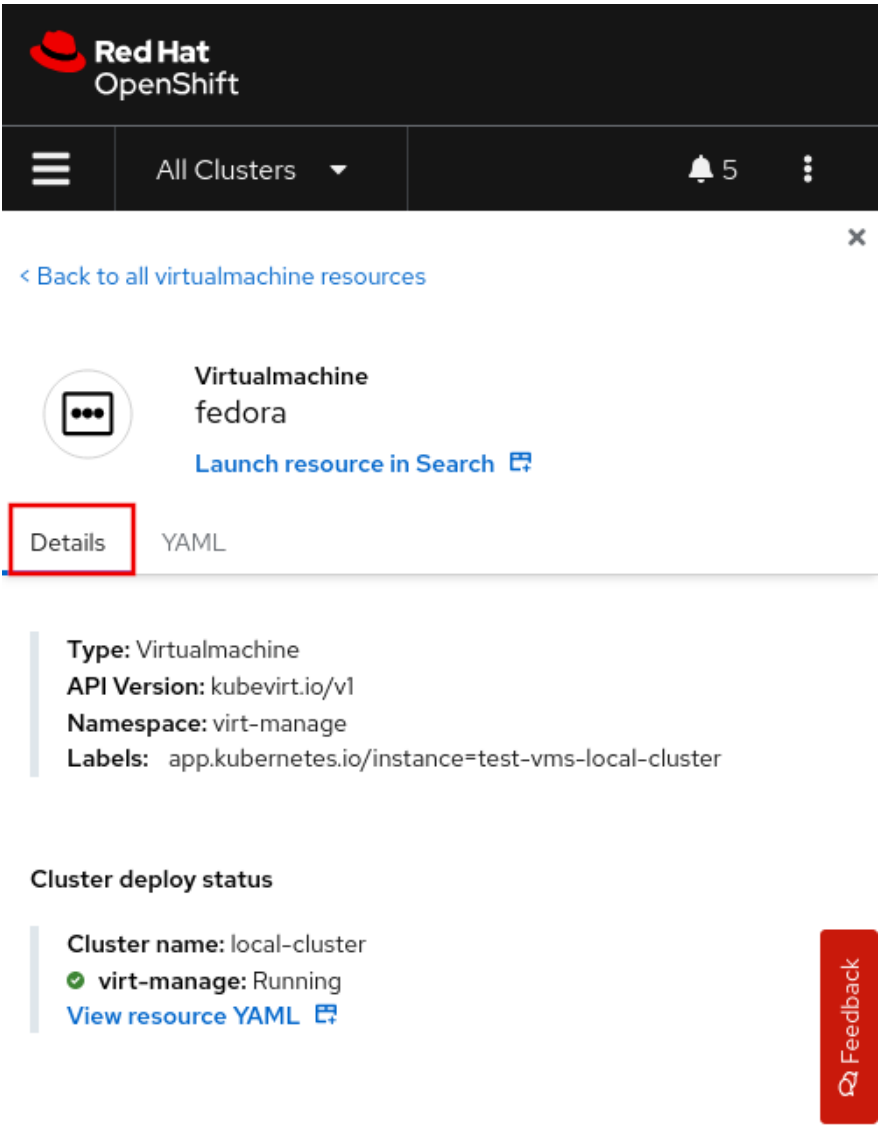


Figure 47. Details of the VM

From the **Topology** tab, you can view the status and information for the other resources that the application set created, such as the associated pods, the persistent volume claims, the data volumes, and more.

 **Note**
Click **How to read topology** to view the meaning of the status icons.

Removing Virtual Machines with an Application Set

You can delete the VMs and their related resources by removing the application set. Go to **Applications** and select the application set to delete. Click **Actions > Delete application** from the **Overview** tab.

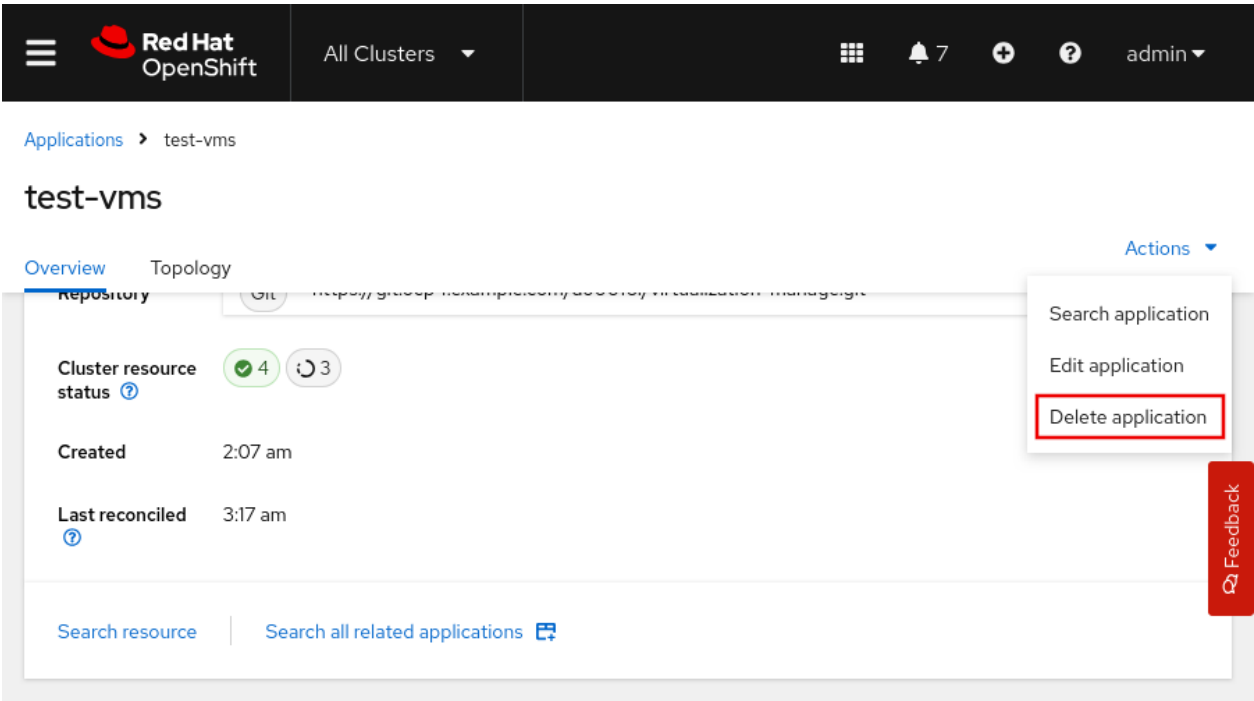


Figure 48. Actions menu

Select the Remove ApplicationSet related resources option and click **Delete**.

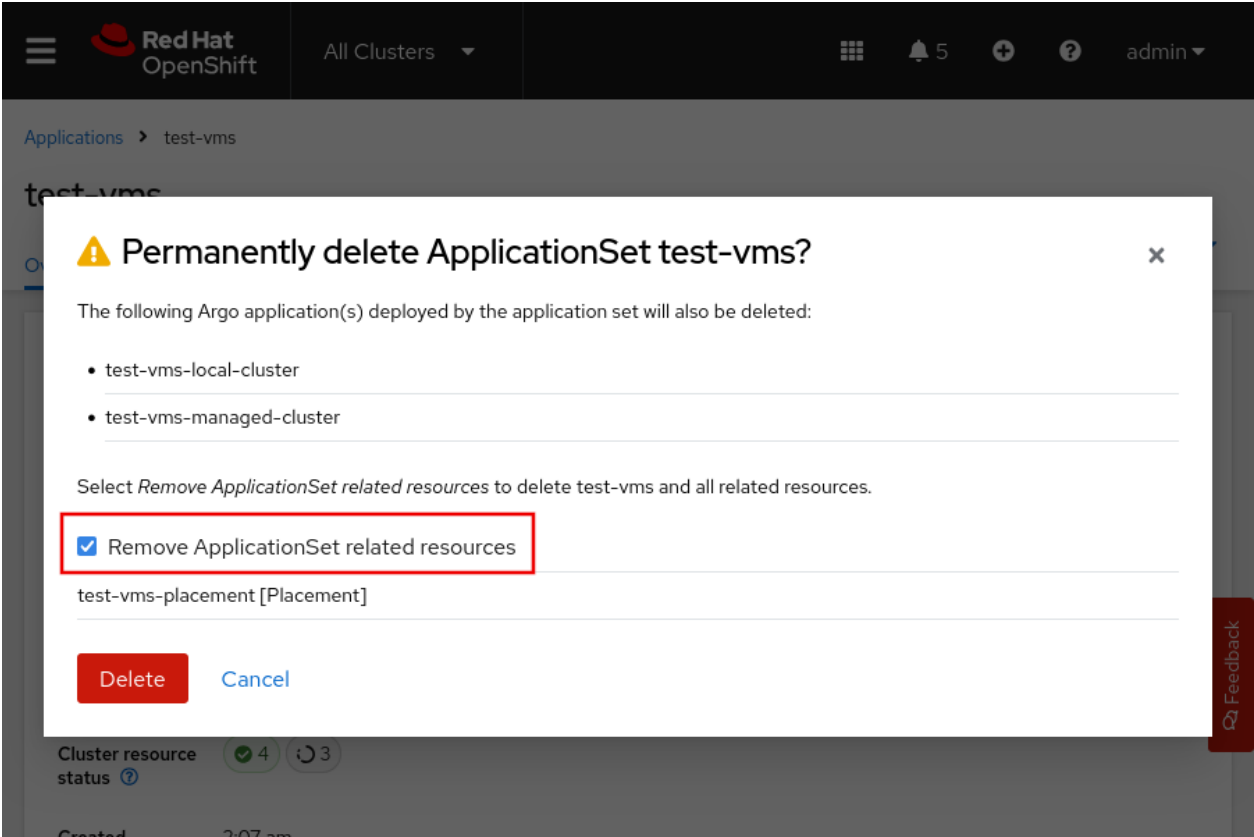


Figure 49. Delete the application set

Navigating the Virtual Machines Page

To review the configuration of the VMs, go to **Infrastructure > Virtual machines**. The page lists the available VMs and shows the information for each of the VMs, such as the status of the VM, the cluster where the VM is running, the namespace, the node, the IP address, and the launch links.

Name	Status	Cluster	Namespace	Node	IP address	Launch links
fedora	Running	local-cluster	virt-manage	master01	10.8.1.195	Launch links
fedora	Running	managed-cluster	virt-manage	master101	10.128.1.140	Launch links

Figure 50. List of available VMs

The Launch links option next to the IP address column gives two links. The Virtual machine details link redirects you to the **Overview** tab of the VM, and the Virtual machine console link redirects you to the **Console** tab of the VM. You can alternatively access these links from the **Details** tab of the VM that you selected.

From the **Details** tab, you can view the similar VM details from the previous page, with additional information such as the labels, the creation date, the annotations, and the owner.

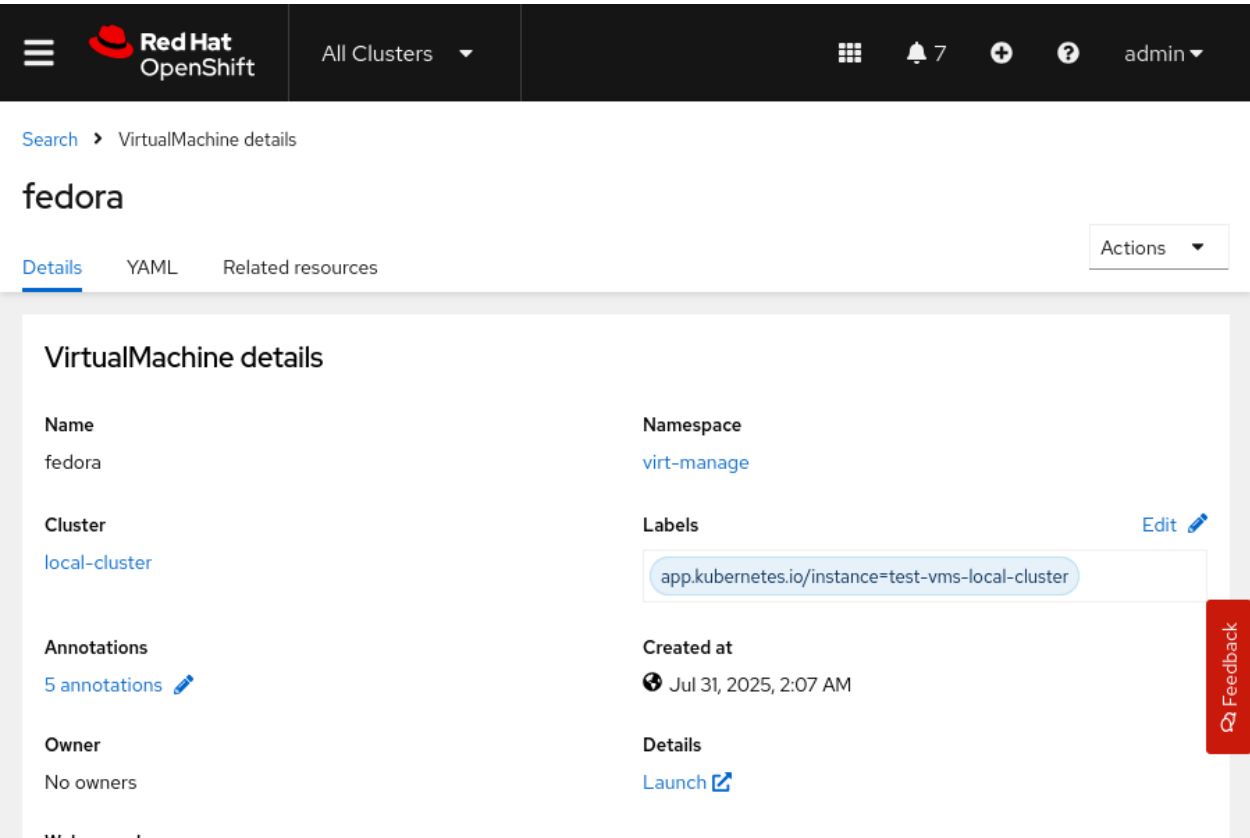


Figure 51. Details of the VM

The **Conditions** section lists the status, the date, and the message for each condition of the VM. The message describes the events of the related resources of the VM, such as the VMIs, the data volumes, and the PVC.

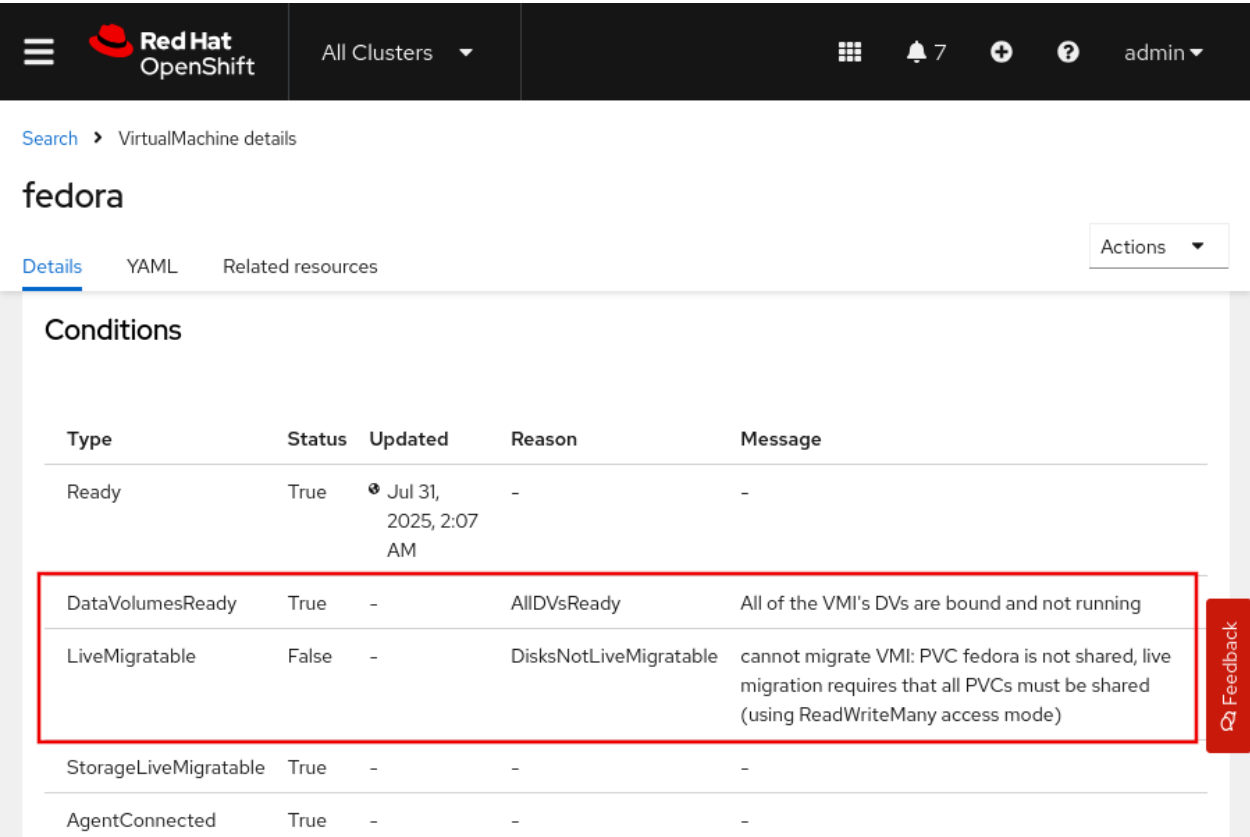


Figure 52. Conditions of the VM

From the **YAML** tab, you can review the YAML manifest of the VM. The **Related resources** tab shows the resources that are related to the VM, such as the application set, the cluster where the VM is running, the data volume, the PV and

PVC, the controller revision, the assigned pods, and the VMI. You can access the related resources to review their configuration.

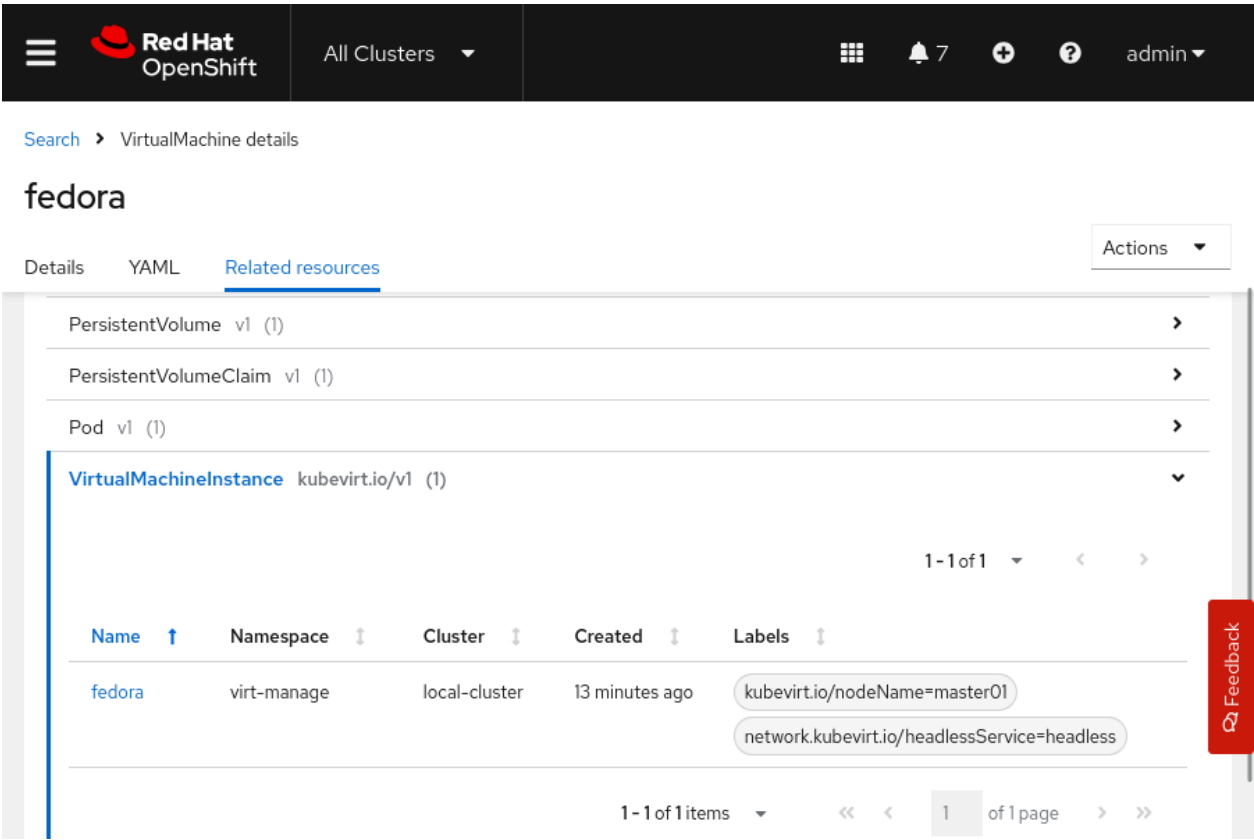


Figure 53. Related resources of the VM

From the **Details** tab, you can access the console of the VM by using the link in the **Web console** section. The link redirects you to the **Console** tab of the VM. You can access the **Overview** page of the VM by using the Launch link in the **Details** section.

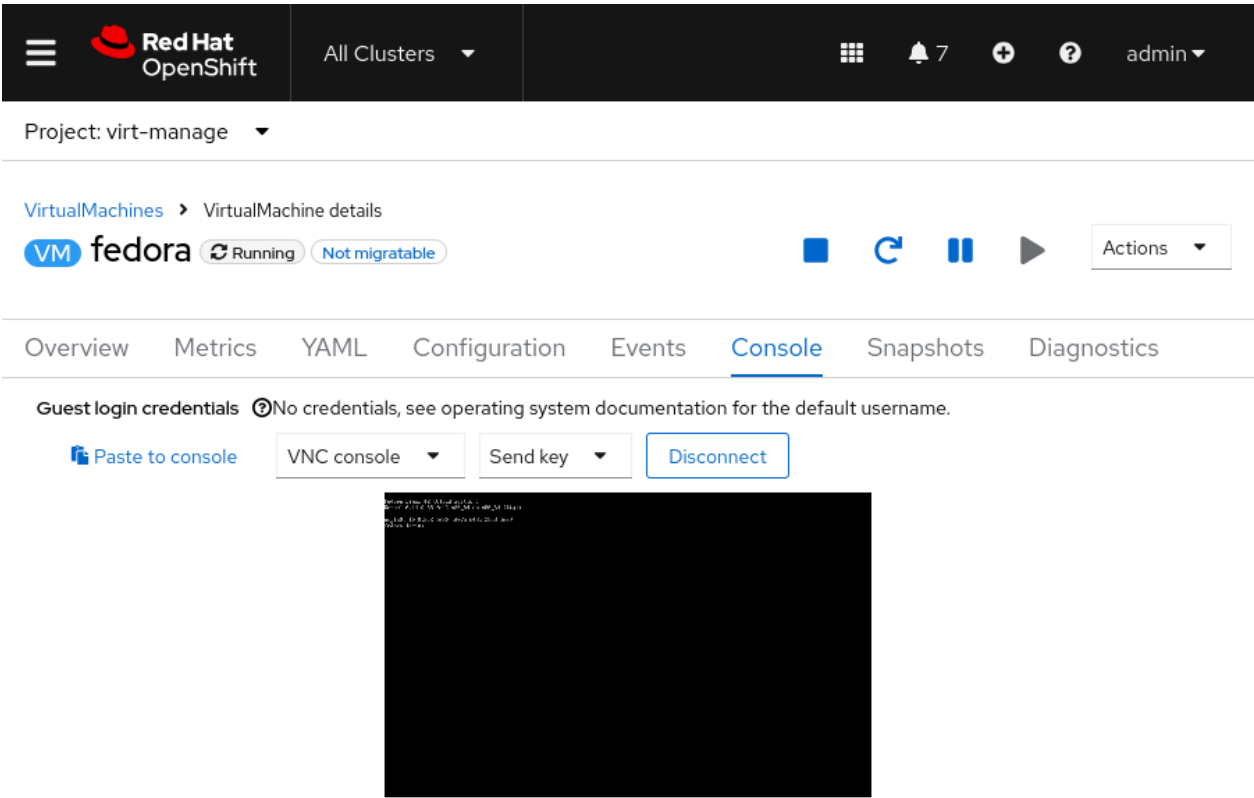


Figure 54. VM console

To update the VM, go to the **Details** tab and click **Actions > Edit VirtualMachine**. You can edit the YAML manifest of the VM to update the configuration.

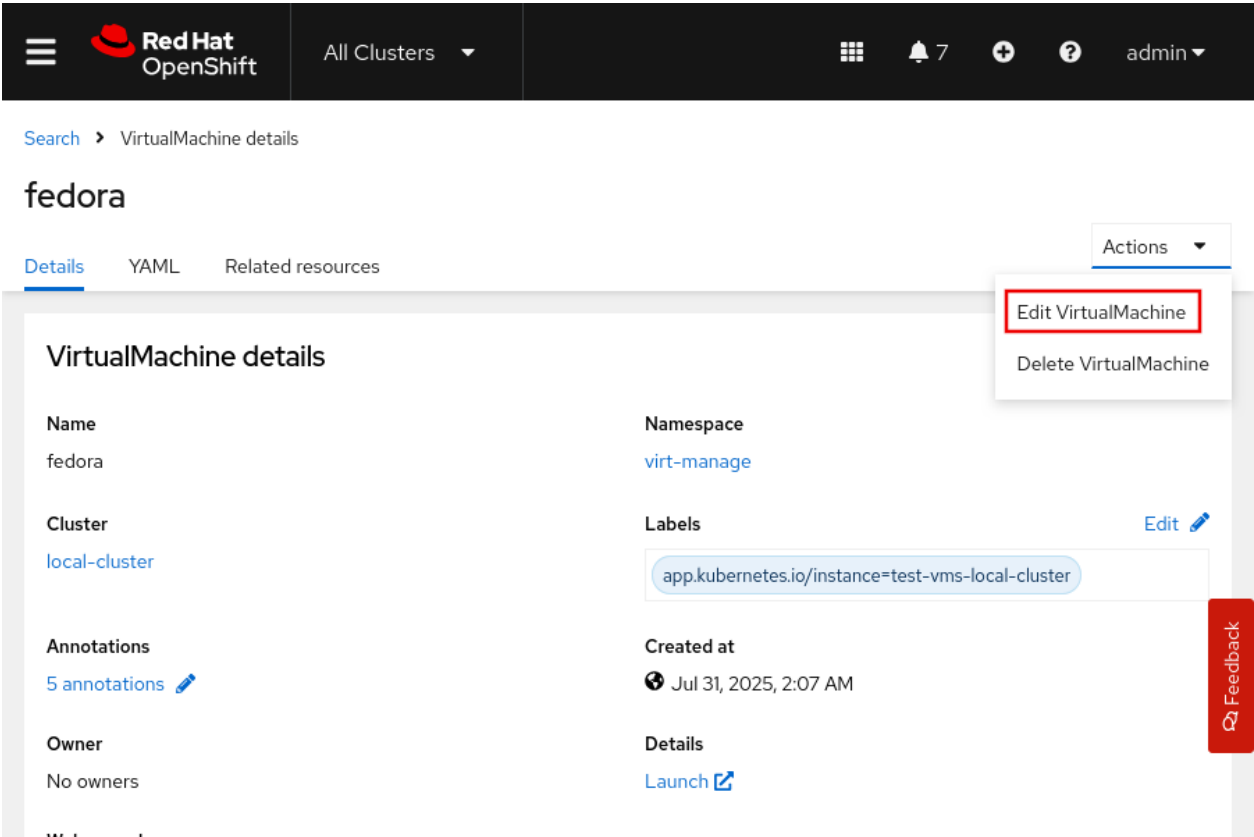


Figure 55. VM options

To delete the VM, click **Actions > Delete VirtualMachine**.

Enabling Virtual Machine Actions

Some VM actions, such as stop, restart, or pause VMs from the RHACM web console, are not enabled by default. Use the `oc annotate search search-v2-operator -n open-cluster-management virtual-machine-preview='true'` command to enable the VM actions.

```
[user@host ~]$ oc annotate search search-v2-operator -n open-cluster-management \
  virtual-machine-preview='true'
search.search.open-cluster-management.io/search-v2-operator annotated
```

Before running the previous command, ensure that the ManagedServiceAccount add-on is enabled in the MultiClusterEngine resource by using the following command:

```
[user@host ~]$ oc describe MultiClusterEngine multiclusterengine
...output omitted...
Spec:
  Availability Config: High
  Overrides:
    Components:
      Config Overrides:
        Enabled: true
        Name: local-cluster
    ...output omitted...
      Config Overrides:
        Enabled: true
        Name: hypershift
      Config Overrides:
        Enabled: true
        Name: managedserviceaccount
    ...output omitted...
```

 **Note**

You can list the configured MultiClusterEngine resource by using the `oc get MultiClusterEngine` command.

After you run the `oc annotate` command to enable the VM actions, go to **Infrastructure > Virtual machines** and click **Actions** to show the new available options that you can apply to the VM. The `runStrategy` field in the YAML manifest of the VM must be set to `Manual` to enable the start, restart, stop, or pause actions.

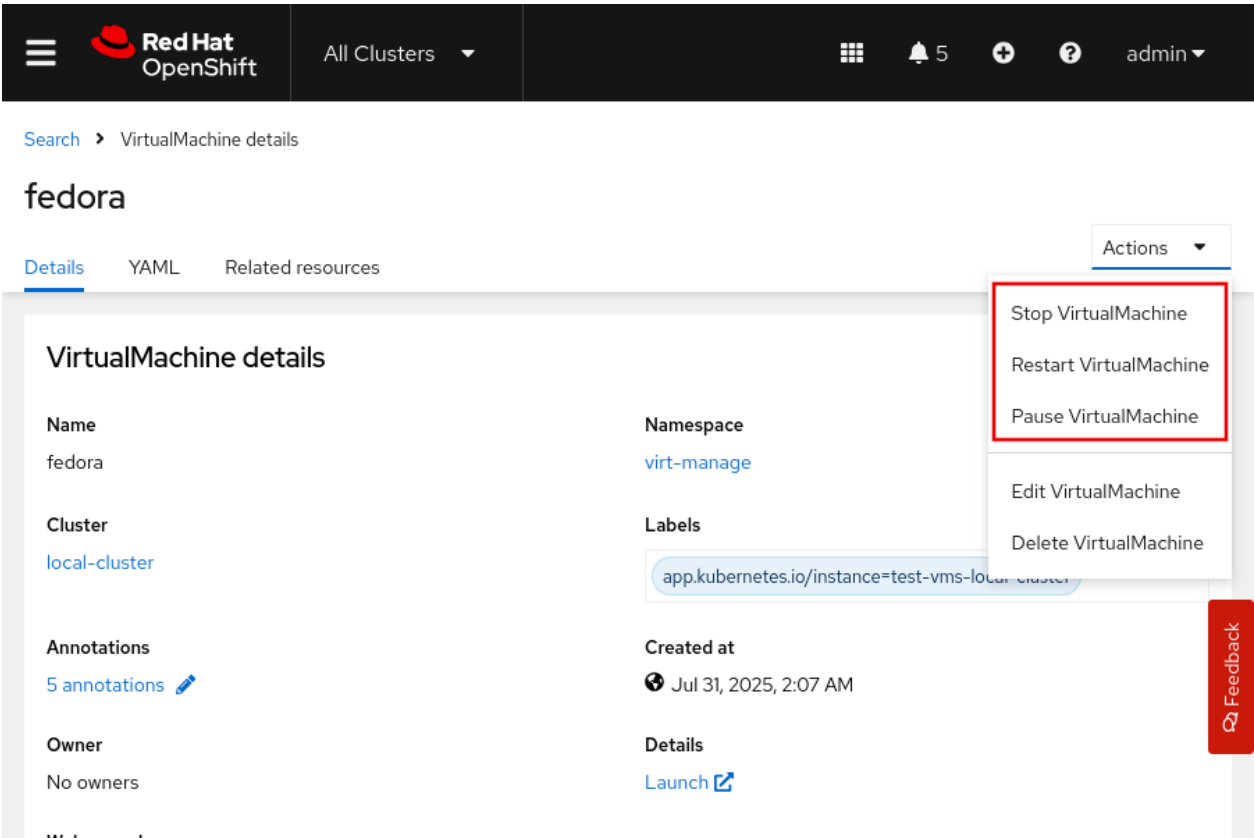


Figure 56. VM new options

To disable the previous VM actions, set the false option to the `virtual-machine-preview` label and use the `--overwrite` option:

```
[user@host ~]$ oc annotate search search-v2-operator -n open-cluster-management \
  virtual-machine-preview='false' --overwrite
search.search.open-cluster-management.io/search-v2-operator annotated
```

Note
The VM actions that you can use for your VMs are a technology preview feature.

References
[Migrate Virtual Machines with Red Hat](#)

[Using Red Hat Advanced Cluster Management and OpenShift GitOps to manage OpenShift Virtualization](#)

For more information about the VM actions in RHACM, refer to the *Enabling Virtual Machine Actions* section in the *Virtualization* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Virtualization* documentation at https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/virtualization/index#enable-vm-actions

6.6. Guided Exercise

Deploy and Manage Multicenter Virtual Machines with RHACM

Deploy and manage virtual machines across a multicenter environment by using RHACM.

Outcomes

- Create an application set to deploy a VM to the clusters in the configured cluster set.
- Use the VMs page to perform VM operations.

Prerequisites

As the student user on the workstation machine, use the lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab install do0016l ; lab start virtualization-manage
```

Instructions

The lab start command deploys the GitOps operator, installs the OpenShift Virtualization operator, and creates a cluster set named gitops-configure that contains the hub cluster and the managed cluster.

1. Log in to the RHOC hub cluster, and switch to the RHACM web console.
 - 1.1. Open a web browser and go to `https://console-openshift-console.apps.ocp4.example.com`
 - 1.2. Select the **Red Hat Identity Management** identity provider and log in as the admin user with redhatocp as the password.
 - 1.3. Ensure that the cluster switcher is set to **All Clusters**, which corresponds to the RHACM web console.
2. Create an application set to deploy VMs from a Git repository.
 - 2.1. Go to **Applications** and select **Create application > Argo CD ApplicationSet - Push model**.
 - 2.2. Set the name of the application to `virtualization-manage`, select the openshift-gitops Argo CD server, leave the default value for **Requeue**, and click **Next**.
 - 2.3. Configure the Git repository as follows, and click **Next**.

Parameter	Value
Type	Git
URL	https://git.ocp4.example.com/do0016l/virtualization-manage.git
Revision	main
Path	virtualization/manifests

Parameter	Value
Remote namespace	virt-manage

Red Hat OpenShift

All Clusters

5

admin

Applications > Create application set - push model

Create application set - push model

YAML

2 Template

URL *

https://git.ocp4.example.com/do0016l/virtualization-manage.git

Revision

main

Path

virtualization/manifests

Next

Back

Cancel

Feedback

2.4. In the Sync policy section, leave the default values and click **Next**.

2.5. Create a placement to select the gitops-configure cluster set. Set **Limit the number of clusters selected** to 2. Click **Next**.

Page 419 | 475

DO432-RHACM2.13-en-2-20260109

Red Hat OpenShift

All Clusters

5

admin

Applications > Create application set - push model

Create application set - push model

YAML

4 Placement

Cluster sets

gitops-configure 1

If no cluster sets are selected, all clusters will be selected from the cluster sets bound to the namespace.

Label expressions

+ Add label expression

Limit the number of clusters selected

-

2

+

Next

Back

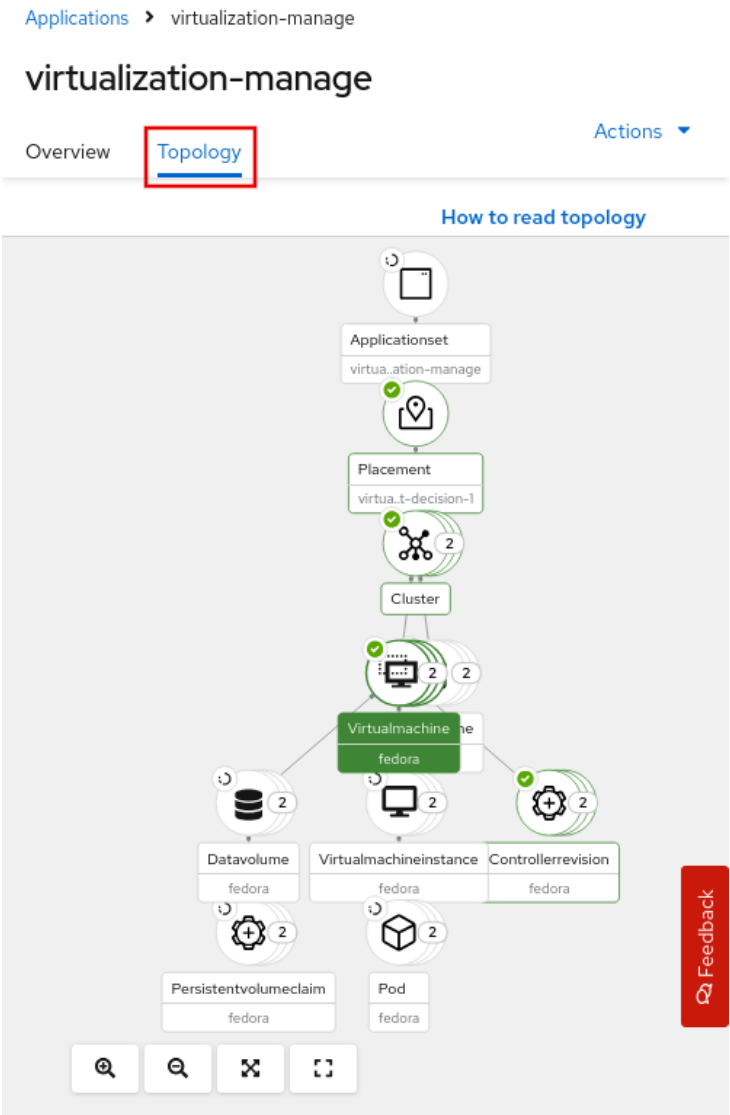
Cancel

Feedback

2.6. Review your configuration and click **Submit**.

3. Verify the deployment of the VMs.

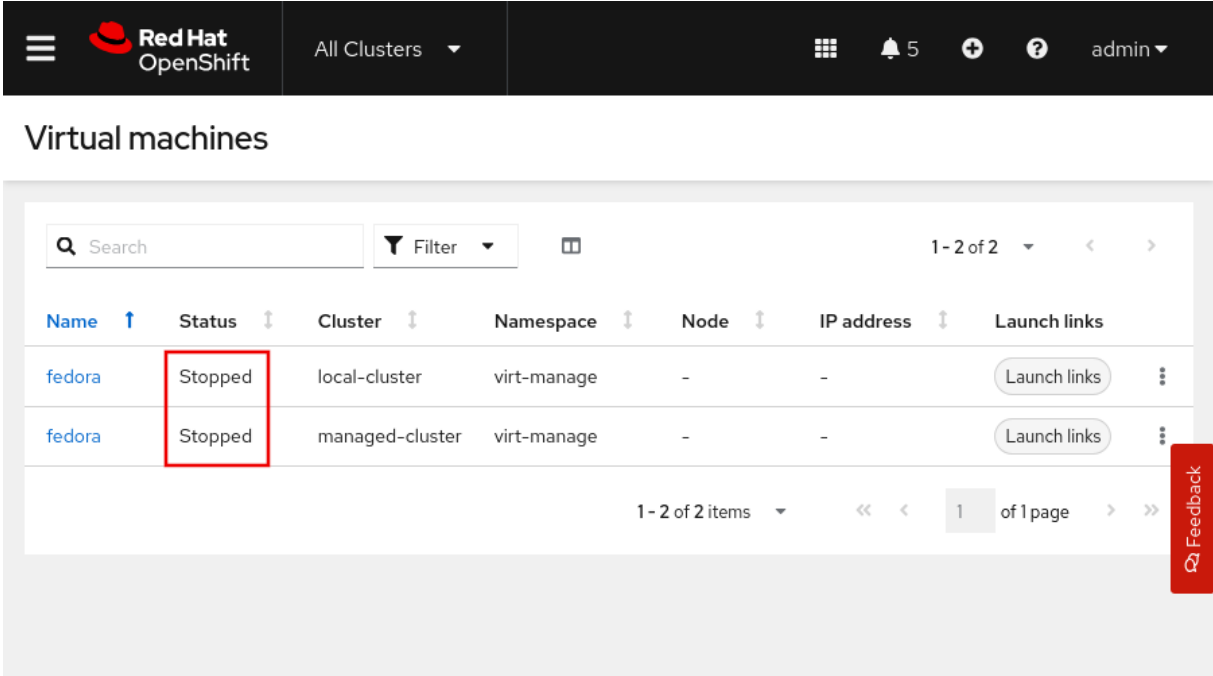
3.1. Click the **Topology** tab and wait up to six minutes for the topology to report the application ready status. The VMI and Pods resources are not available because the VMs are stopped by default.



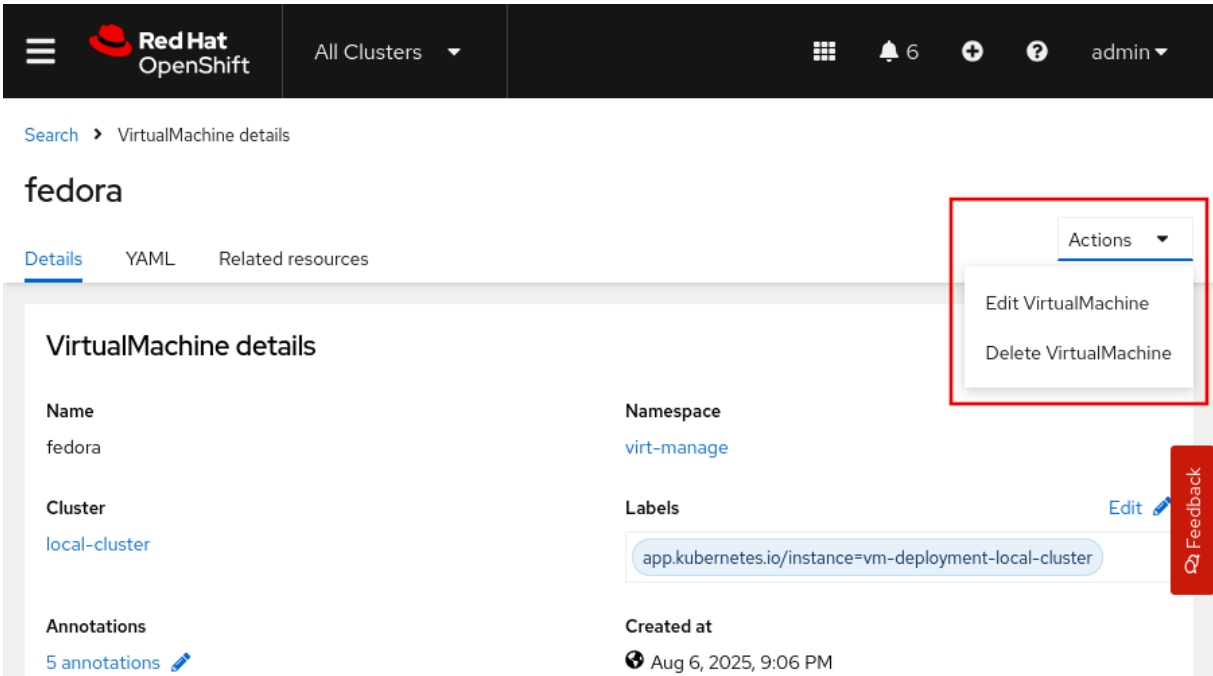
Note

In the **Topology** view, you might see the unknown status icon for DataVolume and PVC. It is a known display issue and does not affect the functioning of the application, and will be resolved in a future update.

3.2. Go **Infrastructure > Virtual machines** and verify that the VMs appear in the list with the Stopped status.



4. Enable the VM actions in the RHACM web console.
- 4.1. Select the `fedora` VM that is deployed in the `local-cluster` cluster. Verify that the stop, restart, and pause actions in the `Actions` menu are not listed.



- 4.2. In the terminal window, log in to the hub cluster as the `admin` user with `redhatocp` as the password.

```
[student@workstation ~]$ oc login -u admin -p redhatocp \
https://api.ocp4.example.com:6443
Login successful
...output omitted...
```

- 4.3. List the available `MultiClusterEngine` resource.

```
[student@workstation ~]$ oc get MultiClusterEngine
NAME                STATUS    AGE      CURRENTVERSION    ...
multicenterengine   Available  166m     2.8.0              ...
```

- 4.4. Use the `oc describe MultiClusterEngine multicenterengine` command to verify that the ManagedServiceAccount add-on is enabled. From the Overrides section, confirm that for the managedserviceaccount option, the true value is assigned to the Enabled field.

```
[student@workstation ~]$ oc describe MultiClusterEngine \
  multicenterengine
...output omitted...
Spec:
  Availability Config: High
Overrides:
  Components:
    Config Overrides:
      Enabled: true
      Name: local-cluster
...output omitted...
    Config Overrides:
      Enabled: true
      Name: hypershift
    Config Overrides:
      Enabled: true
      Name: managedserviceaccount
...output omitted...
```

- 4.5. Use the `oc annotate` command to enable the VM actions.

```
[student@workstation ~]$ oc annotate search \
  search-v2-operator -n open-cluster-management \
  virtual-machine-preview='true' --overwrite
search.search.open-cluster-management.io/search-v2-operator annotated
```

5. Start the fedora VM that is deployed in the local-cluster cluster from the RHACM web console.

- 5.1. Go to **Infrastructure > Virtual machines** and select the fedora VM in the local-cluster cluster.
- 5.2. Click the **Actions > Start VirtualMachine** option to start the VM. If the option is not available, then wait a few minutes and try again.

Red Hat OpenShift

All Clusters

7

admin

Search

VirtualMachine details

fedora

DetailsYAMLRelated resources

VirtualMachine details

Name

fedora

Cluster

local-cluster

Annotations

5 annotations

Namespace

virt-manage

Labels

app.kubernetes.io/instance=vm-deployment-local-cluster

Created at

Aug 6, 2025, 9:06 PM

Actions

Start VirtualMachine

Restart VirtualMachine

Pause VirtualMachine

Edit VirtualMachine

Delete VirtualMachine

Feedback

Note

The VM actions are available only for the VM in the hub cluster. This issue is known from the technology preview feature.

5.3. Go to **Infrastructure > Virtual machines** and wait for the VM to have the Running status.

Red Hat OpenShift

All Clusters

6

admin

Virtual machines

Search

Filter

1 - 2 of 2

Name	Status	Cluster	Namespace	Node	IP address	Launch links
fedora	Running	local-cluster	virt-manage	master01	10.8.0.233	Launch links
fedora	Stopped	managed-cluster	virt-manage	-	-	Launch links

1 - 2 of 2 items

1 of 1 page

Feedback

Finish

On the workstation machine, use the lab command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish virtualization-manage
```

6.7. Monitoring OpenShift Virtualization with RHACM

Objectives

- Describe the OpenShift Virtualization dashboards on the Cluster Monitoring Operator.

OpenShift Virtualization Observability

As organizations increasingly adopt OpenShift Virtualization for critical workloads and across multiple clusters, the need for robust and scalable monitoring becomes crucial. RHACM uses the Cluster Monitoring Operator to add a set of detailed OpenShift Virtualization dashboards to the observability service.

The dashboards provide system administrators with actionable insights, to help them to optimize resources, improve troubleshooting, and ensure smooth operations across clusters.

RHACM OpenShift Virtualization Dashboards

RHACM provides the following OpenShift Virtualization dashboards in Grafana:

- *Clusters Overview*
- *Single Cluster View*
- *Single Virtual Machine View*
- *Virtual Machines Inventory*
- *Virtual Machines by Time in Status*

To access the dashboards in Grafana, from the RHACM web console, go to **Infrastructure > Virtual machines** and click the **Observability dashboards** link in the upper-right corner of the page.

Alternatively, you can go to **Infrastructure > Clusters** and click the **Grafana** link in the upper-right corner of the page. From Grafana, go to **Home > Dashboards > ACM / OpenShift Virtualization**.

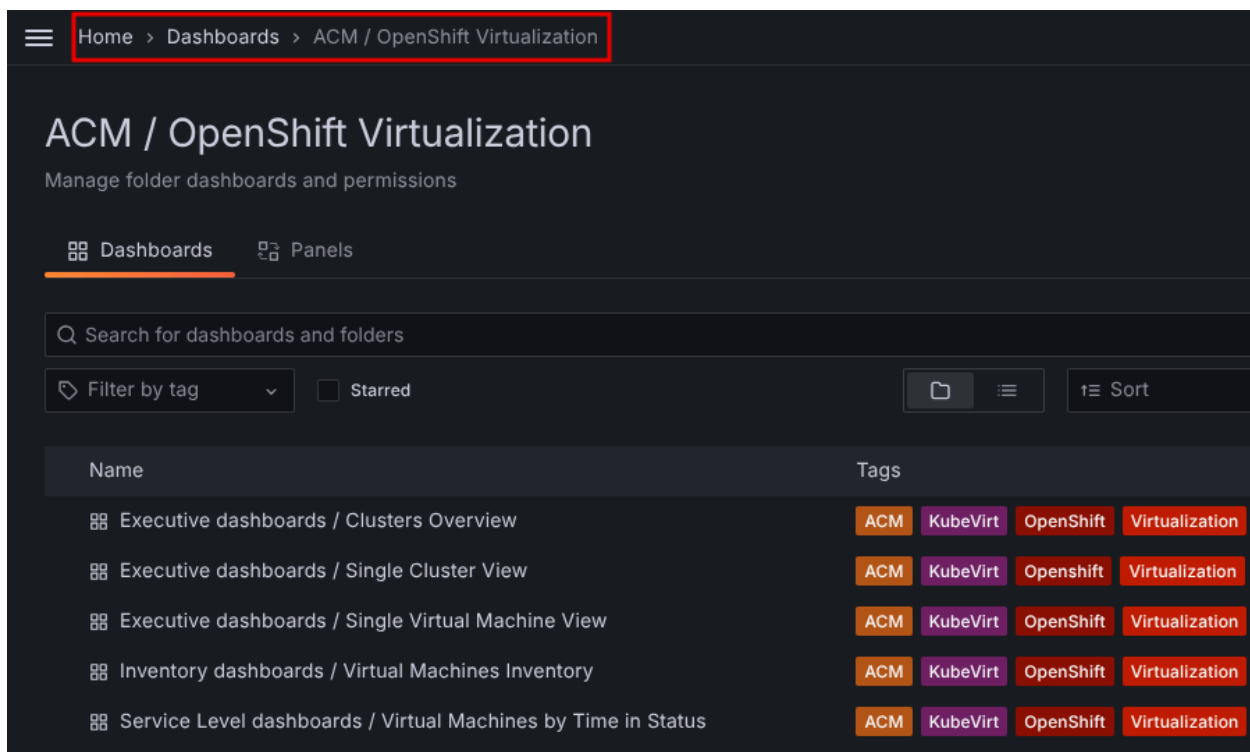


Figure 57. OpenShift Virtualization dashboards

Clusters Overview Dashboard

The *Clusters Overview* dashboard aggregates and displays information from all OpenShift Virtualization clusters across the RHACM management fleet, to showcase the entire virtualization infrastructure's capacity. The dashboard offers significant value to the Operations team, who manage thousands of VMs throughout the fleet.

The dashboard divides the information into the following sections:

- General information
- Operator health
- Additional virtual machine details
- Resources utilization

At the upper-right corner of the dashboard are two drop-down lists, to filter the information by cluster or by operator health.

General Information Section

The section shows the following infrastructure data details:

- Number of clusters that run OpenShift Virtualization
- Clusters in critical health
- Clusters in warning health
- Number of allocatable nodes
- Number of VMs
- Number of VMs by status (running, stopped, error, starting, and migrating)
- Number of VMs that were started in the last seven days
- Clusters by operator version
- Clusters by OpenShift version

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

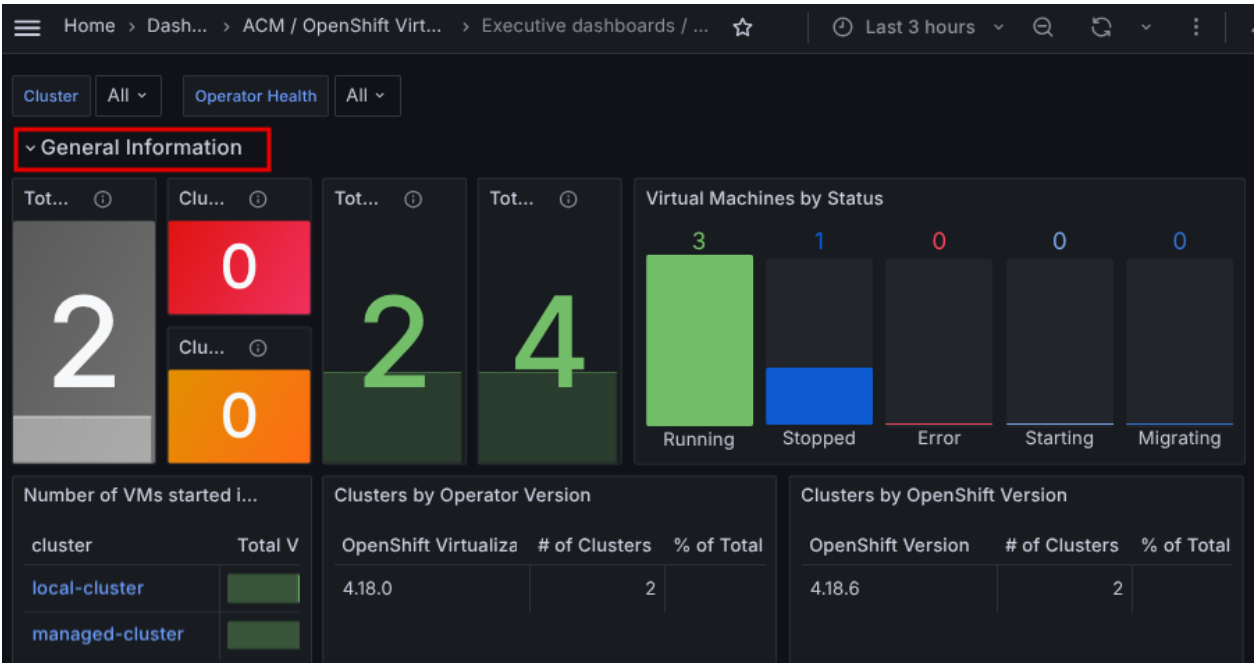


Figure 58. Cluster overview dashboard, general information

Operator Health Section

This section provides the following information about the OpenShift Virtualization operator:

- Operator health (healthy, warning, and critical)
- Alerts with critical impact
- Alerts with warning
- Number of running VMs per cluster

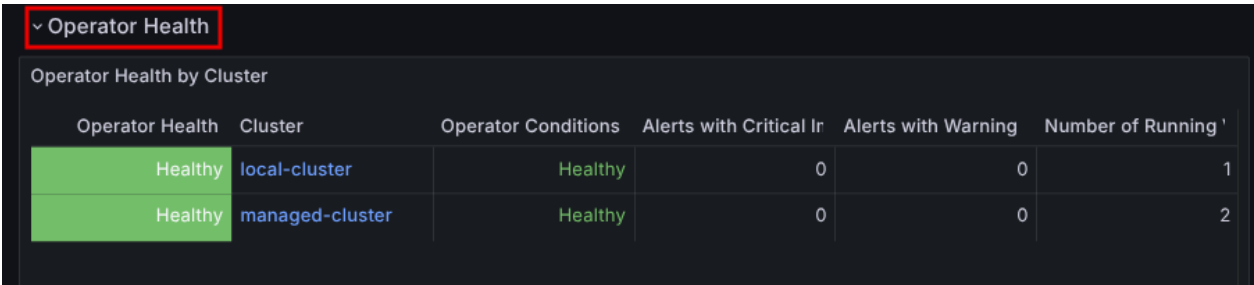


Figure 59. Cluster overview dashboard, operator health

Additional Virtual Machine Details Section

The section displays the details across all virtual machines regarding their statuses and operating systems (OS). Two timeline graphs are shown: one that displays the top 20 running VMs per node, and another that displays the top 20 running VMs per cluster.

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

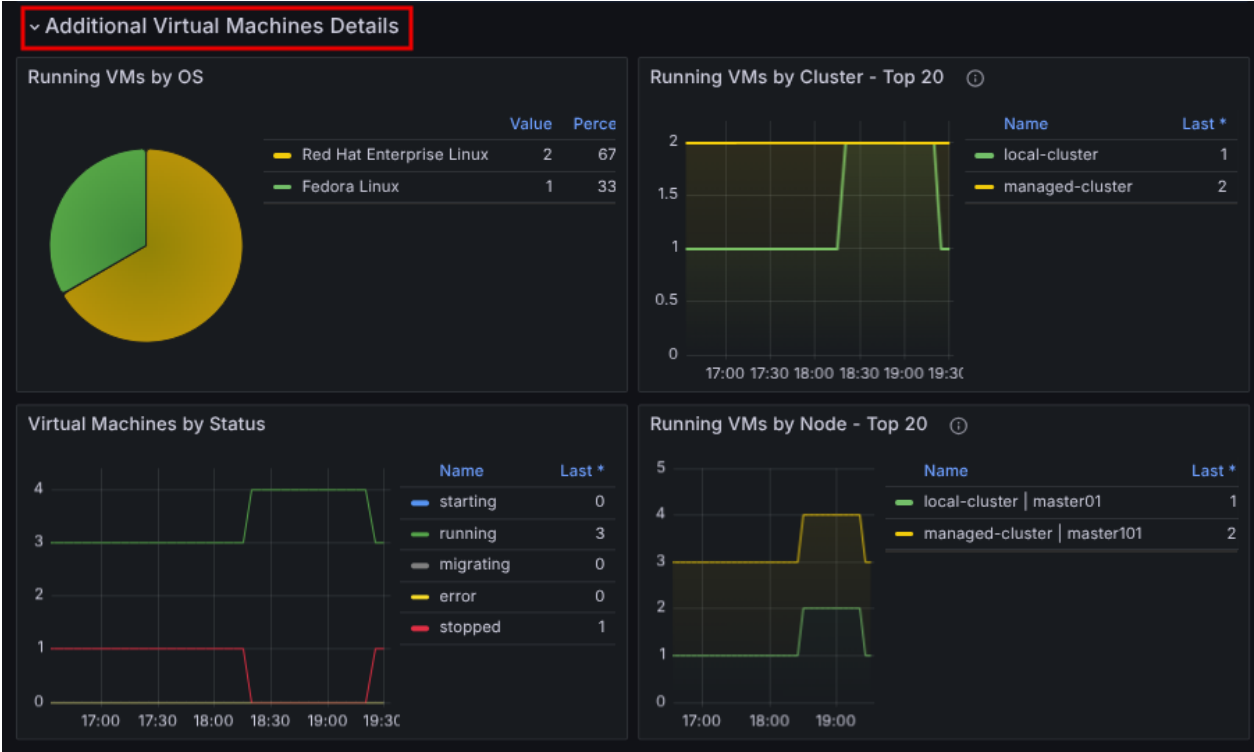


Figure 60. Cluster overview dashboard, additional virtual machine details

Resources Utilization Section

This section gives the detailed history of resource consumption (CPU, memory, network receive, network transmit, and storage IOPS) per cluster. This information helps identify potential bottlenecks, idle resources, or even unexpected consumption peaks per cluster.

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

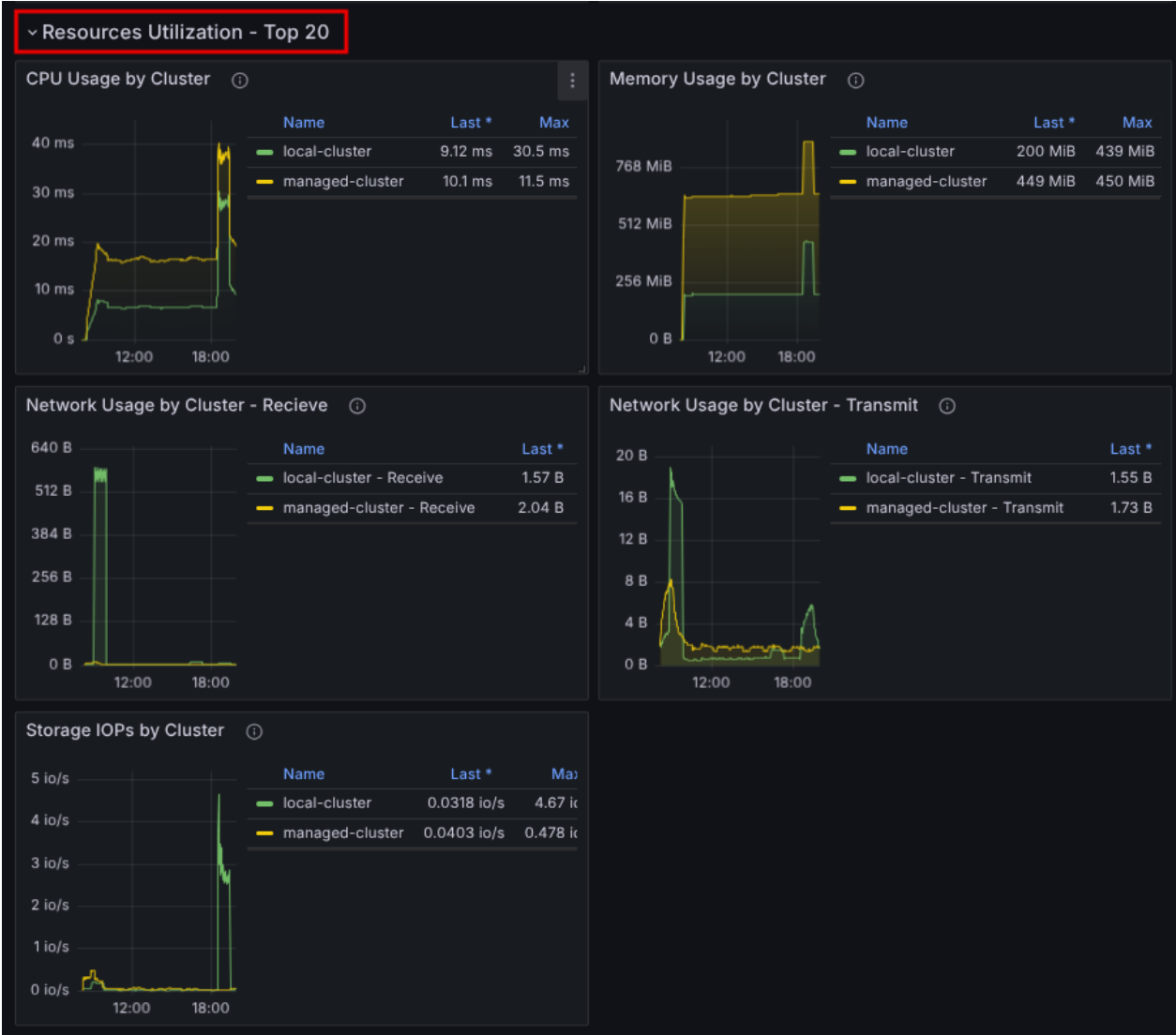


Figure 61. Cluster overview dashboard, resources utilization

Single Cluster View Dashboard

The Single Cluster View dashboard focuses on detailed visualizations in a specific OpenShift Virtualization cluster. At the upper-right corner of the dashboard are three drop-down lists, to filter the information by cluster, by alert impact on operator health, and by alert severity.

The Single Cluster View dashboard and the Clusters Overview dashboard connect to each other. For a closer examination of a single OpenShift Virtualization cluster, click the cluster's name on any relevant panel within the Clusters Overview dashboard. This action redirects you to the Single Cluster View dashboard for that cluster.

The dashboard divides the information into the following sections:

- General information
- Additional virtual machine details
- Resources utilization
- Alerts

General Information Section

The section provides an overview of the following information:

- Cluster name

- The provider that the cluster is running on
- OpenShift Virtualization version
- OpenShift cluster version
- Operator status
- Operator conditions
- Number of nodes that are used to provision VMs
- Number VMs
- Number of VMs by status (running, stopped, error, starting, and migrating)
- Number VMs by OS
- Recent started VMs with their corresponding namespace, and their uptime

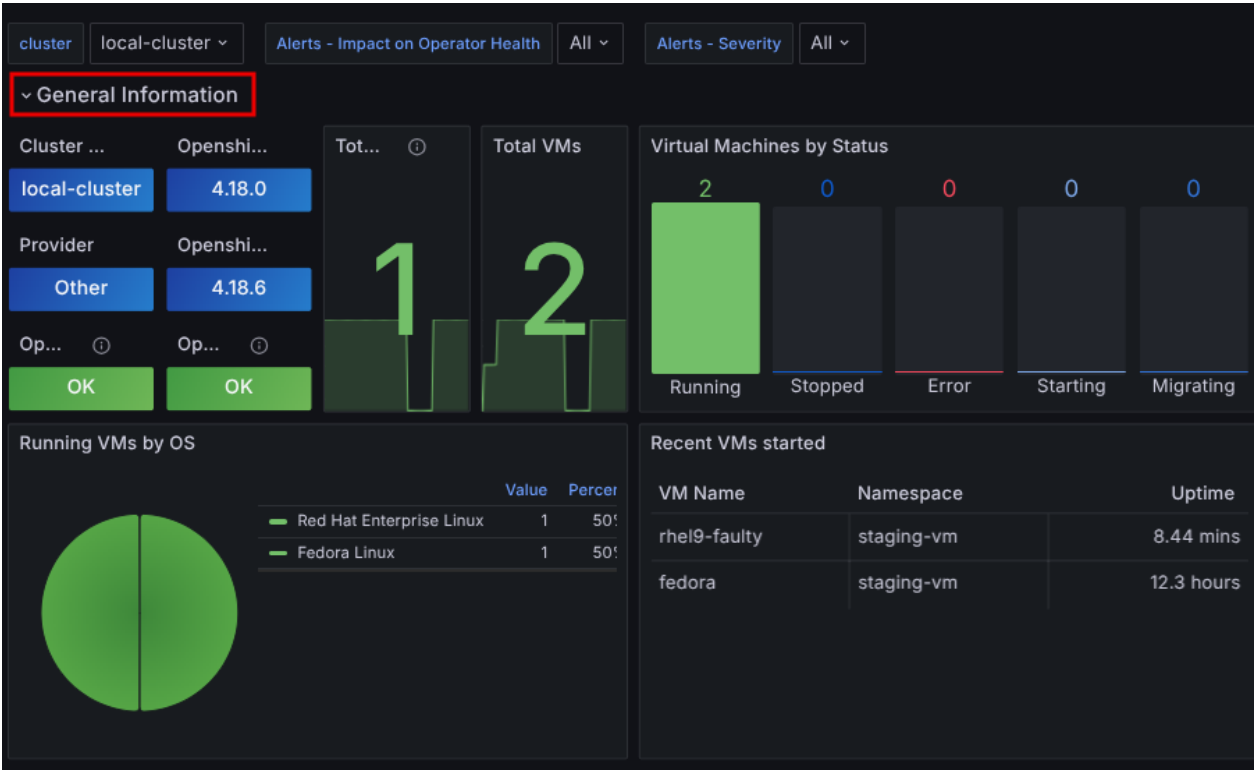


Figure 62. Single Cluster View dashboard, general information

Additional Virtual Machine Details Section

This section shows two timeline graphs of the number of VMs that are running, one per node and the other by status.

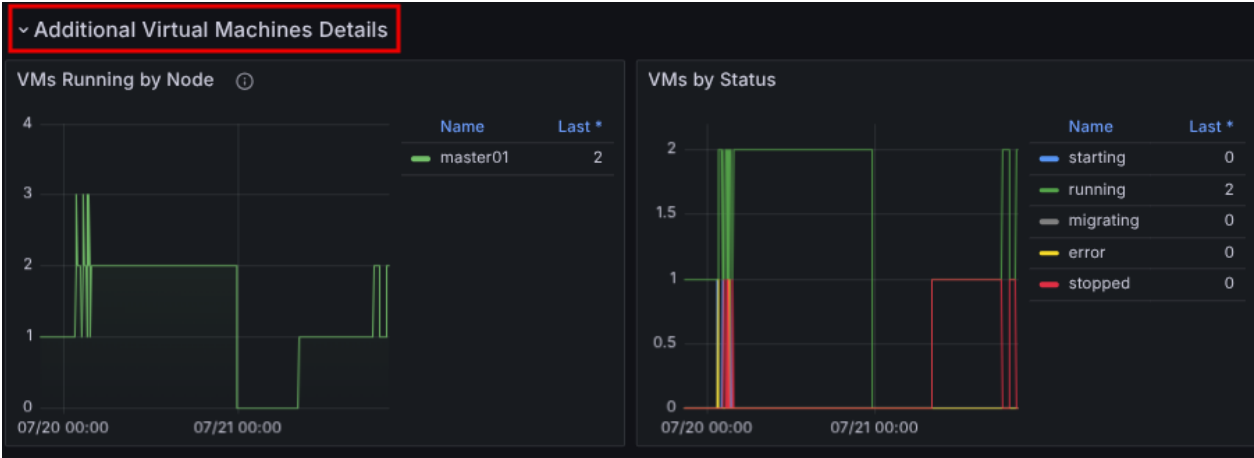


Figure 63. Single Cluster View dashboard, additional virtual machine details

Resources Utilization Section

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

This section is similar to the *Resources Utilization* section within the *Clusters Overview* dashboard; here, the information in the graphs is aggregated per node and per virtual machine. The graphs help you identify both on the VMs and on the nodes, potential bottlenecks, idle resources, or unexpected consumption peaks.

You see the following information in the graphs:

- CPU usage by each node and VM
- Memory usage by each node and VM
- Network usage by each node and VM (both incoming and outgoing traffic)
- Storage usage in IOPs by each node and VM



Figure 64. Single Cluster View dashboard, resources utilization

Alerts Section

This section displays the main alerts, status, and health of the OpenShift Virtualization operator, and all virtualization-related alerts in a specific cluster.

The section divides the information into the following groups:

- Number of alerts by severity (critical, warning, and information)
- Number of alerts that impact operator health
- Number of operator Cluster Service Version issues
- A table that lists alerts with their information

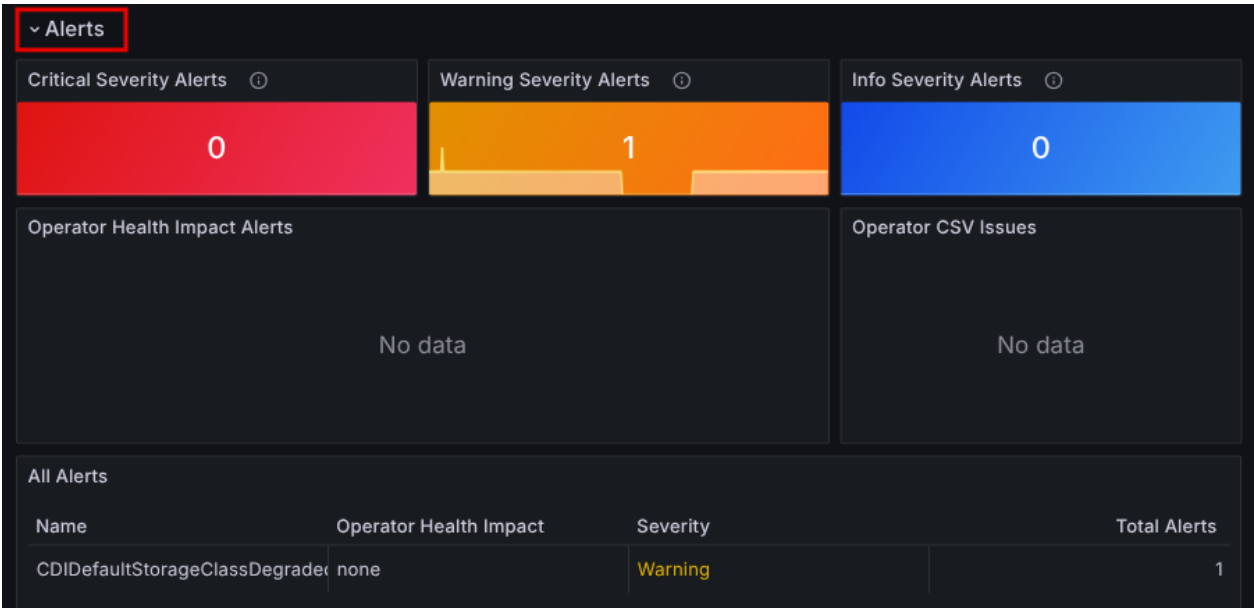


Figure 65. Single Cluster View dashboard, resources utilization

Single Virtual Machine View Dashboard

The Single Virtual Machine View dashboard enables system administrators to track and manage individual VMs. It provides a detailed snapshot of a specific VM, including its configuration, resource usage, and any active alerts.

At the upper-right corner of the dashboard are three drop-down lists, to filter the information by VM name, by namespace, or by cluster.

The dashboard divides the information into the following sections:

- General information
- Resources
- Alerts

General Information Section

This section shows the VM status and details, such as the time in the current status, the namespace, the allocated memory and CPU, the template details, the instance type, and the OS details.

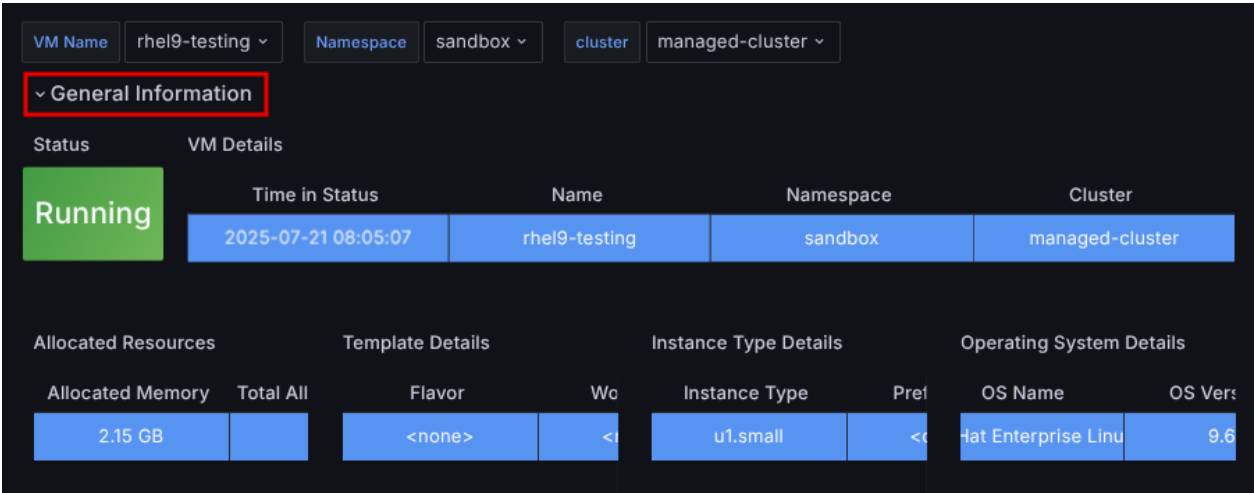


Figure 66. Single virtual machine view dashboard, general information

Resources Section

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

Similar to the previous dashboards, this section displays the history of resource consumption (CPU, memory, network receive, network transmit, and storage IOPs) of the selected VM.

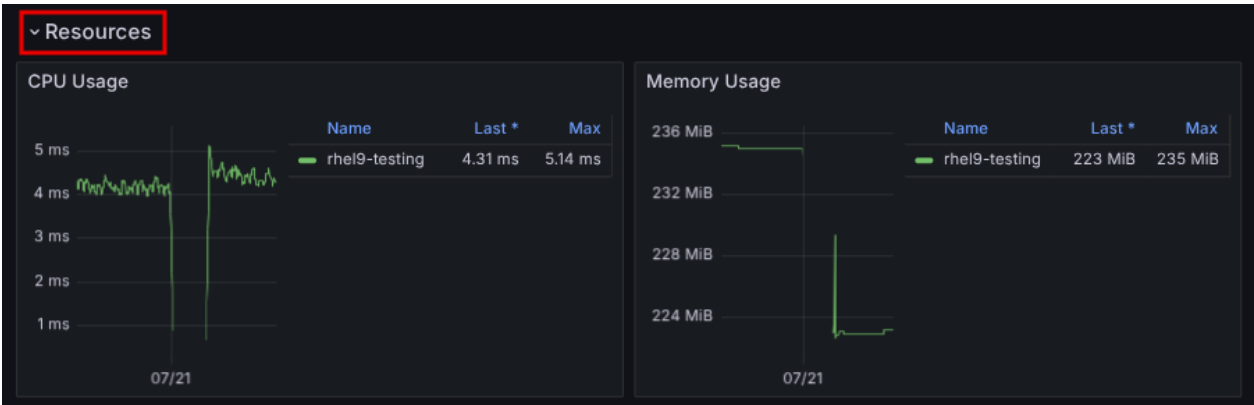


Figure 67. Single virtual machine view dashboard, resources

Alerts Section

The section displays the number of alerts by severity, and the operator alerts.

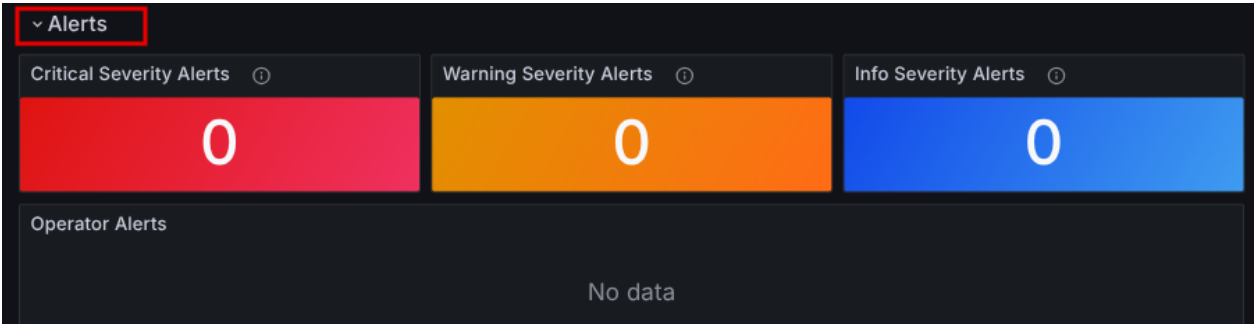


Figure 68. Single virtual machine view dashboard, alerts

Virtual Machines Inventory Dashboard

The *Virtual Machines Inventory* dashboard displays one section with a table that contains an inventory of all the VMs and their information. At the upper-right corner of the dashboard are three drop-down lists, to filter the information by cluster, by namespace, and by VM name.

The inventory helps system administrators to identify specific VMs and to use this list as a reference for performing various operations, such as powering off VMs, deleting VMs, cloning VMs, and initiating VM migrations.

The table enables you to search for VMs based on the following fields:

- Cluster
- Namespace
- VM name
- Status
- Node
- Allocated memory
- Guest agent status
- OS name
- OS version
- Instance type
- Preference
- Workload

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

- Flavor
- Evictable
- Cluster type
- Guest OS architecture
- Outdated

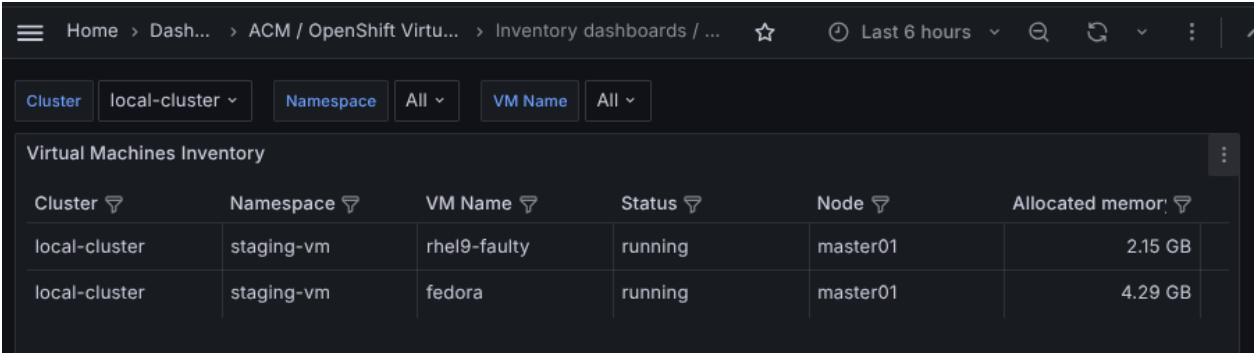


Figure 69. Virtual machines inventory dashboard

Virtual Machines by Time in Status Dashboard

The *Virtual Machines by Time in Status* dashboard displays one section with a table that shows the duration that VMs are in their current state, such as stopped, starting, migrating, and error.

This dashboard is critical for ensuring uptime and reliability. It helps you to maintain Service Level Agreements (SLAs) by identifying VMs in an undesirable state for a long time so you can act to resolve the issue.

You can use the dashboard to identify VMs for cleanup or further investigation.

The drop-down lists at the upper-right corner of the dashboard enable you to filter the data that the table displays.

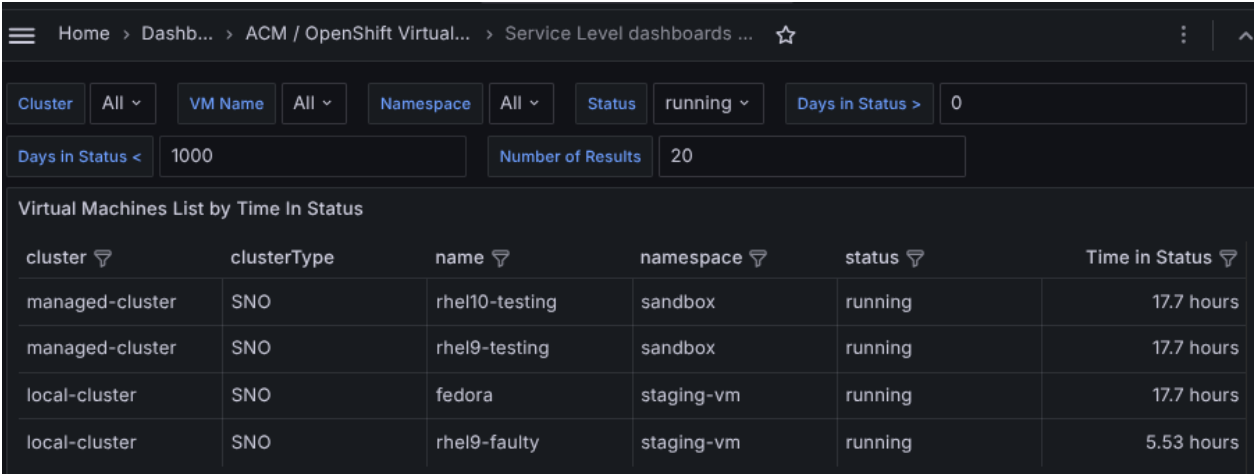


Figure 70. Virtual machines by time in status dashboard

Right-sizing for OpenShift Virtualization

In computing, *right-sizing* refers to optimizing resource allocation to match the needs of an application or workload. Right-sizing involves adjusting resources such as CPU, memory, and storage to avoid overprovisioning (wasting resources) or underprovisioning (impacting performance). By right-sizing, organizations can achieve optimal operational efficiency and significant cost savings.

Right-sizing for OpenShift Virtualization is currently in Technology Preview. It provides dedicated Grafana dashboards in RHACM that are powered by Prometheus recording rules. You enable right-sizing by using RHACM governance

capabilities. The RHACM right-sizing offering provides all preconfigured policies, dashboards, and logic.

Right-sizing OpenShift Virtualization Dashboards

Right-sizing enables two types of Grafana dashboards:

- Right-sizing recommendations at cluster and namespace levels
- Right-sizing recommendations at VM levels

The following list shows the dashboards that right-sizing provides:

- ACM Right Sizing OpenShift Virtualization Dashboard
- ACM Right Sizing OpenShift Virtualization VM Dashboard Overestimation
- ACM Right Sizing OpenShift Virtualization VM Dashboard Underestimated
- ACM Right-Sizing Namespace

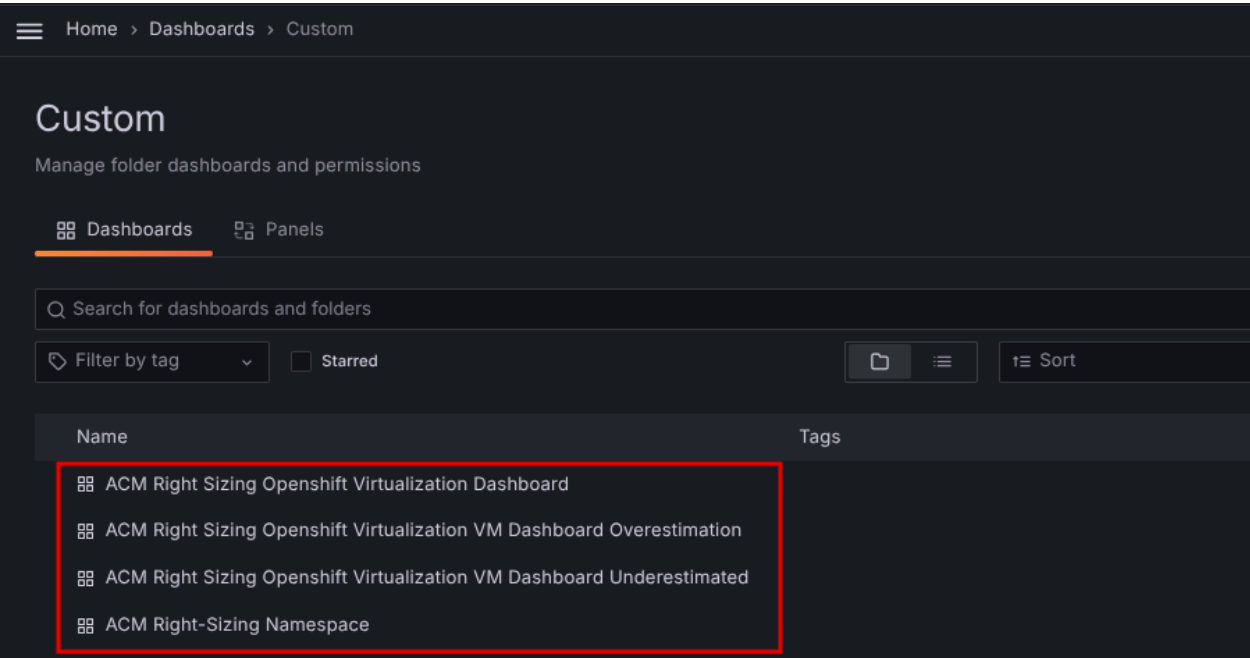


Figure 71. RHACM right-sizing OpenShift Virtualization dashboards

ACM Right-sizing Namespace Dashboard

The dashboard displays the CPU and memory recommendations at cluster and namespace levels. Also, at the upper-right corner of the dashboard is a drop-down list to choose the aggregation time.

In the following screenshot of the dashboard, in the managed-cluster cluster, the sandbox namespace had over 100% of CPU usage for the aggregated time of one day. The dashboard recommends increasing the total CPU requests for the workloads in this namespace to 0.24.

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright



Figure 72. RHACM right-sizing namespace dashboard

RHACM Right-sizing OpenShift Virtualization Dashboard

This dashboard displays the following information:

- The current total CPU overestimation across virtual machines in a cluster
- The current total CPU underestimation across virtual machines in a cluster
- The current total memory overestimation across virtual machines in a cluster
- The current total memory underestimation across virtual machines in a cluster

At the upper-right corner of the dashboard are four drop-down lists to filter the information by cluster, by profile (the only option is *Max Overall*), by aggregation time, or by namespace.

The following screenshot of the dashboard displays a total CPU underestimation of two and a total memory overestimation of four across all virtual machines in the `local-cluster` cluster.

The cluster has two VMs that are running in the `staging-vm` namespace. Both VMs are undersized for CPU. Both VMs are oversized for memory. You use this information to identify the resources that are allocated unnecessarily.

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

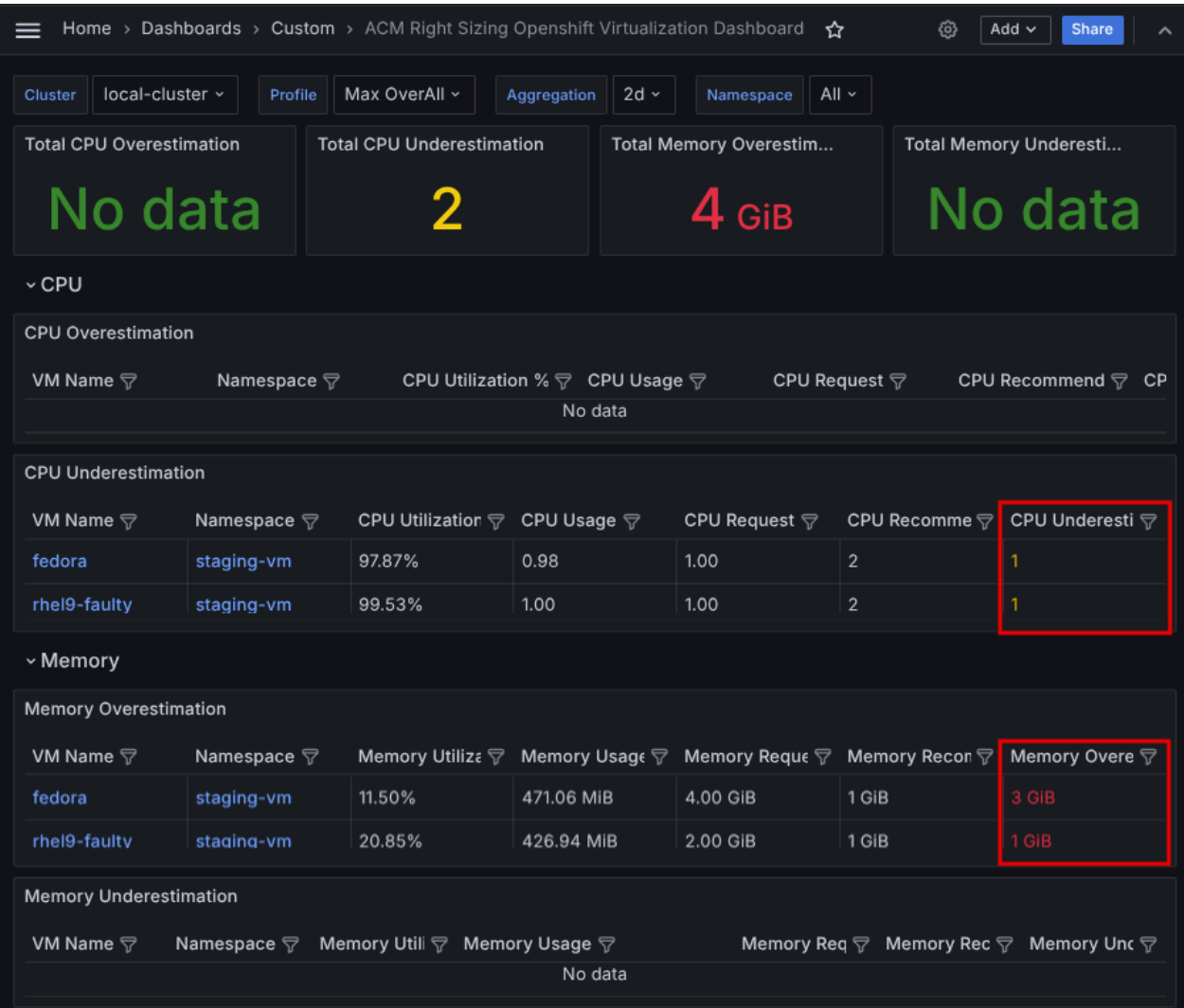


Figure 73. RHACM right-sizing OpenShift Virtualization dashboard

Click the VM name in the dashboard to get deeper information about that specific VM.

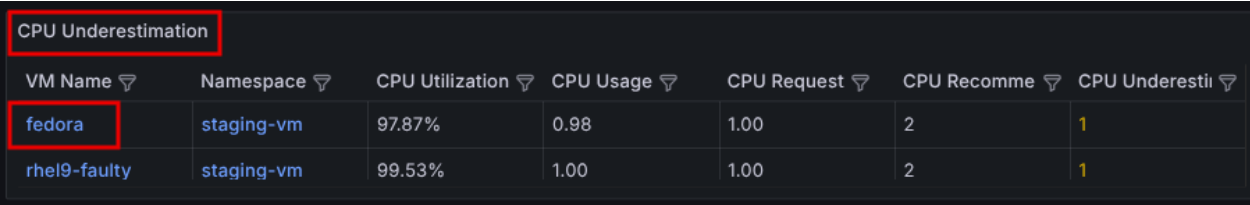


Figure 74. RHACM right-sizing OpenShift Virtualization dashboard

In the following example, by clicking the `fedora` VM in the **CPU Underestimation** table, you are redirected to the *ACM Right Sizing OpenShift Virtualization VM Dashboard Underestimation* dashboard for that VM.

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

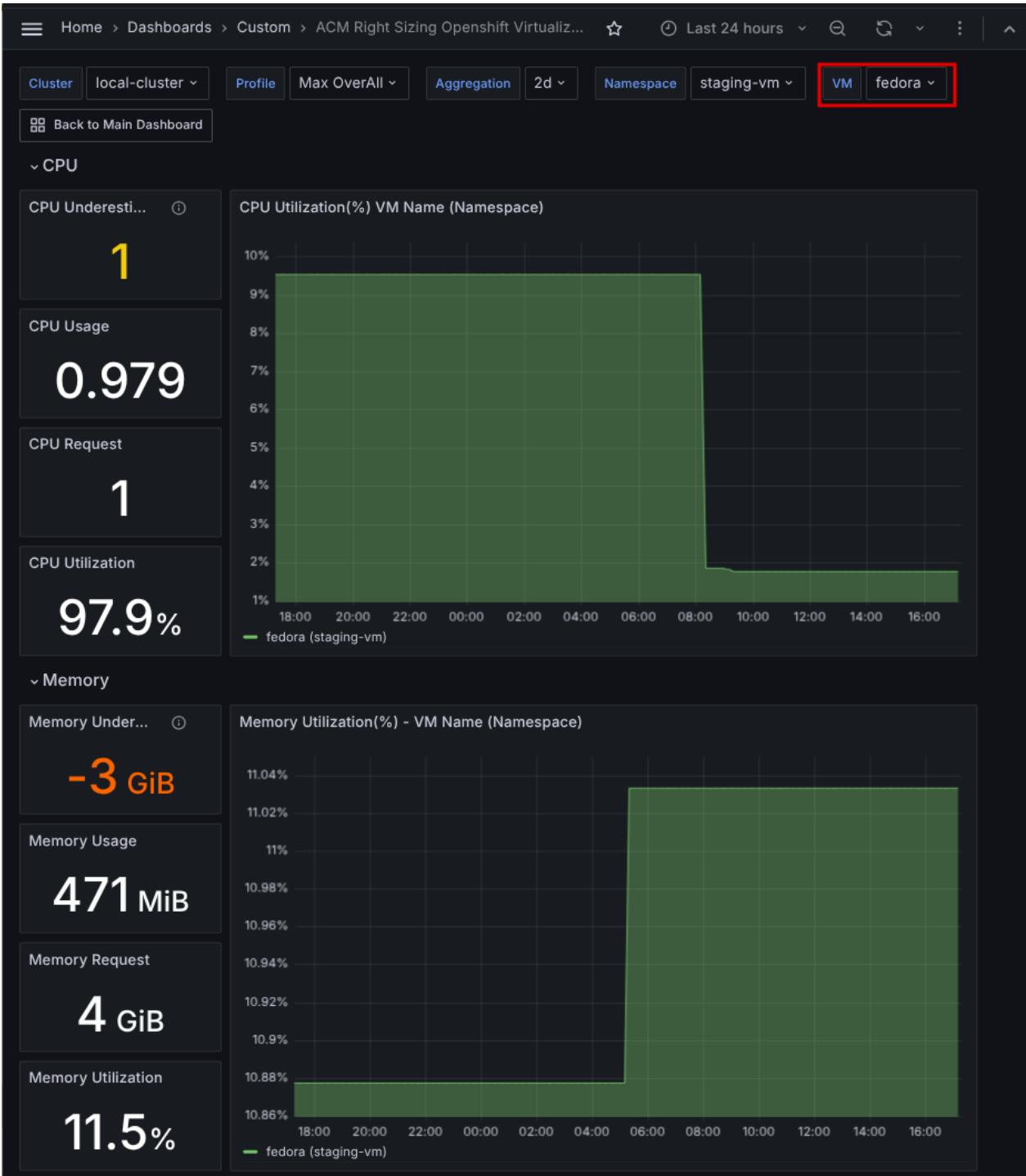


Figure 75. RHACM right-sizing OpenShift Virtualization VM dashboard underestimation

Note
The negative value in the memory underestimation field indicates no memory underestimation.

By clicking the VM name in the **CPU Overestimation** or **Memory Overestimation** table, you are redirected to the *ACM Right Sizing OpenShift Virtualization VM Dashboard Overestimation* dashboard for that VM.

Identify VM Issues with the Virtualization Dashboards

Imagine that you are a member of the Operations team that manages multiple clusters with RHACM. A performance issue arises with a specific VM. Users report a slow application response. Your task is to use the available Grafana dashboards to investigate the root cause and to identify the problem with the VM.

You can use the Clusters Overview dashboard to identify the overall health of the clusters, and inspect the status of the VMs that are running in the clusters. If all VMs are running, then you can check the graphs in the resources utilization section to identify which cluster runs the VM with performance issues.

The VM with issues runs in the local-cluster cluster. You can go to the Single Cluster View dashboard and select the local-cluster cluster. Inspect the graphs in the resources utilization section to identify VMs with issues such as network bottlenecks, storage issues, or abnormal spikes in the CPU and memory usage.

The following graphs show a significant increase in the CPU and memory usage for the fedora VM:



Figure 76. Single cluster view dashboard

You can use the Single Virtual Machine View dashboard to get detailed information about the fedora VM.

The VM CPU and memory usage is close to 100% of the allocated resources. The issue might be with a process that is running within the VM and that consumes all available resources, which in turn slows down the application.

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright ©

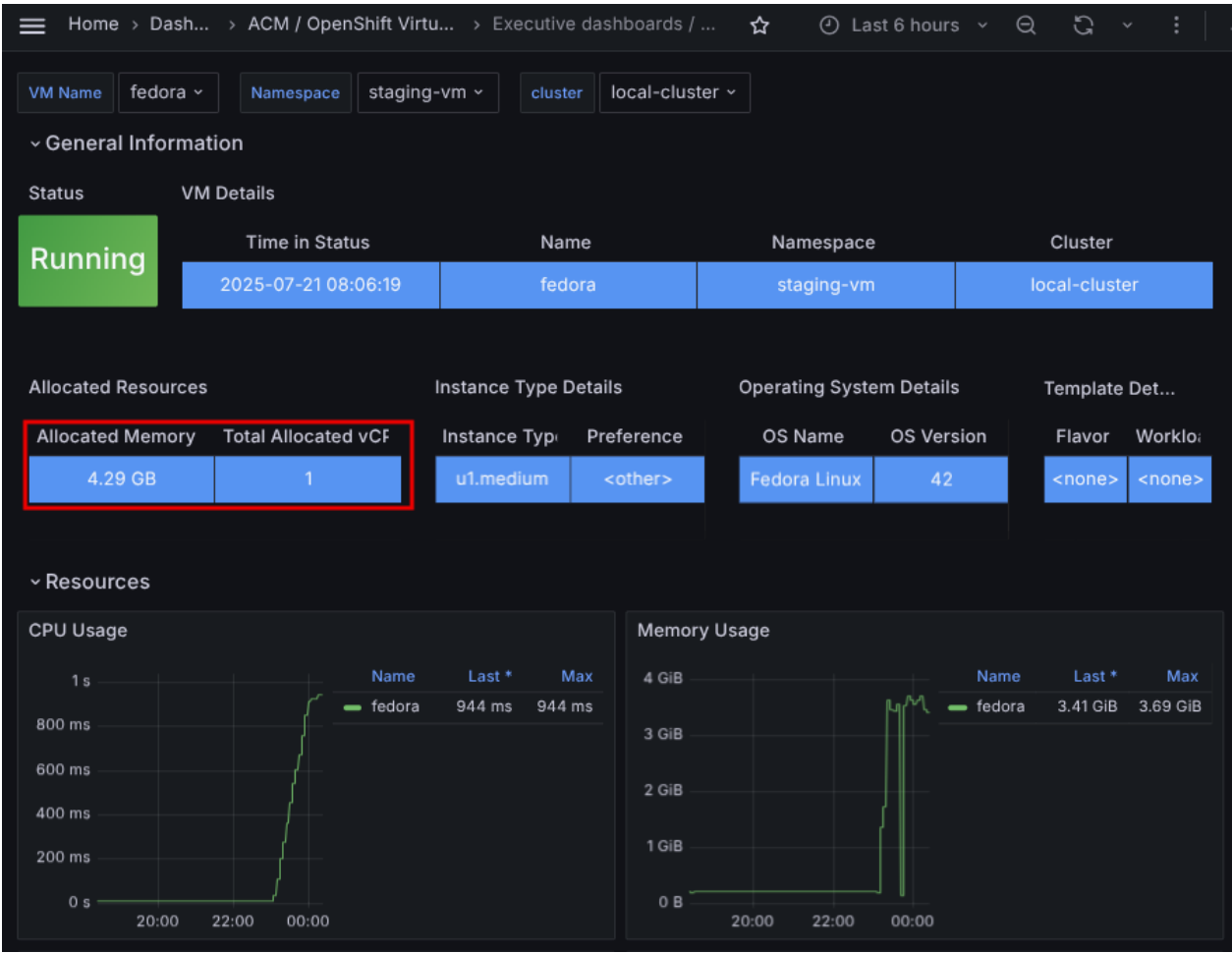


Figure 77. Single virtual machine view dashboard

References

Monitor OpenShift Virtualization at Scale with RHACM: Part 1

Monitor OpenShift Virtualization at Scale with RHACM: Part 2

Announcing Right-sizing for OpenShift Virtualization

For more information about the observability service in RHACM, refer to the *Observability Service* chapter in the *Red Hat Advanced Cluster Management for Kubernetes 2.13 Observability* documentation at https://docs.redhat.com/en/documentation/red_hat_advanced_cluster_management_for_kubernetes/2.13/html-single/observability/index#observing-environments-intro

Lesson 4: *Enabling and Customizing the RHACM Observability Stack*

6.8. Guided Exercise

Monitor OpenShift Virtualization with RHACM

Monitor multicluster virtual machines with various OpenShift Virtualization dashboards and identify any VMs with problems.

Outcomes

- Use the RHACM OpenShift Virtualization Grafana dashboards to monitor a cluster fleet.
- In a fleet of VMs, identify VMs in error status.
- Identify VMs with performance issues.
- Obtain an inventory of the VMs in a managed cluster.
- Review the duration that VMs are in their current state.

Prerequisites

As the student user on the workstation machine, use the lab command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab install do0016l ; lab start virtualization-monitor
```

Instructions

The lab start command performs the following actions:

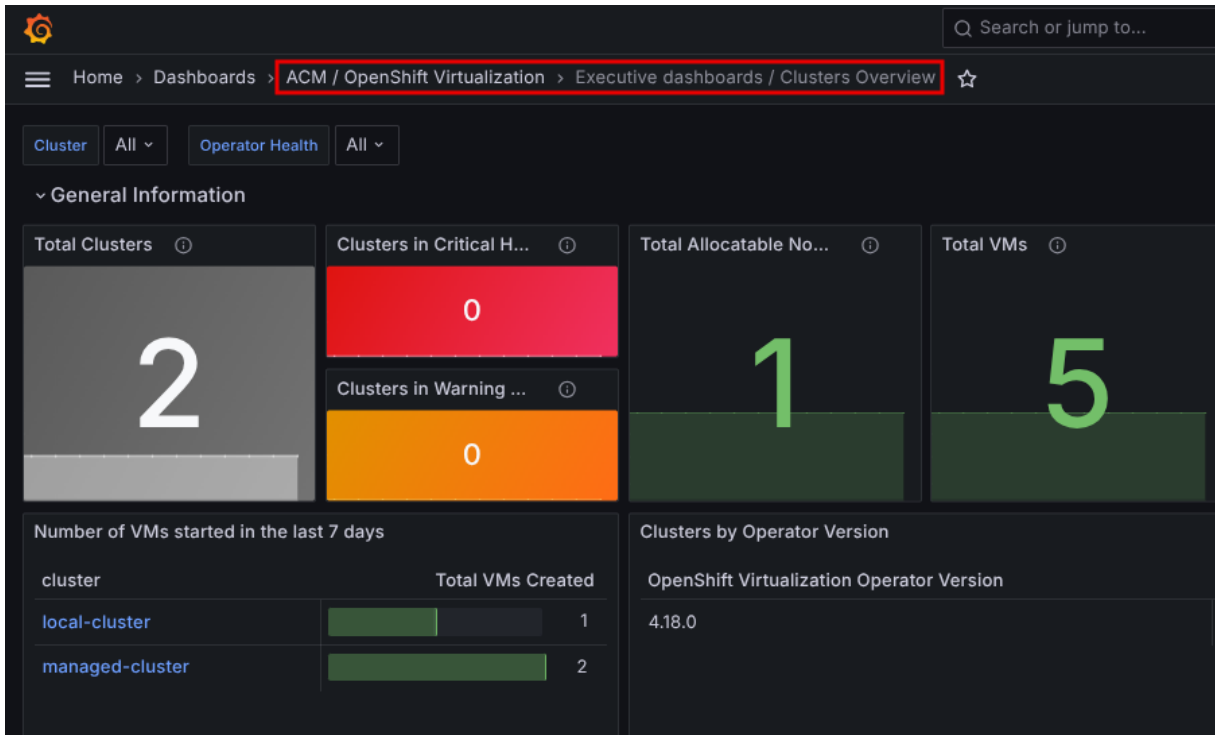
- Install OpenShift Data Foundation and deploy the Multicloud Object Gateway component.
- Install and configure OpenShift Virtualization in the managed-cluster cluster and the local-cluster cluster.
- Deploy and enable the RHACM observability stack.
- Install and deploy OpenShift GitOps in the managed-cluster cluster and the local-cluster cluster.

The lab start command creates the following VMs:

local-cluster	managed-cluster
fedora-hub	fedora-mng
rhel9-hub	rhel9-mng
-	centos10-mng

1. Access the RHACM OpenShift Virtualization dashboards in Grafana from the RHACM console. It can take up to 15 minutes for the information to appear in the dashboards.

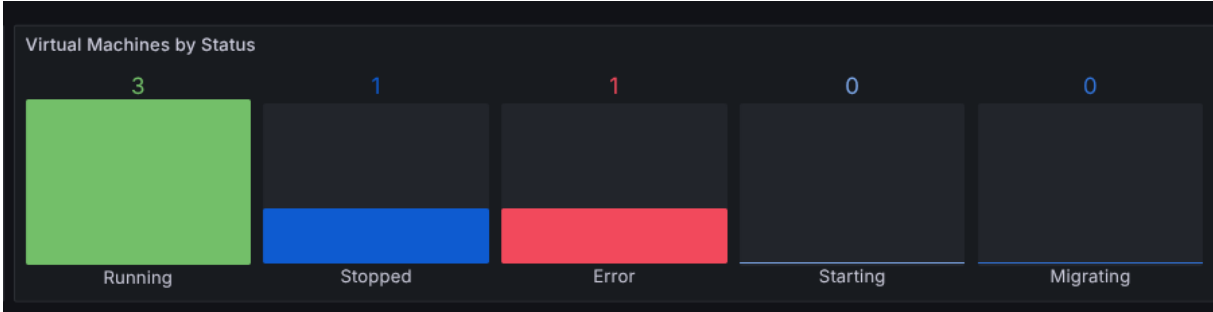
- 1.1. On the workstation machine, open a web browser and go to `https://console-openshift-console.apps.ocp4.example.com`
- 1.2. Select the **Red Hat Identity Management** identity provider and log in as the admin user with `redhatocp` as the password.
- 1.3. Switch to the RHACM web console, and go to **Infrastructure > Virtual machines**.
- 1.4. Click the **Observability dashboards** link in the upper-right corner.
- 1.5. In a new browser tab that opens, log in as the admin user with `redhatocp` as the password.
- 1.6. Grafana loads the **Clusters Overview** dashboard in the **ACM / OpenShift Virtualization** directory.
- 2. Explore the information from all OpenShift Virtualization clusters across the RHACM management fleet by using the **Clusters Overview** dashboard.
- 2.1. Verify that the **General Information** section within the **Clusters Overview** dashboard shows a total of two clusters, one allocatable node, and five VMs. The `local-cluster` cluster has one VM that is running, and the `managed-cluster` cluster has two VMs that are running.



Note

The total allocatable node panel shows only one node, because the PromQL query that the panel uses to calculate the value excludes the `local-cluster` cluster.

- 2.2. Verify that the **Virtual Machines by Status** panel shows three running VMs, one stopped VM, and one VM with error status.

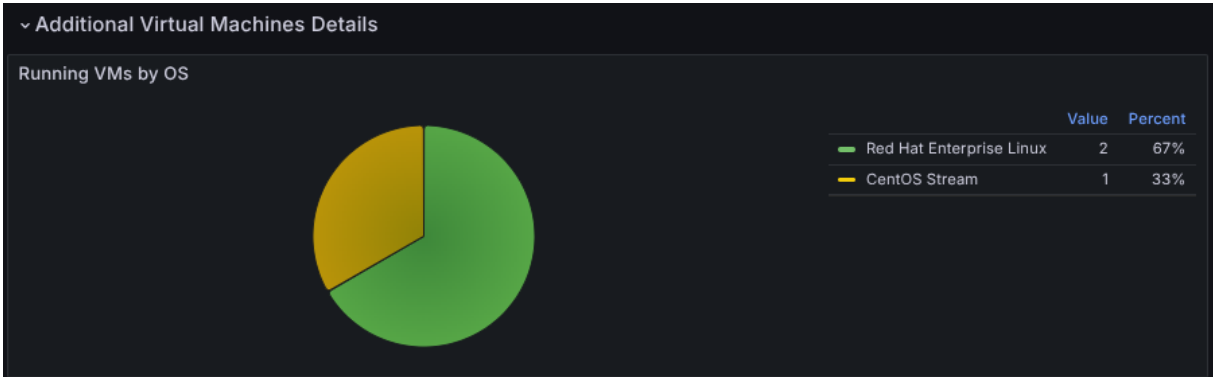


2.3. Check that the OpenShift Virtualization operator is healthy in all clusters.

A table titled "Operator Health by Cluster". It shows the health status of the OpenShift Virtualization operator across two clusters. The "local-cluster" and "managed-cluster" are both marked as "Healthy".

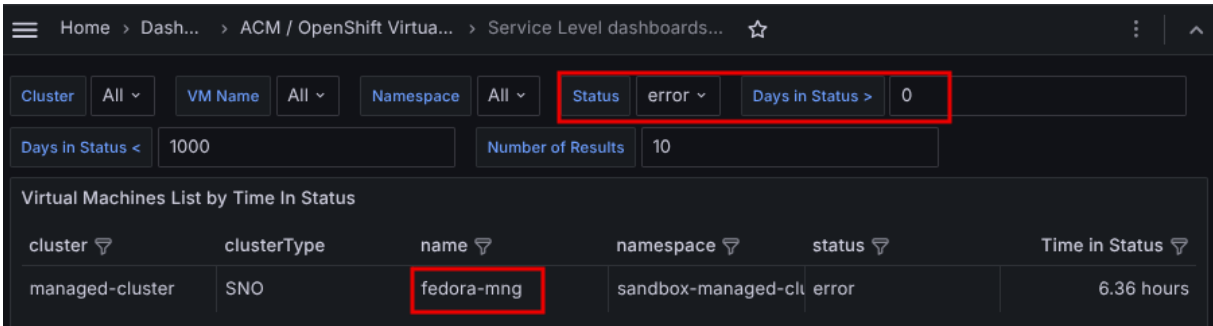
Operator Health	Cluster	Operator Conditions Health
Healthy	local-cluster	Healthy
Healthy	managed-cluster	Healthy

2.4. Verify that from the total VMs that are running across the cluster fleet, two VMs run the Red Hat Enterprise Linux OS, and one VM runs the CentOS Stream OS.

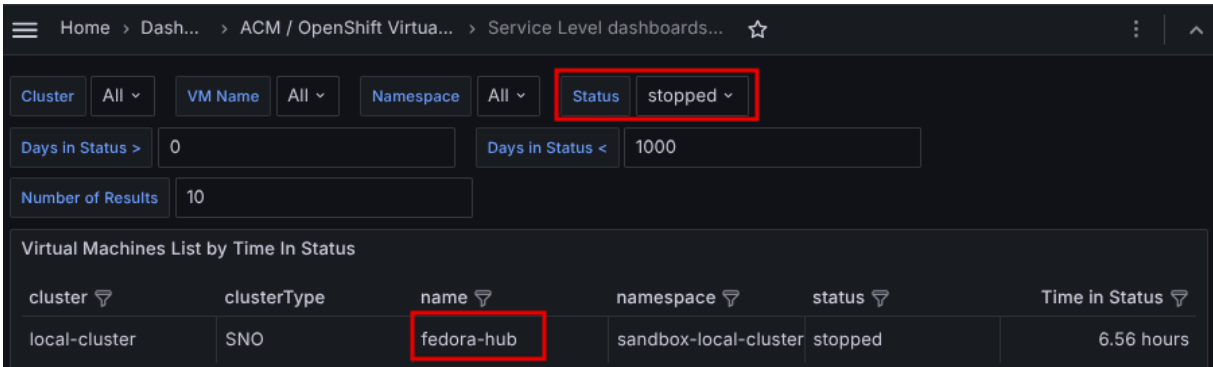


3. Use the **Virtual Machines by Time in Status** dashboard to locate the VMs with stopped or error as the current state.

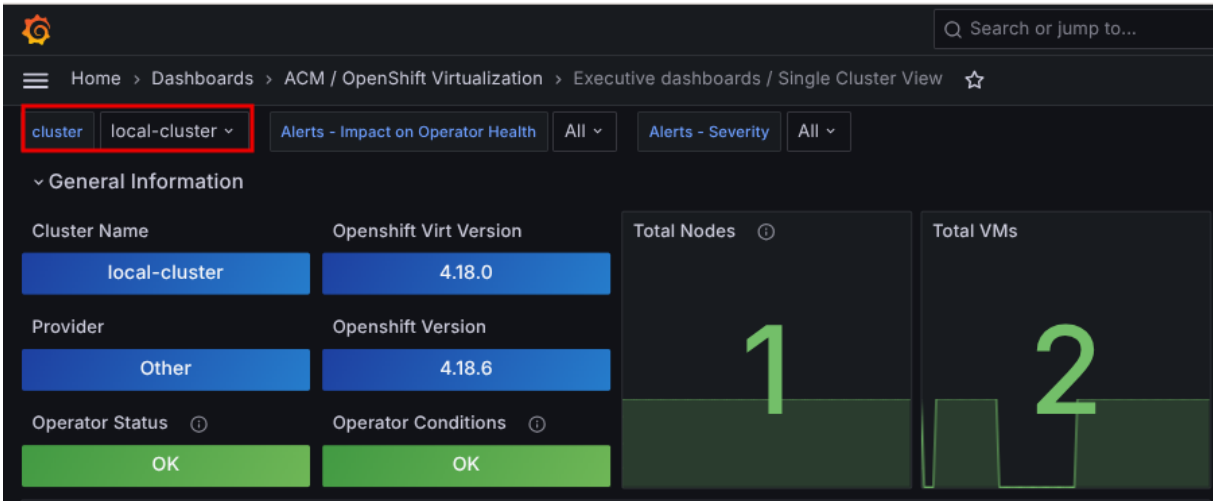
- 3.1. From Grafana, go to **Home > Dashboards > ACM / OpenShift Virtualization**, and click **Virtual Machines by Time in Status**.
- 3.2. In the **Virtual Machines by Time in Status** dashboard, select **error** in the **Status** drop-down list. Set the **Days in Status >** value to 0, and press **Enter**.
- 3.3. The dashboard displays a table with information for the VM named `fedora-mng`. This VM belongs to the `managed-cluster` cluster and to the `sandbox-managed-cluster` namespace. The status is error, and the time in status is 6.36h. The time in status is different in your cluster.



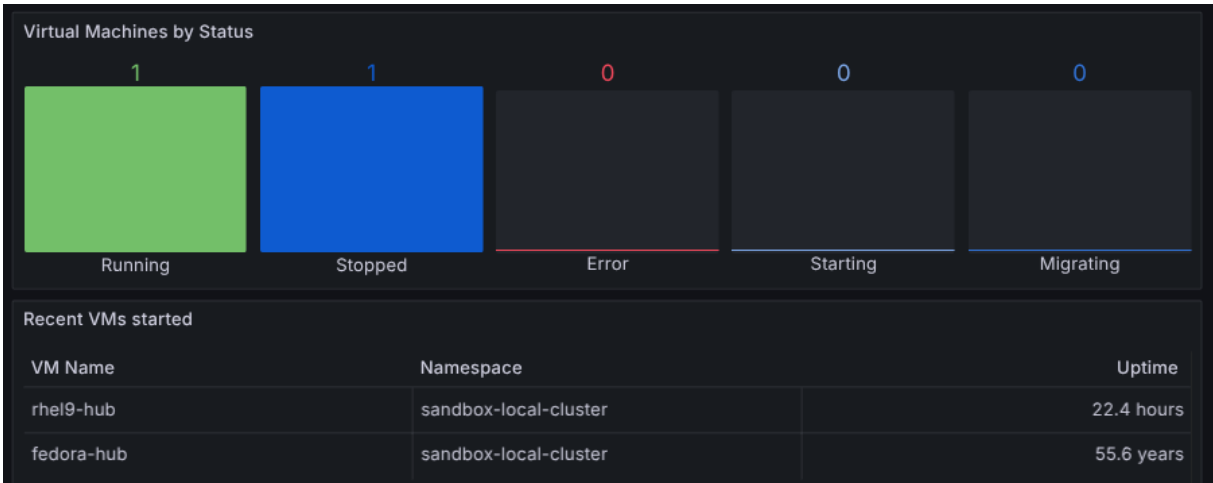
- 3.4. Select **stopped** in the **Status** drop-down list.
- 3.5. The dashboard displays a table with information for the VM named `fedora-hub`. This VM belongs to the `local-cluster` cluster and to the `sandbox-local-cluster` namespace. The status is `stopped`, and the time in status is 6.56h. The time in status is different in your cluster.



4. Use the **Single Cluster View** dashboard to inspect individually each cluster in the fleet.
- 4.1. From Grafana, go to **Home > Dashboards > ACM / OpenShift Virtualization**, and click **Single Cluster View**.
- 4.2. At the upper-left corner of the dashboard, select **local-cluster** in the **cluster** drop-down list.
- 4.3. Verify from the **General Information** section that the OpenShift version is `4.18.6`, the OpenShift Virtualization operator version is `4.18.0`, and that the operator status and operator conditions are OK. The cluster has one node and two VMs.

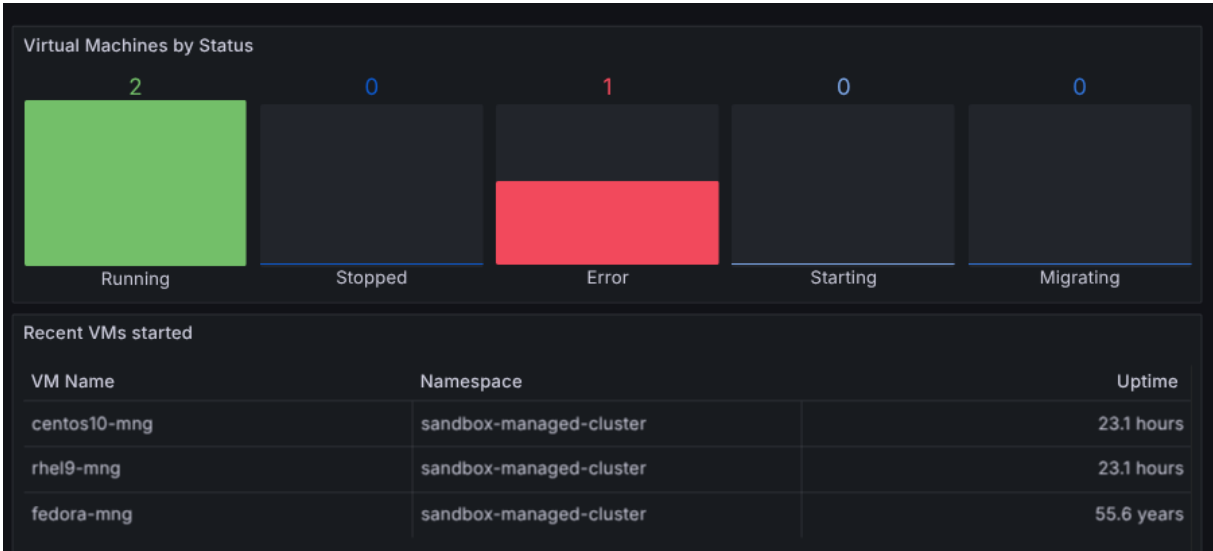


- 4.4. The **Virtual Machines by Status** panel shows that the `rhel9-hub` VM is running with an uptime of 22.4 hours, and the `fedora-hub` VM is stopped. The uptime is different in your cluster.



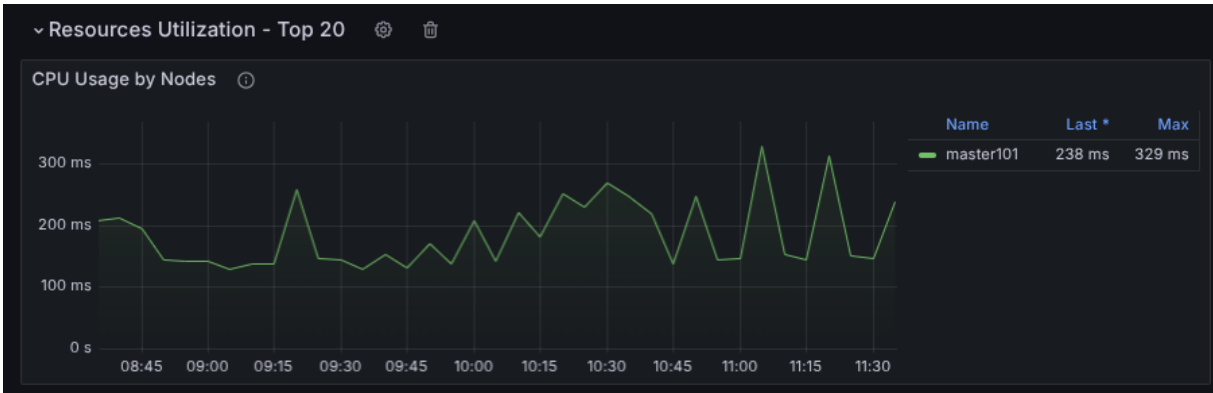
Note
The uptime value for the fedora-hub VM is 55.6 years, which means that the VM is stopped. The PromQL query that the panel uses to calculate the uptime value uses the time() function, which is the number of seconds since 1 January 1970, which equals to 55.6 years as of the time of writing.

- 4.5. At the upper-left corner of the dashboard, select **managed-cluster** in the **cluster** drop-down list.
- 4.6. Verify from the **General Information** section that the cluster has one node and three VMs. The centos10-mng VM and the rhel9-mng VM are running; the fedora-mng VM is not running and reports the error status. The uptime is different in your cluster.

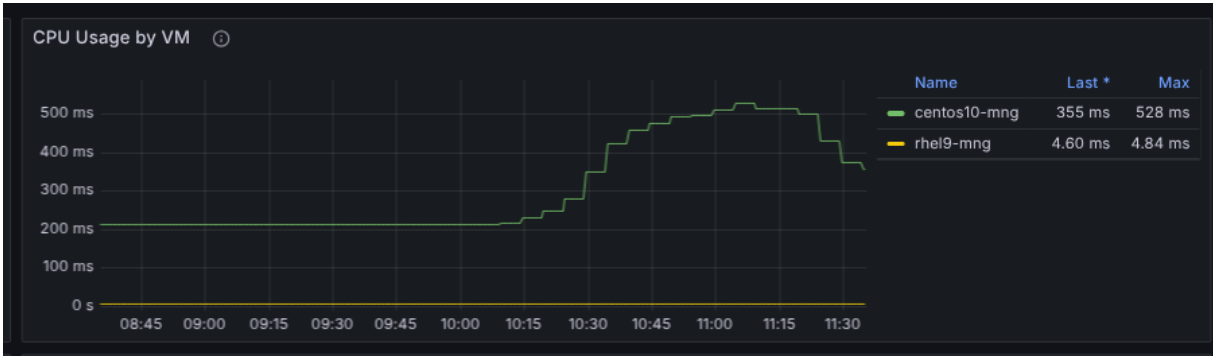


- 4.7. At the upper-right corner of the dashboard, set the time range from the drop-down list to **Last 3 hours**.
- 4.8. Click **Resources Utilization - Top 20** to expand the section. Review the CPU usage graph for the **master101** node. The values for CPU usage are different in your cluster.

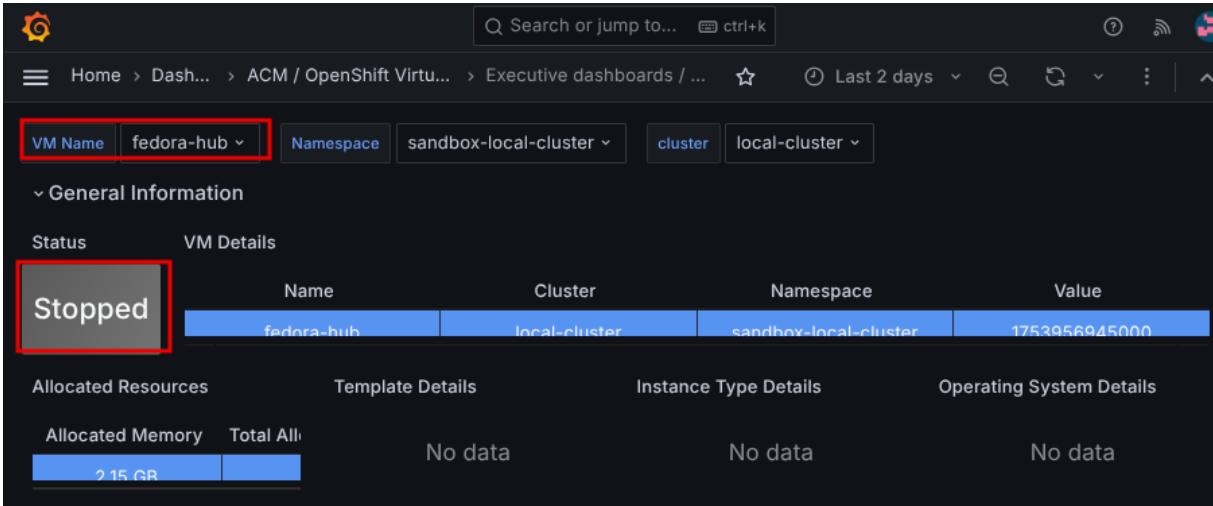
For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright



4.9. Review the CPU usage graph for the VMs that are running in the cluster. The values for CPU usage by VM are different in your cluster.



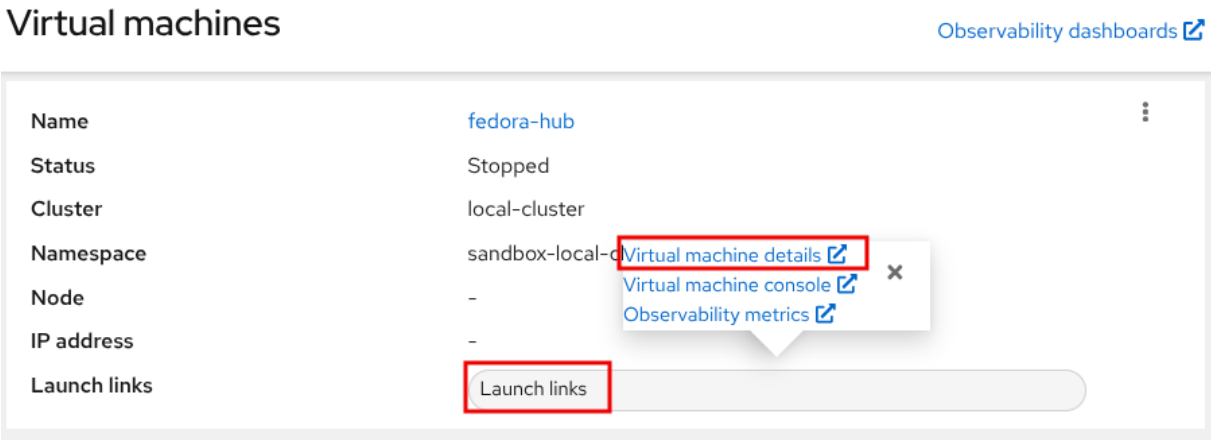
5. Use the **Single Virtual Machine View** dashboard to inspect a single VM status and details.
- 5.1. From Grafana, go to **Home > Dashboards > ACM / OpenShift Virtualization**, and click **Single Virtual Machine View**.
- 5.2. At the upper-left corner of the dashboard, select **fedora-hub** in the **VM Name** drop-down list.
- 5.3. The fedora-hub VM shows the stopped status.



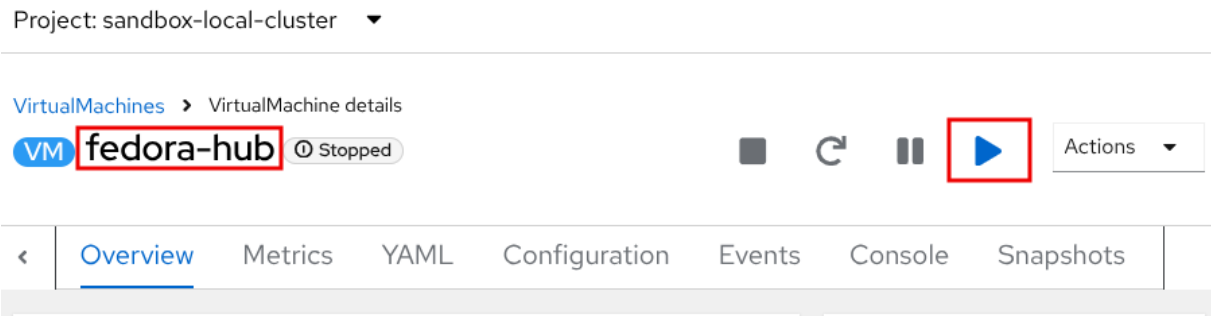
Note
To view the information correctly, you must enlarge the panels for VM Details, Allocated Resources, Template Details, Instance Type Details, and Operating System Details.

5.4. Switch to the RHACM web console browser tab. Go to **Infrastructure > Virtual machines**.

5.5. On the fedora-hub VM, click **Launch links** and click **Virtual machine details**.

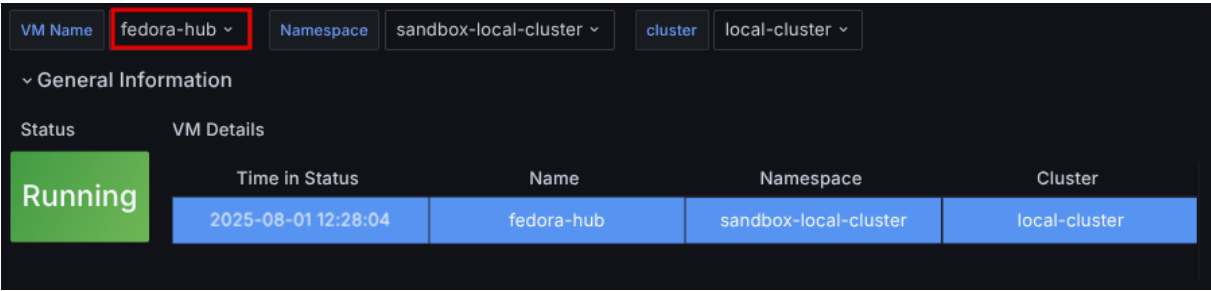


5.6. A new browser tab opens with the fedora-hub VM details. Click the start icon at the upper-right corner.



5.7. Switch back to the Grafana **Single Virtual Machine View** dashboard.

5.8. The fedora-hub VM shows the running status. It can take up to five minutes to reach this status. You can click the refresh icon at the upper-right corner of the dashboard to refresh the information.



5.9. Verify from the **Allocated Resources** panel that the VM has 2.15 GB of allocated memory and 1 vCPU of allocated CPU.

5.10. Verify that the **Operating System Details** panel displays Fedora Linux as the **OS Name** and 42 as the **OS Version**.

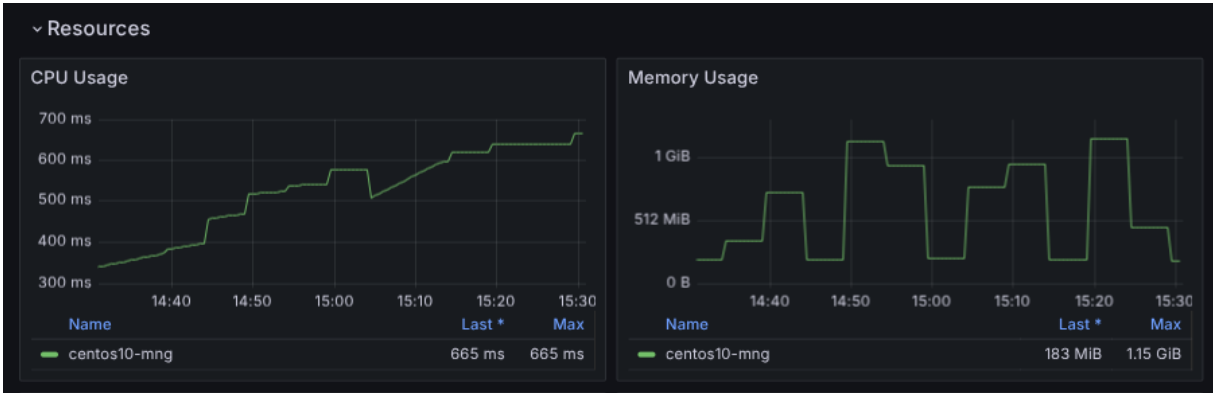
6. Use the **Single Virtual Machine View** dashboard to inspect a VM that runs an application that experiences low performance.

6.1. From Grafana, go to **Home > Dashboards > ACM / OpenShift Virtualization**, and click **Single Virtual Machine View**.

6.2. At the upper-left corner of the dashboard, select **centos10-mng** in the **VM Name** drop-down list. At the upper-right corner of the dashboard, set the time range from the drop-down list to **Last 5 minutes**.

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

6.3. Review the CPU Usage graph and the Memory Usage graph in the **Resources** section. The CPU graph shows a constant increase in CPU usage, and the memory graph shows frequent spikes. The values in the graphs might be different in your cluster depending on the VM uptime.



7. Use the Virtual Machine Inventory dashboard to view the inventory of the VMs on each cluster.

- 7.1. From Grafana, go to **Home > Dashboards > ACM / OpenShift Virtualization**, and click **Virtual Machine Inventory**.
- 7.2. At the upper-left corner of the dashboard, select **local-cluster** in the **Cluster** drop-down list.
- 7.3. Verify that the dashboard displays a table with the fedora-hub VM and the rhel9-hub VM. Both VMs show the running status.

The screenshot shows the Grafana interface for the 'Virtual Machine Inventory' dashboard. The breadcrumb navigation is 'Home > Dashboards > ACM / OpenShift Virtualization > Inventory dashboards / Virtual Machine Inventory'. The 'Cluster' dropdown is set to 'local-cluster'. Below the filters, a table titled 'Virtual Machines Inventory' displays the following data:

Cluster	Namespace	VM Name	Status	Node	Allocated memory
local-cluster	sandbox-local-cluster	fedora-hub	running	master01	2.15 GB
local-cluster	sandbox-local-cluster	rhel9-hub	running	master01	2.15 GB

- 7.4. At the upper-left corner of the dashboard, select **managed-cluster** in the **Cluster** drop-down list.
- 7.5. Verify that the dashboard displays a table with the centos10-mng VM, the rhel9-mng VM, and the fedora-mng VM. The fedora-mng VM shows the error status.

The screenshot shows the Grafana interface for the 'Virtual Machine Inventory' dashboard. The breadcrumb navigation is 'Home > Dashboards > ACM / OpenShift Virtualization > Inventory dashboards / Virtual Machine Inventory'. The 'Cluster' dropdown is set to 'managed-cluster'. Below the filters, a table titled 'Virtual Machines Inventory' displays the following data:

Cluster	Namespace	VM Name	Status	Node	Allocated memory
managed-cluster	sandbox-managed-cluster	centos10-mng	running	master101	2.15 GB
managed-cluster	sandbox-managed-cluster	rhel9-mng	running	master101	2.15 GB
managed-cluster	sandbox-managed-cluster	fedora-mng	error		2.15 GB

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish virtualization-monitor
```

6.9. Lab

Manage Virtual Machines for Multiple Clusters with RHACM

Deploy and manage virtual machines across a multicluster environment by using RHACM. Additionally, identify and pause a VM with performance issues in a managed cluster.

Outcomes

- Deploy and configure the OpenShift Virtualization operator.
- Create VMs by using RHACM and GitOps.
- Monitor VMs and identify VMs with performance issues.

Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab install do0016L ; lab start virtualization-review
```

Instructions

In this exercise, you prepare your two clusters and you deploy VMs by using GitOps. You also identify the VM with performance issues.

The `lab start` command performed the following actions:

- Installed OpenShift Data Foundation and deployed the Multicloud Object Gateway component.
- Deployed and enabled the RHACM observability stack.
- Installed and deployed OpenShift GitOps.
- Created Git repositories with the resources for deploying the OpenShift Virtualization operator and for creating the VMs.

The public Git repositories are available at <https://git.ocp4.example.com/do0016l/virtualization-review-deploy.git> and <https://git.ocp4.example.com/do0016l/virtualization-review-vm.git>

The classroom environment provides two RHOC clusters: an RHACM hub cluster and a managed cluster. You can access those clusters by using the following parameters:

RHACM hub cluster web console URL

<https://console-openshift-console.apps.ocp4.example.com>

RHACM hub cluster API URL

<https://api.ocp4.example.com:6443>

Managed cluster web console URL

<https://console-openshift-console.apps.ocp4-mng.example.com>

Managed cluster API URL

<https://api.ocp4-mng.example.com:6443>

Administrator username for both clusters

admin (select the **Red Hat Identity Management** identity provider)

Administrator password for both clusters

redhatocp

1. Deploy and configure the OpenShift Virtualization operator in your two clusters by using RHACM and the GitOps Push model.

The Git repository that the classroom provides already includes the resources to perform this step. Use the following parameters for your configuration:

Parameter	Value
GitOps model	Push model
Application set name	virt-operator
Git URL	https://git.ocp4.example.com/do0016l/virtualization-review-deploy.git
Git revision	main
Git path	manifests
Remote namespace	openshift-cnv
Placement cluster set	gitops-configure
Placement selected clusters	The two clusters in the gitops-configure cluster set

2. Deploy a VM in the cluster with the env=prod label by using RHACM and the GitOps Push model.

The Git repository that the classroom provides already includes the resources to perform this step. Use the following parameters for your configuration:

Parameter	Value
GitOps model	Push model
Application set name	vm-prod

Parameter	Value
Git URL	https://git.ocp4.example.com/do0016l/virtualization-review-vms.git
Git revision	main
Git path	manifests/prod
Remote namespace	vm-prod
Placement cluster set	gitops-configure
Managed cluster label	env=prod

 Note

In the **Topology** view, you might see an unknown status icon for the Data Volume and the Persistent Volume Claim (PVC) resources. It is a known display issue that does not affect the functioning of the application, and will be resolved in a future update.

3. Deploy a VM in the cluster with the env=test label by using RHACM and the GitOps Push model.

The Git repository that the classroom provides already includes the resources to perform this step. Use the following parameters for your configuration:

Parameter	Value
GitOps model	Push model
Application set name	vm-test
Git URL	https://git.ocp4.example.com/do0016l/virtualization-review-vms.git
Git revision	main
Git path	manifests/test
Remote namespace	vm-test
Placement cluster set	gitops-configure

Parameter	Value
Managed cluster label	env=test

Note

In the **Topology** view, you might see an unknown status icon for the Data Volume and the Persistent Volume Claim (PVC) resources. It is a known display issue that does not affect the functioning of the application, and will be resolved in a future update.

4. One of the VMs consumes an abnormal amount of I/O. Identify the VM and pause it.

Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade virtualization-review
```

Finish

As the student user on the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish virtualization-review
```

Solution

Deploy and manage virtual machines across a multicluster environment by using RHACM. Additionally, identify and pause a VM with performance issues in a managed cluster.

Outcomes

- Deploy and configure the OpenShift Virtualization operator.
- Create VMs by using RHACM and GitOps.
- Monitor VMs and identify VMs with performance issues.

Prerequisites

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab install do0016L ; lab start virtualization-review
```

Instructions

In this exercise, you prepare your two clusters and you deploy VMs by using GitOps. You also identify the VM with performance issues.

The `lab start` command performed the following actions:

- Installed OpenShift Data Foundation and deployed the Multicloud Object Gateway component.
- Deployed and enabled the RHACM observability stack.
- Installed and deployed OpenShift GitOps.
- Created Git repositories with the resources for deploying the OpenShift Virtualization operator and for creating the VMs.

The public Git repositories are available at <https://git.ocp4.example.com/do0016l/virtualization-review-deploy.git> and <https://git.ocp4.example.com/do0016l/virtualization-review-vm.git>

The classroom environment provides two RHOC clusters: an RHACM hub cluster and a managed cluster. You can access those clusters by using the following parameters:

RHACM hub cluster web console URL

<https://console-openshift-console.apps.ocp4.example.com>

RHACM hub cluster API URL

<https://api.ocp4.example.com:6443>

Managed cluster web console URL

<https://console-openshift-console.apps.ocp4-mng.example.com>

Managed cluster API URL

<https://api.ocp4-mng.example.com:6443>

Administrator username for both clusters

admin (select the **Red Hat Identity Management** identity provider)

Administrator password for both clusters

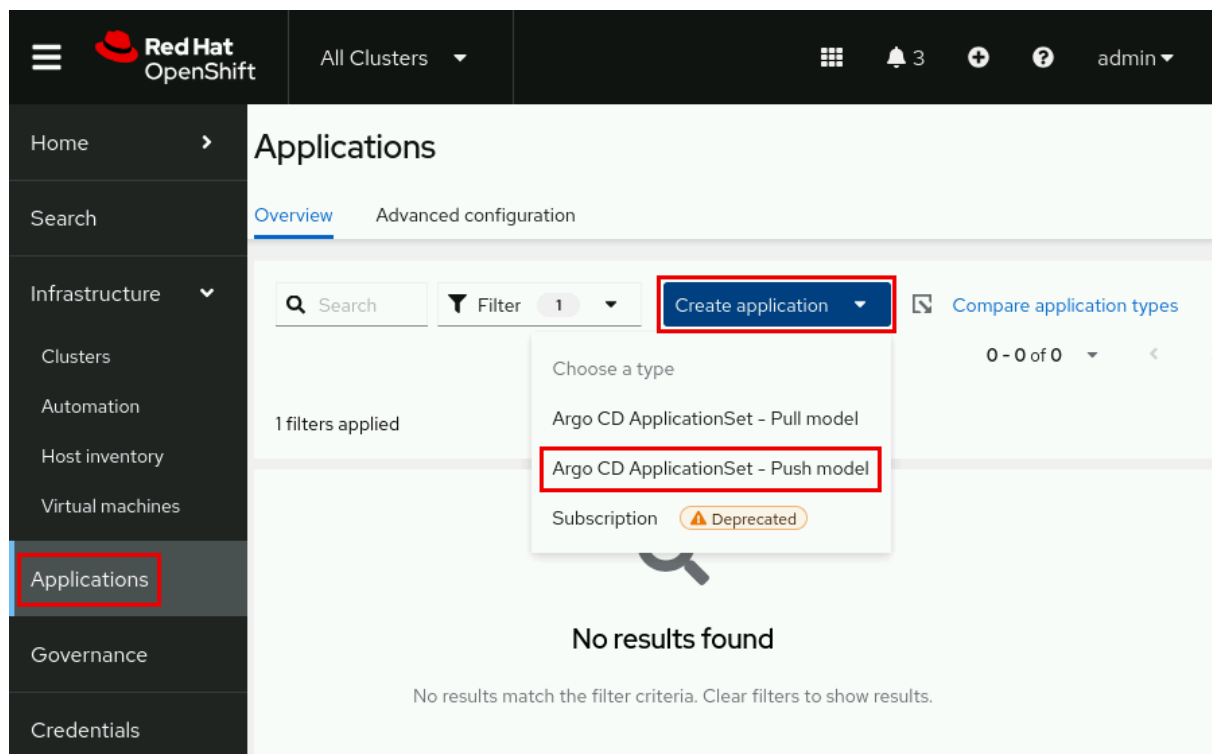
redhatocp

1. Deploy and configure the OpenShift Virtualization operator in your two clusters by using RHACM and the GitOps Push model.

The Git repository that the classroom provides already includes the resources to perform this step. Use the following parameters for your configuration:

Parameter	Value
GitOps model	Push model
Application set name	virt-operator
Git URL	https://git.ocp4.example.com/do0016l/virtualization-review-deploy.git
Git revision	main
Git path	manifests
Remote namespace	openshift-cnv
Placement cluster set	gitops-configure
Placement selected clusters	The two clusters in the gitops-configure cluster set

- 1.1. Open a web browser and go to https://console-openshift-console.apps.ocp4.example.com
- 1.2. Select the **Red Hat Identity Management** identity provider and log in as the admin user with redhatocp as the password.
- 1.3. Ensure that the cluster switcher is set to **All Clusters**, which corresponds to the RHACM web console.
- 1.4. Go to **Applications** and select **Create application > Argo CD ApplicationSet - Push model**.



- 1.5. Complete the form with the provided parameters. In the **General** section, set the application name and select the Argo CD server. Click **Next**.

Applications > Create application set - push model

Create application set - push model

☐ YAML

1 General

General

Name *

virt-operator

Argo server * ?

openshift-gitops

Requeue time * ?

180

Next Back Cancel

- 1.6. In the **Template** section, select **Git** and provide the Git parameters. Click **Next**.

2 Template

Repository

Type
Git https://git.ocp4.example.com/do0016l/virtualization-review-deploy.git



Git
Use a Git repository



Helm
Use a Helm repository

URL * ?
https://git.ocp4.example.com/do0016l/virtualization-review-deploy.git

Revision ?
main

Path ?
manifests

+ Add another repository

Destination

Remote namespace *
openshift-cnv

Next Back Cancel

- 1.7. Do not change anything in the **Sync policy** section. Click **Next**.
- 1.8. In the Placement section, select **New placement**, select the gitops-configure cluster set, and limit the number of clusters to two. Click **Next**.

4 Placement

Placement

How do you want to select clusters?

New placement Existing placement

Cluster sets ?

gitops-configure 1

If no cluster sets are selected, all clusters will be selected from the cluster sets bound to the namespace.

Label expressions ?

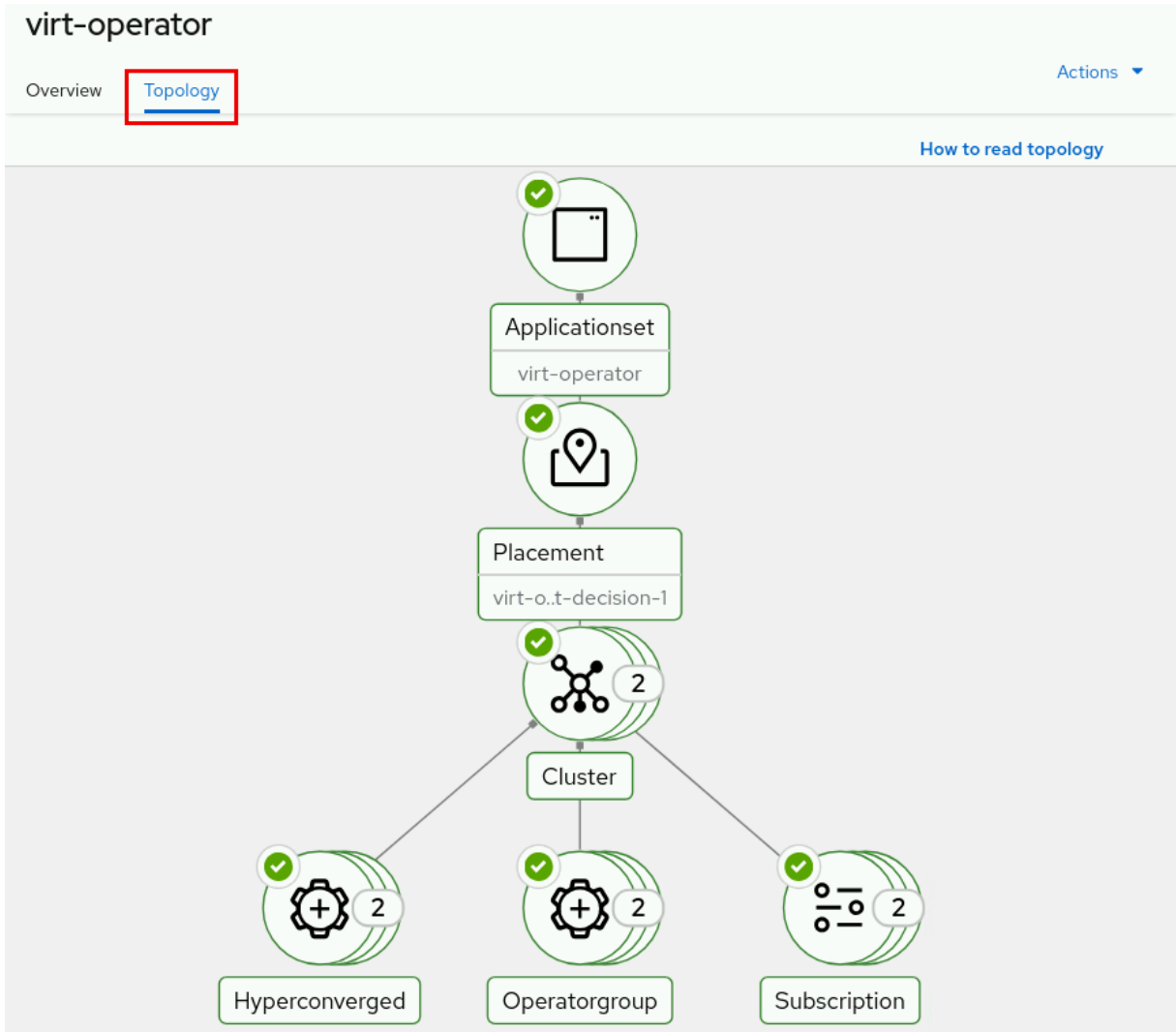
+ Add label expression

Limit the number of clusters selected

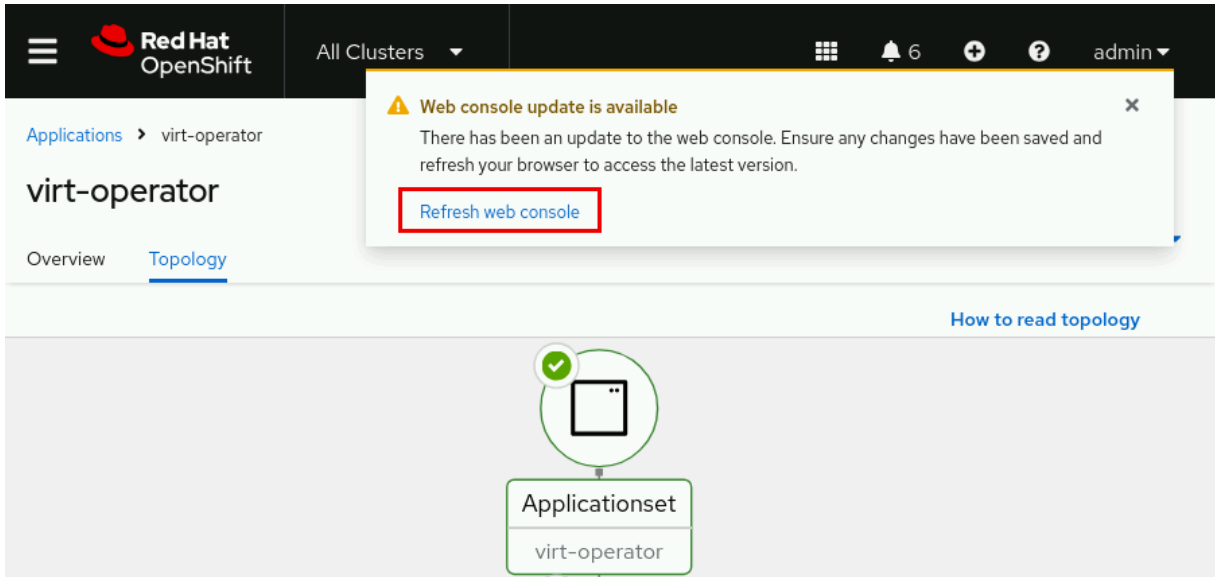
- 2 +

Next Back Cancel

- 1.9. In the Review section, verify the parameters and click **Submit**.
- 1.10. Go to **Topology** and wait up to three minutes for the topology to report the application status.



1.11. If a message about a web console update is displayed, then click **Refresh web console**.



- 1.12. Verify that the operator is available in the `local-cluster` cluster. Ensure that the cluster switcher is set to `local-cluster`, which corresponds to the RHOCP web console.
- 1.13. From the left panel of the OpenShift web console, go to **Operators > Installed Operators** to confirm that the OpenShift Virtualization installation succeeded.

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

Project: All Projects ▾

Installed Operators

Installed Operators are represented by ClusterServiceVersions within this Namespace. For more information, see the [Understanding Operators documentation](#). Or create an Operator and ClusterServiceVersion using the [Operator SDK](#).

Name ▾

Search by name... /

Name	Namespace	Managed Namespaces	Status
 Advanced Cluster Management for Kubernetes 2.13.0 provided by Red Hat	NS open-cluster-management	NS open-cluster-management	✓ Succeeded Up to date
 OpenShift Virtualization 4.18.0 provided by Red Hat	NS openshift-cnv	NS openshift-cnv	✓ Succeeded Up to date

- 1.14. Click **OpenShift Virtualization** to view the operator details.
- 1.15. Click **All instances**, and verify that the **kubevirt-hyperconverged** object status shows the **ReconcileComplete**, **Available**, and **Upgradeable** conditions.

Project: openshift-cnv ▾

Installed Operators ▸

Operator details



OpenShift Virtualization

4.18.0 provided by Red Hat

Actions ▾

Details

YAML

Subscription

Events

All instances

OpenShift Virtualization Deployments

All Instances

Create new ▾

Filter ▾

Name ▾

Search by name... /

Name	Kind	Status
HC kubevirt-hyperconverged	HyperConverged	Conditions: ReconcileComplete, Available, Upgradeable

- 1.16. Switch back to the RHACM web console by setting the cluster switcher to **All Clusters**.
- 1.17. Open a browser tab and go to <https://console-openshift-console.apps.ocp4-mng.example.com> to log in to the RHOCP managed-cluster cluster. Select the **Red Hat Identity Management** identity provider and log in

as the admin user with redhatocp as the password.

- 1.18. From the left panel of the OpenShift web console, go to **Operators > Installed Operators** to confirm that the OpenShift Virtualization installation succeeded.
- 1.19. Click **OpenShift Virtualization** to view the operator details.
- 1.20. Click **All instances**, and verify that the **kubevirt-hyperconverged** object status shows the **ReconcileComplete**, **Available**, and **Upgradeable** conditions.
- 1.21. Switch back to the browser tab with the RHACM web console.

- 2. Deploy a VM in the cluster with the env=prod label by using RHACM and the GitOps Push model.

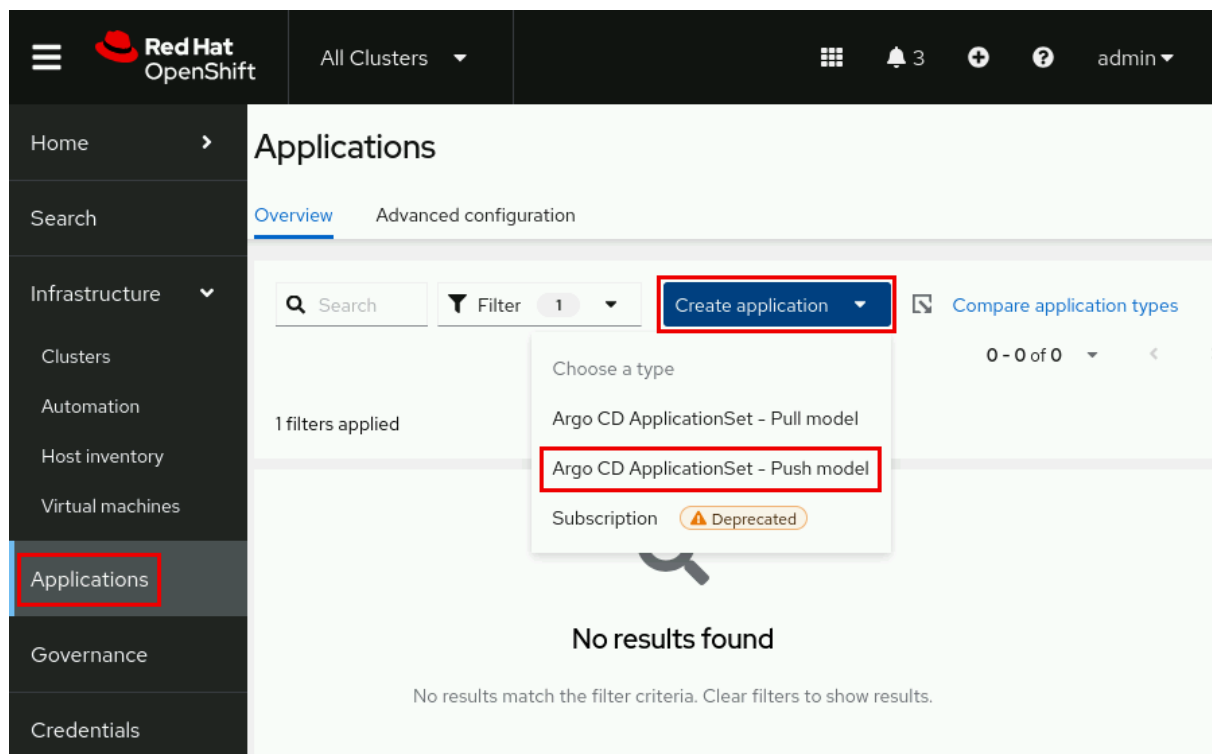
The Git repository that the classroom provides already includes the resources to perform this step. Use the following parameters for your configuration:

Parameter	Value
GitOps model	Push model
Application set name	vm-prod
Git URL	https://git.ocp4.example.com/do0016l/virtualization-review-vms.git
Git revision	main
Git path	manifests/prod
Remote namespace	vm-prod
Placement cluster set	gitops-configure
Managed cluster label	env=prod

 Note

In the **Topology** view, you might see an unknown status icon for the Data Volume and the Persistent Volume Claim (PVC) resources. It is a known display issue that does not affect the functioning of the application, and will be resolved in a future update.

- 2.1. Go to **Applications** and select **Create application > Argo CD ApplicationSet - Push model**.



- 2.2. Complete the form with the provided parameters. In the **General** section, set the application name and select the Argo CD server. Click **Next**.

The screenshot shows the 'Create application set - push model' form. The 'General' section is expanded. The 'Name' field contains 'vm-prod'. The 'Argo server' dropdown is set to 'openshift-gitops'. The 'Requeue time' dropdown is set to 180. At the bottom, the 'Next' button is highlighted with a red box, along with 'Back' and 'Cancel' buttons.

- 2.3. In the **Template** section, select **Git** and provide the Git parameters. Click **Next**.

2 Template

Repository

Type
Git https://git.ocp4.example.com/do0016l/virtualization-review-vms.git



Git
Use a Git repository



Helm
Use a Helm repository

URL * ?
https://git.ocp4.example.com/do0016l/virtualization-review-vms.git

Revision ?
main

Path ?
manifests/prod

+ Add another repository

Destination

Remote namespace *
vm-prod

Next Back Cancel

- 2.4. Do not change anything in the **Sync policy** section. Click **Next**.
- 2.5. In the Placement section, select **New placement** and select the gitops-configure cluster set. Add the expression that selects the cluster with the env=prod label and limit the number of clusters to one. Click **Next**.

4 Placement

Placement

How do you want to select clusters?

New placement Existing placement

Cluster sets ?

gitops-configure 1

If no cluster sets are selected, all clusters will be selected from the cluster sets bound to the namespace.

Label expressions ?

env equals prod

Label *	Operator *	Values *
env	equals any of	prod 1

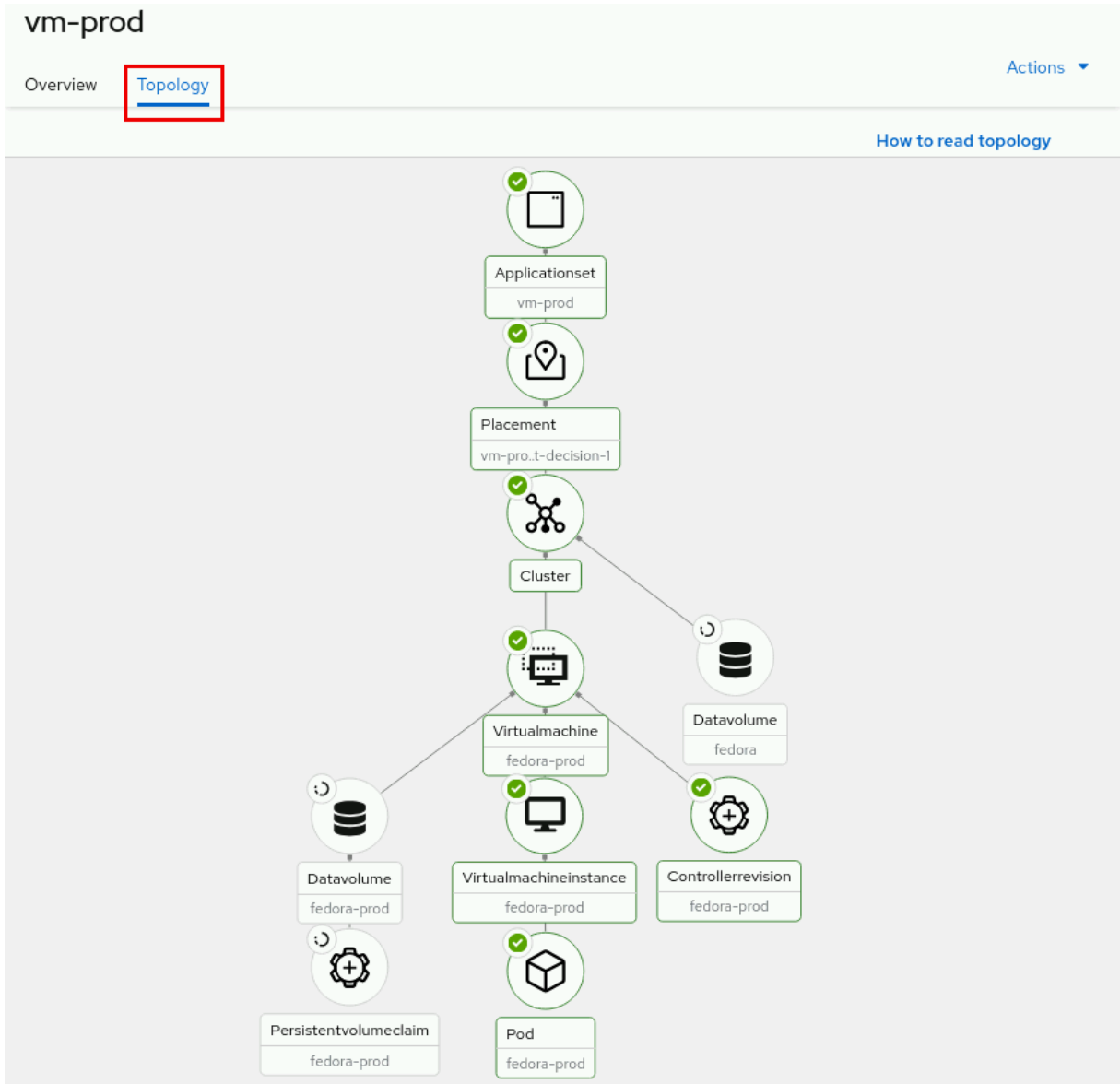
+ Add label expression

Limit the number of clusters selected

- 1 +

Next Back Cancel

- 2.6. In the Review section, verify the parameters and click **Submit**.
- 2.7. Go to **Topology** and wait up to three minutes for the topology to report the application status.



3. Deploy a VM in the cluster with the env=test label by using RHACM and the GitOps Push model.

The Git repository that the classroom provides already includes the resources to perform this step. Use the following parameters for your configuration:

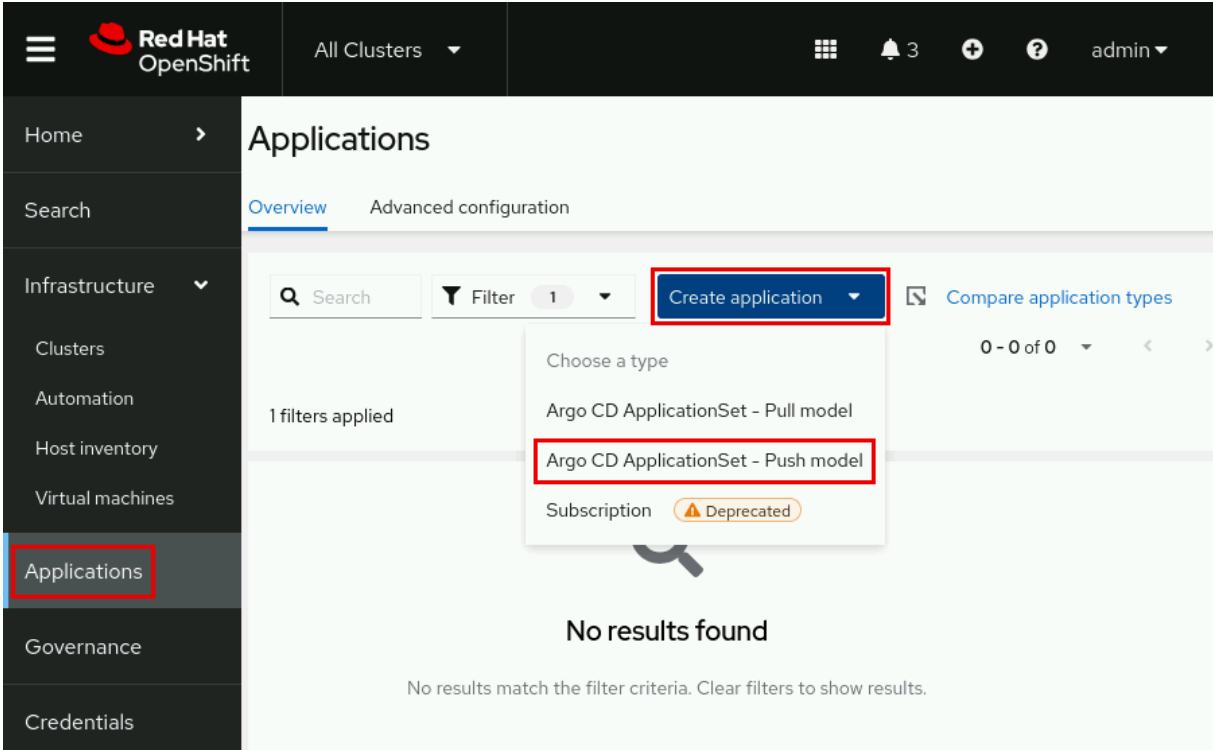
Parameter	Value
GitOps model	Push model
Application set name	vm-test
Git URL	https://git.ocp4.example.com/do0016l/virtualization-review-vms.git
Git revision	main
Git path	manifests/test

Parameter	Value
Remote namespace	vm-test
Placement cluster set	gitops-configure
Managed cluster label	env=test

Note

In the **Topology** view, you might see an unknown status icon for the Data Volume and the Persistent Volume Claim (PVC) resources. It is a known display issue that does not affect the functioning of the application, and will be resolved in a future update.

3.1. Go to **Applications** and select **Create application > Argo CD ApplicationSet - Push model**.



3.2. Complete the form with the provided parameters. In the **General** section, set the application name and select the Argo CD server. Click **Next**.

Applications > Create application set - push model

Create application set - push model

☒ YAML

1

General

General

Name *

vm-test

Argo server * ?

openshift-gitops

Requeue time * ?

180

Next

Back

Cancel

3.3. In the **Template** section, select **Git** and provide the Git parameters. Click **Next**.

2 Template

Repository

Type
Git https://git.ocp4.example.com/do0016l/virtualization-review-vms.git



Git
Use a Git repository



Helm
Use a Helm repository

URL * ?
https://git.ocp4.example.com/do0016l/virtualization-review-vms.git

Revision ?
main

Path ?
manifests/test

+ Add another repository

Destination

Remote namespace *
vm-test

Next Back Cancel

- 3.4. Do not change anything in the **Sync policy** section. Click **Next**.
- 3.5. In the Placement section, select **New placement** and select the gitops-configure cluster set. Add the expression that selects the cluster with the env=test label and limit the number of clusters to one. Click **Next**.

4 Placement

Placement

How do you want to select clusters?

New placement Existing placement

Cluster sets ?

gitops-configure 1

If no cluster sets are selected, all clusters will be selected from the cluster sets bound to the namespace.

Label expressions ?

env equals prod

Label *	Operator *	Values *
env	equals any of	test 1

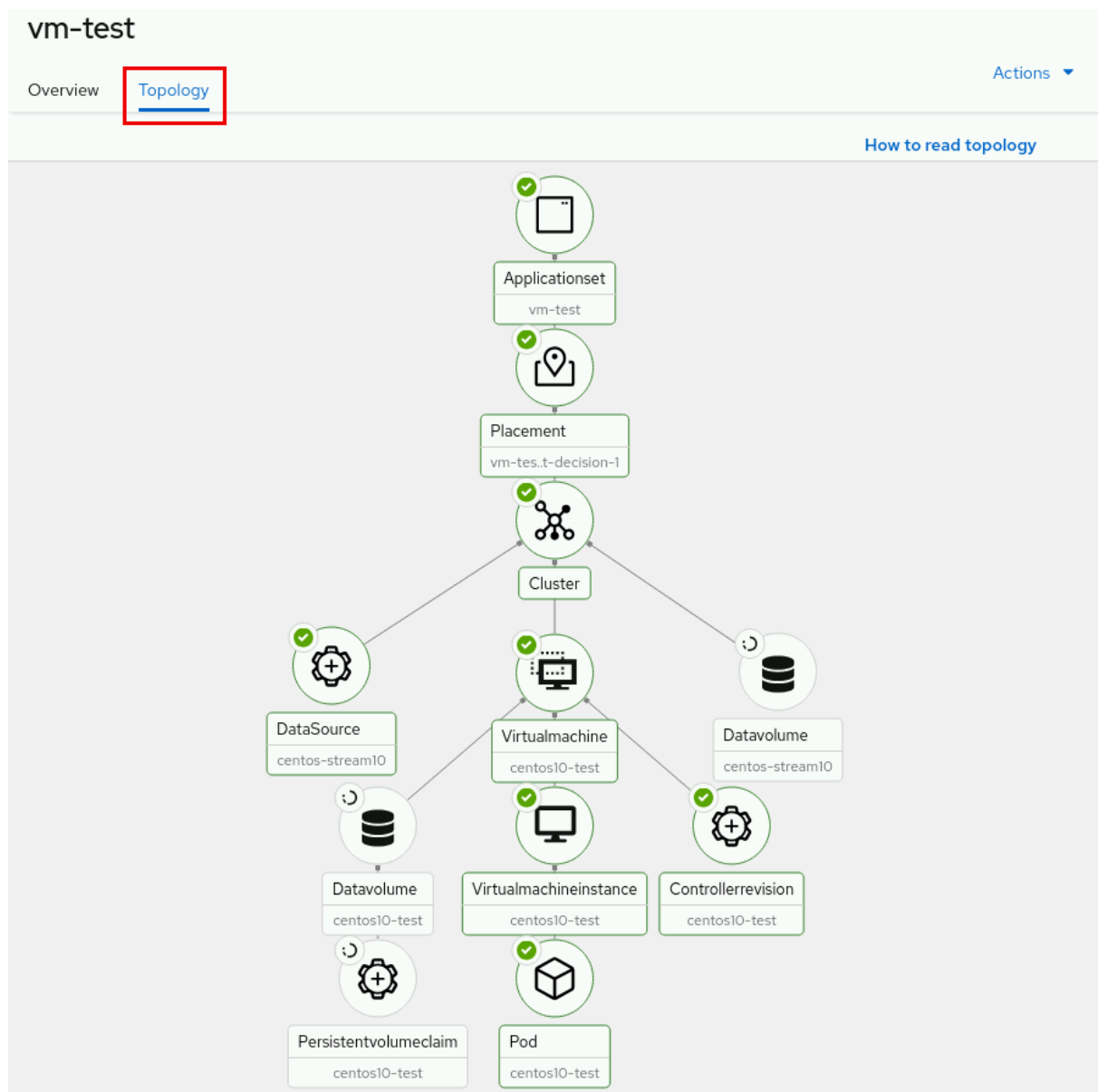
+ Add label expression

Limit the number of clusters selected

- 1 +

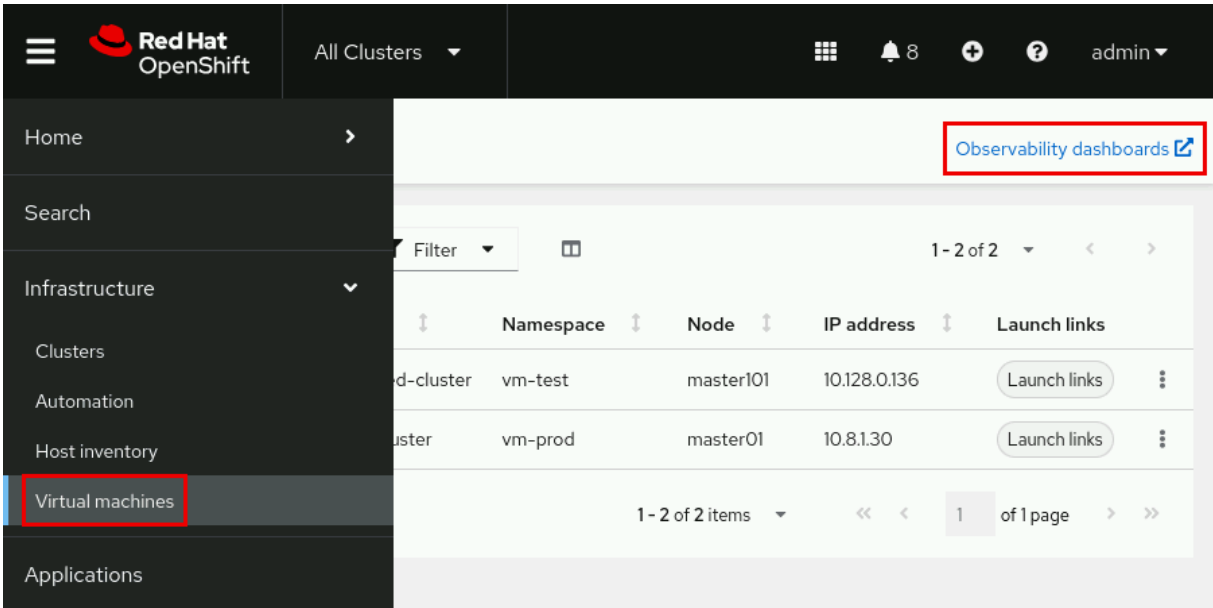
Next Back Cancel

- 3.6. In the Review section, verify the parameters and click **Submit**.
- 3.7. Go to **Topology** and wait up to three minutes for the topology to report the application status.

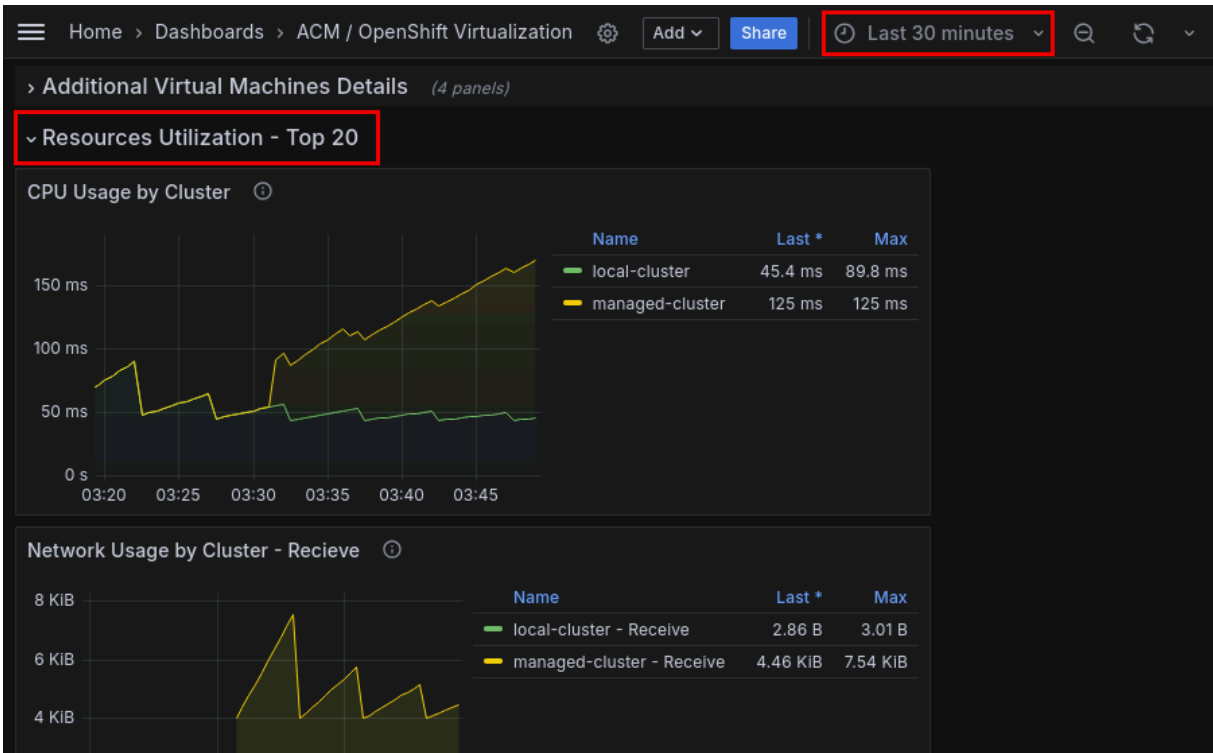


4. One of the VMs consumes an abnormal amount of I/O. Identify the VM and pause it.

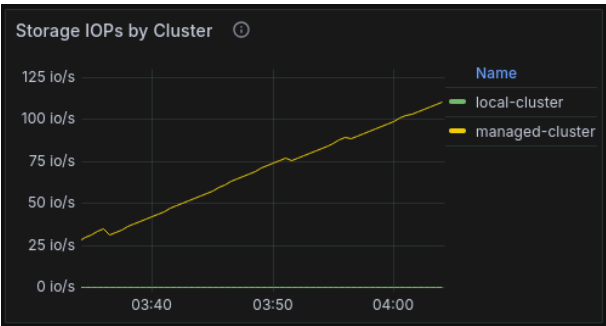
4.1. Go to **Infrastructure > Virtual machines** and click **Observability dashboards**.



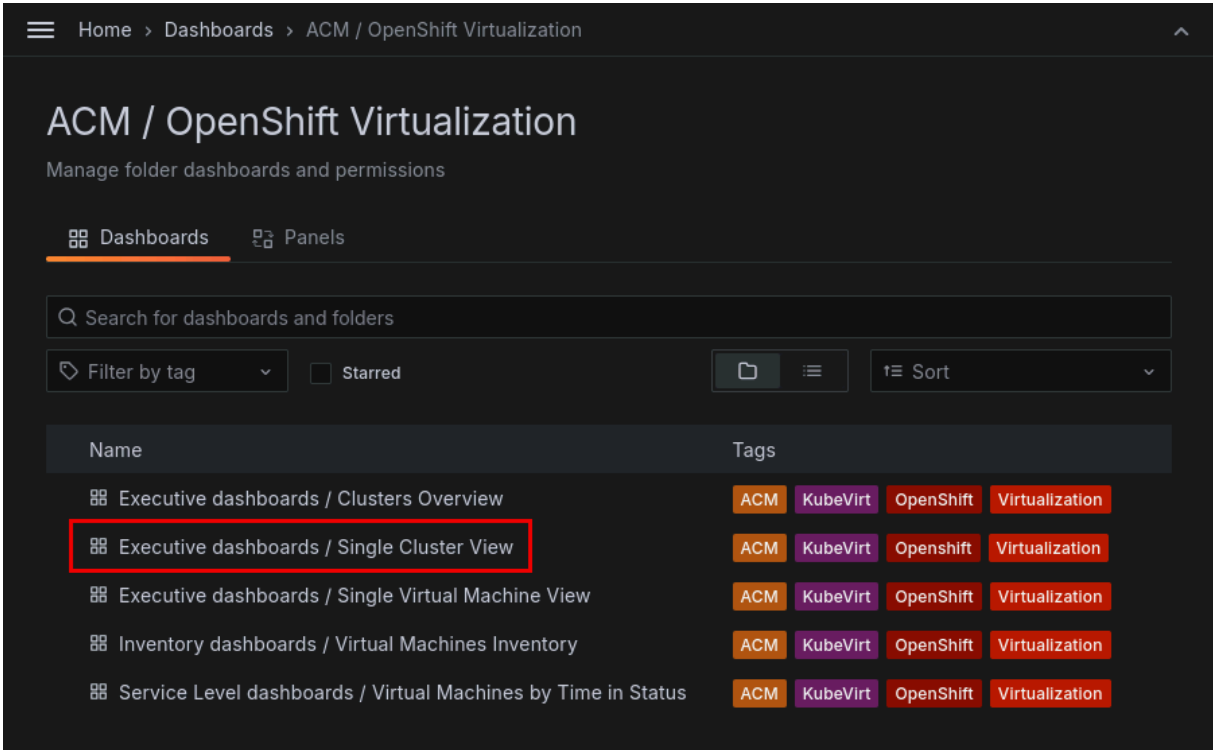
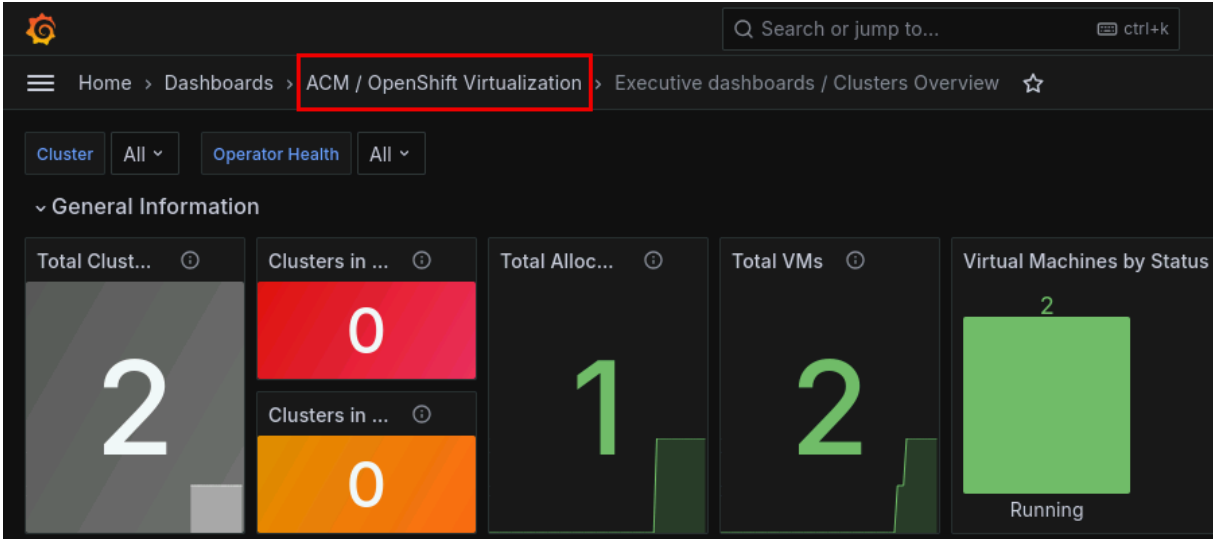
- 4.2. In the new browser tab that opens, click **Log in with OpenShift**, select the **Red Hat Identity Management** identity provider, and log in as the admin user with redhatocp as the password.
- 4.3. In the **Clusters Overview** dashboard in the **ACM / OpenShift Virtualization** directory, expand the **Resources utilization** section at the bottom of the page, and set the time range from the drop-down list to **Last 30 minutes**.



- 4.4. Locate the **Storage IOPs by Cluster** panel, which indicates that the I/O operations are constantly increasing for the managed-cluster cluster.

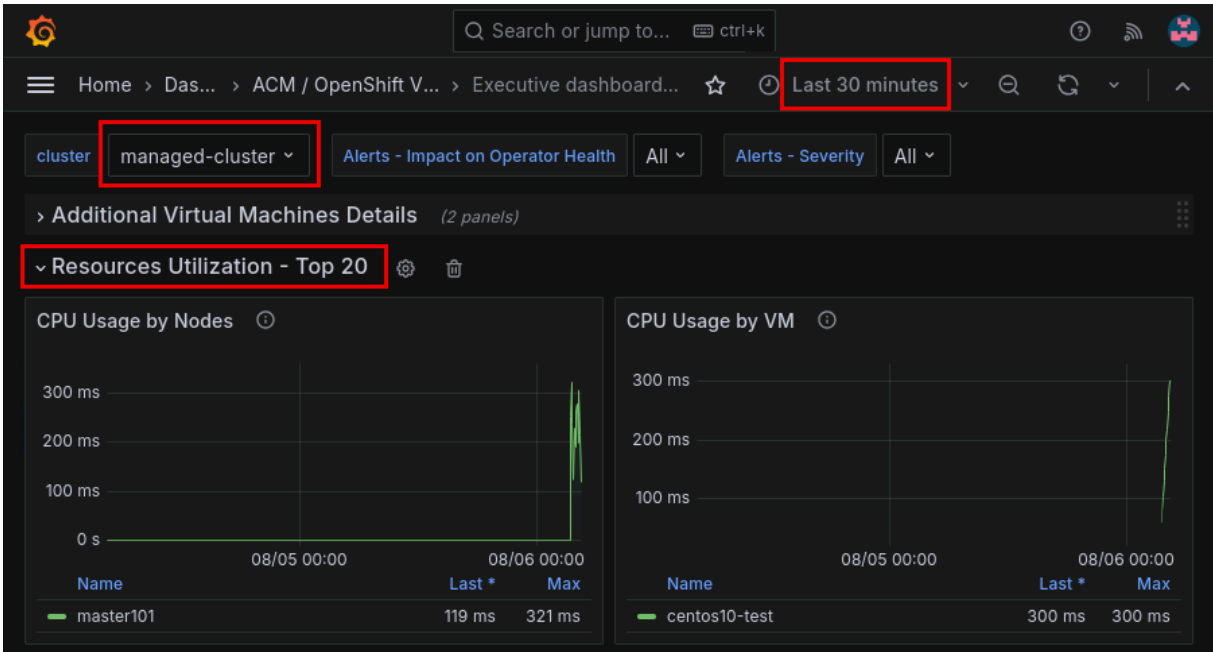


4.5. Review the I/O operations of the VMs that are running in the managed-cluster cluster. Select the **ACM / OpenShift Virtualization** directory, and then select the **Single cluster view** dashboard.

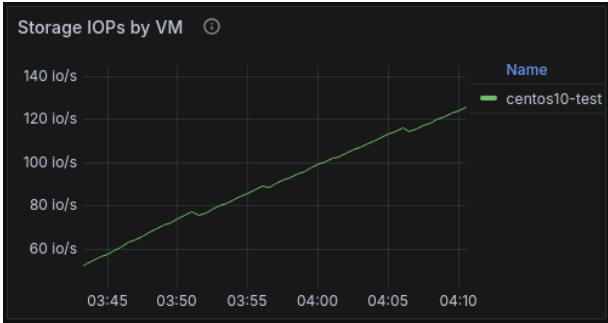


4.6. Select managed-cluster in the **cluster** drop-down list, expand the **Resources Utilization** section at the bottom of the page, and set the time range from the drop-down list to **Last 30 minutes**.

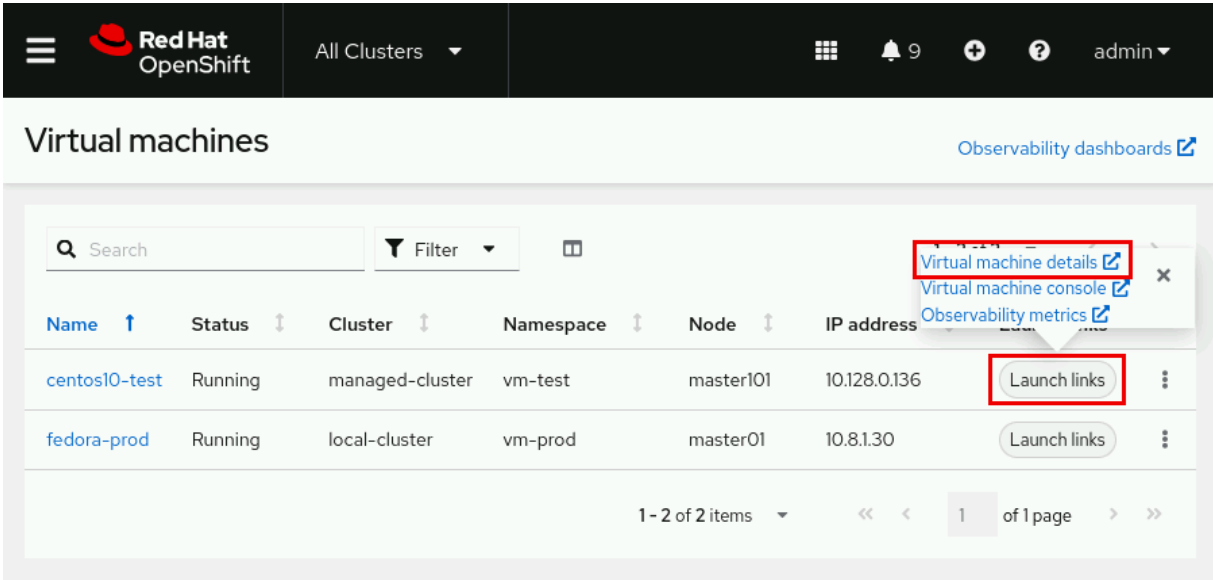
For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright



4.7. Locate the **Storage IOPs by VM** panel, which indicates that the I/O operations are constantly increasing for the centos10-test VM. This is the VM that you must pause.



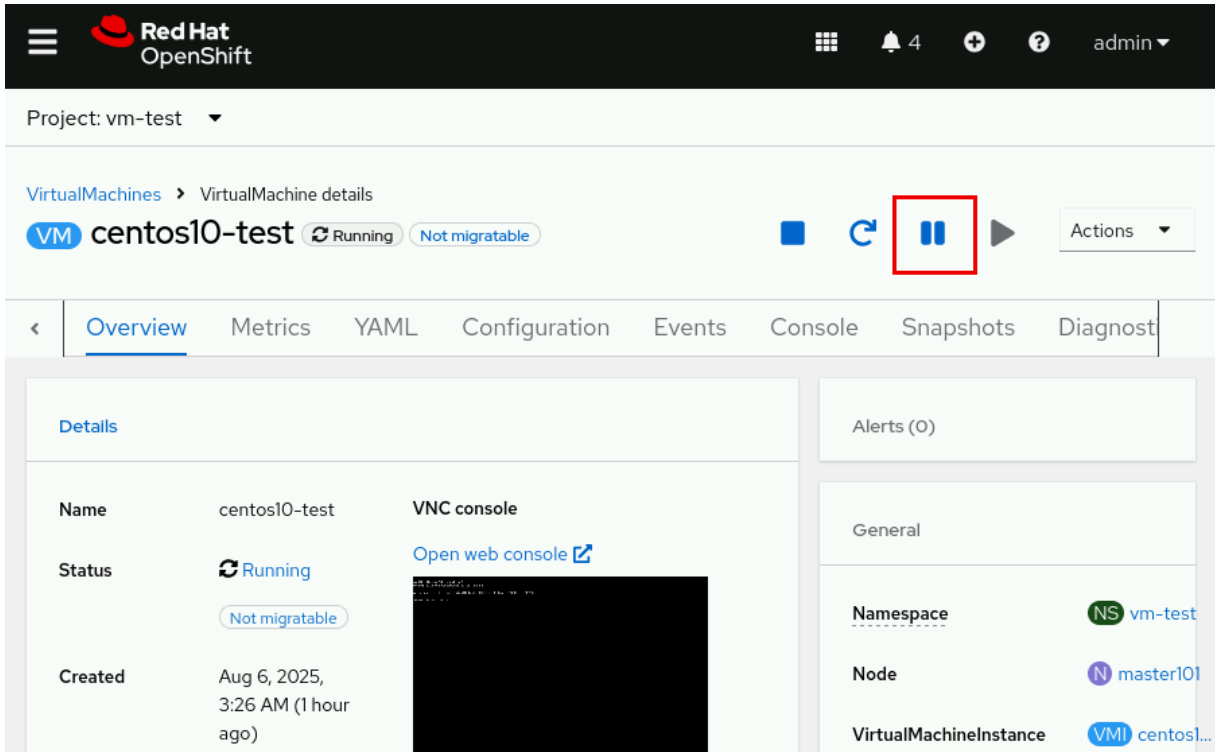
4.8. Switch back to the RHACM web console browser tab. Go to **Infrastructure > Virtual machines**, and click **Launch links > Virtual machine details** for the centos10-test VM.



4.9. The OpenShift web console opens in a new tab. If you are not already logged in to the console, then select the **Red Hat Identity Management** identity provider and log in as the admin user with redhatocp as the password.

For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright © 2026 Red Hat. | For use by Rajesh Ramamorthy ra_6776 ra.6776@gmail.com Copyright

4.10. Click **Pause**.



Evaluation

As the student user on the workstation machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade virtualization-review
```

Finish

As the student user on the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish virtualization-review
```

6.10. Summary

In this lesson, you learned how to manage virtual machines across multiple clusters by using Red Hat Advanced Cluster Management for Kubernetes. The lesson highlights the challenges of traditional VM management and presents RHACM as a centralized solution that extends the Kubernetes control plane to include VM workloads, to simplify administration and improve consistency.

You explored two key methods to deploy and manage the OpenShift Virtualization operator with RHACM:

- **GitOps with Argo CD:** You used this declarative framework to automatically deploy the OpenShift Virtualization operator and its custom resources from a Git repository. This approach uses the Argo CD `Application` and `ApplicationSet` resources, along with sync waves, to ensure a consistent and scalable installation across your fleet of clusters.
- **RHACM policies:** You learned how to use the RHACM policy engine to enforce an intended state for operator installation and configuration. This method is suitable for ensuring compliance and detecting configuration drift across your managed clusters.

Furthermore, you learned how to deploy and manage the lifecycle of the VMs by using the RHACM GitOps capabilities. Finally, the lesson covered how RHACM integrates with the OpenShift observability stack to provide a comprehensive view of your virtualized infrastructure. You explored various Grafana dashboards, such as Clusters Overview, Single Cluster View, and Single Virtual Machine View. These dashboards provide critical metrics for monitoring VM health, identifying performance issues, and informing resource optimization decisions such as right-sizing.

Next Steps

You might explore other learning opportunities on your own, explore further your RHACM installation, or redo some exercises.